Parlez-nous de l'expérience de téléchargement de PDF.

Référence DAX (Data Analysis Expressions)

DAX (Data Analysis Expressions) est une bibliothèque de fonctions et d'opérateurs qui peuvent être combinés pour générer des formules et des expressions dans Power BI, Analysis Services et Power Pivot dans les modèles de données Excel.

Fonctions

RÉFÉRENCE

Fonctions DAX

Instructions DAX

NOUVEAUTÉS

Nouvelles fonctions DAX

Découvrir DAX

VUE D'ENSEMBLE

Vue d'ensemble de DAX

VIDEO

Vidéos

ENTRAINEMENT

Utiliser DAX dans Power BI Desktop

Comprendre

CONCEPT

Glossaire DAX

Opérateurs DAX

Requêtes DAX

Syntaxe DAX

Vue d’ensemble de DAX

Article • 20/10/2023

DAX (Data Analysis Expressions) est un langage d’expression de formule utilisé dans les applications Analysis Services, Power BI et Power Pivot dans Excel. Les formules DAX incluent des fonctions, des opérateurs et des valeurs qui permettent d’effectuer des requêtes et des calculs complexes sur des données de colonnes et tables associées dans des modèles de données tabulaires.

Cet article offre une présentation générale des principaux concepts de DAX. Il décrit DAX tel qu’il s’applique à tous les produits qui l’utilisent. Certaines de ses fonctionnalités peuvent ne pas s’appliquer à des produits ou cas d’usage particuliers. Pour plus d’informations sur l’implémentation de DAX de votre produit, consultez la documentation correspondante.

Calculs

Les formules DAX s’utilisent dans les mesures, les colonnes calculées, les tables calculées et la sécurité au niveau des lignes.

Mesures

Les mesures sont des formules de calcul dynamique dont les résultats changent selon le contexte. Les mesures sont utilisées dans les rapports qui prennent en charge la combinaison et le filtrage des données de modèle à l’aide de plusieurs attributs, comme dans les rapports Power BI ou les tableaux et graphiques croisés dynamiques Excel. Les mesures sont créées au moyen de la barre de formule DAX dans le concepteur de modèle.

Une formule dans une mesure peut utiliser des fonctions d’agrégation standard qui sont créées automatiquement avec une fonction de somme automatique, comme COUNT ou SUM. Vous pouvez également définir votre propre formule à l’aide de la barre de formule DAX. Une mesure nommée peut être passée comme argument à d’autres mesures.

Lorsque vous définissez une formule pour une mesure dans la barre de formule, une info-bulle affiche un aperçu rapide de ce que seront les résultats pour l'ensemble du contexte actuel, mais autrement, aucun résultat n'est immédiatement généré. La raison pour laquelle vous ne pouvez pas voir les résultats (filtrés) du calcul immédiatement tient au fait que le résultat d'une mesure ne peut pas être déterminé sans contexte.

Évaluer une mesure requiert une application cliente de création de rapports qui peut fournir le contexte nécessaire pour récupérer les données relatives à chaque cellule, puis évaluer l'expression pour chaque cellule. Cette application cliente peut être un tableau ou graphique croisé dynamique Excel, un rapport Power BI, ou une expression de table dans une requête DAX dans SSMS (SQL Server Management Studio).

Quelle que soit l'application cliente utilisée, une requête distincte est exécutée pour chaque cellule dans les résultats. Autrement dit, chaque combinaison d'en-têtes de ligne et de colonne dans un tableau croisé dynamique, ou chaque sélection de segments et de filtres dans un rapport Power BI, génère un sous-ensemble de données différent sur lequel la mesure est calculée. Par exemple, c'est ce qui se produit avec cette formule de mesure très simple :

DAX

```
Total Sales = SUM([Sales Amount])
```

Quand un utilisateur ajoute la mesure TotalSales à un rapport, puis ajoute la colonne Catégorie de produit d'une table Produit dans Filtres, la somme des ventes (Sales Amount) est calculée et affichée pour chaque catégorie de produit.

Contrairement aux colonnes calculées, la syntaxe d'une mesure inclut le nom de la mesure avant la formule. Dans l'exemple juste au-dessus, le nom **Total Sales** est placé avant la formule proprement dite. Une fois que vous avez créé une mesure, le nom et sa définition s'affichent dans la liste de champs de l'application cliente de création de rapports et, selon les perspectives et les rôles, la mesure est disponible pour tous les utilisateurs du modèle.

Pour plus d'informations, consultez les rubriques suivantes :
Mesures dans Power BI Desktop
Mesures dans Analysis Services
Mesures dans Power Pivot

Colonnes calculées

Une colonne calculée est une colonne que vous ajoutez à une table existante (dans le concepteur de modèle), puis dans laquelle vous créez une formule DAX qui définit ses valeurs. Quand une colonne calculée contient une formule DAX valide, les valeurs sont calculées pour chaque ligne dès que la formule est entrée. Les valeurs sont ensuite stockées dans le modèle de données en mémoire. Par exemple, dans une table Date, lorsque la formule est entrée dans la barre de formule :

DAX

```
= [Calendar Year] & " Q" & [Calendar Quarter]
```

Une valeur pour chaque ligne de la table est calculée en prenant les valeurs de la colonne Calendar Year (dans la même table Date), en ajoutant un espace et la lettre majuscule Q, puis en ajoutant les valeurs de la colonne Calendar Quarter (toujours dans la même table Date). Le résultat pour chaque ligne de la colonne calculée est calculé immédiatement, puis est affiché sous cette forme : **2017 Q1**. Les valeurs de la colonne sont recalculées uniquement si la table ou une table associée est traitée (actualisée) ou bien si le modèle est déchargé de la mémoire, puis rechargé, comme c'est le cas notamment lors de la fermeture et de la réouverture d'un fichier Power BI Desktop.

Pour en savoir plus, consultez :
Colonnes calculées dans Power BI Desktop
Colonnes calculées dans Analysis Services
Colonnes calculées dans Power Pivot

Tables calculées

Une table calculée est un objet calculé, sur la base d'une expression de formule et d'une partie ou de la totalité des autres tables contenues dans le même modèle. Au lieu d'interroger et de charger les valeurs dans les colonnes de votre nouvelle table à partir d'une source de données, c'est une formule DAX qui définit les valeurs de la table.

Les tables calculées peuvent être utiles dans une dimension de rôle actif. La table Date en est un exemple, où OrderDate, ShipDate et DueDate dépendent de la relation de clé étrangère. En créant explicitement une table calculée pour ShipDate, vous obtenez une table autonome utilisable pour les requêtes et qui fonctionne comme n'importe quelle autre table. Les tables calculées sont également utiles quand vous configurez un ensemble de lignes filtré, ou un sous-ensemble ou sur-ensemble de colonnes issues d'autres tables existantes. Vous pouvez ainsi conserver la table d'origine inchangée et en créer des variantes pour les besoins de scénarios particuliers.

Les tables calculées prennent en charge les relations avec d'autres tables. Les colonnes de votre table calculée ont des types de données et une mise en forme, et elles peuvent appartenir à une catégorie de données. Les tables calculées peuvent être nommées et être exposées ou masquées comme toutes les autres tables. Les tables calculées sont recalculées si l'une des tables dont elles tirent (pull) des données sont actualisées ou mises à jour.

Pour en savoir plus, consultez :
Tables calculées dans Power BI Desktop
Tables calculées dans Analysis Services

Sécurité au niveau des lignes

Avec la sécurité au niveau des lignes, une formule DAX doit évaluer une condition booléenne TRUE/FALSE, qui définit les lignes pouvant être retournées dans les résultats d'une requête par les membres d'un rôle donné. Par exemple, pour les membres du rôle Sales, la table Customers avec la formule DAX suivante :

```
DAX
= Customers[Country] = "USA"
```

Les membres du rôle Sales peuvent uniquement voir les données des clients situés aux États-Unis, et les agrégats, tels que SUM, sont retournés uniquement pour ces clients. La sécurité au niveau des lignes n'est pas disponible dans Power Pivot dans Excel.

Quand vous définissez la sécurité au niveau des lignes à l'aide d'une formule DAX, vous créez un ensemble de lignes autorisées. Cela ne signifie pas que les autres lignes ne sont pas accessibles ; en fait, elles ne sont simplement pas retournées dans l'ensemble de lignes autorisées. D'autres rôles peuvent autoriser l'accès aux lignes qui sont exclues par la formule DAX. Si un utilisateur est membre d'un autre rôle et que la sécurité au niveau des lignes de ce rôle autorise l'accès à cet ensemble de lignes particulier, l'utilisateur peut voir les données de ces lignes.

Les formules de la sécurité au niveau des lignes s'appliquent aux lignes spécifiées ainsi qu'aux lignes associées. Si une table possède plusieurs relations, les filtres appliquent la sécurité de la relation qui est active. Les formules de la sécurité au niveau des lignes se recoupent avec d'autres formules définies pour les tables associées.

Pour en savoir plus, consultez :
Sécurité au niveau des lignes (RLS) avec Power BI
Rôles dans Analysis Services

Requêtes

Les requêtes DAX peuvent être créées et exécutées dans SSMS (SQL Server Management Studio) et des outils open source comme DAX Studio (daxstudio.org). À la différence des formules de calcul DAX, qui peuvent uniquement être créées dans des modèles de données tabulaires, les requêtes DAX peuvent également être exécutées sur

des modèles multidimensionnels dans Analysis Services. Les requêtes DAX sont souvent plus faciles à écrire et plus performantes que les requêtes multidimensionnelles (MDX).

Une requête DAX est une instruction comparable à une instruction SELECT dans T-SQL. Le type de requête DAX le plus simple est une instruction d'*évaluation* (Evaluate). Par exemple,

DAX

```
EVALUATE
 ( FILTER ( 'DimProduct', [SafetyStockLevel] < 200 ) )
ORDER BY [EnglishProductName] ASC
```

Retourne comme résultats une table qui liste uniquement les produits dont le niveau de stock de sécurité (valeur SafetyStockLevel) est inférieur à 200, dans l'ordre croissant du nom de produit (valeur EnglishProductName).

Vous pouvez créer des mesures à utiliser dans la requête. Ces mesures existent seulement pour la durée de la requête. Pour en savoir plus, consultez Requêtes DAX.

Formules

Les formules DAX sont essentielles pour créer des calculs dans des colonnes calculées et des mesures, et pour sécuriser vos données à l'aide de la sécurité au niveau des lignes. Pour créer des formules pour des colonnes calculées et des mesures, utilisez la barre de formule en haut de la fenêtre du concepteur de modèle ou l'éditeur DAX. Pour créer des formules pour la sécurité au niveau des lignes, utilisez la boîte de dialogue Gestionnaire de rôles ou Gérer les rôles. Les informations de cette section sont destinées à vous aider à comprendre les principes de base des formules DAX.

Principes de base des formules

Les formules DAX peuvent être très simples ou assez complexes. Le tableau suivant présente quelques exemples de formules simples qui peuvent être utilisées dans une colonne calculée.

Agrandir le tableau

Formule	Définition
`= TODAY()`	Insère la date du jour dans chaque ligne d'une colonne calculée.
`= 3`	Insère la valeur 3 dans chaque ligne d'une colonne calculée.

Formule	Définition
`= [Column1] + [Column2]`	Ajoute les valeurs dans la même ligne des colonnes [Column1] et [Column2], et insère le résultat dans la colonne calculée de la même ligne.

Que vous souhaitiez créer une formule simple ou complexe, vous pouvez suivre les étapes ci-dessous :

1. Chaque formule doit commencer par un signe égal (=).
2. Tapez ou sélectionnez un nom de fonction, ou tapez une expression.
3. Tapez les premières lettres de la fonction ou du nom que vous souhaitez ; la fonctionnalité Autocomplétion affiche alors une liste de fonctions, tables et colonnes disponibles. Appuyez sur Tab pour insérer un élément de la liste Autocomplétion dans la formule.

   Vous pouvez aussi cliquer sur le bouton **Fx** pour afficher une liste de fonctions disponibles. Pour sélectionner une fonction dans la liste déroulante, utilisez les flèches afin de mettre en surbrillance l'élément, puis cliquez sur **OK** pour ajouter la fonction à la formule.
4. Spécifiez les arguments de la fonction en les sélectionnant dans une liste déroulante de tables et de colonnes possibles, ou en tapant leurs valeurs.
5. Recherchez les éventuelles erreurs de syntaxe : vérifiez que toutes les parenthèses sont fermées et que les colonnes, les tables et les valeurs sont correctement référencées.
6. Appuyez sur Entrée pour valider la formule.

> ⓘ **Notes**
>
> Dans une colonne calculée, dès que la formule entrée est validée, la colonne est remplie avec des valeurs. Dans une mesure, la définition de la mesure est enregistrée avec la table quand vous appuyez sur Entrée. Si une formule n'est pas valide, une erreur s'affiche.

Dans cet exemple, examinons une formule dans une mesure nommée **Days in Current Quarter** :

```
DAX
```

```
Days in Current Quarter = COUNTROWS( DATESBETWEEN( 'Date'[Date],
STARTOFQUARTER( LASTDATE('Date'[Date])), ENDOFQUARTER('Date'[Date])))
```

Cette mesure est utilisée pour créer un ratio de comparaison entre une période incomplète et la période précédente. La formule doit prendre en compte la proportion de la période déjà écoulée et la comparer à la même proportion de la période précédente. Dans ce cas, le ratio [Days Current Quarter to Date]/[Days in Current Quarter] donne la proportion écoulée de la période actuelle.

Cette formule contient les éléments suivants :

Agrandir le tableau

Élément de la formule	Description
`Days in Current Quarter`	Nom de la mesure.
`=`	Le signe égal (=) commence la formule.
`COUNTROWS`	COUNTROWS compte le nombre de lignes dans la table Date.
`()`	Les parenthèses ouvrantes et fermantes spécifient des arguments.
`DATESBETWEEN`	La fonction DATESBETWEEN retourne les dates entre la dernière date pour chaque valeur de la colonne Date dans la table Date.
`'Date'`	Spécifie la table Date. Les tables sont spécifiées entre deux guillemets simples.
`[Date]`	Spécifie la colonne Date dans la table Date. Les colonnes sont spécifiées entre crochets.
`,`	
`STARTOFQUARTER`	La fonction STARTOFQUARTER retourne la date du début du trimestre.
`LASTDATE`	La fonction LASTDATE retourne la dernière date du trimestre.
`'Date'`	Spécifie la table Date.
`[Date]`	Spécifie la colonne Date dans la table Date.
`,`	
`ENDOFQUARTER`	Fonction ENDOFQUARTER
`'Date'`	Spécifie la table Date.

Élément de la formule	Description
`[Date]`	Spécifie la colonne Date dans la table Date.

Utilisation de la fonctionnalité Autocomplétion dans les formules

La saisie semi-automatique vous aide à saisir une syntaxe de formule valide en vous proposant des options pour chaque élément de la formule.

- Vous pouvez utiliser la saisie semi-automatique des formules au milieu d'une formule existante avec les fonctions imbriquées. Le texte immédiatement avant le point d'insertion est utilisé pour afficher des valeurs dans la liste déroulante, et tout le texte après le point d'insertion reste inchangé.

- La saisie semi-automatique n'ajoute pas la parenthèse fermante des fonctions, ni ne met automatiquement en correspondance les parenthèses. Vous devez vous assurer que chaque fonction est correcte syntaxiquement ou vous ne pouvez pas enregistrer ni utiliser la formule.

Utilisation de plusieurs fonctions dans une formule

Vous pouvez imbriquer des fonctions, ce qui signifie que vous utilisez les résultats d'une fonction comme un argument d'une autre fonction. Vous pouvez imbriquer jusqu'à 64 niveaux de fonctions dans les colonnes calculées. Toutefois, l'imbrication peut rendre la création ou le dépannage de formules difficile. De nombreuses fonctions sont conçues pour être utilisées uniquement comme fonctions imbriquées. Ces fonctions retournent une table, qui ne peut pas être enregistrée directement comme résultat ; elle doit être fournie comme entrée à une fonction de table. Par exemple, les fonctions SUMX, AVERAGEX et MINX requièrent toutes une table comme premier argument.

Fonctions

Une fonction est une formule nommée dans une expression. La plupart des fonctions prennent en entrée des arguments obligatoires et facultatifs, également appelés paramètres. Lorsque la fonction est exécutée, une valeur est retournée. DAX fournit des fonctions qui vous permettent d’effectuer des calculs avec des dates et des heures, de créer des valeurs conditionnelles, d’utiliser des chaînes, d’effectuer des recherches basées sur des relations et d’itérer sur une table pour faire des calculs récursifs. Si vous connaissez les formules Excel, plusieurs de ces fonctions vous seront familières ;

toutefois, les formules DAX diffèrent à de nombreux égards. Voici quelques différences de taille :

- Une fonction DAX fait toujours référence à une table ou une colonne complète. Si vous souhaitez utiliser certaines valeurs particulières d'une table ou colonne, vous pouvez ajouter des filtres à la formule.
- Si vous devez personnaliser des calculs en fonction de chaque ligne, DAX fournit des fonctions qui vous permettent d'utiliser la valeur de ligne actuelle ou une valeur associée comme genre de paramètre pour effectuer des calculs qui varient selon le contexte. Pour comprendre le fonctionnement de ces fonctions, consultez la section Contexte dans le présent article.
- DAX inclut de nombreuses fonctions qui retournent une table, plutôt qu'une valeur. La table n'est pas affichée dans un client de création de rapports, mais elle est utilisée pour fournir une entrée à d'autres fonctions. Par exemple, vous pouvez récupérer une table, puis compter les valeurs distinctes qu'elle contient, ou calculer les sommes dynamiques entre les colonnes ou les tables filtrées.
- Les fonctions DAX intègrent une grande diversité de fonctions *d'intelligence temporelle*. Ces fonctions vous permettent de définir ou de sélectionner des plages de dates, et effectuer des calculs dynamiques basés sur ces dates ou plages. Par exemple, vous pouvez comparer les sommes sur des périodes parallèles.

Fonctions d'agrégation

Les fonctions d'agrégation calculent une valeur (scalaire) telle que count, sum, average, minimum ou maximum pour toutes les lignes d'une colonne ou d'une table, comme défini par l'expression. Pour en savoir plus, voir Fonctions d'agrégation.

Fonctions de date et heure

Les fonctions de date et d'heure de DAX sont semblables à celles de Microsoft Excel. Toutefois, les fonctions DAX sont basées sur un type de données **datetime** dont la date de début est le 1er mars 1900. Pour en savoir plus, consultez Fonctions de date et heure.

Fonctions de filtrage

Les fonctions de filtrage de DAX permettent de retourner des types de données spécifiques, de rechercher des valeurs dans les tables associées et de procéder à un filtrage par valeurs associées. Les fonctions de recherche s'appuient sur des tables et des relations, comme une base de données. Les fonctions de filtrage vous permettent de

manipuler le contexte de données pour créer des calculs dynamiques. Pour en savoir plus, consultez Fonctions de filtre.

Fonctions financières

Les fonctions financières dans DAX s'utilisent dans des formules qui effectuent des calculs financiers, par exemple, pour connaître la valeur actuelle nette et le taux de rendement. Ces fonctions sont similaires aux fonctions financières utilisées dans Microsoft Excel. Pour en savoir plus, consultez Fonctions financières.

Fonctions d'information

Une fonction d'information examine la cellule ou la ligne qui est fournie comme argument et vous indique si la valeur correspond au type attendu. Par exemple, la fonction ISERROR retourne TRUE si la valeur que vous référencez contient une erreur. Pour en savoir plus, consultez Fonctions d'information.

Fonctions logiques

Les fonctions logiques agissent sur une expression pour retourner des informations sur les valeurs au sein de l'expression. Par exemple, la fonction TRUE vous permet de savoir si une expression que vous évaluez retourne une valeur TRUE. Pour en savoir plus, consultez Fonctions logiques.

Fonctions mathématiques et trigonométriques

Les fonctions mathématiques dans DAX sont très semblables aux fonctions mathématiques et trigonométriques Excel. Il existe quelques différences mineures dans les types de données numériques utilisés par les fonctions DAX. Pour en savoir plus, consultez Fonctions mathématiques et trigonométriques.

Autres fonctions

Ces fonctions effectuent des actions uniques qui n'entrent dans aucune des catégories définissant la plupart des autres fonctions. Pour en savoir plus, consultez Autres fonctions.

Fonctions de relation

Les fonctions de relation dans DAX vous permettent de retourner des valeurs d'une autre table associée, de spécifier une relation particulière à utiliser dans une expression et de définir la direction du filtrage croisé. Pour en savoir plus, consultez Fonctions de relation.

Fonctions statistiques

Les fonctions statistiques calculent des valeurs liées aux distributions statistiques et à la probabilité, telles que l'écart type et le nombre de permutations. Pour en savoir plus, consultez Fonctions statistiques.

Fonctions de texte

Les fonctions de texte dans DAX sont très semblables à leurs équivalents dans Excel. Vous pouvez retourner une partie d'une chaîne, rechercher un texte dans une chaîne ou concaténer des valeurs de chaîne. DAX fournit également des fonctions pour le contrôle des formats pour les dates, les heures et les nombres. Pour en savoir plus, consultez Fonctions de texte.

Fonctions d'intelligence temporelle

Les fonctions d'intelligence temporelle fournies dans DAX vous permettent de créer des calculs qui utilisent la connaissance intégrée relative aux calendriers et aux dates. En utilisant des plages de dates et d'heures en association avec des agrégations ou des calculs, vous pouvez générer des comparaisons significatives sur des périodes de temps comparables pour les ventes, les stocks, etc. Pour en savoir plus, consultez Fonctions d'intelligence temporelle (DAX).

Fonctions de manipulation de tables

Ces fonctions retournent une table ou manipulent des tables existantes. Par exemple, vous pouvez ajouter des colonnes calculées à une table spécifiée avec la fonction ADDCOLUMNS, ou retourner une table de résumé sur un ensemble de groupes avec la fonction SUMMARIZECOLUMNS. Pour en savoir plus, consultez Fonctions de manipulation de tables.

Variables

Vous pouvez créer des variables dans une expression en utilisant VAR. VAR n'est pas une fonction d'un point de vue technique : c'est un mot clé qui stocke le résultat d'une

expression sous la forme d'une variable nommée. Cette variable peut ensuite être passée comme argument à d'autres expressions de mesure. Par exemple :

```DAX
VAR
    TotalQty = SUM ( Sales[Quantity] )

Return

    IF (
        TotalQty > 1000,
        TotalQty * 0.95,
        TotalQty * 1.25
        )
```

Dans cet exemple, TotalQty peut être passé en tant que variable nommée à d'autres expressions. Les variables peuvent être de n'importe quel type de données scalaire, y compris des tables. L'emploi de variables dans vos formules DAX peut être extrêmement puissant.

Types de données

Vous pouvez importer des données dans un modèle à partir de nombreuses sources de données différentes, qui peuvent prendre en charge des types de données différents. Lorsque vous importez des données dans un modèle, les données sont converties en un des types de données de modèle tabulaire. Quand les données du modèle sont utilisées dans un calcul, les données sont converties en un type de données DAX pour la durée et le résultat du calcul. Lorsque vous créez une formule DAX, les termes utilisés dans la formule déterminent automatiquement le type de données de valeur retournées.

DAX prend en charge les types de données suivants :

Agrandir le tableau

Type de données dans le modèle	Type de données dans DAX	Description
Nombre entier	Valeur entière de 64 bits (huit octets) [1, 2]	Nombres qui n'ont pas de décimales. Les entiers peuvent être des nombres positifs ou négatifs, mais doivent être compris entre -9 223 372 036 854 775 808 (-2^63) et 9 223 372 036 854 775 807 (2^63-1).
Nombre décimal	Nombre réel de 64 bits (huit	Les nombres réels sont des nombres qui peuvent avoir des décimales. Les nombres réels couvrent une large gamme de

Type de données dans le modèle	Type de données dans DAX	Description
		octets) [1, 2] valeurs : Valeurs négatives de -1.79E +308 à -2.23E -308 Zéro Valeurs positives de 2.23E -308 à -1.79E +308 Toutefois, le nombre de bits significatifs est limité à 17 chiffres décimaux.
Booléen	Boolean	Valeur True ou valeur False.
Texte	String	Chaîne de données caractères au format Unicode. Il peut s'agir de chaînes, de nombres ou de dates représentés dans un format texte.
Date	Date/heure	Dates et heures dans une représentation date-heure acceptée. Les dates valides sont toutes les dates après le 1er mars 1900.
Devise	Devise	Le type de données devise autorise des valeurs entre -922 337 203 685 477,5808 et 922 337 203 685 477,5807 avec quatre chiffres décimaux à précision fixe.
N/A	Vide	Le type de données Vide (Blank) de DAX représente et remplace les valeurs Null SQL. Vous pouvez créer une valeur vide à l'aide de la fonction BLANK et tester les valeurs vides à l'aide de la fonction logique ISBLANK.

Les modèles de données tabulaires incluent également le type de données *Table* en entrée ou en sortie dans de nombreuses fonctions DAX. Par exemple, la fonction FILTER prend une table en entrée et génère en sortie une autre table qui contient uniquement les lignes qui répondent aux conditions de filtre. En associant des fonctions de table à des fonctions d'agrégation, vous pouvez effectuer des calculs complexes sur des jeux de données définis de façon dynamique.

Bien que les types de données soient généralement définis automatiquement, il est important de comprendre de quelle façon ils s'appliquent, notamment, aux formules DAX. Des erreurs dans les formules ou des résultats inattendus sont notamment dus à l'emploi d'un opérateur spécifique qui ne peut pas être utilisé avec le type de données spécifié dans un argument. Par exemple, la formule `= 1 & 2`, retourne un résultat de chaîne de 12. Toutefois, la formule `= "1" + "2"` retourne un résultat entier de 3.

Context

Le *contexte* est un concept important qu'il faut bien comprendre quand vous créez des formules DAX. Le contexte vous permet d'effectuer une analyse dynamique dans la mesure où les résultats d'une formule changent pour refléter la sélection de ligne ou de cellule actuelle, ainsi que toutes les données associées. Il est impératif de comprendre ce qu'est un contexte et de savoir utiliser un contexte à bon escient pour créer des analyses dynamiques performantes et pour résoudre les problèmes dans les formules.

Les formules dans les modèles tabulaires peuvent être évaluées dans un contexte différent, en fonction d'autres éléments de conception.

- Filtres appliqués dans un tableau croisé dynamique ou un rapport
- Filtres définis dans une formule
- Relations spécifiées à l'aide de fonctions spéciales dans une formule

Il existe différents types de contexte : *contexte de ligne*, *contexte de requête*et *contexte de filtre*.

Contexte de ligne

Le *contexte de ligne* peut être vu comme « la ligne active ». Si vous créez une formule dans une colonne calculée, le contexte de ligne correspondant à cette formule inclut les valeurs de toutes les colonnes dans la ligne actuelle. Si la table est associée à une autre table, le contenu inclut également toutes les valeurs de cette autre table qui sont mises en relation avec la ligne actuelle.

Par exemple, supposons que vous créez une colonne calculée, `= [Freight] + [Tax]`, qui additionne les valeurs de deux colonnes, Freight et Tax, de la même table. Cette formule obtient automatiquement les valeurs de la ligne actuelle uniquement dans les colonnes spécifiées.

Le contexte de ligne suit également toutes les relations définies entre les tables, notamment les relations définies dans une colonne calculée à l'aide de formules DAX, afin de déterminer les lignes des tables liées qui sont associées à la ligne actuelle.

Par exemple, la formule suivante utilise la fonction RELATED pour extraire une valeur de taxe d'une table associée, selon la région vers laquelle la commande a été expédiée. La valeur de taxe est déterminée par l'utilisation de la valeur de la région dans la table actuelle, la recherche de la région dans la table associée, puis l'obtention du taux d'imposition de cette région à partir de la table associée.

```
DAX
```

```
= [Freight] + RELATED('Region'[TaxRate])
```

Cette formule obtient le taux d'imposition pour la région active de la table Region et l'ajoute à la valeur de la colonne de Freight. Dans les formules DAX, vous n'avez pas besoin de connaître ou spécifier la relation spécifique qui connecte les tables.

Contextes de ligne multiples

Le langage DAX (Data Analysis Expressions) inclut des fonctions qui itèrent des calculs sur une table. Ces fonctions peuvent avoir plusieurs lignes actives, chacune avec son propre contexte de ligne. Fondamentalement, ces fonctions vous permettent de créer des formules qui effectuent des opérations de manière récursive sur une boucle interne et externe.

Par exemple, supposons que votre modèle contienne une table **Products** et une table **Sales** . Les utilisateurs peuvent vouloir parcourir la totalité de la table Sales, remplie de transactions impliquant plusieurs produits, et rechercher la quantité maximale commandée pour chaque produit dans l'ensemble de ces transactions.

Avec DAX, vous pouvez générer une formule unique qui retourne la valeur correcte, et les résultats sont automatiquement mis à jour chaque fois que l'utilisateur ajoute des données aux tables.

DAX

```
= MAXX(FILTER(Sales,[ProdKey] = EARLIER([ProdKey])),Sales[OrderQty])
```

Pour obtenir un exemple détaillé de cette formule, consultez EARLIER.

Pour résumer, la fonction EARLIER stocke le contexte de ligne de l'opération qui a précédé l'opération active. Cette fonction stocke systématiquement en mémoire deux ensembles de contexte : le premier ensemble représente la ligne actuelle pour la boucle interne de la formule, et le second ensemble représente la ligne actuelle pour la boucle externe de la formule. DAX fournit automatiquement les valeurs entre les deux boucles pour vous permettre de créer des agrégats complexes.

Contexte de requête

Un*contexte de requête* fait référence au sous-ensemble de données qui est implicitement récupéré pour une formule. Par exemple, quand un utilisateur insère une mesure ou un champ dans un rapport, le moteur examine les en-têtes de ligne et de colonne, les

segments et les filtres de rapport pour déterminer le contexte. Les requêtes nécessaires sont ensuite exécutées sur les données de modèle pour obtenir le sous-ensemble correct de données, effectuer les calculs définis par la formule, puis remplir les valeurs dans le rapport.

Étant donné que le contexte change en fonction de l'endroit où vous placez la formule, les résultats de la formule peuvent varier également. Supposons, par exemple, que vous créez une formule qui additionne les valeurs de la colonne **Profit** de la table **Sales** : `=SUM('Sales'[Profit])`. Si vous utilisez cette formule dans une colonne calculée dans la table **Sales**, les résultats de la formule seront identiques pour toute la table, car le contexte de requête utilisé par la formule est toujours l'ensemble de données complet de la table **Sales**. Les résultats indiqueront les bénéfices pour toutes les régions, tous les produits, toutes les années, etc.

Cependant, plutôt que d'obtenir le même résultat des centaines de fois, les utilisateurs voudront connaître le bénéfice pour une année, un pays ou un produit en particulier (ou pour une combinaison de ces éléments), avant d'avoir un total général.

Dans un rapport, le contexte peut être changé par le filtrage, l'ajout ou la suppression de champs et l'utilisation de segments. Cela entraîne le changement du contexte de requête dans lequel la mesure est évaluée. Par conséquent, la même formule, utilisée dans une mesure, est évaluée dans un *contexte de requête* différent pour chaque cellule.

Contexte de filtre

Le*contexte de filtre* est l'ensemble de valeurs autorisé dans chaque colonne, ou dans les valeurs récupérées d'une table associée. Des filtres peuvent être appliqués à la colonne dans le concepteur, ou dans la couche de présentation (rapports et tableaux croisés dynamiques). Des filtres peuvent également être définis explicitement par des expressions de filtre dans la formule.

Un contexte de filtre est ajouté lorsque vous spécifiez des contraintes de filtre sur l'ensemble de valeurs autorisé dans une colonne ou une table, en utilisant les arguments d'une formule. Un contexte de filtre s'applique au-dessus des autres contextes, tels qu'un contexte de ligne ou un contexte de requête.

Dans des modèles tabulaires, plusieurs méthodes s'offrent à vous pour créer le contexte de filtre. Dans le contexte des clients consommateurs du modèle, comme les rapports Power BI, les utilisateurs peuvent créer des filtres instantanément en ajoutant des segments ou des filtres de rapport sur les en-têtes de ligne et de colonne. Vous pouvez également spécifier des expressions de filtre directement dans la formule, pour spécifier des valeurs associées, filtrer les tables utilisées comme entrées ou obtenir de façon

dynamique le contexte des valeurs utilisées dans les calculs. Vous pouvez également supprimer entièrement les filtres ou les effacer de manière sélective sur des colonnes spécifiques. C'est très utile lors de la création de formules qui calculent des totaux généraux.

Pour en savoir plus sur la création de filtres dans les formules, consultez FILTER, fonction (DAX).
Pour obtenir un exemple de la procédure de suppression des filtres pour créer des totaux généraux, consultez Fonction ALL (DAX).

Pour obtenir des exemples d'application et de suppression sélective des filtres dans les formules, consultez ALLEXCEPT.

Détermination du contexte dans les formules

Lorsque vous créez une formule DAX, la formule est d'abord testée pour vérifier si la syntaxe est valide, puis pour vérifier que les noms des colonnes et des tables inclus dans la formule figurent dans le contexte actuel. Si aucune colonne ou table spécifiée par la formule ne peut être trouvée, une erreur est retournée.

Le contexte au cours de la validation (et des opérations de recalcul) est déterminé comme décrit dans les sections précédentes, à l'aide des tables disponibles dans le modèle, des relations entre les tables et des éventuels filtres qui ont été appliqués.

Par exemple, si vous venez d'importer des données dans une nouvelle table et que celle-ci n'est pas associée à d'autres tables (et que vous n'avez pas appliqué de filtres), le *contexte actuel* correspond à l'ensemble entier de colonnes dans la table. Si la table est liée par des relations à d'autres tables, le contexte actuel inclut les tables associées. Si vous ajoutez une colonne de la table à un rapport qui contient des segments et éventuellement certains filtres de rapport, le contexte de la formule correspond au sous-ensemble de données dans chaque cellule du rapport.

Le contexte est un concept puissant qui peut également compliquer la résolution des problèmes liés aux formules. Nous vous recommandons de commencer par des formules et des relations simples pour voir comment le contexte s'exécute. La section suivante fournit quelques exemples d'utilisation de différents types de contextes par les formules pour un retour dynamique des résultats.

Opérateurs

Le langage DAX utilise quatre types différents d'opérateurs de calcul dans les formules :

- Opérateurs de comparaison pour comparer des valeurs et retourner une valeur TRUE\FALSE logique.
- Opérateurs arithmétiques pour effectuer des calculs arithmétiques qui retournent des valeurs numériques.
- Opérateurs de concaténation de texte pour joindre deux chaînes de texte ou plus.
- Opérateurs logiques qui combinent deux expressions ou plus pour retourner un résultat unique.

Pour plus d'informations sur les opérateurs utilisés dans les formules DAX, consultez Opérateurs DAX.

Utilisation de tables et de colonnes

Les tables dans les modèles de données tabulaires ont une apparence similaire aux tables Excel, mais elles interagissent différemment avec les données et les formules :

- Les formules fonctionnent uniquement avec les tables et les colonnes, et non avec les cellules individuelles, les références de plage ou les tableaux.
- Les formules peuvent utiliser des relations pour obtenir des valeurs à partir de tables associées. Les valeurs récupérées sont toujours associées à la valeur de ligne actuelle.
- Vous ne pouvez pas avoir de données irrégulières ou « déséquilibrées », comme vous le pouvez dans une feuille de calcul Excel. Chaque ligne d'une table doit contenir le même nombre de colonnes. Toutefois, certaines colonnes peuvent comporter des valeurs vides. Les tables de données Excel et les tables de données de modèle tabulaire ne sont pas interchangeables.
- Un type de données étant défini pour chaque colonne, toutes les valeurs de cette colonne doivent être du même type.

Référence aux tables et aux colonnes dans les formules

Vous pouvez faire référence à toute table et colonne à l'aide de son nom. Par exemple, la formule suivante indique comment faire référence aux colonnes de deux tables à l'aide du nom *complet* :

DAX

```
= SUM('New Sales'[Amount]) + SUM('Past Sales'[Amount])
```

Lorsqu'une formule est évaluée, le générateur de modèles vérifie en premier la syntaxe générale, puis il contrôle les noms des colonnes et des tables que vous fournissez par rapport aux colonnes et tables possibles dans le contexte actuel. Si le nom est ambigu ou si la colonne/la table est introuvable, vous obtiendrez une erreur dans votre formule (une #chaîne ERROR au lieu d'une valeur de données dans les cellules où l'erreur se produit). Pour en savoir plus sur les conventions de nommage des tables, colonnes et autres objets, consultez la section en question dans Syntaxe DAX.

Relations entre les tables

Créer des relations entre les tables vous donne la possibilité d'utiliser des valeurs associées dans d'autres tables pour les calculs. Par exemple, utilisez une colonne calculée pour déterminer tous les enregistrements d'expédition associés au revendeur actuel et calculer ensuite le total des frais d'expédition pour chacun d'eux. Dans de nombreux cas, toutefois, il est possible qu'une relation ne soit pas nécessaire. Vous pouvez utiliser la fonction LOOKUPVALUE dans une formule afin de retourner la valeur dans *result_columnName* pour la ligne répondant aux critères spécifiés dans les arguments *search_column* et *search_value*.

De nombreuses fonctions DAX requièrent l'existence d'une relation entre les tables ou entre plusieurs tables, afin de localiser les colonnes que vous avez référencées et de retourner des résultats qui ont un sens. D'autres fonctions essaient d'identifier la relation ; toutefois, lorsque cela est possible, créez une relation pour optimiser les résultats. Les modèles de données tabulaires prennent en charge les relations multiples entre les tables. Pour éviter toute confusion ou des résultats incorrects, une seule relation à la fois est désignée comme la relation active. Vous pouvez changer la relation active si nécessaire pour parcourir les différentes relations dans les données des calculs. La fonction USERELATIONSHIP permet de spécifier une ou plusieurs relations à utiliser dans un calcul spécifique.

Il est important d'observer les règles de conception de formule suivantes si vous utilisez des relations :

- Quand des tables sont liées par une relation, vous devez vous assurer que les deux colonnes utilisées comme clés ont des valeurs qui correspondent. En effet, l'intégrité référentielle n'étant pas vérifiée ici, il est possible de créer une relation même s'il y a des valeurs sans correspondance dans une colonne clé. Si cela se produit, vous devez savoir que des valeurs vides ou sans correspondance peuvent affecter les résultats des formules.

- Lorsque vous liez des tables dans votre modèle à l'aide de relations, vous agrandissez l'étendue, ou le *contexte*, dans laquelle vos formules sont évaluées.

Des modifications dans le contexte résultant de l'ajout de nouvelles tables, de nouvelles relations ou de modifications de la relation active peuvent entraîner des modifications de vos résultats qu'il n'est pas possible d'anticiper. Pour en savoir plus, consultez Contexte dans cet article.

Traitement et actualisation

Le *traitement* et le *recalcul* sont deux opérations distinctes mais interdépendantes. Vous devez comprendre en détail ces concepts lorsque vous concevez un modèle qui contient des formules complexes, de grandes quantités de données ou des données obtenues de sources de données externes.

Le *traitement (actualisation)* met à jour les données dans un modèle avec de nouvelles données provenant d'une source de données externe.

Le*recalcul* est le processus de mise à jour des résultats des formules afin de refléter toutes les modifications qui leur ont été apportées, ainsi que toutes les modifications des données sous-jacentes. Le recalcul peut affecter les performances des façons suivantes :

- Les valeurs dans une colonne calculée sont calculées et stockées dans le modèle. Pour mettre à jour les valeurs dans la colonne calculée, vous devez traiter le modèle à l'aide de l'une des trois commandes de traitement – Traiter entièrement, Traiter les données ou Traiter le recalcul. Le résultat de la formule doit toujours être recalculé, pour la colonne entière, chaque fois que vous modifiez la formule.

- Les valeurs calculées par une mesure sont évaluées dynamiquement chaque fois qu'un utilisateur ajoute la mesure à un tableau croisé dynamique ou ouvre un rapport. Quand l'utilisateur change le contexte, les valeurs retournées par la mesure changent également. Les résultats de la mesure reflètent toujours les valeurs les plus récentes dans le cache en mémoire.

Le traitement et le recalcul n'ont pas d'impact sur les formules de sécurité au niveau des lignes, sauf si le résultat d'un recalcul retourne une valeur différente, rendant la ligne interrogeable ou non interrogeable par les membres du rôle.

Mises à jour

DAX est constamment amélioré. Les fonctions nouvelles et modifiées sont publiées avec chaque mise à jour suivante disponible, qui est généralement mensuelle. Les mises à jour sont appliquées aux services en premier, puis aux applications installées telles que Power BI Desktop, Excel, SQL Server Management Studio (SSMS) et l'extension de

projets Analysis Services pour Visual Studio (SSDT). SQL Server Analysis Services est mis à jour avec la mise à jour cumulative suivante. Les nouvelles fonctions sont d'abord annoncées et décrites dans la documentation de référence des fonctions DAX, en même temps que les mises à jour de Power BI Desktop.

Les fonctions ne sont pas toutes prises en charge dans les versions antérieures de SQL Server Analysis Services et d'Excel.

Dépannage

Si vous obtenez une erreur lorsque vous définissez une formule, la formule peut contenir une *erreur syntaxique*, une *erreur sémantique*ou une *erreur de calcul*.

Les erreurs syntactiques sont les plus faciles à résoudre. Elles impliquent en général une virgule ou une parenthèse manquante.

Dans les deux autres types d'erreurs, la syntaxe est correcte, mais la valeur ou une colonne référencée n'a pas de sens dans le contexte de la formule. De telles erreurs sémantiques et de calcul peuvent être provoquées par l'un des problèmes suivants :

- La formule fait référence à une colonne, une table ou une fonction non existante.
- La formule semble être correcte, mais lorsque le moteur de données extrait les données, il trouve une incompatibilité de type et génère une erreur.
- La formule transmet un nombre ou un type d'arguments incorrect à une fonction.
- La formule fait référence à une colonne différente qui comporte une erreur, et par conséquent, ses valeurs ne sont pas valides.
- La formule fait référence à une colonne qui n'a pas été traitée, ce qui signifie qu'elle comporte des métadonnées, mais aucune donnée à proprement parler à utiliser pour les calculs.

Dans les quatre premiers cas, DAX signale la colonne entière qui contient la formule non valide. Dans le dernier cas, DAX grise la colonne pour indiquer que la colonne se trouve dans un état non traité.

Applications et outils

Power BI Desktop



Power BI Desktop est une application gratuite conçue pour la modélisation des données et la création de rapports. Le concepteur de modèle comprend un éditeur DAX permettant de créer des formules de calcul DAX.

Power Pivot dans Excel



Le concepteur de modèle Power Pivot dans Excel comprend un éditeur DAX permettant de créer des formules de calcul DAX.

Visual Studio



Visual Studio avec l'extension de projets Analysis Services (VSIX) s'utilise pour créer des projets de modèle Analysis Services. Le concepteur de modèle tabulaire, installé avec l'extension de projets, comprend un éditeur DAX.

SQL Server Management Studio



SSMS (SQL Server Management Studio) est un outil essentiel pour travailler avec Analysis Services. SSMS comprend un éditeur de requête DAX permettant d'interroger à la fois des modèles tabulaires et des modèles multidimensionnels.

DAX Studio



DAX Studio est un outil client open source qui permet de créer et d'exécuter des requêtes DAX sur des modèles Analysis Services, Power BI Desktop et Power Pivot dans Excel.

Éditeur tabulaire



Éditeur tabulaire est un outil open source qui fournit une vue hiérarchique et intuitive de chaque objet dans les métadonnées du modèle tabulaire. Éditeur tabulaire inclut un éditeur DAX avec mise en surbrillance syntaxique, qui permet de modifier facilement des mesures, des colonnes calculées et des expressions de table calculée.

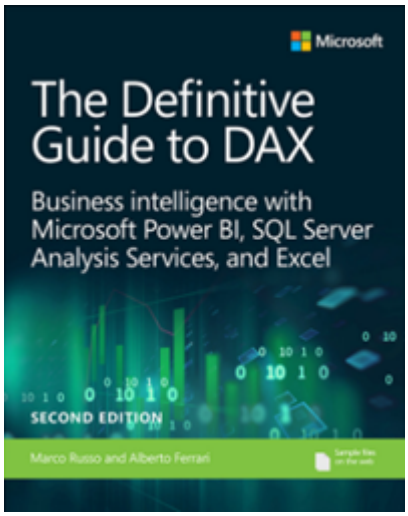
Ressources d'apprentissage

Pendant la phase d'apprentissage de DAX, il est préférable d'utiliser l'application dont vous vous servirez ensuite pour créer vos propres modèles de données. Que ce soit pour Analysis Services, Power BI Desktop ou Power Pivot dans Excel, vous trouverez de nombreux articles et tutoriels qui expliquent pas à pas comment créer des mesures, des colonnes calculées et des filtres de lignes à l'aide du langage DAX. Voici quelques ressources supplémentaires :

Vidéos

Parcours d'apprentissage Utiliser DAX dans Power BI Desktop.

Definitive Guide to DAX, des auteurs Alberto Ferrari et Marco Russo (Microsoft Press). Maintenant dans ce deuxième édition, ce guide complet est destiné aux modélisateurs de données et aux professionnels en Business Intelligence qui débutent. Il fournit des informations générales sur des techniques innovantes et très performantes.



Communauté

DAX a une communauté dynamique, toujours désireuse de partager son expertise. La communauté Power BI de Microsoft a un forum de discussion spécial pour DAX, ses commandes et des conseils.

Commentaires

Cette page a-t-elle été utile ? Yes No

Vidéos

Article • 20/10/2023

Que vous utilisiez Power BI Desktop, Power Pivot dans Excel ou Analysis Services, il est essentiel de bien comprendre le langage DAX (Data Analysis Expressions) pour créer des modèles de données efficaces. Voici quelques vidéos qui vous aideront à commencer à utiliser ce puissant langage d'expression.

Introduction à DAX

Dans cette vidéo d'introduction à DAX, Alberto Ferrari, partenaire Microsoft, présente les principaux concepts de DAX. Au travers d'exemples clairs et pratiques, vous allez découvrir les mesures, les colonnes calculées et les expressions de modélisation des données de base avec DAX.

https://www.youtube-nocookie.com/embed/klQAZLr5vxA

Utilisation avancée de DAX

Dans cette vidéo sur l'utilisation avancée de DAX, Alberto Ferrari, partenaire Microsoft, décrit la théorie de DAX, le contexte de filtre et de ligne ainsi que d'autres concepts essentiels de DAX.

https://www.youtube-nocookie.com/embed/6ncHnWMEdic

DAX pratique pour Power BI

Dans cette vidéo, Phil Seamark, responsable principal de programme chez Microsoft, présente les meilleures pratiques pour l'utilisation de DAX dans Power BI.

https://www.youtube-nocookie.com/embed/1fGfqzS37qs

Commentaires

Cette page a-t-elle été utile ? Yes No

Exemple de modèle DAX

Article • 20/10/2023

L'exemple de modèle **Adventure Works DW 2020** Power BI Desktop est conçu pour prendre en charge votre apprentissage DAX. Le modèle est basé sur l'exemple d'entrepôt de données Adventure Works pour AdventureWorksDW2017. Toutefois, les données ont été modifiées pour s'adapter aux objectifs de l'exemple de modèle.

L'exemple de modèle ne contient aucune formule DAX. Toutefois, il prend en charge des centaines voire des milliers de formules et de requêtes de calcul potentielles. Certains exemples de fonctions, comme ceux de CALCULATE, DATESBETWEEN, DATESIN PERIOD, IF et LOOKUPVALUE, peuvent être ajoutés à l'exemple de modèle sans modification. Nous prévoyons d'inclure davantage d'exemples dans d'autres articles de référence sur les fonctions compatibles avec cet exemple de modèle.

Scénario

La société Adventure Works représente un fabricant de vélos qui vend des vélos et des accessoires à l'échelle internationale. Les données de l'entrepôt de données de la société sont stockées dans Azure SQL Database.

Structure du modèle

Le modèle comporte sept tables :

Agrandir le tableau

Table	Description
Client	Décrit les clients et leur emplacement géographique. Les clients achètent des produits en ligne (Internet Sales, Ventes sur Internet).
Date	Il existe trois relations entre les tables **Date** et **Sales** (Ventes), pour la date de commande, la date d'expédition et la date d'échéance. La relation de la date de

Table	Description
	commande est active. La société génère des rapports sur les ventes à l'aide d'un exercice qui commence le 1er juillet de chaque année. La table est marquée en tant que table de dates à l'aide de la colonne **Date**.
Produit	Stocke uniquement les produits finis.
Reseller	Décrit les revendeurs et leur emplacement géographique. Revendeurs et la vente des produits à leurs clients.
Sales	Stocke les lignes sur le grain de la ligne de commande client. Toutes les valeurs financières sont en dollars américains (USD). La première date de commande est le 1er juillet 2017 et la dernière date de commande est le 15 juin 2020.
Sales Order (Commande client)	Décrit les numéros de commande client et de ligne de commande, ainsi que le canal de vente, **Reseller** (Revendeur) ou **Internet**. Cette table a une relation un-à-un avec la table **Sales** (Ventes).
Sales Territory (Secteur de vente)	Les secteurs de vente sont organisés en groupes (Amérique du Nord, Europe et Pacifique), pays et régions. Seuls les États-Unis vendent des produits au niveau de la région.

Charger l'exemple

Téléchargez le fichier de l'exemple de modèle Power BI Desktop ici.

Contenu connexe

- Parcours d'apprentissage : Utiliser DAX dans Power BI Desktop
- Des questions ? Essayez d'interroger la communauté Power BI
- Vous avez des suggestions ? Envoyez-nous vos idées pour améliorer Power BI

Commentaires

Cette page a-t-elle été utile ? Yes No

Compréhension des fonctions ORDERBY, PARTITIONBY et MATCHBY

Article • 28/07/2023

Les fonctions ORDERBY, PARTITIONBY et MATCHBY dans DAX sont des fonctions spéciales qui ne peuvent être utilisées qu'avec des fonctions Window de DAX : INDEX, OFFSET, WINDOW, RANK, ROWNUMBER.

Il est essentiel de comprendre ORDERBY, PARTITIONBY et MATCHBY pour utiliser correctement les fonctions Window. Si les exemples présentés ici utilisent la fonction OFFSET, ils peuvent s'appliquer de la même façon aux autres fonctions Window.

Scénario

Commençons par un exemple qui n'utilise pas du tout de fonctions Window. Vous trouverez ci-dessous une table qui retourne le total des ventes par couleur et par année civile. Il existe plusieurs façons de définir cette table, mais comme notre objectif est ici de comprendre ce qu'il se passe dans DAX, nous allons utiliser une table calculée. Voici l'expression de table :

DAX

```
BasicTable =
    SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
    )
```

Vous verrez que cette expression de table calculée utilise SUMMARIZECOLUMNS pour calculer la somme (SUM) de la colonne SalesAmount de la table FactInternetSales, en fonction de la colonne Color de la table DimProduct et de la colonne CalendarYear de la table DimDate. Voici le résultat :

Agrandir le tableau

Color	CalendarYear	CurrentYearSales
"Black"	2017	393885

Color	CalendarYear	CurrentYearSales
"Black"	2018	1818835
"Black"	2019	3981638
"Black"	2020	2644054
« Blue »	2019	994448
« Blue »	2020	1284648
"Multi"	2019	48622
"Multi"	2020	57849
"NA"	2019	207822
"NA"	2020	227295
"Red"	2017	2961198
"Red"	2018	3686935
"Red"	2019	900175
"Red"	2020	176022
"Silver"	2017	326399
"Silver"	2018	750026
"Silver"	2019	2165176
"Silver"	2020	1871788
"White"	2019	2517
"White"	2020	2589
"Yellow"	2018	163071
"Yellow"	2019	2072083
"Yellow"	2020	2621602

Imaginons maintenant que nous cherchons à répondre à la question commerciale suivante : quelle a été la différence des ventes pour chaque couleur d'une année sur l'autre ? En fait, nous devons trouver un moyen de calculer les ventes de l'année précédente pour une même couleur et de soustraire ces chiffres des ventes de l'année en cours, en contexte. Par exemple, pour la combinaison [Red, 2019], nous recherchons

les ventes pour [Red, 2018]. Une fois que nous avons ce chiffre, nous pouvons le soustraire des ventes actuelles et retourner la valeur souhaitée.

Utilisation de OFFSET

La fonction OFFSET est parfaite pour les calculs de *comparaison* types pour répondre à la question commerciale décrite ci-dessus, car elle nous permet d'effectuer un mouvement relatif. Notre première tentative pourrait être la suivante :

DAX

```
1stAttempt =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "PreviousColorSales",
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation
            ),
            [CurrentYearSales]
        )
    )
```

Il y a beaucoup à dire sur cette expression. Nous avons utilisé ADDCOLUMNS pour développer la table précédente avec une colonne appelée PreviousColorSales. Le contenu de cette colonne est défini sur CurrentYearSales, qui correspond à SUM(FactInternetSales[SalesAmount]), pour la couleur précédente (extraite avec OFFSET).

Le résultat est le suivant :

Agrandir le tableau

Color	CalendarYear	CurrentYearSales	PreviousColorSales
"Black"	2017	393885	
"Black"	2018	1818835	393885

Color	CalendarYear	CurrentYearSales	PreviousColorSales
"Black"	2019	3981638	1818835
"Black"	2020	2644054	3981638
« Blue »	2019	994448	2644054
« Blue »	2020	1284648	994448
"Multi"	2019	48622	1284648
"Multi"	2020	57849	48622
"NA"	2019	207822	57849
"NA"	2020	227295	207822
"Red"	2017	2961198	227295
"Red"	2018	3686935	2961198
"Red"	2019	900175	3686935
"Red"	2020	176022	900175
"Silver"	2017	326399	176022
"Silver"	2018	750026	326399
"Silver"	2019	2165176	750026
"Silver"	2020	1871788	2165176
"White"	2019	2517	1871788
"White"	2020	2589	2517
"Yellow"	2018	163071	2589
"Yellow"	2019	2072083	163071
"Yellow"	2020	2621602	2072083

C’est un pas de plus vers notre objectif, mais à y regarder de près, ce n’est pas tout à fait ce que nous voulons. Par exemple, pour [Silver, 2017], previousColorSales est défini sur [Red, 2020].

Ajout de ORDERBY

La définition ci-dessus équivaut à :

DAX

```
1stAttemptWithORDERBY =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "PreviousColorSales",
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([Color], ASC, [CalendarYear], ASC,
[CurrentYearSales], ASC)
            ),
            [CurrentYearSales]
        )
    )
```

Dans ce cas, l'appel à OFFSET utilise ORDERBY pour organiser la table en fonction de Color et CalendarYear par ordre croissant, ce qui détermine ce qui est considéré comme étant la ligne précédente retournée.

Si ces deux résultats sont équivalents, c'est parce que ORDERBY contient automatiquement toutes les colonnes de la relation qui ne se trouvent pas dans PARTITIONBY. Comme PARTITIONBY n'a pas été spécifié, ORDERBY est défini sur Color, CalendarYear et CurrentYearSales. Or, sachant que les paires Color et CalendarYear sont uniques dans la relation, l'ajout de CurrentYearSales ne change pas le résultat. En fait, même si nous devions spécifier uniquement Color dans ORDERBY, les résultats seraient les mêmes, car CalendarYear serait automatiquement ajouté. Cela est dû au fait que la fonction ajoute autant de colonnes que nécessaire à ORDERBY afin que chaque ligne puisse être identifiée de manière unique par les colonnes ORDERBY et PARTITIONBY :

DAX

```
1stAttemptWithORDERBY =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
```

```
    ADDCOLUMNS(
        vRelation,
        "PreviousColorSales",
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([Color])
            ),
            [CurrentYearSales]
        )
    )
```

Ajout de PARTITIONBY

Maintenant, pour obtenir *pratiquement* le résultat recherché, nous pouvons utiliser PARTITIONBY, comme dans l'expression de table calculée suivante :

DAX

```
UsingPARTITIONBY =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "PreviousColorSales",
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([CalendarYear]),
                PARTITIONBY ([Color])
            ),
            [CurrentYearSales]
        )
    )
```

Notez que la spécification de ORDERBY est ici facultative, car ORDERBY contient automatiquement toutes les colonnes de la relation qui ne sont pas spécifiées dans PARTITIONBY. Par conséquent, l'expression suivante retourne les mêmes résultats, car ORDERBY est défini automatiquement sur CalendarYear et CurrentYearSales :

DAX

```
UsingPARTITIONBYWithoutORDERBY =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "PreviousColorSales",
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                PARTITIONBY ([Color])
            ),
            [CurrentYearSales]
        )
    )
```

Notes

Même si ORDERBY est automatiquement défini sur CalendarYear et CurrentYearSales, rien ne permet de dire dans quel l'ordre elles seront ajoutées. Si la colonne CurrentYearSales est ajoutée avant CalendarYear, l'ordre obtenu ne sera pas celui attendu. **Pour éviter toute confusion et des résultats inattendus, spécifiez ORDERBY et PARTITIONBY de manière explicite**.

Les deux expressions retournent le résultat qui nous voulions :

Agrandir le tableau

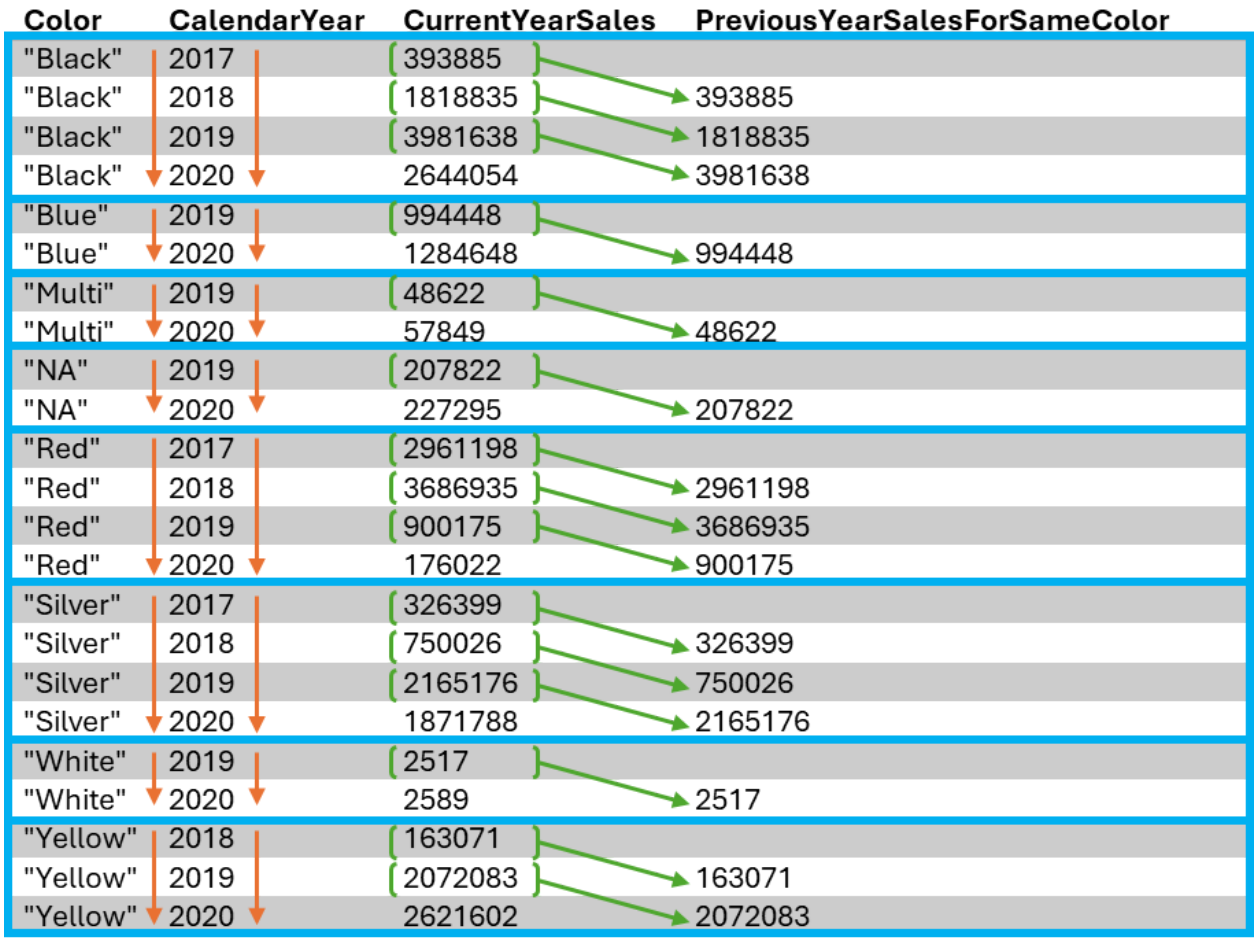
Color	CalendarYear	CurrentYearSales	PreviousYearSalesForSameColor
"Black"	2017	393885	
"Black"	2018	1818835	393885
"Black"	2019	3981638	1818835
"Black"	2020	2644054	3981638
« Blue »	2019	994448	

Color	CalendarYear	CurrentYearSales	PreviousYearSalesForSameColor
« Blue »	2020	1284648	994448
"Multi"	2019	48622	
"Multi"	2020	57849	48622
"NA"	2019	207822	
"NA"	2020	227295	207822
"Red"	2017	2961198	
"Red"	2018	3686935	2961198
"Red"	2019	900175	3686935
"Red"	2020	176022	900175
"Silver"	2017	326399	
"Silver"	2018	750026	326399
"Silver"	2019	2165176	750026
"Silver"	2020	1871788	2165176
"White"	2019	2517	
"White"	2020	2589	2517
"Yellow"	2018	163071	
"Yellow"	2019	2072083	163071
"Yellow"	2020	2621602	2072083

Comme vous pouvez le voir dans cette table, la colonne PreviousYearSalesForSameColor affiche les ventes de l'année précédente pour la même couleur. Pour [Red, 2020], elle retourne les ventes pour [Red, 2019], et ainsi de suite. S'il n'y a pas d'année précédente, comme c'est le cas pour [Red, 2017], aucune valeur n'est retournée.

Vous pouvez voir PARTITIONBY comme un moyen de diviser la table en différentes parties dans lesquelles exécuter le calcul de OFFSET. Dans l'exemple ci-dessus, la table est divisée en autant de parties qu'il y a de couleurs, une pour chaque couleur. Ensuite, dans chaque partie, OFFSET est calculé, avec un tri en fonction de CalendarYear.

Visuellement, voici ce qu'il se passe :



Color	CalendarYear	CurrentYearSales	PreviousYearSalesForSameColor
"Black"	2017	393885	
"Black"	2018	1818835	393885
"Black"	2019	3981638	1818835
"Black"	2020	2644054	3981638
"Blue"	2019	994448	
"Blue"	2020	1284648	994448
"Multi"	2019	48622	
"Multi"	2020	57849	48622
"NA"	2019	207822	
"NA"	2020	227295	207822
"Red"	2017	2961198	
"Red"	2018	3686935	2961198
"Red"	2019	900175	3686935
"Red"	2020	176022	900175
"Silver"	2017	326399	
"Silver"	2018	750026	326399
"Silver"	2019	2165176	750026
"Silver"	2020	1871788	2165176
"White"	2019	2517	
"White"	2020	2589	2517
"Yellow"	2018	163071	
"Yellow"	2019	2072083	163071
"Yellow"	2020	2621602	2072083

Tout d'abord, l'appel à PARTITIONBY a pour effet de diviser la table en différentes parties, une pour chaque couleur. Cela est représenté par les zones bleu clair dans l'image de la table. Ensuite, ORDERBY fait en sorte que chaque partie soit triée en fonction de la colonne CalendarYear (ce qui est représenté par les flèches orange). Enfin, dans chaque partie triée, OFFSET recherche la ligne précédente et retourne cette valeur dans la colonne PreviousYearSalesForSameColor. Comme aucune ligne ne précède la première de ligne de chaque partie, la colonne PreviousYearSalesForSameColor est vide pour cette ligne.

Pour obtenir le résultat final, il nous suffit de soustraire CurrentYearSales des ventes de l'année précédente pour la même couleur retournées par l'appel à OFFSET. Comme nous ne sommes pas intéressés par l'affichage des ventes de l'année précédente pour la même couleur, mais par seulement les ventes de l'année en cours et la différence d'une année sur l'autre, voici l'expression de table calculée finale :

DAX

```
FinalResult =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
```

```
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "YoYSalesForSameColor",
        [CurrentYearSales] -
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([CalendarYear]),
                PARTITIONBY ([Color])
            ),
            [CurrentYearSales]
        )
    )
```

Et voici le résultat de cette expression :

Agrandir le tableau

Color	CalendarYear	CurrentYearSales	YoYSalesForSameColor
"Black"	2017	393885	393885
"Black"	2018	1818835	1424950
"Black"	2019	3981638	2162803
"Black"	2020	2644054	-1337584
« Blue »	2019	994448	994448
« Blue »	2020	1284648	290200
"Multi"	2019	48622	48622
"Multi"	2020	57849	9227
"NA"	2019	207822	207822
"NA"	2020	227295	19473
"Red"	2017	2961198	2961198
"Red"	2018	3686935	725737
"Red"	2019	900175	-2786760
"Red"	2020	176022	-724153

Color	CalendarYear	CurrentYearSales	YoYSalesForSameColor
"Silver"	2017	326399	326399
"Silver"	2018	750026	423627
"Silver"	2019	2165176	1415150
"Silver"	2020	1871788	-293388
"White"	2019	2517	2517
"White"	2020	2589	72
"Yellow"	2018	163071	163071
"Yellow"	2019	2072083	1909012
"Yellow"	2020	2621602	549519

Utilisation de MATCHBY

Vous avez peut-être remarqué que nous n'avons pas du tout spécifié MATCHBY. Dans ce cas, ce n'est pas nécessaire. Les colonnes spécifiées dans ORDERBY et PARTITIONBY (pour autant qu'elles aient été spécifiées dans les exemples ci-dessus) suffisent à identifier chaque ligne de manière unique. Comme nous n'avons pas spécifié MATCHBY, les colonnes spécifiées dans ORDERBY et PARTITIONBY sont utilisées pour identifier de manière unique chaque ligne afin qu'elles puissent être comparées pour permettre à OFFSET de produire un résultat significatif. Si les colonnes spécifiées dans ORDERBY et PARTITIONBY ne peuvent pas identifier chaque ligne de manière unique, des colonnes supplémentaires peuvent être ajoutées à la clause ORDERBY si ces colonnes supplémentaires permettent d'identifier chaque ligne de manière unique. Si ce n'est pas possible, une erreur est retournée. Dans ce dernier cas, la spécification de MATCHBY peut contribuer à résoudre l'erreur.

Si MATCHBY est spécifié, les colonnes spécifiées dans MATCHBY et PARTITIONBY sont utilisées pour identifier chaque ligne de manière unique. Si ce n'est pas possible, une erreur est retournée. Même si MATCHBY n'est pas obligatoire, envisagez de spécifier explicitement MATCHBY pour éviter toute confusion.

Dans la continuité des exemples ci-dessus, voici la dernière expression :

DAX

```
FinalResult =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
```

```
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "YoYSalesForSameColor",
        [CurrentYearSales] -
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([CalendarYear]),
                PARTITIONBY ([Color])
            ),
            [CurrentYearSales]
        )
    )
```

Si nous voulons être explicites sur la façon dont les lignes doivent être identifiées de manière unique, nous pouvons spécifier MATCHBY comme dans l'expression équivalente suivante :

DAX

```
FinalResultWithExplicitMATCHBYOnColorAndCalendarYear =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "YoYSalesForSameColor",
        [CurrentYearSales] -
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([CalendarYear]),
                PARTITIONBY ([Color]),
                MATCHBY ([Color], [CalendarYear])
            ),
            [CurrentYearSales]
        )
    )
```

MATCHBY étant spécifié, les colonnes spécifiées à la fois dans MATCHBY et PARTITIONBY sont utilisées pour identifier les lignes de manière unique. Comme Color est spécifié à la fois dans MATCHBY et PARTITIONBY, l'expression suivante est équivalente à l'expression précédente :

DAX

```
FinalResultWithExplicitMATCHBYOnCalendarYear =
    VAR vRelation = SUMMARIZECOLUMNS (
        DimProduct[Color],
        DimDate[CalendarYear],
        "CurrentYearSales", ROUND ( SUM ( FactInternetSales[SalesAmount] ),
0 )
        )
    RETURN
    ADDCOLUMNS (
        vRelation,
        "YoYSalesForSameColor",
        [CurrentYearSales] -
        SELECTCOLUMNS (
            OFFSET (
                -1,
                vRelation,
                ORDERBY ([CalendarYear]),
                PARTITIONBY ([Color]),
                MATCHBY ([CalendarYear])
            ),
            [CurrentYearSales]
        )
    )
```

Sachant qu'il n'était pas nécessaire de spécifier MATCHBY dans les exemples que nous avons vus jusqu'à présent, intéressons-nous à présent à un exemple légèrement différent qui nécessite MATCHBY. Dans ce cas, nous avons une liste de lignes de commande. Chaque ligne représente une ligne de commande pour une commande. Une commande peut être constituée de plusieurs lignes de commande et la ligne de commande 1 apparaît sur de nombreuses commandes. Par ailleurs, pour chaque ligne de commande, nous avons un élément ProductKey (Clé de produit) et un élément SalesAmount (Montant de la vente). Voici un aperçu des colonnes importantes de la table :

Agrandir le tableau

SalesOrderNumber	SalesOrderLineNumber	ProductKey	SalesAmount
SO51900	1	528	4.99

SalesOrderNumber	SalesOrderLineNumber	ProductKey	SalesAmount
SO51948	1	528	5,99
SO52043	1	528	4.99
SO52045	1	528	4.99
SO52094	1	528	4.99
SO52175	1	528	4.99
SO52190	1	528	4.99
SO52232	1	528	4.99
SO52234	1	528	4.99
SO52234	2	529	3.99

Notez que SalesOrderNumber et SalesOrderLineNumber sont tous deux nécessaires pour identifier les lignes de manière unique.

Pour chaque commande, nous voulons obtenir le montant de ventes précédent du même produit (représenté par ProductKey) commandé à hauteur de SalesAmount par ordre décroissant. L'expression suivante ne fonctionnera pas, car il peut exister plusieurs lignes dans vRelation au moment où ce dernier est passé à OFFSET :

DAX

```
ThisExpressionFailsBecauseMATCHBYIsMissing =
    ADDCOLUMNS (
        FactInternetSales,
        "Previous Sales Amount",
            SELECTCOLUMNS (
                OFFSET (
                    -1,
                    FactInternetSales,
                    ORDERBY ( FactInternetSales[SalesAmount], DESC ),
                    PARTITIONBY ( FactInternetSales[ProductKey] )
                ),
                FactInternetSales[SalesAmount]
            )
    )
```

Cette expression retourne une erreur indiquant que le paramètre Relation de OFFSET comportement potentiellement des lignes en double, ce qui n'est pas autorisé.

Pour que cette expression fonctionne, MATCHBY doit être spécifié et inclure toutes les colonnes qui définissent une ligne de manière unique. MATCHBY est nécessaire ici, car la relation, FactInternetSales, ne contient pas de clés explicites ni de colonnes uniques. Cependant, les colonnes SalesOrderNumber et SalesOrderLineNumber forment ensemble une *clé composite*, où leur existence conjointe est unique dans la relation et peut donc identifier chaque ligne de manière unique. Le simple fait de spécifier SalesOrderNumber ou SalesOrderLineNumber ne suffit pas, car les deux colonnes contiennent des valeurs qui se répètent. L'expression suivante résout le problème :

DAX

```
ThisExpressionWorksBecauseOfMATCHBY =
    ADDCOLUMNS (
        FactInternetSales,
        "Previous Sales Amount",
            SELECTCOLUMNS (
                OFFSET (
                    -1,
                    FactInternetSales,
                    ORDERBY ( FactInternetSales[SalesAmount], DESC ),
                    PARTITIONBY ( FactInternetSales[ProductKey] ),
                    MATCHBY ( FactInternetSales[SalesOrderNumber],
                                FactInternetSales[SalesOrderLineNumber] )
                ),
                FactInternetSales[SalesAmount]
            )
    )
```

Et cette expression retourne en effet les résultats qui nous intéressent :

Agrandir le tableau

SalesOrderNumber	SalesOrderLineNumber	ProductKey	SalesAmount	Montant de ventes précédent
SO51900	1	528	5,99	
SO51948	1	528	4.99	5,99
SO52043	1	528	4.99	4.99
SO52045	1	528	4.99	4.99
SO52094	1	528	4.99	4.99
SO52175	1	528	4.99	4.99

SalesOrderNumber	SalesOrderLineNumber	ProductKey	SalesAmount	Montant de ventes précédent
SO52190	1	528	4.99	4.99
SO52232	1	528	4.99	4.99
SO52234	1	528	4.99	4.99
SO52234	2	529	3.99	

Contenu connexe

ORDERBY
PARTITIONBY
MATCHBY
INDEX
DÉCALAGE
WINDOW
RANK
ROWNUMBER

Commentaires

Cette page a-t-elle été utile ? Yes No

Utilisation appropriée des fonctions d’erreur

Article • 06/04/2023

En tant que modeleur de données, quand vous écrivez une expression DAX qui pourrait entraîner une erreur de temps d’évaluation, vous pouvez utiliser deux fonctions DAX pratiques.

- La fonction ISERROR, qui prend une seule expression et retourne la valeur TRUE si cette expression entraîne une erreur.
- La fonction IFERROR, qui prend deux expressions. Si la première expression entraîne une erreur, la valeur de la seconde expression est retournée. Il s’agit en fait d’une meilleure implémentation de l'imbrication de la fonction ISERROR dans une fonction IF.

Cependant, même si ces fonctions peuvent être utiles et faciliter la compréhension des expressions, elles peuvent aussi dégrader considérablement les performances des calculs. Cela peut arriver parce que ces fonctions augmentent le nombre d'analyses requises par le moteur de stockage.

La plupart des erreurs de temps d’évaluation sont dues à des valeurs vides (BLANK) inattendues ou à des valeurs nulles, ou à une conversion de type de données non valide.

Recommandations

Il vaut mieux éviter d'utiliser les fonctions ISERROR et IFERROR. Appliquez plutôt des stratégies défensives lors de la conception du modèle et de l'écriture des expressions. Les stratégies peuvent inclure :

- **S'assurer que des données de qualité sont chargées dans le modèle :** Utilisez les transformations Power Query pour supprimer ou substituer des valeurs non valides ou manquantes, et pour définir des types de données corrects. Une transformation Power Query peut également être utilisée pour filtrer les lignes lorsque des erreurs, notamment une conversion de données non valide, surviennent.

  La qualité des données peut également être contrôlée en désactivant la propriété **Is Nullable** de la colonne du modèle, ce qui entraînera l’échec de l’actualisation des données si des valeurs vides (BLANK) sont détectées. Si cet échec se produit, les données chargées à la suite d'une actualisation réussie resteront dans les tables.

- **Utilisation de la fonction IF :** L'expression de test logique de la fonction IF peut déterminer si un résultat d'erreur se produirait. Remarque : comme les fonctions ISERROR et IFERROR, cette fonction peut entraîner des analyses supplémentaires du moteur de stockage, mais elle sera probablement plus performante car aucune erreur ne doit être déclenchée.

- **Utilisation de fonctions à tolérance d'erreur :** Certaines fonctions DAX testent et compensent les conditions d'erreur. Ces fonctions vous permettent de saisir un autre résultat qui serait renvoyé à la place. La fonction DIVIDE en est un exemple. Pour plus d'informations sur cette fonction, lisez l'article DAX : fonction DIVIDE ou opérateur de division (/).

Exemple

L'expression de mesure suivante teste si une erreur s'affichera. Elle retourne une valeur BLANK dans ce cas (ce qui est le cas lorsque vous ne fournissez pas à la fonction IF une expression value-if-false).

DAX

```
Profit Margin
= IF(ISERROR([Profit] / [Sales]))
```

Cette prochaine version de l'expression de mesure a été améliorée en utilisant la fonction IFERROR à la place des fonctions IF et ISERROR.

DAX

```
Profit Margin
= IFERROR([Profit] / [Sales], BLANK())
```

Mais cette version finale de l'expression de mesure aboutit au même résultat, mais de façon plus efficace et fluide.

DAX

```
Profit Margin
= DIVIDE([Profit], [Sales])
```

Contenu connexe

- Parcours d'apprentissage : Utiliser DAX dans Power BI Desktop

- Des questions ? Essayez d'interroger la communauté Power BI
- Vous avez des suggestions ? Envoyez-nous vos idées pour améliorer Power BI

Commentaires

Cette page a-t-elle été utile ? Yes No

Éviter de convertir des blancs (BLANK) en valeurs

Article • 06/04/2023

En tant que modeleur de données, quand vous écrivez des expressions de mesure, vous pouvez rencontrer des cas où une valeur significative ne peut pas être retournée. Dès lors, vous pouvez être tenté de retourner une valeur (comme zéro) à la place. Nous vous suggérons de déterminer soigneusement si cette conception est efficace et pratique.

Considérez la définition de mesure suivante qui convertit explicitement les résultats BLANK en zéro.

DAX

```
Sales (No Blank) =
IF(
    ISBLANK([Sales]),
    0,
    [Sales]
)
```

Prenons une autre définition de mesure qui convertit également les résultats BLANK en zéro.

DAX

```
Profit Margin =
DIVIDE([Profit], [Sales], 0)
```

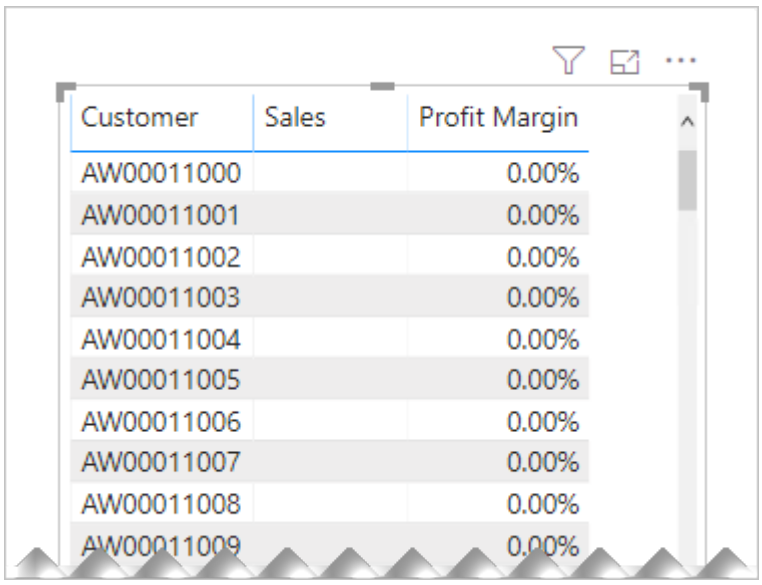
La fonction DIVIDE divise la mesure **Profit** par la mesure **Sales**. Si le résultat est zéro ou BLANK, le troisième argument (le résultat alternatif facultatif) est retourné. Dans cet exemple, étant donné que zéro est passé comme résultat alternatif, la mesure est certaine de toujours retourner une valeur.

Ces conceptions de mesures sont inefficaces et entraînent des conceptions de rapports médiocres.

Quand elles sont ajoutées à un visuel de rapport, Power BI tente de récupérer tous les regroupements dans le contexte de filtre. L'évaluation et la récupération de résultats de requête volumineux entraînent souvent un ralentissement du rendu des rapports. Chaque exemple de mesure convertit efficacement un calcul fragmenté en calcul dense, ce qui oblige Power BI à utiliser plus de mémoire que nécessaire.

De plus, un trop grand nombre de regroupements accable souvent les utilisateurs de vos rapports.

Voyons ce qui se passe lorsque la mesure **Profit Margin** est ajoutée à un visuel de table, avec un regroupement par client.



Customer	Sales	Profit Margin
AW00011000		0.00%
AW00011001		0.00%
AW00011002		0.00%
AW00011003		0.00%
AW00011004		0.00%
AW00011005		0.00%
AW00011006		0.00%
AW00011007		0.00%
AW00011008		0.00%
AW00011009		0.00%

Le visuel de table affiche un nombre décourageant de lignes. (Il existe en fait 18 484 clients dans le modèle et la table essaie donc de tous les afficher.) Remarquez que les clients en vue n'ont réalisé aucune vente. Toutefois, puisque la mesure **Profit Margin** retourne toujours une valeur, ils sont affichés.

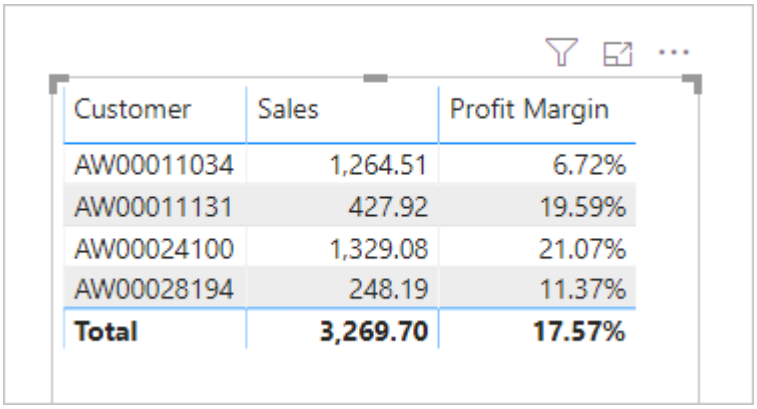
> ⓘ **Notes**
>
> Lorsqu'il y a trop de points de données à afficher dans un visuel, Power BI peut utiliser des stratégies de réduction des données pour supprimer ou synthétiser des résultats de requêtes volumineux. Pour plus d'informations, consultez **Limites et stratégies par type de visuel du point de données**.

Voyons ce qui se passe lorsque la mesure **Profit Margin** est améliorée. Elle retourne désormais une seule valeur quand la mesure **Sales** n'est pas BLANK (ou zéro).

DAX

```
Profit Margin =
DIVIDE([Profit], [Sales])
```

Le visuel de table présente maintenant uniquement les clients qui ont effectué des ventes dans le contexte de filtre actuel. La mesure ainsi améliorée engendre une expérience plus efficace et plus pratique pour les utilisateurs de vos rapports.



Customer	Sales	Profit Margin
AW00011034	1,264.51	6.72%
AW00011131	427.92	19.59%
AW00024100	1,329.08	21.07%
AW00028194	248.19	11.37%
Total	**3,269.70**	**17.57%**

Conseil

Si nécessaire, vous pouvez configurer un visuel pour afficher tous les groupes (qui retournent des valeurs ou BLANK) dans le contexte de filtre en activant l'option **Afficher les éléments sans données**.

Recommandation

Il est recommandé que vos mesures retournent BLANK quand aucune valeur significative ne peut être retournée.

Cette approche de conception est efficace, car elle permet à Power BI d'afficher les rapports plus rapidement. De plus, il est préférable de retourner BLANK parce que les visuels de rapport éliminent par défaut les regroupements quand les totalisations ont la valeur BLANK.

Contenu connexe

- Parcours d'apprentissage : Utiliser DAX dans Power BI Desktop
- Des questions ? Essayez d'interroger la communauté Power BI
- Vous avez des suggestions ? Envoyez-nous vos idées pour améliorer Power BI

Commentaires

Cette page a-t-elle été utile ? Yes No

Évitez d'utiliser FILTER comme argument de filtre

Article • 06/04/2023

En tant que modeleur de données, vous êtes fréquemment amené à écrire des expressions DAX qui doivent être évaluées dans un contexte de filtre modifié. Par exemple, vous pouvez écrire une définition de mesure pour calculer les ventes de « produits à marge élevée ». Nous décrirons ce calcul plus loin dans cet article.

> ⓘ **Notes**
>
> Cet article est particulièrement utile pour les calculs de modèle qui appliquent des filtres aux tables d'importation.

Les fonctions DAX CALCULATE et CALCULATETABLE sont des fonctions importantes et très utiles. Elles vous permettent d'écrire des calculs qui suppriment ou ajoutent des filtres, et de modifier des chemins de relation. Pour cela, vous devez passer des arguments de filtre (expressions booléennes, expressions de table ou fonctions de filtre spécial). Dans cet article, nous aborderons uniquement les expressions booléennes et les expressions de table.

Intéressons-nous maintenant à la définition de mesure suivante, qui calcule les ventes de produits de couleur rouge à l'aide d'une expression de table. Celle-ci va remplacer tous les filtres qui sont appliqués à la table **Product**.

DAX

```
Red Sales =
CALCULATE(
    [Sales],
    FILTER('Product', 'Product'[Color] = "Red")
)
```

La fonction CALCULATE accepte une expression de table retournée par la fonction DAX FILTER, qui évalue son expression de filtre pour chaque ligne de la table **Product**. Elle obtient le bon résultat, c'est-à-dire le résultat des ventes des produits de couleur rouge. Toutefois, le résultat peut être obtenu plus efficacement à l'aide d'une expression booléenne.

Voici une définition de mesure améliorée, qui utilise une expression booléenne au lieu d'une expression de table. La fonction DAX KEEPFILTERS garantit que tous les filtres

existants appliqués à la colonne **Couleur** sont conservés et ne sont pas remplacés.

DAX

```
Red Sales =
CALCULATE(
    [Sales],
    KEEPFILTERS('Product'[Color] = "Red")
)
```

Dans la mesure du possible, il est recommandé de passer des arguments de filtre en tant qu'expressions booléennes. Ceci est recommandé car les tables de modèle d'importation sont des magasins de colonnes en mémoire. Ils sont optimisés de manière explicite pour filtrer efficacement les colonnes ainsi.

Toutefois, des restrictions s'appliquent aux expressions booléennes lorsqu'elles sont utilisées en tant qu'arguments de filtre. Voici ces restrictions :

- Impossible de référencer des colonnes de plusieurs tables
- Elles ne peuvent pas référencer une mesure.
- Elles ne peuvent pas utiliser des fonctions CALCULATE imbriquées.
- Elles ne peuvent pas utiliser des fonctions qui analysent ou retournent une table.

Cela signifie que vous devez utiliser des expressions de table lorsque les exigences de filtre sont complexes.

Voyons maintenant une autre définition de mesure. L'exigence consiste à calculer les ventes, mais uniquement pour les mois où l'entreprise a réalisé des bénéfices.

DAX

```
Sales for Profitable Months =
CALCULATE(
    [Sales],
    FILTER(
        VALUES('Date'[Month]),
        [Profit] > 0)
    )
)
```

Dans cet exemple, vous pouvez utiliser une fonction FILTER. Ceci est dû au fait qu'il est nécessaire d'évaluer la mesure **Profit** pour éliminer les mois où aucun bénéfice n'est réalisé. Vous ne pouvez pas utiliser de mesure dans une expression booléenne quand celle-ci est utilisée comme un argument de filtre.

Recommandations

Pour des performances optimales, il est recommandé d'utiliser des expressions booléennes comme arguments de filtre chaque fois que cela vous est possible.

Par conséquent, vous ne devez utiliser les fonctions de filtre que lorsque cela est nécessaire. Vous pouvez les utiliser pour effectuer des comparaisons de colonnes à filtre complexe. Ces comparaisons de colonnes peuvent impliquer :

- Mesures
- D'autres colonnes
- L'utilisation de la fonction DAX ou , ou de l'opérateur logique OR (||)

Contenu connexe

- Fonctions de filtrage (DAX)
- Parcours d'apprentissage : Utiliser DAX dans Power BI Desktop
- Des questions ? Essayez d'interroger la communauté Power BI
- Vous avez des suggestions ? Envoyez-nous vos idées pour améliorer Power BI

Commentaires

Cette page a-t-elle été utile ? Yes No

Références de colonne et de mesure

Article • 06/04/2023

Parce qu'elles modélisent les données, vos expressions DAX référencent des colonnes et des mesures de modèle. Les colonnes et les mesures sont toujours associées à des tables de modèle, mais ces associations sont différentes. Nous avons donc des recommandations différentes sur la façon de les référencer dans vos expressions.

Colonnes

Une colonne est un objet de niveau table et son nom doit être unique dans une table. Vous pouvez donc avoir le même nom de colonne plusieurs fois dans votre modèle, à condition que ces colonnes appartiennent à des tables différentes. Il existe une autre règle : un nom de colonne ne peut pas être identique à un nom de mesure ou un nom de hiérarchie qui existe dans la même table.

En général, DAX n'impose pas de référence *complète* à une colonne. Une référence complète signifie que le nom de table précède le nom de colonne.

Voici un exemple de définition de colonne calculée qui utilise uniquement des références de nom de colonne. Les colonnes **Sales** et **Cost** appartiennent toutes deux à une table nommée **Orders**.

DAX

```
Profit = [Sales] - [Cost]
```

La même définition peut être réécrite avec des références de colonne complètes.

DAX

```
Profit = Orders[Sales] - Orders[Cost]
```

Vous devez parfois utiliser des références de colonne complètes quand Power BI détecte une ambiguïté. Quand vous entrez une formule, un message d'erreur et un trait ondulé rouge vous avertissent. Par ailleurs, certaines fonctions DAX, comme LOOKUPVALUE, nécessitent l'utilisation de références de colonne complètes.

Il est recommandé de toujours utiliser des références de colonne complètes. Les raisons sont fournies dans la section Recommandations.

Mesures

Une mesure est un objet de niveau modèle. Pour cette raison, les noms de mesure doivent être uniques dans le modèle. Toutefois, dans le volet **Champs**, les auteurs de rapports voient chaque mesure associée à une seule table de modèle. Cette association est définie pour des raisons esthétiques, et vous pouvez la configurer en définissant la propriété **Table principale** de la mesure. Pour plus d'informations, consultez Mesures dans Power BI Desktop (Organisation de vos mesures).

Vous pouvez utiliser une référence de mesure complète dans vos expressions. La fonctionnalité IntelliSense de DAX vous offre même la suggestion. Toutefois, ce n'est pas nécessaire et ce n'est pas une pratique recommandée. Si vous changez la table principale d'une mesure, toute expression qui utilise une référence de mesure complète s'arrête. Vous devez ensuite modifier chaque formule rompue pour supprimer (ou mettre à jour) la référence de mesure.

Il est recommandé de ne jamais utiliser de références complètes de mesure. Les raisons sont fournies dans la section Recommandations.

Recommandations

Nos recommandations sont simples et faciles à mémoriser :

- Toujours utiliser des références de colonne complètes
- Ne jamais utiliser de références de mesure complètes

Voici pourquoi :

- **Entrée de formule** : Les expressions sont acceptées, car il n'y a pas de référence ambiguë à résoudre. Par ailleurs, vous respectez les exigences de ces fonctions DAX qui demandent des références de colonne complètes.
- **Robustesse** : Les expressions continuent de fonctionner, même quand vous changez la propriété de table principale d'une mesure.
- **Lisibilité** : Les expressions sont rapides et faciles à comprendre : vous déterminez rapidement s'il s'agit d'une colonne ou d'une mesure, selon que sa référence est complète ou non.

Contenu connexe

- Parcours d'apprentissage : Utiliser DAX dans Power BI Desktop
- Des questions ? Essayez d'interroger la communauté Power BI
- Vous avez des suggestions ? Envoyez-nous vos idées pour améliorer Power BI

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonction DIVIDE ou opérateur de division (/)

Article • 06/04/2023

En tant que modeleur de données, quand vous écrivez une expression DAX pour diviser un numérateur par un dénominateur, vous pouvez choisir d'utiliser la fonction DIVIDE ou l'opérateur de division (/, barre oblique).

Lorsque vous utilisez la fonction DIVIDE, vous devez passer les expressions de numérateur et dénominateur. Si vous le souhaitez, vous pouvez passer une valeur qui représente un *autre résultat*.

DAX

```
DIVIDE(<numerator>, <denominator> [,<alternateresult>])
```

La fonction DIVIDE a été conçue pour gérer automatiquement les cas de division par zéro. Si aucun autre résultat n'est passé et que le dénominateur a la valeur zéro ou BLANK, la fonction retourne BLANK. Quand un autre résultat est passé, il est retourné à la place de BLANK.

La fonction DIVIDE est pratique, car elle évite à votre expression d'avoir à commencer par tester la valeur du dénominateur. La fonction est également mieux optimisée pour tester la valeur de dénominateur que la fonction IF. Le gain de performance est significatif, car la vérification de division par zéro est coûteuse. L'utilisation poussée de DIVIDE produit une expression plus concise et plus fluide.

Exemple

L'expression de mesure suivante produit une division sans risque, mais elle implique l'utilisation de quatre fonctions DAX.

DAX

```
Profit Margin =
IF(
    OR(
        ISBLANK([Sales]),
        [Sales] == 0
    ),
    BLANK(),
```

```
    [Profit] / [Sales]
)
```

Cette expression de mesure aboutit au même résultat, mais de façon plus efficace et fluide.

DAX

```
Profit Margin =
DIVIDE([Profit], [Sales])
```

Recommandations

Il est recommandé d'utiliser la fonction DIVIDE chaque fois que le dénominateur est une expression qui *peut* retourner zéro ou BLANK.

Dans le cas où le dénominateur est une valeur constante, nous vous recommandons d'utiliser l'opérateur de division. Dans ce cas, la réussite de la division est garantie et votre expression est plus performante, car elle évite les tests inutiles.

Déterminez avec soin si la fonction DIVIDE doit retourner une autre valeur. Pour les mesures, il est généralement préférable qu'elle retourne BLANK. Il est préférable de retourner BLANK parce que les visuels de rapport éliminent par défaut les regroupements quand les totalisations ont la valeur BLANK. Cela permet au visuel de fixer son attention sur les groupes où il existe des données. Si nécessaire, dans Power BI, vous pouvez configurer le visuel pour afficher tous les groupes (qui retournent des valeurs ou BLANK) dans le contexte de filtre en activant l'option Afficher les éléments sans données.

Contenu connexe

- Parcours d'apprentissage : Utiliser DAX dans Power BI Desktop
- Des questions ? Essayez d'interroger la communauté Power BI
- Vous avez des suggestions ? Envoyez-nous vos idées pour améliorer Power BI

Commentaires

Cette page a-t-elle été utile ? Yes No

Utiliser SELECTEDVALUE à la place de VALUES

Article • 14/12/2023

En tant que modeleur de données, vous pouvez parfois avoir besoin d'écrire une expression DAX qui teste si une colonne est filtrée par une valeur spécifique.

Dans les versions antérieures de DAX, cette condition a été remplie sans problème avec un modèle impliquant trois fonctions DAX ; IF, HASONEVALUE et VALUES. La définition de mesure suivante présente un exemple. Elle calcule le montant des taxes de vente, mais uniquement pour les ventes effectuées auprès de clients australiens.

DAX

```
Australian Sales Tax =
IF(
    HASONEVALUE(Customer[Country-Region]),
    IF(
        VALUES(Customer[Country-Region]) = "Australia",
        [Sales] * 0.10
    )
)
```

Dans l'exemple, la fonction HASONEVALUE retourne TRUE uniquement lorsqu'une valeur de la colonne **Country-Region** est visible dans le contexte filtré actuel. Quand la valeur est TRUE, la fonction VALUES est comparée au texte littéral « Australia ». Lorsque la fonction VALUES retourne la valeur TRUE, la mesure **Sales** est multipliée par 0,10 (qui représente 10 %). Si la fonction HASONEVALUE retourne la valeur FALSE (car plusieurs valeurs filtrent la colonne), la première fonction IF retourne BLANK.

L'utilisation de HASONEVALUE est une technique défensive. Elle est nécessaire parce qu'il est possible que plusieurs valeurs filtrent la colonne **Country-Region**. Dans ce cas, la fonction VALUES retourne une table de plusieurs lignes. La comparaison d'une table de plusieurs lignes à une valeur scalaire génère une erreur.

Recommandation

Il est recommandé d'utiliser la fonction SELECTEDVALUE. Elle permet d'obtenir le même résultat que le modèle décrit dans cet article, mais de manière plus efficace et plus élégante.

En utilisant la fonction SELECTEDVALUE, l'exemple de définition de mesure est maintenant réécrit.

DAX

```
Australian Sales Tax =
IF(
    SELECTEDVALUE(Customer[Country-Region]) = "Australia",
    [Sales] * 0.10
)
```

Conseil

Il est possible de passer une valeur de *résultat alternatif* dans la fonction SELECTEDVALUE. La valeur de résultat alternatif est retournée quand aucun filtre n'est appliqué ou plusieurs filtres sont appliqués à la colonne.

Contenu connexe

- Parcours d'apprentissage : Utiliser DAX dans Power BI Desktop
- Des questions ? Essayez d'interroger la communauté Power BI
- Vous avez des suggestions ? Envoyez-nous vos idées pour améliorer Power BI

Commentaires

Cette page a-t-elle été utile ? Yes No

Utiliser COUNTROWS à la place de COUNT

Article • 06/04/2023

En tant que modeleur de données, vous pouvez parfois avoir besoin d'écrire une expression DAX qui compte les lignes de la table. La table peut être une table de modèle ou une expression qui retourne une table.

Cette condition peut être remplie de deux manières. Vous pouvez utiliser la fonction COUNT pour compter les valeurs de colonne ou vous pouvez utiliser la fonction COUNTROWS pour compter les lignes de la table. Les deux fonctions aboutissent au même résultat, dans la mesure où la colonne comptée ne contient pas de valeurs BLANK.

La définition de mesure suivante présente un exemple. Elle calcule le nombre de valeurs de colonne **OrderDate**.

DAX

```
Sales Orders =
COUNT(Sales[OrderDate])
```

Si la précision de la table **Sales** correspond à une seule ligne par commande et que la colonne **OrderDate** ne contient pas de valeurs BLANK, alors la mesure retourne un résultat correct.

Toutefois, la définition de mesure suivante est une meilleure solution.

DAX

```
Sales Orders =
COUNTROWS(Sales)
```

Il existe trois raisons pour lesquelles la deuxième définition de mesure est meilleure :

- Elle est plus efficace et elle est donc plus performante.
- Elle ne tient pas compte des valeurs BLANK contenues dans les colonnes de la table.
- L'objectif de la formule est plus clair et parvient à s'autodécrire.

Recommandation

Lorsque vous avez l'intention de compter les lignes de la table, il est recommandé de toujours utiliser la fonction COUNTROWS.

Contenu connexe

- Parcours d'apprentissage : Utiliser DAX dans Power BI Desktop
- Des questions ? Essayez d'interroger la communauté Power BI
- Vous avez des suggestions ? Envoyez-nous vos idées pour améliorer Power BI

Commentaires

Cette page a-t-elle été utile ? Yes No

Utiliser des variables pour améliorer vos formules DAX

Article • 06/04/2023

En tant que modeleur de données, l'écriture et le débogage de certains calculs DAX peuvent s'avérer difficiles. Les exigences de calcul complexes impliquent souvent d'écrire des expressions composées ou complexes. Les expressions composées peuvent impliquer l'utilisation de nombreuses fonctions imbriquées, et éventuellement la réutilisation de la logique d'expression.

L'utilisation de variables dans vos formules DAX peut vous aider à écrire des calculs plus complexes et efficaces. Les variables peuvent améliorer les performances, la fiabilité et la lisibilité, et réduire la complexité.

Dans cet article, nous allons démontrer les trois premiers avantages en utilisant un exemple de mesure de la croissance des ventes année par année. (La formule de croissance des ventes année par année est la période de vente moins les ventes de la même période de l'année précédente, divisée par les ventes de la même période de l'année précédente.)

Commençons par la définition de mesure suivante.

DAX

```
Sales YoY Growth % =
DIVIDE(
    ([Sales] - CALCULATE([Sales], PARALLELPERIOD('Date'[Date], -12,
MONTH))),
    CALCULATE([Sales], PARALLELPERIOD('Date'[Date], -12, MONTH))
)
```

La mesure produit le résultat correct, mais intéressons-nous maintenant à la manière de l'améliorer.

Améliorer les performances

Notez que la formule répète l'expression qui calcule « même période de l'année précédente ». Cette formule est inefficace, car elle demande que Power BI évalue deux fois la même expression. La définition de mesure peut être rendue plus efficace à l'aide d'une variable, VAR.

La définition de mesure suivante représente une amélioration. Elle utilise une expression pour affecter le résultat « même période de l'année précédente » à une variable nommée **SalesPriorYear**. La variable est ensuite utilisée deux fois dans l'expression RETURN.

DAX

```
Sales YoY Growth % =
VAR SalesPriorYear =
    CALCULATE([Sales], PARALLELPERIOD('Date'[Date], -12, MONTH))
RETURN
    DIVIDE(([Sales] - SalesPriorYear), SalesPriorYear)
```

La mesure continue à produire le résultat correct et le fait environ deux fois plus vite.

Améliorer la lisibilité

Dans la définition de mesure précédente, remarquez comme le choix du nom de la variable rend l'expression RETURN plus simple à comprendre. L'expression est courte et auto-descriptive.

Simplifier le débogage

Les variables peuvent également vous aider à déboguer une formule. Pour tester une expression affectée à une variable, vous réécrivez provisoirement l'expression RETURN pour générer la variable.

La définition de mesure suivante retourne uniquement la variable **SalesPriorYear**. Remarquez comme elle commente l'expression RETURN prévue. Cette technique vous permet de la rétablir facilement une fois que le débogage est terminé.

DAX

```
Sales YoY Growth % =
VAR SalesPriorYear =
    CALCULATE([Sales], PARALLELPERIOD('Date'[Date], -12, MONTH))
RETURN
    --DIVIDE(([Sales] - SalesPriorYear), SalesPriorYear)
    SalesPriorYear
```

Réduire la complexité

Dans les versions antérieures de DAX, les variables n'étaient pas encore prises en charge. Les expressions complexes qui ont introduit de nouveaux contextes de filtre devaient utiliser les fonctions DAX EARLIER ou EARLIEST pour faire référence à des contextes de filtre externes. Malheureusement, les modeleurs de données ont trouvé ces fonctions difficiles à comprendre et à utiliser.

Les variables sont toujours évaluées en dehors des filtres que votre expression RETURN applique. C'est pourquoi, lorsque vous utilisez une variable dans un contexte de filtre modifié, elle obtient le même résultat que la fonction EARLIEST. L'utilisation des fonctions EARLIER ou EARLIEST peut donc être évitée. Cela signifie que vous pouvez désormais écrire des formules qui sont moins complexes et plus faciles à comprendre.

Considérez la définition de colonne calculée suivante ajoutée à la table **Subcategory**. Elle évalue un classement pour chaque sous-catégorie de produit en fonction des valeurs de la colonne **Subcategory Sales**.

DAX

```
Subcategory Sales Rank =
COUNTROWS(
    FILTER(
        Subcategory,
        EARLIER(Subcategory[Subcategory Sales]) < Subcategory[Subcategory
Sales]
    )
) + 1
```

La fonction EARLIER est utilisée pour faire référence à la valeur de la colonne **Subcategory Sales***dans le contexte de ligne actuel*.

La définition de colonne calculée peut être améliorée à l'aide d'une variable au lieu de la fonction EARLIER. La variable **CurrentSubcategorySales** stocke la valeur de la colonne **Subcategory Sales***dans le contexte de ligne actuel*, puis l'expression RETURN l'utilise dans un contexte de filtre modifié.

DAX

```
Subcategory Sales Rank =
VAR CurrentSubcategorySales = Subcategory[Subcategory Sales]
RETURN
    COUNTROWS(
        FILTER(
            Subcategory,
            CurrentSubcategorySales < Subcategory[Subcategory Sales]
        )
    ) + 1
```

Contenu connexe

- Article DAX VAR
- Parcours d'apprentissage : Utiliser DAX dans Power BI Desktop
- Des questions ? Essayez d'interroger la communauté Power BI

Commentaires

Cette page a-t-elle été utile ? Yes No

Référence des fonctions DAX

Article • 20/10/2023

Les informations de référence sur les fonctions DAX fournissent des détails, notamment la syntaxe, les paramètres, les valeurs de retour et les exemples pour chacune des plus de 250 fonctions utilisées dans les formules DAX (Data Analysis Expression).

> **ⓘ Important**
>
> Toutes les fonctions DAX ne sont pas prises en charge ou incluses dans les versions antérieures de Power BI Desktop, Analysis Services et Power Pivot dans Excel.

Dans cette section

Nouvelles fonctions DAX – Ces fonctions sont nouvelles ou correspondent à des fonctions existantes qui ont fait l'objet d'une mise à jour importante.

Fonctions d'agrégation – Ces fonctions calculent une valeur (scalaire) telle que count, sum, average, minimum ou maximum pour toutes les lignes d'une colonne ou d'une table, comme défini par l'expression.

Fonctions de date et heure – Ces fonctions DAX sont similaires aux fonctions de date et heure dans Microsoft Excel. Toutefois, les fonctions DAX sont basées sur les types de données datetime utilisés par Microsoft SQL Server.

Fonctions de filtre – Ces fonctions vous permettent de retourner des types de données spécifiques, de rechercher des valeurs dans des tables associées et de filtrer les données en fonction de valeurs associées. Les fonctions de recherche utilisent des tables et des relations entre ces tables. Avec les fonctions de filtre, vous pouvez manipuler le contexte des données pour créer des calculs dynamiques.

Fonctions financières – Ces fonctions s'utilisent dans des formules qui effectuent des calculs financiers, par exemple pour connaître la valeur actuelle nette et le taux de rendement.

Fonctions informatives – Ces fonctions examinent une table ou une colonne fournie comme argument à une autre fonction et retournent des informations indiquant si la valeur correspond au type attendu. Par exemple, la fonction ISERROR retourne TRUE si la valeur que vous référencez contient une erreur.

Fonctions logiques – Ces fonctions retournent des informations sur les valeurs figurant dans une expression. Par exemple, la fonction TRUE vous permet de savoir si une expression que vous évaluez retourne une valeur TRUE.

Fonctions mathématiques et trigonométriques – Les fonctions mathématiques dans DAX sont similaires aux fonctions mathématiques et trigonométriques dans Excel. Toutefois, il existe quelques différences concernant les types de données numériques utilisés par les fonctions DAX.

Autres fonctions – Ces fonctions effectuent des actions uniques qui n'entrent dans aucune des catégories définissant la plupart des autres fonctions.

Fonctions parents et enfants – Ces fonctions aident les utilisateurs à gérer des données dans le cadre d'une hiérarchie parent/enfant dans leurs modèles de données.

Fonctions de relation – Ces fonctions permettent de gérer et d'utiliser les relations entre les tables. Par exemple, vous pouvez spécifier une relation particulière à utiliser dans un calcul.

Fonctions statistiques – Ces fonctions calculent des valeurs liées aux distributions statistiques et à la probabilité, telles que l'écart type et le nombre de permutations.

Fonctions de manipulation de table – Ces fonctions retournent une table ou manipulent des tables existantes.

Fonctions de texte – Ces fonctions vous permettent de retourner une partie d'une chaîne, de rechercher du texte dans une chaîne ou de concaténer des valeurs de chaîne. Des fonctions supplémentaires ont été conçues pour contrôler les formats des dates, des heures et des nombres.

Fonctions d'intelligence temporelle – Ces fonctions vous permettent de créer des calculs qui utilisent une connaissance intégrée des calendriers et des dates. En utilisant des plages de dates et d'heures en association avec des agrégations ou des calculs, vous pouvez générer des comparaisons significatives sur des périodes de temps comparables pour les ventes, les stocks, etc.

Contenu connexe

Informations de référence sur la syntaxe DAX
Référence des opérateurs DAX
Conventions de nommage des paramètres DAX

Commentaires

Cette page a-t-elle été utile ? Yes No

Nouvelles fonctions DAX

Article • 22/02/2024

DAX est en permanence amélioré avec de nouvelles fonctions et capacités pour prendre en charge de nouvelles fonctionnalités. De nouvelles fonctions et mises à jour sont incluses dans les mises à jour des services, des applications et des outils qui, dans la plupart des cas, sont mensuelles.

Tandis que des fonctions et capacités sont en permanence en cours de mise à jour, seules les mises à jour qui présentent un changement visible et fonctionnel exposé aux utilisateurs sont décrites dans la documentation. Sont indiquées ici les nouvelles fonctions et mises à jour de fonctions existantes datant de l'année dernière.

> **Important**
>
> Les fonctions ne sont pas toutes prises en charge dans toutes les versions de Power BI Desktop, d'Analysis Services et de Power Pivot dans Excel. Les fonctionnalités nouvelles et mises à jour sont généralement introduites dans Power BI Desktop, puis incluses ultérieurement dans Analysis Services, Power Pivot dans Excel et les outils.

Nouvelles fonctions

Agrandir le tableau

Fonction	Month	Description
FIRST	Janvier 2024	Utilisé uniquement dans les calculs de visuels. Récupère une valeur dans la matrice du visuel à partir de la première ligne d'un axe.
LAST	Janvier 2024	Utilisé uniquement dans les calculs de visuels. Récupère une valeur dans la matrice du visuel à partir de la dernière ligne d'un axe.
SUIVANT	Janvier 2024	Utilisé uniquement dans les calculs de visuels. Récupère une valeur dans la ligne suivante d'un axe situé dans la matrice du visuel.
PREVIOUS	Janvier 2024	Utilisé uniquement dans les calculs de visuels. Récupère une valeur dans la ligne précédente d'un axe situé dans la matrice du visuel.

Fonction	Month	Description
MATCHBY	Mai 2023	Définir les colonnes utilisées pour faire correspondre les données et identifier la ligne actuelle, dans une expression de fonction de fenêtre.
RANK	Avril 2023	Retourne le classement du contexte actuel dans la partition spécifiée triée selon l'ordre spécifié.
ROWNUMBER	Avril 2023	Retourne le classement unique du contexte actuel dans la partition spécifiée triée selon l'ordre spécifié.
LINEST	Février 2023	Utilise la méthode des moindres carrés pour calculer une ligne droite qui correspond le mieux aux données spécifiées.
LINESTX	Février 2023	Utilise la méthode des moindres carrés pour calculer une ligne droite qui correspond le mieux aux données spécifiées. Les données sont le résultat des expressions évaluées pour chaque ligne d'une table.

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonctions d’agrégation

Article • 20/10/2023

Les fonctions d’agrégation calculent une valeur (scalaire) telle que count, sum, average, minimum ou maximum pour toutes les lignes d’une colonne ou d’une table, comme défini par l’expression.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
APPROXIMATEDISTINCTCOUNT	Retourne un nombre *estimé* de valeurs uniques dans une colonne.
AVERAGE	Retourne la moyenne (arithmétique) de tous les nombres d’une colonne.
AVERAGEA	Retourne la moyenne (arithmétique) des valeurs d’une colonne.
AVERAGEX	Calcule la moyenne (arithmétique) d’un ensemble d’expressions évaluées sur une table.
COUNT	Compte le nombre de lignes dans la colonne spécifiée qui contiennent des valeurs non vides.
COUNTA	Compte le nombre de lignes dans la colonne spécifiée qui contiennent des valeurs non vides.
COUNTAX	Compte les résultats non vides lors de l'évaluation du résultat d'une expression sur une table.
COUNTBLANK	Compte le nombre de cellules vides dans une colonne.
COUNTROWS	Compte le nombre de lignes dans la table spécifiée ou dans une table définie par une expression.
COUNTX	Compte le nombre de lignes qui contiennent un nombre ou une expression qui prend la valeur d’un nombre lors de l’évaluation d’une expression sur une table.
DISTINCTCOUNT	Compte le nombre de valeurs distinctes dans une colonne.
DISTINCTCOUNTNOBLANK	Compte le nombre de valeurs distinctes dans une colonne.
MAX	Retourne la plus grande valeur numérique d’une colonne ou

Fonction	Description
	entre deux expressions scalaires.
MAXA	Retourne la plus grande valeur d'une colonne.
MAXX	Évalue une expression pour chaque ligne d'une table et retourne la plus grande valeur numérique.
MIN	Retourne la plus petite valeur numérique d'une colonne ou entre deux expressions scalaires.
MINA	Retourne la plus petite valeur d'une colonne, notamment les valeurs logiques et les nombres représentés sous forme de texte.
MINX	Retourne la plus petite valeur numérique qui résulte de l'évaluation d'une expression pour chaque ligne d'une table.
PRODUCT	Retourne le produit des nombres d'une colonne.
PRODUCTX	Retourne le produit d'une expression évaluée pour chaque ligne d'une table.
SUM	Additionne tous les nombres d'une colonne.
SUMX	Retourne la somme d'une expression évaluée pour chaque ligne d'une table.

Commentaires

Cette page a-t-elle été utile ? Yes No

APPROXIMATEDISTINCTCOUNT

Article • 20/10/2023

Retourne un nombre *estimé* de valeurs uniques dans une colonne. Cette fonction appelle une opération d'agrégation correspondante dans la source de données, qui est optimisée pour les performances des requêtes, mais avec une précision légèrement moindre. Cette fonction peut être utilisée avec les sources de données suivantes : Azure SQL, pool SQL dédiée Azure Synapse, BigQuery, Databricks et Snowflake. Cette fonction nécessite le mode DirectQuery. Le mode d'importation et le mode de stockage double ne sont pas pris en charge.

Syntaxe

DAX

```
APPROXIMATEDISTINCTCOUNT(<columnName>)
```

Paramètres

Agrandir le tableau

Terme	Description
colonne	Colonne contenant les valeurs à compter. Il ne peut pas s'agir d'une expression.

Valeur renvoyée

Nombre approximatif de valeurs distinctes dans une *colonne*.

Notes

Le seul argument de cette fonction est une colonne. Vous pouvez utiliser des colonnes contenant n'importe quel type de données. Quand la fonction ne trouve aucune ligne à compter, elle retourne une valeur BLANK ; sinon, elle retourne le nombre de valeurs distinctes.

Commentaires

Cette page a-t-elle été utile ? Yes No

AVERAGE

Article • 20/10/2023

Retourne la moyenne (arithmétique) de tous les nombres d'une colonne.

Syntaxe

```
DAX

AVERAGE(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne contenant les nombres dont vous voulez obtenir la moyenne.

Valeur renvoyée

Retourne un nombre décimal qui représente la moyenne arithmétique des nombres de la colonne.

Notes

- Cette fonction prend la colonne spécifiée en tant qu'argument et trouve la moyenne des valeurs de cette colonne. Si vous souhaitez trouver la moyenne d'une expression qui prend la valeur d'un ensemble de nombres, utilisez plutôt la fonction AVERAGEX.
- Les valeurs non numériques de la colonne sont gérées comme suit :
  - Si la colonne contient du texte, aucune agrégation ne peut être effectuée et les fonctions retournent des blancs.
  - Si la colonne contient des valeurs logiques ou des cellules vides, ces valeurs sont ignorées.
  - Les cellules dont la valeur est égale à zéro sont incluses.

- Quand vous calculez la moyenne des cellules, vous devez garder à l'esprit la différence entre une cellule vide et une cellule qui contient la valeur 0 (zéro). Quand une cellule contient 0, elle est ajoutée à la somme des nombres et la ligne est comptée dans le nombre de lignes utilisées comme diviseur. Toutefois, lorsqu'une cellule contient un blanc, la ligne n'est pas comptée.

- Quand il n'y a pas de lignes à agréger, la fonction retourne une valeur vide. En revanche, s'il y a des lignes, mais qu'aucune d'entre elles ne répond aux critères spécifiés, la fonction retourne 0. Excel retourne aussi un zéro si aucune ligne répondant aux conditions n'est trouvée.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La formule suivante retourne la moyenne des valeurs de la colonne ExtendedSalesAmount dans la table InternetSales.

DAX

```
= AVERAGE(InternetSales[ExtendedSalesAmount])
```

Fonctions connexes

La fonction AVERAGEX peut prendre comme argument une expression qui est évaluée pour chaque ligne d'une table. Cela vous permet d'effectuer des calculs, puis de prendre la moyenne des valeurs calculées.

La fonction AVERAGEA prend une colonne comme argument, mais elle est semblable à la fonction Excel du même nom. En utilisant la fonction AVERAGEA, vous pouvez calculer une moyenne sur une colonne qui contient des valeurs vides.

Commentaires

Cette page a-t-elle été utile ? Yes No

AVERAGEA

Article • 20/10/2023

Retourne la moyenne (arithmétique) des valeurs d'une colonne. Gère les valeurs texte et non numériques.

Syntaxe

DAX

```
AVERAGEA(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne qui contient les valeurs dont vous voulez obtenir la moyenne.

Valeur de retour

Nombre décimal.

Remarques

- La fonction AVERAGEA prend une colonne et calcule la moyenne des nombres qu'elle contient, mais gère également les types de données non numériques conformément aux règles suivantes :
  - Les valeurs TRUE comptent comme 1.
  - Les valeurs FALSE comptent comme 0 (zéro).
  - Les valeurs qui contiennent du texte non numérique comptent comme 0 (zéro).
  - Le texte vide ("") compte comme 0 (zéro).
- Si vous ne souhaitez pas inclure de valeurs logiques ni de représentations textuelles de nombres dans une référence dans le cadre du calcul, utilisez la fonction AVERAGE.

- Chaque fois qu'il n'existe aucune ligne à agréger, elle retourne un espace. En revanche, s'il y a des lignes, mais qu'aucune d'entre elles ne répond aux critères spécifiés, la fonction retourne 0. Microsoft Excel retourne aussi un zéro si aucune ligne répondant aux conditions n'est trouvée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant retourne la moyenne des cellules non vides dans la colonne référencée, selon le tableau suivant. Si vous avez utilisé la fonction AVERAGE, la moyenne est 21/2 ; avec la fonction AVERAGEA, le résultat est 22/5.

Agrandir le tableau

ID de transaction	Amount	Résultat
0000123	1	Compte comme 1
0000124	20	Compte comme 20
0000125	n/a	Compte comme 0
0000126		Compte comme 0
0000126	VRAI	Compte comme 1

```DAX
= AVERAGEA([Amount])
```

Contenu connexe

AVERAGE, fonction
AVERAGEX, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

AVERAGEX

Article • 20/10/2023

Calcule la moyenne (arithmétique) d'un ensemble d'expressions évaluées sur une table.

Syntaxe

DAX

```
AVERAGEX(<table>,<expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Nom d'une table ou expression qui spécifie la table sur laquelle l'agrégation peut être effectuée.
expression	Expression avec un résultat scalaire, qui sera évaluée pour chaque ligne de la table spécifiée dans le premier argument.

Valeur de retour

Nombre décimal.

Remarques

- La fonction AVERAGEX vous permet d'évaluer des expressions pour chaque ligne d'une table, puis de calculer la moyenne arithmétique du jeu de valeurs obtenu. La fonction utilise donc une table comme premier argument et une expression comme deuxième argument.
- Pour le reste, la fonction AVERAGEX suit les mêmes règles que la fonction AVERAGE. Vous ne pouvez pas inclure des cellules non numériques ou Null. Les arguments de table et d'expression sont requis.
- Quand il n'existe aucune ligne à agréger, la fonction retourne une valeur vide. Si des lignes sont présentes mais qu'aucune d'entre elles ne répond aux critères

spécifiés, la fonction retourne 0.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant calcule les frais de transport et de taxe moyens pour chaque commande de la table InternetSales. Pour cela, la somme des valeurs Freight et TaxAmt est calculée pour chaque ligne, puis la moyenne des sommes est établie.

DAX

```
= AVERAGEX(InternetSales, InternetSales[Freight]+ InternetSales[TaxAmt])
```

Si vous utilisez plusieurs opérations dans l'expression spécifiée comme deuxième argument, vous devez utiliser des parenthèses pour définir l'ordre des calculs. Pour plus d'informations, consultez le document de référence sur la syntaxe DAX.

Contenu connexe

AVERAGE, fonction
AVERAGEA, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

COUNT

Article • 14/12/2023

Compte le nombre de lignes dans la colonne spécifiée qui contiennent des valeurs non vides.

Syntaxe

DAX

```
COUNT(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne contenant les valeurs à compter.

Valeur de retour

Nombre entier.

Remarques

- Le seul argument autorisé pour cette fonction est une colonne. La fonction COUNT compte les lignes qui contiennent les types de valeurs suivants :
  - Nombres
  - Dates
  - Chaînes
- Quand la fonction ne trouve aucune ligne à compter, elle retourne une valeur vide.
- Les valeurs vides sont ignorées. Les valeurs TRUE/FALSE ne sont pas prises en charge.
- Si vous souhaitez évaluer une colonne de valeurs TRUE/FALSE, utilisez la fonction COUNTA.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).
- Pour connaître les meilleures pratiques lors de l'utilisation de COUNT, consultez Utiliser COUNTROWS au lieu de Count.

Exemple

L'exemple suivant montre comment compter le nombre de valeurs dans la colonne ShipDate.

```
DAX
= COUNT([ShipDate])
```

Pour compter les valeurs logiques ou du texte, utilisez les fonctions COUNTA ou COUNTAX.

Contenu connexe

COUNTA, fonction
COUNTAX, fonction
COUNTX, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

COUNTA

Article • 14/12/2023

Compte le nombre de lignes dans la colonne spécifiée qui contiennent des valeurs non vides.

Syntaxe

DAX

```
COUNTA(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne contenant les valeurs à compter.

Valeur de retour

Nombre entier.

Remarques

- Quand la fonction ne trouve aucune ligne à compter, la fonction retourne une valeur vide.
- Contrairement à COUNT, COUNTA prend en charge le type de données booléen.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant retourne toutes les lignes de la table `Reseller` pour lesquelles la colonne des numéros de téléphone contient un type de données quelconque.

DAX

```
= COUNTA(Reseller[Phone])
```

Contenu connexe

COUNT, fonction
COUNTAX, fonction
COUNTX, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

COUNTAX

Article • 20/10/2023

La fonction COUNTAX compte les résultats non vides lors de l'évaluation du résultat d'une expression sur une table. Autrement dit, elle fonctionne comme la fonction COUNTA, mais est utilisée pour parcourir les lignes d'une table et compter les lignes où les expressions spécifiées génèrent un résultat non vide.

Syntaxe

DAX

```
COUNTAX(<table>,<expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table contenant les lignes pour lesquelles l'expression est évaluée.
expression	Expression à évaluer pour chaque ligne de la table.

Valeur de retour

Nombre entier.

Remarques

- À l'instar de la fonction COUNTA, la fonction COUNTAX compte les cellules contenant n'importe quel type d'informations, y compris d'autres expressions. Par exemple, si la colonne contient une expression qui prend la valeur d'une chaîne vide, la fonction COUNTAX traite ce résultat comme non vide. En règle générale, la fonction COUNTAX ne compte pas les cellules vides mais, dans ce cas, la cellule contient une formule et elle est donc comptée.
- Chaque fois que la fonction ne trouve aucune ligne à agréger, elle retourne un espace.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant compte le nombre de lignes non vides dans la colonne Phone à l'aide de la table qui résulte du filtrage de la table Reseller sur [Status] = **Active**.

DAX

```
= COUNTAX(FILTER('Reseller',[Status]="Active"),[Phone])
```

Contenu connexe

COUNT, fonction
COUNTA, fonction
COUNTX, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

COUNTBLANK

Article • 14/12/2023

Compte le nombre de cellules vides dans une colonne.

Syntaxe

DAX

```
COUNTBLANK(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne qui contient les cellules vides à compter.

Valeur de retour

Nombre entier. Si aucune ligne répondant à la condition n'est trouvée, des valeurs vides sont retournées.

Remarques

- Le seul argument autorisé pour cette fonction est une colonne. Vous pouvez utiliser des colonnes contenant n'importe quel type de données, mais seules les cellules vides sont comptées. Les cellules qui ont la valeur zéro (0) ne sont pas comptées, car zéro est considéré comme une valeur numérique et non comme une valeur vide.
- Quand il n'y a pas de lignes à agréger, la fonction retourne une valeur vide. En revanche, s'il y a des lignes, mais qu'aucune d'entre elles ne répond aux critères spécifiés, la fonction retourne 0. Microsoft Excel retourne aussi zéro si aucune ligne remplissant les conditions n'est trouvée.
- Autrement dit, si la fonction COUNTBLANK ne trouve pas de valeurs vides, le résultat est zéro, mais s'il n'y a aucune ligne à vérifier, le résultat est une valeur

vide.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant montre comment compter le nombre de lignes dans la table Reseller où BankName présente des valeurs vides.

```
DAX
= COUNTBLANK(Reseller[BankName])
```

Pour compter les valeurs logiques ou du texte, utilisez les fonctions COUNTA ou COUNTAX.

Contenu connexe

COUNT, fonction
COUNTA, fonction
COUNTAX, fonction
COUNTX, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

COUNTROWS

Article • 20/10/2023

La fonction COUNTROWS compte le nombre de lignes dans la table spécifiée ou dans une table définie par une expression.

Syntaxe

DAX

```
COUNTROWS([<table>])
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	(Facultatif) Nom de la table qui contient les lignes à compter, ou expression qui retourne une table. Lorsqu'elle n'est pas fournie, la valeur par défaut est la table principale de l'expression actuelle.

Valeur de retour

Nombre entier.

Remarques

- Cette fonction peut être utilisée pour compter le nombre de lignes dans une table de base, mais le plus souvent, elle est utilisée pour compter le nombre de lignes obtenues après l'application d'un filtre ou d'un contexte à une table.
- Quand il n'y a pas de lignes à agréger, la fonction retourne une valeur vide. En revanche, s'il y a des lignes, mais qu'aucune d'entre elles ne répond aux critères spécifiés, la fonction retourne 0. Microsoft Excel retourne aussi un zéro si aucune ligne répondant aux conditions n'est trouvée.
- Pour en savoir plus sur les meilleures pratiques lors de l'utilisation de COUNT et COUNTROWS, consultez Utiliser COUNTROWS au lieu de Count dans DAX.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple 1

L'exemple suivant montre comment compter le nombre de lignes dans la table Orders. Le résultat attendu est 52761.

DAX

```
= COUNTROWS('Orders')
```

Exemple 2

L'exemple suivant montre comment utiliser COUNTROWS avec un contexte de ligne. Dans ce scénario, il y a deux ensembles de données liés entre eux par le numéro de commande. La table Reseller présente une ligne par revendeur ; la table ResellerSales comporte plusieurs lignes pour chacune des commandes, chaque ligne correspondant à une commande d'un revendeur particulier. Les tables sont connectées par une relation établie sur la colonne ResellerKey.

La formule obtient la valeur de ResellerKey, puis compte le nombre de lignes dans la table associée qui contiennent le même ID de revendeur. Le résultat est affiché dans la colonne **CalculatedColumn1**.

DAX

```
= COUNTROWS(RELATEDTABLE(ResellerSales))
```

Le tableau suivant présente une partie des résultats attendus :

Agrandir le tableau

ResellerKey	CalculatedColumn1
1	73
2	70
3	394

Contenu connexe

COUNT, fonction
COUNTA, fonction
COUNTAX, fonction
COUNTX, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

COUNTX

Article • 14/12/2023

Compte le nombre de lignes contenant une valeur non vide ou une expression qui prend une valeur non vide quand une expression est évaluée sur une table.

Syntaxe

```
DAX

COUNTX(<table>,<expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table contenant les lignes à compter.
expression	Expression qui retourne le jeu de valeurs qui contient les valeurs que vous souhaitez compter.

Valeur de retour

Entier.

Remarques

- La fonction COUNTX utilise deux arguments. Le premier argument doit toujours être une table ou toute expression qui retourne une table. Le deuxième argument est la colonne ou l'expression recherchée par la fonction COUNTX.
- La fonction COUNTX compte uniquement les valeurs, les dates et les chaînes. Si la fonction ne trouve aucune ligne à compter, elle retourne une valeur vide.
- Si vous souhaitez compter des valeurs logiques, utilisez la fonction COUNTAX.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au

niveau des lignes (RLS).

Exemple 1

La formule suivante retourne le nombre total de lignes de la table Product contenant un prix catalogue.

DAX

```
= COUNTX(Product,[ListPrice])
```

Exemple 2

La formule suivante montre comment transmettre une table filtrée à la fonction COUNTX pour le premier argument. La formule utilise une expression de filtre pour obtenir uniquement les lignes de la table Product qui remplissent la condition ProductSubCategory = "Caps", puis elle compte les lignes de la table obtenue qui contiennent un prix catalogue. L'expression FILTER s'applique à la table Products mais utilise une valeur que vous recherchez dans la table associée, ProductSubCategory.

DAX

```
=
COUNTX(FILTER(Product,RELATED(ProductSubcategory[EnglishProductSubcategoryNa
me])="Caps"), Product[ListPrice])
```

Contenu connexe

COUNT, fonction
COUNTA, fonction
COUNTAX, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

DISTINCTCOUNT

Article • 14/12/2023

Compte le nombre de valeurs distinctes dans une colonne.

Syntaxe

DAX

```
DISTINCTCOUNT(<column>)
```

Paramètres

Agrandir le tableau

Terme	Description
colonne	Colonne qui contient les valeurs à compter.

Valeur de retour

Nombre de valeurs distinctes contenues dans *column*.

Remarques

- Le seul argument autorisé pour cette fonction est une colonne. Vous pouvez utiliser des colonnes contenant n'importe quel type de données. Quand la fonction ne trouve aucune ligne à compter, elle retourne une valeur BLANK ; sinon, elle retourne le nombre de valeurs distinctes.
- La fonction DISTINCTCOUNT compte la valeur BLANK. Pour ignorer la valeur BLANK, utilisez la fonction DISTINCTCOUNTNOBLANK.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant montre comment compter le nombre de commandes client distinctes dans la colonne ResellerSales_USD[SalesOrderNumber].

```
DAX
= DISTINCTCOUNT(ResellerSales_USD[SalesOrderNumber])
```

En utilisant la mesure ci-dessus dans une table avec l'année civile sur le côté et la catégorie de produit en haut, les résultats suivants sont retournés :

Agrandir le tableau

Étiquettes de ligne	Accessories	Bikes	Clothing	Components	-	Total général
2005	135	345	242	205		366
2006	356	850	644	702		1015
2007	531	1234	963	1138		1521
2008	293	724	561	601		894
					1	1
Total général	**1315**	**3153**	**2410**	**2646**	**1**	**3797**

Dans l'exemple ci-dessus, notez que les valeurs des lignes Total général ne s'additionnent pas ; cela est dû au fait que la même commande peut contenir des articles, dans le même ordre, de catégories de produits distinctes.

Contenu connexe

COUNT, fonction
COUNTA, fonction
COUNTAX, fonction
COUNTX, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

DISTINCTCOUNTNOBLANK

Article • 14/12/2023

Compte le nombre de valeurs distinctes dans une colonne.

Syntaxe

DAX

```
DISTINCTCOUNTNOBLANK (<column>)
```

Paramètres

Agrandir le tableau

Terme	Description
colonne	Colonne qui contient les valeurs à compter.

Valeur de retour

Nombre de valeurs distinctes contenues dans *column*.

Remarques

- Contrairement à la fonction DISTINCTCOUNT, DISTINCTCOUNTNOBLANK ne compte pas la valeur BLANK.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant montre comment compter le nombre de commandes client distinctes dans la colonne ResellerSales_USD[SalesOrderNumber].

DAX

```
= DISTINCTCOUNT(ResellerSales_USD[SalesOrderNumber])
```

Requête DAX

DAX

```
EVALUATE
    ROW(
        "DistinctCountNoBlank", DISTINCTCOUNTNOBLANK(DimProduct[EndDate]),
        "DistinctCount", DISTINCTCOUNT(DimProduct[EndDate])
    )
```

Agrandir le tableau

[DistinctCountNoBlank]	[DistinctCount]
2	3

Contenu connexe

DISTINCTCOUNT

Commentaires

Cette page a-t-elle été utile ? Yes No

MAX

Article • 20/10/2023

Retourne la plus grande valeur d'une colonne ou entre deux expressions scalaires.

Syntaxe

DAX

```
MAX(<column>)
```

DAX

```
MAX(<expression1>, <expression2>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne dans laquelle vous voulez rechercher la plus grande valeur.
expression	Toute expression DAX qui retourne une seule valeur.

Valeur renvoyée

Valeur la plus grande.

Remarques

- Quand deux expressions sont comparées, un blanc (blank) est traité comme 0 lors de la comparaison. Autrement dit, Max(1,Blank()) retourne 1, tandis que Max(-1, Blank()) retourne 0. Si les deux arguments sont vides, MAX retourne un blanc. Si l'une des expressions retourne une valeur qui n'est pas autorisée, MAX retourne une erreur.
- Les valeurs TRUE/FALSE ne sont pas prises en charge. Si vous souhaitez évaluer une colonne de valeurs TRUE/FALSE, utilisez la fonction MAXA.

Exemple 1

L'exemple suivant retourne la plus grande valeur trouvée dans la colonne ExtendedAmount de la table InternetSales.

```
DAX

= MAX(InternetSales[ExtendedAmount])
```

Exemple 2

L'exemple suivant retourne la plus grande valeur du résultat de deux expressions.

```
DAX

= Max([TotalSales], [TotalPurchases])
```

Contenu connexe

MAXA, fonction
MAXX, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

MAXA

Article • 14/12/2023

Retourne la plus grande valeur d'une colonne.

Syntaxe

DAX

```
MAXA(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne dans laquelle vous voulez rechercher la plus grande valeur.

Valeur de retour

Valeur la plus grande.

Remarques

- La fonction MAXA prend comme argument une colonne et recherche la plus grande valeur parmi les types de valeurs suivants :
  - Nombres
  - Dates
- Valeurs logiques, telles que TRUE et FALSE. Les lignes qui équivalent à TRUE comptent pour 1. Les lignes qui équivalent à FALSE comptent pour 0 (zéro).
- Les cellules vides sont ignorées. Si la colonne ne contient aucune valeur pouvant être utilisée, MAXA retourne 0 (zéro).
- Si vous souhaitez comparer des valeurs de texte, utilisez la fonction MAX.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au

niveau des lignes (RLS).

Exemple 1

L'exemple suivant retourne la plus grande valeur d'une colonne calculée, nommée **ResellerMargin**, qui calcule la différence entre le prix catalogue et le prix revendeur.

```
DAX
= MAXA([ResellerMargin])
```

Exemple 2

L'exemple suivant retourne la plus grande valeur d'une colonne qui contient des dates et des heures. Par conséquent, cette formule obtient la date de transaction la plus récente.

```
DAX
= MAXA([TransactionDate])
```

Contenu connexe

MAX, fonction
MAXX, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

MAXX

Article • 14/12/2023

Renvoie la valeur la plus élevée résultant de l'évaluation d'une expression pour chaque ligne d'une table.

Syntaxe

```DAX
MAXX(<table>,<expression>,[<variant>])
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table contenant les lignes pour lesquelles l'expression est évaluée.
expression	Expression à évaluer pour chaque ligne de la table.
variant	(Facultatif) Si VRAI, et s'il y a des variantes ou des types de valeurs mixtes, la valeur la plus élevée basée sur TRIER PAR DESC est renvoyée.

Valeur retournée

Valeur la plus élevée.

Remarques

- L'argument **table** de la fonction MAXX peut être un nom de table ou une expression qui correspond à une table. Le deuxième argument indique l'expression à évaluer pour chaque ligne de la table.
- Parmi les valeurs à évaluer, seules les suivantes sont comptées :
  - Nombres
  - Textes
  - Dates

- Les valeurs vides sont ignorées. Les valeurs TRUE/FALSE ne sont pas prises en charge.
- Si l'expression comporte des variantes ou des types de valeurs mixtes tels que du texte et des nombres, MAXX ne prend en compte par défaut que les nombres. Si `<variant> = TRUE`, la valeur maximale est retournée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple 1

La formule suivante utilise une expression comme deuxième argument pour calculer le montant total des taxes et des frais d'expédition pour chaque commande de la table InternetSales. Le résultat attendu est 375,7184.

DAX

```
= MAXX(InternetSales, InternetSales[TaxAmt]+ InternetSales[Freight])
```

Exemple 2

La formule suivante filtre d'abord la table InternetSales à l'aide d'une expression FILTER afin de retourner un sous-ensemble de commandes pour une région de vente spécifique, définie comme suit : [SalesTerritory] = 5. La fonction MAXX évalue ensuite l'expression utilisée comme deuxième argument pour chaque ligne de la table filtrée et retourne le montant des taxes et des frais d'expédition le plus élevé pour ces commandes. Le résultat attendu est 250,3724.

DAX

```
= MAXX(FILTER(InternetSales,[SalesTerritoryCode]="5"),
InternetSales[TaxAmt]+ InternetSales[Freight])
```

Contenu connexe

MAX, fonction
MAXA, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

MIN

Article • 20/10/2023

Retourne la plus petite valeur d'une colonne ou entre deux expressions scalaires.

Syntaxe

DAX

```
MIN(<column>)
```

DAX

```
MIN(<expression1>, <expression2>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne dans laquelle vous voulez trouver la plus petite valeur numérique.
expression	Toute expression DAX qui retourne une seule valeur.

Valeur renvoyée

La plus petite valeur.

Remarques

- La fonction MIN prend une colonne ou deux expressions comme argument et retourne la plus petite valeur. Les types de valeurs suivants dans les colonnes sont comptabilisés :
  - Nombres
  - Textes
  - Dates
  - Espaces

- Lors de la comparaison d'expressions, un blanc (blank) est traité comme 0 lors de la comparaison. Autrement dit, Min(1,Blank()) retourne 0, tandis que Min(-1, Blank()) retourne -1. Si les deux arguments sont vides, MIN retourne un blanc. Si l'une des expressions retourne une valeur qui n'est pas autorisée, MIN retourne une erreur.

- Les valeurs TRUE/FALSE ne sont pas prises en charge. Si vous souhaitez évaluer une colonne de valeurs TRUE/FALSE, utilisez la fonction MINA.

Exemple 1

L'exemple suivant retourne la plus petite valeur de la colonne calculée, ResellerMargin.

DAX

```
= MIN([ResellerMargin])
```

Exemple 2

L'exemple suivant retourne la plus petite valeur d'une colonne qui contient des dates et des heures, TransactionDate. Par conséquent, cette formule retourne la date la plus ancienne.

DAX

```
= MIN([TransactionDate])
```

Exemple 3

L'exemple suivant retourne la plus petite valeur du résultat de deux expressions scalaires.

DAX

```
= Min([TotalSales], [TotalPurchases])
```

Contenu connexe

MINA, fonction
MINX, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

MINA

Article • 20/10/2023

Retourne la plus petite valeur d'une colonne.

Syntaxe

DAX

```
MINA(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne pour laquelle vous souhaitez rechercher la valeur minimale.

Valeur de retour

La plus petite valeur.

Remarques

- La fonction MINA prend comme argument une colonne qui contient des nombres et détermine la plus petite valeur comme suit :
  - Si la colonne ne contient aucune valeur, MINA retourne 0 (zéro).
  - Les lignes de la colonne qui donne des valeurs logiques, comme TRUE et FALSE, sont traitées comme égales à 1 si TRUE et à 0 (zéro) si FALSE.
  - Les cellules vides sont ignorées.
- Si vous souhaitez comparer des valeurs de texte, utilisez la fonction MIN.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple 1

L'expression suivante retourne les frais de transport minimaux de la table InternetSales.

```DAX
= MINA(InternetSales[Freight])
```

Exemple 2

L'expression suivante retourne la valeur minimale dans la colonne PostalCode. Étant donné que le type de données de la colonne est texte, la fonction ne trouve aucune valeur et la formule retourne zéro (0).

```DAX
= MINA([PostalCode])
```

Contenu connexe

MIN, fonction
MINX, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

MINX

Article • 20/10/2023

Renvoie la valeur la plus basse résultant de l'évaluation d'une expression pour chaque ligne d'un tableau.

Syntaxe

DAX

```
MINX(<table>, < expression>,[<variant>])
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table contenant les lignes pour lesquelles l'expression est évaluée.
expression	Expression à évaluer pour chaque ligne de la table.
variant	(Facultatif) Si VRAI, et s'il y a des variantes ou des types de valeurs mixtes, la valeur la plus basse basée sur TRIER PAR ASC est renvoyée.

Valeur retournée

Valeur la plus basse.

Remarques

- La fonction MINX prend comme premier argument une table ou une expression qui retourne une table. Le deuxième argument contient l'expression à évaluer pour chaque ligne de la table.
- Les valeurs vides sont ignorées. Les valeurs TRUE/FALSE ne sont pas prises en charge.
- Si l'expression comporte des variantes ou des types de valeurs mixtes tels que du texte et des nombres, MINX ne prend en compte par défaut que les nombres. Si

`<variant> = TRUE`, la valeur minimale est retournée.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple 1

L'exemple suivant filtre la table InternetSales et ne renvoie que les lignes correspondant à un territoire de vente spécifique. La formule trouve ensuite la valeur minimale dans la colonne Fret.

DAX

```
= MINX( FILTER(InternetSales, [SalesTerritoryKey] = 5),[Freight])
```

Exemple 2

L'exemple suivant utilise la même table filtrée que dans l'exemple précédent, mais au lieu de rechercher simplement des valeurs dans la colonne pour chaque ligne de la table filtrée, la fonction calcule la somme de deux colonnes, Freight et TaxAmt, et retourne la valeur la plus petite résultant de ce calcul.

DAX

```
= MINX( FILTER(InternetSales, InternetSales[SalesTerritoryKey] = 5),
InternetSales[Freight] + InternetSales[TaxAmt])
```

Dans le premier exemple, les noms des colonnes ne sont pas qualifiés. Dans le second exemple, les noms des colonnes sont complets.

Contenu connexe

MIN, fonction
MINA, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

PRODUIT

Article • 14/12/2023

Retourne le produit des nombres d'une colonne.

Syntaxe

DAX

```
PRODUCT(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
Colonne	Colonne qui contient les nombres dont le produit doit être calculé.

Valeur renvoyée

Nombre décimal.

Notes

- Pour retourner le produit d'une expression évaluée pour chaque ligne d'une table, utilisez la fonction PRODUCTX.
- Seuls les nombres de la colonne sont comptés. Les valeurs vides, les valeurs logiques et le texte sont ignorés. Par exemple,

  `PRODUCT( Table[Column] )` équivaut à `PRODUCTX( Table, Table[Column] )`.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Le code suivant calcule le produit de la colonne AdjustedRates dans une table Annuity :

```
DAX
= PRODUCT( Annuity[AdjustedRates] )
```

Contenu connexe

PRODUCTX

Commentaires

Cette page a-t-elle été utile ? Yes No

PRODUCTX

Article • 20/10/2023

Retourne le produit d'une expression évaluée pour chaque ligne d'une table.

Syntaxe

DAX

```
PRODUCTX(<table>, <expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table contenant les lignes pour lesquelles l'expression est évaluée.
expression	Expression à évaluer pour chaque ligne de la table.

Valeur renvoyée

Nombre décimal.

Notes

- Pour retourner le produit des nombres d'une colonne, utilisez PRODUCT.
- La fonction PRODUCTX prend une table comme premier argument ou une expression qui retourne une table. Le deuxième argument est une colonne qui contient les nombres dont vous voulez calculer le produit ou une expression qui est évaluée à une colonne.
- Seuls les nombres de la colonne sont comptés. Les valeurs vides, les valeurs logiques et le texte sont ignorés.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant calcule la valeur capitalisée d'un investissement :

```DAX
= [PresentValue] * PRODUCTX( AnnuityPeriods, 1+[FixedInterestRate] )
```

Contenu connexe

PRODUCT

Commentaires

Cette page a-t-elle été utile ? Yes No

SUM

Article • 14/12/2023

Additionne tous les nombres d'une colonne.

Syntaxe

```DAX
SUM(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne contenant les nombres à additionner.

Valeur renvoyée

Nombre décimal.

Notes

Si vous souhaitez filtrer les valeurs que vous additionnez, vous pouvez utiliser la fonction SUMX et spécifier une expression à additionner.

Exemple

L'exemple suivant additionne tous les nombres contenus dans la colonne Amt de la table Sales.

```DAX
= SUM(Sales[Amt])
```

Contenu connexe

SUMX

Commentaires

Cette page a-t-elle été utile ? Yes No

SUMX

Article • 20/10/2023

Retourne la somme d'une expression évaluée pour chaque ligne d'une table.

Syntaxe

DAX

```
SUMX(<table>, <expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table contenant les lignes pour lesquelles l'expression est évaluée.
expression	Expression à évaluer pour chaque ligne de la table.

Valeur renvoyée

Nombre décimal.

Remarques

- La fonction SUMX prend comme premier argument une table ou une expression qui retourne une table. Le deuxième argument est une colonne qui contient les nombres dont vous voulez calculer la somme ou une expression correspondant à une colonne.
- Seuls les nombres de la colonne sont comptés. Les valeurs vides, les valeurs logiques et le texte sont ignorés.
- Pour obtenir des exemples plus complexes de SUMX dans les formules, consultez ALL et CALCULATETABLE.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au

niveau des lignes (RLS).

Exemple

L'exemple suivant filtre d'abord la table InternetSales sur l'expression 'InternetSales[SalesTerritoryID] = 5`, puis renvoie la somme de toutes les valeurs de la colonne Freight. En d'autres termes, l'expression retourne la somme des frais de transport uniquement pour le secteur de vente spécifié.

DAX

```
= SUMX(FILTER(InternetSales, InternetSales[SalesTerritoryID]=5),[Freight])
```

Si vous n'avez pas besoin de filtrer la colonne, utilisez la fonction SUM. Cette fonction est similaire à la fonction SUM dans Excel. Cependant, elle prend une colonne comme référence.

Contenu connexe

SUM
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonctions de date et heure

Article • 20/10/2023

Ces fonctions vous aident à créer des calculs basés sur les dates et l'heure. La plupart des fonctions de DAX sont similaires aux fonctions de date et d'heure d'Excel. Toutefois, les fonctions DAX utilisent un type de données **datetime** et peuvent prendre des valeurs d'une colonne en tant qu'argument.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
CALENDAR	Retourne une table avec une seule colonne nommée « Date » qui contient un ensemble de dates contigu.
CALENDARAUTO	Retourne une table avec une seule colonne nommée « Date » qui contient un ensemble de dates contigu.
DATE	Retourne la date spécifiée au format datetime.
DATEDIFF	Renvoie le nombre de limites d'intervalle entre deux dates.
DATEVALUE	Convertit une date au format texte en une date au format datetime.
DAY	Retourne le jour du mois, un nombre compris entre 1 et 31.
EDATE	Retourne la date qui correspond au nombre de mois indiqué avant ou après la date de début.
EOMONTH	Retourne la date au format datetime du dernier jour du mois avant ou après un nombre de mois spécifié.
HOUR	Retourne l'heure sous la forme d'un nombre compris entre 0 (00:00) et 23 (23:00).
MINUTE	Retourne la minute sous la forme d'un nombre compris entre 0 et 59, à partir d'une valeur de date et d'heure.
MONTH	Retourne le mois sous la forme d'un nombre compris entre 1 (janvier) et 12 (décembre).
NETWORKDAYS	Renvoie le nombre de jours ouvrés entiers entre deux dates.
NOW	Retourne la date et l'heure actuelles au format DateHeure.

Fonction	Description
QUARTER	Retourne le trimestre sous forme de nombre compris entre 1 et 4.
SECOND	Retourne les secondes d'une valeur d'heure, sous la forme d'un nombre compris entre 0 et 59.
TIME	Convertit les heures, minutes et secondes données sous forme de nombres en heure au format datetime.
TIMEVALUE	Convertit une heure au format texte en une heure au format datetime.
TODAY	Retourne la date du jour.
UTCNOW	Retourne la date et l'heure UTC actuelles.
UTCTODAY	Retourne la date UTC actuelle.
WEEKDAY	Retourne un nombre compris entre 1 et 7 identifiant le jour de la semaine d'une date.
WEEKNUM	Retourne le numéro de semaine pour la date et l'année données en fonction de la valeur return_type.
YEAR	Retourne l'année d'une date sous forme d'un entier à quatre chiffres compris dans la plage 1900-9999.
YEARFRAC	Calcule la fraction de l'année représentée par le nombre de jours entiers entre deux dates.

Commentaires

Cette page a-t-elle été utile ? Yes No

CALENDAR

Article • 20/10/2023

Retourne une table avec une seule colonne nommée « Date » qui contient un ensemble de dates contigu. La plage de dates est la période allant de la date de début spécifiée à la date de fin spécifiée, incluses.

Syntaxe

DAX

```
CALENDAR(<start_date>, <end_date>)
```

Paramètres

Agrandir le tableau

Terme	Définition
start_date	Toute expression DAX qui retourne une valeur DateHeure.
end_date	Toute expression DAX qui retourne une valeur DateHeure.

Valeur de retour

Retourne une table avec une seule colonne nommée « Date » qui contient un ensemble de dates contigu. La plage de dates est la période allant de la date de début spécifiée à la date de fin spécifiée, incluses.

Remarques

- Une erreur est retournée si la date start_date est postérieure à la date end_date.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

La formule suivante renvoie un tableau avec des dates comprises entre le 1er janvier 2015 et le 31 décembre 2021.

```
DAX
= CALENDAR (DATE (2015, 1, 1), DATE (2021, 12, 31))
```

Pour un modèle de données qui inclut des données de ventes réelles et des prévisions de ventes futures, l'expression suivante renvoie une table de dates couvrant la plage de dates dans les tables Ventes et Prévision.

```
DAX
= CALENDAR (MINX (Sales, [Date]), MAXX (Forecast, [Date]))
```

Commentaires

Cette page a-t-elle été utile ? Yes No

CALENDARAUTO

Article • 20/10/2023

Retourne une table avec une seule colonne nommée « Date » qui contient un ensemble de dates contigu. La plage de dates est calculée automatiquement en fonction des données du modèle.

Syntaxe

```DAX
CALENDARAUTO([fiscal_year_end_month])
```

Paramètres

Agrandir le tableau

Terme	Définition
fiscal_year_end_month	Toute expression DAX qui retourne un entier compris entre 1 et 12. En cas d'omission, la valeur par défaut est celle spécifiée dans le modèle de table de calendrier de l'utilisateur actuel (le cas échéant) ; sinon, la valeur par défaut est 12.

Valeur de retour

Retourne une table avec une seule colonne nommée « Date » qui contient un ensemble de dates contigu. La plage de dates est calculée automatiquement en fonction des données du modèle.

Remarques

- La plage de dates est calculée comme suit :
  - La première date du modèle qui n'est pas dans une colonne ou une table calculée est considérée comme MinDate.
  - La dernière date du modèle qui n'est pas dans une colonne ou une table calculée est considérée comme MaxDate.
  - La plage de dates retournée est comprise entre le début de l'exercice fiscal associé à MinDate et la fin de l'exercice fiscal associé à MaxDate.

- Une erreur est retournée si le modèle ne contient pas de valeurs DateHeure qui ne sont pas dans des colonnes calculées ou des tables calculées.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Dans cet exemple, les informations MinDate et MaxDate dans le modèle de données sont le 1er juillet 2010 et le 30 juin 2011.

`CALENDARAUTO()` retourne toutes les dates comprises entre le 1er janvier 2010 et le 31 décembre 2011.

`CALENDARAUTO(3)` retourne toutes les dates comprises entre le 1er avril 2010 et le 31 mars 2012.

Commentaires

Cette page a-t-elle été utile ? Yes No

DATE

Article • 20/10/2023

Retourne la date spécifiée au format **datetime**.

Syntaxe

DAX

```
DATE(<year>, <month>, <day>)
```

Paramètres

Agrandir le tableau

Terme	Définition
year	Nombre représentant l'année. La valeur de l'argument **year** peut inclure d'un à quatre chiffres. L'argument **year** est interprété en fonction du système de date utilisé par votre ordinateur. Les dates à partir du 1er mars 1900 sont prises en charge. Si vous entrez un nombre comportant des décimales, il est arrondi. Pour les valeurs supérieures à 9 999 ou inférieures à zéro (valeurs négatives), la fonction retourne une erreur **#VALUE!**. Si la valeur de **year** est comprise entre 0 et 1899, elle est ajoutée à 1900 pour générer la valeur finale. Considérons les exemples ci-dessous. **Remarque :** Vous devez utiliser quatre chiffres pour l'argument **year** chaque fois que possible pour éviter des résultats indésirables. Par exemple, l'utilisation de 07 retourne 1907 comme valeur de year.
month	Nombre représentant le mois ou un calcul conforme aux règles suivantes : Les entiers négatifs ne sont pas pris en charge. Les valeurs valides sont comprises entre 1 et 12. Si **month** est un nombre compris entre 1 et 12, il représente un mois de l'année. 1 représente le mois de janvier, 2 le mois de février, et ainsi de suite jusqu'à 12 qui représente le mois de décembre. Si vous entrez un entier supérieur à 12, le calcul suivant est effectué : la date est calculée

Terme	Définition
	en ajoutant la valeur de **month** à **year**. Par exemple, la formule DATE(2008, 18, 1) retourne une valeur datetime équivalente au 1er juin 2009, car 18 mois sont ajoutés au début de 2008, ce qui équivaut à juin 2009. Voir les exemples ci-dessous.
day	Nombre représentant le jour ou un calcul conforme aux règles suivantes : Les entiers négatifs ne sont pas pris en charge. Les valeurs valides sont comprises entre 1 et 31. Si **day** est un nombre compris entre 1 et le dernier jour du mois donné, il représente un jour du mois. Si vous entrez un entier supérieur au dernier jour du mois donné, le calcul suivant est effectué : la date est calculée en ajoutant la valeur de **day** à **month**. Par exemple, dans la formule `DATE( 2008, 3, 32)`, la fonction DATE retourne une valeur **datetime** équivalente au 1er avril 2008, car 32 jours sont ajoutés au début du mois de mars, ce qui équivaut au 1er avril. Si **day** contient une partie décimale, la valeur est arrondie à l'entier le plus proche.

Valeur renvoyée

Retourne la date spécifiée (**datetime)**.

Notes

- La fonction DATE prend les entiers entrés comme arguments et génère la date correspondante. La fonction DATE est particulièrement utile dans les situations où l'année, le mois et le jour sont fournis par des formules. Par exemple, les données sous-jacentes peuvent contenir des dates dans un format qui n'est pas reconnu comme une date, tel que AAAAMMJJ. Vous pouvez utiliser la fonction DATE conjointement à d'autres fonctions pour convertir les dates en un nombre pouvant être reconnu comme une date.
- Contrairement à Microsoft Excel, qui stocke les dates en tant que nombres ordinaux, les fonctions de date DAX retournent toujours un type de données **datetime**. Toutefois, vous pouvez utiliser la mise en forme pour afficher les dates en tant que nombres ordinaux si vous le souhaitez.
- Les valeurs Date et DateHeure peuvent également être spécifiées en tant que littéral au format `dt"YYYY-MM-DD"`, `dt"YYYY-MM-DDThh:mm:ss"` ou `dt"YYYY-MM-DD hh:mm:ss"`. Quand elle est spécifiée en tant que littéral, il n'est pas nécessaire

d'utiliser la fonction DATE dans l'expression. Pour en savoir plus, consultez Syntaxe DAX | Date et heure.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

Date simple

La formule suivante retourne la date du 8 juillet 2009 :

```
DAX
= DATE(2009,7,8)
```

Années avant 1899

Si la valeur que vous entrez pour l'argument **year** est comprise entre 0 (zéro) et 1899 (inclus), cette valeur est ajoutée à 1900 pour calculer l'année. La formule suivante retourne le 2 janvier 1908 : (1900+08).

```
DAX
= DATE(08,1,2)
```

Années après 1899

Si la valeur **year** est comprise entre 1900 et 9999 (inclus), elle est utilisée comme année. La formule suivante retourne le 2 janvier 2008 :

```
DAX
= DATE(2008,1,2)
```

Mois

Si la valeur **month** est supérieure à 12, **month** ajoute ce nombre de mois au premier mois de l'année spécifiée. La formule suivante retourne la date du 2 février 2009 :

DAX

```
= DATE(2008,14,2)
```

Jours

Si la valeur **day** est supérieure au nombre de jours du mois spécifié, **day** ajoute ce nombre de jours au premier jour du mois. La formule suivante retourne la date du 4 février 2008 :

DAX

```
= DATE(2008,1,35)
```

Contenu connexe

Fonctions de date et heure
DAY, fonction
TODAY, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

DATEDIFF

Article • 20/10/2023

Renvoie le nombre de limites d'intervalle entre deux dates.

Syntaxe

DAX

```
DATEDIFF(<Date1>, <Date2>, <Interval>)
```

Paramètres

Agrandir le tableau

Terme	Définition
Date1	Valeur datetime scalaire.
Date2	Valeur datetime scalaire.
Intervalle	Intervalle à utiliser durant la comparaison de dates. Il peut s'agir de l'une des valeurs suivantes : - SECOND - MINUTE - HOUR - DAY - WEEK - MONTH - QUARTER - YEAR

Valeur renvoyée

Le nombre de limites d'intervalle entre deux dates.

Remarques

Un résultat positif est renvoyé si Date2 est supérieur à Date1. Un résultat négatif est renvoyé si Date1 est supérieur à Date2.

Exemple

Les exemples de cet article peuvent être utilisés avec l'exemple de modèle Adventure Works DW 2020 Power BI Desktop. Pour obtenir le modèle, consultez Exemple de modèle DAX.

La requête DAX suivante :

DAX

```
EVALUATE
VAR StartDate =  DATE ( 2019, 07, 01 )
VAR EndDate =    DATE ( 2021, 12, 31 )
RETURN
    {
        ( "Year",     DATEDIFF ( StartDate, EndDate, YEAR ) ),
        ( "Quarter",  DATEDIFF ( StartDate, EndDate, QUARTER ) ),
        ( "Month",    DATEDIFF ( StartDate, EndDate, MONTH ) ),
        ( "Week",     DATEDIFF ( StartDate, EndDate, WEEK ) ),
        ( "Day",      DATEDIFF ( StartDate, EndDate, DAY ) )
    }
```

retourne les résultats suivants :

Agrandir le tableau

Value1	Value2
Year	2
Quarter (Trimestre)	9
Month	29
Semaine	130
Jour	914

Commentaires

Cette page a-t-elle été utile ? Yes No

DATEVALUE

Article • 20/10/2023

Convertit une date au format texte en une date au format datetime.

Syntaxe

```DAX
DATEVALUE(date_text)
```

Paramètres

Agrandir le tableau

Terme	Définition
date_text	Texte représentant une date.

Valeur de propriété / Valeur de retour

Date au format **DateHeure**.

Notes

- Lors de la conversion, DATEVALUE utilise les paramètres régionaux, ainsi que les paramètres de date et d'heure du modèle pour déterminer une valeur de date. Si les paramètres de date/heure du modèle représentent des dates au format mois/jour/année, la chaîne « 1/8/2009 » est convertie en valeur **DateHeure** correspondant au 8 janvier 2009. Cependant, si les paramètres de date/heure du modèle représentent des dates au format jour/mois/année, cette chaîne est convertie en valeur **DateHeure** correspondant au 1er août 2009.
- Si la conversion à l'aide des paramètres régionaux et des paramètres date/heure du modèle échoue, DATEVALUE tente d'utiliser d'autres formats de date. Dans ce cas, certaines lignes peuvent être converties à l'aide d'un format et d'autres lignes sont converties à l'aide d'un format différent. Par exemple, « 5/4/2018 » peut être converti en 4 mai 2018 et « 20/4/2018 » peut être converti en 20 avril.

- Si la partie année de l'argument **date_text** est omise, la fonction DATEVALUE utilise l'année en cours de l'horloge intégrée de l'ordinateur. Les informations d'heure de l'argument **date_text** sont ignorées.
- Les paramètres régionaux et les paramètres date/heure du modèle sont initialement déterminés par l'application et l'ordinateur lors de la création du modèle.
- Les valeurs Date et DateHeure peuvent également être spécifiées en tant que littéral au format `dt"YYYY-MM-DD"`, `dt"YYYY-MM-DDThh:mm:ss"` ou `dt"YYYY-MM-DD hh:mm:ss"`. Quand elle est spécifiée en tant que littéral, il n'est pas nécessaire d'utiliser la fonction DATEVALUE dans l'expression. Pour en savoir plus, consultez Syntaxe DAX | Date et heure.

Exemple

L'exemple suivant retourne une valeur **DateHeure** différente selon les paramètres régionaux du modèle et les paramètres d'affichage de date/heure.

- Avec des paramètres de date/heure où le jour précède le mois, l'exemple retourne une valeur **DateHeure** correspondant au 8 janvier 2009.
- Avec des paramètres de date/heure où le mois précède le jour, l'exemple retourne une valeur **DateHeure** correspondant au 1er août 2009.

DAX

```
= DATEVALUE("8/1/2009")
```

Contenu connexe

Fonctions de date et heure

Commentaires

Cette page a-t-elle été utile ? Yes No

DAY

Article • 14/12/2023

Retourne le jour du mois, un nombre compris entre 1 et 31.

Syntaxe

```
DAX

DAY(<date>)
```

Paramètres

Agrandir le tableau

Terme	Définition
date	Date au format **DateHeure** ou représentation textuelle d'une date.

Valeur de retour

Nombre entier indiquant le jour du mois.

Remarques

- La fonction DAY prend comme argument la date du jour que vous recherchez. Vous pouvez spécifier des dates dans la fonction à l'aide d'une autre fonction de date, en utilisant une expression qui retourne une date ou en tapant une date au format **DateHeure**. Vous pouvez également taper une date en respectant l'un des formats de chaîne acceptés pour les dates.

- Les valeurs retournées par les fonctions YEAR, MONTH et DAY sont des valeurs grégoriennes, quel que soit le format d'affichage de la valeur de date fournie. Par exemple, si le format d'affichage de la date fournie est basé sur le calendrier hégirien, les valeurs retournées par les fonctions YEAR, MONTH et DAY sont associées à la date grégorienne équivalente.

- Quand l'argument de date est une représentation textuelle de la date, la fonction DAY utilise les paramètres régionaux et de date/heure de l'ordinateur client pour

interpréter la valeur de texte et effectuer la conversion. Si les paramètres de date/heure actuels représentent des dates au format mois/jour/année, la chaîne « 1/8/2009 » est interprétée comme une valeur **DateHeure** correspondant au 8 janvier 2009 et la fonction retourne le chiffre 8. Cependant, si les paramètres de date/heure actuels représentent des dates au format jour/mois/année, cette chaîne est interprétée comme une valeur **DateHeure** correspondant au 1er août 2009 et la fonction retourne le chiffre 1.

Exemple : Obtention du jour à partir d'une colonne de date

La formule suivante retourne le jour de la date figurant dans la colonne [BirthDate].

DAX

```
= DAY([Birthdate])
```

Exemple - Obtention du jour à partir d'une date sous forme de chaîne

Les formules suivantes retournent le jour 4 en utilisant les dates fournies sous forme de chaînes dans un format de texte accepté.

DAX

```
= DAY("3-4-1007")
= DAY("March 4 2007")
```

Exemple - Utilisation d'une valeur de jour comme condition

L'expression suivante retourne le jour auquel chaque commande client a été passée et marque la ligne comme article promotionnel si la commande a été passée le 10 du mois.

DAX

```
= IF( DAY([SalesDate])=10,"promotion","")
```

Contenu connexe

Fonctions de date et heure
TODAY, fonction
DATE, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

EDATE

Article • 20/10/2023

Retourne la date qui correspond au nombre de mois indiqué avant ou après la date de début. Utilisez EDATE pour calculer les dates d'échéance correspondant au même jour du mois que la date d'émission.

Syntaxe

DAX

```
EDATE(<start_date>, <months>)
```

Paramètres

Agrandir le tableau

Terme	Définition
start_date	Date au format **DateHeure** ou **texte** représentant la date de début.
mois	Nombre entier représentant le nombre de mois avant ou après **start_date**.

Valeur de retour

Date (**DateHeure**).

Remarques

- Contrairement à Microsoft Excel, qui stocke les dates comme des numéros de série séquentiels, DAX utilise les dates au format **DateHeure**. Les dates stockées dans d'autres formats sont converties implicitement.
- Si **start_date** n'est pas une date valide, la fonction EDATE retourne une erreur. Assurez-vous que la référence de colonne ou la date que vous spécifiez comme premier argument est une date.
- Si la valeur **months** n'est pas un entier, elle est tronquée.

- Quand l'argument de date est une représentation textuelle de la date, la fonction EDATE utilise les paramètres régionaux et de date et heure de l'ordinateur client pour interpréter la valeur de texte et effectuer la conversion. Si les paramètres de date et heure actuels représentent une date au format mois/jour/année, la chaîne « 1/8/2009 » est interprétée comme une valeur DateHeure correspondant au 8 janvier 2009. Cependant, si les paramètres de date et heure actuels représentent une date au format jour/mois/année, cette chaîne est interprétée comme une valeur DateHeure correspondant au 1er août 2009.

- Si la date demandée est postérieure au dernier jour du mois correspondant, le dernier jour du mois est retourné. Par exemple, les fonctions suivantes : EDATE("2009-01-29", 1), EDATE("2009-01-30", 1), EDATE("2009-01-31", 1) retournent 28 février 2009 ; cela correspond à un mois après la date de début.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant retourne la date trois mois après la date de commande, qui est stockée dans la colonne [TransactionDate].

DAX

```
= EDATE([TransactionDate],3)
```

Contenu connexe

EOMONTH, fonction
Fonctions de date et heure

Commentaires

Cette page a-t-elle été utile ? Yes No

EOMONTH

Article • 20/10/2023

Retourne la date au format **datetime** du dernier jour du mois avant ou après un nombre de mois spécifié. Utilisez EOMONTH pour calculer les dates d'échéance qui se situent le dernier jour du mois.

Syntaxe

DAX

```
EOMONTH(<start_date>, <months>)
```

Paramètres

Agrandir le tableau

Terme	Définition
start_date	Date de début au format **datetime** ou dans une représentation textuelle acceptée d'une date.
mois	Nombre représentant le nombre de mois avant ou après **start_date**. **Remarque :** Si vous entrez un nombre qui n'est pas un entier, le nombre est arrondi à l'entier immédiatement inférieur ou supérieur.

Valeur de retour

Date (**DateHeure**).

Remarques

- Contrairement à Microsoft Excel, qui stocke les dates comme des numéros de série séquentiels, DAX utilise les dates au format **DateHeure**. La fonction EOMONTH peut accepter des dates sous d'autres formats, avec les restrictions suivantes :
- Si **start_date** n'est pas une date valide, EOMONTH retourne une erreur.
- Si **start_date** est une valeur numérique qui n'est pas au format **datetime**, EOMONTH convertit le nombre en une date. Pour éviter des résultats inattendus,

convertissez le nombre au format **datetime** avant d'utiliser la fonction EOMONTH.

- Si **start_date** avec months génère une date non valide, EOMONTH retourne une erreur. Les dates antérieures au 1er mars 1900 et postérieures au 31 décembre 9999 ne sont pas valides.
- Quand l'argument date est une représentation textuelle de la date, la fonction EDATE utilise les paramètres régionaux et les paramètres de date et d'heure de l'ordinateur client pour comprendre la valeur texte afin d'effectuer la conversion. Si les paramètres de date et d'heure actuels représentent une date au format mois/jour/année, la chaîne suivante « 1/8/2009 » est interprétée comme une valeur datetime équivalente au 8 janvier 2009. Cependant, si les paramètres de date et heure actuels représentent une date au format jour/mois/année, cette chaîne est interprétée comme une valeur DateHeure correspondant au 1er août 2009.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'expression suivante retourne le 31 mai 2008, car l'argument **months** est arrondi à 2.

DAX

```
= EOMONTH("March 3, 2008",1.5)
```

Contenu connexe

EDATE, fonction
Fonctions de date et heure

Commentaires

Cette page a-t-elle été utile ? Yes No

HOUR

Article • 15/12/2023

Retourne l'heure sous la forme d'un nombre compris entre 0 (00:00) et 23 (23:00).

Syntaxe

DAX

```
HOUR(<datetime>)
```

Paramètres

Agrandir le tableau

Terme	Définition
datetime	Valeur **datetime**, par exemple 16:48:00 ou 4:48 PM.

Valeur renvoyée

Nombre entier compris entre 0 et 23.

Notes

- La fonction HOUR prend comme argument l'heure qui contient l'heure que vous voulez rechercher. Vous pouvez fournir l'heure en utilisant une fonction date/heure, une expression qui retourne un **datetime**ou en tapant directement la valeur dans l'un des formats d'heure acceptés. Vous pouvez aussi entrer les heures sous la forme d'une représentation textuelle acceptée d'une heure.

- Quand l'argument **datetime** est une représentation textuelle de la date et de l'heure, la fonction utilise les paramètres régionaux et les paramètres de date/heure de l'ordinateur client pour comprendre la valeur texte et effectuer la conversion. La plupart des paramètres régionaux utilisent le signe deux-points (:) comme séparateur horaire et tout texte d'entrée utilisant des signes deux-points comme séparateurs d'heure est analysé correctement. Examinez vos paramètres régionaux pour comprendre vos résultats.

Exemple 1

L'exemple suivant retourne l'heure de la colonne **TransactionTime** d'une table nommée **Orders**.

```
DAX

= HOUR('Orders'[TransactionTime])
```

Exemple 2

L'exemple suivant retourne 15, c'est-à-dire l'heure correspondant à 3 PM dans un format 24 heures. La valeur texte est automatiquement analysée et convertie en valeur date/heure.

```
DAX

= HOUR("March 3, 2008 3:00 PM")
```

Contenu connexe

Fonctions de date et heure
MINUTE, fonction
YEAR, fonction
SECOND, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

MINUTE

Article • 14/12/2023

Retourne la minute sous forme de nombre compris entre 0 et 59, à partir d'une valeur de date et d'heure.

Syntaxe

DAX

```
MINUTE(<datetime>)
```

Paramètres

Agrandir le tableau

Terme	Définition
datetime	Valeur **datetime** ou texte dans un format d'heure accepté, par exemple 16:48:00 ou 4:48 PM.

Valeur de retour

Nombre entier compris entre 0 et 59.

Remarques

- Contrairement à Microsoft Excel, qui stocke les dates et les heures dans un format numérique de série, DAX utilise un type de données **datetime** pour les dates et les heures. Vous pouvez fournir la valeur **datetime** à la fonction MINUTE en référençant une colonne qui stocke des dates et des heures, à l'aide d'une fonction date/time ou en utilisant une expression qui retourne une date et une heure.
- Quand l'argument **datetime** est une représentation textuelle de la date et de l'heure, la fonction utilise les paramètres régionaux et les paramètres de date/heure de l'ordinateur client pour comprendre la valeur texte et effectuer la conversion. La plupart des paramètres régionaux utilisent le signe deux-points (:) comme séparateur horaire et tout texte d'entrée utilisant des signes deux-points

comme séparateurs d'heure est analysé correctement. Vérifiez vos paramètres régionaux pour comprendre vos résultats.

Exemple 1

L'exemple suivant retourne la minute à partir de la valeur stockée dans la colonne **TransactionTime** de la table **Orders**.

```
DAX

= MINUTE(Orders[TransactionTime])
```

Exemple 2

L'exemple suivant retourne 45, ce qui correspond au nombre de minutes dans l'heure 1:45 PM.

```
DAX

= MINUTE("March 23, 2008 1:45 PM")
```

Contenu connexe

Fonctions de date et heure
HOUR, fonction
YEAR, fonction
SECOND, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

MONTH

Article • 14/12/2023

Retourne le mois sous la forme d'un nombre compris entre 1 (janvier) et 12 (décembre).

Syntaxe

```DAX
MONTH(<datetime>)
```

Paramètres

Agrandir le tableau

Terme	Définition
date	Date au format **datetime** ou texte.

Valeur de retour

Nombre entier compris entre 1 et 12.

Remarques

- Contrairement à Microsoft Excel, qui stocke les dates comme des numéros de série, DAX utilise un format **datetime** lors de l'utilisation de dates. Vous pouvez entrer la date utilisée comme argument de la fonction MONTH en tapant un format **datetime** accepté, en fournissant une référence à une colonne qui contient des dates ou en utilisant une expression qui retourne une date.
- Les valeurs retournées par les fonctions YEAR, MONTH et DAY sont des valeurs grégoriennes, quel que soit le format d'affichage de la valeur de date fournie. Par exemple, si le format d'affichage de la date fournie est Hijri, les valeurs retournées pour les fonctions YEAR, MONTH et DAY sont associées à la date grégorienne équivalente.
- Quand l'argument date est une représentation textuelle de la date, la fonction utilise les paramètres régionaux et les paramètres de date et d'heure de

l'ordinateur client pour comprendre la valeur texte afin d'effectuer la conversion. Si les paramètres de date et d'heure actuels représentent une date au format mois/jour/année, la chaîne suivante « 1/8/2009 » est interprétée comme une valeur datetime équivalente au 8 janvier 2009 et la fonction génère le résultat 1. Toutefois, si les paramètres de date et d'heure actuels représentent une date au format jour/mois/année, la même chaîne est interprétée comme une valeur datetime équivalente au 1er août 2009 et la fonction génère le résultat 8.

- Si la représentation textuelle de la date ne peut pas être correctement convertie en valeur datetime, la fonction retourne une erreur.

Exemple 1

L'expression suivante retourne 3, qui est l'entier correspondant à mars, le mois dans l'argument **date**.

```
DAX

= MONTH("March 3, 2008 3:45 PM")
```

Exemple 2

L'expression suivante retourne le mois à partir de la date dans la colonne **TransactionDate** de la table **Orders**.

```
DAX

= MONTH(Orders[TransactionDate])
```

Contenu connexe

Fonctions de date et heure
HOUR, fonction
MINUTE, fonction
YEAR, fonction
SECOND, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

NETWORKDAYS

Article • 20/10/2023

Renvoie le nombre de jours ouvrés entiers entre deux dates (inclus). Les paramètres précisent quels jours et combien de jours constituent un weekend. Les jours de weekend et les jours spécifiés comme jours fériés ne sont pas considérés comme des jours ouvrés.

Syntaxe

DAX

```
NETWORKDAYS(<start_date>, <end_date>[, <weekend>, <holidays>])
```

Paramètres

Agrandir le tableau

Terme	Définition
start_date	Date qui représente la date de début. Dates pour lesquelles la différence doit être calculée. La valeur start_date peut être antérieure, égale ou ultérieure à la valeur end_date.
end_date	Date qui représente la date de fin. Dates pour lesquelles la différence doit être calculée. La valeur start_date peut être antérieure, égale ou ultérieure à la valeur end_date.
Weekend	Indique les jours de la semaine qui représentent des jours de weekend et qui ne sont pas inclus dans le nombre de jours ouvrables entiers entre la date de début et la date de fin. Weekend est un numéro de weekend qui spécifie le moment où surviennent les weekends. Les valeurs numériques weekend indiquent les jours de weekend suivants : 1 ou omis : samedi, dimanche 2 : dimanche, lundi 3: lundi, mardi 4 : mardi, mercredi 5 : mercredi, jeudi 6 : jeudi, vendredi 7 : vendredi, samedi 11 : dimanche uniquement 12 : lundi uniquement 13 : mardi uniquement 14 : mercredi uniquement 15 : jeudi uniquement

Terme	Définition
	16 : vendredi uniquement 17 : samedi uniquement
jours fériés	Table à colonne d'une ou de plusieurs dates à exclure du calendrier des jours ouvrés.

Valeur de retour

Nombre entier de jours ouvrés entiers.

Notes

- Cette fonction DAX est similaire aux fonctions Excel NETWORKDAYS.INTL et NETWORKDAYS.
- Si les valeurs start_date et end_date sont BLANK, la valeur de sortie est également BLANK.
- Si la valeur start_date ou end_date est BLANK, la valeur start_date ou end_date BLANK sera traitée comme Date(1899, 12, 30).
- Les dates doivent être spécifiées à l'aide de la fonction DATE ou en tant que résultat d'une autre expression. Par exemple, `DATE ( 2022, 5, 30 )` correspond à 30 mai 2022. Les dates peuvent également être spécifiées comme un littéral au format `(dt"2022-05-30")`. Ne spécifiez pas de dates en tant que texte.

Exemple

L'expression suivante :

DAX

```
= NETWORKDAYS (
    DATE ( 2022, 5, 28 ),
    DATE ( 2022, 5, 30 ),
    1,
    {
        DATE ( 2022, 5, 30 )
    }
)
```

Retourne les informations suivantes :

Agrandir le tableau

[Valeur]
0

Dans cet exemple, la valeur 0 est retournée car la date de début est un samedi et la date de fin est un lundi. Le paramètre weekend spécifie que le weekend inclut le samedi et le dimanche, de sorte que ces jours ne sont pas des jours ouvrés. Le paramètre holiday marque le 30 (date de fin) comme un jour férié. Il ne reste donc aucun jour ouvré.

Commentaires

Cette page a-t-elle été utile ? Yes No

NOW

Article • 14/12/2023

Retourne la date et l'heure actuelles au format **DateHeure**.

La fonction NOW est utile quand vous devez faire apparaître la date et l'heure actuelles dans une feuille de calcul ou calculer une valeur en fonction de la date et de l'heure actuelles et que cette valeur doit être mise à jour chaque fois que vous ouvrez la feuille de calcul.

Syntaxe

```DAX
NOW()
```

Valeur de retour

Date (**DateHeure**).

Remarques

- Le résultat de la fonction NOW change uniquement quand la colonne contenant la formule est actualisée. Il n'est pas mis à jour en continu.
- Dans le service Power BI, le résultat de la fonction NOW est toujours exprimé selon le fuseau horaire UTC.
- La fonction TODAY retourne la même date, mais pas une heure exacte : l'heure retournée est toujours 12:00:00 AM et seule la date est mise à jour.

Exemple

L'exemple suivant retourne la date et l'heure actuelles plus 3,5 jours :

```DAX
= NOW()+3.5
```

Contenu connexe

UTCNOW, fonction
TODAY, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

QUARTER

Article • 14/12/2023

Retourne le trimestre sous forme de nombre compris entre 1 (janvier-mars) et 4 (octobre-décembre).

Syntaxe

```
DAX

QUARTER(<date>)
```

Paramètres

Agrandir le tableau

Terme	Définition
date	Une date.

Valeur renvoyée

Nombre entier compris entre 1 et 4.

Notes

Si la valeur d'entrée est VIDE, la valeur de sortie est également VIDE.

Exemple 1

La requête DAX suivante :

```
DAX

EVALUATE { QUARTER(DATE(2019, 2, 1)), QUARTER(DATE(2018, 12, 31)) }
```

Retourne :

Agrandir le tableau

[Value]
1
4

Exemple 2

La requête DAX suivante :

```
DAX

EVALUATE
ADDCOLUMNS(
    FILTER(
        VALUES(
            FactInternetSales[OrderDate]),
            [OrderDate] >= DATE(2008, 3, 31) && [OrderDate] <= DATE(2008, 4,
1)
        ),
    "Quarter", QUARTER([OrderDate])
)
```

Retourne :

Agrandir le tableau

FactInternetSales[OrderDate]	[Quarter]
3/31/2008	1
4/1/2008	2

Contenu connexe

YEAR
MONTH
DAY

Commentaires

Cette page a-t-elle été utile ? Yes No

SECOND

Article • 20/10/2023

Retourne les secondes d'une valeur d'heure, sous la forme d'un nombre compris entre 0 et 59.

Syntaxe

```
DAX

SECOND(<time>)
```

Paramètres

Agrandir le tableau

Terme	Définition
time	Heure au format **datetime**, par exemple 16:48:23 ou 4:48:47 PM.

Valeur de retour

Nombre entier compris entre 0 et 59.

Remarques

- Contrairement à Microsoft Excel, qui stocke les dates et les heures sous forme de numéros de série, DAX utilise un format **datetime** pour les dates et les heures. Si les données source ne sont pas dans ce format, DAX convertit implicitement les données. Vous pouvez utiliser une mise en forme pour afficher les dates et les heures sous forme de numéros de série, le cas échéant.
- La valeur de date/heure que vous fournissez comme argument à la fonction SECOND peut être entrée sous forme de chaîne de texte entre guillemets (par exemple, "6:45 PM"). Vous pouvez aussi fournir une valeur d'heure comme résultat d'une autre expression ou comme référence à une colonne qui contient des heures.

- Si vous fournissez une valeur numérique d'un autre type de données, par exemple 13.60, la valeur est interprétée comme étant un numéro de série et est représentée comme type de données **datetime** avant l'extraction de la valeur pour les secondes. Pour mieux comprendre vos résultats, vous pouvez représenter ces nombres sous forme de dates avant de les utiliser dans la fonction SECOND. Par exemple, si vous utilisez SECOND avec une colonne qui contient une valeur numérique comme **25.56**, la formule retourne 24. En effet, quand elle est mise en forme en tant que date, la valeur 25.56 correspond au 25 janvier 1900, 1:26:24 PM.

- Quand l'argument **time** est une représentation textuelle d'une date et heure, la fonction utilise les paramètres régionaux et les paramètres de date/heure de l'ordinateur client pour comprendre la valeur texte et effectuer la conversion. La plupart des paramètres régionaux utilisent le signe deux-points (:) comme séparateur horaire et tout texte d'entrée utilisant des signes deux-points comme séparateurs d'heure est analysé correctement. Examinez vos paramètres régionaux pour comprendre vos résultats.

Exemple 1

La formule suivante retourne le nombre de secondes contenues dans l'heure figurant dans la colonne **TransactionTime** d'une table nommée **Orders**.

DAX

```
= SECOND('Orders'[TransactionTime])
```

Exemple 2

La formule suivante retourne 3, qui est le nombre de secondes contenues dans l'heure représentée par la valeur **3 mars 2008 12:00:03**.

DAX

```
= SECOND("March 3, 2008 12:00:03")
```

Contenu connexe

Fonctions de date et heure
HOUR

MINUTE
YEAR

Commentaires

Cette page a-t-elle été utile ? Yes No

TEMPS

Article • 20/10/2023

Convertit les heures, minutes et secondes données sous forme de nombres en heure au format **DateHeure**.

Syntaxe

DAX

```
TIME(hour, minute, second)
```

Paramètres

Agrandir le tableau

Terme	Définition
hour	**Mode Importation :** nombre compris entre 0 et 32767, représentant les heures. Toute valeur supérieure à 23 est divisée par 24. Le reste constitue la valeur d'heure, représentée sous la forme d'une fraction d'un jour. Par exemple, TIME(27,0,0) = TIME(3,0,0) = 3:00:00 AM **Mode DirectQuery :** nombre compris entre 0 et 23, représentant les heures.
minute	**Mode Importation :** nombre compris entre 0 et 32767, représentant les minutes. Toute valeur supérieure à 59 minutes est convertie en heures et en minutes. Toute valeur supérieure à 1440 (24 heures) ne modifie pas la partie de la date. À la place, elle est divisée par 1440 et le reste est traité comme la valeur de la minute, représentée sous la forme d'une fraction d'un jour. Par exemple, TIME(0,2190,0) = TIME(0,750,0) = TIME(12,30,0) = 12:30:00 PM **Mode DirectQuery :** nombre compris entre 0 et 59, représentant les minutes.
second	**Mode Importation :** nombre compris entre 0 et 32767, représentant les secondes. Toute valeur supérieure à 59 est convertie en heures, minutes et secondes. Par exemple, TIME(0,0,2000) = TIME(0,33,20) = 12:33:20 AM **Mode DirectQuery :** nombre compris entre 0 et 59, représentant les secondes.

Valeur retournée

Une heure (**datetime**) comprise entre 00:00:00 (12:00:00 AM) et 23:59:59 (11:59:59 PM).

Remarques

- Contrairement à Microsoft Excel qui stocke les dates et les heures sous la forme de numéros de série, DAX utilise les valeurs de date et d'heure au format **DateHeure**. Les nombres dans d'autres formats sont implicitement convertis quand vous utilisez une valeur de date/heure dans une fonction DAX. Si vous avez besoin d'utiliser des numéros de série, vous pouvez utiliser la mise en forme pour changer la façon dont les nombres sont affichés.

- Les valeurs d'heure font partie d'une valeur de date et sont représentées par un nombre décimal dans le système de numéros de série. Ainsi, la valeur **DateHeure** 12:00 PM équivaut à 0,5 car elle correspond à la moitié de la journée.

- Vous pouvez fournir les arguments à la fonction TIME de plusieurs manières : en tapant directement des valeurs, en spécifiant le résultat d'une autre expression ou en faisant référence à une colonne qui contient une valeur numérique.

- Les valeurs Date et DateHeure peuvent également être spécifiées en tant que littéral au format `dt"YYYY-MM-DD"`, `dt"YYYY-MM-DDThh:mm:ss"` ou `dt"YYYY-MM-DD hh:mm:ss"`. Quand elle est spécifiée en tant que littéral, il n'est pas nécessaire d'utiliser la fonction TIME dans l'expression. Pour en savoir plus, consultez Syntaxe DAX | Date et heure.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple 1

Les exemples suivants retournent tous les deux l'heure, 3:00 AM :

DAX

```
= TIME(27,0,0)
```

DAX

```
= TIME(3,0,0)
```

Exemple 2

Les exemples suivants retournent tous les deux l'heure, 12:30 PM :

```
DAX

= TIME(0,750,0)
```

```
DAX

= TIME(12,30,0)
```

Exemple 3

L'exemple suivant crée une heure basée sur les valeurs des colonnes `intHours`, `intMinutes` et `intSeconds` :

```
DAX

= TIME([intHours],[intMinutes],[intSeconds])
```

Contenu connexe

DATE
Fonctions de date et heure

Commentaires

Cette page a-t-elle été utile ? Yes No

TIMEVALUE

Article • 14/12/2023

Convertit une heure au format texte en une heure au format datetime.

Syntaxe

```
DAX

TIMEVALUE(time_text)
```

Paramètres

Agrandir le tableau

Terme	Définition
time_text	Chaîne de texte qui représente une certaine heure de la journée. Toute information de date incluse dans l'argument **time_text** est ignorée.

Valeur de retour

Date (**DateHeure**).

Remarques

- Les valeurs d'heure font partie d'une valeur de date et sont représentées par un nombre décimal. Par exemple, 12:00 PM est représenté sous la forme 0,5 (soit une demi-journée).
- Quand l'argument **time_text** est une représentation textuelle de la date et de l'heure, la fonction utilise les paramètres régionaux et les paramètres date/heure du modèle pour comprendre la valeur texte et effectuer la conversion. La plupart des paramètres régionaux utilisent le signe deux-points (:) comme séparateur horaire ; tout texte d'entrée utilisant des signes deux-points comme séparateurs d'heure est analysé correctement. Examinez vos paramètres régionaux pour comprendre vos résultats.

- Les valeurs Date et DateHeure peuvent également être spécifiées en tant que littéral au format `dt"YYYY-MM-DD"`, `dt"YYYY-MM-DDThh:mm:ss"` ou `dt"YYYY-MM-DD hh:mm:ss"`. Quand elle est spécifiée en tant que littéral, il n'est pas nécessaire d'utiliser la fonction TIMEVALUE dans l'expression. Pour en savoir plus, consultez Syntaxe DAX | Date et heure.

Exemple

DAX

```
= TIMEVALUE("20:45:30")
```

Contenu connexe

Fonctions de date et heure

Commentaires

Cette page a-t-elle été utile ? Yes No

TODAY

Article • 14/12/2023

Retourne la date actuelle.

Syntaxe

```DAX
TODAY()
```

Valeur de retour

Date (**DateHeure**).

Remarques

- La fonction TODAY est utile quand vous devez faire apparaître la date actuelle dans une feuille de calcul, quel que soit le moment où vous ouvrez le classeur. Elle est également utile pour le calcul d'intervalles.
- Si la fonction TODAY ne met pas à jour la date au moment souhaité, vous devrez peut-être changer les paramètres qui contrôlent le moment auquel la colonne ou le classeur est actualisé.
- La fonction NOW est similaire. Cependant, elle retourne l'heure exacte, tandis que TODAY retourne la valeur d'heure 12:00:00 PM pour toutes les dates.

Exemple

Si vous savez qu'une personne est née en 1963, vous pouvez utiliser la formule suivante pour déterminer son âge au jour de son anniversaire cette année :

```DAX
= YEAR(TODAY())-1963
```

Cette formule utilise la fonction TODAY comme argument de la fonction YEAR pour obtenir l'année actuelle. Elle soustrait ensuite 1963 pour retourner l'âge de la personne.

Contenu connexe

Fonctions de date et heure
NOW

Commentaires

Cette page a-t-elle été utile ? Yes No

UTCNOW

Article • 14/12/2023

Retourne la date et l'heure UTC actuelles.

Syntaxe

```
DAX

UTCNOW()
```

Valeur de retour

Une valeur **datetime**.

Remarques

Le résultat de la fonction UTCNOW change uniquement quand la formule est actualisée. Il n'est pas mis à jour en continu.

Exemple

La formule :

```
DAX

EVALUATE { FORMAT(UTCNOW(), "General Date") }
```

Retourne :

Agrandir le tableau

[Value]
2/2/2018 4:48:08 AM

Contenu connexe

NOW, fonction

UTCTODAY, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

UTCTODAY

Article • 14/12/2023

Retourne la date UTC actuelle.

Syntaxe

```
DAX

UTCTODAY()
```

Valeur de retour

Une date.

Remarques

- UTCTODAY retourne la valeur horaire 12:00:00 PM pour toutes les dates.
- La fonction UTCNOW est similaire, mais retourne la date et l'heure exactes.

Exemple

La formule :

```
DAX

EVALUATE { FORMAT(UTCTODAY(), "General Date") }
```

Retourne :

Agrandir le tableau

[Value]
2/2/2018

Contenu connexe

NOW, fonction
UTCNOW, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

WEEKDAY

Article • 20/10/2023

Retourne un nombre compris entre 1 et 7 identifiant le jour de la semaine d'une date. Par défaut, le jour est compris entre 1 (dimanche) et 7 (samedi).

Syntaxe

DAX

```
WEEKDAY(<date>, <return_type>)
```

Paramètres

Agrandir le tableau

Terme	Définition
date	Date au format **DateHeure**. Les dates doivent être entrées à l'aide de la fonction DATE, en utilisant des expressions qui génèrent une date, ou comme résultat d'autres formules.
return_type	Nombre qui détermine la valeur de retour : Type de retour : **1**, la semaine commence le dimanche (1) et se termine le samedi (7). numérotation de 1 à 7. Type de retour : **2**, la semaine commence le lundi (1) et se termine le dimanche (7). Type de retour : **3**, la semaine commence le lundi (0) et se termine le dimanche (6).numéroration de 1 à 7.

Valeur renvoyée

Nombre entier compris entre 1 et 7.

Notes

- Contrairement à Microsoft Excel, qui stocke les dates sous forme de numéros de série, DAX utilise des dates et des heures de format **datetime**. Si vous devez

afficher les dates sous forme de numéros de série, vous pouvez utiliser les options de mise en forme d'Excel.

- Vous pouvez aussi taper les dates dans une représentation textuelle acceptée de date, mais pour éviter des résultats inattendus, il est préférable de convertir d'abord la date de format texte dans un format **datetime**.

- Quand l'argument date est une représentation textuelle de la date, la fonction utilise les paramètres régionaux et les paramètres de date et d'heure de l'ordinateur client pour comprendre la valeur texte et effectuer la conversion. Si les paramètres de date/heure actuels représentent des dates au format mois/jour/année, la chaîne « 1/8/2009 » est interprétée comme une valeur **datetime** correspondant au 8 janvier 2009. Cependant, si les paramètres de date/heure actuels représentent des dates au format jour/mois/année, la même chaîne est interprétée comme une valeur **datetime** correspondant au 1er août 2009.

Exemple

L'exemple suivant obtient la date de la colonne [HireDate], ajoute 1 et affiche le jour de la semaine correspondant à cette date. Comme l'argument **return_type** a été omis, le format par défaut est utilisé, dans lequel 1 est dimanche et 7 samedi. Si le résultat est 4, le jour est mercredi.

DAX

```
= WEEKDAY([HireDate]+1)
```

Contenu connexe

Fonctions de date et heure
WEEKNUM, fonction
YEARFRAC, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

WEEKNUM

Article • 20/10/2023

Retourne le numéro de semaine pour la date donnée en fonction de la valeur **return_type**. Le numéro de semaine indique la position numérique de la semaine dans une année.

Il existe deux *systèmes* utilisés pour cette fonction :

- **Système 1** : la semaine contenant le 1er janvier est la première semaine de l'année et est numérotée semaine 1.
- **Système 2** : la semaine contenant le premier jeudi de l'année est la première semaine de l'année et est numérotée semaine 1. Ce système suit la méthodologie spécifiée par la norme ISO 8601, communément appelée « système européen de numérotation des semaines ».

Syntaxe

DAX

```
WEEKNUM(<date>[, <return_type>])
```

Paramètres

Agrandir le tableau

Terme	Définition
date	Date au format **datetime**.
return_type	(Facultatif) Nombre qui détermine le jour de début de la semaine. 1 constitue la valeur par défaut. Consultez la section Notes.

Valeur de retour

Nombre entier.

Remarques

- Par défaut, la fonction WEEKNUM utilise une convention de calendrier selon laquelle la semaine contenant le 1er janvier est considérée comme la première semaine de l'année. Toutefois, la norme internationale ISO 8601, largement utilisée en Europe, définit la première semaine du calendrier comme celle où la majorité des jours (quatre ou plus) tombent dans la nouvelle année. Par conséquent, si la valeur **return_type** est une valeur valide autre que 21, pour les années où il y a trois jours ou moins dans la première semaine de janvier, la fonction WEEKNUM retourne des numéros de semaine différents de ceux de la norme ISO 8601.

- Pour **return_type**, à l'exception de la valeur 21, les valeurs valides suivantes peuvent ne pas être prises en charge par certaines sources de données DirectQuery :

Agrandir le tableau

return_type	La semaine commence le	Système
1 ou omise	Dimanche	1
2	Lundi	1
11	Lundi	1
12	Mardi	1
13	Mercredi	1
14	Thursday	1
15	Vendredi	1
16	Samedi	1
17	Dimanche	1
21	Lundi	2

Exemple 1

L'exemple suivant retourne le numéro de semaine correspondant à la date du 14 février 2010. Ce calcul suppose que les semaines commencent le lundi.

DAX

```
= WEEKNUM("Feb 14, 2010", 2)
```

Exemple 2

L'exemple suivant retourne le numéro de semaine pour la date stockée dans la colonne **HireDate** de la table **Employees**. Ce calcul suppose que les semaines commencent le dimanche.

```
DAX
= WEEKNUM('Employees'[HireDate])
```

Contenu connexe

YEARFRAC, fonction
WEEKDAY, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

YEAR

Article • 20/10/2023

Retourne l'année d'une date sous forme d'un entier à quatre chiffres compris dans la plage 1900-9999.

Syntaxe

DAX

```
YEAR(<date>)
```

Paramètres

Agrandir le tableau

Terme	Définition
date	Date au format **DateHeure** ou texte contenant l'année que vous recherchez.

Valeur de retour

Entier compris dans la plage 1900-9999.

Remarques

- Contrairement à Microsoft Excel qui stocke les dates sous la forme de numéros de série, DAX utilise un type de données **DateHeure** lors de l'utilisation de dates et d'heures.
- Les dates doivent être entrées à l'aide de la fonction DATE ou doivent résulter d'autres formules ou fonctions. Vous pouvez également entrer des dates dans les représentations textuelles acceptées, par exemple 3 mars 2007 ou Mar-3-2003.
- Les valeurs retournées par les fonctions YEAR, MONTH et DAY sont des valeurs grégoriennes, quel que soit le format d'affichage de la valeur de date fournie. Par exemple, si le format d'affichage de la date fournie utilise le calendrier Hijri, les valeurs retournées pour les fonctions YEAR, MONTH et DAY sont associées à la date grégorienne équivalente.

- Quand l'argument date est une représentation textuelle de la date, la fonction utilise les paramètres régionaux et les paramètres de date et d'heure de l'ordinateur client pour comprendre la valeur texte afin d'effectuer la conversion. Des erreurs peuvent se produire si le format des chaînes est incompatible avec les paramètres régionaux actuels. Par exemple, si vos paramètres régionaux spécifient le format de date mois/jour/année et que la date est fournie au format jour/mois/année, 25/1/2009 n'est pas interprété comme le 25 janvier 2009 mais comme une date non valide.

Exemple

L'exemple suivant retourne 2007.

DAX

```
= YEAR("March 2007")
```

Exemple - Date comme résultat d'une expression

Description

L'exemple suivant retourne l'année de la date du jour.

DAX

```
= YEAR(TODAY())
```

Contenu connexe

Fonctions de date et heure
HOUR, fonction
MINUTE, fonction
YEAR, fonction
SECOND, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

YEARFRAC

Article • 14/12/2023

Calcule la fraction de l'année représentée par le nombre de jours entiers entre deux dates. Utilisez la fonction de feuille de calcul YEARFRAC pour identifier la proportion des avantages ou des obligations d'une année entière à assigner à un terme spécifique.

Syntaxe

DAX

```
YEARFRAC(<start_date>, <end_date>, <basis>)
```

Paramètres

Agrandir le tableau

Terme	Définition
start_date	Date de début au format **datetime**.
end_date	Date de fin au format **datetime**.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Tous les arguments sont tronqués en entiers. Basis – Description 0 – ÉTATS-UNIS (NASD) 30/360 (valeur par défaut) 1 - Réel/réel 2 – Réel/360 3 - Réel/365 4 – Européen 30/360

Valeur de retour

Nombre décimal. Le type de données interne est un nombre à virgule flottante double précision IEEE 64 bits (8 octets) signé.

Remarques

- Contrairement à Microsoft Excel qui stocke les dates en tant que nombres ordinaux, DAX utilise un format **datetime** pour utiliser les dates et les heures. Si vous devez afficher les dates en tant que nombres ordinaux, vous pouvez utiliser les options de mise en forme d'Excel.
- Si **start_date** ou **end_date** ne sont pas des dates valides, YEARFRAC retourne une erreur.
- Si **basis**< 0 ou si **basis**> 4, YEARFRAC retourne une erreur.

Exemple 1

L'exemple suivant retourne la fraction d'une année représentée par la différence entre les dates dans les deux colonnes, `TransactionDate` et `ShippingDate` :

```
DAX

= YEARFRAC(Orders[TransactionDate],Orders[ShippingDate])
```

Exemple 2

L'exemple suivant retourne la fraction d'une année représentée par la différence entre les dates, 1er janvier et 1er mars :

```
DAX

= YEARFRAC("Jan 1 2007","Mar 1 2007")
```

Utilisez des années à quatre chiffres chaque fois que possible pour éviter d'obtenir des résultats inattendus. Lorsque l'année est tronquée, l'année en cours est supposée. Lorsque la date est omise, la première date du mois est supposée.

Le deuxième argument, **basis**, a également été omis. Par conséquent, la fraction d'année est calculée selon la norme américaine (NASD) 30/360.

Contenu connexe

Fonctions de date et heure
WEEKNUM, fonction

YEARFRAC, fonction

WEEKDAY, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonctions de filtrage

Article • 29/02/2024

Les fonctions de filtre et de valeur de DAX comptent parmi les plus complexes et les plus puissantes. Elles présentent des différences notables avec les fonctions Excel. Les fonctions de recherche utilisent des tables et des relations, comme une base de données. Avec les fonctions de filtre, vous pouvez manipuler le contexte des données pour créer des calculs dynamiques.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
ALL	Retourne toutes les lignes d'une table, ou toutes les valeurs d'une colonne, en ignorant les filtres qui ont été éventuellement appliqués.
ALLCROSSFILTERED	Efface tous les filtres qui sont appliqués à une table.
ALLEXCEPT	Supprime tous les filtres de contexte de la table, à l'exception de ceux qui ont été appliqués aux colonnes spécifiées.
ALLNOBLANKROW	Retourne, à partir de la table parente d'une relation, toutes les lignes sauf la ligne vide ou toutes les valeurs distinctes d'une colonne sauf la ligne vide, et ignore tous les filtres de contexte qui peuvent exister.
ALLSELECTED	Supprime les filtres de contexte des colonnes et des lignes dans la requête actuelle, tout en conservant tous les autres filtres de contexte ou filtres explicites.
CALCULATE	Évalue une expression dans un contexte de filtre modifié.
CALCULATETABLE	Évalue une expression de table dans un contexte de filtre modifié.
EARLIER	Retourne la valeur actuelle de la colonne spécifiée dans un test d'évaluation externe de la colonne mentionnée.
EARLIEST	Retourne la valeur actuelle de la colonne spécifiée dans un test d'évaluation externe de la colonne mentionnée.
FILTER	Retourne une table qui représente un sous-ensemble d'une autre table ou expression.
FIRST	Utilisé uniquement dans les calculs de visuels. Récupère une valeur dans la matrice du visuel à partir de la première ligne d'un axe.

Fonction	Description
INDEX	Récupère une ligne à une position absolue, spécifiée par le paramètre de position, dans la partition spécifiée triée selon l'ordre spécifié ou sur l'axe spécifié.
KEEPFILTERS	Modifie la façon dont les filtres sont appliqués quand une fonction CALCULATE ou CALCULATETABLE est évaluée.
LAST	Utilisé uniquement dans les calculs de visuels. Récupère une valeur dans la matrice du visuel à partir de la dernière ligne d'un axe.
LOOKUPVALUE	Retourne la valeur pour la ligne qui remplit tous les critères spécifiés par les conditions de recherche. La fonction peut appliquer une ou plusieurs conditions de recherche.
MATCHBY	Dans les fonctions de fenêtre, définit les colonnes utilisées pour déterminer la manière de faire correspondre des données et d'identifier la *ligne actuelle*.
MOVINGAVERAGE	Retourne une moyenne mobile calculée le long de l'axe donné de la grille de données du calcul de visuel.
SUIVANT	Utilisé uniquement dans les calculs de visuels. Récupère une valeur dans la ligne suivante d'un axe situé dans la matrice du visuel.
DÉCALAGE	Retourne une seule ligne qui est positionnée avant ou après la *ligne actuelle* dans la même table, avec un décalage donné.
ORDERBY	Définit les colonnes qui déterminent l'ordre de tri dans chacune des partitions d'une fonction de fenêtre.
PARTITIONBY	Définit les colonnes utilisées pour partitionner le paramètre <relation> d'une fonction WINDOW.
PREVIOUS	Utilisé uniquement dans les calculs de visuels. Récupère une valeur dans la ligne précédente d'un axe situé dans la matrice du visuel.
PLAGE	Retourne un intervalle de lignes dans l'axe donné, par rapport à la ligne actuelle. Un raccourci pour WINDOW.
RANK	Retourne le classement d'une ligne dans l'intervalle donné.
REMOVEFILTERS	Efface les filtres des tables ou colonnes spécifiées.
ROWNUMBER	Retourne le classement unique d'une ligne dans l'intervalle donné.
RUNNINGSUM	Retourne une somme en exécution calculée le long de l'axe donné de la grille de données du calcul visuel.
SELECTEDVALUE	Retourne la valeur quand le contexte pour columnName a été réduit par filtrage à une seule valeur distincte. Sinon, retourne alternateResult.

Fonction	Description
WINDOW	Retourne plusieurs lignes qui sont positionnées dans l'intervalle donné.

Commentaires

Cette page a-t-elle été utile ? Yes No

ALL

Article • 20/10/2023

Retourne toutes les lignes d'une table, ou toutes les valeurs d'une colonne, en ignorant les filtres qui ont été éventuellement appliqués. Cette fonction est utile pour effacer les filtres et créer des calculs sur toutes les lignes d'une table.

Syntaxe

DAX

```
ALL( [<table> | <column>[, <column>[, <column>[,…]]]] )
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table dont vous voulez effacer les filtres.
colonne	Colonne dont vous voulez effacer les filtres.

L'argument passé à la fonction ALL doit être une référence à une table de base ou à une colonne de base. Vous ne pouvez pas utiliser d'expressions de table ni d'expressions de colonne avec la fonction ALL.

Valeur de retour

Table ou colonne dont les filtres ont été supprimés.

Remarques

- Cette fonction ne s'utilise pas seule : elle fait office de fonction intermédiaire qui permet de changer le jeu de résultats sur duquel un autre calcul est effectué.
- Le comportement normal des expressions DAX contenant la fonction ALL() consiste à ignorer tous les filtres appliqués. Toutefois, il existe des scénarios où ce n'est pas le cas en raison d'*existence automatique*, technologie DAX qui optimise le filtrage afin de réduire la quantité de traitement nécessaire pour certaines requêtes DAX.

Par exemple, l'existence automatique et ALL() fournissent des résultats inattendus lors du filtrage d'au moins deux colonnes de la même table (comme lors de l'utilisation de segments) et s'il y a une mesure sur cette table qui utilise ALL(). Dans ce cas, l'existence automatique *fusionne* les différents filtres en un seul et filtre uniquement les combinaisons de valeurs existantes. En raison de cette fusion, la mesure est calculée sur les combinaisons de valeurs existantes et le résultat est basé sur les valeurs filtrées et non sur toutes les valeurs comme prévu. Pour en savoir plus sur l'existence automatique et son effet sur les calculs, consultez l'article Understanding DAX Auto-Exist du MVP Microsoft, Alberto Ferrari, sur sql.bi.com.

- Le tableau suivant décrit comment vous pouvez utiliser les fonctions ALL et ALLEXCEPT dans différents scénarios.

Agrandir le tableau

Fonction et utilisation	Description
ALL()	Supprime tous les filtres partout. La fonction ALL() peut être utilisée uniquement pour effacer des filtres, et non pour retourner une table.
ALL(Table)	Supprime tous les filtres de la table spécifiée. En effet, ALL(Table) retourne toutes les valeurs de la table, en supprimant tous les filtres du contexte qui auraient pu autrement être appliqués. Cette fonction s'avère utile quand vous utilisez divers niveaux de regroupement et que vous voulez créer un calcul qui établit un rapport entre une valeur agrégée et la valeur totale. Le premier exemple illustre ce scénario.
ALL (Column[, Column[, ...]])	Supprime tous les filtres des colonnes spécifiées de la table ; tous les autres filtres des autres colonnes de la table continuent de s'appliquer. Tous les arguments de colonne doivent provenir de la même table. La variante ALL(Column) est utile quand vous voulez supprimer les filtres de contexte pour une ou plusieurs colonnes spécifiques et conserver tous les autres filtres de contexte. Les deuxième et troisième exemples illustrent ce scénario.
ALLEXCEPT(Table, Column1 [,Column2]...)	Supprime tous les filtres de contexte de la table, à l'exception des filtres appliqués aux colonnes spécifiées. C'est une méthode simple à utiliser si vous souhaitez supprimer les filtres sur beaucoup de colonnes, mais pas sur toutes, dans une table.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple 1

Calculer le ratio des ventes par catégorie par rapport au total des ventes

Supposez que vous voulez calculer le montant des ventes pour la cellule active, dans votre tableau croisé dynamique, divisé par le total des ventes pour tous les revendeurs. Pour faire en sorte que le dénominateur soit le même quel que soit le mode de filtrage ou de regroupement de données employé par l'utilisateur du tableau croisé dynamique, vous définissez une formule qui utilise ALL pour créer le total général exact.

Le tableau suivant présente les résultats obtenus quand la nouvelle mesure **All Reseller Sales Ratio** est créée en à partir de la formule présentée dans la section de code. Pour voir comment cela fonctionne, ajoutez le champ CalendarYear à la zone **Étiquettes de lignes** du tableau croisé dynamique, puis ajoutez le champ ProductCategoryName à la zone **Étiquettes de colonnes**. Ensuite, faites glisser la mesure **All Reseller Sales Ratio** vers la zone **Valeurs** du tableau croisé dynamique. Pour afficher les résultats sous forme de pourcentages, utilisez les fonctionnalités de mise en forme d'Excel pour appliquer une mise en forme de pourcentage aux cellules qui contiennent la mesure.

Agrandir le tableau

Étiquettes de ligne	Accessories	Bikes	Clothing	Components	Total général
2005	0,02 %	9,10 %	0,04 %	0,75 %	9,91 %
2006	0,11 %	24,71 %	0,60 %	4,48 %	29,90 %
2007	0,36 %	31,71 %	1,07 %	6,79 %	39,93 %
2008	0,20 %	16,95 %	0,48 %	2,63 %	20,26 %
Total général	0,70 %	82,47 %	2,18 %	14,65 %	100,00 %

Formule

DAX

```
= SUMX(ResellerSales_USD,
ResellerSales_USD[SalesAmount_USD])/SUMX(ALL(ResellerSales_USD),
ResellerSales_USD[SalesAmount_USD])
```

La formule est construite comme suit :

1. Le numérateur `SUMX(ResellerSales_USD, ResellerSales_USD[SalesAmount_USD])` est la somme des valeurs de ResellerSales_USD[SalesAmount_USD] pour la cellule

active du tableau croisé dynamique, avec les filtres de contexte appliqués à CalendarYear et ProductCategoryName.

2. Pour le dénominateur, vous commencez par spécifier une table, ResellerSales_USD, et utilisez la fonction ALL pour supprimer tous les filtres de contexte de la table.

3. Vous utilisez ensuite la fonction SUMX pour calculer la somme des valeurs de la colonne ResellerSales_USD[SalesAmount_USD]. Autrement dit, vous obtenez la somme de ResellerSales_USD[SalesAmount_USD] pour les ventes de tous les revendeurs.

Exemple 2

Calculer le ratio des ventes de produits par rapport au total des ventes tout au long de l'année en cours

Supposez que vous voulez créer un tableau comparant le pourcentage de ventes au fil des ans pour chaque catégorie de produit (ProductCategoryName). Pour obtenir le pourcentage pour chaque année et chaque valeur de ProductCategoryName, vous devez diviser la somme des ventes pour l'année et la catégorie de produit en question par la somme des ventes de cette même catégorie de produit de toutes les années. En d'autres termes, vous devez conserver le filtre sur ProductCategoryName, mais supprimer celui appliqué à l'année au moment de calculer le dénominateur du pourcentage.

Le tableau suivant présente les résultats obtenus quand la nouvelle mesure **Reseller Sales Year** est créée à partir de la formule présentée dans la section de code. Pour voir comment cela fonctionne, ajoutez le champ CalendarYear à la zone **Étiquettes de ligne** d'un tableau croisé dynamique et ajoutez le champ ProductCategoryName à la zone **Étiquettes de colonne**. Pour afficher les résultats sous forme de pourcentages, utilisez les fonctionnalités de mise en forme d'Excel pour appliquer un format numérique de pourcentage aux cellules contenant la mesure **Reseller Sales Year**.

Agrandir le tableau

Étiquettes de ligne	Accessories	Bikes	Clothing	Components	Total général
2005	3,48 %	11,03 %	1,91 %	5,12 %	9,91 %
2006	16,21 %	29,96 %	27,29 %	30,59 %	29,90 %
2007	51,62 %	38,45 %	48,86 %	46,36 %	39,93 %
2008	28,69 %	20,56 %	21,95 %	17,92 %	20,26 %

Étiquettes de ligne	Accessories	Bikes	Clothing	Components	Total général
Total général	100,00 %	100,00 %	100,00 %	100,00 %	100,00 %

Formule

```
DAX
= SUMX(ResellerSales_USD, ResellerSales_USD[SalesAmount_USD])/CALCULATE(
SUM( ResellerSales_USD[SalesAmount_USD]), ALL(DateTime[CalendarYear]))
```

La formule est construite comme suit :

1. Le numérateur `SUMX(ResellerSales_USD, ResellerSales_USD[SalesAmount_USD])` est la somme des valeurs de ResellerSales_USD[SalesAmount_USD] pour la cellule active du tableau croisé dynamique, avec les filtres de contexte appliqués aux colonnes CalendarYear et ProductCategoryName.

2. Pour le dénominateur, vous supprimez le filtre existant de CalendarYear à l'aide de la fonction ALL(Column). Cette fonction calcule la somme sur les lignes restantes de la table ResellerSales_USD, après application des filtres de contexte existants des étiquettes de colonnes. Résultat final pour le dénominateur : la somme est calculée sur la colonne ProductCategoryName sélectionnée (filtre de contexte implicite) et pour toutes les valeurs de l'année.

Exemple 3

Calculer la contribution des catégories de produit par rapport au total des ventes par an

Supposez que vous voulez créer une table qui présente le pourcentage des ventes pour chaque catégorie de produit, année par année. Pour obtenir le pourcentage de chaque catégorie de produit pour une année déterminée, vous devez calculer la somme des ventes dans ladite catégorie de produit (ProductCategoryName) pour l'année n, puis diviser la valeur obtenue par la somme des ventes de l'année n pour toutes les catégories de produit. En d'autres termes, vous devez conserver le filtre sur l'année, mais supprimer celui appliqué à ProductCategoryName au moment de calculer le dénominateur du pourcentage.

Le tableau suivant présente les résultats obtenus quand la nouvelle mesure **Reseller Sales CategoryName** est créée à partir de la formule présentée dans la section de code. Pour voir comment cela fonctionne, ajoutez le champ CalendarYear à la zone **Étiquettes de lignes** du tableau croisé dynamique, puis ajoutez le champ ProductCategoryName à la zone **Étiquettes de colonnes**. Ajoutez ensuite la nouvelle mesure à la zone **Valeurs** du

tableau croisé dynamique. Pour afficher les résultats sous forme de pourcentages, utilisez les fonctionnalités de mise en forme d'Excel pour appliquer un format numérique de pourcentage aux cellules qui contiennent la nouvelle mesure, **Reseller Sales CategoryName**.

Agrandir le tableau

Étiquettes de ligne	Accessories	Bikes	Clothing	Components	Total général
2005	0,25 %	91,76 %	0,42 %	7,57 %	100,00 %
2006	0,38 %	82,64 %	1,99 %	14,99 %	100,00 %
2007	0,90 %	79,42 %	2,67 %	17,01 %	100,00 %
2008	0,99 %	83,69 %	2,37 %	12,96 %	100,00 %
Total général	0,70 %	82,47 %	2,18 %	14,65 %	100,00 %

Formule

DAX

```
= SUMX(ResellerSales_USD, ResellerSales_USD[SalesAmount_USD])/CALCULATE(
SUM( ResellerSales_USD[SalesAmount_USD]),
ALL(ProductCategory[ProductCategoryName]))
```

La formule est construite comme suit :

1. Le numérateur `SUMX(ResellerSales_USD, ResellerSales_USD[SalesAmount_USD])` est la somme des valeurs de ResellerSales_USD[SalesAmount_USD] pour la cellule active du tableau croisé dynamique, avec les filtres de contexte appliqués aux champs CalendarYear et ProductCategoryName.

2. Pour le dénominateur, vous utilisez la fonction ALL(Column) pour supprimer le filtre de ProductCategoryName et calculer la somme sur les lignes restantes de la table ResellerSales_USD, après application des filtres de contexte existants des étiquettes de lignes. Résultat final pour le dénominateur : la somme est calculée pour l'année sélectionnée (filtre de contexte implicite) et toutes les valeurs de ProductCategoryName.

Contenu connexe

Fonctions de filtrage
ALL, fonction

ALLEXCEPT, fonction

FILTER, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

ALLCROSSFILTERED

Article • 20/10/2023

Efface tous les filtres qui sont appliqués à une table.

Syntaxe

```DAX
ALLCROSSFILTERED(<table>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table dont vous voulez effacer les filtres.

Valeur renvoyée

N/A. Consultez la section Remarques.

Notes

- La fonction ALLCROSSFILTERED peut uniquement être utilisée pour effacer des filtres, mais pas pour retourner une table.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

```DAX
DEFINE
MEASURE FactInternetSales[TotalQuantity1] =
    CALCULATE(SUM(FactInternetSales[OrderQuantity]),
```

```
ALLCROSSFILTERED(FactInternetSales))
MEASURE FactInternetSales[TotalQuantity2] =
    CALCULATE(SUM(FactInternetSales[OrderQuantity]), ALL(FactInternetSales))
EVALUATE
    SUMMARIZECOLUMNS(DimSalesReason[SalesReasonName],
        "TotalQuantity1", [TotalQuantity1],
        "TotalQuantity2", [TotalQuantity2])
    ORDER BY DimSalesReason[SalesReasonName]
```

Retourne :

Agrandir le tableau

DimSalesReason[SalesReasonName]	[TotalQuantity1]	[TotalQuantity2]
Événement de démonstration	60398	
Publicité dans un magazine	60398	
Fabricant	60398	1818
En promotion	60398	7390
Autre	60398	3653
Price	60398	47733
Qualité	60398	1551
Révision	60398	1640
Parrainage	60398	
Publicité à la télévision	60398	730

Notes

Il existe une relation plusieurs à plusieurs directe ou indirecte entre la table FactInternetSales et la table DimSalesReason.

Commentaires

Cette page a-t-elle été utile ? Yes No

ALLEXCEPT

Article • 20/10/2023

Supprime tous les filtres de contexte de la table, à l'exception de ceux qui ont été appliqués aux colonnes spécifiées.

Syntaxe

```
DAX

ALLEXCEPT(<table>,<column>[,<column>[,…]])
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table de laquelle sont supprimés tous les filtres de contexte, à l'exception des filtres appliqués aux colonnes spécifiées dans les arguments suivants.
colonne	Colonne pour laquelle les filtres de contexte doivent être conservés.

Le premier argument de la fonction ALLEXCEPT doit être une référence à une table de base. Tous les arguments suivants doivent être des références à des colonnes de base. Vous ne pouvez pas utiliser d'expressions de table ni d'expressions de colonne avec la fonction ALLEXCEPT.

Valeur de retour

Table dans laquelle tous les filtres ont été supprimés, à l'exception des filtres appliqués aux colonnes spécifiées.

Remarques

- Cette fonction ne s'utilise pas seule : elle fait office de fonction intermédiaire qui permet de changer le jeu de résultats sur duquel un autre calcul est effectué.
- Les fonctions ALL et ALLEXCEPT peuvent être utilisées dans différents scénarios :

Agrandir le tableau

Fonction et utilisation	Description
ALL(Table)	Supprime tous les filtres de la table spécifiée. En effet, ALL(Table) retourne toutes les valeurs de la table, en supprimant tous les filtres du contexte qui auraient pu autrement être appliqués. Cette fonction s'avère utile quand vous travaillez avec de nombreux niveaux de regroupement et que vous voulez créer un calcul qui établisse un rapport entre une valeur agrégée et la valeur totale.
ALL (Column[, Column[, ...]])	Supprime tous les filtres des colonnes spécifiées de la table ; tous les autres filtres des autres colonnes de la table continuent de s'appliquer. Tous les arguments de colonne doivent provenir de la même table. La variante ALL(Column) est utile quand vous voulez supprimer les filtres de contexte pour une ou plusieurs colonnes spécifiques et conserver tous les autres filtres de contexte.
ALLEXCEPT(Table, Column1 [,Column2]...)	Supprime tous les filtres de contexte de la table, à l'exception des filtres appliqués aux colonnes spécifiées. C'est une méthode simple à utiliser si vous souhaitez supprimer les filtres sur beaucoup de colonnes, mais pas sur toutes, dans une table.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La formule de mesure suivante calcule le total SalesAmount_USD et utilise la fonction ALLEXCEPT pour supprimer tous les filtres de contexte sur la table DateTime, sauf les filtres qui ont été appliqués à la colonne CalendarYear.

DAX

```
= CALCULATE(SUM(ResellerSales_USD[SalesAmount_USD]), ALLEXCEPT(DateTime,
DateTime[CalendarYear]))
```

Du fait que la formule utilise la fonction ALLEXCEPT, quand l'une des colonnes (à l'exception de CalendarYear) de la table DateTime est utilisée pour segmenter les données d'une visualisation, la formule supprime tous les filtres de segment, et retourne une valeur égale à la somme de SalesAmount_USD. Toutefois, si la colonne CalendarYear est utilisée pour segmenter la visualisation, les résultats sont différents. Étant donné que CalendarYear est spécifié comme argument dans la fonction ALLEXCEPT, quand les

données sont segmentées sur l'année, un filtre est appliqué sur les années au niveau des lignes

Contenu connexe

Fonctions de filtrage
ALL, fonction
FILTER, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

ALLNOBLANKROW

Article • 20/10/2023

À partir de la table parent d'une relation, retourne toutes les lignes sauf la ligne vide, ou toutes les valeurs distinctes d'une colonne sauf la ligne vide, et ignore tous les filtres de contexte qui peuvent exister.

Syntaxe

DAX

```
ALLNOBLANKROW( {<table> | <column>[, <column>[, <column>[,…]]]} )
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table sur laquelle tous les filtres de contexte sont supprimés.
colonne	Colonne sur laquelle tous les filtres de contexte sont supprimés.

Un seul paramètre doit être passé ; le paramètre est soit une table, soit une colonne.

Valeur de retour

Une table, quand le paramètre passé était une table, ou une colonne de valeurs, quand le paramètre passé était une colonne.

Remarques

- La fonction ALLNOBLANKROW filtre uniquement la ligne vide qu'une table parent, dans une relation, affiche quand les valeurs d'une ou plusieurs lignes de la table enfant n'ont pas de valeurs correspondantes dans la colonne parent. Consultez l'exemple ci-dessous pour obtenir une explication détaillée.
- Le tableau suivant récapitule les variantes de ALL proposées dans DAX et leurs différences :

Agrandir le tableau

Fonction et utilisation	Description
ALL(Column)	Supprime tous les filtres de la colonne spécifiée de la table ; tous les autres filtres sur les autres colonnes de la table continuent de s'appliquer.
ALL(Table)	Supprime tous les filtres de la table spécifiée.
ALLEXCEPT(Table,Col1,Col2...)	Remplace tous les filtres de contexte de la table, sauf sur les colonnes spécifiées.
ALLNOBLANK(table|colonne)	À partir de la table parent d'une relation, retourne toutes les lignes sauf la ligne vide, ou toutes les valeurs distinctes d'une colonne sauf la ligne vide, et ignore tous les filtres de contexte qui peuvent exister.

Pour obtenir une description générale de la façon dont la fonction ALL opère ainsi que des exemples pas à pas qui utilisent ALL(Table) et ALL(Column), consultez ALL, fonction.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Dans les données d'exemple, la table ResellerSales_USD contient une ligne sans valeurs qui ne peut donc être associée à aucune table parent dans les relations du classeur. Vous allez utiliser cette table dans un tableau croisé dynamique pour observer le comportement des lignes vides et voir comment gérer les décomptes dans les données non liées.

Étape 1 : Vérifier les données non liées

Ouvrez la **fenêtre Power Pivot**, puis sélectionnez la table ResellerSales_USD. Dans la colonne ProductKey, filtrez les valeurs vides. Il reste une ligne. Dans cette ligne, toutes les valeurs de colonne doivent être vides, sauf pour SalesOrderLineNumber.

Étape 2 : Créer un tableau croisé dynamique

Créez un tableau croisé dynamique, puis faites glisser la colonne datetime.[Année civile] vers le volet Étiquettes de ligne. Le tableau suivant présente les résultats attendus :

Agrandir le tableau

Étiquettes de ligne
2005
2006
2007
2008
Total général

Comme vous pouvez le constater, il y a une étiquette vide entre **2008** et **Total général**. Cette étiquette vide représente le Membre inconnu, qui est un groupe spécial créé pour prendre en compte les valeurs de la table enfant qui n'ont pas de valeur correspondante dans la table parent, dans cet exemple la colonne datetime.[Année civile].

La présence de cette étiquette vide dans le tableau croisé dynamique est le signe qu'il y a soit des valeurs vides, soit des valeurs sans correspondance dans certaines tables associées à la colonne, datetime.[Année civile]. La table parent est celle dans laquelle figure l'étiquette vide, mais les lignes qui ne correspondent pas se trouvent dans une ou plusieurs des tables enfant.

Les lignes ajoutées au groupe de l'étiquette vide sont des valeurs qui ne correspondent à aucune valeur dans la table parent (par exemple, une date qui n'existe pas dans la table datetime) ou des valeurs null, c'est-à-dire sans aucune valeur pour la date. Dans cet exemple, nous avons placé une valeur vide dans toutes les colonnes de la table « sales » enfant. Le fait que la table parent contienne plus de valeurs que les tables enfant ne pose pas de problème.

Étape 3 : Compter les lignes avec ALL et ALLNOBLANK

Ajoutez les deux mesures suivantes à la table datetime pour compter les lignes de la table : **Countrows ALLNOBLANK of datetime**, **Countrows ALL of datetime**. Les formules que vous pouvez utiliser pour définir ces mesures sont les suivantes :

DAX

```
// Countrows ALLNOBLANK of datetime
= COUNTROWS(ALLNOBLANKROW('DateTime'))

// Countrows ALL of datetime
= COUNTROWS(ALL('DateTime'))
```

```
// Countrows ALLNOBLANKROW of ResellerSales_USD
= COUNTROWS(ALLNOBLANKROW('ResellerSales_USD'))

// Countrows ALL of ResellerSales_USD
= COUNTROWS(ALL('ResellerSales_USD'))
```

Dans un tableau croisé dynamique vide, ajoutez datetime.[Année civile] aux étiquettes de ligne, puis ajoutez les mesures nouvellement créées. Les résultats doivent ressembler au tableau suivant :

Agrandir le tableau

Étiquettes de ligne	Countrows ALLNOBLANK of datetime	Countrows ALL of datetime
2005	1 280	1281
2006	1 280	1281
2007	1 280	1281
2008	1 280	1281
	1 280	1281
Total général	1 280	1281

Les résultats montrent une différence de 1 ligne dans le nombre de lignes de table. Cependant, si vous ouvrez la **fenêtre Power Pivot** et que vous sélectionnez la table datetime, vous ne trouverez aucune ligne vide dans la table, car la ligne vide spéciale mentionnée ici est le Membre inconnu.

Étape 4 : Vérifier que le nombre est juste

Pour prouver que ALLNOBLANKROW ne compte pas de lignes véritablement vides et ne gère que la ligne vide spéciale de la table parente, ajoutez les deux mesures suivantes à la table ResellerSales_USD : **Countrows ALLNOBLANKROW of ResellerSales_USD**, **Countrows ALL of ResellerSales_USD**.

Créez un tableau croisé dynamique, puis faites glisser la colonne datetime.[Année civile] vers le volet Étiquettes de ligne. Ajoutez maintenant les mesures que vous venez de créer. Les résultats doivent se présenter comme suit :

Agrandir le tableau

Étiquettes de ligne	Countrows ALLNOBLANKROW of ResellerSales_USD	Countrows ALL of ResellerSales_USD
2005	60856	60856
2006	60856	60856
2007	60856	60856
2008	60856	60856
	60856	60856
Total général	60856	60856

À présent, les deux mesures présentent les mêmes résultats. Cela est dû au fait que la fonction ALLNOBLANKROW ne compte pas les lignes véritablement vides dans une table, mais traite uniquement la ligne vide qui est un cas spécial généré dans une table parent quand une ou plusieurs des tables enfant de la relation contiennent des valeurs non correspondantes ou des valeurs vides.

Contenu connexe

Fonctions de filtrage
ALL, fonction
FILTER, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

ALLSELECTED

Article • 20/10/2023

Supprime les filtres de contexte des colonnes et des lignes dans la requête actuelle, tout en conservant tous les autres filtres de contexte ou filtres explicites.

La fonction ALLSELECTED obtient le contexte qui représente toutes les lignes et colonnes dans la requête, tout en conservant les contextes et les filtres explicites autres que les filtres de lignes et de colonnes. Cette fonction peut être utilisée pour obtenir des valeurs totales affichées dans des requêtes.

Syntaxe

DAX

```
ALLSELECTED([<tableName> | <columnName>[, <columnName>[, <columnName>[,…]]]]
)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableName	Nom d'une table existante, spécifié avec la syntaxe DAX standard. Ce paramètre ne peut pas être une expression. Ce paramètre est facultatif.
columnName	Nom, généralement complet, d'une colonne existante utilisant la syntaxe DAX standard. Il ne peut pas s'agir d'une expression. Ce paramètre est facultatif.

Valeur renvoyée

Contexte de la requête sans filtres de lignes ni de colonnes.

Notes

- S'il existe un argument, l'argument est *tableName* ou *columnName*. S'il existe plusieurs arguments, ils doivent être des colonnes de la même table.

- Cette fonction est différente de ALL() car elle conserve tous les filtres définis explicitement dans la requête et conserve tous les filtres de contexte autres que les filtres de lignes et de colonnes.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant montre comment générer des niveaux différents de valeurs totales affichées dans un rapport de table à l'aide d'expressions DAX. Dans le rapport, deux (2) filtres précédents ont été appliqués aux données des ventes du revendeur : l'un sur le groupe du secteur de vente (Sales Territory Group) = *Europe* et l'autre sur le type de promotion (Promotion Type) = *Remise de volume* (Volume Discount). Une fois les filtres appliqués, des valeurs totales affichées peuvent être calculées pour l'intégralité du rapport, pour toutes les années ou pour toutes les catégories de produits. En outre, à des fins d'illustration, le total général de toutes les ventes du revendeur est également obtenu, en supprimant tous les filtres du rapport. L'évaluation de l'expression DAX suivante génère une table contenant toutes les informations nécessaires à la création d'une table avec des valeurs totales affichées.

DAX

```
define
measure 'Reseller Sales'[Reseller Sales Amount]=sum('Reseller Sales'[Sales
Amount])
measure 'Reseller Sales'[Reseller Grand Total]=calculate(sum('Reseller
Sales'[Sales Amount]), ALL('Reseller Sales'))
measure 'Reseller Sales'[Reseller Visual Total]=calculate(sum('Reseller
Sales'[Sales Amount]), ALLSELECTED())
measure 'Reseller Sales'[Reseller Visual Total for All of Calendar
Year]=calculate(sum('Reseller Sales'[Sales Amount]),
ALLSELECTED('Date'[Calendar Year]))
measure 'Reseller Sales'[Reseller Visual Total for All of Product Category
Name]=calculate(sum('Reseller Sales'[Sales Amount]), ALLSELECTED('Product
Category'[Product Category Name]))
evaluate
CalculateTable(
    //CT table expression
    summarize(
//summarize table expression
crossjoin(distinct('Product Category'[Product Category Name]),
distinct('Date'[Calendar Year]))
//First Group by expression
, 'Product Category'[Product Category Name]
//Second Group by expression
```

```
, 'Date'[Calendar Year]
//Summary expressions
, "Reseller Sales Amount", [Reseller Sales Amount]
, "Reseller Grand Total", [Reseller Grand Total]
, "Reseller Visual Total", [Reseller Visual Total]
, "Reseller Visual Total for All of Calendar Year", [Reseller Visual Total
for All of Calendar Year]
, "Reseller Visual Total for All of Product Category Name", [Reseller Visual
Total for All of Product Category Name]
)
//CT filters
, 'Sales Territory'[Sales Territory Group]="Europe", 'Promotion'[Promotion
Type]="Volume Discount"
)
order by [Product Category Name], [Calendar Year]
```

Après l'exécution de l'expression ci-dessus dans SQL Server Management Studio par rapport au modèle tabulaire AdventureWorks DW, vous obtenez les résultats suivants :

Agrandir le tableau

[Product Category Name]	[Calendar Year]	[Reseller Sales Amount]	[Reseller Grand Total]	[Reseller Visual Total]	[Reseller Visual Total for All of Calendar Year]	[Reseller Visual Total for All of Product Category Name]
Accessories	2000		80450596.9823	877006.7987	38786.018	
Accessories	2001		80450596.9823	877006.7987	38786.018	
Accessories	2002	625.7933	80450596.9823	877006.7987	38786.018	91495.3104
Accessories	2003	26037.3132	80450596.9823	877006.7987	38786.018	572927.0136
Accessories	2004	12122.9115	80450596.9823	877006.7987	38786.018	212584.4747
Accessories	2005		80450596.9823	877006.7987	38786.018	
Accessories	2006		80450596.9823	877006.7987	38786.018	
Bikes	2000		80450596.9823	877006.7987	689287.7939	
Bikes	2001		80450596.9823	877006.7987	689287.7939	
Bikes	2002	73778.938	80450596.9823	877006.7987	689287.7939	91495.3104
Bikes	2003	439771.4136	80450596.9823	877006.7987	689287.7939	572927.0136
Bikes	2004	175737.4423	80450596.9823	877006.7987	689287.7939	212584.4747

[Product Category Name]	[Calendar Year]	[Reseller Sales Amount]	[Reseller Grand Total]	[Reseller Visual Total]	[Reseller Visual Total for All of Calendar Year]	[Reseller Visual Total for All of Product Category Name]
Bikes	2005		80450596.9823	877006.7987	689287.7939	
Bikes	2006		80450596.9823	877006.7987	689287.7939	
Clothing	2000		80450596.9823	877006.7987	95090.7757	
Clothing	2001		80450596.9823	877006.7987	95090.7757	
Clothing	2002	12132.4334	80450596.9823	877006.7987	95090.7757	91495.3104
Clothing	2003	58234.2214	80450596.9823	877006.7987	95090.7757	572927.0136
Clothing	2004	24724.1209	80450596.9823	877006.7987	95090.7757	212584.4747
Clothing	2005		80450596.9823	877006.7987	95090.7757	
Clothing	2006		80450596.9823	877006.7987	95090.7757	
Components	2000		80450596.9823	877006.7987	53842.2111	
Components	2001		80450596.9823	877006.7987	53842.2111	
Components	2002	4958.1457	80450596.9823	877006.7987	53842.2111	91495.3104
Components	2003	48884.0654	80450596.9823	877006.7987	53842.2111	572927.0136
Components	2004		80450596.9823	877006.7987	53842.2111	212584.4747
Components	2005		80450596.9823	877006.7987	53842.2111	
Components	2006		80450596.9823	877006.7987	53842.2111	

Les colonnes du rapport sont les suivantes :

Reseller Sales Amount
Valeur réelle des ventes du revendeur pour l'année et la catégorie de produits. Cette valeur apparaît dans une cellule au centre de votre rapport, à l'intersection de l'année et de la catégorie.

Reseller Visual Total for All of Calendar Year (Valeur totale affichée du revendeur pour toutes les années civiles)
Valeur totale pour une catégorie de produits sur toutes les années. Cette valeur apparaît

à la fin d'une colonne ou d'une ligne d'une catégorie de produits donnée, pour toutes les années du rapport.

Reseller Visual Total for All of Product Category Name (Valeur totale affichée du revendeur pour tous les noms des catégories de produits)
Valeur totale sur une année pour toutes les catégories de produits. Cette valeur apparaît à la fin d'une colonne ou d'une ligne d'une année donnée, pour toutes les catégories de produits du rapport.

Reseller Visual Total (Valeur totale affichée du revendeur)
Valeur totale pour toutes les années et toutes les catégories de produits. Cette valeur figure généralement dans le coin inférieur droit de la table.

Reseller Grand Total (Total général du revendeur)
Il s'agit du total général de toutes les ventes du revendeur, avant l'application d'un filtre quelconque. Vous devez noter la différence avec [Reseller Visual Total]. N'oubliez pas que ce rapport comprend deux (2) filtres, l'un sur le groupe de catégories de produits et l'autre sur le type de promotion.

> **Notes**
>
> Si vous avez des filtres explicites dans votre expression, ils sont également appliqués à l'expression.

Commentaires

Cette page a-t-elle été utile ? Yes No

CALCULATE

Article • 20/10/2023

Évalue une expression dans un contexte de filtre modifié.

> ⓘ **Notes**
>
> Vous avez également la fonction **CALCULATETABLE**. Elle opère exactement de la même façon, sauf qu'elle modifie le **contexte de filtre** appliqué à une expression qui retourne un *objet de table*.

Syntaxe

DAX

```
CALCULATE(<expression>[, <filter1> [, <filter2> [, …]]])
```

Paramètres

Agrandir le tableau

Terme	Définition
expression	Expression à évaluer.
filter1, filter2, etc.	(Facultatif) Expressions booléennes ou de table qui définissent des filtres ou des fonctions de modification de filtre.

L'expression utilisée comme premier paramètre équivaut essentiellement à une mesure.

Exemples de filtres :

- Expressions de filtre booléennes
- Expressions de filtre de table
- Fonctions de modification de filtre

Lorsqu'il existe plusieurs filtres, ils peuvent être évalués avec l'opérateur logique AND (&&), ce qui signifie que toutes les conditions doivent être TRUE, ou avec l'opérateur logique OR (||), ce qui signifie que l'une des conditions peut être TRUE.

Expressions de filtre booléennes

Un filtre d'expression booléenne est une expression qui prend la valeur TRUE ou FALSE. Ces filtres doivent respecter plusieurs règles :

- Ils peuvent faire référence aux colonnes d'une seule table.
- Ils ne peuvent pas référencer des mesures.
- Ils ne peuvent pas utiliser une fonction CALCULATE imbriquée.

Depuis la version de septembre 2021 de Power BI Desktop, les conditions suivantes s'appliquent également :

- Ils ne peuvent pas utiliser les fonctions qui analysent ou retournent une table, sauf s'ils sont passés en tant qu'arguments aux fonctions d'agrégation.
- Ils *peuvent* contenir une fonction d'agrégation qui retourne une valeur scalaire. Par exemple,

```DAX
Total sales on the last selected date =
CALCULATE (
    SUM ( Sales[Sales Amount] ),
    'Sales'[OrderDateKey] = MAX ( 'Sales'[OrderDateKey] )
)
```

Expression de filtre de table

Un filtre d'expression de table applique un objet de table en tant que filtre. Vous pouvez avoir une référence à une table de modèle, mais il s'agit plus vraisemblablement d'une fonction qui retourne un objet de table. Vous pouvez utiliser la fonction FILTER pour appliquer des conditions de filtre complexes, comme celles qui ne peuvent pas être définies par une expression de filtre booléenne.

Fonctions de modification de filtre

Les fonctions de modification de filtre vous permettent d'effectuer des actions qui vont au-delà du simple ajout de filtres. Elles vous offrent davantage de contrôle lors de la modification du contexte de filtre.

Agrandir le tableau

Fonction	Objectif
REMOVEFILTERS	Supprimer tous les filtres ou ceux d'une ou plusieurs colonnes d'une table ou de toutes les colonnes d'une seule table.
ALL[1], ALLEXCEPT, ALLNOBLANKROW	Supprimer les filtres d'une ou plusieurs colonnes ou de toutes les colonnes d'une seule table.
KEEPFILTERS	Ajouter un filtre sans supprimer les filtres existants sur les mêmes colonnes.
USERELATIONSHIP	Engager une relation inactive entre des colonnes associées, auquel cas la relation active devient automatiquement inactive.
CROSSFILTER	Modifier la direction du filtre (des deux directions à une direction ou vice-versa) ou désactiver une relation.

[1] La fonction ALL et ses variantes se comportent à la fois comme des modificateurs de filtre et comme des fonctions qui retournent des objets de table. Si la fonction REMOVEFILTERS est prise en charge par votre outil, il est préférable de l'utiliser pour supprimer des filtres.

Valeur renvoyée

Valeur résultant de l'expression.

Notes

- Quand des expressions de filtre sont fournies, la fonction CALCULATE modifie le contexte de filtre pour évaluer l'expression. Pour chaque expression de filtre, il existe deux résultats standard possibles quand l'expression de filtre n'est pas wrappée dans la fonction KEEPFILTERS :
  - Si les colonnes (ou tables) ne sont pas dans le contexte du filtre, de nouveaux filtres sont ajoutés au contexte de filtre pour évaluer l'expression.
  - Si les colonnes (ou tables) sont déjà dans le contexte du filtre, les filtres existants sont remplacés par les nouveaux filtres pour évaluer l'expression CALCULATE.

- La fonction CALCULATE utilisée *sans filtres* répond à une exigence spécifique. Elle effectue la transition du contexte de ligne au contexte de filtre. Elle est obligatoire quand une expression (autre qu'une mesure de modèle) qui synthétise des données de modèle doit être évaluée dans le contexte de ligne. Ce scénario peut se produire dans une formule de colonne calculée ou lors de l'évaluation d'une

expression dans une fonction d'itérateur. Notez que lorsqu'une mesure de modèle est utilisée dans le contexte de ligne, la transition du contexte est automatique.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

La définition de mesure pour la table **Sales** suivante produit un résultat sous forme de chiffre d'affaires, mais uniquement pour les produits bleus.

Les exemples de cet article peuvent être utilisés avec l'exemple de modèle Adventure Works DW 2020 Power BI Desktop. Pour obtenir le modèle, consultez Exemple de modèle DAX.

DAX

```
Blue Revenue =
CALCULATE(
    SUM(Sales[Sales Amount]),
    'Product'[Color] = "Blue"
)
```

Agrandir le tableau

Category	Sales Amount	Chiffre d'affaires des produits bleus
Accessories	1 272 057,89 US$	165 406,62 US$
Bikes	94 620 526,21 US$	8 374 313,88 US$
Clothing	2 117 613,45 US$	259 488,37 US$
Composants	11 799 076,66 $	803 642,10 US$
Total	**109 809 274,20 US$**	**9 602 850,97 US$**

La fonction CALCULATE évalue la somme de la colonne **Sales Amount** de la table **Sales** dans un contexte de filtre modifié. Un nouveau filtre est ajouté à la colonne **Color** de la table **Product**, ou le filtre remplace tout filtre déjà appliqué à la colonne.

La définition de mesure pour la table **Sales** suivante produit le pourcentage des ventes par rapport au total de tous les canaux de vente.

Agrandir le tableau

Canal	Sales Amount	% des ventes par rapport au total
Internet	29 358 677,22 $	26,74 %
Reseller	80 450 596,98 $	73,26 %
Total	**109 809 274,20 US$**	**100,00 %**

DAX

```
Revenue % Total Channel =
DIVIDE(
    SUM(Sales[Sales Amount]),
    CALCULATE(
        SUM(Sales[Sales Amount]),
        REMOVEFILTERS('Sales Order'[Channel])
    )
)
```

La fonction DIVIDE divise une expression qui calcule la somme des valeurs de la colonne **Sales Amount** de la table **Sales** (dans le contexte de filtre) par la même expression dans un contexte de filtre modifié. C'est la fonction CALCULATE qui modifie le contexte de filtre en utilisant la fonction REMOVEFILTERS, qui est une fonction de modification de filtre. Elle supprime les filtres de la colonne **Channel** de la table **Sales Order**.

La définition de colonne calculée de la table **Customer** suivante classifie les clients dans une classe de fidélité. Ce scénario est très simple : Quand le chiffre d'affaires produit par le client est inférieur à 2 500 US$, il est classifié dans la classe *Low* ; sinon, il est classifié dans la classe *High*.

DAX

```
Customer Segment =
IF(
    CALCULATE(SUM(Sales[Sales Amount]), ALLEXCEPT(Customer,
Customer[CustomerKey])) < 2500,
    "Low",
    "High"
)
```

Dans cet exemple, le contexte de ligne est converti en contexte de filtre. On parle dans ce cas de *transition de contexte*. La fonction ALLEXCEPT supprime les filtres de toutes les colonnes de la table **Customer**, à l'exception de la colonne **CustomerKey**.

Contenu connexe

Contexte de filtre
Contexte de ligne
CALCULATETABLE, fonction
Fonctions de filtrage

Commentaires

Cette page a-t-elle été utile ? Yes No

CALCULATETABLE

Article • 20/10/2023

Évalue une expression de table dans un contexte de filtre modifié.

> ⓘ **Notes**
>
> Vous avez également la fonction **CALCULATE**. Elle opère exactement de la même façon, sauf qu'elle modifie le **contexte de filtre** appliqué à une expression qui retourne une *valeur scalaire*.

Syntaxe

DAX

```
CALCULATETABLE(<expression>[, <filter1> [, <filter2> [, …]]])
```

Paramètres

Agrandir le tableau

Terme	Définition
expression	Expression de table à évaluer.
filter1, filter2, etc.	(Facultatif) Expressions booléennes ou de table qui définissent des filtres ou des fonctions de modification de filtre.

L'expression utilisée comme premier paramètre doit être un modèle de table ou une fonction qui retourne une table.

Exemples de filtres :

- Expressions de filtre booléennes
- Expressions de filtre de table
- Fonctions de modification de filtre

Si vous avez plusieurs filtres, ils sont évalués à l'aide de l'opérateur logique AND. Cela signifie que toutes les conditions doivent avoir la valeur TRUE en même temps.

Expressions de filtre booléennes

Un filtre d'expression booléenne est une expression qui prend la valeur TRUE ou FALSE. Ces filtres doivent respecter plusieurs règles :

- Ils ne peuvent référencer qu'une seule colonne.
- Ils ne peuvent pas référencer des mesures.
- Ils ne peuvent pas utiliser une fonction CALCULATE imbriquée.

Depuis la version de septembre 2021 de Power BI Desktop, les conditions suivantes s'appliquent également :

- Ils ne peuvent pas utiliser les fonctions qui analysent ou retournent une table, sauf s'ils sont passés en tant qu'arguments aux fonctions d'agrégation.
- Ils *peuvent* contenir une fonction d'agrégation qui retourne une valeur scalaire.

Expression de filtre de table

Un filtre d'expression de table applique un objet de table en tant que filtre. Vous pouvez avoir une référence à une table de modèle, mais il s'agit plus vraisemblablement d'une fonction qui retourne un objet de table. Vous pouvez utiliser la fonction FILTER pour appliquer des conditions de filtre complexes, comme celles qui ne peuvent pas être définies par une expression de filtre booléenne.

Fonctions de modification de filtre

Les fonctions de modification de filtre vous permettent d'effectuer des actions qui vont au-delà du simple ajout de filtres. Elles vous offrent davantage de contrôle lors de la modification du contexte de filtre.

Agrandir le tableau

Fonction	Objectif
REMOVEFILTERS	Supprimer tous les filtres ou ceux d'une ou plusieurs colonnes d'une table ou de toutes les colonnes d'une seule table.
ALL[1], ALLEXCEPT, ALLNOBLANKROW	Supprimer les filtres d'une ou plusieurs colonnes ou de toutes les colonnes d'une seule table.
KEEPFILTERS	Ajouter un filtre sans supprimer les filtres existants sur les mêmes colonnes.
USERELATIONSHIP	Engager une relation inactive entre des colonnes associées, auquel cas la relation active devient automatiquement inactive.

Fonction	Objectif
CROSSFILTER	Modifier la direction du filtre (des deux directions à une direction ou vice-versa) ou désactiver une relation.

[1] La fonction ALL et ses variantes se comportent à la fois comme des modificateurs de filtre et comme des fonctions qui retournent des objets de table. Si la fonction REMOVEFILTERS est prise en charge par votre outil, il est préférable de l'utiliser pour supprimer des filtres.

Valeur renvoyée

Table de valeurs.

Notes

- Quand des expressions de filtre sont fournies, la fonction CALCULATETABLE modifie le contexte de filtre pour évaluer l'expression. Pour chaque expression de filtre, il existe deux résultats standard possibles quand l'expression de filtre n'est pas wrappée dans la fonction KEEPFILTERS :
  - Si les colonnes (ou tables) ne sont pas dans le contexte du filtre, de nouveaux filtres sont ajoutés au contexte de filtre pour évaluer l'expression.
  - Si les colonnes (ou tables) sont déjà dans le contexte du filtre, les filtres existants sont remplacés par les nouveaux filtres pour évaluer l'expression CALCULATETABLE.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant utilise la fonction CALCULATETABLE pour obtenir la somme des ventes Internet pour 2006. Cette valeur sera utilisée ultérieurement pour calculer le rapport entre les ventes Internet et l'ensemble des ventes sur l'année 2006.

La formule suivante :

DAX

```
= SUMX(
    CALCULATETABLE(
```

```
        'InternetSales_USD',
        'DateTime'[CalendarYear] = 2006
    ),
    [SalesAmount_USD]
)
```

Le tableau suivant est obtenu :

Agrandir le tableau

Étiquettes de ligne	Montant des ventes Internet (USD)	CalculateTable - Ventes Internet en 2006	Rapport entre les ventes Internet et les ventes sur 2006
2005	$2,627,031.40	$5,681,440.58	0.46
2006	$5,681,440.58	$5,681,440.58	1.00
2007	$8,705,066.67	$5,681,440.58	1,53
2008	$9,041,288.80	$5,681,440.58	1.59
Total général	26 054 827,45 $	$5,681,440.58	4.59

Contenu connexe

- Contexte de filtre
- CALCULATE, fonction (DAX)
- Fonctions de filtrage (DAX)

Commentaires

Cette page a-t-elle été utile ? Yes No

EARLIER

Article • 20/10/2023

Retourne la valeur actuelle de la colonne spécifiée dans un test d'évaluation externe de la colonne mentionnée.

EARLIER est utile pour les calculs imbriqués où vous souhaitez utiliser une certaine valeur comme entrée et produire des calculs basés sur cette entrée. Dans Microsoft Excel, vous ne pouvez effectuer de tels calculs que dans le contexte de la ligne actuelle. Toutefois, dans DAX, vous pouvez stocker la valeur de l'entrée, puis effectuer le calcul à l'aide des données de la table entière.

EARLIER est principalement utilisée dans le contexte des colonnes calculées.

Syntaxe

DAX

```
EARLIER(<column>, <number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne ou expression qui correspond à une colonne.
num	(Facultatif) Nombre positif pour le test d'évaluation externe. Le niveau d'évaluation externe suivant est représenté par 1 ; deux niveaux externes sont représentés par 2, et ainsi de suite. En cas d'omission, la valeur par défaut est 1.

Valeur renvoyée

Valeur actuelle de la ligne, à partir de **column**, au niveau **number** des tests d'évaluations externes.

Exceptions

Description des erreurs

Remarques

- **EARLIER** réussit s'il existe un contexte de ligne avant le début de l'analyse de table. Sinon, elle retourne une erreur.
- Les performances de la fonction **EARLIER** peuvent être lentes, car elle peut avoir théoriquement à effectuer un nombre d'opérations proche du nombre total de lignes (dans la colonne) multiplié par le même nombre (en fonction de la syntaxe de l'expression). Par exemple, si vous avez 10 lignes dans la colonne, environ 100 opérations peuvent être nécessaires. Si vous avez 100 lignes, près de 10 000 opérations peuvent être effectuées.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

> **Notes**
>
> Dans la pratique, le moteur d'analytique en mémoire VertiPaq effectue des optimisations pour réduire le nombre réel de calculs, mais vous devez être prudent lors de la création de formules qui impliquent la récursivité.

Exemple

Pour illustrer l'utilisation de la fonction EARLIER, il est nécessaire de générer un scénario qui calcule une valeur de classement, puis utilise cette valeur de classement dans d'autres calculs.

L'exemple suivant est basé sur cette table simple, **ProductSubcategory**, qui affiche le total des ventes pour chaque ProductSubcategory.

La table finale, dont la colonne de classement, est présentée ici.

Agrandir le tableau

ProductSubcategoryKey	EnglishProductSubcategoryName	TotalSubcategorySales	SubcategoryRanking
18	Cuissards	156 167,88 $	18
26	Porte-vélos	220 720,70 $	14
27	Supports à vélos	35 628,69 $	30
28	Bidons et porte-bidons	59 342,43 $	24
5	Jeux de pédalier	48 643,47 $	27
6	Freins	62 113,16 $	23
19	Capuchons	47 934,54 $	28

ProductSubcategoryKey	EnglishProductSubcategoryName	TotalSubcategorySales	SubcategoryRanking
7	Chaînes	8 847,08 $	35
29	Produits d’entretien	16 882,62 $	32
8	Pédaliers	191 522,09 $	15
9	Dérailleurs	64 965,33 $	22
30	Garde-boue	41 974,10 $	29
10	Fourches	74 727,66 $	21
20	Gants	228 353,58 $	12
4	Guidons	163 257,06 $	17
11	Oreillettes	57 659,99 $	25
31	Casques	451 192,31 $	9
32	Sacs à eau	96 893,78 $	20
21	Maillots	699 429,78 $	7
33	Lumières		36
34	Verrous	15 059,47 $	33
1	VTT	34 305 864,29 $	2
12	Cadres VTT	4 511 170,68 $	4
35	Sacoches		36
13	Pédales	140 422,20 $	19
36	Pompes	12 695,18 $	34
2	Vélos de route	40 551 696,34 $	1
14	Cadres de cyclisme	3 636 398,71 $	5
15	Selles	52 526,47 $	26
22	Shorts	385 707,80 $	10
23	Chaussettes	28 337,85 $	31
24	Collants	189 179,37 $	16
37	Pneus et chambres à air	224 832,81 $	13
3	Vélos de tourisme	13 334 864,18 $	3
16	Cadres de tourisme	1 545 344,02 $	6

ProductSubcategoryKey	EnglishProductSubcategoryName	TotalSubcategorySales	SubcategoryRanking
25	Gilets	240 990,04 $	11
17	Roues	648 240,04 $	8

Création d'une valeur de classement

Une façon d'obtenir une valeur de classement pour une valeur donnée dans une ligne est de compter le nombre de lignes, dans la même table, qui ont une valeur supérieure (ou inférieure) à celle qui est comparée. Cette technique retourne une valeur vide ou nulle pour la valeur la plus élevée dans la table, alors que les valeurs égales auront la même valeur de classement et la valeur suivante (après les valeurs égales) aura une valeur de classement non consécutive. Consultez l'exemple ci-dessous.

Une nouvelle colonne calculée, **SubCategorySalesRanking**, est créée à l'aide de la formule suivante.

DAX

```
= COUNTROWS(FILTER(ProductSubcategory,
EARLIER(ProductSubcategory[TotalSubcategorySales])
<ProductSubcategory[TotalSubcategorySales]))+1
```

Les étapes suivantes décrivent plus en détail la méthode de calcul.

1. La fonction **EARLIER** obtient la valeur de *TotalSubcategorySales* pour la ligne actuelle dans la table. Dans ce cas, comme le processus démarre, il s'agit de la première ligne de la table

2. **EARLIER**([*TotalSubcategorySales*]) prend la valeur 156 167,88 $, la ligne actuelle dans la boucle externe.

3. La fonction **FILTER** retourne désormais une table dans laquelle toutes les lignes ont une valeur pour *TotalSubcategorySales* supérieure à 156 167,88 $ (qui est la valeur actuelle pour **EARLIER**).

4. La fonction **COUNTROWS** compte les lignes de la table filtrée et assigne cette valeur à la nouvelle colonne calculée dans la ligne actuelle plus 1. L'ajout de 1 est nécessaire pour éviter que la valeur la mieux classée ne devienne un espace.

5. La formule de colonne calculée passe à la ligne suivante et répète les étapes 1 à 4. Ces étapes sont répétées jusqu'à ce que la fin de la table soit atteinte.

La fonction **EARLIER** obtiendra toujours la valeur de la colonne avant l'opération de table actuelle. Si vous avez besoin d'obtenir une valeur de la boucle avant cela, définissez le deuxième argument sur 2.

Contenu connexe

EARLIEST, fonction
Fonctions de filtrage

Commentaires

Cette page a-t-elle été utile ? Yes No

EARLIEST

Article • 14/12/2023

Retourne la valeur actuelle de la colonne spécifiée dans un test d'évaluation externe de la colonne mentionnée.

Syntaxe

DAX

```
EARLIEST(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Référence à une colonne.

Valeur renvoyée

Colonne dont les filtres ont été supprimés.

Notes

- La fonction EARLIEST est similaire à EARLIER, mais vous permet de spécifier un niveau supplémentaire de récursivité.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de données actuel ne prend pas en charge ce scénario.

DAX

```
= EARLIEST(<column>)
```

Contenu connexe

EARLIER, fonction
Fonctions de filtrage

Commentaires

Cette page a-t-elle été utile ? Yes No

FILTER

Article • 20/10/2023

Retourne une table qui représente un sous-ensemble d'une autre table ou expression.

Syntaxe

DAX

```
FILTER(<table>,<filter>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table à filtrer. La table peut également être une expression produisant une table.
filter	Expression booléenne à évaluer pour chaque ligne de la table. Par exemple, `[Amount] > 0` ou `[Region] = "France"`.

Valeur de retour

Table contenant uniquement les lignes filtrées.

Remarques

- Vous pouvez utiliser FILTER pour réduire le nombre de lignes dans la table avec laquelle vous travaillez et utiliser uniquement des données spécifiques dans les calculs. La fonction FILTER n'étant pas indépendante, elle doit être utilisée en tant que fonction incorporée dans d'autres fonctions nécessitant une table comme argument.
- Pour connaître les meilleures pratiques lors de l'utilisation d'un filtre, consultez Éviter d'utiliser le filtre en tant qu'argument de filtre.

Utiliser COUNTROWS à la place de COUNT dans DAX

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant crée un rapport des ventes sur Internet hors États-Unis à l'aide d'une mesure qui exclut par filtrage les ventes réalisées aux États-Unis, puis découpe les ventes par année civile et catégorie de produit. Pour créer cette mesure, filtrez la table InternetSales_USD à l'aide du secteur de vente (SalesTerritory), puis utilisez la table filtrée dans une fonction SUMX.

Dans cet exemple, l'expression :

DAX

```
FILTER('InternetSales_USD', RELATED('SalesTerritory'[SalesTerritoryCountry])
<>"United States")
```

Retourne une table qui est un sous-ensemble des ventes sur Internet, moins toutes les lignes appartenant au secteur de vente États-Unis. La fonction RELATED est ce qui lie la clé Territory dans la table InternetSales à SalesTerritoryCountry dans la table SalesTerritory.

La table suivante démontre le bien-fondé de la mesure Ventes sur Internet hors États-Unis dont la formule est indiquée dans la section de code ci-dessous. La table compare toutes les ventes sur Internet aux ventes sur Internet hors États-Unis pour montrer que l'expression de filtre fonctionne (les ventes des États-Unis sont exclues du calcul).

Pour recréer cette table, ajoutez le champ SalesTerritoryCountry à la zone **Étiquettes de ligne** d'un rapport ou tableau croisé dynamique.

Tableau 1. Comparaison des ventes totales réalisées aux États-Unis et dans toutes les autres régions

Agrandir le tableau

Étiquettes de ligne	Internet Sales	Ventes sur Internet hors États-Unis
Australie	4 999 021,84 $	4 999 021,84 $
Canada	1,343,109.10 $	1,343,109.10 $

Étiquettes de ligne	Internet Sales	Ventes sur Internet hors États-Unis
France	2 490 944,57 $	2 490 944,57 $
Allemagne	2 775 195,60 $	2 775 195,60 $
Royaume-Uni	5 057 076,55 $	5 057 076,55 $
États-Unis	9 389 479,79 $	
Total général	26 054 827,45 $	16 665 347,67 $

La table du rapport final montre les résultats quand vous créez un tableau croisé dynamique à l'aide de la mesure Ventes sur Internet hors États-Unis. Ajoutez le champ CalendarYear à la zone **Étiquettes de ligne** du tableau croisé dynamique, puis ajoutez le champ ProductCategoryName à la zone **Étiquettes de colonne**.

Tableau 2. Comparaison des ventes hors États-Unis par catégorie de produit

Agrandir le tableau

Étiquettes de ligne	Accessories	Bikes	Clothing	Total général
2005		1 526 481,95 $		1 526 481,95 $
2006		3 554 744,04 $		3 554 744,04 $
2007	156 480,18 $	5 640 106,05 $	70 142,77 $	5 866 729,00 $
2008	228 159,45 $	5 386 558,19 $	102 675,04 $	5 717 392,68 $
Total général	384 639,63 $	16 107 890,23 $	172 817,81 $	16 665 347,67 $

DAX

```
SUMX(FILTER('InternetSales_USD',
RELATED('SalesTerritory'[SalesTerritoryCountry])<>"United States")
     ,'InternetSales_USD'[SalesAmount_USD])
```

Contenu connexe

Fonctions de filtrage
ALL, fonction
ALLEXCEPT, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

INDEX

Article • 22/02/2024

Récupère une ligne à une position absolue, spécifiée par le paramètre de position, dans la partition spécifiée triée selon l'ordre spécifié. Si la partition actuelle ne peut pas être déduite en une seule partition, plusieurs lignes peuvent être retournées.

Syntaxe

DAX

```
INDEX(<position>[, <relation> or <axis>][, <orderBy>][, <blanks>][,
<partitionBy>][, <matchBy>][, <reset>] )
```

Paramètres

Agrandir le tableau

Terme	Définition
position	La position absolue (basée sur 1) à partir de laquelle obtenir les données : - <position> est positive : 1 est la première ligne, 2 est la deuxième ligne, etc. - <position> est négative : -1 est la dernière ligne, -2 est l'avant-dernière ligne, etc. Lorsque <position> est hors limite, zéro ou BLANK(), INDEX renvoie un tableau vide. Peut être toute expression DAX qui retourne une valeur scalaire.
relation	(Facultatif) Expression de table à partir de laquelle la sortie est retournée Si la valeur est spécifiée, toutes les colonnes de <partitionBy> doivent provenir de celle-ci ou d'une table associée. En cas d'omission : - <orderBy> doit être spécifié explicitement. - Toutes les expressions <orderBy> et <partitionBy> doivent être des noms complets de colonnes et provenir d'une table unique. - La valeur par défaut est ALLSELECTED() de toutes les colonnes dans <orderBy> et <partitionBy>.
axis	(Facultatif) Axe de la forme du visuel. Disponible seulement dans les calculs de visuel et remplace <relation>.
orderBy	(Facultatif) Clause ORDERBY() contenant les expressions qui définissent le tri de chaque partition. En cas d'omission : - la <relation> doit être spécifiée explicitement. - Par défaut, le tri est effectué par chaque colonne dans une <relation> qui n'est pas déjà spécifiée dans <partitionBy>.

Terme	Définition
vides	(Facultatif) Énumération qui définit comment gérer des valeurs vides lors du tri. Ce paramètre est réservé à un usage futur. Actuellement, la seule valeur prise en charge est DEFAULT, où le comportement pour les valeurs numériques est que les valeurs vides sont classées entre les valeurs zéro et les valeurs négatives. Le comportement des chaînes est que les valeurs vides sont triées avant toutes les chaînes, y compris les chaînes vides.
partitionBy	(Facultatif) Clause PARTITIONBY() contenant les colonnes qui définissent la façon dont la <relation> est partitionnée. En cas d'omission, la <relation> est traitée comme une partition unique.
matchBy	(Facultatif) Clause MATCHBY() contenant les colonnes qui définissent la manière de faire correspondre les données et d'identifier la ligne actuelle.
reset	(Facultatif) Disponible seulement dans les calculs de visuel. Indique si le calcul est réinitialisé et à quel niveau de la hiérarchie des colonnes de la forme du visuel. Les valeurs acceptées sont : NONE, LOWESTPARENT, HIGHESTPARENT ou un entier. Le comportement dépend du signe de l'entier : - S'il est égal à zéro ou s'il est omis, le calcul n'est pas réinitialisé. Équivalent à NONE. - S'il est positif, l'entier identifie la colonne en commençant par la plus élevée, indépendamment du fragment. HIGHESTPARENT équivaut à 1. - S'il est négatif, l'entier identifie la colonne en commençant par la plus basse, par rapport au fragment actuel. LOWESTPARENT équivaut à -1.

Valeur retournée

Une ligne à une position absolue.

Notes

Chaque colonne <partitionBy> et <matchBy> doit avoir une valeur externe correspondante permettant de définir la « partition actuelle » à traiter, avec le comportement suivant :

- S'il existe exactement une colonne externe correspondante, sa valeur est utilisée.
- S'il n'existe aucune colonne externe correspondante :
  - INDEX détermine d'abord toutes les colonnes <partitionBy> et <matchBy> qui n'ont aucune colonne externe correspondante.
  - Pour chaque combinaison de valeurs existantes pour ces colonnes dans le contexte parent d'INDEX, INDEX est évalué, et une ligne est retournée.
  - La sortie finale d'INDEX est une union de ces lignes.
- S'il existe plusieurs colonnes externes correspondantes, une erreur est retournée.

Si <matchBy> est présent, INDEX essaie d'utiliser les colonnes <matchBy> et <partitionBy> pour identifier la ligne.
Si <matchBy> n'est pas présent et que les colonnes spécifiées dans <orderBy> et <partitionBy> ne peuvent pas identifier de manière unique chaque ligne dans la <relation> :

- INDEX essaie de trouver le moins de colonnes supplémentaires requises pour identifier chaque ligne de manière unique.
- Si de telles colonnes sont trouvées, INDEX ajoute automatiquement ces nouvelles colonnes à <orderBy>, et chaque partition est triée à l'aide de ce nouvel ensemble de colonnes OrderBy.
- Si ces colonnes sont introuvables, une erreur est retournée.

Une table vide est retournée si :

- La valeur externe correspondante d'une colonne PartitionBy n'existe pas dans la <relation>.
- La valeur <position> fait référence à une position qui n'existe pas dans la partition.

Si INDEX est utilisé dans une colonne calculée définie sur la même table que <relation> et que <orderBy> est omis, une erreur est retournée.

<reset> peut être utilisé seulement dans les calculs de visuel, et ne peut pas être utilisé en combinaison avec <orderBy> ou <partitionBy>. Si <reset> est présent, <axis> peut être spécifié, mais pas <relation>.

Exemple 1 : colonne calculée

La requête DAX suivante :

DAX

```
EVALUATE INDEX(1, ALL(DimDate[CalendarYear]))
```

retourne la table suivante :

Agrandir le tableau

DimDate[CalendarYear]
2005

Exemple 2 : colonne calculée

La requête DAX suivante :

```DAX
EVALUATE
SUMMARIZECOLUMNS (
    FactInternetSales[ProductKey],
    DimDate[MonthNumberOfYear],
    FILTER (
            VALUES(FactInternetSales[ProductKey]),
            [ProductKey] < 222
    ),
    "CurrentSales", SUM(FactInternetSales[SalesAmount]),
    "LastMonthSales",
    CALCULATE (
        SUM(FactInternetSales[SalesAmount]),
        INDEX(-1, ORDERBY(DimDate[MonthNumberOfYear]))
    )
)
ORDER BY [ProductKey], [MonthNumberOfYear]
```

retourne la table suivante :

Agrandir le tableau

FactInternetSales[ProductKey]	DimDate[MonthNumberOfYear]	[CurrentSales]	[LastMonthSales]
214	1	5423,45	8047,7
214	2	4968,58	8047,7
214	3	5598,4	8047,7
214	4	5073,55	8047,7
214	5	5248,5	8047,7
214	6	7487,86	8047,7
214	7	7382,89	8047,7
214	8	6543,13	8047,7
214	9	6788,06	8047,7
214	10	6858,04	8047,7
214	11	8607,54	8047,7
214	12	8047,7	8047,7
217	1	5353,47	7767,78

FactInternetSales[ProductKey]	DimDate[MonthNumberOfYear]	[CurrentSales]	[LastMonthSales]
217	2	4268,78	7767,78
217	3	5773,35	7767,78
217	4	5738,36	7767,78
217	5	6158,24	7767,78
217	6	6998	7767,78
217	7	5563,41	7767,78
217	8	5913,31	7767,78
217	9	5913,31	7767,78
217	10	6823,05	7767,78
217	11	6683,09	7767,78
217	12	7767,78	7767,78

Exemple 3 : calcul de visuel

Les requêtes DAX de calcul de visuel suivantes :

DAX

```
SalesComparedToBeginningOfYear = [SalesAmount] -
CALCULATE(SUM([SalesAmount]), INDEX(1, ROWS, HIGHESTPARENT))

SalesComparedToBeginningOfQuarter = [SalesAmount] -
CALCULATE(SUM([SalesAmount]), INDEX(1, , -1))
```

Améliorez une table afin qu'elle contienne pour chaque mois :
- le montant total des ventes ;
- la différence avec le premier mois de l'année respective ;
- et la différence avec le premier mois du trimestre respectif.

La capture d'écran ci-dessous montre la matrice du visuel et l'expression du premier calcul de visuel :

```
1 SalesComparedToBeginningOfYear = [SalesAmount] - CALCULATE(SUM([SalesAmount]), INDEX(1, ROWS, HIGHESTPARENT))
```

CalendarYear	CalendarQuarter	MonthNumberOfYear	SalesAmount	SalesComparedToBeginningOfYear	SalesComparedToBeginningOfQuarter
2011	1	1	$323,743.2056	0.00	0.00
		2	$437,372.4476	113,629.24	113,629.24
		3	$514,918.2284	191,175.02	191,175.02
		Total	**$1,276,033.8816**	**0.00**	**0.00**
	2	4	$523,950.0258	200,206.82	0.00
		5	$539,469.0004	215,725.79	15,518.97
		6	$647,524.214	323,781.01	123,574.19

Contenu connexe

DÉCALAGE
ORDERBY
PARTITIONBY
WINDOW
RANK
ROWNUMBER

Commentaires

Cette page a-t-elle été utile ? Yes No

KEEPFILTERS

Article • 20/10/2023

Modifie la façon dont les filtres sont appliqués quand une fonction CALCULATE ou CALCULATETABLE est évaluée.

Syntaxe

DAX

```
KEEPFILTERS(<expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
expression	Toute expression.

Valeur de retour

Table de valeurs.

Remarques

- Vous utilisez KEEPFILTERS dans les fonctions CALCULATE et CALCULATETABLE portant sur le contexte pour remplacer le comportement standard de ces fonctions.
- Par défaut, les arguments de filtre dans des fonctions telles que CALCULATE sont utilisés comme contexte pour l'évaluation de l'expression. Ainsi, les arguments de filtre pour la fonction CALCULATE remplacent tous les filtres existants sur les mêmes colonnes. Le nouveau contexte affecté par l'argument de filtre pour la fonction CALCULATE affecte uniquement les filtres existants sur les colonnes mentionnées dans le cadre de l'argument de filtre. Les filtres sur des colonnes autres que celles mentionnées dans les arguments de la fonction CALCULATE ou d'autres fonctions associées continuent de s'appliquer sans modification.

- La fonction KEEPFILTERS vous permet de modifier ce comportement. Quand vous utilisez KEEPFILTERS, tous les filtres existants dans le contexte actuel sont comparés aux colonnes dans les arguments de filtre, et l'intersection de ces arguments est utilisée comme contexte pour l'évaluation de l'expression. L'effet net sur une colonne, quelle qu'elle soit, est le suivant : les deux jeux d'arguments s'appliquent, à savoir les arguments de filtre utilisés dans la fonction CALCULATE et ceux de la fonction KEEPFILTERS. En d'autres termes, alors que les filtres CALCULATE remplacent le contexte actuel, KEEPFILTERS ajoute des filtres au contexte actuel.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant vous explique, pour différents scénarios courants, comment utiliser la fonction KEEPFILTERS dans le cadre d'une formule CALCULATE ou CALCULATETABLE.

Les trois premières expressions permettent d'obtenir des données simples, qui seront utilisées à des fins de comparaison :

- Ventes Internet pour l'État de Washington.

- Ventes Internet pour les États de Washington et de l'Oregon (combinés).

- Ventes Internet pour l'État de Washington et la province de Colombie-Britannique (combinés).

La quatrième expression calcule les ventes Internet pour les États de Washington et de l'Oregon avec application du filtre pour Washington et la Colombie-Britannique.

L'expression suivante calcule les ventes Internet pour les États de Washington et de l'Oregon et utilise la fonction KEEPFILTERS. Le filtre pour Washington et la Colombie-Britannique fait partie du contexte précédent.

DAX

```
EVALUATE ROW(
  "$$ in WA"
    , CALCULATE('Internet Sales'[Internet Total Sales]
                , 'Geography'[State Province Code]="WA"
      )
, "$$ in WA and OR"
    , CALCULATE('Internet Sales'[Internet Total Sales]
                , 'Geography'[State Province Code]="WA"
                  || 'Geography'[State Province Code]="OR"
```

```
      )
, "$$ in WA and BC"
    , CALCULATE('Internet Sales'[Internet Total Sales]
                , 'Geography'[State Province Code]="WA"
                  || 'Geography'[State Province Code]="BC"
      )
, "$$ in WA and OR ??"
    , CALCULATE(
          CALCULATE('Internet Sales'[Internet Total Sales]
                    ,'Geography'[State Province Code]="WA"
                      || 'Geography'[State Province Code]="OR"
          )
          , 'Geography'[State Province Code]="WA"
            || 'Geography'[State Province Code]="BC"
      )
, "$$ in WA !!"
    , CALCULATE(
          CALCULATE('Internet Sales'[Internet Total Sales]
                  , KEEPFILTERS('Geography'[State Province Code]="WA"
                                || 'Geography'[State Province Code]="OR"
                    )
          )
          , 'Geography'[State Province Code]="WA"
            || 'Geography'[State Province Code]="BC"
      )
)
```

Quand cette expression est évaluée par rapport à l'exemple de base de données AdventureWorks DW, les résultats suivants sont obtenus.

Agrandir le tableau

Colonne	Valeur
[$$ in WA]	$ 2,467,248.34
[$$ in WA and OR]	$ 3,638,239.88
[$$ in WA and BC]	$ 4,422,588.44
[$$ in WA and OR ??]	$ 3,638,239.88
[$$ in WA !!]	$ 2,467,248.34

> ⓘ **Notes**
>
> À des fins pédagogiques, les résultats ci-dessus ont été mis en forme dans une table et non sur une ligne unique.

Tout d'abord, examinez l'expression **[$$ in WA and OR ??]** . Vous vous demandez peut-être comment cette formule peut retourner la valeur des ventes dans les États de Washington et de l'Oregon puisque l'expression CALCULATE externe comprend un filtre pour Washington et la Colombie-Britannique. En fait, par défaut, la fonction CALCULATE remplace les filtres externes dans 'Geography'[State Province Code] et applique ses propres arguments de filtre, car les filtres s'appliquent à la même colonne.

Examinez à présent l'expression **[$$ in WA !!]** . Vous vous demandez peut-être comment cette formule peut retourner la valeur des ventes dans l'État de Washington uniquement puisque l'argument de filtre inclut l'Oregon et que l'expression CALCULATE externe comprend un filtre pour Washington et la Colombie-Britannique. En fait, la fonction KEEPFILTERS modifie le comportement par défaut de l'expression CALCULATE et ajoute un filtre supplémentaire. Étant donné que l'intersection des filtres est utilisée, le filtre externe **'Geography'[State Province Code]="WA" || 'Geography'[State Province Code]="BC")** est ajouté à l'argument de filtre **'Geography'[State Province Code]="WA" || 'Geography'[State Province Code]="OR"** . Étant donné que les deux filtres s'appliquent à la même colonne, c'est le filtre **'Geography'[State Province Code]="WA"** obtenu qui est appliqué quand l'expression est évaluée.

Contenu connexe

Fonctions de filtrage
CALCULATE, fonction
CALCULATETABLE, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

LOOKUPVALUE

Article • 20/10/2023

Retourne la valeur pour la ligne qui remplit tous les critères spécifiés par une ou plusieurs conditions de recherche.

Syntaxe

DAX

```
LOOKUPVALUE (
    <result_columnName>,
    <search_columnName>,
    <search_value>
    [, <search2_columnName>, <search2_value>]…
    [, <alternateResult>]
)
```

Paramètres

Agrandir le tableau

Terme	Définition
result_columnName	Nom d'une colonne existante qui contient la valeur que vous voulez retourner. Il ne peut pas s'agir d'une expression.
search_columnName	Nom d'une colonne existante. Elle peut se trouver dans la même table que result_columnName ou dans une table associée. Il ne peut pas s'agir d'une expression. Plusieurs paires de search_columnName et de search_value peuvent être spécifiées.
search_value	Valeur à rechercher dans search_columnName. Plusieurs paires de search_columnName et de search_value peuvent être spécifiées.
alternateResult	(Facultatif) Valeur retournée quand le contexte de result_columnName a été filtré sur zéro ou plusieurs valeurs distinctes. Si elle n'est pas spécifiée, la fonction retourne BLANK lorsque result_columnName est filtrée à zéro valeur ou une erreur lorsqu'il existe plusieurs valeurs distinctes dans le contexte pour result_columnName.

Valeur de retour

Valeur de **result_column** sur la ligne où toutes les paires de **search_column** et **search_value** ont une correspondance exacte.

En l'absence de résultat qui correspond à toutes les valeurs de recherche, la valeur BLANK ou **alternateResult** (si elle est fourni) est retournée. En d'autres termes, la fonction ne retourne pas de valeur de recherche si seuls certains critères sont remplis.

Si plusieurs lignes correspondent aux valeurs de recherche et que les valeurs du **result_columnName** pour ces lignes sont identiques, cette valeur est retournée. Cependant, si **result_column** retourne des valeurs différentes, une erreur ou **alternateResult** (s'il est fourni) est retourné.

Remarques

- S'il existe une relation entre les tables qui contiennent la colonne de résultat et les tables qui contiennent les colonnes de recherche, dans la plupart des cas, l'utilisation de la fonction RELATED au lieu de LOOKUPVALUE est plus efficace et offre de meilleures performances.
- Plusieurs paires de **search_columnName** et de **search_value** peuvent être spécifiées.
- Les paramètres **search_value** et **alternateResult** sont évalués avant l'itération de la fonction dans les lignes de la table de recherche.
- Évitez d'utiliser les fonctions ISERROR ou IFERROR pour capturer une erreur retournée par LOOKUPVALUE. Si certaines entrées de la fonction provoquent une erreur quand une seule valeur de sortie ne peut pas être déterminée, le fait de fournir un paramètre **alternateResult** est le moyen le plus fiable et le plus performant pour gérer l'erreur.
- Le paramètre **alternateResult** retourne une erreur s'il est spécifié dans une colonne calculée PowerPivot.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple 1

Dans cet exemple, LOOKUPVALUE est utilisé pour rechercher le taux moyen de la devise utilisée pour payer la commande le jour où la commande a été passée :

DAX

```
Exchange Rate =
LOOKUPVALUE (
    'Currency Rate'[Average Rate],
    'Currency Rate'[CurrencyKey], [CurrencyKey],
    'Currency Rate'[DateKey], [OrderDateKey]
)
```

La date de commande et la devise sont requises pour trouver le taux moyen pour la date et la devise appropriées. OrderDateKey et CurrencyKey sont les clés utilisées pour rechercher le taux moyen dans la table Taux monétaire.

Vous pouvez utiliser le taux de change pour calculer le montant des ventes en devise locale avec :

DAX

```
Sales Amount Local Currency = [Sales Amount] * [Exchange Rate]
```

Exemple 2

Dans cet exemple, la colonne calculée suivante définie dans la table **Ventes** utilise la fonction LOOKUPVALUE pour retourner les valeurs de canal de la table **Commande client**.

DAX

```
CHANNEL =
LOOKUPVALUE (
    'Sales Order'[Channel],
    'Sales Order'[SalesOrderLineKey],
    [SalesOrderLineKey]
)
```

Toutefois, dans le cas présent, étant donné qu'il existe une relation entre les tables **Commande client** et **Ventes**, il est plus efficace d'utiliser la fonction RELATED.

DAX

```
CHANNEL = RELATED('Sales Order'[Channel])
```

Contenu connexe

RELATED, fonction (DAX)
Fonctions d'information

Commentaires

Cette page a-t-elle été utile ? Yes No

MATCHBY

Article • 14/12/2023

Dans les fonctions de fenêtre, définit les colonnes utilisées pour déterminer la manière de faire correspondre des données et d'identifier la ligne actuelle.

Syntaxe

DAX

```
MATCHBY ( [<matchBy_columnName>[, matchBy_columnName [, …]]] )
```

Paramètres

Agrandir le tableau

Terme	Définition
matchBy_columnName	(Facultatif) Nom d'une colonne existante à utiliser pour identifier la ligne actuelle dans la <relation> d'une fonction de fenêtre. RELATED() peut également être utilisé pour faire référence à une colonne dans une table liée à une <relation>.

Valeur renvoyée

Cette fonction ne retourne pas de valeur.

Notes

Cette fonction s'utilise uniquement dans une expression de fonction de fenêtre.

Exemple

Voir OFFSET.

Contenu connexe

Compréhension des fonctions ORDERBY, PARTITIONBY et MATCHBY
INDEX
DÉCALAGE
ORDERBY
PARTITIONBY
WINDOW
RANK
ROWNUMBER

Commentaires

Cette page a-t-elle été utile ? Yes No

MOVINGAVERAGE

Article • 24/02/2024

Retourne une moyenne mobile calculée le long de l'axe donné de la grille de données du calcul de visuel. Autrement dit, la moyenne de la colonne donnée calculée sur les lignes de la dernière <windowSize>.

Syntaxe

DAX

```
MOVINGAVERAGE ( <column>, <windowSize>[, <includeCurrent>][, <axis>][,
<blanks>][, <reset>] )
```

Paramètres

Agrandir le tableau

Terme	Définition
N°	La colonne qui fournit la valeur pour chaque ligne.
windowSize	Le nombre de lignes à inclure dans le calcul. Doit être une valeur constante.
includeCurrent	(Facultatif) Une valeur logique spécifiant s'il faut ou non inclure la ligne active dans la plage. La valeur par défaut est True.
axis	(Facultatif) Une référence d'axe, la direction selon laquelle la moyenne mobile sera calculée.
vides	(Facultatif) Énumération qui définit comment gérer des valeurs vides lors du tri. Ce paramètre est réservé à un usage futur.
reset	(Facultatif) Indique si le calcul est réinitialisé et à quel niveau de la hiérarchie des colonnes de la forme visuelle. Les valeurs acceptées sont : NONE, LOWESTPARENT, HIGHESTPARENT ou un entier. Le comportement dépend du signe de l'entier : - S'il est égal à zéro ou s'il est omis, le calcul n'est pas réinitialisé. Équivalent à NONE. - S'il est positif, l'entier identifie la colonne en commençant par la plus élevée, indépendamment du fragment. HIGHESTPARENT équivaut à 1. - S'il est négatif, l'entier identifie la colonne en commençant par la plus basse, par rapport au fragment actuel. LOWESTPARENT équivaut à -1.

Valeur retournée

Une valeur scalaire, la moyenne mobile au niveau de la ligne actuelle.

Notes

Cette fonction peut être utilisée seulement dans les calculs de visuel.

Les paramètres <includeCurrent>, <axis>, <blanks> et <reset> peuvent être omis.

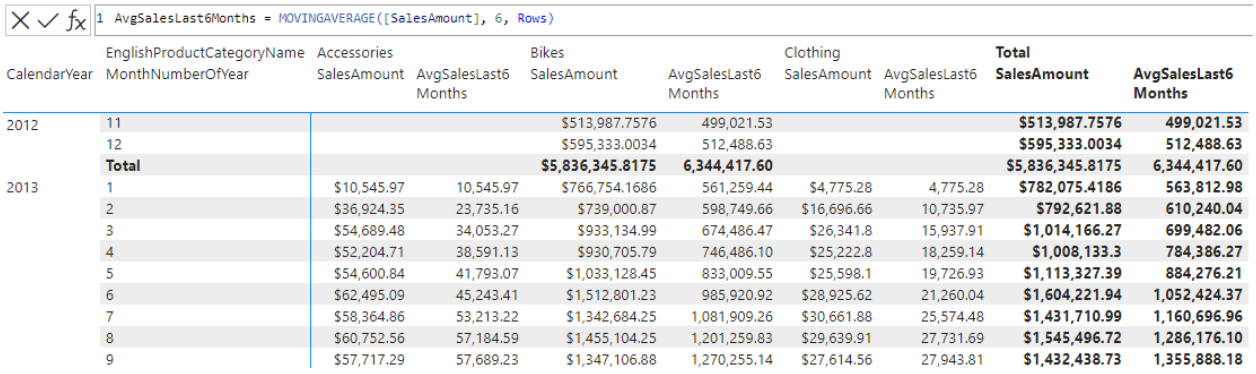
Exemple 1

Dans une table récapitulant les ventes totales pour chaque catégorie de produit et chaque mois calendaire, la requête DAX suivante ajoute une colonne contenant la moyenne des ventes totales pour cette catégorie au cours des 6 derniers mois :

DAX

```
AvgSalesLast6Months = MOVINGAVERAGE([SalesAmount], 6, Rows)
```

La capture d'écran ci-dessous montre la matrice du visuel et l'expression du calcul de visuel :



```
1 AvgSalesLast6Months = MOVINGAVERAGE([SalesAmount], 6, Rows)
```

CalendarYear	EnglishProductCategoryName MonthNumberOfYear	Accessories SalesAmount	 AvgSalesLast6 Months	Bikes SalesAmount	 AvgSalesLast6 Months	Clothing SalesAmount	 AvgSalesLast6 Months	**Total SalesAmount**	**AvgSalesLast6 Months**
2012	11			$513,987.7576	499,021.53			**$513,987.7576**	**499,021.53**
	12			$595,333.0034	512,488.63			**$595,333.0034**	**512,488.63**
	Total			**$5,836,345.8175**	**6,344,417.60**			**$5,836,345.8175**	**6,344,417.60**
2013	1	$10,545.97	10,545.97	$766,754.1686	561,259.44	$4,775.28	4,775.28	**$782,075.4186**	**563,812.98**
	2	$36,924.35	23,735.16	$739,000.87	598,749.66	$16,696.66	10,735.97	**$792,621.88**	**610,240.04**
	3	$54,689.48	34,053.27	$933,134.99	674,486.47	$26,341.8	15,937.91	**$1,014,166.27**	**699,482.06**
	4	$52,204.71	38,591.13	$930,705.79	746,486.10	$25,222.8	18,259.14	**$1,008,133.3**	**784,386.27**
	5	$54,600.84	41,793.07	$1,033,128.45	833,009.55	$25,598.1	19,726.93	**$1,113,327.39**	**884,276.21**
	6	$62,495.09	45,243.41	$1,512,801.23	985,920.92	$28,925.62	21,260.04	**$1,604,221.94**	**1,052,424.37**
	7	$58,364.86	53,213.22	$1,342,684.25	1,081,909.26	$30,661.88	25,574.48	**$1,431,710.99**	**1,160,696.96**
	8	$60,752.56	57,184.59	$1,455,104.25	1,201,259.83	$29,639.91	27,731.69	**$1,545,496.72**	**1,286,176.10**
	9	$57,717.29	57,689.23	$1,347,106.88	1,270,255.14	$27,614.56	27,943.81	**$1,432,438.73**	**1,355,888.18**

Exemple 2

Dans la même table, la requête DAX suivante ajoute une colonne contenant la moyenne des ventes totales pour cette catégorie au cours des 12 derniers mois (sans y inclure le mois actuel) :

DAX

```
AvgSalesPrev12Months = MOVINGAVERAGE([SalesAmount], 12, FALSE, Rows, KEEP)
```

Voir aussi

INDEX
ORDERBY
PARTITIONBY
RUNNINGSUM
WINDOW

Commentaires

Cette page a-t-elle été utile ? Yes No

OFFSET

Article • 21/02/2024

Retourne une seule ligne qui est positionnée avant ou après la *ligne actuelle* dans la même table, avec un décalage donné. Si la ligne actuelle ne peut pas être déduite en une seule ligne, plusieurs lignes peuvent être retournées.

Syntaxe

DAX

```
OFFSET ( <delta>[, <relation> or <axis>][, <orderBy>][, <blanks>][,
<partitionBy>][, <matchBy>][, <reset>] )
```

Paramètres

Agrandir le tableau

Terme	Définition
delta	Nombre de lignes avant (valeur négative) ou après (valeur positive) de la ligne actuelle à partir desquelles les données doivent être obtenues. Peut être toute expression DAX qui retourne une valeur scalaire.
relation	(Facultatif) Expression de table à partir de laquelle la ligne de sortie est retournée Si la valeur est spécifiée, toutes les colonnes de <partitionBy> doivent provenir de celle-ci ou d'une table associée. En cas d'omission : – <orderBy> doit être spécifié explicitement. - Toutes les colonnes <orderBy> et <partitionBy> doivent être des noms complets de colonnes et provenir d'une table unique. - La valeur par défaut est ALLSELECTED() de toutes les colonnes dans <orderBy> et <partitionBy>.
axis	(Facultatif) Axe de la forme du visuel. Disponible seulement dans les calculs de visuel et remplace <relation>.
orderBy	(Facultatif) Clause ORDERBY() contenant les expressions qui définissent le tri de chaque partition. En cas d'omission : - la <relation> doit être spécifiée explicitement. - Par défaut, le tri est effectué par chaque colonne dans une <relation> qui n'est pas déjà spécifiée dans <partitionBy>.

Terme	Définition
vides	(Facultatif) Énumération qui définit comment gérer des valeurs vides lors du tri. Ce paramètre est réservé à un usage futur. Actuellement, la seule valeur prise en charge est DEFAULT, où le comportement pour les valeurs numériques est que les valeurs vides sont classées entre les valeurs zéro et les valeurs négatives. Le comportement des chaînes est que les valeurs vides sont triées avant toutes les chaînes, y compris les chaînes vides.
partitionBy	(Facultatif) Clause PARTITIONBY() contenant les colonnes qui définissent la façon dont la <relation> est partitionnée. En cas d'omission, la <relation> est traitée comme une partition unique.
matchBy	(Facultatif) Clause MATCHBY() contenant les colonnes qui définissent la manière de faire correspondre les données et d'identifier la ligne actuelle.
reset	(Facultatif) Disponible seulement dans les calculs de visuel. Indique si le calcul est réinitialisé et à quel niveau de la hiérarchie des colonnes de la forme du visuel. Les valeurs acceptées sont : NONE, LOWESTPARENT, HIGHESTPARENT ou un entier. Le comportement dépend du signe de l'entier : : s'il est égal à zéro ou s'il est omis, le calcul ne se réinitialise pas. Équivalent à NONE. : s'il est positif, l'entier identifie la colonne en commençant depuis la plus élevée, indépendamment du fragment. HIGHESTPARENT équivaut à 1. : s'il est négatif, l'entier identifie la colonne en commençant par la plus basse, par rapport au fragment actuel. LOWESTPARENT équivaut à 1.

Valeur retournée

Une ou plusieurs lignes de la <relation>.

Notes

À l'exception des colonnes ajoutées par les fonctions de table DAX, chaque colonne dans la <relation>, quand <matchBy> n'est pas présent, ou chaque colonne dans <matchBy> et <partitionBy>, quand <matchBy> est présent, doit avoir une valeur externe correspondante permettant de définir la ligne actuelle à traiter, avec le comportement suivant :

- S'il existe exactement une colonne externe correspondante, sa valeur est utilisée.
- S'il n'existe aucune colonne externe correspondante, alors :
  - OFFSET détermine d'abord toutes les colonnes qui n'ont aucune colonne externe correspondante.
  - Pour chaque combinaison de valeurs existantes pour ces colonnes dans le contexte parent de OFFSET, OFFSET est évalué, et une ligne est retournée.
  - La sortie finale d'OFFSET est une union de ces lignes.

- S'il existe plusieurs colonnes externes correspondantes, une erreur est retournée.

Si toutes les colonnes de <relation> ont été ajoutées par les fonctions de table DAX, une erreur est retournée.

Si <matchBy> est présent, OFFSET essaie d'utiliser les colonnes <matchBy> et <partitionBy> pour identifier la ligne.
Si <matchBy> n'est pas présent et que les colonnes spécifiées dans <orderBy> et <partitionBy> ne peuvent pas identifier de manière unique chaque ligne dans la <relation> :

- OFFSET essaie de trouver le moins de colonnes supplémentaires requises pour identifier chaque ligne de manière unique.
- Si de telles colonnes sont trouvées, OFFSET ajoute automatiquement ces nouvelles colonnes à <orderBy>, et chaque partition est triée à l'aide de ce nouvel ensemble de colonnes OrderBy.
- Si ces colonnes sont introuvables, une erreur est retournée.

Une table vide est retournée si :

- La valeur externe correspondante d'une colonne OrderBy ou PartitionBy n'existe pas dans la <relation>.
- La valeur <delta> entraîne un décalage vers une ligne qui n'existe pas dans la partition.

Si OFFSET est utilisé dans une colonne calculée définie sur la même table que <relation> et que <orderBy> est omis, une erreur est retournée.

<reset> peut être utilisé seulement dans les calculs de visuel, et ne peut pas être utilisé en combinaison avec <orderBy> ou <partitionBy>. Si <reset> est présent, <axis> peut être spécifié, mais pas <relation>.

Exemple 1 : colonne calculée

La requête DAX suivante :

```
DAX

DEFINE
VAR vRelation = SUMMARIZECOLUMNS (
                    DimProductCategory[EnglishProductCategoryName],
                    DimDate[CalendarYear],
                    "CurrentYearSales", SUM(FactInternetSales[SalesAmount])
                )
EVALUATE
```

```
ADDCOLUMNS (
    vRelation,
    "PreviousYearSales",
    SELECTCOLUMNS(
        OFFSET (
                -1,
                vRelation,
                ORDERBY([CalendarYear]),
                PARTITIONBY([EnglishProductCategoryName])
        ),
        [CurrentYearSales]
    )
)
```

Retourne une table qui résume le total des ventes pour chaque catégorie de produit et année civile, ainsi que le total des ventes de cette catégorie au cours de l'année précédente.

Exemple 2 : mesure

La requête DAX suivante :

DAX

```
DEFINE
MEASURE DimProduct[CurrentYearSales] = SUM(FactInternetSales[SalesAmount])
MEASURE DimProduct[PreviousYearSales] =
CALCULATE(SUM(FactInternetSales[SalesAmount]), OFFSET(-1, ,
ORDERBY(DimDate[CalendarYear])))
EVALUATE
SUMMARIZECOLUMNS (
    DimDate[CalendarYear],
    "CurrentYearSales", DimProduct[CurrentYearSales],
    "PreviousYearSales", DimProduct[PreviousYearSales]
)
```

Utilise OFFSET() dans une mesure pour renvoyer une table qui résume les ventes totales pour chaque année civile et les ventes totales de l'année précédente.

Exemple 3 : colonne calculée

La requête DAX suivante :

DAX

```
EVALUATE
ADDCOLUMNS (
    FactInternetSales,
    "Previous Sales Amount",
        SELECTCOLUMNS (
            OFFSET (
                -1,
                FactInternetSales,
                ORDERBY ( FactInternetSales[SalesAmount], DESC ),
                PARTITIONBY ( FactInternetSales[ProductKey] ),
                MATCHBY( FactInternetSales[SalesOrderNumber],
FactInternetSales[SalesOrderLineNumber] )
            ),
            FactInternetSales[SalesAmount]
        )
)
```

Retourne la table FactInternetSales en ajoutant une colonne qui indique pour chaque vente le montant de sa vente précédente, du même produit, dans l'ordre décroissant du montant des ventes, la vente actuelle étant identifiée par SalesOrderNumber et SalesOrderLineNumber. Sans MATCHBY, la requête retournerait une erreur, car la table FactInternetSales ne comprend pas de colonnes de clés.

Exemple 4 : calcul de visuel

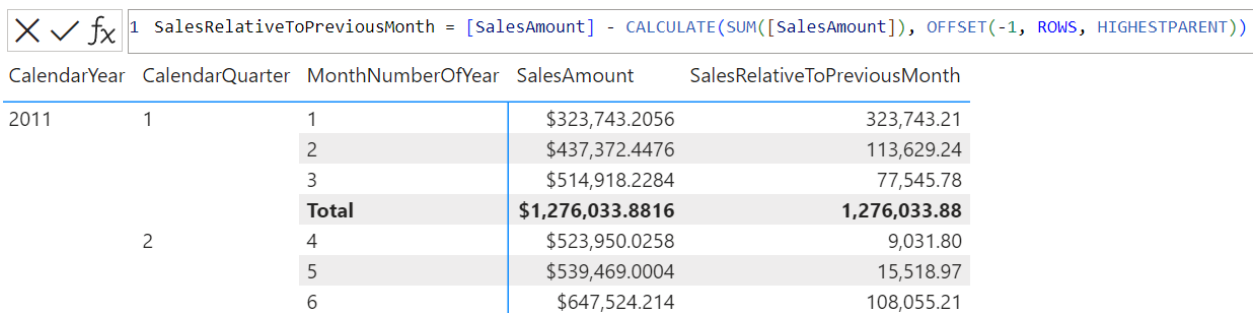
La requête DAX de calcul de visuel suivante :

DAX

```
SalesRelativeToPreviousMonth = [SalesAmount] - CALCULATE(SUM([SalesAmount]),
OFFSET(-1, ROWS, HIGHESTPARENT))
```

Retourne la différence dans le total des ventes entre chaque mois et le mois précédent au cours de la même année.

La capture d'écran ci-dessous montre la matrice du visuel et l'expression du calcul de visuel :



CalendarYear	CalendarQuarter	MonthNumberOfYear	SalesAmount	SalesRelativeToPreviousMonth
2011	1	1	$323,743.2056	323,743.21
		2	$437,372.4476	113,629.24
		3	$514,918.2284	77,545.78
		Total	**$1,276,033.8816**	**1,276,033.88**
	2	4	$523,950.0258	9,031.80
		5	$539,469.0004	15,518.97
		6	$647,524.214	108,055.21

Contenu connexe

INDEX
ORDERBY
PARTITIONBY
MATCHBY
WINDOW
RANK
ROWNUMBER

Commentaires

Cette page a-t-elle été utile ? Yes No

ORDERBY

Article • 20/10/2023

Définit les expressions qui déterminent l'ordre de tri dans chacune des partitions d'une fonction WINDOW.

Syntaxe

DAX

```
ORDERBY ( [<orderBy_expression>[, <order>[, <orderBy_expression>[, <order>]]
…]] )
```

Paramètres

Agrandir le tableau

Terme	Définition
orderBy_expression	(Facultatif) Toute expression scalaire utilisée pour trier les données dans chacune des partitions d'une fonction WINDOW.
order	(Facultatif) Valeur en deux parties de la forme « <Sens_tri> [<Gestion_valeurs_vides>] ». <OrderDirection> spécifie comment trier les valeurs <orderBy_expression> (par ordre croissant ou décroissant). Les valeurs valides sont les suivantes : **DESC**. Valeur alternative : **0**(zéro)/**FALSE**. Trie par ordre décroissant les valeurs <orderBy_expression>. **ASC**. Valeur alternative : **1**/**TRUE**. Trie par ordre croissant les valeurs <orderBy_expression>. Il s'agit de la valeur par défaut lorsque le paramètre <order> est omis. La partie <Gestion_valeurs_vides> est facultative. Elle spécifie comment les valeurs vides sont triées. Les valeurs valides sont les suivantes : **BLANKS DEFAULT**. Il s'agit de la valeur par défaut. Le comportement pour les valeurs numériques est que les valeurs vides sont classées entre les valeurs zéro et les valeurs négatives. Le comportement des chaînes est que les valeurs vides sont triées avant toutes les chaînes, y compris les chaînes vides. **BLANKS FIRST**. Les valeurs vides sont toujours classées au début, que l'ordre de tri soit croissant ou décroissant. **BLANKS LAST**. Les valeurs vides sont toujours classées à la fin, que l'ordre de tri soit croissant ou décroissant.

Valeur renvoyée

Cette fonction ne retourne pas de valeur.

Notes

Cette fonction s'utilise uniquement dans une expression de fonction de fenêtre.

Exemple

Voir OFFSET.

Contenu connexe

Compréhension des fonctions ORDERBY, PARTITIONBY et MATCHBY
INDEX
DÉCALAGE
PARTITIONBY
MATCHBY
WINDOW
RANK
ROWNUMBER

Commentaires

Cette page a-t-elle été utile ? Yes No

PARTITIONBY

Article • 14/12/2023

Définit les colonnes utilisées pour partitionner le paramètre <relation> d'une fonction WINDOW.

Syntaxe

DAX

```
PARTITIONBY ( [<partitionBy_columnName>[, partitionBy_columnName [, …]]] )
```

Paramètres

Agrandir le tableau

Terme	Définition
partitionBy_columnName	(Facultatif) Nom d'une colonne existante à utiliser pour partitionner la <relation> d'une fonction WINDOW. RELATED() peut également être utilisé pour faire référence à une colonne dans une table liée à une <relation>.

Valeur renvoyée

Cette fonction ne retourne pas de valeur.

Notes

Cette fonction s'utilise uniquement dans une expression de fonction de fenêtre.

Exemple

Voir OFFSET.

Contenu connexe

Compréhension des fonctions ORDERBY, PARTITIONBY et MATCHBY
INDEX
DÉCALAGE
ORDERBY
MATCHBY
WINDOW
RANK
ROWNUMBER

Commentaires

Cette page a-t-elle été utile ? Yes No

RANGE

Article • 24/02/2024

Retourne un intervalle de lignes dans l'axe donné, par rapport à la ligne actuelle. Cet intervalle sera constitué des dernières lignes de l'<étape> avant l'étape actuelle, ou des premières lignes de l'<étape> après l'étape actuelle.

Syntaxe

DAX

```
RANGE ( <step>[, <includeCurrent>][, <axis>][, <blanks>][, <reset>] )
```

Paramètres

Agrandir le tableau

Terme	Définition
step	Le nombre de lignes avant (valeur négative) ou après (valeur positive) la ligne actuelle à inclure dans la plage. Doit être une valeur constante. : Si elle est négative, la fenêtre contiendra les dernières lignes de l'étape avant la ligne actuelle. : sinon, la fenêtre contiendra les premières lignes de l'étape après la ligne actuelle.
includeCurrent	(Facultatif) Une valeur logique spécifiant s'il faut ou non inclure la ligne active dans la plage. La valeur par défaut est True.
axis	(Facultatif) Une référence d'axe, la direction le long de laquelle l'intervalle sera créé.
vides	(Facultatif) Énumération qui définit comment gérer des valeurs vides lors du tri. Ce paramètre est réservé à un usage futur.
reset	(Facultatif) Indique si le calcul est réinitialisé et à quel niveau de la hiérarchie des colonnes de la forme visuelle. Les valeurs acceptées sont : NONE, LOWESTPARENT, HIGHESTPARENT ou un entier. Le comportement dépend du signe de l'entier : - S'il est égal à zéro ou s'il est omis, le calcul n'est pas réinitialisé. Équivalent à NONE. - S'il est positif, l'entier identifie la colonne en commençant par la plus élevée, indépendamment du fragment. HIGHESTPARENT équivaut à 1.

Terme	Définition
	- S'il est négatif, l'entier identifie la colonne en commençant par la plus basse, par rapport au fragment actuel. LOWESTPARENT équivaut à -1.

Valeur retournée

Un intervalle de lignes de données.

Notes

Cette fonction peut être utilisée seulement dans les calculs de visuel.

Les paramètres <includeCurrent>, <axis>, <blanks> et <reset> peuvent être omis.

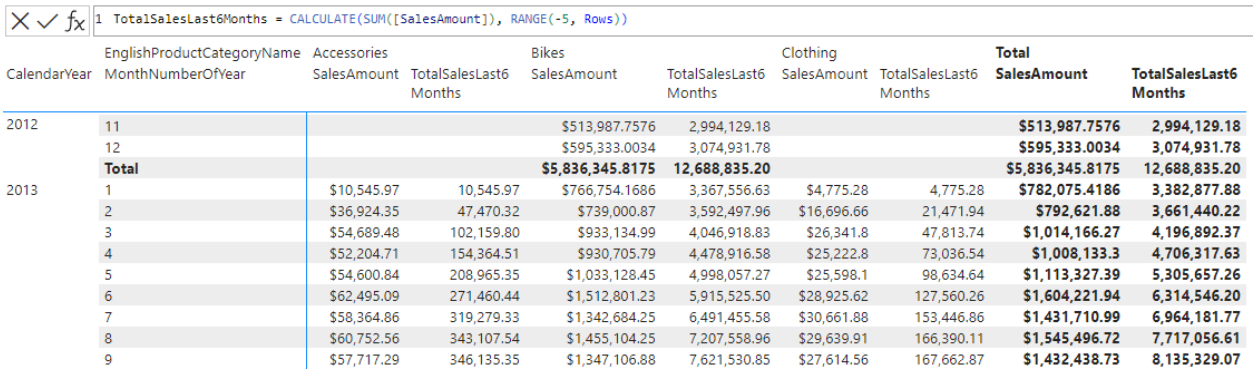
Exemple 1

Dans une table récapitulant les ventes totales pour chaque catégorie de produit et chaque mois, la requête DAX suivante ajoute une colonne contenant les ventes totales au cours des 6 derniers mois :

DAX

```
TotalSalesLast6Months = CALCULATE(SUM([SalesAmount]), RANGE(-5, Rows))
```

La capture d'écran ci-dessous montre la matrice du visuel et l'expression du calcul de visuel :



```
1 TotalSalesLast6Months = CALCULATE(SUM([SalesAmount]), RANGE(-5, Rows))
```

CalendarYear	EnglishProductCategoryName MonthNumberOfYear	Accessories SalesAmount	 TotalSalesLast6 Months	Bikes SalesAmount	 TotalSalesLast6 Months	Clothing SalesAmount	 TotalSalesLast6 Months	**Total SalesAmount**	** TotalSalesLast6 Months**
2012	11			$513,987.7576	2,994,129.18			**$513,987.7576**	**2,994,129.18**
	12			$595,333.0034	3,074,931.78			**$595,333.0034**	**3,074,931.78**
	Total			**$5,836,345.8175**	**12,688,835.20**			**$5,836,345.8175**	**12,688,835.20**
2013	1	$10,545.97	10,545.97	$766,754.1686	3,367,556.63	$4,775.28	4,775.28	**$782,075.4186**	**3,382,877.88**
	2	$36,924.35	47,470.32	$739,000.87	3,592,497.96	$16,696.66	21,471.94	**$792,621.88**	**3,661,440.22**
	3	$54,689.48	102,159.80	$933,134.99	4,046,918.83	$26,341.8	47,813.74	**$1,014,166.27**	**4,196,892.37**
	4	$52,204.71	154,364.51	$930,705.79	4,478,916.58	$25,222.8	73,036.54	**$1,008,133.3**	**4,706,317.63**
	5	$54,600.84	208,965.35	$1,033,128.45	4,998,057.27	$25,598.1	98,634.64	**$1,113,327.39**	**5,305,657.26**
	6	$62,495.09	271,460.44	$1,512,801.23	5,915,525.50	$28,925.62	127,560.26	**$1,604,221.94**	**6,314,546.20**
	7	$58,364.86	319,279.33	$1,342,684.25	6,491,455.58	$30,661.88	153,446.86	**$1,431,710.99**	**6,964,181.77**
	8	$60,752.56	343,107.54	$1,455,104.25	7,207,558.96	$29,639.91	166,390.11	**$1,545,496.72**	**7,717,056.61**
	9	$57,717.29	346,135.35	$1,347,106.88	7,621,530.85	$27,614.56	167,662.87	**$1,432,438.73**	**8,135,329.07**

Exemple 2

Dans la même table, la requête DAX suivante ajoute une colonne avec le total des ventes dans les 12 mois suivants, sans y inclure le mois actuel :

DAX

```
TotalSalesFollowingYear = CALCULATE(SUM([SalesAmount]), RANGE(12, FALSE,
Rows, KEEP))
```

Voir aussi

INDEX
ORDERBY
PARTITIONBY
WINDOW

Commentaires

Cette page a-t-elle été utile ? Yes No

RANK

Article • 21/02/2024

Retourne le classement du contexte actuel dans la partition spécifiée triée selon l'ordre spécifié. Si aucune correspondance n'est trouvée, rank est vide.

Syntaxe

DAX

```
RANK ( [<ties>][, <relation> or <axis>][, <orderBy>][, <blanks>][,
<partitionBy>][, <matchBy>][, <reset>] )
```

Paramètres

Agrandir le tableau

Terme	Définition
ties	(Facultatif) Définit comment gérer le classement quand deux lignes ou plus sont à égalité. Si la valeur est spécifiée, la valeur prise en charge est DENSE ou SKIP. Si aucune valeur n'est spécifiée : SKIP par défaut
relation	(Facultatif) Expression de table à partir de laquelle la ligne de sortie est retournée Si vous en spécifiez une, toutes les colonnes <orderBy> et <partitionBy> doivent provenir de celle-ci. En cas d'omission : - <orderBy> doit être spécifié explicitement. - Toutes les colonnes <orderBy> et <partitionBy> doivent être complètes et provenir d'une seule table. - La valeur par défaut est ALLSELECTED() de toutes les colonnes dans <orderBy> et <partitionBy>.
axis	(Facultatif) Axe de la forme du visuel. Disponible seulement dans les calculs de visuel et remplace <relation>.
orderBy	(Facultatif) Clause ORDERBY() contenant les colonnes qui définissent le tri de chaque partition. En cas d'omission : - la <relation> doit être spécifiée explicitement. - Par défaut, le tri est effectué par chaque colonne dans une <relation> qui n'est pas déjà spécifiée dans <partitionBy>.

Terme	Définition
vides	(Facultatif) Énumération qui définit comment gérer des valeurs vides lors du tri. Les valeurs prises en charge sont : • KEEP (la valeur par défaut), où le comportement pour les valeurs numériques est que les valeurs vides sont classées entre zéro et les valeurs négatives. Le comportement des chaînes est que les valeurs vides sont triées avant toutes les chaînes, y compris les chaînes vides. • FIRST, les valeurs vides sont toujours classées au début, que l'ordre de tri soit croissant ou décroissant. • LAST, les valeurs vides sont toujours classées à la fin, que l'ordre de tri soit croissant ou décroissant. Notez que quand le paramètres <blanks> et des valeurs vides dans la fonction ORDERBY() sur une expression individuelle sont tous deux spécifiés, les valeurs <vide> sur l'expression orderBy individuelle sont prioritaires pour l'expression orderBy appropriée, et les expressions orderBy sans que <blanks> soit spécifié respectent le paramètre <blanks> sur la fonction de la fenêtre parente.
partitionBy	(Facultatif) Clause PARTITIONBY() contenant les colonnes qui définissent la façon dont la <relation> est partitionnée. En cas d'omission, la <relation> est traitée comme une partition unique.
matchBy	(Facultatif) Clause MATCHBY() contenant les colonnes qui définissent la manière de faire correspondre les données et d'identifier la ligne actuelle.
reset	(Facultatif) Disponible seulement dans les calculs de visuel. Indique si le calcul est réinitialisé et à quel niveau de la hiérarchie des colonnes de la forme du visuel. Les valeurs acceptées sont : NONE, LOWESTPARENT, HIGHESTPARENT ou un entier. Le comportement dépend du signe de l'entier : : s'il est égal à zéro ou s'il est omis, le calcul ne se réinitialise pas. Équivalent à NONE. : s'il est positif, l'entier identifie la colonne en commençant depuis la plus élevée, indépendamment du fragment. HIGHESTPARENT équivaut à 1. : s'il est négatif, l'entier identifie la colonne en commençant par la plus basse, par rapport au fragment actuel. LOWESTPARENT équivaut à 1.

Valeur retournée

Numéro de classement du contexte actuel.

Notes

- Chaque colonne <orderBy>, <partitionBy> et <matchBy> doit avoir une valeur externe correspondante permettant de définir la ligne actuelle à traiter, avec le comportement suivant :

    - S'il existe exactement une colonne externe correspondante, sa valeur est utilisée.
    - S'il n'existe aucune colonne externe correspondante, alors :
        - RANK détermine d'abord toutes les colonnes <orderBy>, <partitionBy> et <matchBy> qui n'ont aucune colonne externe correspondante.
        - Pour chaque combinaison de valeurs existantes pour ces colonnes dans le contexte parent de RANK, RANK est évalué, et une ligne est retournée.
        - La sortie finale de RANK est un numéro de classement.

- Si <matchBy> est présent, RANK essaie d'utiliser les colonnes dans <matchBy> et <partitionBy> pour identifier la ligne actuelle.

- Si les colonnes spécifiées dans <orderBy> et <partitionBy> ne peuvent pas identifier de manière unique chaque ligne dans la <relation>, deux lignes ou plus peuvent avoir le même classement et le classement est déterminé par le paramètre Ties.

- RANK retourne une valeur vide pour le nombre total de lignes. Il est recommandé de tester votre expression de manière approfondie.

- RANK n'est pas comparable à RANKX comme SUM est comparable à SUMX.

- <reset> peut être utilisé seulement dans les calculs de visuel, et ne peut pas être utilisé en combinaison avec <orderBy> ou <partitionBy>. Si <reset> est présent, <axis> peut être spécifié, mais pas <relation>.

Exemple 1 : colonne calculée

La requête DAX suivante :

DAX

```
EVALUATE
ADDCOLUMNS(
    'DimGeography',
    "Rank",
    RANK(
        DENSE,
        'DimGeography',
        ORDERBY(
            'DimGeography'[StateProvinceName], desc,
            'DimGeography'[City], asc),
        LAST,
        PARTITIONBY(
            'DimGeography'[EnglishCountryRegionName])))
```

```
ORDER BY [EnglishCountryRegionName] asc, [StateProvinceName] desc, [City]
asc
```

Renvoie une table qui classe chaque zone géographique avec le même EnglishCountryRegionName, par StateProvinceName et City. Les valeurs de colonne <orderBy> vides sont triées à la fin.

Exemple 2 : calcul de visuel

Les requêtes DAX de calcul de visuel suivantes :

DAX

```
SalesRankWithinYear = RANK(DENSE, ORDERBY([SalesAmount], DESC),
PARTITIONBY([CalendarYear]))

SalesRankAllHistory = RANK(DENSE, ORDERBY([SalesAmount], DESC))
```

Créent deux colonnes qui classent chaque mois par total des ventes, une pour chaque année l'autre dans tout l'historique.

La capture d'écran ci-dessous montre la matrice du visuel et l'expression du premier calcul de visuel :



CalendarYear	CalendarQuarter	MonthNumberOfYear	SalesAmount	SalesRankWithinYear	SalesRankAllHistory
2011	1	1	$323,743.2056	12.00	37.00
		2	$437,372.4476	11.00	34.00
		3	$514,918.2284	10.00	25.00
		Total	**$1,276,033.8816**	**4.00**	**11.00**
	2	4	$523,950.0258	9.00	24.00
		5	$539,469.0004	8.00	23.00
		6	$647,524.214	4.00	16.00
		Total	**$1,710,943.2402**	**3.00**	**7.00**
	3	7	$648,343.8132	3.00	15.00
		8	$599,874.2732	7.00	20.00
		9	$609,121.865	6.00	19.00
		Total	**$1,857,339.9514**	**2.00**	**6.00**
	4	10	$655,170.3294	2.00	14.00
		11	$716,748.7178	1.00	13.00
		12	$636,253.2642	5.00	17.00
		Total	**$2,008,172.3114**	**1.00**	**5.00**
	Total		**$6,852,489.3846**	**1.00**	**2.00**

Contenu connexe

INDEX
ORDERBY
PARTITIONBY
WINDOW
ROWNUMBER

Commentaires

Cette page a-t-elle été utile ? Yes No

REMOVEFILTERS

Article • 20/10/2023

Efface les filtres des tables ou colonnes spécifiées.

Syntaxe

```DAX
REMOVEFILTERS([<table> | <column>[, <column>[, <column>[,…]]]])
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table dont vous voulez effacer les filtres.
colonne	Colonne dont vous voulez effacer les filtres.

Valeur renvoyée

N/A. Consultez la section Remarques.

Notes

- La fonction REMOVEFILTERS peut être utilisée uniquement pour effacer des filtres et non pour retourner une table.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple 1

Requête DAX

```DAX
DEFINE
MEASURE FactInternetSales[TotalSales] = SUM(FactInternetSales[SalesAmount])
MEASURE FactInternetSales[%Sales] = DIVIDE([TotalSales],
CALCULATE([TotalSales],REMOVEFILTERS()))
```

```
EVALUATE
    SUMMARIZECOLUMNS(
      ROLLUPADDISSUBTOTAL(DimProductCategory[EnglishProductCategoryName],
"IsGrandTotal"),
      "TotalSales", [TotalSales],
      "%Sales", [%Sales]
    )
ORDER BY
  [IsGrandTotal] DESC, [TotalSales] DESC
```

retourne :

Agrandir le tableau

DimProductCategory[EnglishProductCategoryName]	[IsGrandTotal]	[TotalSales]	[%Sales]
Ligne1	True	29358677,2207	1
Bikes	False	28318144,6507	0,964557920570538
Accessories	False	700759,96	0,023868921434441
Clothing	False	339772,61	0,0115731579950215

Exemple 2

Requête DAX

DAX

```
DEFINE
MEASURE FactInternetSales[TotalSales] = SUM(FactInternetSales[SalesAmount])
MEASURE FactInternetSales[%Sales] = DIVIDE([TotalSales],
CALCULATE([TotalSales],REMOVEFILTERS(DimProductSubcategory[EnglishProductSubcategory
Name])))

EVALUATE
    SUMMARIZECOLUMNS(
      DimProductCategory[EnglishProductCategoryName],
      DimProductSubcategory[EnglishProductSubcategoryName],
      "TotalSales", [TotalSales],
      "%Sales", [%Sales]
    )
ORDER BY
  DimProductCategory[EnglishProductCategoryName] ASC,
  DimProductSubcategory[EnglishProductSubcategoryName] ASC
```

retourne :

Agrandir le tableau

DimProductCategory [EnglishProductCategoryName]	DimProductSubcategory [EnglishProductSubcategoryName]	[TotalSales]	[%Sales]
Accessories	Porte-vélos	39360	0,05616759
Accessories	Supports à vélos	39591	0,05649723
Accessories	Bidons et porte-bidons	56798,19	0,08105228
Accessories	Produits d'entretien	7218,6	0,0103011
Accessories	Garde-boue	46619,58	0,06652717
Accessories	Casques	225335,6	0,3215589
Accessories	Sacs à eau	40307,67	0,05751994
Accessories	Pneus et chambres à air	245529,32	0,35037578
Bikes	VTT	9952759,564	0,35146228
Bikes	Vélos de route	14520584,04	0,51276608
Bikes	Vélos de tourisme	3844801,05	0,13577164
Clothing	Limites	19688,1	0,05794493
Clothing	Gants	35020,7	0,10307099
Clothing	Maillots	172950,68	0,5090189
Clothing	Shorts	71319,81	0,20990453
Clothing	Chaussettes	5106,32	0,01502864
Clothing	Gilets	35687	0,10503201

Commentaires

Cette page a-t-elle été utile ? Yes No

ROWNUMBER

Article • 27/02/2024

Retourne le classement unique du contexte actuel dans la partition spécifiée triée selon l'ordre spécifié. Si aucune correspondance n'est trouvée, rownumber est vide.

Syntaxe

DAX

```
ROWNUMBER ( [<relation> or <axis>][, <orderBy>][, <blanks>][, <partitionBy>]
[, <matchBy>][, <reset>] )
```

Paramètres

Agrandir le tableau

Terme	Définition
relation	(Facultatif) Expression de table à partir de laquelle la ligne de sortie est retournée Si vous en spécifiez une, toutes les colonnes <orderBy> et <partitionBy> doivent provenir de celle-ci. En cas d'omission : - <orderBy> doit être spécifié explicitement. - Toutes les colonnes <orderBy> et <partitionBy> doivent être complètes et provenir d'une seule table. - La valeur par défaut est ALLSELECTED() de toutes les colonnes dans <orderBy> et <partitionBy>.
axis	(Facultatif) Axe de la forme du visuel. Disponible seulement dans les calculs de visuel et remplace <relation>.
orderBy	(Facultatif) Clause ORDERBY() contenant les colonnes qui définissent le tri de chaque partition. En cas d'omission : - la <relation> doit être spécifiée explicitement. - Par défaut, le tri est effectué par chaque colonne dans une <relation> qui n'est pas déjà spécifiée dans <partitionBy>.
vides	(Facultatif) Énumération qui définit comment gérer des valeurs vides lors du tri. Les valeurs prises en charge sont : • DEFAULT (la valeur par défaut), où le comportement pour les valeurs numériques est que les valeurs vides sont classées entre les valeurs zéro et les

Terme	Définition
	valeurs négatives. Le comportement des chaînes est que les valeurs vides sont triées avant toutes les chaînes, y compris les chaînes vides. • FIRST, les valeurs vides sont toujours classées au début, que l'ordre de tri soit croissant ou décroissant. • LAST, les valeurs vides sont toujours classées à la fin, que l'ordre de tri soit croissant ou décroissant. Notez que quand le paramètres <blanks> et des valeurs vides dans la fonction ORDERBY() sur une expression individuelle sont tous deux spécifiés, les valeurs <vide> sur l'expression orderBy individuelle sont prioritaires pour l'expression orderBy appropriée, et les expressions orderBy sans que <blanks> soit spécifié respectent le paramètre <blanks> sur la fonction de la fenêtre parente.
partitionBy	(Facultatif) Clause PARTITIONBY() contenant les colonnes qui définissent la façon dont la <relation> est partitionnée. En cas d'omission, la <relation> est traitée comme une partition unique.
matchBy	(Facultatif) Clause MATCHBY() contenant les colonnes qui définissent la manière de faire correspondre les données et d'identifier la ligne actuelle.
reset	(Facultatif) Disponible seulement dans les calculs de visuel. Indique si le calcul est réinitialisé et à quel niveau de la hiérarchie des colonnes de la forme du visuel. Les valeurs acceptées sont : NONE, LOWESTPARENT, HIGHESTPARENT ou un entier. Le comportement dépend du signe de l'entier : - S'il est égal à zéro ou s'il est omis, le calcul n'est pas réinitialisé. Équivalent à NONE. - S'il est positif, l'entier identifie la colonne en commençant par la plus élevée, indépendamment du fragment. HIGHESTPARENT équivaut à 1. - S'il est négatif, l'entier identifie la colonne en commençant par la plus basse, par rapport au fragment actuel. LOWESTPARENT équivaut à -1.

Valeur retournée

Numéro rownumber du contexte actuel.

Notes

Chaque colonne <orderBy>, <partitionBy> et <matchBy> doit avoir une valeur externe correspondante permettant de définir la ligne actuelle à traiter, avec le comportement suivant :

- S'il existe exactement une colonne externe correspondante, sa valeur est utilisée.
- S'il n'existe aucune colonne externe correspondante, alors :

    - ROWNUMBER détermine d'abord toutes les colonnes <orderBy>, <partitionBy> et <matchBy> qui n'ont aucune colonne externe correspondante.
    - Pour chaque combinaison de valeurs existantes pour ces colonnes dans le contexte parent de ROWNUMBER, ROWNUMBER est évalué, et une ligne est retournée.
    - La sortie finale de ROWNUMBER est une union de ces lignes.
- S'il existe plusieurs colonnes externes correspondantes, une erreur est retournée.

Si <matchBy> est présent, ROWNUMBER essaie d'utiliser les colonnes dans <matchBy> et <partitionBy> pour identifier la ligne actuelle.
Si les colonnes spécifiées dans <orderBy> et <partitionBy> ne peuvent pas identifier de manière unique chaque ligne dans la <relation>, alors :

- ROWNUMBER essaie de trouver le moins de colonnes supplémentaires requises pour identifier chaque ligne de manière unique.
- Si de telles colonnes sont disponibles, ROWNUMBER
    - Essaie de trouver le moins de colonnes supplémentaires requises pour identifier chaque ligne de manière unique.
    - Ajoute automatiquement ces nouvelles colonnes à la clause <orderBy>.
    - Trie chaque partition à l'aide de ce nouvel ensemble de colonnes orderBy.
- Si ces colonnes sont introuvables et que la fonction détecte une égalité au moment de l'exécution, une erreur est retournée.

<reset> peut être utilisé seulement dans les calculs de visuel, et ne peut pas être utilisé en combinaison avec <orderBy> ou <partitionBy>. Si <reset> est présent, <axis> peut être spécifié, mais pas <relation>.

Exemple 1 : colonne calculée

La requête DAX suivante :

DAX

```
EVALUATE
ADDCOLUMNS(
    'DimGeography',
    "UniqueRank",
    ROWNUMBER(
        'DimGeography',
        ORDERBY(
            'DimGeography'[StateProvinceName], desc,
            'DimGeography'[City], asc),
        PARTITIONBY(
            'DimGeography'[EnglishCountryRegionName])))
```

```
ORDER BY [EnglishCountryRegionName] asc, [StateProvinceName] desc, [City]
asc
```

Renvoie une table qui classe de manière unique chaque zone géographique avec le même EnglishCountryRegionName, par StateProvinceName et City.

Exemple 2 : calcul de visuel

Les requêtes DAX de calcul de visuel suivantes :

DAX

```
SalesRankWithinYear = ROWNUMBER(ORDERBY([SalesAmount], DESC),
PARTITIONBY([CalendarYear]))

SalesRankAllHistory = ROWNUMBER(ORDERBY([SalesAmount], DESC))
```

Créent deux colonnes qui classent de manière unique chaque mois par total des ventes, une pour chaque année l'autre dans tout l'historique.

La capture d'écran ci-dessous montre la matrice du visuel et l'expression du premier calcul de visuel :

```
1 SalesRankWithinYear = ROWNUMBER(ORDERBY([SalesAmount], DESC), PARTITIONBY([CalendarYear]))
```

CalendarYear	CalendarQuarter	MonthNumberOfYear	SalesAmount	SalesRankWithinYear	SalesRankAllHistory
2011	1	1	$323,743.2056	12.00	37.00
		2	$437,372.4476	11.00	34.00
		3	$514,918.2284	10.00	25.00
		Total	**$1,276,033.8816**	**4.00**	**11.00**
	2	4	$523,950.0258	9.00	24.00
		5	$539,469.0004	8.00	23.00
		6	$647,524.214	4.00	16.00
		Total	**$1,710,943.2402**	**3.00**	**7.00**
	3	7	$648,343.8132	3.00	15.00
		8	$599,874.2732	7.00	20.00
		9	$609,121.865	6.00	19.00
		Total	**$1,857,339.9514**	**2.00**	**6.00**
	4	10	$655,170.3294	2.00	14.00
		11	$716,748.7178	1.00	13.00
		12	$636,253.2642	5.00	17.00
		Total	**$2,008,172.3114**	**1.00**	**5.00**
	Total		**$6,852,489.3846**	**1.00**	**2.00**

Contenu connexe

INDEX
ORDERBY

PARTITIONBY
WINDOW
RANK

Commentaires

Cette page a-t-elle été utile ? Yes No

RUNNINGSUM

Article • 27/02/2024

Retourne une somme cumulée calculée sur l'axe donné de la grille de données du calcul du visuel. C'est-à-dire, la somme de la colonne donnée calculée de toutes les lignes jusqu'à la ligne actuelle.

Syntaxe

DAX

```
RUNNINGSUM ( <column>[, <axis>][, <blanks>][, <reset>] )
```

Paramètres

Agrandir le tableau

Terme	Définition
N°	La colonne qui fournit la valeur pour chaque ligne.
axis	(Facultatif) Référence d'axe, c'est-à-dire le sens dans lequel la somme cumulée est calculée.
vides	(Facultatif) Énumération qui définit comment gérer des valeurs vides lors du tri. Ce paramètre est réservé à un usage futur.
reset	(Facultatif) Indique si le calcul est réinitialisé et à quel niveau de la hiérarchie des colonnes de la forme visuelle. Les valeurs acceptées sont : NONE, LOWESTPARENT, HIGHESTPARENT ou un entier. Le comportement dépend du signe de l'entier : - S'il est égal à zéro ou s'il est omis, le calcul n'est pas réinitialisé. Équivalent à NONE. - S'il est positif, l'entier identifie la colonne en commençant par la plus élevée, indépendamment du fragment. HIGHESTPARENT équivaut à 1. - S'il est négatif, l'entier identifie la colonne en commençant par la plus basse, par rapport au fragment actuel. LOWESTPARENT équivaut à -1.

Valeur retournée

Valeur scalaire, c'est-à-dire la somme cumulée jusqu'à la ligne actuelle.

Notes

Cette fonction peut être utilisée seulement dans les calculs de visuel.

Les paramètres <axis>, <blanks> et <reset> peuvent être omis.

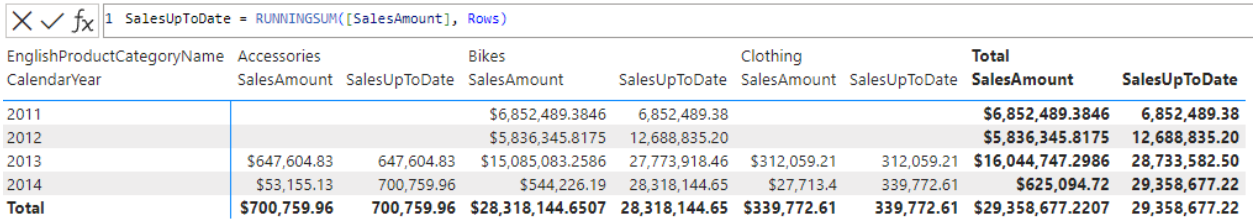
Exemple

Dans une table récapitulant les ventes totales pour chaque catégorie de produit et chaque année calendaire, la requête DAX suivante ajoute une colonne contenant le total des ventes pour cette catégorie jusqu'à une année donnée.

DAX

```
SalesUpToDate = RUNNINGSUM([SalesAmount], Rows)
```

La capture d'écran ci-dessous montre la matrice du visuel et l'expression du calcul de visuel :



```
1 SalesUpToDate = RUNNINGSUM([SalesAmount], Rows)
```

EnglishProductCategoryName	Accessories		Bikes		Clothing		**Total**	
CalendarYear	SalesAmount	SalesUpToDate	SalesAmount	SalesUpToDate	SalesAmount	SalesUpToDate	**SalesAmount**	**SalesUpToDate**
2011			$6,852,489.3846	6,852,489.38			**$6,852,489.3846**	**6,852,489.38**
2012			$5,836,345.8175	12,688,835.20			**$5,836,345.8175**	**12,688,835.20**
2013	$647,604.83	647,604.83	$15,085,083.2586	27,773,918.46	$312,059.21	312,059.21	**$16,044,747.2986**	**28,733,582.50**
2014	$53,155.13	700,759.96	$544,226.19	28,318,144.65	$27,713.4	339,772.61	**$625,094.72**	**29,358,677.22**
Total	**$700,759.96**	**700,759.96**	**$28,318,144.6507**	**28,318,144.65**	**$339,772.61**	**339,772.61**	**$29,358,677.2207**	**29,358,677.22**

Voir aussi

INDEX
MOVINGAVERAGE
ORDERBY
PARTITIONBY
WINDOW

Commentaires

Cette page a-t-elle été utile ? Yes No

SELECTEDVALUE

Article • 20/10/2023

Retourne la valeur quand le contexte pour columnName a été réduit par filtrage à une seule valeur distincte. Sinon, retourne alternateResult.

Syntaxe

DAX

```
SELECTEDVALUE(<columnName>[, <alternateResult>])
```

Paramètres

Agrandir le tableau

Terme	Définition
columnName	Nom d'une colonne existante, spécifié avec la syntaxe DAX standard. Il ne peut pas s'agir d'une expression.
alternateResult	(Facultatif) Valeur retournée quand le contexte pour columnName a été réduit par filtrage à zéro ou à plusieurs valeurs distinctes. Quand elle est omise, la valeur par défaut est BLANK().

Valeur de retour

Valeur quand le contexte pour columnName a été réduit par filtrage à une seule valeur distincte. Sinon, alternateResult.

Notes

- `IF(HASONEVALUE(<columnName>), VALUES(<columnName>), <alternateResult>)` est une expression équivalente de `SELECTEDVALUE(<columnName>, <alternateResult>)`.
- Pour en savoir plus sur les meilleures pratiques lors de l'utilisation de SELECTEDVALUE, consultez Utiliser SELECTEDVALUE au lieu de valeurs dans DAX.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au

niveau des lignes (RLS).

Exemple

La requête DAX suivante :

```DAX
DEFINE
 MEASURE DimProduct[Selected Color] = SELECTEDVALUE(DimProduct[Color], "No Single Selection")
EVALUATE
 SUMMARIZECOLUMNS
   (ROLLUPADDISSUBTOTAL(DimProduct[Color], "Is Total"),
   "Selected Color", [Selected Color])ORDER BY [Is Total] ASC,
   [Color] ASC
```

retourne les résultats suivants :

Agrandir le tableau

DimProduct[Color]	[Is Total]	[Selected Color]
Noir	FALSE	Noir
Bleu	FALSE	Blue
Grey	FALSE	Grey
Multiple	FALSE	Multiple
N/D	FALSE	N/A
Rouge	FALSE	Rouge
Argent	FALSE	Argent
Silver/Black	FALSE	Silver/Black
Blancs	FALSE	Blancs
Jaune	FALSE	Jaune
	true	Aucune sélection unique

Commentaires

Cette page a-t-elle été utile ? Yes No

WINDOW

Article • 21/02/2024

Retourne plusieurs lignes qui sont positionnées dans l'intervalle donné.

Syntaxe

DAX

```
WINDOW ( from[, from_type], to[, to_type][, <relation> or <axis>][,
<orderBy>][, <blanks>][, <partitionBy>][, <matchBy>][, <reset>] )
```

Paramètres

Agrandir le tableau

Terme	Définition
de	Indique l'emplacement de démarrage de la fenêtre. Peut être toute expression DAX qui retourne une valeur scalaire. Le comportement dépend du paramètre <from_type> : - Si <from_type> est REL, il s'agit du nombre de lignes duquel revenir en arrière (valeur négative) ou avancer (valeur positive) à partir de la ligne active pour obtenir la première ligne dans la fenêtre. - Si <from_type> a la valeur ABS et que la valeur de <from> est positive, il s'agit de la position du début de la fenêtre à partir du début de la partition. L'indexation commence à 1. Par exemple, 1 signifie que la fenêtre démarre au début de la partition. Si la valeur de <from> est négative, il s'agit de la position du début de la fenêtre à partir de la fin de la partition. -1 signifie la dernière ligne de la partition.
from_type	Modifie le comportement du paramètre <from>. Les valeurs possibles sont ABS (absolu) et REL (relatif). La valeur par défaut est REL.
to	Identique à <from>, mais indique la fin de la fenêtre. La dernière ligne est incluse dans la fenêtre.
to_type	Identique à <from_type>, mais modifie le comportement de <à>.
relation	(Facultatif) Expression de table à partir de laquelle les lignes de sortie sont retournées. Si la valeur est spécifiée, toutes les colonnes de <partitionBy> doivent provenir de celle-ci ou d'une table associée. En cas d'omission : – <orderBy> doit être spécifié explicitement. - Toutes les colonnes <orderBy> et <partitionBy> doivent être des noms complets

Terme	Définition
	de colonnes et provenir d'une table unique. - La valeur par défaut est ALLSELECTED() de toutes les colonnes dans <orderBy> et <partitionBy>.
axis	(Facultatif) Axe de la forme du visuel. Disponible seulement dans les calculs de visuel et remplace <relation>.
orderBy	(Facultatif) Clause ORDERBY() contenant les expressions qui définissent le tri de chaque partition. En cas d'omission : - la <relation> doit être spécifiée explicitement. - Par défaut, le tri est effectué par chaque colonne dans une <relation> qui n'est pas déjà spécifiée dans <partitionBy>.
vides	(Facultatif) Énumération qui définit comment gérer des valeurs vides lors du tri. Ce paramètre est réservé à un usage futur. Actuellement, la seule valeur prise en charge est DEFAULT, où le comportement pour les valeurs numériques est que les valeurs vides sont classées entre les valeurs zéro et les valeurs négatives. Le comportement des chaînes est que les valeurs vides sont triées avant toutes les chaînes, y compris les chaînes vides.
partitionBy	(Facultatif) Clause PARTITIONBY() contenant les colonnes qui définissent la façon dont la <relation> est partitionnée. En cas d'omission, la <relation> est traitée comme une partition unique.
matchBy	(Facultatif) Clause MATCHBY() contenant les colonnes qui définissent la manière de faire correspondre les données et d'identifier la ligne actuelle.
reset	(Facultatif) Disponible seulement dans les calculs de visuel. Indique si le calcul est réinitialisé et à quel niveau de la hiérarchie des colonnes de la forme du visuel. Les valeurs acceptées sont : NONE, LOWESTPARENT, HIGHESTPARENT ou un entier. Le comportement dépend du signe de l'entier : : s'il est égal à zéro ou s'il est omis, le calcul ne se réinitialise pas. Équivalent à NONE. : s'il est positif, l'entier identifie la colonne en commençant depuis la plus élevée, indépendamment du fragment. HIGHESTPARENT équivaut à 1. : s'il est négatif, l'entier identifie la colonne en commençant par la plus basse, par rapport au fragment actuel. LOWESTPARENT équivaut à 1.

Valeur retournée

Toutes les lignes de la fenêtre.

Notes

À l'exception des colonnes ajoutées par les fonctions de table DAX, chaque colonne dans la <relation>, quand <matchBy> n'est pas présent, ou chaque colonne dans

<matchBy> et <partitionBy>, quand <matchBy> est présent, doit avoir une valeur externe correspondante permettant de définir la ligne actuelle à traiter. Si <from_type> et <to_type> ont la valeur ABS, ce qui suit s'applique uniquement aux colonnes <partitionBy> :

- S'il existe exactement une colonne externe correspondante, sa valeur est utilisée.
- S'il n'existe aucune colonne externe correspondante :
  - WINDOW détermine d'abord toutes les colonnes qui n'ont aucune colonne externe correspondante.
  - Pour chaque combinaison de valeurs existantes pour ces colonnes dans le contexte parent de WINDOW, WINDOW est évalué, et les lignes correspondantes sont retournées.
  - La sortie finale de WINDOW est une union de ces lignes.
- S'il existe plusieurs colonnes externes correspondantes, une erreur est retournée.

Si toutes les colonnes de <relation> ont été ajoutées par les fonctions de table DAX, une erreur est retournée.

Si <matchBy> est présent, WINDOW essaie d'utiliser les colonnes <matchBy> et <partitionBy> pour identifier la ligne.
Si <matchBy> n'est pas présent et que les colonnes spécifiées dans <orderBy> et <partitionBy> ne peuvent pas identifier de manière unique chaque ligne dans la <relation> :

- WINDOW essaie de trouver le moins de colonnes supplémentaires requises pour identifier chaque ligne de manière unique.
- Si de telles colonnes sont trouvées, WINDOW ajoute automatiquement ces nouvelles colonnes à <orderBy>, et chaque partition est triée à l'aide de ce nouvel ensemble de colonnes orderBy.
- Si ces colonnes sont introuvables, une erreur est retournée.

Une table vide est retournée si :

- La valeur externe correspondante d'une colonne <orderBy> ou <partitionBy> n'existe pas dans la <relation>.
- La fenêtre entière se trouve à l'extérieur de la partition, ou le début de la fenêtre se trouve après sa fin.

Si WINDOW est utilisé dans une colonne calculée définie sur la même table que <relation> et que <orderBy> est omis, une erreur est retournée.

Si le début de la fenêtre se trouve avant la première ligne, la valeur est définie sur la première ligne. De même, si la fin de la fenêtre se trouve après la dernière ligne de la

partition, elle est définie sur la dernière ligne.

<reset> peut être utilisé seulement dans les calculs de visuel, et ne peut pas être utilisé en combinaison avec <orderBy> ou <partitionBy>. Si <reset> est présent, <axis> peut être spécifié, mais pas <relation>.

Exemple 1 : mesure

La mesure suivante :

DAX

```
3-day Average Price =
AVERAGEX(
    WINDOW(
        -2,REL,0,REL,
        SUMMARIZE(ALLSELECTED('Sales'), 'Date'[Date], 'Product'[Product]),
        ORDERBY('Date'[Date]),
        KEEP,
        PARTITIONBY('Product'[Product])
    ),
    CALCULATE(AVERAGE(Sales[Unit Price]))
)
```

Retourne la moyenne sur 3 jours des prix unitaires pour chaque produit. Notez que la fenêtre de 3 jours se compose de trois jours pendant lesquels le produit a des ventes, pas nécessairement trois jours calendaires consécutifs.

Exemple 2 : mesure

La mesure suivante :

DAX

```
RunningSum =
SUMX (
    WINDOW (
        1, ABS, 0, REL,
        ALLSELECTED (
            'Date'[Fiscal Year],
            'Date'[Month Number Of Year]
        ),
        PARTITIONBY ( 'Date'[Fiscal Year] )
    ),
    [Total Sales]
```

```
)
```

Retourne la somme actuelle pour le Total des ventes sur [nombre du mois de l'année], en redémarrant pour chaque exercice financier :

Agrandir le tableau

Year	Nombre du mois de l'année	Sales Amount	RunningSum
EF2018	1	1 327 675 $	1 327 675 $
EF2018	2	3 936 463 $	5 264 138 $
EF2018	3	700 873 $	5 965 011 $
EF2018	4	1 519 275 $	7 484 286 $
EF2018	5	2 960 378 $	10 444 664 $
EF2018	6	1 487 671 $	11 932 336 $
EF2018	7	1 423 357 $	13 355 693 $
EF2018	8	2 057 902 $	15 413 595 $
EF2018	9	2 523 948 $	17 937 543 $
EF2018	10	561 681 $	18 499 224 $
EF2018	11	4 764 920 $	23 264 145 $
EF2018	12	596 747 $	23 860 891 $
EF2019	1	1 847 692 $	1 847 692 $
EF2019	2	2 829 362 $	4 677 054 $
EF2019	3	2 092 434 $	6 769 488 $
EF2019	4	2 405 971 $	9 175 459 $
EF2019	5	3 459 444 $	12 634 903 $
EF2019	6	2 850 649 $	15 485 552 $
EF2019	7	2 939 691 $	18 425 243 $
EF2019	8	3 964 801 $	22 390 045 $
EF2019	9	3 287 606 $	25 677 650 $

Year	Nombre du mois de l'année	Sales Amount	RunningSum
EF2019	10	2 157 287 $	27 834 938 $
EF2019	11	3 611 092 $	31 446 030 $
EF2019	12	2 624 078 $	34 070 109 $
EF2020	1	3 235 187 $	3 235 187 $
EF2020	2	4 070 046 $	7 305 233 $
EF2020	3	4 429 833 $	11 735 066 $
EF2020	4	4 002 614 $	15 737 680 $
EF2020	5	5 265 797 $	21 003 477 $
EF2020	6	3 465 241 $	24 468 717 $
EF2020	7	3 513 064 $	27 981 781 $
EF2020	8	5 247 165 $	33 228 947 $
EF2020	9	5 104 088 $	38 333 035 $
EF2020	10	3 542 150 $	41 875 184 $
EF2020	11	5 151 897 $	47 027 081 $
EF2020	12	4 851 194 $	51 878 275 $

Exemple 3 : calcul de visuel

La requête DAX de calcul de visuel suivante :

DAX

```
TotalSalesRunningSumByYear = SUMX(WINDOW(0, ABS, 0, REL, ROWS,
HIGHESTPARENT), [SalesAmount])
```

Retourne le total cumulé des ventes par mois, calculé au fil de chaque année. Les valeurs 1 et -2 pourraient être utilisées au lieu de HIGHESTPARENT, avec les mêmes résultats.

La capture d'écran ci-dessous montre la matrice du visuel et l'expression du calcul de visuel :

```
1 TotalSalesRunningSumByYear = SUMX(WINDOW(0, ABS, 0, REL, ROWS, HIGHESTPARENT), [SalesAmount])
```

CalendarYear	CalendarQuarter	MonthNumberOfYear	SalesAmount	TotalSalesRunningSumByYear	TotalSalesRunningSumByQuarter
2011	1	1	$323,743.2056	323,743.21	323,743.21
		2	$437,372.4476	761,115.65	761,115.65
		3	$514,918.2284	1,276,033.88	1,276,033.88
		Total	**$1,276,033.8816**	**1,276,033.88**	**1,276,033.88**
	2	4	$523,950.0258	1,799,983.91	523,950.03
		5	$539,469.0004	2,339,452.91	1,063,419.03
		6	$647,524.214	2,986,977.12	1,710,943.24

Exemple 4 : calcul de visuel

La requête DAX de calcul de visuel suivante :

DAX

```
TotalSalesRunningSumByQuarter = SUMX(WINDOW(0, ABS, 0, REL, , -1),
[SalesAmount])
```

Retourne le total cumulé des ventes par mois, calculé au fil de chaque trimestre.

Contenu connexe

INDEX
MOVINGAVERAGE
DÉCALAGE
ORDERBY
PARTITIONBY
RANGE
RANK
ROWNUMBERRUNNINGSUM

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonctions financières

Article • 20/10/2023

Les fonctions financières dans DAX s'utilisent dans des formules qui effectuent des calculs financiers, par exemple, pour connaître la valeur actuelle nette et le taux de rendement. Ces fonctions sont similaires aux fonctions financières utilisées dans Microsoft Excel.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
ACCRINT	Retourne les intérêts courus pour un titre qui rapporte des intérêts périodiquement.
ACCRINTM	Retourne les intérêts courus pour un titre qui rapporte des intérêts à l'échéance.
AMORDEGRC	Retourne l'amortissement pour chaque période comptable. Fonction similaire à AMORLINC, à ceci près qu'un coefficient d'amortissement est appliqué en fonction de la durée de vie des actifs.
AMORLINC	Retourne l'amortissement pour chaque période comptable.
COUPDAYBS	Retourne le nombre de jours entre le début de la période d'un coupon et sa date de règlement.
COUPDAYS	Retourne le nombre de jours dans la période de coupon qui contient la date de règlement.
COUPDAYSNC	Retourne le nombre de jours entre la date de règlement et la date du prochain coupon.
COUPNCD	Retourne la date du prochain coupon après la date du règlement.
COUPNUM	Retourne le nombre de coupons dus entre la date de règlement et la date d'échéance, arrondi au nombre entier de coupons le plus proche.
COUPPCD	Retourne la date du coupon précédent avant la date de règlement.
CUMIPMT	Retourne les intérêts cumulés payés sur un prêt entre start_period et end_period.
CUMPRINC	Retourne le capital cumulé payé sur un prêt entre start_period et end_period.
BdD	Retourne l'amortissement d'un actif pour une période spécifiée, calculé avec la méthode de l'amortissement dégressif à taux fixe.

Fonction	Description
DDB	Retourne l'amortissement d'un actif pour une période spécifiée, calculé selon la méthode de l'amortissement dégressif à taux double ou une autre méthode que vous spécifiez.
DISC	Retourne le taux d'escompte pour un titre.
DOLLARDE	Convertit un prix en dollars exprimé sous la forme d'une partie entière et d'une partie fractionnaire, comme 1,02, en prix en dollars exprimé sous la forme d'un nombre décimal.
DOLLARFR	Convertit un prix en dollars exprimé sous la forme d'une partie entière et d'une partie fractionnaire, comme 1,02, en prix en dollars exprimé sous la forme d'un nombre décimal.
DURATION	Retourne la durée de Macaulay pour une valeur nominale supposée de 100 $.
EFFECT	Retourne le taux d'intérêt annuel effectif, compte tenu du taux d'intérêt annuel nominal et du nombre de périodes de calcul de l'intérêt par an.
VC	Calcule la valeur future d'un investissement sur la base d'un taux d'intérêt constant.
INTRATE	Retourne le taux d'intérêt d'un titre totalement investi.
IPMT	Retourne les intérêts payés sur une période donnée pour un investissement avec des paiements périodiques constants et un taux d'intérêt constant.
ISPMT	Calcule les intérêts payés (ou reçus) pour la période spécifiée d'un prêt (ou d'un investissement) avec des paiements de principal identiques.
MDURATION	Retourne la durée de Macauley modifiée pour un titre ayant une valeur nominale hypothétique de 100 $.
NOMINAL	Retourne le taux d'intérêt annuel nominal compte tenu du taux effectif et du nombre de périodes de calcul de l'intérêt par an.
NPER	Retourne le nombre de périodes pour un investissement avec des paiements périodiques constants et un taux d'intérêt constant.
ODDFPRICE	Retourne le cours par valeur nominale de 100 USD d'un titre dont la première période est décalée (courte ou longue).
ODDFYIELD	Retourne le rendement d'un titre dont la première période est inhabituelle (courte ou longue).
ODDLPRICE	Retourne le prix par valeur nominale de 100 $ d'un titre dont la période du dernier coupon est inhabituelle (courte ou longue).
ODDLYIELD	Retourne le rendement d'un titre dont la dernière période est inhabituelle (courte ou longue).

Fonction	Description
PDURATION	Retourne le nombre de périodes requises pour qu'un investissement atteigne une valeur spécifiée.
PMT	Calcule le remboursement d'un emprunt sur la base de paiements constants et d'un taux d'intérêt constant.
PPMT	Retourne, pour une période donnée, la part du paiement du principal d'un investissement avec des paiements périodiques constants et un taux d'intérêt constant.
PRICE	Retourne le prix par valeur nominale de 100 USD d'un titre rapportant des intérêts périodiques.
PRICEDISC	Retourne le cours par valeur nominale de 100 USD pour un titre au-dessous du pair.
PRICEMAT	Retourne le prix par valeur nominale de 100 $ d'un titre rapportant des intérêts à l'échéance.
PV	Calcule la valeur actuelle d'un prêt ou d'un investissement sur la base d'un taux d'intérêt constant.
RATE	Retourne le taux d'intérêt par période d'une annuité.
RECEIVED	Retourne le montant reçu à l'échéance d'un titre totalement investi.
RRI	Retourne un taux d'intérêt équivalent pour la croissance d'un investissement.
AmorLin	Retourne l'amortissement linéaire d'un actif pour une période.
SYD	Retourne l'amortissement d'un actif pour une période spécifiée, calculé selon la méthode de l'amortissement proportionnel à l'ordre numérique inversé des années.
TBILLEQ	Retourne le rendement équivalent obligataire d'un bon du Trésor.
TBILLPRICE	Retourne le prix d'un bon du Trésor par valeur nominale de 100 $.
TBILLYIELD	Retourne le rendement d'un bon du Trésor.
VDB	Retourne l'amortissement d'un actif pour une période spécifiée (périodes partielles incluses), calculé selon la méthode de l'amortissement dégressif à taux double ou une autre méthode que vous spécifiez.
XIRR	Retourne le taux de retour interne pour une planification de flux de trésorerie qui n'est pas nécessairement périodique.
XNPV	Retourne la valeur actualisée pour une planification de flux de trésorerie qui n'est pas nécessairement périodique.

Fonction	Description
YIELD	Retourne le rendement d'un titre qui rapporte des intérêts périodiquement.
YIELDDISC	Retourne le rendement annuel d'un titre escompté.
YIELDMAT	Retourne le rendement annuel d'un titre qui rapporte des intérêts à l'échéance.

Commentaires

Cette page a-t-elle été utile ? Yes No

ACCRINT

Article • 20/10/2023

Retourne les intérêts courus pour un titre qui rapporte des intérêts périodiques.

Syntaxe

DAX

```
ACCRINT(<issue>, <first_interest>, <settlement>, <rate>, <par>, <frequency>
[, <basis>[, <calc_method>]])
```

Paramètres

Agrandir le tableau

Terme	Définition
problème	Date d'émission du titre.
first_interest	Date du premier intérêt du titre.
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
rate	Taux d'intérêt nominal annuel du titre.
par	Valeur nominale du titre.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.
calc_method	(Facultatif) Valeur logique qui spécifie le mode de calcul du total des intérêts courus est calculé quand la date de règlement est postérieure à la date de first_interest. Si calc_method est omis, sa valeur présumée est TRUE. - Si calc_method a la valeur TRUE ou est omis, ACCRINT retourne le total des intérêts courus entre l'émission (issue) et le règlement (settlement). - Si calc_method a la valeur FALSE, ACCRINT retourne les intérêts courus entre first_interest et le règlement.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365
4	Européen 30/360

Valeur de retour

Intérêts courus.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. En DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 correspond au jour 39 448, car il s'agit du 39 448 e jour après le 30 décembre 1899.

- ACCRINT se calcule comme suit :

$$\mathrm{ACCRINT} = \mathrm{par} \times \frac{\mathrm{rate}}{\mathrm{frequency}} \times \sum_{i=1}^{\mathrm{NC}} \frac{\mathrm{A}_i}{\mathrm{NL}_i}$$

  où :
  - A_i = nombre de jours courus pour la période de quasi-coupon i^{th} comprise dans une période décalée.
  - NC = nombre de périodes de quasi-coupon comprises dans une période décalée. Si ce nombre contient une fraction, arrondissez-le au nombre entier supérieur.
  - NL_i = longueur normale en jours de la période de quasi-coupon dans une période décalée.

- issue, first_interest et settlement sont tronqués en entiers.

- frequency et basis sont arrondis à l'entier le plus proche.

- Une erreur est retournée si :

- issue, first_interest ou settlement n'est pas une date valide.
- issue ≥ settlement.
- rate ≤ 0.
- par ≤ 0.
- frequency est un nombre différent de 1, 2 ou 4.
- basis < 0 ou basis > 4.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

Agrandir le tableau

Données	Description
1er mars 2007	Date d'émission
31 août 2008	Date du premier intérêt
1er mai 2008	Date de règlement
10 %	Taux d'intérêt nominal
1 000	Valeur nominale
2	La fréquence est semestrielle (voir ci-dessus)
0	Base 30/360 (voir ci-dessus)

Exemple 1

La requête DAX suivante :

DAX

```
EVALUATE
{
  ACCRINT(DATE(2007,3,1), DATE(2008,8,31), DATE(2008,5,1), 0.1, 1000, 2, 0)
}
```

Retourne l'intérêt couru entre l'émission et le règlement, pour un titre dont les termes sont spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
116,944444444444

Exemple 2

La requête DAX suivante :

DAX

```
EVALUATE
{
  ACCRINT(DATE(2007,3,1), DATE(2008,8,31), DATE(2008,5,1), 0.1, 1000, 2, 0,
FALSE)
}
```

Retourne l'intérêt couru entre first_interest et settlement, pour un titre dont les termes sont spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
66,9444444444445

Commentaires

Cette page a-t-elle été utile ? Yes No

ACCRINTM

Article • 20/10/2023

Retourne les intérêts courus pour un titre qui rapporte des intérêts à l'échéance.

Syntaxe

DAX

```
ACCRINTM(<issue>, <maturity>, <rate>, <par>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
problème	Date d'émission du titre.
maturity	Date d'échéance du titre.
rate	Taux d'intérêt nominal annuel du titre.
par	Valeur nominale du titre.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365
4	Européen 30/360

Valeur de retour

Intérêts courus.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.

- ACCRINTM se calcule comme suit :

$$\mathrm{ACCRINTM} = \mathrm{par} \times \mathrm{rate} \times \frac{\mathrm{A}}{\mathrm{D}}$$

  où :
  - A = nombre de jours courus comptés selon une base mensuelle. Pour les intérêts à l'échéance, le nombre de jours entre la date d'émission et la date d'échéance est utilisé.
  - D = Base Annuelle.

- issue et maturity sont tronqués en entiers.

- basis est arrondi à l'entier le plus proche.

- Une erreur est retournée si :
  - issue ou maturity n'est pas une date valide.
  - issue ≥ maturity.
  - rate ≤ 0.
  - par ≤ 0.
  - basis < 0 ou basis > 4.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
1 avril 2008	Date d'émission
15 juin 2008	Date d'échéance
10 %	Coupon en pourcentage
1 000	Valeur nominale
3	Base Réel/365 (voir ci-dessus)

La requête DAX suivante :

DAX

```
EVALUATE
{
  ACCRINTM(DATE(2008,4,1), DATE(2008,6,15), 0.1, 1000, 3)
}
```

Retourne l'intérêt couru pour un titre aux conditions spécifiées ci-dessus.

Agrandir le tableau

[Valeur]
20,5479452054795

Commentaires

Cette page a-t-elle été utile ? Yes No

AMORDEGRC

Article • 20/10/2023

Retourne l'amortissement pour chaque période comptable. Cette fonction est fournie pour le système comptable français. Si un actif est acheté au milieu de la période comptable, l'amortissement calculé au prorata est pris en compte. La fonction est similaire à AMORLINC, à ceci près qu'un coefficient d'amortissement est appliqué dans le calcul en fonction de la durée de vie des actifs.

Syntaxe

DAX

```
AMORDEGRC(<cost>, <date_purchased>, <first_period>, <salvage>, <period>,
<rate>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
cost	Coût de l'actif.
date_purchased	Date d'achat de l'actif.
first_period	Date de fin de la première période.
salvage	Valeur résiduelle à la fin de vie de l'actif.
period	Période.
rate	Taux d'amortissement.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Système de date
0 ou omis	360 jours (méthode NASD)
1	Réel
3	Année de 365 jours
4	Année de 360 jours (méthode européenne)

Valeur de retour

Amortissement pour chaque période comptable.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.
- Cette fonction retourne l'amortissement jusqu'à la dernière période du cycle de vie des actifs ou jusqu'à ce que la valeur cumulée de l'amortissement soit supérieure au coût des actifs moins la valeur résiduelle.
- Les coefficients d'amortissement sont les suivants :

Agrandir le tableau

Durée de vie des actifs (1/rate)	Coefficient d'amortissement
Entre 3 et 4 ans	1.5
Entre 5 et 6 ans	2
Plus de 6 ans	2.5

- Le taux d'amortissement passe à 50 % pour la période précédant la dernière période et à 100 % pour la dernière période.
- period et basis sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - cost < 0.
  - first_period ou date_purchased n'est pas une date valide.

- date_purchased > first_period.
- salvage < 0 ou salvage > cost.
- period < 0.
- rate ≤ 0.
- La durée de vie (life) des actifs est comprise entre 0 (zéro) et 1, 1 et 2, 2 et 3 ou 4 et 5.
- basis est un nombre différent de 0, 1, 3 ou 4.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
2 400	Coût
19 août 2008	Date d'achat
31 décembre 2008	Fin de la première période
300	Valeur résiduelle
1	Période
15 %	Taux d'amortissement
1	Base Réel (voir ci-dessus)

La requête DAX suivante :

DAX

```
EVALUATE
{
  AMORDEGRC(2400, DATE(2008,8,19), DATE(2008,12,31), 300, 1, 0.15, 1)
}
```

Retourne l'amortissement de la première période selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
776

Commentaires

Cette page a-t-elle été utile ? Yes No

AMORLINC

Article • 20/10/2023

Retourne l'amortissement pour chaque période comptable. Cette fonction est fournie pour le système comptable français. Si un bien est acheté au milieu de la période comptable, l'amortissement calculé au prorata est pris en compte.

Syntaxe

DAX

```
AMORLINC(<cost>, <date_purchased>, <first_period>, <salvage>, <period>,
<rate>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
cost	Coût de l'actif.
date_purchased	Date d'achat de l'actif.
first_period	Date de fin de la première période.
salvage	Valeur résiduelle à la fin de vie de l'actif.
period	Période.
rate	Taux d'amortissement.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Système de date
0 ou omis	360 jours (méthode NASD)

Base	Système de date
1	Réel
3	Année de 365 jours
4	Année de 360 jours (méthode européenne)

Valeur de retour

Amortissement pour chaque période comptable.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. En DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 correspond au jour 39 448, car il s'agit du 39 448 e jour après le 30 décembre 1899.
- period et basis sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - cost < 0.
  - first_period ou date_purchased n'est pas une date valide.
  - date_purchased > first_period.
  - salvage < 0 ou salvage > cost.
  - period < 0.
  - rate ≤ 0.
  - basis est un nombre différent de 0, 1, 3 ou 4.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
2 400	Coût

Données	Description
19 août 2008	Date d'achat
31 décembre 2008	Fin de la première période
300	Valeur résiduelle
1	Période
15 %	Taux d'amortissement
1	Base Réel (voir ci-dessus)

La requête DAX suivante :

```
DAX

EVALUATE
{
  AMORLINC(2400, DATE(2008,8,19), DATE(2008,12,31), 300, 1, 0.15, 1)
}
```

Retourne l'amortissement de la première période selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
360

Commentaires

Cette page a-t-elle été utile ? Yes No

COUPDAYBS

Article • 20/10/2023

Retourne le nombre de jours entre le début d'une période de coupon et sa date de règlement.

Syntaxe

DAX

```
COUPDAYBS(<settlement>, <maturity>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365

Base	Base de comptage des jours
4	Européen 30/360

Valeur de retour

Nombre de jours entre le début de la période d'un coupon et sa date de règlement (settlement).

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.

- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après le 1er janvier 2008, date d'émission.

- settlement et maturity sont tronqués en entiers.

- frequency et basis sont arrondis à l'entier le plus proche.

- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - frequency est un nombre différent de 1, 2 ou 4.
  - basis < 0 ou basis > 4.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
25 janvier 2011	Date de règlement
15 novembre 2011	Date d'échéance
2	Coupon semestriel (voir ci-dessus)
1	Base Réel/réel (voir ci-dessus)

La requête DAX suivante :

DAX

```
EVALUATE
{
  COUPDAYBS(DATE(2011,1,25), DATE(2011,11,15), 2, 1)
}
```

Retourne le nombre de jours entre le début de la période d'un coupon et la date de règlement (settlement) pour une obligation selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
71

Commentaires

Cette page a-t-elle été utile ? Yes No

COUPDAYS

Article • 20/10/2023

Retourne le nombre de jours dans la période du coupon qui contient la date de règlement.

Syntaxe

```
DAX
COUPDAYS(<settlement>, <maturity>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365

Base	Base de comptage des jours
4	Européen 30/360

Valeur de retour

Nombre de jours dans la période du coupon qui contient la date de règlement.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.
- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après le 1er janvier 2008, date d'émission.
- settlement et maturity sont tronqués en entiers.
- frequency et basis sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - frequency est un nombre différent de 1, 2 ou 4.
  - basis < 0 ou basis > 4.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
25 janvier 2011	Date de règlement
15 novembre 2011	Date d'échéance
2	Coupon semestriel (voir ci-dessus)
1	Base Réel/réel (voir ci-dessus)

La requête DAX suivante :

DAX

```
EVALUATE
{
  COUPDAYS(DATE(2011,1,25), DATE(2011,11,15), 2, 1)
}
```

Retourne le nombre de jours dans la période du coupon qui contient la date de règlement pour une obligation selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
181

Commentaires

Cette page a-t-elle été utile ? Yes No

COUPDAYSNC

Article • 20/10/2023

Retourne le nombre de jours entre la date de règlement et la date du prochain coupon.

Syntaxe

DAX

```
COUPDAYSNC(<settlement>, <maturity>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365
4	Européen 30/360

Valeur de retour

Nombre de jours entre la date de règlement et la date du prochain coupon.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.
- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après la date d'émission, le 1er janvier 2008.
- settlement et maturity sont tronqués en entiers.
- frequency et basis sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - frequency est un nombre différent de 1, 2 ou 4.
  - basis < 0 ou basis > 4.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
25 janvier 2011	Date de règlement
15 novembre 2011	Date d'échéance

Données	Description
2	Coupon semestriel (voir ci-dessus)
1	Base Réel/réel (voir ci-dessus)

La requête DAX suivante :

```
DAX

EVALUATE
{
  COUPDAYSNC(DATE(2011,1,25), DATE(2011,11,15), 2, 1)
}
```

Retourne le nombre de jours entre la date de règlement et la date du prochain coupon pour une obligation dont les termes sont spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
110

Commentaires

Cette page a-t-elle été utile ? Yes No

COUPNCD

Article • 20/10/2023

Retourne la date du prochain coupon après la date du règlement.

Syntaxe

DAX

```
COUPNCD(<settlement>, <maturity>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365
4	Européen 30/360

Valeur de retour

Date du prochain coupon après la date du règlement.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.
- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après le 1er janvier 2008, date d'émission.
- settlement et maturity sont tronqués en entiers.
- frequency et basis sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - frequency est un nombre différent de 1, 2 ou 4.
  - basis < 0 ou basis > 4.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
25 janvier 2011	Date de règlement
15 novembre 2011	Date d'échéance

Données	Description
2	Coupon semestriel (voir ci-dessus)
1	Base Réel/réel (voir ci-dessus)

La requête DAX suivante :

```
DAX

EVALUATE
{
  COUPNCD(DATE(2011,1,25), DATE(2011,11,15), 2, 1)
}
```

Retourne la date du coupon suivant après la date de règlement, pour une obligation avec les conditions spécifiées ci-dessus.

Agrandir le tableau

[Valeur]
15/05/2011 12:00:00

Commentaires

Cette page a-t-elle été utile ? Yes No

COUPNUM

Article • 20/10/2023

Retourne le nombre de coupons dus entre la date de règlement et la date d'échéance, arrondi au nombre entier de coupons le plus proche.

Syntaxe

DAX

```
COUPNUM(<settlement>, <maturity>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365

Base	Base de comptage des jours
4	Européen 30/360

Valeur de retour

Nombre de coupons dus entre la date de règlement (settlement) et la date d'échéance (maturity).

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.
- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après la date d'émission, le 1er janvier 2008.
- settlement et maturity sont tronqués en entiers.
- frequency et basis sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - frequency est un nombre différent de 1, 2 ou 4.
  - basis < 0 ou basis > 4.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
25 janvier 2007	Date de règlement
15 novembre 2008	Date d'échéance
2	Coupon semestriel (voir ci-dessus)
1	Base Réel/réel (voir ci-dessus)

La requête DAX suivante :

```DAX
EVALUATE
{
  COUPNUM(DATE(2007,1,25), DATE(2008,11,15), 2, 1)
}
```

Retourne le nombre de versements de coupon pour une obligation aux conditions spécifiées ci-dessus.

Agrandir le tableau

[Valeur]
4

Commentaires

Cette page a-t-elle été utile ? Yes No

COUPPCD

Article • 20/10/2023

Retourne la date de coupon précédente avant la date de règlement.

Syntaxe

DAX

```
COUPPCD(<settlement>, <maturity>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365
4	Européen 30/360

Valeur de retour

Date de coupon précédente avant la date de règlement.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.
- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après la date d'émission, le 1er janvier 2008.
- settlement et maturity sont tronqués en entiers.
- frequency et basis sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - frequency est un nombre différent de 1, 2 ou 4.
  - basis < 0 ou basis > 4.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
25 janvier 2011	Date de règlement
15 novembre 2011	Date d'échéance

Données	Description
2	Coupon semestriel (voir ci-dessus)
1	Base Réel/réel (voir ci-dessus)

La requête DAX suivante :

```
DAX

EVALUATE
{
  COUPPCD(DATE(2011,1,25), DATE(2011,11,15), 2, 1)
}
```

Retourne la date de coupon précédente avant la date de règlement, pour une obligation utilisant les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
15/11/2010 00:00:00

Commentaires

Cette page a-t-elle été utile ? Yes No

CUMIPMT

Article • 20/10/2023

Retourne les intérêts cumulés payés sur un emprunt entre start_period et end_period.

Syntaxe

DAX

```
CUMIPMT(<rate>, <nper>, <pv>, <start_period>, <end_period>, <type>)
```

Paramètres

Agrandir le tableau

Terme	Définition
rate	Taux d'intérêt.
nper	Nombre total de périodes de remboursement.
pv	Valeur actuelle.
start_period	Première période prise en compte dans le calcul. Doit être comprise entre 1 et end_period (incluse).
end_period	Dernière période prise en compte dans le calcul. Doit être comprise entre start_period et nper (inclus).
type	Moment choisi du remboursement. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **type** accepte les valeurs suivantes :

Agrandir le tableau

Type	Minutage
0 (zéro)	Remboursement à la fin de la période
1	Remboursement au début de la période

Valeur de retour

Intérêts cumulés versés au cours de la période spécifiée.

Notes

- Au moment de spécifier rate et nper, veillez à la cohérence des unités que vous utilisez. Si vous effectuez des paiements mensuels pour un emprunt de quatre ans à un taux d'intérêt annuel de 10 pour cent, spécifiez 0,1/12 pour rate et 4*12 pour nper. Si vous effectuez des paiements annuels pour le même prêt, utilisez 0,1 pour rate et 4 pour nper.
- start_period, end_period et type sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - rate ≤ 0.
  - nper < 1.
  - pv ≤ 0.
  - start_period < 1 ou start_period > end_period.
  - end_period < start_period ou end_period > nper.
  - type est un nombre différent de 0 ou 1.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

Agrandir le tableau

Données	Description
9 %	Taux d'intérêt annuel
30	Durée de l'emprunt en années
125 000	Valeur actuelle

Exemple 1

La requête DAX suivante :

```
DAX
```

```
EVALUATE
{
  CUMIPMT(0.09/12, 30*12, 125000, 13, 24, 1)
}
```

Retourne le montant total des intérêts payés la deuxième année de remboursement, des périodes 13 à 24, en partant du principe que les paiements sont effectués au début de chaque mois.

Agrandir le tableau

[Valeur]
-11052.3395838718

Exemple 2

La requête DAX suivante :

DAX

```
EVALUATE
{
  CUMIPMT(0.09/12, 30*12, 125000, 1, 1, 0)
}
```

Retourne les intérêts payés en une seul fois le premier mois, en partant du principe que le paiement est effectué à la fin du mois.

Agrandir le tableau

[Valeur]
-937,5

Commentaires

Cette page a-t-elle été utile ? Yes No

CUMPRINC

Article • 20/10/2023

Retourne le capital accumulé sur un prêt entre start_period et end_period.

Syntaxe

DAX

```
CUMPRINC(<rate>, <nper>, <pv>, <start_period>, <end_period>, <type>)
```

Paramètres

Agrandir le tableau

Terme	Définition
rate	Taux d'intérêt.
nper	Nombre total de périodes de remboursement.
pv	Valeur actuelle.
start_period	Première période prise en compte dans le calcul. Doit être comprise entre 1 et end_period (incluse).
end_period	Dernière période prise en compte dans le calcul. Doit être comprise entre start_period et nper (inclus).
type	Moment choisi du remboursement. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **type** accepte les valeurs suivantes :

Agrandir le tableau

Type	Minutage
0 (zéro)	Remboursement à la fin de la période
1	Remboursement au début de la période

Valeur de retour

Capital accumulé remboursé au cours de la période spécifiée.

Notes

- Au moment de spécifier rate et nper, veillez à la cohérence des unités que vous utilisez. Si vous effectuez des paiements mensuels pour un emprunt de quatre ans à un taux d'intérêt annuel de 10 pour cent, spécifiez 0,1/12 pour rate et 4*12 pour nper. Si vous effectuez des paiements annuels pour le même prêt, utilisez 0,1 pour rate et 4 pour nper.
- start_period, end_period et type sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - rate ≤ 0.
  - nper < 1.
  - pv ≤ 0.
  - start_period < 1 ou start_period > end_period.
  - end_period < start_period ou end_period > nper.
  - type est un nombre différent de 0 ou 1.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

Agrandir le tableau

Données	Description
9 %	Taux d'intérêt annuel
30	Durée en années
125 000	Valeur actuelle

Exemple 1

La requête DAX suivante :

```
DAX
```

```
EVALUATE
{
  CUMPRINC(0.09/12, 30*12, 125000, 13, 24, 1)
}
```

Retourne le montant total du capital remboursé au cours de la deuxième année de remboursement, de la période 13 à 24, en partant du principe que les paiements sont effectués au début de chaque mois.

Agrandir le tableau

[Valeur]
-927,153472378062

Exemple 2

La requête DAX suivante :

DAX

```
EVALUATE
{
  CUMPRINC(0.09/12, 30*12, 125000, 1, 1, 0)
}
```

Retourne le capital remboursé en une seule fois le premier mois, en partant du principe que le paiement est effectué à la fin du mois.

Agrandir le tableau

[Valeur]
-68,2782711809784

Commentaires

Cette page a-t-elle été utile ? Yes No

DB

Article • 20/10/2023

Retourne l'amortissement d'un actif pour une période spécifiée, calculé avec la méthode de l'amortissement dégressif à taux fixe.

Syntaxe

```
DAX

DB(<cost>, <salvage>, <life>, <period>[, <month>])
```

Paramètres

Agrandir le tableau

Terme	Définition
cost	Coût initial du bien.
salvage	Valeur au terme de l'amortissement (parfois appelée valeur résiduelle de l'actif). Cette valeur peut être égale à 0.
à faire parler vos données	Nombre de périodes pendant lesquelles l'actif est amorti (parfois appelé durée de vie utile de l'actif).
period	Période pour laquelle vous souhaitez calculer l'amortissement. Elle doit utiliser les mêmes unités que life. Doit être comprise entre 1 et la valeur de life (incluse).
month	(Facultatif) Nombre de mois dans la première année. Si month est omis, sa valeur supposée est 12.

Valeur de retour

Amortissement sur la période spécifiée.

Notes

- La méthode de l'amortissement dégressif fixe calcule l'amortissement avec un taux fixe. DB utilise la formule suivante pour calculer l'amortissement au cours d'une

période :

$$(\text{cost} - \text{amortissement total des périodes précédentes}) \times \text{rate}$$

où :
- $\text{rate} = 1 - ((\frac{\text{salvage}}{\text{cost}})^{(\frac{1}{\text{life}})})$, arrondi à trois décimales

- L'amortissement sur les première et dernière périodes est un cas particulier.
  - Pour la première période, DB utilise cette formule :

$$\frac{\text{cost} \times \text{rate} \times \text{month}}{12}$$

  - Pour la dernière période, DB utilise cette formule :

$$\frac{(\text{cost} - \text{amortissement total des périodes précédentes}) \times \text{rate} \times (12 - \text{month})}{12}$$

- period et month sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - cost < 0.
  - salvage < 0.
  - life < 1.
  - period < 1 ou period > life.
  - month < 1 ou month > 12.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

Exemple 1

La requête DAX suivante :

DAX

```
EVALUATE
{
  DB(1000000, 0, 6, 1, 2)
}
```

Retourne l'amortissement d'un actif sur les deux derniers mois de la première année, en supposant qu'il sera de 0 USD après 6 ans.

Agrandir le tableau

[Valeur]
166666,666666667

Exemple 2

L'exemple suivant calcule l'amortissement total de tous les actifs sur plusieurs années pendant leur durée de vie. Ici, la première année inclut seulement 7 mois d'amortissement et la dernière année, uniquement 5 mois.

DAX

```
DEFINE
VAR NumDepreciationPeriods = MAX(Asset[LifeTimeYears])+1
VAR DepreciationPeriods = GENERATESERIES(1, NumDepreciationPeriods)
EVALUATE
ADDCOLUMNS (
  DepreciationPeriods,
  "Current Period Total Depreciation",
  SUMX (
    FILTER (
      Asset,
      [Value] <= [LifetimeYears]+1
    ),
    DB([InitialCost], [SalvageValue], [LifetimeYears], [Value], 7)
  )
)
```

Commentaires

Cette page a-t-elle été utile ? Yes No

DDB

Article • 20/10/2023

Retourne l'amortissement d'un actif pour une période spécifiée, calculé selon la méthode de l'amortissement dégressif à taux double ou une autre méthode que vous spécifiez.

Syntaxe

```
DAX

DDB(<cost>, <salvage>, <life>, <period>[, <factor>])
```

Paramètres

Agrandir le tableau

Terme	Définition
cost	Coût initial du bien.
salvage	Valeur au terme de l'amortissement (parfois appelée valeur résiduelle de l'actif). Cette valeur peut être égale à 0.
à faire parler vos données	Nombre de périodes pendant lesquelles l'actif est amorti (parfois appelé durée de vie utile de l'actif).
period	Période pour laquelle vous souhaitez calculer l'amortissement. Elle doit utiliser les mêmes unités que life. Doit être comprise entre 1 et life (inclus).
facteur	(Facultatif) Taux auquel le solde diminue. Si factor est omis, la valeur par défaut est 2 (méthode de l'amortissement dégressif à taux double).

Valeur de retour

Amortissement sur la période spécifiée.

Notes

- La méthode de l'amortissement dégressif à taux double calcule l'amortissement avec un taux accéléré. L'amortissement est au plus haut pendant la première période, puis diminue au cours des périodes suivantes. DDB utilise la formule suivante pour calculer l'amortissement au cours d'une période :

  $$\text{Min}((\text{cost} - \text{amortissement total des périodes précédentes}) \times (\frac{\text{factor}}{\text{life}}), (\text{cost} - \text{salvage} - \text{amortissement total des périodes précé}$$

- Modifiez factor si vous ne souhaitez pas utiliser la méthode de l'amortissement dégressif à taux double.
- Utilisez la fonction VDB pour passer à la méthode de l'amortissement linéaire quand l'amortissement est supérieur à celui obtenu par la méthode de l'amortissement dégressif.
- period est arrondi à l'entier le plus proche.
- Une erreur est retournée si :
  - cost < 0.
  - salvage < 0.
  - life < 1.
  - period < 1 ou period > life.
  - factor ≤ 0.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

Exemple 1

La requête DAX suivante :

```
DAX

EVALUATE
{
  DDB(1000000, 0, 10, 5, 1.5)
}
```

Retourne l'amortissement d'un actif la 5e année, en supposant que sa valeur sera égale à 0 USD après 10 ans. Ce calcul utilise un facteur de 1,5.

Agrandir le tableau

[Valeur]
78300,9375

Exemple 2

L'exemple suivant calcule l'amortissement total de tous les actifs sur plusieurs années pendant leur durée de vie. Ce calcul utilise le facteur par défaut de 2 (méthode de l'amortissement dégressif à taux double).

```
DAX

DEFINE
VAR NumDepreciationPeriods = MAX(Asset[LifeTimeYears])
VAR DepreciationPeriods = GENERATESERIES(1, NumDepreciationPeriods)
EVALUATE
  ADDCOLUMNS (
  DepreciationPeriods,
  "Current Period Total Depreciation",
  SUMX (
    FILTER (
      Asset,
      [Value] <= [LifetimeYears]
    ),
    DDB([InitialCost], [SalvageValue], [LifetimeYears], [Value])
  )
)
```

Commentaires

Cette page a-t-elle été utile ? Yes No

DISC

Article • 20/10/2023

Retourne le taux d'actualisation d'un titre.

Syntaxe

DAX

```
DISC(<settlement>, <maturity>, <pr>, <redemption>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
pr	Cours du titre par valeur nominale de 100 USD.
redemption	Valeur de rachat du titre par valeur nominale de 100 USD.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365

Base	Base de comptage des jours
4	Européen 30/360

Valeur de retour

Taux d'actualisation.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.

- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation de 30 ans a été émise le 1er janvier 2018 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2018, la date de règlement (settlement) le 1er juillet 2018 et la date d'échéance (maturity) le 1er janvier 2048, soit 30 ans après le 1er janvier 2008, la date d'émission.

- DISC se calcule comme suit :

$$\mathrm{DISC} = \frac{\mathrm{redemption} - \mathrm{par}}{\mathrm{redemption}} \times \frac{\mathrm{B}}{\mathrm{DSM}}$$

  où :

  - B = nombre de jours dans une année, selon la base annuelle.
  - DSM = nombre de jours entre le règlement (settlement) et l'échéance (maturity).

- settlement et maturity sont tronqués en entiers.

- basis est arrondi à l'entier le plus proche.

- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - pr ≤ 0.

- redemption ≤ 0.
- basis < 0 ou basis > 4.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
01/07/2018	Date de règlement
01/01/2048	Date d'échéance
97,975	Price
100	Valeur de rachat
1	Base Réel/réel (voir ci-dessus)

La requête DAX suivante :

```
DAX

EVALUATE
{
  DISC(DATE(2018,7,1), DATE(2048,1,1), 97.975, 100, 1)
}
```

Retourne le taux d'actualisation d'une obligation dont les termes sont spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
0,000686384169121348

Commentaires

Cette page a-t-elle été utile ? Yes No

DOLLARDE

Article • 20/10/2023

Convertit un prix en dollars exprimé sous la forme d'une partie entière et d'une partie fractionnaire, comme 1,02, en prix en dollars exprimé sous la forme d'un nombre décimal. Les nombres fractionnaires en dollars sont parfois utilisés pour les cours des titres.

Syntaxe

DAX

```
DOLLARDE(<fractional_dollar>, <fraction>)
```

Paramètres

Agrandir le tableau

Terme	Définition
fractional_dollar	Nombre exprimé sous la forme d'une partie entière et d'une partie fractionnaire, séparées par un symbole décimal.
fraction	Entier à utiliser dans le dénominateur de la fraction.

Valeur de retour

Valeur décimale de *fractional_dollar*.

Notes

- La partie fractionnaire de la valeur est divisée par un entier que vous spécifiez. Par exemple, si vous souhaitez que votre prix soit exprimé avec une précision de 1/16 de dollar, vous divisez la partie fractionnaire par 16. Dans ce cas, 1.02 représente $1.125 ($1 + 2/16 = $1.125).
- fraction est arrondi à l'entier le plus proche.
- Une erreur est retournée si :
  - fraction < 1.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La requête DAX suivante :

DAX

```
EVALUATE
{
  DOLLARDE(1.02, 16)
}
```

Retourne 1,125, le prix décimal du prix fractionnaire d’origine, 1,02, lu comme 1 et 2/16. Comme la fraction a la valeur 16, le prix a une précision de 1/16 de dollar.

Commentaires

Cette page a-t-elle été utile ? Yes No

DOLLARFR

Article • 20/10/2023

Convertit un prix en dollars exprimé sous la forme d'un nombre décimal en prix en dollars exprimé sous la forme d'une partie entière et d'une partie fractionnaire, comme 1,02. Les montants en dollars fractionnaires sont parfois utilisés pour les cours des titres.

Syntaxe

DAX

```
DOLLARFR(<decimal_dollar>, <fraction>)
```

Paramètres

Agrandir le tableau

Terme	Définition
decimal_dollar	Nombre décimal.
fraction	Entier à utiliser dans le dénominateur de la fraction.

Valeur de retour

Valeur fractionnaire de *decimal_dollar*, exprimée sous la forme d'une partie entière et d'une partie fractionnaire.

Notes

- fraction est arrondi à l'entier le plus proche.
- Une erreur est retournée si :
  - fraction < 1.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La requête DAX suivante :

```
DAX
EVALUATE
{
  DOLLARFR(1.125, 16)
}
```

Retourne 1,02, qui se lit 1 et 2/16, qui est le prix fractionnaire correspondant du prix décimal d'origine, 1,125. Comme la valeur de fraction est égale à 16, le prix a une précision de 1/16 de dollar.

Commentaires

Cette page a-t-elle été utile ? Yes No

DURATION

Article • 20/10/2023

Retourne la durée de Macaulay pour une valeur nominale hypothétique de 100 USD. La durée est définie comme la moyenne pondérée de la valeur actuelle des flux de trésorerie et est utilisée comme mesure de la réponse d'un prix d'obligation aux changements de rendement.

Syntaxe

DAX

```
DURATION(<settlement>, <maturity>, <coupon>, <yld>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
coupon	Taux d'intérêt nominal annuel du titre.
yld	Rendement annuel du titre.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360

Base	Base de comptage des jours
1	Réel/réel
2	Réel/360
3	Réel/365
4	Européen 30/360

Valeur de retour

Durée de Macaulay.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.
- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après la date d'émission, le 1er janvier 2008.
- settlement et maturity sont tronqués en entiers.
- frequency et basis sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - coupon < 0.
  - yld < 0
  - frequency est un nombre différent de 1, 2 ou 4.
  - basis < 0 ou basis > 4.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au

niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
01/07/2018	Date de règlement
01/01/2048	Date d'échéance
8.0%	Coupon (pourcentage)
9.0%	Taux de rendement
2	La fréquence est semestrielle (voir ci-dessus)
1	Base Réel/réel (voir ci-dessus)

La requête DAX suivante :

DAX

```
EVALUATE
{
  DURATION(DATE(2018,7,1), DATE(2048,1,1), 0.08, 0.09, 2, 1)
}
```

Retourne la durée de Macaulay d'une obligation selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
10,9191452815919

Commentaires

Cette page a-t-elle été utile ? Yes No

EFFECT

Article • 20/10/2023

Retourne le taux d'intérêt annuel effectif, compte tenu du taux d'intérêt annuel nominal et du nombre de périodes de calcul de l'intérêt par an.

Syntaxe

DAX

```
EFFECT(<nominal_rate>, <npery>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nominal_rate	Taux d'intérêt nominal.
npery	Nombre de périodes de calcul de l'intérêt par an.

Valeur de retour

Taux d'intérêt annuel effectif.

Notes

- EFFECT se calcule comme suit :

$$\text{EFFECT} = \left(1 + \frac{\text{nominal_rate}}{\text{npery}}\right)^{\text{npery}} - 1$$

- npery est arrondi à l'entier le plus proche.
- Une erreur est retournée si :
  - nominal_rate ≤ 0.
  - npery < 1.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
5,25 %	Taux d’intérêt nominal
4	Nombre de périodes de calcul de l’intérêt par an

La requête DAX suivante :

DAX

```
EVALUATE
{
  EFFECT(0.0525, 4)
}
```

Retourne le taux d’intérêt effectif selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
0,0535426673707584

Commentaires

Cette page a-t-elle été utile ? Yes No

VC

Article • 20/10/2023

Calcule la valeur future d'un investissement sur la base d'un taux d'intérêt constant. Vous pouvez utiliser la fonction FV pour des paiements périodiques, des paiements constants et/ou un paiement forfaitaire unique.

Syntaxe

DAX

```
FV(<rate>, <nper>, <pmt>[, <pv>[, <type>]])
```

Paramètres

Agrandir le tableau

Terme	Définition
rate	Taux d'intérêt par période.
nper	Nombre total de périodes de paiement dans une annuité.
pmt	Paiement effectué à chaque période ; il ne peut pas changer pendant la durée de l'annuité. En général, pmt inclut le principal et les intérêts, mais pas d'autres frais ou taxes.
pv	(Facultatif) Valeur actuelle ou montant forfaitaire actuel d'une série de paiements futurs. Si pv est omis, sa valeur supposée est BLANK.
type	(Facultatif) Valeur 0 ou 1 qui indique à quel moment les paiements sont dus. Si type est omis, la valeur par défaut est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **type** accepte les valeurs suivantes :

Agrandir le tableau

Définir le type avec la valeur	Si les paiements sont dus
0 ou omis	À la fin de la période
1	Au début de la période

Remarque : Pour avoir une description plus complète des arguments de la fonction FV ainsi que des informations supplémentaires sur les fonctions d'annuité, consultez la fonction PV.

Valeur de retour

Valeur future d'un investissement.

Notes

- Au moment de spécifier rate et nper, veillez à la cohérence des unités que vous utilisez. Si vous effectuez des paiements mensuels pour un prêt de quatre ans à un taux d'intérêt annuel de 12 %, spécifiez 0,12/12 pour rate et 4*12 pour nper. Si vous effectuez des paiements annuels pour le même prêt, utilisez 0,12 pour rate et 4 pour nper.
- Pour tous les arguments, les décaissements, tels que les dépôts sur un compte d'épargne, sont représentés par des nombres négatifs alors que les encaissements, tels que les versements de dividendes, sont représentés par des nombres positifs.
- type est arrondi à l'entier le plus proche.
- Une erreur est retournée si :
  - nper < 1
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
6 %	Taux d'intérêt annuel
10	Nombre de paiements
-200	Montant du paiement
-500	Valeur actuelle

Données	Description
1	Le paiement est dû au début de la période (0 indique que le paiement est dû à la fin de la période)

La requête DAX suivante :

```
DAX

EVALUATE
{
  FV(0.06/12, 10, -200, -500, 1)
}
```

Retourne la valeur future d'un investissement selon les conditions spécifiées ci-dessus.

Agrandir le tableau

[Valeur]
2581,40337406012

Commentaires

Cette page a-t-elle été utile ? Yes No

INTRATE

Article • 20/10/2023

Retourne le taux d'intérêt d'un titre totalement investi.

Syntaxe

DAX

```
INTRATE(<settlement>, <maturity>, <investment>, <redemption>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
investment	Montant investi dans le titre.
redemption	Montant à percevoir à l'échéance.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365

Base	Base de comptage des jours
4	Européen 30/360

Valeur de retour

Taux d'intérêt.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.

- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après la date d'émission, le 1er janvier 2008.

- INTRATE se calcule comme suit :

$$\mathrm{INTRATE} = \frac{\mathrm{redemption} - \mathrm{investment}}{\mathrm{investment}} \times \frac{\mathrm{B}}{\mathrm{DIM}}$$

  où :
  - B = nombre de jours dans une année, selon la base annuelle.
  - DIM = nombre de jours entre la date de règlement (settlement) et la date d'échéance (maturity).

- settlement et maturity sont tronqués en entiers.

- basis est arrondi à l'entier le plus proche.

- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - investment ≤ 0.
  - redemption ≤ 0.
  - basis < 0 ou basis > 4.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
15/2/2008	Date de règlement
15/5/2008	Date d'échéance
$1,000,000	Investissement
$1,014,420	Valeur de rachat
2	Base Réel/360

La requête DAX suivante :

```
DAX

EVALUATE
{
  INTRATE(DATE(2008,2,15), DATE(2008,5,15), 1000000, 1014420, 2)
}
```

Retourne le taux d'actualisation d'une obligation selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
0,05768

Commentaires

Cette page a-t-elle été utile ? Yes No

IPMT

Article • 20/10/2023

Retourne les intérêts payés sur une période donnée pour un investissement avec des paiements périodiques constants et un taux d'intérêt constant.

Syntaxe

DAX

```
IPMT(<rate>, <per>, <nper>, <pv>[, <fv>[, <type>]])
```

Paramètres

Agrandir le tableau

Terme	Définition
rate	Taux d'intérêt par période.
per	Période pour laquelle vous souhaitez déterminer l'intérêt. Doit être comprise entre 1 et nper (inclus).
nper	Nombre total de périodes de paiement dans une annuité.
pv	Valeur actuelle ou montant forfaitaire actuel d'une série de paiements futurs.
fv	(Facultatif) Valeur capitalisée ou solde de caisse que vous souhaitez obtenir après le dernier paiement. Si fv est omis, sa valeur présumée est BLANK.
type	(Facultatif) Valeur 0 ou 1 qui indique à quel moment les paiements sont dus. Si type est omis, la valeur par défaut est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **type** accepte les valeurs suivantes :

Agrandir le tableau

Définir le type avec la valeur	Si les paiements sont dus
0 ou omis	À la fin de la période
1	Au début de la période

Valeur de retour

Paiement des intérêts pour la période donnée.

Notes

- Au moment de spécifier rate et nper, veillez à la cohérence des unités que vous utilisez. Si vous effectuez des paiements mensuels pour un prêt de quatre ans à un taux d'intérêt annuel de 12 %, spécifiez 0,12/12 pour rate et 4*12 pour nper. Si vous effectuez des paiements annuels pour le même prêt, utilisez 0,12 pour rate et 4 pour nper.
- Pour tous les arguments, les décaissements, tels que les dépôts sur un compte d'épargne, sont représentés par des nombres négatifs alors que les encaissements, tels que les versements de dividendes, sont représentés par des nombres positifs.
- type est arrondi à l'entier le plus proche.
- Une erreur est retournée si :
  - per < 1 ou per > nper
  - nper < 1
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

Agrandir le tableau

Données	Description
10,00 %	Taux d'intérêt annuel
3	Nombre d'années du prêt
$8,000	Valeur actuelle du prêt

Exemple 1

La requête DAX suivante :

```
DAX
```

```
EVALUATE
{
  IPMT(0.1/12, 1, 3*12, 8000)
}
```

Retourne les intérêts mensuels dus le premier mois pour un prêt avec les conditions spécifiées ci-dessus.

Agrandir le tableau

[Valeur]
-66,6666666666667

Exemple 2

La requête DAX suivante :

DAX

```
EVALUATE
{
  IPMT(0.1, 3, 3, 8000)
}
```

Retourne les intérêts annuels dus la dernière année pour un prêt avec les conditions spécifiées ci-dessus, où les remboursements sont effectués annuellement.

Agrandir le tableau

[Valeur]
-292,447129909366

Commentaires

Cette page a-t-elle été utile ? Yes No

ISPMT

Article • 20/10/2023

Calcule les intérêts payés (ou reçus) pour la période spécifiée d'un prêt (ou d'un investissement) avec des paiements de principal identiques.

Syntaxe

DAX

```
ISPMT(<rate>, <per>, <nper>, <pv>)
```

Paramètres

Agrandir le tableau

Terme	Définition
rate	Taux d'intérêt pour l'investissement.
per	Période pour laquelle vous souhaitez trouver l'intérêt. Doit être comprise entre 0 et nper-1 (inclus).
nper	Nombre total de périodes de paiement pour l'investissement.
pv	Valeur actuelle de l'investissement. Pour un prêt, pv est le montant du prêt.

Valeur de retour

Intérêts payés (ou reçus) pour la période spécifiée.

Notes

- Au moment de spécifier rate et nper, veillez à la cohérence des unités que vous utilisez. Si vous effectuez des paiements mensuels pour un emprunt de quatre ans à un taux d'intérêt annuel de 12 %, spécifiez 0,12/12 pour rate et 4*12 pour nper. Si vous effectuez des paiements annuels pour le même prêt, utilisez 0,12 pour rate et 4 pour nper.

- Pour tous les arguments, les décaissements, tels que les dépôts sur un compte d'épargne ou tout autre retrait, sont représentés par des nombres négatifs alors que les encaissements, tels que les chèques de dividendes et autres dépôts, sont représentés par des nombres positifs.
- ISPMT compte chaque période commençant par zéro, et non pas un.
- La plupart des prêts utilisent un calendrier de remboursement avec des paiements réguliers. La fonction IPMT retourne le paiement des intérêts sur une période donnée pour ce type de prêt.
- Certains prêts utilisent un calendrier de remboursement avec des paiements de principal identiques. La fonction ISPMT retourne le paiement des intérêts sur une période donnée pour ce type de prêt.
- Une erreur est retournée si :
  - nper = 0.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
$4,000	Valeur actuelle
4	Nombre de périodes
10 %	Tarif

Pour vous montrer quand utiliser ISPMT, le tableau d'amortissement ci-dessous utilise un calendrier de remboursement du principal avec des paiements identiques selon les termes spécifiés ci-dessus. Les frais d'intérêt pour chaque période sont calculés en multipliant le taux par le solde impayé de la période précédente. Quant au paiement pour chaque période, il est égal au montant du principal (identique d'une période à l'autre) plus l'intérêt pour la période.

Agrandir le tableau

Période	Paiement du principal	Paiement des intérêts	Paiement total	Balance
				4 000,00
1	1 000,00	400,00	1 400,00	3 000,00
2	1 000,00	300,00	1 300,00	2 000,00
3	1 000,00	200.00	1 200,00	1 000,00
4	1 000,00	100,00	1 100,00	0,00

La requête DAX suivante :

DAX

```
DEFINE
VAR NumPaymentPeriods = 4
VAR PaymentPeriods = GENERATESERIES(0, NumPaymentPeriods-1)
EVALUATE
ADDCOLUMNS (
  PaymentPeriods,
  "Interest Payment",
  ISPMT(0.1, [Value], NumPaymentPeriods, 4000)
)
```

Retourne l'intérêt payé au cours de chaque période, calculé selon le programme de remboursement du principal avec des paiements identiques et les termes spécifiés ci-dessus. Les valeurs sont négatives pour indiquer qu'il s'agit d'un intérêt payé, non reçu.

Agrandir le tableau

[Valeur]	[Paiement des intérêts]
0	-400
1	-300
2	-200
3	-100

Commentaires

Cette page a-t-elle été utile ? Yes No

MDURATION

Article • 20/10/2023

Retourne la durée de Macauley modifiée pour un titre ayant une valeur nominale hypothétique de 100 USD.

Syntaxe

DAX

```
MDURATION(<settlement>, <maturity>, <coupon>, <yld>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
coupon	Taux d'intérêt nominal annuel du titre.
yld	Rendement annuel du titre.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel

Base	Base de comptage des jours
2	Réel/360
3	Réel/365
4	Européen 30/360

Valeur de retour

Durée de Macauley modifiée.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.

- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après la date d'émission, le 1er janvier 2008.

- La durée modifiée est définie comme suit :

$$\text{MDURATION} = \frac{\text{DURATION}}{1 + (\frac{\text{Market yield}}{\text{Coupon payments per year}})}$$

- settlement et maturity sont tronqués en entiers.

- frequency et basis sont arrondis à l'entier le plus proche.

- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - coupon < 0.
  - yld < 0
  - frequency est un nombre différent de 1, 2 ou 4.
  - basis < 0 ou basis > 4.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
1/1/2008	Date de règlement
1/1/2016	Date d’échéance
8 %	Coupon en pourcentage
9 %	Taux de rendement
2	La fréquence est semestrielle (voir ci-dessus)
1	Base Réel/réel (voir ci-dessus)

La requête DAX suivante :

DAX

```
EVALUATE
{
  MDURATION(DATE(2008,1,1), DATE(2016,1,1), 0.08, 0.09, 2, 1)
}
```

Retourne la durée de Macaulay modifiée pour une obligation aux conditions spécifiées ci-dessus.

Agrandir le tableau

[Valeur]
5,73566981391884

Commentaires

Cette page a-t-elle été utile ? Yes No

NOMINAL

Article • 20/10/2023

Retourne le taux d'intérêt annuel nominal en fonction du taux effectif et du nombre de périodes de calcul de l'intérêt par an.

Syntaxe

```
DAX
NOMINAL(<effect_rate>, <npery>)
```

Paramètres

Agrandir le tableau

Terme	Définition
effect_rate	Taux d'intérêt effectif.
npery	Nombre de périodes de calcul de l'intérêt par an.

Valeur de retour

Taux d'intérêt annuel nominal.

Notes

- La relation entre NOMINAL et EFFET est illustrée dans l'équation suivante :

$$\text{EFFECT} = \left(1 + \frac{\text{nominal_rate}}{\text{npery}}\right)^{\text{npery}} - 1$$

- npery est arrondi à l'entier le plus proche.
- Une erreur est retournée si :
  - effect_rate ≤ 0.
  - npery < 1.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au

niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
5,3543 %	Taux d'intérêt effectif
4	Nombre de périodes de calcul de l'intérêt par an

La requête DAX suivante :

DAX

```
EVALUATE
{
  NOMINAL(0.053543, 4)
}
```

Retourne le taux d'intérêt effectif selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
0,052500319868356

Commentaires

Cette page a-t-elle été utile ? Yes No

NPER

Article • 20/10/2023

Retourne le nombre de périodes pour un investissement avec des paiements périodiques constants et un taux d'intérêt constant.

Syntaxe

DAX

```
NPER(<rate>, <pmt>, <pv>[, <fv>[, <type>]])
```

Paramètres

Agrandir le tableau

Terme	Définition
rate	Taux d'intérêt par période.
pmt	Paiement effectué à chaque période ; il ne peut pas changer pendant la durée de l'annuité. En général, pmt inclut le principal et les intérêts, mais pas d'autres frais ou taxes.
pv	Valeur actuelle ou montant forfaitaire actuel d'une série de paiements futurs.
fv	(Facultatif) Valeur capitalisée ou solde de caisse que vous souhaitez obtenir après le dernier paiement. Si fv est omis, sa valeur présumée est BLANK.
type	(Facultatif) Valeur 0 ou 1 qui indique à quel moment les paiements sont dus. Si type est omis, la valeur par défaut est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **type** accepte les valeurs suivantes :

Agrandir le tableau

Définir le type avec la valeur	Si les paiements sont dus
0 ou omis	À la fin de la période
1	Au début de la période

Valeur de retour

Nombre de périodes d'un investissement.

Notes

- type est arrondi à l'entier le plus proche.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
12 %	Taux d'intérêt annuel
-100	Paiement effectué à chaque période
-1 000	Valeur actuelle
10000	Valeur capitalisée
1	Le paiement est dû au début de la période (voir ci-dessus)

La requête DAX suivante :

DAX

```
EVALUATE
{
  NPER(0.12/12, -100, -1000, 10000, 1)
}
```

Retourne le nombre de périodes de l'investissement décrit par les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
59,6738656742946

Commentaires

Cette page a-t-elle été utile ? Yes No

ODDFPRICE

Article • 20/10/2023

Retourne le cours par valeur nominale de 100 USD d'un titre dont la première période est décalée (courte ou longue).

Syntaxe

```
DAX

ODDFPRICE(<settlement>, <maturity>, <issue>, <first_coupon>, <rate>, <yld>, <redemption>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
problème	Date d'émission du titre.
first_coupon	Date du premier coupon du titre.
rate	Taux d'intérêt du titre.
yld	Rendement annuel du titre.
redemption	Valeur de rachat du titre par valeur nominale de 100 USD.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365
4	Européen 30/360

Valeur de retour

Cours par valeur nominale de 100 USD.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.
- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) est le 1er juillet 2008 et la date d'échéance (maturity) est le 1er janvier 2038, soit 30 ans après le 1er janvier 2008, la date d'émission.

- ODDFPRICE se calcule comme suit :

  Premier coupon à détachement précoce :

$$\mathrm{ODDFPRICE} = \left[\frac{\mathrm{redemption}}{(1+\frac{\mathrm{yld}}{\mathrm{frequency}})^{(N-1+\frac{\mathrm{DSC}}{\mathrm{E}})}}\right] + \left[\frac{100 \times \frac{\mathrm{rate}}{\mathrm{frequency}} \times \frac{\mathrm{DFC}}{\mathrm{E}}}{(1+\frac{\mathrm{yld}}{\mathrm{frequency}})^{(\frac{\mathrm{DSC}}{\mathrm{E}})}}\right] + \left[\sum_{k=2}^{N} \frac{100 \times \frac{\mathrm{rate}}{\mathrm{frequency}}}{(1+\frac{\mathrm{yld}}{\mathrm{frequency}})^{(k-1+\frac{\mathrm{DSC}}{\mathrm{E}})}}\right] - \left[100 \times \frac{\mathrm{rate}}{\mathrm{frequency}} \times \right.$$

  où :
  - A = nombre de jours entre le début de la période de coupon et la date de règlement (jours courus).
  - DSC = nombre de jours entre le règlement et la date du prochain coupon.
  - DFC = nombre de jours entre le début du premier coupon décalé et la date du premier coupon.
  - E = nombre de jours dans la période de coupon.
  - N = nombre de coupons payables entre la date de règlement et la date de rachat. (Si ce nombre contient une fraction, il est arrondi au nombre entier supérieur.)

  Premier coupon à détachement tardif :

$$\mathrm{ODDFPRICE} = \left[\frac{\mathrm{redemption}}{(1+\frac{\mathrm{yld}}{\mathrm{frequency}})^{(\mathrm{N}+\mathrm{N}_q+\frac{\mathrm{DSC}}{\mathrm{E}})}}\right] + \left[\frac{100 \times \frac{\mathrm{rate}}{\mathrm{frequency}} \times \left[\sum_{i=1}^{\mathrm{NC}} \frac{\mathrm{DC}_i}{\mathrm{NL}_i}\right]}{(1+\frac{\mathrm{yld}}{\mathrm{frequency}})^{(\mathrm{N}_q+\frac{\mathrm{DSC}}{\mathrm{E}})}}\right] + \left[\sum_{k=1}^{\mathrm{N}} \frac{100 \times \frac{\mathrm{rate}}{\mathrm{frequency}}}{(1+\frac{\mathrm{yld}}{\mathrm{frequency}})^{(k-\mathrm{N}_q+\frac{\mathrm{DSC}}{\mathrm{E}})}}\right] - \left[100 \times \overline{\mathrm{fr}}\right.$$

  où :
  - A_i = nombre de jours depuis le début de la i^e ou dernière période de quasi-coupon dans la période décalée.
  - DC_i = nombre de jours entre la date de valeur (ou date d'émission) et le premier quasi-coupon ($i = 1$) ou nombre de jours dans le quasi-coupon ($i = 2$,..., $i = \mathrm{NC}$).
  - DSC = nombre de jours entre le règlement et la date du prochain coupon.
  - E = nombre de jours dans la période de coupon.
  - N = nombre de coupons payables entre la date du premier coupon réel et la date de rachat. (Si ce nombre contient une fraction, il est arrondi au nombre entier supérieur.)
  - NC = nombre de périodes de quasi-coupon comprises dans la période décalée. (Si ce nombre contient une fraction, il est arrondi au nombre entier supérieur.)
  - NL_i = longueur normale en jours de la dernière ou i^e période complète de quasi-coupon dans la période décalée.
  - N_q = nombre de périodes de quasi-coupon entières entre la date de règlement et le premier coupon.

- settlement, maturity, issue et first_coupon sont tronqués en entiers.
- basis et frequency sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - settlement, maturity, issue ou first_coupon n'est pas une valeur valide.
  - l'ordre maturity > first_coupon > settlement > issue n'est pas respecté.
  - rate < 0.
  - yld < 0.
  - redemption ≤ 0.
  - frequency est un nombre différent de 1, 2 ou 4.
  - basis < 0 ou basis > 4.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description des arguments
11/11/2008	Date de règlement
1/3/2021	Date d'échéance
15/10/2008	Date d'émission
1/3/2009	Date du premier coupon
7,85 %	Coupon en pourcentage
6,25 %	Rendement en pourcentage

Données	Description des arguments
$100.00	Valeur de rachat
2	Fréquence semestrielle
1	Base Réel/réel

La requête DAX suivante :

DAX

```
EVALUATE
{
  ODDFPRICE(DATE(2008,11,11), DATE(2021,3,1), DATE(2008,10,15), DATE(2009,3,1), 0.0785, 0.0625, 100.00, 2, 1)
}
```

Retourne le cours par valeur nominale de 100 USD d'un titre dont la première période est décalée (courte ou longue) selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
113,597717474079

Commentaires

Cette page a-t-elle été utile ? Yes No

ODDFYIELD

Article • 20/10/2023

Retourne le rendement d'un titre dont la première période est inhabituelle (courte ou longue).

Syntaxe

```
DAX

ODDFYIELD(<settlement>, <maturity>, <issue>, <first_coupon>, <rate>, <pr>,
<redemption>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
problème	Date d'émission du titre.
first_coupon	Date du premier coupon du titre.
rate	Taux d'intérêt du titre.
pr	Prix du titre.
redemption	Valeur de rachat du titre par valeur nominale de 100 USD.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365
4	Européen 30/360

Valeur de retour

Rendement du titre.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.

- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après le 1er janvier 2008, date d'émission.

- ODDFYIELD se calcule selon une méthode itérative. Elle utilise la méthode Newton basée sur la formule employée pour la fonction ODDFPRICE. Le rendement change en 100 itérations jusqu'à ce que le prix estimé avec le rendement donné soit proche du prix. Consultez ODDFPRICE pour connaître la formule utilisée par ODDFYIELD.

- settlement, maturity, issue et first_coupon sont tronqués en entiers.

- basis et frequency sont arrondis à l'entier le plus proche.

- Une erreur est retournée si :
  - settlement, maturity, issue ou first_coupon n'est pas une valeur valide.
  - l'ordre maturity > first_coupon > settlement > issue n'est pas respecté.

    - rate < 0.
    - pr ≤ 0.
    - redemption ≤ 0.
    - frequency est un nombre différent de 1, 2 ou 4.
    - basis < 0 ou basis > 4.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description des arguments
11 novembre 2008	Date de règlement
1 mars 2021	Date d'échéance
15 octobre 2008	Date d'émission
1 mars 2009	Date du premier coupon
5,75 %	Coupon en pourcentage
84,50	Price
100	Valeur de rachat
2	Fréquence semestrielle
0	Base 30/360

La requête DAX suivante :

DAX

```
EVALUATE
{
  ODDFYIELD(DATE(2008,11,11), DATE(2021,3,1), DATE(2008,10,15),
DATE(2009,3,1), 0.0575, 84.50, 100, 2, 0)
}
```

Retourne le rendement d'un titre dont la première période est inhabituelle (courte ou longue), selon les conditions spécifiées ci-dessus.

Agrandir le tableau

[Valeur]
0,0772455415972989

Commentaires

Cette page a-t-elle été utile ? Yes No

ODDLPRICE

Article • 20/10/2023

Retourne le prix par valeur nominale de 100 USD d'un titre dont la période du dernier coupon est inhabituelle (courte ou longue).

Syntaxe

DAX

```
ODDLPRICE(<settlement>, <maturity>, <last_interest>, <rate>, <yld>,
<redemption>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
last_interest	Date du dernier coupon du titre.
rate	Taux d'intérêt du titre.
yld	Rendement annuel du titre.
redemption	Valeur de rachat du titre par valeur nominale de 100 USD.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365
4	Européen 30/360

Valeur de retour

Cours par valeur nominale de 100 USD.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.
- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après la date d'émission, le 1er janvier 2008.
- settlement, maturity et last_interest sont tronqués en entiers.
- basis et frequency sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - settlement, maturity ou last_interest n'est pas une date valide.
  - maturity > settlement > last_interest n'est pas respecté.
  - rate < 0.
  - yld < 0.
  - redemption ≤ 0.
  - frequency est un nombre différent de 1, 2 ou 4.
  - basis < 0 ou basis > 4.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La requête DAX suivante :

Agrandir le tableau

Données	Description des arguments
7 février 2008	Date de règlement
15 juin 2008	Date d’échéance
15 octobre 2007	Date du dernier intérêt
3.75%	Coupon (pourcentage)
4,05 %	Taux de rendement
$100	Valeur de rachat
2	Fréquence semestrielle
0	Base 30/360

DAX

```
EVALUATE
{
  ODDLPRICE(DATE(2008,2,7), DATE(2008,6,15), DATE(2007,10,15), 0.0375,
0.0405, 100, 2, 0)
}
```

Retourne le prix par valeur nominale de 100 USD d’un titre dont la période du dernier coupon est inhabituelle (courte ou longue), selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
99,8782860147213

Commentaires

Cette page a-t-elle été utile ? Yes No

ODDLYIELD

Article • 20/10/2023

Retourne le rendement d'un titre dont la dernière période est inhabituelle (courte ou longue).

Syntaxe

```
DAX

ODDLYIELD(<settlement>, <maturity>, <last_interest>, <rate>, <pr>, <redemption>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
last_interest	Date du dernier coupon du titre.
rate	Taux d'intérêt du titre.
pr	Prix du titre.
redemption	Valeur de rachat du titre par valeur nominale de 100 USD.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365
4	Européen 30/360

Valeur de retour

Rendement du titre.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour

après le 30 décembre 1899.

- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après le 1er janvier 2008, date d'émission.

- ODDLYIELD se calcule comme suit :

$$\text{ODDLYIELD} = \left[\frac{(\text{redemption} + ((\sum_{i=1}^{\text{NC}} \frac{\text{DC}_i}{\text{NL}_i}) \times \frac{100 \times \text{rate}}{\text{frequency}})) - (\text{par} + ((\sum_{i=1}^{\text{NC}} \frac{\text{A}_i}{\text{NL}_i}) \times \frac{100 \times \text{rate}}{\text{frequency}}))}{\text{par} + ((\sum_{i=1}^{\text{NC}} \frac{\text{A}_i}{\text{NL}_i}) \times \frac{100 \times \text{rate}}{\text{frequency}})}\right] \times \left[\frac{\text{frequency}}{(\sum_{i=1}^{\text{NC}} \frac{\text{DSC}_i}{\text{NL}_i})}\right]$$

  où :
  - A_i = nombre de jours courus pour la i^e ou dernière période de quasi-coupon incluse dans la période irrégulière, en partant de la date du dernier intérêt avant le rachat.
  - DC_i = nombre de jours comptés dans la i^e ou dernière période de quasi-coupon, délimitée par la durée de la période de coupon actuelle.
  - NC = nombre de périodes de quasi-coupon incluses dans la période irrégulière ; si ce nombre comprend une fraction, il est élevé au nombre entier immédiatement supérieur.
  - NL_i = durée normale en jours de la i^e ou dernière période de quasi-coupon incluse dans la période de coupon irrégulière.

- settlement, maturity et last_interest sont tronqués en entiers.

- basis et frequency sont arrondis à l'entier le plus proche.

- Une erreur est retournée si :
  - settlement, maturity ou last_interest n'est pas une date valide.
  - maturity > settlement > last_interest n'est pas respecté.
  - rate < 0.
  - pr ≤ 0.
  - redemption ≤ 0.
  - frequency est un nombre différent de 1, 2 ou 4.
  - basis < 0 ou basis > 4.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La requête DAX suivante :

Agrandir le tableau

Données	Description des arguments
20/04/2008	Date de règlement
15/06/2008	Date d'échéance
24/12/2007	Date du dernier intérêt
3.75%	Coupon (pourcentage)
$99.875	Price
$100	Valeur de rachat
2	Fréquence semestrielle

Données	Description des arguments
0	Base 30/360

DAX

```
EVALUATE
{
  ODDLYIELD(DATE(2008,4,20), DATE(2008,6,15), DATE(2007,12,24), 0.0375, 99.875, 100, 2, 0)
}
```

Retourne le rendement d'un titre dont la dernière période est inhabituelle (courte ou longue), selon les conditions spécifiées ci-dessus.

Agrandir le tableau

[Valeur]
0,0451922356291692

Commentaires

Cette page a-t-elle été utile ? Yes No

PDURATION

Article • 20/10/2023

Retourne le nombre de périodes requises par un investissement pour atteindre une valeur spécifiée.

Syntaxe

DAX

```
PDURATION(<rate>, <pv>, <fv>)
```

Paramètres

Agrandir le tableau

Terme	Définition
rate	Taux d'intérêt par période.
pv	Valeur actuelle de l'investissement.
fv	Valeur future souhaitée de l'investissement.

Valeur de retour

Nombre de périodes.

Notes

- PDURATION utilise l'équation suivante :

$$\text{PDURATION} = \frac{log(\text{fv}) - log(\text{pv})}{log(1 + \text{rate})}$$

- Une erreur est retournée si :
  - rate ≤ 0.
  - pv ≤ 0.
  - fv ≤ 0.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple 1

La requête DAX suivante :

```
DAX

EVALUATE
{
  PDURATION(0.025, 2000, 2200)
}
```

Retourne le nombre d’années requises pour qu’un investissement de 2 000 USD, rapportant 2,5 % par an, atteigne 2 200 USD.

Agrandir le tableau

[Valeur]
3,85986616262266

Exemple 2

La requête DAX suivante :

```
DAX

EVALUATE
{
  PDURATION(0.025/12, 1000, 1200)
}
```

Retourne le nombre de mois requis pour qu’un investissement de 1 000 USD, rapportant 2,5 % par an, atteigne 1 200 USD.

Agrandir le tableau

[Valeur]
87,6054764193714

Commentaires

Cette page a-t-elle été utile ? Yes No

PMT

Article • 20/10/2023

Calcule le remboursement d'un emprunt sur la base de paiements constants et d'un taux d'intérêt constant.

Syntaxe

DAX

```
PMT(<rate>, <nper>, <pv>[, <fv>[, <type>]])
```

Paramètres

Agrandir le tableau

Terme	Définition
rate	Taux d'intérêt de l'emprunt.
nper	Nombre total de paiements pour l'emprunt.
pv	Valeur actuelle ou montant total que représente à la date d'aujourd'hui une série de remboursements futurs ; il s'agit du capital.
fv	(Facultatif) Valeur capitalisée ou solde de caisse que vous souhaitez obtenir après le dernier paiement. Si fv est omis, sa valeur présumée est BLANK.
type	(Facultatif) Valeur 0 ou 1 qui indique à quel moment les paiements sont dus. Si type est omis, la valeur par défaut est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **type** accepte les valeurs suivantes :

Agrandir le tableau

Définir le type avec la valeur	Si les paiements sont dus
0 ou omis	À la fin de la période
1	Au début de la période

Remarque : Pour une description plus complète des arguments de PMT, consultez la fonction PV.

Valeur de retour

Montant d'un seul remboursement d'emprunt.

Notes

- Le paiement retourné par PMT comprend le capital et les intérêts, mais pas les taxes, les paiements de réserves ni les frais parfois associés aux prêts.
- Au moment de spécifier rate et nper, veillez à la cohérence des unités que vous utilisez. Si vous effectuez des paiements mensuels pour un emprunt de quatre ans à un taux d'intérêt annuel de 12 %, spécifiez 0,12/12 pour rate et 4*12 pour nper. Si vous effectuez des paiements annuels pour le même emprunt, utilisez 0,12 pour rate et 4 pour nper.
- type est arrondi à l'entier le plus proche.
- Une erreur est retournée si :
  - nper < 1

Conseil : Pour déterminer le montant total payé sur la durée de l'emprunt, multipliez la valeur PMT retournée par nper.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

Exemple 1

Agrandir le tableau

Données	Description
8 %	Taux d'intérêt annuel
10	Nombre de mois de remboursement

Données	Description
$10,000	Montant du prêt

La requête DAX suivante :

```
DAX

EVALUATE
{
  PMT(0.08/12, 10, 10000, 0, 1)
}
```

Retourne le montant de remboursement mensuel, payé au début du mois, pour un emprunt dont les termes sont spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
-1030,16432717797

Remarque : 1030,16432717797 est le remboursement périodique. Par conséquent, le montant total payé sur la durée de l'emprunt est d'environ 1030,16 * 10 = 10 301,60 USD. En d'autres termes, le montant d'intérêts payés est d'environ 301,60 USD.

Exemple 2

Agrandir le tableau

Données	Description
6 %	Taux d'intérêt annuel
18	Nombre d'années de remboursement
$50,000	Montant du prêt

La requête DAX suivante :

```
DAX

EVALUATE
{
```

```
  PMT(0.06/12, 18*12, 0, 50000)
}
```

Agrandir le tableau

[Valeur]
-129,081160867991

Retourne le montant à épargner chaque mois pour disposer de 50 000 USD au bout de 18 ans, selon les termes spécifiés ci-dessus.

Commentaires

Cette page a-t-elle été utile ? Yes No

PPMT

Article • 20/10/2023

Retourne le paiement du principal pour une période donnée pour un investissement basé sur des paiements périodiques constants et un taux d'intérêt fixe.

Syntaxe

DAX

```
PPMT(<rate>, <per>, <nper>, <pv>[, <fv>[, <type>]])
```

Paramètres

Agrandir le tableau

Terme	Définition
rate	Taux d'intérêt du prêt.
per	Spécifie la période. Doit être comprise entre 1 et nper (inclus).
nper	Nombre total de périodes de paiement dans une annuité.
pv	Valeur actuelle ou valeur que représente à la date d'aujourd'hui une série de remboursements futurs.
fv	(Facultatif) Valeur capitalisée ou solde de caisse que vous souhaitez obtenir après le dernier paiement. Si fv est omis, sa valeur présumée est BLANK.
type	(Facultatif) Valeur 0 ou 1 qui indique à quel moment les paiements sont dus. Si type est omis, la valeur par défaut est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **type** accepte les valeurs suivantes :

Agrandir le tableau

Définir le type avec la valeur	Si les paiements sont dus
0 ou omis	À la fin de la période
1	Au début de la période

Remarque : Pour obtenir une description plus complète des arguments dans PPMT, consultez la fonction PV.

Valeur de retour

Paiement sur le principal pour une période donnée.

Notes

- Au moment de spécifier rate et nper, veillez à la cohérence des unités que vous utilisez. Si vous effectuez des paiements mensuels pour un emprunt de quatre ans à un taux d'intérêt annuel de 12 %, spécifiez 0,12/12 pour rate et 4*12 pour nper. Si vous effectuez des paiements annuels pour le même emprunt, utilisez 0,12 pour rate et 4 pour nper.
- type est arrondi à l'entier le plus proche.
- Une erreur est retournée si :
  - per < 1 ou per > nper
  - nper < 1
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple 1

Agrandir le tableau

Données	Description des arguments
10 %	Taux d'intérêt annuel
2	Nombre d'années du prêt
$2,000.00	Montant du prêt

La requête DAX suivante :

```
DAX

EVALUATE
{
```

```
  PPMT(0.1/12, 1, 2*12, 2000.00)
}
```

Retourne le paiement du principal effectué au cours du premier mois d'un prêt selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
-75,6231860083663

Exemple 2

Agrandir le tableau

Données	Description des arguments
8 %	Taux d'intérêt annuel
10	Nombre d'années du prêt
$200,000.00	Montant du prêt

La requête DAX suivante :

```
DAX

EVALUATE
{
  PPMT(0.08, 10, 10, 200000.00)
}
```

Retourne le paiement du principal effectué au cours de la 10e année d'un prêt selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
-27598,0534624214

Commentaires

Cette page a-t-elle été utile ? Yes No

PRICE

Article • 20/10/2023

Retourne le prix par valeur nominale de 100 USD d'un titre rapportant des intérêts périodiques.

Syntaxe

```
DAX

PRICE(<settlement>, <maturity>, <rate>, <yld>, <redemption>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
rate	Taux d'intérêt nominal annuel du titre.
yld	Rendement annuel du titre.
redemption	Valeur de rachat du titre par valeur nominale de 100 USD.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365
4	Européen 30/360

Valeur de retour

Cours par valeur nominale de 100 USD.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.
- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après la date d'émission, le 1er janvier 2008.
- settlement et maturity sont tronqués en entiers.
- basis et frequency sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - rate < 0.
  - yld < 0.
  - redemption ≤ 0.
  - frequency est un nombre différent de 1, 2 ou 4.
  - basis < 0 ou basis > 4.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Important :

- Quand N > 1, où N est le nombre de coupons payables entre la date de règlement (settlement) et la date de rachat (redemption), **PRICE** est calculé comme suit :

$$\text{PRICE} = \left[\frac{\text{redemption}}{(1+\frac{\text{yld}}{\text{frequency}})^{(N-1+\frac{\text{DSC}}{\text{E}})}}\right] + \left[\sum_{k=1}^{N}\frac{100\times\frac{\text{rate}}{\text{frequency}}}{(1+\frac{\text{yld}}{\text{frequency}})^{(k-1+\frac{\text{DSC}}{\text{E}})}}\right] - \left[100\times\frac{\text{rate}}{\text{frequency}}\times\frac{\text{A}}{\text{E}}\right]$$

- Quand N = 1, où N est le nombre de coupons payables entre la date de règlement (settlement) et la date de rachat (redemption), **PRICE** est calculé comme suit :

$$\text{DSR} = \text{E} - \text{A}$$

$$\text{T1} = 100\times\frac{\text{rate}}{\text{frequency}} + \text{redemption}$$

$$\text{T2} = \frac{\text{yld}}{\text{frequency}}\times\frac{\text{DSR}}{\text{E}} + 1$$

$$\text{T3} = 100\times\frac{\text{rate}}{\text{frequency}}\times\frac{\text{A}}{\text{E}}$$

$$\text{PRICE} = \frac{\text{T1}}{\text{T2}} - \text{T3}$$

où :

- DSC = nombre de jours entre le règlement (settlement) et la date du prochain coupon.
- E = nombre de jours dans la période du coupon qui contient la date de règlement (settlement).
- A = nombre de jours entre le début de la période du coupon et la date de règlement (settlement).

Exemple

Agrandir le tableau

Données	Description des arguments
15/2/2008	Date de règlement
15/11/2017	Date d'échéance
5,75 %	Taux de coupon semestriel
6,50 %	Taux de rendement
$100	Valeur de rachat
2	Fréquence semestrielle
0	Base 30/360

La requête DAX suivante :

DAX

```
EVALUATE
{
  PRICE(DATE(2008,2,15), DATE(2017,11,15), 0.0575, 0.065, 100, 2, 0)
}
```

Retourne le prix d'une obligation selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
94,6343616213221

Commentaires

Cette page a-t-elle été utile ? Yes No

PRICEDISC

Article • 20/10/2023

Retourne le cours par valeur nominale de 100 USD pour un titre au-dessous du pair.

Syntaxe

DAX

```
PRICEDISC(<settlement>, <maturity>, <discount>, <redemption>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
discount	Taux d'actualisation du titre.
redemption	Valeur de rachat du titre par valeur nominale de 100 USD.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365

Base	Base de comptage des jours
4	Européen 30/360

Valeur de retour

Cours par valeur nominale de 100 USD.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.

- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation de 30 ans a été émise le 1er janvier 2018 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2018, la date de règlement (settlement) le 1er juillet 2018 et la date d'échéance (maturity) le 1er janvier 2048, soit 30 ans après le 1er janvier 2008, la date d'émission.

- PRICEDISC se calcule comme suit :

$$\text{PRICEDISC} = \text{redemption} - \text{discount} \times \text{redemption} \times \frac{\text{DSM}}{\text{B}}$$

  où :
  - B = nombre de jours dans une année, qui dépend de la base annuelle.
  - DIM = nombre de jours entre le règlement (settlement) et l'échéance (maturity).

- settlement et maturity sont tronqués en entiers.

- basis est arrondi à l'entier le plus proche.

- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - discount ≤ 0.
  - redemption ≤ 0.
  - basis < 0 ou basis > 4.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description des arguments
16/2/2008	Date de règlement
1/3/2008	Date d'échéance
5,25 %	Taux d'actualisation en pourcentage
$100	Valeur de rachat
2	Base Réel/360

La requête DAX suivante :

```DAX
EVALUATE
{
  PRICEDISC(DATE(2008,2,16), DATE(2008,3,1), 0.0525, 100, 2)
}
```

Retourne le cours par valeur nominale de 100 USD pour l'obligation dont les termes sont spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
99,7958333333333

Commentaires

Cette page a-t-elle été utile ? Yes No

PRICEMAT

Article • 20/10/2023

Retourne le cours par valeur nominale de 100 USD d'un titre rapportant des intérêts à l'échéance.

Syntaxe

```
DAX
PRICEMAT(<settlement>, <maturity>, <issue>, <rate>, <yld>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
problème	Date d'émission du titre.
rate	Taux d'intérêt du titre à la date d'émission.
yld	Rendement annuel du titre.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360

Base	Base de comptage des jours
3	Réel/365
4	Européen 30/360

Valeur de retour

Cours par valeur nominale de 100 USD.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.

- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) est le 1er juillet 2008 et la date d'échéance (maturity) est le 1er janvier 2038, soit 30 ans après le 1er janvier 2008, la date d'émission.

- PRICEMAT se calcule comme suit :

$$\text{PRICEMAT} = \frac{100 + (\frac{\text{DIM}}{\text{B}} \times \text{rate} \times 100)}{1 + (\frac{\text{DSM}}{\text{B}} \times \text{yld})} - (\frac{\text{A}}{\text{B}} \times \text{rate} \times 100)$$

  où :
  - B = nombre de jours dans une année, qui dépend de la base annuelle.
  - DIM = nombre de jours entre le règlement (settlement) et l'échéance (maturity).
  - DIM = nombre de jours entre l'émission (issue) et l'échéance (maturity).
  - A = nombre de jours entre l'émission (issue) et le règlement (settlement).

- settlement, maturity et issue sont tronqués en entiers.

- basis est arrondi à l'entier le plus proche.

- Une erreur est retournée si :
  - settlement, maturity ou issue n'est pas une date valide.

- maturity > settlement > issue n’est pas respecté.
- rate < 0.
- yld < 0.
- basis < 0 ou basis > 4.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La requête DAX suivante :

Agrandir le tableau

Données	Description
15/2/2008	Date de règlement
13/4/2008	Date d’échéance
11/11/2007	Date d’émission
6,10 %	Coupon semestriel en pourcentage
6,10 %	Rendement en pourcentage
0	Base 30/360

DAX

```
EVALUATE
{
  PRICEMAT(DATE(2008,2,15), DATE(2008,4,13), DATE(2007,11,11), 0.061, 0.061,
0)
}
```

Retourne le cours par valeur nominale de 100 USD d’un titre dont les termes sont spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
99,9844988755569

Commentaires

Cette page a-t-elle été utile ? Yes No

PV

Article • 20/10/2023

Calcule la valeur actuelle d'un emprunt ou d'un investissement sur la base d'un taux d'intérêt constant. Vous pouvez utiliser la fonction PV avec des paiements périodiques constants (tels qu'un crédit immobilier ou un autre emprunt) et/ou une valeur future qui correspond à votre objectif d'investissement.

Syntaxe

DAX

```
PV(<rate>, <nper>, <pmt>[, <fv>[, <type>]])
```

Paramètres

Agrandir le tableau

Terme	Définition
rate	Taux d'intérêt par période. Par exemple, si vous obtenez un crédit automobile à un taux d'intérêt annuel de 10 % et que vous effectuez des remboursements mensuels, votre taux d'intérêt par mois est de 0,1/12, soit 0,0083. Vous devez entrer 0,1/12 ou 0,0083 dans la formule pour rate.
nper	Nombre total de périodes de remboursement dans une annuité. Par exemple, si vous obtenez un crédit auto d'une durée de quatre ans et que vous effectuez des remboursements mensuels, votre prêt comptera 4*12 (ou 48) périodes. Vous devez entrer 48 dans la formule pour nper.
pmt	Paiement effectué à chaque période qui ne peut pas changer pendant la durée de l'annuité. En règle générale, pmt inclut le capital et les intérêts, mais pas d'autres frais ou taxes. Par exemple, le montant des remboursements mensuels pour un crédit auto de 10 000 USD sur quatre ans à 12 pour cent est de 263,33 USD. Vous devez entrer -263,33 dans la formule pour pmt.
fv	(Facultatif) Valeur capitalisée ou solde de caisse que vous souhaitez obtenir après le dernier paiement. Si fv est omis, sa valeur présumée est BLANK. Par exemple, si vous souhaitez épargner 50 000 USD pour financer un projet spécial dans 18 ans, 50 000 USD est la valeur capitalisée. Vous pouvez ensuite faire une estimation à partir d'un taux d'intérêt raisonnable pour déterminer le montant que vous devez épargner chaque mois.
type	(Facultatif) Valeur 0 ou 1 qui indique à quel moment les paiements sont dus. Si type est omis, la valeur par défaut est 0. Les valeurs acceptées sont listées en dessous de ce

Terme	Définition
	tableau.

Le paramètre **type** accepte les valeurs suivantes :

Agrandir le tableau

Définir le type avec la valeur	Si les paiements sont dus
0 ou omis	À la fin de la période
1	Au début de la période

Valeur de retour

Valeur actuelle d'un emprunt ou investissement.

Notes

- Au moment de spécifier rate et nper, veillez à la cohérence des unités que vous utilisez. Si vous effectuez des paiements mensuels pour un prêt de quatre ans à un taux d'intérêt annuel de 12 %, spécifiez 0,12/12 pour rate et 4*12 pour nper. Si vous effectuez des paiements annuels pour le même emprunt, utilisez 0,12 pour rate et 4 pour nper.
- Les fonctions suivantes s'appliquent uniquement aux annuités :
  - CUMIPMT
  - CUMPRINC
  - VC
  - IPMT
  - PMT
  - PPMT
  - PV
  - RATE
  - XIRR
  - XNPV
- Une annuité est une série de paiements comptants fixes effectués sur une période continue. Par exemple, un crédit automobile ou un crédit immobilier est une annuité. Pour plus d'informations, consultez la description de chaque fonction d'annuité.

- Dans les fonctions d'annuité, les montants que vous déboursez, par un versement au titre d'un placement, sont représentées par un nombre négatif ; les montants que vous recevez, par exemple un chèque de dividendes, sont représentés par un nombre positif. Par exemple, un montant de 1 000 USD déposé à la banque serait représenté par l'argument -1000 si vous étiez le déposant et par l'argument 1000 si vous étiez la banque.
- Un argument financier est résolu selon les termes des autres.
  - Si rate n'est pas égal à 0, alors :

    $$\text{pv} \times (1 + \text{rate})^{\text{nper}} + \text{pmt}(1 + \text{rate} \times \text{type}) \times \left(\frac{(1 + \text{rate})^{\text{nper}} - 1}{\text{rate}} \right) + \text{fv} = 0$$

  - Si rate est égal à 0, alors :

    $$(\text{pmt} \times \text{nper}) + \text{pv} + \text{fv} = 0$$

- type est arrondi à l'entier le plus proche.
- Une erreur est retournée si :
  - nper < 1 ou vide
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
$500.00	Somme déboursée au titre d'une annuité d'assurance à la fin de chaque mois.
8 %	Taux d'intérêt acquis sur la somme déboursée.
20	Année à laquelle les versements prendront fin.

La requête DAX suivante :

DAX

```
EVALUATE
{
```

```
  PV(0.08/12, 12*20, 500.00, 0, 0)
}
```

Retourne la valeur actuelle d'une annuité selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
-59777,1458511878

Commentaires

Cette page a-t-elle été utile ? Yes No

RATE

Article • 20/10/2023

Retourne le taux d'intérêt par période d'une annuité. RATE est calculé par itération et peut avoir zéro ou plusieurs solutions. Si les résultats successifs de RATE ne convergent pas après 20 itérations (avec une précision de 0,0000001), une erreur est retournée.

Syntaxe

DAX

```
RATE(<nper>, <pmt>, <pv>[, <fv>[, <type>[, <guess>]]])
```

Paramètres

Agrandir le tableau

Terme	Définition
nper	Nombre total de périodes de paiement dans une annuité.
pmt	Paiement effectué à chaque période qui ne peut pas changer pendant la durée de l'annuité. En général, pmt inclut le principal et les intérêts, mais pas d'autres frais ou taxes.
pv	Valeur actuelle ou valeur que représente à la date d'aujourd'hui une série de remboursements futurs.
fv	(Facultatif) Valeur capitalisée ou solde de caisse que vous souhaitez obtenir après le dernier paiement. Si fv est omis, la valeur par défaut est 0 (la valeur future d'un prêt, par exemple, est 0).
type	(Facultatif) Valeur 0 ou 1 qui indique à quel moment les paiements sont dus. Si type est omis, la valeur par défaut est 0. Les valeurs acceptées sont listées en dessous de ce tableau.
guess	(Facultatif) Votre estimation du taux. - En cas d'omission, la valeur par défaut est 10 %. -Si RATE ne converge pas, essayez des valeurs différentes de guess. RATE converge habituellement si guess est compris entre 0 et 1.

Le paramètre **type** accepte les valeurs suivantes :

Agrandir le tableau

Définir le type avec la valeur	Si les paiements sont dus
0 ou omis	À la fin de la période
1	Au début de la période

Valeur de retour

Taux d'intérêt par période.

Notes

- Au moment de spécifier guess et nper, veillez à la cohérence des unités que vous utilisez. Si vous effectuez des paiements mensuels pour un prêt de quatre ans à un taux d'intérêt annuel de 12 %, spécifiez 0,12/12 pour guess et 4*12 pour nper. Si vous effectuez des paiements annuels pour le même prêt, utilisez 0,12 pour guess et 4 pour nper.
- type est arrondi à l'entier le plus proche.
- Une erreur est retournée si :
  - nper ≤ 0.
  - RATE ne converge pas après 20 itérations (avec une précision de 0,0000001)
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

Agrandir le tableau

Données	Description
4	Années du prêt
-200	Paiement mensuel
8000	Montant du prêt

Exemple 1

La requête DAX suivante :

```
DAX

EVALUATE
{
  RATE(4*12, -200, 8000)
}
```

Retourne le taux mensuel du prêt selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
0,00770147248820137

Exemple 2

La requête DAX suivante :

```
DAX

EVALUATE
{
  RATE(4*12, -200, 8000) * 12
}
```

Retourne le taux annuel du prêt selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
0,0924176698584164

Commentaires

Cette page a-t-elle été utile ? Yes No

RECEIVED

Article • 20/10/2023

Retourne le montant reçu à l'échéance d'un titre totalement investi.

Syntaxe

DAX

```
RECEIVED(<settlement>, <maturity>, <investment>, <discount>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
investment	Montant investi dans le titre.
discount	Taux d'escompte du titre.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365

Base	Base de comptage des jours
4	Européen 30/360

Valeur de retour

Montant reçu à l'échéance.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.

- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après le 1er janvier 2008, date d'émission.

- RECEIVED se calcule comme suit :

$$\text{RECEIVED} = \frac{\text{investment}}{1 - (\text{discount} \times \frac{\text{DIM}}{\text{B}})}$$

  où :
  - B = nombre de jours dans une année, selon la base annuelle.
  - DIM = nombre de jours entre la date d'émission (issue) et la date d'échéance (maturity).

- settlement et maturity sont tronqués en entiers.

- basis est arrondi à l'entier le plus proche.

- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - investment ≤ 0.
  - discount ≤ 0.

- basis < 0 ou basis > 4.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La requête DAX suivante :

Agrandir le tableau

Données	Description
15 février 2008	Date du règlement (émission)
15 mai 2008	Date d'échéance
$1,000,000.00	Investissement
5,75 %	Taux d'escompte
2	Base Réel/360

DAX

```
EVALUATE
{
  RECEIVED(DATE(2008,2,15), DATE(2008,5,15), 1000000.00, 0.0575, 2)
}
```

Retourne le montant total à recevoir à l'échéance, pour une obligation avec les conditions spécifiées ci-dessus.

Agrandir le tableau

[Valeur]
1014584,6544071

Commentaires

Cette page a-t-elle été utile ? Yes No

RRI

Article • 20/10/2023

Retourne un taux d'intérêt équivalent pour la croissance d'un investissement.

Syntaxe

```
DAX
RRI(<nper>, <pv>, <fv>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nper	Nombre de périodes de l'investissement.
pv	Valeur actuelle de l'investissement.
fv	Valeur capitalisée de l'investissement.

Valeur de retour

Taux d'intérêt équivalent.

Notes

- RRI retourne le taux d'intérêt calculé à l'aide de l'équation suivante, où nper est le nombre de périodes, pv la valeur actuelle et fv la valeur future :

$$\left(\frac{\mathrm{fv}}{\mathrm{pv}}\right)^{\left(\frac{1}{nper}\right)} - 1$$

- Une erreur est retournée si :
  - nper ≤ 0.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au

niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
$10,000	Valeur actuelle
$21,000	Valeur capitalisée
4	Années investies

La requête DAX suivante :

DAX

```
EVALUATE
{
  RRI(4*12, 10000, 21000)
}
```

Retourne un taux d'intérêt équivalent pour la croissance d'un investissement selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
0,0155771057566627

Commentaires

Cette page a-t-elle été utile ? Yes No

AmorLin

Article • 20/10/2023

Retourne l'amortissement linéaire d'un bien pour une période.

Syntaxe

DAX

```
SLN(<cost>, <salvage>, <life>)
```

Paramètres

Agrandir le tableau

Terme	Définition
cost	Coût initial du bien.
salvage	Valeur au terme de l'amortissement (parfois appelée valeur résiduelle de l'actif).
à faire parler vos données	Nombre de périodes à l'issue desquelles le bien est amorti (parfois appelé durée de vie utile du bien).

Valeur de retour

Amortissement linéaire d'un bien pour une période.

Notes

- Une erreur est retournée si :
  life = 0.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
$30,000	Coût
$7,500	Valeur résiduelle
10	Années de vie utile

La requête DAX suivante :

```
DAX

EVALUATE
{
  SLN(30000, 7500, 10)
}
```

Retourne la provision pour amortissement annuelle en utilisant les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
2 250

Commentaires

Cette page a-t-elle été utile ? Yes No

SYD

Article • 20/10/2023

Retourne l'amortissement d'un actif pour une période spécifiée, calculé selon la méthode de l'amortissement proportionnel à l'ordre numérique inversé des années.

Syntaxe

```
DAX
SYD(<cost>, <salvage>, <life>, <per>)
```

Paramètres

Agrandir le tableau

Terme	Définition
cost	Coût initial du bien.
salvage	Valeur au terme de l'amortissement (parfois appelée valeur résiduelle de l'actif).
à faire parler vos données	Nombre de périodes à l'issue desquelles l'actif est amorti (parfois appelé durée de vie utile de l'actif).
per	Période. Doit être spécifiée dans les mêmes unités que life. Doit être comprise entre 1 et la valeur de life (incluse).

Valeur de retour

Amortissement pour une période spécifiée, calculé selon la méthode de l'amortissement proportionnel à l'ordre numérique inversé des années.

Notes

- SYD se calcule comme suit :

$$\text{SYD} = \frac{(\text{cost} - \text{salvage}) \times (\text{life} - \text{per} + 1) \times 2}{(\text{life}) \times (\text{life} + 1)}$$

- Une erreur est retournée si :
  - life < 1.
  - per < 1 ou per > life.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

Agrandir le tableau

Données	Description
$30,000.00	Coût initial
$7,500.00	Valeur résiduelle
10	Durée de vie en années

Exemple 1

La requête DAX suivante :

```DAX
EVALUATE
{
  SYD(30000.00, 7500.00, 10, 1)
}
```

Retourne la provision d'amortissement sur la première année, selon la méthode de l'amortissement proportionnel à l'ordre numérique inversé des années, pour un actif avec les conditions spécifiées ci-dessus.

Agrandir le tableau

[Valeur]
4090,90909090909

Exemple 2

La requête DAX suivante :

```
DAX

EVALUATE
{
  SYD(30000.00, 7500.00, 10, 10)
}
```

Retourne la provision d'amortissement sur la dixième (et dernière) année, selon la méthode de l'amortissement proportionnel à l'ordre numérique inversé des années, pour un actif avec les conditions spécifiées ci-dessus.

Agrandir le tableau

[Valeur]
409,090909090909

Commentaires

Cette page a-t-elle été utile ? Yes No

TBILLEQ

Article • 20/10/2023

Retourne le rendement équivalent à une obligation d'un bon du Trésor.

Syntaxe

DAX

```
TBILLEQ(<settlement>, <maturity>, <discount>)
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du bon du Trésor. La date de règlement du titre correspond à la date suivant la date d'émission, quand le bon du Trésor est cédé à l'acheteur.
maturity	Date d'échéance du bon du Trésor. La date d'échéance correspond à la date d'expiration du bon du Trésor.
discount	Taux d'actualisation du bon du Trésor.

Valeur de retour

Rendement équivalent à une obligation du bon du Trésor.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. En DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 correspond au jour 39 448, car il s'agit du 39 448 e jour après le 30 décembre 1899.
- TBILLEQ se calcule comme suit :

$$\mathrm{TBILLEQ} = \frac{365 \times \mathrm{discount}}{360 - (\mathrm{discount} \times \mathrm{DSM})}$$

où :
    - DSM est le nombre de jours entre le règlement (settlement) et l'échéance (maturity) calculé sur la base d'une année de 360 jours.

- settlement et maturity sont tronqués en entiers.
- Une erreur est retournée si :
    - settlement ou maturity n'est pas une date valide.
    - settlement ≥ maturity ou maturity est postérieur de plus d'un an à settlement.
    - discount ≤ 0.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
3/31/2008	Date de règlement
1/6/2008	Date d'échéance
9,14 %	Taux d'actualisation en pourcentage

La requête DAX suivante :

```DAX
EVALUATE
{
  TBILLEQ(DATE(2008,3,31), DATE(2008,6,1), 0.0914)
}
```

Retourne le rendement équivalent à une obligation d'un bon du Trésor selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
0,094151493565943

Commentaires

Cette page a-t-elle été utile ? Yes No

TBILLPRICE

Article • 20/10/2023

Retourne le cours par valeur nominale de 100 USD pour un bon du Trésor.

Syntaxe

DAX

```
TBILLPRICE(<settlement>, <maturity>, <discount>)
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du bon du Trésor. La date de règlement du titre correspond à la date suivant la date d'émission, quand le bon du Trésor est cédé à l'acheteur.
maturity	Date d'échéance du bon du Trésor. La date d'échéance correspond à la date d'expiration du bon du Trésor.
discount	Taux d'actualisation du bon du Trésor.

Valeur de retour

Cours du bon du Trésor par valeur nominale de 100 USD.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. En DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 correspond au jour 39 448, car il s'agit du 39 448 e jour après le 30 décembre 1899.
- TBILLPRICE se calcule comme suit :

$$\text{TBILLPRICE} = 100 \times (1 - \frac{\text{discount} \times \text{DSM}}{360})$$

où :
    - DSM = nombre de jours entre le règlement (settlement) et l'échéance (maturity), à l'exclusion de toute date d'échéance postérieure de plus d'une année civile à la date de règlement.

- settlement et maturity sont tronqués en entiers.

- Une erreur est retournée si :
    - settlement ou maturity n'est pas une date valide.
    - settlement ≥ maturity ou maturity est postérieur de plus d'un an à settlement.
    - discount ≤ 0.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
3/31/2008	Date de règlement
1/6/2008	Date d'échéance
9.0%	Taux d'actualisation en pourcentage

La requête DAX suivante :

DAX

```
EVALUATE
{
  TBILLPRICE(DATE(2008,3,31), DATE(2008,6,1), 0.09)
}
```

Retourne le cours du bon du Trésor par valeur nominale de 100 USD, selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
98,45

Commentaires

Cette page a-t-elle été utile ? Yes No

TBILLYIELD

Article • 20/10/2023

Retourne le rendement d'un bon du Trésor.

Syntaxe

```
DAX

TBILLYIELD(<settlement>, <maturity>, <pr>)
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du bon du Trésor. La date de règlement du titre correspond à la date suivant la date d'émission, quand le bon du Trésor est cédé à l'acheteur.
maturity	Date d'échéance du bon du Trésor. La date d'échéance correspond à la date d'expiration du bon du Trésor.
pr	Cours du bon du Trésor par valeur nominale de 100 USD.

Valeur de retour

Rendement du bon du Trésor.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. En DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 correspond au jour 39 448, car il s'agit du 39 448 e jour après le 30 décembre 1899.
- TBILLYIELD se calcule comme suit :

$$\text{TBILLYIELD} = \frac{100 - \text{pr}}{\text{pr}} \times \frac{360}{\text{DSM}}$$

où :
    - DSM = nombre de jours entre le règlement (settlement) et l'échéance (maturity), à l'exclusion de toute date d'échéance postérieure de plus d'une année civile à la date de règlement.

- settlement et maturity sont tronqués en entiers.

- Une erreur est retournée si :
    - settlement ou maturity n'est pas une date valide.
    - settlement ≥ maturity ou maturity est postérieur de plus d'un an à settlement.
    - pr ≤ 0.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La requête DAX suivante :

Agrandir le tableau

Données	Description
3/31/2008	Date de règlement
1/6/2008	Date d'échéance
$98.45	Cours par valeur nominale de 100 USD

DAX

```
EVALUATE
{
  TBILLYIELD(DATE(2008,3,31), DATE(2008,6,1), 98.45)
}
```

Retourne le rendement d'un bon du Trésor selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
0,0914169629253426

Commentaires

Cette page a-t-elle été utile ? Yes No

VDB

Article • 20/10/2023

Retourne l'amortissement d'un actif pour une période donnée (périodes partielles incluses), calculé selon la méthode de l'amortissement dégressif à taux double ou une autre méthode que vous spécifiez. VDB est l'acronyme de Variable Declining Balance (solde dégressif variable).

Syntaxe

DAX

```
VDB(<cost>, <salvage>, <life>, <start_period>, <end_period>[, <factor>[, <no_switch>]])
```

Paramètres

Agrandir le tableau

Terme	Définition
cost	Coût initial du bien.
salvage	Valeur au terme de l'amortissement (parfois appelée valeur résiduelle de l'actif). Cette valeur peut être égale à 0.
à faire parler vos données	Nombre de périodes pendant lesquelles l'actif est amorti (parfois appelé durée de vie utile de l'actif).
start_period	Période de départ pour laquelle vous souhaitez calculer l'amortissement. start_period doit utiliser les mêmes unités que life. Doit être comprise entre 1 et life (inclus).
end_period	Période de fin pour laquelle vous souhaitez calculer l'amortissement. end_period doit utiliser les mêmes unités que life. Doit être comprise entre start_period et life (inclus).
facteur	(Facultatif) Taux auquel le solde diminue. Si factor est omis, la valeur par défaut est 2 (méthode de l'amortissement dégressif à taux double). Modifiez factor si vous ne souhaitez pas utiliser la méthode de l'amortissement dégressif à taux double. Pour obtenir une description de la méthode de l'amortissement dégressif à taux double, consultez DDB.
no_switch	(Facultatif) Valeur logique spécifiant s'il faut passer à la méthode de l'amortissement linéaire quand l'amortissement est supérieur à celui obtenu par

Terme	Définition
	la méthode de l'amortissement dégressif. En cas d'omission, la valeur par défaut est FALSE. - Si no_switch prend la valeur TRUE, VDB ne passe pas à la méthode de l'amortissement linéaire, même si l'amortissement est supérieur à celui obtenu par la méthode de l'amortissement dégressif. - Si no_switch prend la valeur FALSE ou est omis, VDB passe à la méthode de l'amortissement linéaire si l'amortissement est supérieur à celui obtenu par la méthode de l'amortissement dégressif.

Valeur de retour

Amortissement sur la période spécifiée.

Notes

- Une erreur est retournée si :
  - cost < 0.
  - salvage < 0.
  - life < 1.
  - start_period < 1 ou start_period > end_period.
  - end_period < start_period ou end_period > life.
  - factor < 0.
  - no_switch ne correspond pas à la valeur TRUE ou FALSE.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

Agrandir le tableau

Données	Description
2 400	Coût initial
300	Valeur résiduelle
10	Durée de vie en années

Exemple 1

La requête DAX suivante :

```
DAX

EVALUATE
{
  VDB(2400, 300, 10*365, 0, 1)
}
```

Retourne l'amortissement du premier jour d'un actif avec un facteur de 2.

Agrandir le tableau

[Valeur]
1,31506849315068

Exemple 2

La requête DAX suivante :

```
DAX

EVALUATE
{
  VDB(2400, 300, 10*12, 6, 18, 3)
}
```

Retourne l'amortissement d'un actif entre le 6^e mois et le 18^e mois. Ce calcul utilise un facteur de 3.

Agrandir le tableau

[Valeur]
540,185558199698

Exemple 3

La requête DAX suivante :

```
DAX
```

```
EVALUATE
{
  VDB(2400, 300, 10, 0, 0.875, 1.5)
}
```

Retourne l'amortissement d'un actif au cours du premier exercice après son acquisition, en supposant que les lois fiscales vous limitent à 150 % d'amortissement du solde dégressif. L'actif est acquis au milieu du premier trimestre de l'exercice.

Agrandir le tableau

[Valeur]
315

Commentaires

Cette page a-t-elle été utile ? Yes No

XIRR

Article • 20/12/2023

Retourne le taux de retour interne pour une planification de flux de trésorerie qui n'est pas nécessairement périodique.

Syntaxe

DAX

```
XIRR(<table>, <values>, <dates>, [, <guess>[, <alternateResult>]])
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table pour laquelle les expressions de valeurs et de dates doivent être calculées.
values	Expression qui retourne la valeur de flux de trésorerie pour chaque ligne de la table.
dates	Expression qui retourne la date de flux de trésorerie pour chaque ligne de la table.
guess	(Facultatif) Estimation initiale du taux de retour interne. En cas d'omission, l'estimation par défaut de 0.1 est utilisée.
alternateResult	(Facultatif) Valeur retournée à la place d'une erreur lorsqu'une solution ne peut pas être déterminée.

Valeur de retour

Taux de retour interne pour les entrées données. Si le calcul ne parvient pas à retourner un résultat valide, une erreur ou une valeur spécifiée en tant que alternateResult est retournée.

Remarques

- La valeur est calculée comme taux qui satisfait la fonction suivante :

$$0 = \sum_{j=1}^{N} \frac{P_j}{(1+\text{rate})^{\frac{d_j - d_1}{365}}}$$

Où :

- P_j est le j^e paiement
- d_j est la j^e date de paiement
- d_1 est la première date de paiement

- La série de valeurs de flux de trésorerie doit contenir au moins un nombre positif et un nombre négatif.
- Évitez d'utiliser les fonctions ISERROR ou IFERROR pour capturer une erreur retournée par XIRR. Si certaines entrées de la fonction peuvent entraîner une erreur sans solution, le fait de fournir un paramètre alternateResult est le moyen le plus fiable et le plus performant pour gérer l'erreur.
- Pour en savoir plus sur l'utilisation du paramètre alternateResult, consultez cette vidéo.
- Lorsque la valeur absolue du paiement initial est faible, le calcul ne parvient probablement pas à retourner un résultat valide.
- Évitez d'inclure des valeurs de 0 dans les paiements. Elles n'auront aucun impact sur le résultat final et l'utilisation de 0 en tant que paiement initial entraînera systématiquement l'échec du calcul XIRR().
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La formule suivante calcule le taux de retour interne de la table CashFlows :

DAX

```
= XIRR( CashFlows, [Payment], [Date] )
```

Agrandir le tableau

Date	Paiement
1/1/2014	-10000

Date	Paiement
1/3/2014	2750
30/10/2014	4250
15/02/2015	3250
1/4/2015	2750

Taux de retour = 37,49 %

Commentaires

Cette page a-t-elle été utile ? Yes No

XNPV

Article • 20/10/2023

Retourne la valeur actuelle pour une planification de flux de trésorerie qui n'est pas nécessairement périodique.

Syntaxe

DAX

```
XNPV(<table>, <values>, <dates>, <rate>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table pour laquelle les expressions de valeurs et de dates doivent être calculées.
values	Expression qui retourne la valeur de flux de trésorerie pour chaque ligne de la table.
dates	Expression qui retourne la date de flux de trésorerie pour chaque ligne de la table.
rate	Taux de remise à appliquer au cash flow pour chaque ligne de la table.

Valeur de retour

Valeur nette actuelle.

Remarques

- La valeur est calculée comme suit :

$$\sum_{j=1}^{N} \frac{P_j}{(1+\text{rate})^{\frac{d_j - d_1}{365}}}$$

  Où :
  - P_j est le j^e paiement
  - d_j est la j^e date de paiement

- d_1 est la première date de paiement

- La série de valeurs de flux de trésorerie doit contenir au moins un nombre positif et un nombre négatif.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant calcule la valeur actuelle de la table CashFlows :

DAX

```
= XNPV( CashFlows, [Payment], [Date], 0.09 )
```

Agrandir le tableau

Date	Paiement
1/1/2014	-10000
1/3/2014	2750
30/10/2014	4250
15/02/2015	3250
1/4/2015	2750

Valeur actuelle = 2086.65

Commentaires

Cette page a-t-elle été utile ? Yes No

YIELD

Article • 20/10/2023

Retourne le rendement d'un titre qui rapporte des intérêts périodiques. Utilisez YIELD pour calculer le rendement d'une obligation.

Syntaxe

DAX

```
YIELD(<settlement>, <maturity>, <rate>, <pr>, <redemption>, <frequency>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
rate	Taux d'intérêt nominal annuel de l'obligation à coupons.
pr	Cours du titre par valeur nominale de 100 USD.
redemption	Valeur de rachat du titre par valeur nominale de 100 USD.
frequency	Nombre de versements de coupons par an. Si le paiement est annuel, frequency = 1 ; s'il est semestriel, frequency = 2 ; et s'il est trimestriel, frequency = 4.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360

Base	Base de comptage des jours
1	Réel/réel
2	Réel/360
3	Réel/365
4	Européen 30/360

Valeur de retour

Rendement du titre.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.

- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) est le 1er juillet 2008 et la date d'échéance (maturity) est le 1er janvier 2038, soit 30 ans après la date d'émission, le 1er janvier 2008.

- S'il existe une période de coupon ou moins avant le rachat, YIELD se calcule comme suit :

$$\text{YIELD} = \frac{\left(\frac{\text{redemption}}{100} + \frac{\text{rate}}{\text{frequency}}\right) - \left(\frac{\text{par}}{100} + \left(\frac{\text{A}}{\text{E}} \times \frac{\text{rate}}{\text{frequency}}\right)\right)}{\frac{\text{par}}{100} + \left(\frac{\text{A}}{\text{E}} \times \frac{\text{rate}}{\text{frequency}}\right)} \times \frac{\text{frequency} \times \text{E}}{\text{DSR}}$$

  où :
  - A = nombre de jours entre le début de la période de coupon et la date de règlement (jours courus).
  - DSR = nombre de jours entre la date de règlement (settlement) et la date de rachat (redemption).
  - E = nombre de jours dans la période de coupon.

- S'il existe plusieurs périodes de coupon avant le rachat, YIELD se calcule par une centaine d'itérations. La résolution utilise la méthode Newton, basée sur la formule utilisée pour la fonction PRICE. Le rendement change tant que le cours estimé par rapport au rendement n'est pas proche du cours.
- settlement et maturity sont tronqués en entiers.
- frequency et basis sont arrondis à l'entier le plus proche.
- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - rate < 0.
  - pr ≤ 0.
  - redemption ≤ 0.
  - frequency est un nombre différent de 1, 2 ou 4.
  - basis < 0 ou basis > 4.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
15 février 2008	Date de règlement
15 novembre 2016	Date d'échéance
5,75 %	Coupon en pourcentage
95,04287	Prx
$100	Valeur de rachat
2	La fréquence est semestrielle (voir ci-dessus)
0	Base 30/360 (voir ci-dessus)

La requête DAX suivante :

```
DAX
```

```
EVALUATE
{
  YIELD(DATE(2008,2,15), DATE(2016,11,15), 0.0575, 95.04287, 100, 2,0)
}
```

Retourne le rendement d'une obligation avec les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
0,0650000068807314

Commentaires

Cette page a-t-elle été utile ? Yes No

YIELDDISC

Article • 20/10/2023

Retourne le rendement annuel d'un titre au-dessous du pair.

Syntaxe

DAX

```
YIELDDISC(<settlement>, <maturity>, <pr>, <redemption>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
pr	Cours du titre par valeur nominale de 100 USD.
redemption	Valeur de rachat du titre par valeur nominale de 100 USD.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365

Base	Base de comptage des jours
4	Européen 30/360

Valeur de retour

Rendement annuel.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.
- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après la date d'émission, le 1er janvier 2008.
- settlement et maturity sont tronqués en entiers.
- basis est arrondi à l'entier le plus proche.
- Une erreur est retournée si :
  - settlement ou maturity n'est pas une date valide.
  - settlement ≥ maturity.
  - pr ≤ 0.
  - redemption ≤ 0.
  - basis < 0 ou basis > 4.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	-
16 février 2008	Date de règlement
1er mars 2008	Date d'échéance
99,795	Price
$100	Valeur de rachat
2	Base Réel/360

La requête DAX suivante :

DAX

```
EVALUATE
{
  YIELDDISC(DATE(2008,2,16), DATE(2008,3,1), 99.795, 100, 2)
}
```

Retourne le rendement annuel du titre, compte tenu des termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
0,0528225719868583

Commentaires

Cette page a-t-elle été utile ? Yes No

YIELDMAT

Article • 20/10/2023

Retourne le rendement annuel d'un titre qui rapporte des intérêts à l'échéance.

Syntaxe

```
DAX
YIELDMAT(<settlement>, <maturity>, <issue>, <rate>, <pr>[, <basis>])
```

Paramètres

Agrandir le tableau

Terme	Définition
settlement	Date de règlement du titre. La date de règlement du titre correspond à la date suivant la date d'émission, quand le titre est cédé à l'acheteur.
maturity	Date d'échéance du titre. La date d'échéance correspond à la date d'expiration du titre.
problème	Date d'émission du titre.
rate	Taux d'intérêt du titre à la date d'émission.
pr	Cours du titre par valeur nominale de 100 USD.
basis	(Facultatif) Type de la base de comptage des jours à utiliser. Si basis est omis, sa valeur supposée est 0. Les valeurs acceptées sont listées en dessous de ce tableau.

Le paramètre **basis** accepte les valeurs suivantes :

Agrandir le tableau

Base	Base de comptage des jours
0 ou omis	US (NASD) 30/360
1	Réel/réel
2	Réel/360
3	Réel/365

Base	Base de comptage des jours
4	Européen 30/360

Valeur de retour

Rendement annuel.

Notes

- Les dates sont stockées sous forme de numéros de série séquentiels de façon à pouvoir être utilisées dans les calculs. Dans DAX, le 30 décembre 1899 correspond au jour 0 et le 1er janvier 2008 au jour 39 448, car c'est le 39 448 e jour après le 30 décembre 1899.
- La date de règlement (settlement) est la date à laquelle un acheteur achète un coupon, par exemple une obligation. La date d'échéance (maturity) correspond à la date d'expiration du coupon. Par exemple, supposez qu'une obligation d'une durée de 30 ans a été émise le 1er janvier 2008 et qu'elle a été achetée six mois plus tard. La date d'émission est le 1er janvier 2008, la date de règlement (settlement) le 1er juillet 2008 et la date d'échéance (maturity) le 1er janvier 2038, soit 30 ans après la date d'émission, le 1er janvier 2008.
- settlement, maturity et issue sont tronqués en entiers.
- basis est arrondi à l'entier le plus proche.
- Une erreur est retournée si :
  - settlement, maturity ou issue n'est pas une date valide.
  - maturity > settlement > issue n'est pas respecté.
  - rate < 0.
  - pr ≤ 0.
  - basis < 0 ou basis > 4.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Données	Description
15-Mar-08	Date de règlement
3-Nov-08	Date d'échéance
8-Nov-07	Date d'émission
6,25 %	Taux de coupon semestriel
100,0123	Price
0	Base 30/360 (voir ci-dessus)

La requête DAX suivante :

```DAX
EVALUATE
{
  YIELDMAT(DATE(2008,3,15), DATE(2008,11,3), DATE(2007,11,8), 0.0625,
100.0123, 0)
}
```

Retourne le rendement d'un titre selon les termes spécifiés ci-dessus.

Agrandir le tableau

[Valeur]
0,0609543336915387

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonctions d'information

Article • 20/10/2023

Les fonctions d'information DAX examinent la cellule ou la ligne qui est fournie comme argument, puis indique si la valeur correspond au type attendu. Par exemple, la fonction ISERROR retourne TRUE si la valeur que vous référencez contient une erreur.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
COLUMNSTATISTICS	Retourne une table de statistiques concernant chaque colonne de chaque table du modèle.
CONTAINS	Retourne la valeur true si les valeurs de toutes les colonnes référencées existent ou sont contenues dans ces colonnes ; dans le cas contraire, la fonction retourne la valeur false.
CONTAINSROW	Retourne TRUE si une ligne de valeurs existe ou est contenue dans une table, sinon retourne FALSE.
CONTAINSSTRING	Retourne TRUE ou FALSE indiquant si une chaîne contient une autre chaîne.
CONTAINSSTRINGEXACT	Retourne TRUE ou FALSE indiquant si une chaîne contient une autre chaîne.
CUSTOMDATA	Retourne le contenu de la propriété CustomData dans la chaîne de connexion.
HASONEFILTER	Retourne TRUE quand le nombre de valeurs filtrées directement sur *columnName* est égal à un ; sinon, retourne FALSE.
HASONEVALUE	Retourne TRUE quand il ne reste qu'une valeur distincte après filtrage du contexte pour *columnName*. Sinon, FALSE.
ISAFTER	Fonction booléenne qui émule le comportement d'une clause Start At et retourne la valeur true pour une ligne qui remplit tous les paramètres de condition.
ISBLANK	Vérifie si une valeur est vide et retourne TRUE ou FALSE.
ISCROSSFILTERED	Retourne TRUE quand *columnName* ou une autre colonne dans la même table ou une table associée est filtrée.

Fonction	Description
ISEMPTY	Vérifie si une table est vide.
ISERROR	Vérifie si une valeur est erronée et retourne TRUE ou FALSE.
ISEVEN	Retourne TRUE si le nombre est pair ou FALSE si le nombre est impair.
ISFILTERED	Retourne TRUE quand un filtre est appliqué directement à *columnName*.
ISINSCOPE	Retourne la valeur true quand la colonne spécifiée est le niveau dans une hiérarchie de niveaux.
ISLOGICAL	Vérifie si une valeur est une valeur logique (TRUE ou FALSE) et retourne TRUE ou FALSE.
ISNONTEXT	Vérifie si une valeur n'est pas du texte (les cellules vides ne sont pas du texte), puis retourne TRUE ou FALSE.
ISNUMBER	Vérifie si une valeur est un nombre et retourne TRUE ou FALSE.
ISODD	Retourne TRUE si le nombre est impair ou FALSE si le nombre est pair.
ISONORAFTER	Fonction booléenne qui émule le comportement d'une clause Start At et retourne la valeur true pour une ligne qui remplit tous les paramètres de condition.
ISSELECTEDMEASURE	Utilisée par les expressions d'éléments de calcul pour déterminer si la mesure dans le contexte est l'une des mesures spécifiées dans une liste de mesures.
ISSUBTOTAL	Crée une autre colonne dans une expression SUMMARIZE qui retourne True si la ligne contient des valeurs de sous-total pour la colonne fournie comme argument et False dans l'autre cas.
ISTEXT	Vérifie si une valeur est du texte et retourne TRUE ou FALSE.
NONVISUAL	Marque un filtre de valeur dans une expression SUMMARIZECOLUMNS comme non visuel.
SELECTEDMEASURE	Fonction utilisée par les expressions d'éléments de calcul pour référencer la mesure qui est dans le contexte.
SELECTEDMEASUREFORMATSTRING	Utilisé par les expressions d'éléments de calcul pour récupérer la chaîne de format de la mesure qui est dans le contexte.

Fonction	Description
SELECTEDMEASURENAME	Utilisée par les expressions d'éléments de calcul pour déterminer la mesure qui est dans le contexte par nom.
USERCULTURE	Retourne les paramètres régionaux de l'utilisateur actuel.
USERNAME	Retourne le nom de domaine et le nom d'utilisateur à partir des informations d'identification fournies au système au moment de la connexion.
USEROBJECTID	Retourne l'ID d'objet ou le SID de l'utilisateur actuel.
USERPRINCIPALNAME	Retourne le nom principal de l'utilisateur.

Commentaires

Cette page a-t-elle été utile ? Yes No

COLUMNSTATISTICS

Article • 15/12/2023

Retourne une table de statistiques concernant chaque colonne de chaque table du modèle.

Syntaxe

```DAX
COLUMNSTATISTICS ()
```

Paramètres

Cette fonction ne prend aucun paramètre.

Valeur de retour

Table de statistiques. Chaque ligne de cette table représente une colonne différente dans le modèle. Les colonnes de la table sont :

- **Nom de la table** : Table de la colonne active.
- **Nom de la colonne** : Nom de la colonne active.
- **Min** : Valeur minimale trouvée dans la colonne active.
- **Max** : Valeur maximale trouvée dans la colonne active.
- **Cardinalité** : Nombre de valeurs distinctes trouvées dans la colonne active.
- **Longueur maximale** : Longueur de la chaîne la plus longue trouvée dans la colonne active (applicable uniquement pour les colonnes de chaîne).

Notes

- Les colonnes dans un état d'erreur et les colonnes des tables calculées requête-étendue n'apparaissent pas dans la table de résultats.
- Si un filtre du contexte de filtre est appliqué à COLUMNSTATISTICS(), une erreur est retournée.

Exemple

Les exemples de cet article peuvent être utilisés avec l'exemple de modèle Adventure Works DW 2020 Power BI Desktop. Pour obtenir le modèle, consultez Exemple de modèle DAX.

La requête DAX suivante :

DAX

```
DEFINE
    TABLE FilteredProduct =
        FILTER (
            Product,
            [Color] == "Blue"
        )
    COLUMN Customer[Location] = [State-Province] & " " & [Country-Region]

EVALUATE
COLUMNSTATISTICS ()
```

Retourne une table avec des statistiques concernant toutes les colonnes de toutes les tables du modèle. La table comprend également des statistiques pour la colonne calculée requête-étendue, Customer[Location]. Par contre, la table ne comprend pas les colonnes de la table calculée requête-étendue, FilteredProduct.

Table Name	Column Name	Min	Max	Cardinality	Max Length
Customer	RowNumber-2662979B-1795-4F74-8F37-6A1BA8059B61			18485	
Customer	CustomerKey	-1	29483	18485	
Customer	Customer ID	[Not Applicable]	AW00029483	18485	16
Customer	Customer	[Not Applicable]	Zoe Watson	18401	26
Customer	City	[Not Applicable]	York	270	21
Customer	State-Province	[Not Applicable]	Yveline	54	19
Customer	Country-Region	[Not Applicable]	United States	7	16
Customer	Postal Code	[Not Applicable]	YO15	324	16
Customer	Location	[Not Applicable] [Not Applicable]	Yveline France	54	33
Date	RowNumber-2662979B-1795-4F74-8F37-6A1BA8059B61			1461	
Date	DateKey	20170701	20210630	1461	
Date	Date	7/1/2017 12:00:00 AM	6/30/2021 12:00:00 AM	1461	
Date	Fiscal Year	FY2018	FY2021	4	6
Date	Fiscal Quarter	FY2018 Q1	FY2021 Q4	16	9
Date	Month	2017 Aug	2021 May	48	8
Date	MonthKey	201707	202106	48	
Date	Full Date	2017 Aug, 01	2021 May, 31	1461	12
Sales Territory	RowNumber-2662979B-1795-4F74-8F37-6A1BA8059B61			11	
Sales Territory	SalesTerritoryKey	1	11	11	
Sales Territory	SalesTerritoryRegion	Aust	United Kingdom	11	

Contenu connexe

Contexte de filtre
CALCULATETABLE, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

CONTAINS

Article • 20/10/2023

Retourne la valeur TRUE si les valeurs de toutes les colonnes référencées existent ou sont contenues dans ces colonnes ; sinon, la fonction retourne la valeur FALSE.

Syntaxe

DAX

```
CONTAINS(<table>, <columnName>, <value>[, <columnName>, <value>]…)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Toute expression DAX qui retourne une table de données.
columnName	Nom d'une colonne existante, spécifié avec la syntaxe DAX standard. Il ne peut pas s'agir d'une expression.
valeur	Toute expression DAX qui retourne une valeur scalaire unique à rechercher dans *columnName*. L'expression doit être évaluée une seule fois et avant d'être passée à la liste d'arguments.

Valeur de retour

Valeur **TRUE** si chaque valeur (*value*) spécifiée peut être trouvée ou est contenue dans la colonne (*columnName*) correspondante ; sinon, la fonction retourne **FALSE**.

Remarques

- Les arguments *columnName* et *value* doivent être fournis ensemble ; sinon, une erreur est retournée.
- *columnName* doit appartenir à la *table* spécifiée ou à une autre table qui est associée à cette *table*.

- Si *columnName* référence une colonne d'une table associée, le nom complet doit être fourni ; sinon, une erreur est retournée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant crée une mesure calculée qui indique si des ventes sur Internet ont été enregistrées à la fois pour le produit 214 et le client 11185.

DAX

```
= CONTAINS(InternetSales, [ProductKey], 214, [CustomerKey], 11185)
```

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonction CONTAINSROW

Article • 20/10/2023

Renvoie TRUE s'il existe au moins une ligne où toutes les colonnes ont des valeurs spécifiées.

Syntaxe

DAX

```
CONTAINSROW(<Table>, <Value> [, <Value> [, …] ] )
```

Paramètres

Agrandir le tableau

Terme	Définition
Table de charge de travail	Un tableau à tester.
Valeur	Toute expression DAX valide qui retourne une valeur scalaire.

Valeur renvoyée

TRUE ou FALSE.

Notes

- À l'exception de leur syntaxe, l'opérateur **IN** et la fonction CONTAINSROW sont équivalents sur un plan fonctionnel.

  DAX

  ```
  <scalarExpr> IN <tableExpr>
  ( <scalarExpr1>, <scalarExpr2>, … ) IN <tableExpr>
  ```

  - Le nombre de scalarExprN doit correspondre au nombre de colonnes dans tableExpr.
  - NOT IN n'est pas un opérateur dans DAX. Pour effectuer la négation logique de l'opérateur IN, placez NOT devant l'expression entière. Par exemple, NOT [Color]

IN { "Red", "Yellow", "Blue" }.

- Contrairement à l'opérateur =, l'opérateur IN et la fonction CONTAINSROW effectuent une comparaison stricte. Par exemple, la valeur BLANK ne correspond pas à 0.

Exemples

Les exemples de cet article peuvent être utilisés avec l'exemple de modèle Adventure Works DW 2020 Power BI Desktop. Pour obtenir le modèle, consultez Exemple de modèle DAX.

Exemple 1

Les requêtes DAX suivantes :

DAX

```
EVALUATE
FILTER (
    ALL ( Product[Color] ),
    ( [Color] )
        IN {
        "Red",
        "Yellow",
        "Blue"
    }
)
ORDER BY [Color]
```

et

DAX

```
EVALUATE
FILTER (
    ALL ( Product[Color] ),
    CONTAINSROW (
        {
            "Red",
            "Yellow",
            "Blue"
        },
        [Color]
    )
)
ORDER BY [Color]
```

Retournent la table suivante avec une seule colonne :

Agrandir le tableau

[Color]
Bleu
Rouge
Jaune

Exemple 2

Les requêtes DAX équivalentes suivantes :

DAX

```
EVALUATE
FILTER (
    ALL ( Product[Color] ),
    NOT [Color]
        IN {
        "Red",
        "Yellow",
        "Blue"
    }
)
ORDER BY [Color]
```

et

DAX

```
EVALUATE
FILTER (
    ALL ( Product[Color] ),
    NOT CONTAINSROW (
        {
            "Red",
            "Yellow",
            "Blue"
        },
        [Color]
    )
)
ORDER BY [Color]
```

Retournent la table suivante avec une seule colonne :

Agrandir le tableau

[Color]
Noir
Grey
Multiple
N/D
Argent
Silver\Black
White

Contenu connexe

opérateur IN
Requêtes DAX

Commentaires

Cette page a-t-elle été utile ? Yes No

CONTAINSSTRING

Article • 20/10/2023

Retourne TRUE ou FALSE indiquant si une chaîne contient une autre chaîne.

Syntaxe

```
DAX

CONTAINSSTRING(<within_text>, <find_text>)
```

Paramètres

Agrandir le tableau

Terme	Définition
within_text	Texte dans lequel vous voulez rechercher find_text.
find_text	Texte à rechercher.

Valeur renvoyée

TRUE si find_text est une sous-chaîne de within_text ; sinon, FALSE.

Notes

- CONTAINSSTRING ne respecte pas la casse.
- Vous pouvez utiliser les caractères génériques `?` et `*`. Utilisez `~` pour échapper les caractères génériques.

Exemple

Requête DAX

```
DAX

EVALUATE
    ROW(
```

```
        "Case 1", CONTAINSSTRING("abcd", "bc"),
        "Case 2", CONTAINSSTRING("abcd", "BC"),
        "Case 3", CONTAINSSTRING("abcd", "a*d"),
        "Case 4", CONTAINSSTRING("abcd", "ef")
    )
```

retourne :

Agrandir le tableau

[Cas 1]	[Cas 2]	[Cas 3]	[Cas 4]
VRAI	VRAI	VRAI	FALSE

Commentaires

Cette page a-t-elle été utile ? Yes No

CONTAINSSTRINGEXACT

Article • 20/10/2023

Retourne TRUE ou FALSE indiquant si une chaîne contient une autre chaîne.

Syntaxe

DAX

```
CONTAINSSTRINGEXACT(<within_text>, <find_text>)
```

Paramètres

Agrandir le tableau

Terme	Définition
within_text	Texte dans lequel vous voulez rechercher find_text.
find_text	Texte à rechercher.

Valeur renvoyée

TRUE si find_text est une sous-chaîne de within_text ; sinon, FALSE.

Notes

CONTAINSSTRINGEXACT est sensible à la casse.

Exemple

Requête DAX

DAX

```
EVALUATE
    ROW(
        "Case 1", CONTAINSSTRINGEXACT("abcd", "bc"),
        "Case 2", CONTAINSSTRINGEXACT("abcd", "BC"),
        "Case 3", CONTAINSSTRINGEXACT("abcd", "a*d"),
```

```
        "Case 4", CONTAINSSTRINGEXACT("abcd", "ef")
    )
```

retourne :

Agrandir le tableau

[Cas 1]	[Cas 2]	[Cas 3]	[Cas 4]
VRAI	FALSE	FAUX	FALSE

Commentaires

Cette page a-t-elle été utile ? Yes No

CUSTOMDATA

Article • 20/10/2023

Retourne le contenu de la propriété **CustomData** dans la chaîne de connexion.

Syntaxe

```DAX
CUSTOMDATA()
```

Valeur de retour

Contenu de la propriété **CustomData** dans la chaîne de connexion.

Vide si la propriété **CustomData** n'a pas été définie au moment de la connexion.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La formule DAX suivante vérifie si la propriété CustomData a été définie sur **"OK"** .

```DAX
= IF(CUSTOMDATA()="OK", "Correct Custom data in connection string", "No
custom data in connection string property or unexpected value")
```

Commentaires

Cette page a-t-elle été utile ? Yes No

HASONEFILTER

Article • 20/10/2023

Retourne **TRUE** quand le nombre de valeurs directement filtrées sur *columnName* est égal à un ; sinon, retourne **FALSE**.

Syntaxe

DAX

```
HASONEFILTER(<columnName>)
```

Paramètres

Agrandir le tableau

Terme	Définition
columnName	Nom d'une colonne existante, spécifié avec la syntaxe DAX standard. Il ne peut pas s'agir d'une expression.

Valeur de retour

TRUE quand le nombre de valeurs directement filtrées sur *columnName* est égal à un ; sinon, retourne **FALSE**.

Notes

- Cette fonction est similaire à HASONEVALUE(), à la différence que HASONEVALUE() fonctionne selon un filtrage croisé alors que HASONEFILTER() fonctionne par un filtre direct.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant montre comment utiliser HASONEFILTER() afin de retourner le filtre pour ResellerSales_USD[ProductKey]) s'il existe un filtre, ou afin de retourner BLANK s'il n'y a pas de filtre ou s'il existe plusieurs filtres sur ResellerSales_USD[ProductKey]).

DAX

```
=
IF(HASONEFILTER(ResellerSales_USD[ProductKey]),FILTERS(ResellerSales_USD[Pro
ductKey]),BLANK())
```

Commentaires

Cette page a-t-elle été utile ? Yes No

HASONEVALUE

Article • 20/10/2023

Retourne **TRUE** quand il ne reste qu'une valeur distincte après filtrage du contexte pour *columnName*. Sinon, **FALSE**.

Syntaxe

```
HTML
HASONEVALUE(<columnName>)
```

Paramètres

Agrandir le tableau

Terme	Définition
columnName	Nom d'une colonne existante, spécifié avec la syntaxe DAX standard. Il ne peut pas s'agir d'une expression.

Valeur de retour

TRUE quand il ne reste qu'une valeur distincte après filtrage du contexte pour *columnName*. Sinon, **FALSE**.

Remarques

- Une expression équivalente pour HASONEVALUE() est `COUNTROWS(VALUES(<columnName>)) = 1`.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La formule de mesure suivante vérifie si le contexte est découpé par une valeur afin d'estimer un pourcentage par rapport à un scénario prédéfini. Dans ce cas, vous souhaitez comparer les ventes du revendeur aux ventes de 2007, puis vous devez savoir si le contexte est filtré par des années individuelles. En outre, si la comparaison n'a pas de sens, vous souhaitez retourner BLANK.

DAX

```
=
IF(HASONEVALUE(DateTime[CalendarYear]),SUM(ResellerSales_USD[SalesAmount_USD
])/CALCULATE(SUM(ResellerSales_USD[SalesAmount_USD]),DateTime[CalendarYear]=
2007),BLANK())
```

Commentaires

Cette page a-t-elle été utile ? Yes No

ISAFTER

Article • 20/10/2023

Fonction booléenne qui émule le comportement d'une clause « Start At » et retourne true pour une ligne qui remplit tous les paramètres de condition.

En fonction de l'ordre de tri, le premier paramètre est comparé au second. Si l'ordre de tri est croissant, la comparaison à effectuer est la suivante : premier paramètre supérieur au second. Si l'ordre de tri est décroissant, la comparaison à effectuer est la suivante : second paramètre inférieur au premier.

Syntaxe

DAX

```
ISAFTER(<scalar_expression>, <scalar_expression>[, sort_order [,
<scalar_expression>, <scalar_expression>[, sort_order]]…)
```

Paramètres

Agrandir le tableau

Terme	Définition
scalar_expression	Toute expression qui retourne une valeur scalaire comme une référence de colonne, un entier ou une valeur de chaîne. En général, le premier paramètre est une référence de colonne et le deuxième est une valeur scalaire.
sort_order	(Facultatif) Ordre de tri de la colonne. Il peut être croissant (ASC) ou décroissant (DEC). Par défaut, l'ordre de tri est croissant.

Valeur de retour

True ou False

Notes

Cette fonction est similaire à ISONORAFTER. La différence est que la fonction ISAFTER retourne true pour les valeurs triées de façon stricte *après* les valeurs de filtre, tandis que

la fonction ISONORAFTER retourne la valeur true pour les valeurs triées *sur ou après* les valeurs de filtre.

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Nom de la table : 'Info'

Agrandir le tableau

Pays/région	State	Count	Total
IND	JK	20	800
IND	MH	25	1 000
IND	WB	10	900
États-Unis	CA	5	500
États-Unis	WA	10	900

L'expression suivante :

DAX

```
FILTER (
    Info,
    ISAFTER (
        Info[Country], "IND", ASC,
        Info[State], "MH", ASC )
)
```

Retourne les informations suivantes :

Agrandir le tableau

Pays ou région	State	Count	Total
IND	WB	10	900
États-Unis	CA	5	500
États-Unis	WA	10	900

Contenu connexe

ISONORAFTER

Commentaires

Cette page a-t-elle été utile ? Yes No

ISBLANK

Article • 14/12/2023

Vérifie si une valeur est vide et retourne TRUE ou FALSE.

Syntaxe

```
DAX

ISBLANK(<value>)
```

Paramètres

Agrandir le tableau

Terme	Définition
valeur	Valeur ou expression à vérifier.

Valeur renvoyée

Valeur booléenne TRUE si la valeur est vide ; sinon, FALSE.

Remarques

Pour en savoir plus sur les meilleures pratiques lorsque vous travaillez avec des valeurs vides (BLANK), consultez Éviter de convertir des valeurs vides (BLANK) en valeurs dans DAX.

Exemple

Cette formule calcule le taux d'augmentation ou de baisse des ventes par rapport à l'année précédente. L'exemple utilise la fonction IF pour vérifier la valeur des ventes de l'année précédente afin d'éviter une erreur de division par zéro.

```
DAX

//Sales to Previous Year Ratio
```

```
= IF( ISBLANK('CalculatedMeasures'[PreviousYearTotalSales])
   , BLANK()
   , ( 'CalculatedMeasures'[Total Sales]-
'CalculatedMeasures'[PreviousYearTotalSales] )
      /'CalculatedMeasures'[PreviousYearTotalSales])
```

Result,

Agrandir le tableau

Étiquettes de ligne	Total Sales	Total Sales Previous Year	Sales to Previous Year Ratio
2005	$10,209,985.08		
2006	$28,553,348.43	$10,209,985.08	179.66%
2007	$39,248,847.52	$28,553,348.43	37.46%
2008	$24,542,444.68	$39,248,847.52	-37.47%
Total général	$102,554,625.71		

Contenu connexe

Fonctions d'information

Commentaires

Cette page a-t-elle été utile ? Yes No

ISCROSSFILTERED

Article • 14/12/2023

Retourne TRUE si un filtre croisé est appliqué à la table ou la colonne spécifiée.

Syntaxe

DAX

```
ISCROSSFILTERED(<TableNameOrColumnName>)
```

Paramètres

Agrandir le tableau

Terme	Définition
TableNameOrColumnName	Nom d'une table ou d'une colonne existante. Il ne peut pas s'agir d'une expression.

Valeur de retour

TRUE quand *ColumnName* ou une colonne de *TableName* a un filtrage croisé. Sinon, retourne la valeur FALSE.

Notes

- Une colonne ou une table est dite à filtrage croisé quand un filtre est appliqué à *ColumnName*, à une colonne de *TableName* ou à une colonne d'une table associée.
- Une colonne ou une table est dite filtrée directement quand un filtre est appliqué à *ColumnName* ou à une colonne de *TableName*. Par conséquent, la fonction ISFILTERED retourne également TRUE quand *ColumnName* ou une colonne de *TableName* sont filtrés.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Contenu connexe

ISFILTERED, fonction
FILTERS, fonction
HASONEFILTER, fonction
HASONEVALUE, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

ISEMPTY

Article • 20/10/2023

Vérifie si une table est vide.

Syntaxe

```DAX
ISEMPTY(<table_expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
table_expression	Référence de table ou expression DAX qui retourne une table.

Valeur de retour

TRUE si la table est vide (ne contient aucune ligne) ; sinon, FALSE.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Pour la table ci-dessous nommée « Info » :

Agrandir le tableau

Pays/région	State	County	Total
IND	JK	20	800

Pays/région	State	County	Total
IND	MH	25	1 000
IND	WB	10	900
États-Unis	CA	5	500
États-Unis	WA	10	900

DAX

```
EVALUATE
ROW("Any countries with count > 25?", NOT(ISEMPTY(FILTER(Info,
[County]>25))))
```

Valeur de retour : **FALSE**

Commentaires

Cette page a-t-elle été utile ? Yes No

ISERROR

Article • 14/12/2023

Vérifie si une valeur est erronée et retourne TRUE ou FALSE.

Syntaxe

```
DAX

ISERROR(<value>)
```

Paramètres

Agrandir le tableau

Terme	Définition
valeur	Valeur à tester.

Valeur renvoyée

Valeur booléenne TRUE si la valeur est erronée ; sinon, FALSE.

Notes

- Pour connaître les meilleures pratiques lors de l'utilisation de ISERROR, consultez Utilisation appropriée des fonctions d'erreur.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant calcule le ratio du total des ventes sur Internet par rapport au total des ventes du revendeur. La fonction ISERROR est utilisée pour rechercher des erreurs, telles que la division par zéro. En cas d'erreur, une valeur vide est retournée ; sinon, le ratio est retourné.

DAX

```
= IF( ISERROR(
       SUM('ResellerSales_USD'[SalesAmount_USD])
       /SUM('InternetSales_USD'[SalesAmount_USD])
            )
    , BLANK()
    , SUM('ResellerSales_USD'[SalesAmount_USD])
      /SUM('InternetSales_USD'[SalesAmount_USD])
    )
```

Contenu connexe

Fonctions d'information

IFERROR, fonction

IF, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

ISEVEN

Article • 20/10/2023

Retourne TRUE si le nombre est pair ou FALSE si le nombre est impair.

Syntaxe

DAX

```
ISEVEN(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Valeur à tester. S'il ne s'agit pas d'un entier, elle est tronquée.

Valeur renvoyée

Retourne TRUE si le nombre est pair ou FALSE si le nombre est impair.

Notes

- Si nombre n'est pas numérique, ISEVEN retourne la valeur d'erreur #NUM!.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Commentaires

Cette page a-t-elle été utile ? Yes No

ISFILTERED

Article • 20/10/2023

Retourne TRUE quand la table ou la colonne spécifiée est filtrée directement.

Syntaxe

```
DAX

ISFILTERED(<TableNameOrColumnName>)
```

Paramètres

Agrandir le tableau

Terme	Définition
TableNameOrColumnName	Nom d'une table ou d'une colonne existante. Il ne peut pas s'agir d'une expression.

Valeur de retour

TRUE quand *ColumnName* ou une colonne de *TableName* est filtrée directement. Sinon, retourne la valeur FALSE.

Notes

- Une colonne ou une table est dite filtrée directement quand un filtre est appliqué à *ColumnName* ou à une colonne de *TableName*.
- Une colonne ou une table est dite à filtrage croisé quand un filtre est appliqué à *ColumnName*, à une colonne de *TableName* ou à une colonne d'une table associée. Par conséquent, la fonction ISCROSSFILTERED retourne également TRUE quand *ColumnName*, une colonne de *TableName* ou une colonne d'une table associée sont filtrés.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Contenu connexe

ISCROSSFILTERED, fonction
FILTERS, fonction
HASONEFILTER, fonction
HASONEVALUE, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

ISINSCOPE

Article • 14/12/2023

Retourne la valeur true quand la colonne spécifiée est le niveau dans une hiérarchie de niveaux.

Syntaxe

DAX

```
ISINSCOPE(<columnName>)
```

Paramètres

Agrandir le tableau

Terme	Définition
columnName	Nom d'une colonne existante, spécifié avec la syntaxe DAX standard. Il ne peut pas s'agir d'une expression.

Valeur de retour

TRUE quand la colonne spécifiée est le niveau dans une hiérarchie de niveaux.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

DAX

```
DEFINE
MEASURE FactInternetSales[% of Parent] =
  SWITCH (TRUE(),
    ISINSCOPE(DimProduct[Subcategory]),
      DIVIDE(
        SUM(FactInternetSales[Sales Amount]),
        CALCULATE(
          SUM(FactInternetSales[Sales Amount]),
          ALLSELECTED(DimProduct[Subcategory]))
```

```
      ),
    ISINSCOPE(DimProduct[Category]),
      DIVIDE(
        SUM(FactInternetSales[Sales Amount]),
        CALCULATE(
          SUM(FactInternetSales[Sales Amount]),
          ALLSELECTED(DimProduct[Category]))
      ),
    1
  ) * 100
EVALUATE
  SUMMARIZECOLUMNS
  (
    ROLLUPADDISSUBTOTAL
    (
      DimProduct[Category], "Category Subtotal",
      DimProduct[Subcategory], "Subcategory Subtotal"
    ),
    TREATAS(
      {"Bike Racks", "Bike Stands", "Mountain Bikes", "Road Bikes", "Touring
Bikes"},
      DimProduct[Subcategory]),
    "Sales", SUM(FactInternetSales[Sales Amount]),
    "% of Parent", [% of Parent]
  )
  ORDER BY
    [Category Subtotal] DESC, [Category],
    [Subcategory Subtotal] DESC, [Subcategory]
```

Retourne :

Agrandir le tableau

DimProduct[Category]	DimProduct[SubCategory]	[Category Subtotal]	[Subcategory Subtotal]	[Sales]	[% of Parent]
		TRUE	TRUE	28 397 095,65	100,00
Accessories		FAUX	true	78 951,00	0,28
Accessories	Porte-vélos	FALSE	FALSE	39 360,00	49,85
Accessories	Supports à vélos	FALSE	FALSE	39 591,00	50.15
Bikes		false	true	28 318 144,65	99,72
Bikes	VTT	FALSE	FALSE	9 952 759,56	35,15
Bikes	Vélos de route	FALSE	FALSE	14 520 584,04	51,28
Bikes	Vélos de tourisme	FALSE	FALSE	3 844 801,05	13,58

Contenu connexe

SUMMARIZECOLUMNS
CALCULATE, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

ISLOGICAL

Article • 14/12/2023

Vérifie si une valeur est une valeur logique (TRUE ou FALSE) et retourne TRUE ou FALSE.

Syntaxe

DAX

```
ISLOGICAL(<value>)
```

Paramètres

Agrandir le tableau

Terme	Définition
valeur	Valeur à tester.

Valeur renvoyée

TRUE si la valeur est une valeur logique ; FALSE pour toute valeur autre que TRUE ou FALSE.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Les trois exemples suivants illustrent le comportement de la fonction ISLOGICAL.

DAX

```
//RETURNS: Is Boolean type or Logical
= IF(ISLOGICAL(true), "Is Boolean type or Logical", "Is different type")
```

```
//RETURNS: Is Boolean type or Logical
= IF(ISLOGICAL(false), "Is Boolean type or Logical", "Is different type")

//RETURNS: Is different type
= IF(ISLOGICAL(25), "Is Boolean type or Logical", "Is different type")
```

Contenu connexe

Fonctions d'information

Commentaires

Cette page a-t-elle été utile ? Yes No

ISNONTEXT

Article • 20/10/2023

Vérifie si une valeur n'est pas du texte (les cellules vides ne sont pas du texte), puis retourne TRUE ou FALSE.

Syntaxe

DAX

```
ISNONTEXT(<value>)
```

Paramètres

Agrandir le tableau

Terme	Définition
valeur	Valeur à vérifier.

Valeur renvoyée

TRUE si la valeur n'est pas du texte ou si elle est vide ; FALSE si la valeur est du texte.

Notes

- Une chaîne vide est considérée comme du texte.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Les exemples suivants illustrent le comportement de la fonction ISNONTEXT.

DAX

```
//RETURNS: Is Non-Text
= IF(ISNONTEXT(1), "Is Non-Text", "Is Text")

//RETURNS: Is Non-Text
= IF(ISNONTEXT(BLANK()), "Is Non-Text", "Is Text")

//RETURNS: Is Text
= IF(ISNONTEXT(""), "Is Non-Text", "Is Text")
```

Contenu connexe

Fonctions d'information

Commentaires

Cette page a-t-elle été utile ? Yes No

ISNUMBER

Article • 14/12/2023

Vérifie si une valeur est un nombre et retourne TRUE ou FALSE.

Syntaxe

DAX

```
ISNUMBER(<value>)
```

Paramètres

Agrandir le tableau

Terme	Définition
valeur	Valeur à tester.

Valeur renvoyée

TRUE si la valeur est numérique ; sinon, FALSE.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Les trois exemples suivants illustrent le comportement de la fonction ISNUMBER.

DAX

```
//RETURNS: Is number
= IF(ISNUMBER(0), "Is number", "Is Not number")

//RETURNS: Is number
= IF(ISNUMBER(3.1E-1),"Is number", "Is Not number")
```

```
//RETURNS: Is Not number
= IF(ISNUMBER("123"), "Is number", "Is Not number")
```

Contenu connexe

Fonctions d'information

Commentaires

Cette page a-t-elle été utile ? Yes No

ISODD

Article • 20/10/2023

Retourne TRUE si le nombre est impair ou FALSE si le nombre est pair.

Syntaxe

```
DAX

ISODD(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Valeur à tester. S'il ne s'agit pas d'un entier, elle est tronquée.

Valeur renvoyée

Retourne TRUE si le nombre est impair ou FALSE si le nombre est pair.

Notes

- Si le paramètre number n'est pas numérique, ISODD retourne la valeur d'erreur #VALUE! #NUM!.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Commentaires

Cette page a-t-elle été utile ? Yes No

ISONORAFTER

Article • 20/10/2023

Fonction booléenne qui émule le comportement d'une clause Start At et retourne la valeur true pour une ligne qui remplit tous les paramètres de condition.

En fonction de l'ordre de tri, le premier paramètre est comparé au second. Si l'ordre de tri est croissant, la comparaison à effectuer est la suivante : premier paramètre supérieur au second. Si l'ordre de tri est décroissant, la comparaison à effectuer est la suivante : second paramètre inférieur au premier.

Syntaxe

DAX

```
ISONORAFTER(<scalar_expression>, <scalar_expression>[, sort_order [,
<scalar_expression>, <scalar_expression>[, sort_order]]…)
```

Paramètres

Agrandir le tableau

Terme	Définition
scalar_expression	Toute expression qui retourne une valeur scalaire comme une référence de colonne, un entier ou une valeur de chaîne. En général, le premier paramètre est une référence de colonne et le deuxième est une valeur scalaire.
sort_order	(Facultatif) Ordre de tri de la colonne. Peut être ascendant (ASC) ou descendant (DESC). Par défaut, l'ordre de tri est croissant.

Valeur de retour

True ou False

Notes

Cette fonction est similaire à ISAFTER. La différence est que la fonction ISONORAFTER retourne la valeur true pour les valeurs triées *sur ou après* les valeurs de filtre, tandis que

la fonction ISAFTER retourne la valeur true pour les valeurs triées strictement *après* les valeurs de filtre.

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Pour le tableau suivant nommé, Info :

Agrandir le tableau

Pays/région	State	Count	Total
IND	JK	20	800
IND	MH	25	1 000
IND	WB	10	900
États-Unis	CA	5	500
États-Unis	WA	10	900

L'expression suivante :

DAX

```
FILTER (
    Info,
    ISONORAFTER (
        Info[Country], "IND", ASC,
        Info[State], "MH", ASC )
)
```

Retourne les informations suivantes :

Agrandir le tableau

Pays/région	State	Count	Total
IND	MH	25	1 000
IND	WB	10	900
États-Unis	CA	5	500

Pays/région	State	Count	Total
États-Unis	WA	10	900

Contenu connexe

ISAFTER

Commentaires

Cette page a-t-elle été utile ? Yes No

ISSELECTEDMEASURE

Article • 20/10/2023

Utilisée par les expressions d'éléments de calcul pour déterminer si la mesure dans le contexte est l'une des mesures spécifiées dans une liste de mesures.

Syntaxe

DAX

```
ISSELECTEDMEASURE( M1, M2, ... )
```

Paramètres

Agrandir le tableau

Terme	Définition
M1, M2, ...	Liste de mesures.

Valeur de retour

Valeur booléenne indiquant si la mesure actuellement dans le contexte est l'une des mesures spécifiées dans la liste de paramètres.

Remarques

- Ne peut être référencée que dans l'expression d'un élément de calcul.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'expression d'élément de calcul suivante vérifie si la mesure actuelle est l'une des mesures spécifiées dans la liste de paramètres. Si les mesures sont renommées, la correction de la formule reflète les changements de nom dans l'expression.

DAX

```
IF (
    ISSELECTEDMEASURE ( [Expense Ratio 1], [Expense Ratio 2] ),
    SELECTEDMEASURE (),
    DIVIDE ( SELECTEDMEASURE (), COUNTROWS ( DimDate ) )
)
```

Contenu connexe

SELECTEDMEASURE
SELECTEDMEASURENAME

Commentaires

Cette page a-t-elle été utile ? Yes No

ISSUBTOTAL

Article • 20/10/2023

Crée une autre colonne dans une expression SUMMARIZE qui retourne True si la ligne contient des valeurs de sous-total pour la colonne fournie comme argument, False dans l'autre cas.

Syntaxe

DAX

```
ISSUBTOTAL(<columnName>)
```

Avec SUMMARIZE,

DAX

```
SUMMARIZE(<table>, <groupBy_columnName>[, <groupBy_columnName>]…[,
ROLLUP(<groupBy_columnName>[,< groupBy_columnName>…])][, <name>,
{<expression>|ISSUBTOTAL(<columnName>)}]…)
```

Paramètres

Agrandir le tableau

Terme	Définition
columnName	Nom d'une colonne quelconque dans la table de la fonction SUMMARIZE ou d'une colonne d'une table associée à cette table.

Valeur renvoyée

Valeur True si la ligne contient une valeur de sous-total pour la colonne fournie comme argument, False dans l'autre cas.

Notes

- ISSUBTOTAL peut uniquement être utilisé dans l'expression d'une fonction SUMMARIZE.

- Cette fonction doit être précédée du nom de la colonne booléenne.

Exemple

Consultez SUMMARIZE.

Commentaires

Cette page a-t-elle été utile ? Yes No

ISTEXT

Article • 14/12/2023

Vérifie si une valeur est du texte et retourne TRUE ou FALSE.

Syntaxe

```DAX
ISTEXT(<value>)
```

Paramètres

Agrandir le tableau

Terme	Définition
valeur	Valeur à vérifier.

Valeur renvoyée

TRUE si la valeur est du texte ; sinon, FALSE.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Les exemples suivants illustrent le comportement de la fonction ISTEXT.

```DAX
//RETURNS: Is Text
= IF(ISTEXT("text"), "Is Text", "Is Non-Text")

//RETURNS: Is Text
= IF(ISTEXT(""), "Is Text", "Is Non-Text")
```

```
//RETURNS: Is Non-Text
= IF(ISTEXT(1), "Is Text", "Is Non-Text")

//RETURNS: Is Non-Text
= IF(ISTEXT(BLANK()), "Is Text", "Is Non-Text")
```

Contenu connexe

Fonctions d'information

Commentaires

Cette page a-t-elle été utile ? Yes No

NONVISUAL

Article • 20/10/2023

Marque un filtre de valeur dans une expression SUMMARIZECOLUMNS comme non visuel. Cette fonction s'utilise uniquement dans une expression SUMMARIZECOLUMNS.

Syntaxe

DAX

```
NONVISUAL(<expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
expression	Toute expression DAX qui retourne une seule valeur (et non une table).

Valeur renvoyée

Table de valeurs.

Notes

- Marque un filtre de valeur dans SUMMARIZECOLUMNS comme n'affectant pas les valeurs de mesure, mais uniquement comme s'appliquant à des colonnes groupBy.
- Cette fonction s'utilise uniquement dans une expression SUMMARIZECOLUMNS. Elle est utilisée comme argument filterTable de la fonction SUMMARIZECOLUMNS ou comme argument groupLevelFilter de la fonction ROLLUPADDISSUBTOTAL ou ROLLUPISSUBTOTAL.

Exemple

Consultez SUMMARIZECOLUMNS.

Commentaires

Cette page a-t-elle été utile ? Yes No

SELECTEDMEASURE

Article • 20/10/2023

Fonction utilisée par les expressions pour les éléments de calcul ou les chaînes à format dynamique afin de référencer la mesure qui est dans le contexte.

Syntaxe

```DAX
SELECTEDMEASURE()
```

Paramètres

Aucune

Valeur de retour

Référence à la mesure qui est dans le contexte au moment de l'évaluation de l'élément de calcul ou de la chaîne de format.

Notes

- Ne peut être référencée que dans l'expression pour un élément de calcul ou une chaîne de format.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'expression d'élément de calcul suivante calcule le cumul annuel jusqu'à ce jour pour la mesure qui est dans le contexte.

```DAX
CALCULATE(SELECTEDMEASURE(), DATESYTD(DimDate[Date]))
```

L'expression suivante peut être utilisée pour ajuster dynamiquement la chaîne de format d'une mesure selon qu'une valeur est égale à des centaines, des milliers ou des millions.

DAX

```
SWITCH(
TRUE(),
SELECTEDMEASURE() < 1000,"$#,##0",            //Values less than 1000 have
no text after them
SELECTEDMEASURE() < 1000000, "$#,##0,.0 K",   //Values between 1000 and
1000000 are formatted as #.## K
"$#,##0,,.0 M"                                //Values greater than 1000000
are formatted as #.## M
)
```

Contenu connexe

SELECTEDMEASURENAME
ISSELECTEDMEASURE

Commentaires

Cette page a-t-elle été utile ? Yes No

SELECTEDMEASUREFORMATSTRING

Article • 20/10/2023

Utilisé par les expressions d'éléments de calcul pour récupérer la chaîne de format de la mesure qui est dans le contexte.

Syntaxe

```
DAX

SELECTEDMEASUREFORMATSTRING()
```

Paramètres

Aucune

Valeur de retour

Chaîne contenant la chaîne de format de la mesure qui est dans le contexte au moment de l'évaluation de l'élément de calcul.

Remarques

- Cette fonction peut être référencée uniquement dans des expressions d'éléments de calcul dans des groupes de calcul. Elle est conçue pour être utilisée par la propriété **Expression de chaîne de format** des éléments de calcul.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'expression suivante est évaluée par la propriété Expression de chaîne de format pour un élément de calcul. S'il y a une seule devise dans le contexte de filtre, la chaîne de format est récupérée de la colonne DimCurrency[FormatString] ; sinon, la chaîne de format de la mesure actuellement dans le contexte est utilisée.

```
```

DAX

```
SELECTEDVALUE( DimCurrency[FormatString], SELECTEDMEASUREFORMATSTRING() )
```

Contenu connexe

SELECTEDMEASURE
ISSELECTEDMEASURE

Commentaires

Cette page a-t-elle été utile ? Yes No

SELECTEDMEASURENAME

Article • 14/12/2023

Utilisée par les expressions d'éléments de calcul pour déterminer la mesure qui est dans le contexte par nom.

Syntaxe

DAX

```
SELECTEDMEASURENAME()
```

Paramètres

Aucune

Valeur de retour

Valeur de chaîne contenant le nom de la mesure actuellement dans le contexte quand l'élément de calcul est évalué.

Remarques

- Ne peut être référencée que dans l'expression d'un élément de calcul.
- Cette fonction est souvent utilisée à des fins de débogage lors de la création de groupes de calcul.

Exemple

L'expression d'élément de calcul suivante vérifie si la mesure actuelle correspond à « Expense Ratio » et applique la logique de calcul de manière conditionnelle. Étant donné que la vérification est basée sur une comparaison de chaînes, elle ne bénéficie pas de la correction de formule et de la répercussion automatique du renommage d'objet. Pour une comparaison similaire bénéficiant de la correction de formule, reportez-vous à la fonction ISSLECTEDMEASURE.

DAX

```
IF (
    SELECTEDMEASURENAME = "Expense Ratio",
    SELECTEDMEASURE (),
    DIVIDE ( SELECTEDMEASURE (), COUNTROWS ( DimDate ) )
)
```

Contenu connexe

SELECTEDMEASURE

ISSELECTEDMEASURE

Commentaires

Cette page a-t-elle été utile ? Yes No

USERCULTURE

Article • 20/10/2023

Retourne les paramètres régionaux (code de langue-pays) de l'utilisateur actuel, déterminés par le système d'exploitation, les paramètres du navigateur ou le service Power BI.

Remarque : Cette fonction est actuellement prise en charge dans Power BI Premium par capacité, Power BI Premium par utilisateur et Power BI Embedded uniquement.

Syntaxe

```DAX
USERCULTURE()
```

Paramètres

Cette expression n'a pas de paramètres.

Valeur de retour

Paramètres régionaux sous forme de chaîne.

Notes

- Dans le service Power BI, les paramètres régionaux sont déterminés par **Paramètres** > **Langue** > **Paramètres de langue**. La valeur par défaut est déterminée par le paramètre de langue du navigateur de l'utilisateur.
- Quand il est utilisé dans les expressions de table calculée et de colonne calculée, le résultat peut différer selon que la table est en mode DirectQuery ou Importation. En mode DirectQuery, le résultat est déterminé par la langue (paramètres régionaux) spécifiée dans Paramètres de langue dans le service Power BI. La valeur par défaut dans Paramètres de langue qui spécifie les paramètres régionaux est déterminée par le paramètre de langue du navigateur de l'utilisateur, ce qui signifie que la même table ou colonne calculée peut retourner des résultats différents en fonction des paramètres de langue du navigateur de chaque utilisateur. En mode Importation, le résultat est déterminé statiquement pendant

l'actualisation et ne varie pas au moment de la requête. Pour les actualisations gérées, c'est-à-dire planifiées ou interactives, les paramètres régionaux ne sont pas basés sur le paramètre de langue du navigateur de l'utilisateur, mais utilisent des paramètres régionaux invariants. Toutefois, les paramètres régionaux invariants peuvent être remplacés en utilisant le point de terminaison XMLA pour spécifier des paramètres régionaux personnalisés.

- Lorsqu'il est combiné à la fonctionnalité Paramètres de champ de Power BI, USERCULTURE peut être utilisé pour traduire de manière fiable les titres et les légendes de la visualisation dynamique lorsqu'ils sont utilisés dans les expressions d'objet RLS et de mesure au sein d'un même modèle. Toutefois, les expressions contenant USERCULTURE qui sont appelées en dehors du modèle, telles que les requêtes et les mesures des rapports Live Connect, ne doivent pas être utilisées pour les titres et les légendes qui ont été bien traduits.

- USERCULTURE retourne les paramètres régionaux utilisateur appropriés lorsqu'ils sont utilisés dans les expressions d'objet appelées à partir du modèle, telles que les mesures, la sécurité au niveau des lignes (RLS) et les éléments de calcul. Toutefois, il peut ne pas retourner les paramètres régionaux utilisateur appropriés lorsqu'ils sont utilisés dans des expressions extérieures au modèle, telles que les requêtes et les mesures des rapports Live Connect.

- Dans les rapports Live Connect, USERCULTURE peut ne pas retourner les paramètres régionaux utilisateur appropriés lorsqu'il est appelé à partir d'une expression de mesure de rapport.

Exemple

Pour l'expression suivante,

DAX

```
FORMAT(TODAY(), "dddd", USERCULTURE())
```

Selon le paramètre de langue de l'utilisateur actuel, USERCULTURE retourne le jour actuel, par exemple :

Agrandir le tableau

Paramètres régionaux	Jour de la semaine mis en forme
de-DE	Dienstag

Paramètres régionaux	Jour de la semaine mis en forme
fr-FR	Tuesday
es-ES_tradnl	martes
eu-ES	asteartea
it-IT	martedì
nl-NL	dinsdag
pl-PL	wtorek
ro-RO	marți
ru-RU	вторник
uk-UA	вівторок

Contenu connexe

Titres basés sur des expressions dans Power BI
USERNAME
USERPRINCIPALNAME
USEROBJECTID

Commentaires

Cette page a-t-elle été utile ? Yes No

USERNAME

Article • 20/10/2023

Retourne le nom de domaine et le nom d'utilisateur à partir des informations d'identification fournies au système au moment de la connexion.

Syntaxe

```DAX
USERNAME()
```

Paramètres

Cette expression n'a pas de paramètres.

Valeur de retour

Nom d'utilisateur provenant des informations d'identification fournies au système au moment de la connexion

Exemple

La formule suivante vérifie si la connexion de l'utilisateur fait partie de UsersTable.

```DAX
= IF(CONTAINS(UsersTable,UsersTable[login], USERNAME()), "Allowed", BLANK())
```

Commentaires

Cette page a-t-elle été utile ? Yes No

USEROBJECTID

Article • 25/10/2023

Retourne l'ID d'objet de l'utilisateur actuel provenant de Microsoft Entra ID ou de l'identificateur de sécurité (SID).

Syntaxe

DAX

```
USEROBJECTID()
```

Paramètres

Cette expression n'a pas de paramètres.

Valeur retournée

ID d'objet de l'utilisateur actuel provenant de modèles Microsoft Entra ID pour Power BI ou Azure Analysis Services ou bien de SID pour modèles SQL Server Analysis Services.

Commentaires

Cette page a-t-elle été utile ? Yes No

USERPRINCIPALNAME

Article • 20/10/2023

Retourne le nom principal de l'utilisateur.

Syntaxe

DAX

```
USERPRINCIPALNAME()
```

Paramètres

Cette expression n’a pas de paramètres.

Valeur de retour

UserPrincipalName au moment de la connexion.

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonctions logiques

Article • 20/10/2023

Les fonctions logiques agissent sur une expression pour retourner des informations sur les valeurs ou les jeux de l'expression. Par exemple, vous pouvez utiliser la fonction IF pour vérifier le résultat d'une expression et créer des résultats conditionnels.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
AND	Vérifie si les deux arguments ont la valeur TRUE, puis retourne TRUE si tel est le cas.
BITAND	Retourne au niveau du bit AND de deux nombres.
BITLSHIFT	Retourne un nombre décalé vers la gauche selon le nombre de bits spécifié.
BITOR	Retourne au niveau du bit OR de deux nombres.
BITRSHIFT	Retourne un nombre décalé vers la droite selon le nombre de bits spécifié.
BITXOR	Retourne au niveau du bit XOR de deux nombres.
COALESCE	Retourne la première expression qui ne donne pas la valeur BLANK (vide).
FALSE	Retourne la valeur logique FALSE.
IF	Vérifie une condition et retourne une valeur si TRUE ; sinon, retourne une deuxième valeur.
IF.EAGER	Vérifie une condition et retourne une valeur si TRUE ; sinon, retourne une deuxième valeur. Utilise un plan d'exécution *eager* qui exécute toujours les expressions de branche, quelle que soit l'expression de condition.
IFERROR	Évalue une expression et retourne une valeur spécifiée si l'expression retourne une erreur
NOT	Change FALSE en TRUE, ou TRUE en FALSE.
OR	Vérifie si l'un des arguments a la valeur TRUE pour retourner TRUE.
SWITCH	Évalue une expression par rapport à une liste de valeurs et retourne l'une des différentes expressions de résultat possibles.
TRUE	Retourne la valeur logique TRUE.

Commentaires

Cette page a-t-elle été utile ? Yes No

AND

Article • 22/01/2024

Vérifie si les deux arguments ont la valeur TRUE, puis retourne TRUE si tel est le cas. Sinon, retourne FALSE.

Syntaxe

```
DAX

AND(<logical1>,<logical2>)
```

Paramètres

Agrandir le tableau

Terme	Définition
logical_1, logical_2	Valeurs logiques à tester.

Valeur de retour

Retourne true ou false en fonction de la combinaison des valeurs testées.

Remarques

La fonction DAX **AND** accepte uniquement deux (2) arguments. Si vous devez effectuer une opération AND sur plusieurs expressions, vous pouvez créer une série de calculs ou, encore mieux, utiliser l'opérateur AND (**&&**) pour toutes les joindre dans une expression plus simple.

Exemple 1

La formule suivante illustre la syntaxe de la fonction AND.

```
DAX

= IF(AND(10 > 9, -10 < -1), "All true", "One or more false"
```

Étant donné que les deux conditions passées comme arguments de la fonction AND ont la valeur true, la formule retourne « All True ».

Exemple 2

L'exemple suivant utilise la fonction AND avec des formules imbriquées pour comparer deux jeux de calculs simultanément. Pour chaque catégorie de produit, la formule détermine si les ventes Internet de l'année en cours et de l'année précédente sont supérieures à celles du réseau de revendeurs sur les mêmes périodes. Si les deux conditions sont vraies, pour chaque catégorie, la formule retourne la valeur « Internet Hit ».

DAX

```
= IF( AND(  SUM( 'InternetSales_USD'[SalesAmount_USD])
           >SUM('ResellerSales_USD'[SalesAmount_USD])
          , CALCULATE(SUM('InternetSales_USD'[SalesAmount_USD]),
PREVIOUSYEAR('DateTime'[DateKey] ))
           >CALCULATE(SUM('ResellerSales_USD'[SalesAmount_USD]),
PREVIOUSYEAR('DateTime'[DateKey] ))
          )
     , "Internet Hit"
     , ""
     )
```

retourne :

Agrandir le tableau

Étiquettes de ligne	2005	2006	2007	2008	-	Total général
Cuissards						
Porte-vélos						
Supports à vélos				Internet Hit		
Bidons et porte-bidons				Internet Hit		
Jeux de pédalier						
Freins						
Casquettes						
Chaînes						
Produits d'entretien						

Étiquettes de ligne	2005	2006	2007	2008	-	Total général
Pédaliers						
Dérailleurs						
Garde-boue				Internet Hit		
Fourches						
Gants						
Guidons						
Jeux de direction						
Casques						
Sacs à eau						
Jerseys						
Feux						
Verrous						
VTT						
Cadres de VTT						
Sacoches						
Pédales						
Pompes						
Vélos de route						
Cadres de vélo de route						
Selles						
Shorts						
Chaussettes						
Collants						
Pneus et chambres à air				Internet Hit		
Vélos de tourisme						
Cadres de vélo de tourisme						

Étiquettes de ligne	2005	2006	2007	2008	-	Total général
Gilets						
Roues						
Total général						

Contenu connexe

Fonctions logiques

Commentaires

Cette page a-t-elle été utile ? Yes No

BITAND

Article • 14/12/2023

Retourne au niveau du bit AND de deux nombres.

Syntaxe

```
DAX

BITAND(<number>, <number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
Nombre	Toute expression scalaire qui retourne un nombre. Si la valeur n'est pas un entier, elle est tronquée.

Valeur retournée

Au niveau du bit AND de deux nombres.

Remarques

- Cette fonction prend en charge les nombres positifs et négatifs.

Exemple

La requête DAX suivante :

```
DAX

EVALUATE { BITAND(13, 11) }
```

Retourne 9.

Contenu connexe

BITLSHIFT
BITRSHIFT
BITOR
BITXOR

Commentaires

Cette page a-t-elle été utile ? Yes No

BITLSHIFT

Article • 20/10/2023

Retourne un nombre décalé vers la gauche selon le nombre de bits spécifié.

Syntaxe

```
DAX

BITLSHIFT(<Number>, <Shift_Amount>)
```

Paramètres

Agrandir le tableau

Terme	Définition
Nombre	Toute expression DAX qui retourne une expression d'entier.
Shift_Amount	Toute expression DAX qui retourne une expression d'entier.

Valeur retournée

Valeur entière.

Remarques

- Veillez à bien comprendre la nature des opérations de décalage de bits et le dépassement de capacité positif ou négatif des entiers avant d'utiliser les fonctions DAX de décalage de bits.
- Si Shift_Amount est négatif, le décalage s'effectue dans la direction opposée.
- Si la valeur absolue de Shift_Amount est supérieure à 64, aucune erreur n'est générée, mais cela entraîne un dépassement de capacité positif/négatif.
- Il n'existe aucune limite sur le nombre, mais le résultat peut correspondre à un dépassement de capacité positif/négatif.

Exemples

Exemple 1

La requête DAX suivante :

```
DAX

EVALUATE
    { BITLSHIFT(2, 3) }
```

Retourne 16.

Exemple 2

La requête DAX suivante :

```
DAX

EVALUATE
    { BITLSHIFT(128, -1) }
```

Retourne 64.

Exemple 3

La requête DAX suivante :

```
DAX

Define
    Measure Sales[LeftShift] = BITLSHIFT(SELECTEDVALUE(Sales[Amount]), 3)

EVALUATE
SUMMARIZECOLUMNS(
    Sales[Amount],
    "LEFTSHIFT",
    [LeftShift]
)
```

Décale vers la gauche chaque montant de vente avec 3 bits et retourne le volume de ventes décalé en bits.

Contenu connexe

BITRSHIFT
BITAND
BITOR
BITXOR

Commentaires

Cette page a-t-elle été utile ? Yes No

BITOR

Article • 14/12/2023

Retourne au niveau du bit OR de deux nombres.

Syntaxe

```
DAX

BITOR(<number>, <number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
Nombre	Toute expression scalaire qui retourne un nombre. Si la valeur n'est pas un entier, elle est tronquée.

Valeur retournée

Au niveau du bit OR de deux nombres.

Remarques

- Cette fonction prend en charge les nombres positifs et négatifs.

Exemple

La requête DAX suivante :

```
DAX

EVALUATE
    { BITOR(9, 10) }
```

Retourne 11.

Contenu connexe

BITAND
BITXOR
BITLSHIFT
BITRSHIFT

Commentaires

Cette page a-t-elle été utile ? Yes No

BITRSHIFT

Article • 14/12/2023

Retourne un nombre décalé vers la droite selon le nombre de bits spécifié.

Syntaxe

DAX

```
BITRSHIFT(<Number>, <Shift_Amount>)
```

Paramètres

Agrandir le tableau

Terme	Définition
Nombre	Toute expression DAX qui retourne une expression d'entier.
Shift_Amount	Toute expression DAX qui retourne une expression d'entier.

Valeur retournée

Valeur entière.

Remarques

- Veillez à bien comprendre la nature des opérations de décalage de bits et le dépassement de capacité positif ou négatif des entiers avant d'utiliser les fonctions DAX de décalage de bits.
- Si Shift_Amount est négatif, le décalage s'effectue dans la direction opposée.
- Si la valeur absolue de Shift_Amount est supérieure à 64, aucune erreur n'est générée, mais cela entraîne un dépassement de capacité positif/négatif.
- Il n'existe aucune limite sur le nombre, mais le résultat peut correspondre à un dépassement de capacité positif/négatif.

Exemples

Exemple 1

La requête DAX suivante :

```
DAX

EVALUATE
    { BITRSHIFT(16, 3) }
```

Retourne 2.

Exemple 2

La requête DAX suivante :

```
DAX

EVALUATE
    { BITRSHIFT(1024, -3) }
```

Retourne 8192.

Exemple 3

La requête DAX suivante :

```
DAX

Define
    Measure Sales[RightShift] = BITRSHIFT(SELECTEDVALUE(Sales[Amount]), 3)

EVALUATE
SUMMARIZECOLUMNS(
    Sales[Amount],
    "RIGHTSHIFT",
    [RightShift]
)
```

Décale vers la droite chaque montant de vente avec 3 bits et retourne le volume de ventes décalé en bits.

Contenu connexe

BITLSHIFT
BITAND
BITOR
BITXOR

Commentaires

Cette page a-t-elle été utile ? Yes No

BITXOR

Article • 14/12/2023

Retourne au niveau du bit XOR de deux nombres.

Syntaxe

```
DAX

BITXOR(<number>, <number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
Nombre	Toute expression scalaire qui retourne un nombre. Si la valeur n'est pas un entier, elle est tronquée.

Valeur retournée

Au niveau du bit XOR de deux nombres.

Remarques

- Cette fonction prend en charge les nombres positifs et négatifs.

Exemple

La requête DAX suivante :

```
DAX

EVALUATE { BITXOR(9, 10) }
```

Retourne 3.

Contenu connexe

BITOR
BITAND
BITLSHIFT
BITRSHIFT

Commentaires

Cette page a-t-elle été utile ? Yes No

COALESCE

Article • 20/10/2023

Retourne la première expression qui ne donne pas la valeur BLANK (vide). Si toutes les expressions prennent la valeur BLANK, la valeur BLANK est retournée.

Syntaxe

DAX

```
COALESCE(<expression>, <expression>[, <expression>]…)
```

Paramètres

Agrandir le tableau

Terme	Définition
expression	Expression DAX qui retourne une expression scalaire.

Valeur de retour

Valeur scalaire provenant de l'une des expressions ou BLANK (vide) si toutes les expressions donnent la valeur BLANK.

Remarques

Les expressions d'entrée peuvent être de différents types de données.

Exemple 1

La requête DAX suivante :

DAX

```
EVALUATE { COALESCE(BLANK(), 10, DATE(2008, 3, 3)) }
```

Retourne `10`, c'est-à-dire la première expression qui ne donne pas la valeur BLANK (vide).

Exemple 2

L'expression DAX suivante :

```
DAX
= COALESCE(SUM(FactInternetSales[SalesAmount]), 0)
```

Retourne la somme de toutes les valeurs de la colonne SalesAmount de la table FactInternetSales, ou `0`. Elle peut être utilisée pour convertir les valeurs BLANK (vides) du total des ventes en `0`.

Commentaires

Cette page a-t-elle été utile ? Yes No

FALSE

Article • 14/12/2023

Retourne la valeur logique FALSE.

Syntaxe

```DAX
FALSE()
```

Valeur de retour

Toujours FALSE.

Remarques

Le mot FALSE est également interprété comme la valeur logique FALSE.

Exemple

La formule retourne la valeur logique FALSE quand la valeur dans la colonne, 'InternetSales_USD'[SalesAmount_USD], est inférieure ou égale à 200000.

```DAX
= IF(SUM('InternetSales_USD'[SalesAmount_USD]) >200000, TRUE(), false())
```

La table suivante montre les résultats quand l'exemple de formule est utilisé avec 'ProductCategory'[ProductCategoryName] dans Étiquettes de ligne et 'DateTime'[CalendarYear] dans Étiquettes de colonne.

Agrandir le tableau

Étiquettes de ligne	2005	2006	2007	2008	-	Total général
Accessories	FAUX	FAUX	VRAI	VRAI	FAUX	VRAI
Bikes	VRAI	VRAI	VRAI	VRAI	FAUX	VRAI

Étiquettes de ligne	2005	2006	2007	2008	-	Total général
Clothing	FAUX	FAUX	FAUX	FAUX	FAUX	VRAI
Components	FAUX	FAUX	FAUX	FAUX	FAUX	FAUX
	FAUX	FAUX	FAUX	FAUX	FAUX	FAUX
Total général	VRAI	VRAI	VRAI	VRAI	FAUX	VRAI

Contenu connexe

TRUE, fonction
NOT, fonction
IF, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

IF

Article • 20/10/2023

Vérifie une condition et retourne une valeur quand sa valeur est TRUE ; sinon, retourne une deuxième valeur.

Syntaxe

DAX

```
IF(<logical_test>, <value_if_true>[, <value_if_false>])
```

Paramètres

Agrandir le tableau

Terme	Définition
logical_test	Valeur ou expression dont le résultat de l'évaluation peut être TRUE ou FALSE.
value_if_true	Valeur retournée si le test logique est TRUE.
value_if_false	(Facultatif) Valeur retournée si le test logique est FALSE. S'il n'est pas spécifié, la valeur BLANK est retournée.

Valeur renvoyée

value_if_true, **value_if_false** ou BLANK.

Notes

- La fonction IF peut retourner un type de données Variant si **value_if_true** et **value_if_false** sont de types de données différents, mais la fonction tente de retourner un type de données unique si **value_if_true** et **value_if_false** sont tous deux de types de données numériques. Dans ce dernier cas, la fonction IF convertira implicitement les types de données de façon à gérer les deux valeurs.

  Par exemple, la formule `IF(<condition>, TRUE(), 0)` retourne TRUE ou 0, mais la formule `IF(<condition>, 1.0, 0)` retourne uniquement des valeurs décimales,

même si **value_if_false** est du type de données nombre entier. Pour en savoir plus sur la conversion implicite de types de données, consultez Types de données.

- Pour exécuter les expressions de branche indépendamment de l'expression de condition, utilisez plutôt IF EAGER.

Exemples

Les définitions de colonnes calculées de la table **Product** suivantes utilisent la fonction IF de différentes façons pour classifer chaque produit en fonction de son prix catalogue.

Le premier exemple teste si la valeur de la colonne **List Price** est inférieure à 500. Si cette condition est true, la valeur **Low** est retournée. Étant donné qu'il n'y a pas de valeur **value_if_false**, BLANK est retourné.

Les exemples de cet article peuvent être utilisés avec l'exemple de modèle Adventure Works DW 2020 Power BI Desktop. Pour obtenir le modèle, consultez Exemple de modèle DAX.

DAX

```
Price Group =
IF(
    'Product'[List Price] < 500,
    "Low"
)
```

Le deuxième exemple utilise le même test, mais cette fois-ci avec une valeur **value_if_false**. La formule classifie donc chaque produit comme suit : **Low** ou **High**.

DAX

```
Price Group =
IF(
    'Product'[List Price] < 500,
    "Low",
    "High"
)
```

Le troisième exemple utilise le même test, mais cette fois imbrique une fonction IF pour effectuer un test supplémentaire. La formule classifie donc chaque produit comme suit : **Low**, **Medium** ou **High**.

DAX

```
Price Group =
IF(
    'Product'[List Price] < 500,
    "Low",
    IF(
        'Product'[List Price] < 1500,
        "Medium",
        "High"
    )
)
```

Conseil

Si vous devez imbriquer plusieurs fonctions IF, il peut être préférable d'utiliser la fonction **SWITCH**. Cette fonction permet d'écrire plus élégamment une expression qui retourne plus de deux valeurs possibles.

Contenu connexe

IF.EAGER, fonction
SWITCH, fonction (DAX)
Fonctions logiques

Commentaires

Cette page a-t-elle été utile ? Yes No

IF.EAGER

Article • 14/12/2023

Vérifie une condition et retourne une valeur si TRUE ; sinon, retourne une deuxième valeur. Il utilise un plan d'exécution *eager* qui exécute toujours les expressions de branche, quelle que soit l'expression de condition.

Syntaxe

DAX

```
IF.EAGER(<logical_test>, <value_if_true>[, <value_if_false>])
```

Paramètres

Agrandir le tableau

Terme	Définition
logical_test	Valeur ou expression dont le résultat de l'évaluation peut être TRUE ou FALSE.
value_if_true	Valeur retournée si le test logique est TRUE.
value_if_false	(Facultatif) Valeur retournée si le test logique est FALSE. S'il n'est pas spécifié, la valeur BLANK est retournée.

Valeur renvoyée

value_if_true, **value_if_false** ou BLANK.

Notes

- La fonction IF.EAGER peut retourner un type de données Variant si value_if_true et value_if_false sont de types de données différents, mais la fonction tente de retourner un type de données unique si **value_if_true** et **value_if_false** sont tous deux de types de données numériques. Dans ce dernier cas, la fonction IF.EAGER convertira implicitement les types de données de façon à gérer les deux valeurs.

  Par exemple, la formule `IF.EAGER(<condition>, TRUE(), 0)` retourne TRUE ou 0, mais la formule `IF.EAGER(<condition>, 1.0, 0)` retourne uniquement des valeurs

décimales, même si **value_if_false** est du type de données nombre entier. Pour en savoir plus sur la conversion implicite de types de données, consultez Types de données.

- IF.EAGER a le même comportement fonctionnel que la fonction IF, mais les performances peuvent différer en raison de différences dans les plans d'exécution. `IF.EAGER(<logical_test>, <value_if_true>, <value_if_false>)` a le même plan d'exécution que l'expression DAX suivante :

  ```
  DAX

  VAR _value_if_true = <value_if_true>
  VAR _value_if_false = <value_if_false>
  RETURN
  IF (<logical_test>, _value_if_true, _value_if_false)
  ```

  Remarque : Les deux expressions de branche sont évaluées indépendamment de l'expression de condition.

Exemples

Consultez Exemples IF.

Contenu connexe

IF, fonction
Fonctions logiques

Commentaires

Cette page a-t-elle été utile ? Yes No

IFERROR

Article • 14/12/2023

Évalue une expression et retourne une valeur spécifiée si l'expression retourne une erreur. Sinon, retourne la valeur de l'expression elle-même.

Syntaxe

DAX

```
IFERROR(value, value_if_error)
```

Paramètres

Agrandir le tableau

Terme	Définition
valeur	Valeur ou expression quelconque.
value_if_error	Valeur ou expression quelconque.

Valeur de retour

Scalaire du même type que **value**

Remarques

- Vous pouvez utiliser la fonction IFERROR pour intercepter et gérer les erreurs dans une expression.
- Si **value** ou **value_if_error** est une cellule vide, IFERROR la traite comme une valeur de chaîne vide ("").
- La fonction IFERROR est basée sur la fonction IF et utilise les mêmes messages d'erreur. Toutefois, elle a moins d'arguments. La relation entre la fonction IFERROR et la fonction IF est la suivante :

  `IFERROR(A,B) := IF(ISERROR(A), B, A)`

Les valeurs retournées pour A et B doivent être du même type de données. La colonne ou l'expression utilisée pour **value** et la valeur retournée pour **value_if_error** doivent donc être du même type de données.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).
- Pour connaître les meilleures pratiques lors de l'utilisation de IFERROR, consultez Utilisation appropriée des fonctions d'erreur.

Exemple

L'exemple suivant retourne 9999 si l'expression 25/0 donne une erreur. Si l'expression retourne une valeur autre qu'une erreur, cette valeur est passée à l'expression appelante.

DAX

```
= IFERROR(25/0,9999)
```

Contenu connexe

Fonctions logiques

Commentaires

Cette page a-t-elle été utile ? Yes No

NOT

Article • 20/10/2023

Change FALSE en TRUE, ou TRUE en FALSE.

Syntaxe

```
DAX

NOT(<logical>)
```

Paramètres

Agrandir le tableau

Terme	Définition
logical	Valeur ou expression qui peut prendre la valeur TRUE ou FALSE.

Valeur de retour

TRUE OU FALSE.

Exemple

L'exemple suivant récupère des valeurs de la colonne calculée qui a été créée pour illustrer la fonction IF. Pour cet exemple, la colonne calculée a été nommée en utilisant le nom par défaut, **Calculated Column1**, et contient la formule suivante : `= IF([Orders]<300,"true","false")`

La formule vérifie la valeur dans la colonne [Orders] et retourne « true » si le nombre de commandes est inférieur à 300.

À présent, créez une colonne calculée, **Calculated Column2**, puis tapez la formule suivante.

```
DAX

= NOT([CalculatedColumn1])
```

Pour chaque ligne de **Calculated Column1**, les valeurs « true » et « false » sont interprétées comme les valeurs logiques TRUE ou FALSE, et la fonction NOT retourne l'opposé logique de cette valeur.

Contenu connexe

TRUE, fonction
FALSE, fonction
IF, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

OR

Article • 20/10/2023

Vérifie si l'un des arguments a la valeur TRUE pour retourner TRUE. La fonction retourne FALSE si les deux arguments ont la valeur FALSE.

Syntaxe

```
DAX

OR(<logical1>,<logical2>)
```

Paramètres

Agrandir le tableau

Terme	Définition
logical_1, logical_2	Valeurs logiques à tester.

Valeur de retour

Valeur booléenne. La valeur est TRUE si l'un des deux arguments a la valeur TRUE ; la valeur est FALSE si les deux arguments ont la valeur FALSE.

Notes

- La fonction **OR** dans DAX n'accepte que deux (2) arguments. Si vous devez effectuer une opération OR sur plusieurs expressions, vous pouvez créer une série de calculs ou, mieux, utiliser l'opérateur OR (**||**) pour joindre tous les calculs dans une expression plus simple.
- La fonction évalue les arguments jusqu'au premier argument TRUE, puis retourne TRUE.

Exemple

L’exemple suivant montre comment utiliser la fonction OR pour obtenir les vendeurs appartenant au cercle d’excellence (« Circle of Excellence »). Ce cercle regroupe les vendeurs dont les ventes de vélos de cyclotourisme (« Touring Bikes ») totalisent plus d’un million de dollars ou dont les ventes totales dépassent deux millions et demi de dollars en 2007.

DAX

```
IF(   OR(   CALCULATE(SUM('ResellerSales_USD'[SalesAmount_USD]),
'ProductSubcategory'[ProductSubcategoryName]="Touring Bikes") > 1000000
         ,   CALCULATE(SUM('ResellerSales_USD'[SalesAmount_USD]),
'DateTime'[CalendarYear]=2007) > 2500000
         )
   , "Circle of Excellence"
   , ""
   )
```

retourne :

Agrandir le tableau

Étiquettes de ligne	2005	2006	2007	2008	-	Total général
Abbas, Syed E						
Alberts, Amy E						
Ansman-Wolfe, Pamela O						
Blythe, Michael G	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence
Campbell, David R						
Carson, Jillian	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence
Ito, Shu K						
Jiang, Stephen Y						

Étiquettes de ligne	2005	2006	2007	2008	-	Total général
Mensa-Annan, Tete A						
Mitchell, Linda C	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence
Pak, Jae B	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence
Reiter, Tsvi Michael						
Saraiva, José Edvaldo	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence
Tsoflias, Lynn N						
Valdez, Rachel B						
Vargas, Garrett R						
Varkey Chudukatil, Ranjit R						Circle of Excellence
Total général	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence	Circle of Excellence

Contenu connexe

Fonctions logiques

Commentaires

Cette page a-t-elle été utile ? Yes No

SWITCH

Article • 20/10/2023

Évalue une expression par rapport à une liste de valeurs et retourne l'une des différentes expressions de résultat possibles. Cette fonction peut être utilisée pour éviter d'avoir plusieurs instructions SI imbriquées.

Syntaxe

DAX

```
SWITCH(<expression>, <value>, <result>[, <value>, <result>]…[, <else>])
```

Paramètres

Agrandir le tableau

Terme	Définition
expression	Toute expression DAX qui retourne une valeur scalaire unique où l'expression doit être évaluée plusieurs fois (pour chaque ligne/contexte).
value	Valeur constante à mettre en correspondance avec les résultats d'**expression**.
result	Toute expression scalaire à évaluer si les résultats d'**expression** correspondent au paramètre **value** correspondant.
else	Toute expression scalaire à évaluer si les résultats d'**expression** ne correspondent à aucun argument du paramètre **value**.

Valeur renvoyée

S'il existe une correspondance avec une **valeur**, une valeur scalaire du **résultat** correspondant est renvoyée. S'il n'y a pas de correspondance avec une **valeur**, une valeur **d'autre** est retournée. Si aucune des **valeurs** ne correspond et **qu'aucune autre** n'est spécifiée, BLANK est retourné.

Remarques

- **L'expression** à évaluer peut être une valeur constante ou une expression. Une utilisation courante de cette fonction consiste à fixer le premier paramètre à TRUE. Voir les exemples ci-dessous.
- Toutes les expressions de **résultat** et l'expression **else** doivent être du même type de données.
- L'ordre des conditions est important. Dès qu'une **valeur** correspond, le **résultat** correspondant est retourné et les autres **valeurs** suivantes ne sont pas évaluées. Assurez-vous que les **valeurs** les plus restrictives à évaluer sont spécifiées avant les **valeurs** moins restrictives. Voir les exemples ci-dessous.

Exemples

Une utilisation courante de SWITCH consiste à comparer **l'expression** avec des **valeurs** constantes. L'exemple suivant crée une colonne calculée de noms de mois :

DAX

```
= SWITCH (
         [Month Number Of Year],
         1, "January",
         2, "February",
         3, "March",
         4, "April",
         5, "May",
         6, "June",
         7, "July",
         8, "August",
         9, "September",
         10, "October",
         11, "November",
         12, "December",
         "Unknown month number"
        )
```

Une autre utilisation courante de SWITCH consiste à remplacer plusieurs instructions IF imbriquées. Pour ce faire, l'expression doit être définie comme VRAIE, comme le montre l'exemple suivant, qui compare le point de commande et le niveau de stock de sécurité pour les produits afin d'identifier les risques potentiels de rupture de stock :

DAX

```
= SWITCH (
        TRUE,
        [Reorder Point] > [Safety Stock Level], "Good: Safety stock level
exceeded",
        [Reorder Point] = [Safety Stock Level], "Minimal: Safety stock level
```

```
met",
        [Reorder Point] < [Safety Stock Level], "At risk: Safety stock level
not met",
        ISBLANK ( [Reorder Point] ), "Incomplete: Reorder point not set",
        ISBLANK ( [Safety Stock Level] ), "Incomplete: Safety stock level
not set",
        "Unknown"
        )
```

L'ordre des **valeurs** est important. Dans l'exemple suivant, le deuxième **résultat** n'est jamais retourné, car la première valeur est moins restrictive que la seconde. Le résultat de cet exemple est toujours « A » ou « C », mais jamais « B ».

DAX

```
= SWITCH (
         TRUE,
         Product[Standard Cost] < 100, "A",
         Product[Standard Cost] < 10, "B",
         "C"
         )
```

L'instruction suivante retourne une erreur, car les types de données dans les arguments **de résultat** sont différents. N'oubliez pas que les types de données dans tous les arguments **de résultat** et **d'autres** doivent être identiques.

DAX

```
= SWITCH (
        [Class],
        "L", "Large",
        "H", 0.1
        )
```

Commentaires

Cette page a-t-elle été utile ? Yes No

TRUE

Article • 14/12/2023

Retourne la valeur logique TRUE.

Syntaxe

```DAX
TRUE()
```

Valeur de retour

Toujours TRUE.

Remarques

Le mot TRUE est également interprété comme la valeur logique TRUE.

Exemple

La formule retourne la valeur logique TRUE quand la valeur dans la colonne, 'InternetSales_USD'[SalesAmount_USD], est supérieure à 200000.

```DAX
= IF(SUM('InternetSales_USD'[SalesAmount_USD]) >200000, TRUE(), false())
```

Le tableau suivant présente les résultats obtenus quand l'exemple de formule est utilisé dans un rapport, avec 'ProductCategory'[ProductCategoryName] en étiquettes de ligne et 'DateTime'[CalendarYear] en étiquettes de colonne.

Agrandir le tableau

Étiquettes de ligne	2005	2006	2007	2008	-	Total général
Accessories	FAUX	FAUX	VRAI	VRAI	FAUX	VRAI
Bikes	VRAI	VRAI	VRAI	VRAI	FAUX	VRAI

Étiquettes de ligne	2005	2006	2007	2008	-	Total général
Clothing	FAUX	FAUX	FAUX	FAUX	FAUX	VRAI
Components	FAUX	FAUX	FAUX	FAUX	FAUX	FAUX
	FAUX	FAUX	FAUX	FAUX	FAUX	FAUX
Total général	VRAI	VRAI	VRAI	VRAI	FAUX	VRAI

Contenu connexe

FAUX
NOT
IF

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonctions mathématiques et trigonométriques

Article • 20/10/2023

Les fonctions mathématiques dans DAX (Data Analysis Expressions) ressemblent beaucoup aux fonctions mathématiques et trigonométriques dans Excel. Cette section liste les fonctions mathématiques fournies par DAX.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
ABS	Retourne la valeur absolue d'un nombre.
ACOS	Retourne l'arc cosinus, ou le cosinus inverse, d'un nombre.
ACOSH	Retourne le cosinus hyperbolique inverse d'un nombre.
ACOT	Retourne l'arccotangente ou la cotangente inverse d'un nombre.
ACOTH	Retourne la cotangente hyperbolique inverse d'un nombre.
ASIN	Retourne l'arc sinus, ou le sinus inverse, d'un nombre.
ASINH	Retourne le sinus hyperbolique inverse d'un nombre.
ATAN	Retourne l'arc tangente, ou la tangente inverse, d'un nombre.
ATANH	Retourne la tangente hyperbolique inverse d'un nombre.
CEILING	Arrondit un nombre à l'entier immédiatement supérieur ou au multiple de l'argument de précision immédiatement supérieur.
CONVERT	Convertit une expression d'un type de données à un autre.
COS	Retourne le cosinus de l'angle donné.
COSH	Retourne le cosinus hyperbolique d'un nombre.
COT	Retourne la cotangente d'un angle calculé en radians.
COTH	Retourne la cotangente hyperbolique d'un angle hyperbolique.
CURRENCY	Évalue l'argument et retourne le résultat en utilisant un type de données monétaire.

Fonction	Description
DEGREES	Convertit les radians en degrés.
DIVIDE	Effectue une division et retourne un autre résultat ou BLANK() lors d'une division par 0.
EVEN	Retourne le nombre arrondi à l'entier pair immédiatement supérieur.
EXP	Retourne e élevé à la puissance d'un nombre donné.
FACT	Retourne la factorielle d'un nombre, égale à la série 1*2*3*...* et se terminant par le nombre donné.
FLOOR	Arrondit vers le bas, en direction de zéro, un nombre à l'entier ou au multiple le plus proche de l'unité de précision.
GCD	Retourne le plus grand commun diviseur (PGCD) de deux entiers ou plus.
INT	Arrondit un nombre à l'entier immédiatement inférieur.
ISO.CEILING	Arrondit un nombre à l'entier ou au multiple de précision le plus proche.
LCM	Retourne le plus petit commun multiple des entiers.
LN	Retourne le logarithme naturel d'un nombre.
LOG	Retourne le logarithme d'un nombre dans la base que vous spécifiez.
LOG10	Retourne le logarithme de base 10 d'un nombre.
MOD	Retourne le reste d'une division d'un nombre par un diviseur. Le résultat a toujours le même signe que le diviseur.
MROUND	Retourne un nombre arrondi au multiple souhaité.
ODD	Retourne le nombre arrondi à l'entier impair supérieur le plus proche.
PI	Retourne la valeur de Pi, 3,14159265358979, avec une précision de 15 chiffres.
POWER	Retourne le résultat d'un nombre élevé à une puissance.
QUOTIENT	Effectue une division et retourne uniquement la partie entière du résultat.
RADIANS	Convertit les degrés en radians.
RAND	Retourne un nombre aléatoire supérieur ou égal à 0 et inférieur à 1, distribué équitablement.
RANDBETWEEN	Retourne un nombre aléatoire compris entre les nombres que vous spécifiez.
ROUND	Arrondit un nombre au nombre de chiffres spécifié.

Fonction	Description
ROUNDDOWN	Arrondit un nombre vers le bas, vers zéro.
ROUNDUP	Arrondit un nombre vers le haut, en s'éloignant de 0 (zéro).
SIGN	Détermine le signe d'un nombre, le résultat d'un calcul ou une valeur dans une colonne.
SIN	Retourne le sinus de l'angle donné.
SINH	Retourne le sinus hyperbolique d'un nombre.
SQRT	Retourne la racine carrée d'un nombre.
SQRTPI	Retourne la racine carrée de (number * pi).
TAN	Retourne la tangente de l'angle donné.
TANH	Retourne la tangente hyperbolique d'un nombre.
TRUNC	Tronque un nombre en entier en supprimant la partie décimale, ou fractionnaire, du nombre.

Commentaires

Cette page a-t-elle été utile ? Yes No

ABS

Article • 14/12/2023

Retourne la valeur absolue d'un nombre.

Syntaxe

DAX

```
ABS(<number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre dont vous voulez obtenir la valeur absolue.

Valeur de retour

Nombre décimal.

Remarques

La valeur absolue d'un nombre est un nombre décimal, entier ou décimal, sans son signe. Vous pouvez utiliser la fonction ABS pour faire en sorte que seuls des nombres non négatifs soient retournés d'expressions quand elles sont imbriquées dans des fonctions qui exigent un nombre positif.

Exemple

L'exemple suivant retourne la valeur absolue de la différence entre le prix catalogue et le prix du détaillant, que vous pouvez utiliser dans une nouvelle colonne calculée, **DealerMarkup**.

DAX

```
= ABS([DealerPrice]-[ListPrice])
```

Contenu connexe

Fonctions mathématiques et trigonométriques
SIGN (fonction)

Commentaires

Cette page a-t-elle été utile ? Yes No

ACOS

Article • 20/10/2023

Retourne l'arc cosinus, ou le cosinus inverse, d'un nombre. L'arc cosinus correspond à l'angle dont le cosinus est l'argument *number*. L'angle retourné est exprimé en radians, dans une plage comprise entre 0 (zéro) et pi.

Syntaxe

```
DAX

ACOS(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
Nombre	Cosinus de l'angle souhaité, compris entre -1 et 1.

Valeur de retour

Retourne l'arc cosinus, ou le cosinus inverse, d'un nombre.

Remarques

Si vous souhaitez convertir en degrés un résultat en radians, multipliez-le par 180/PI() ou utilisez la fonction DEGREES.

Exemple

Agrandir le tableau

Formule	Description	Résultat
= ACOS(-0.5)	Arc cosinus de -0,5 en radians, 2*pi/3.	2,094395102
= ACOS(-0.5)*180/PI()	Arc cosinus de -0,5 en degrés.	120

Commentaires

Cette page a-t-elle été utile ? Yes No

ACOSH

Article • 20/10/2023

Retourne le cosinus hyperbolique inverse d'un nombre. Le nombre doit être supérieur ou égal à 1. Le cosinus hyperbolique inverse est la valeur dont le cosinus hyperbolique est le nombre (*number*). ACOSH(COSH(number)) est donc égal à ce dernier.

Syntaxe

DAX

```
ACOSH(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre réel égal ou supérieur à 1.

Valeur de retour

Retourne le cosinus hyperbolique inverse d'un nombre.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Formule	Description	Résultats
= ACOSH(1)	Cosinus hyperbolique inverse de 1.	0

Formule	Description	Résultats
= ACOSH(10)	Cosinus hyperbolique inverse de 10.	2,993228

Commentaires

Cette page a-t-elle été utile ? Yes No

ACOT

Article • 20/10/2023

Retourne la valeur principale de l'arccotangente ou de la cotangente inverse d'un nombre.

Syntaxe

DAX

```
ACOT(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
Nombre	Cosinus de l’angle souhaité. Doit être un nombre réel.

Valeur renvoyée

Valeur décimale unique.

Commentaires

Cette page a-t-elle été utile ? Yes No

ACOTH

Article • 20/10/2023

Retourne la cotangente hyperbolique inverse d'un nombre.

Syntaxe

DAX

```
ACOTH(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
Nombre	La valeur absolue du nombre doit être supérieure à 1.

Valeur renvoyée

Valeur décimale unique.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Commentaires

Cette page a-t-elle été utile ? Yes No

ASIN

Article • 20/10/2023

Retourne l'arc sinus, ou sinus inverse, d'un nombre. L'arc sinus correspond à l'angle dont le sinus est le nombre (*number*). L'angle retourné est exprimé en radians, dans une plage comprise entre -pi/2 et pi/2.

Syntaxe

DAX

```
ASIN(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Sinus de l'angle souhaité, compris entre -1 et 1.

Valeur de retour

Retourne l'arc sinus, ou le sinus inverse, d'un nombre.

Remarques

Pour exprimer l'arc sinus en degrés, multipliez le résultat par 180/PI( ) ou utilisez la fonction DEGREES.

Exemple

Agrandir le tableau

Formule	Description	Résultat
= ASIN(-0.5)	Arc sinus de -0,5 en radians, -pi/6	-0,523598776
= ASIN(-0.5)*180/PI()	Arc sinus de -0,5 en degrés	-30

Formule	Description	Résultat
= DEGREES(ASIN(-0.5))	Arc sinus de -0,5 en degrés	-30

Commentaires

Cette page a-t-elle été utile ? Yes No

ASINH

Article • 20/10/2023

Retourne le sinus hyperbolique inverse d'un nombre. Le sinus hyperbolique inverse est la valeur dont le sinus hyperbolique est le nombre (*number*) : ASINH(SINH(number)) est égal à *number*.

Syntaxe

```
DAX
ASINH(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Tout nombre réel.

Valeur de retour

Retourne le sinus hyperbolique inverse d'un nombre.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Formule	Description	Résultats
= ASINH(-2.5)	Sinus hyperbolique inverse de -2,5	-1,647231146

Formule	Description	Résultats
= ASINH(10)	Sinus hyperbolique inverse de 10	2,99822295

Commentaires

Cette page a-t-elle été utile ? Yes No

ATAN

Article • 20/10/2023

Retourne l'arc tangente, ou la tangente inverse, d'un nombre. L'arc tangente correspond à l'angle dont la tangente est l'argument *number*. L'angle retourné est exprimé en radians, dans une plage comprise entre -pi/2 et pi/2.

Syntaxe

DAX

```
ATAN(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Tangente de l'angle souhaité.

Valeur de retour

Retourne la tangente hyperbolique inverse d'un nombre.

Remarques

Pour exprimer l'arc tangente en degrés, multipliez le résultat par 180/PI( ) ou utilisez la fonction DEGREES.

Exemple

Agrandir le tableau

Formule	Description	Résultat
= ATAN(1)	Arc tangente de 1 en radians, pi/4	0,785398163
= ATAN(1)*180/PI()	Arc tangente de 1 en degrés	45

Commentaires

Cette page a-t-elle été utile ? Yes No

ATANH

Article • 14/12/2023

Retourne la tangente hyperbolique inverse d'un nombre. La valeur du nombre (number) doit être comprise entre -1 et 1 (valeurs -1 et 1 exclues). La tangente hyperbolique inverse est la valeur dont la tangente hyperbolique est *number*. La formule ATANH(TANH(number)) est donc égale à *number*.

Syntaxe

DAX

```
ATANH(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre réel compris entre 1 et -1.

Valeur de retour

Retourne la tangente hyperbolique inverse d'un nombre.

Remarques

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Formule	Description	Résultats
= ATANH(0.76159416)	Tangente hyperbolique inverse de 0,76159416	1,00000001
= ATANH(-0.1)		-0,100335348

Contenu connexe

ATAN, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

CEILING

Article • 20/10/2023

Arrondit un nombre à l'entier ou au multiple de précision le plus proche.

Syntaxe

DAX

```
CEILING(<number>, <significance>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre que vous souhaitez arrondir ou référence à une colonne contenant des nombres.
significance	Multiple de précision auquel vous voulez arrondir. Par exemple, pour arrondir à l'entier le plus proche, entrez 1.

Valeur de retour

Nombre arrondi tel que spécifié.

Remarques

- Il existe deux fonctions CEILING dans DAX. Elles diffèrent par les aspects suivants :
  - La fonction CEILING émule le comportement de la fonction CEILING dans Excel.
  - La fonction ISO.CEILING suit le comportement défini par la norme ISO pour déterminer la valeur de plafond.
- Les deux fonctions retournent la même valeur pour les nombres positifs, mais des valeurs différentes pour les nombres négatifs. Quand vous utilisez un multiple de précision positif, les fonctions CEILING et ISO.CEILING arrondissent les nombres négatifs à la valeur supérieure (vers l'infini positif). Quand vous utilisez un multiple de précision négatif, la fonction CEILING arrondit les nombres négatifs à la valeur

inférieure (vers l'infini négatif) et la fonction ISO.CEILING arrondit les nombres négatifs à la valeur supérieure (vers l'infini positif).

- Le type de retour est généralement du même type que l'argument de précision, avec les exceptions suivantes :
  - Si le type d'argument de nombre est devise, le type de retour est devise.
  - Si le type d'argument de précision est booléen, le type de retour est entier.
  - Si le type d'argument de précision est non numérique, le type de retour est réel.

Exemple 1

La formule suivante retourne 4.45. Elle peut être utile si vous souhaitez éviter d'utiliser des unités inférieures dans vos tarifs. Si un produit existant est au prix de 4,42 $, vous pouvez utiliser CEILING pour arrondir les prix à l'unité supérieure la plus proche de cinq cents.

```
DAX

= CEILING(4.42,0.05)
```

Exemple 2

La formule suivante retourne des résultats similaires à l'exemple précédent, mais utilise des valeurs numériques stockées dans la colonne, **ProductPrice**.

```
DAX

= CEILING([ProductPrice],0.05)
```

Contenu connexe

Fonctions mathématiques et trigonométriques
FLOOR, fonction
ISO.CEILING, fonction
ROUNDUP, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

CONVERT

Article • 20/10/2023

Convertit une expression d'un type de données en un autre.

Syntaxe

DAX

```
CONVERT(<Expression>, <Datatype>)
```

Paramètres

Agrandir le tableau

Terme	Définition
Expression	Toute expression valide.
Datatype	Énumération contenant : INTEGER(nombre entier), DOUBLE(nombre décimal), STRING(texte), BOOLEAN(True/False), CURRENCY(nombre décimal fixe), DATETIME(date, heure, etc.).

Valeur de retour

Retourne la valeur de <Expression > convertie en <Datatype>.

Remarques

- La fonction retourne une erreur lorsqu'une valeur ne peut pas être convertie dans le type de données spécifié.
- Les colonnes calculées DAX doivent être d'un type de données unique. Dans la mesure où les fonctions MEDIAN et MEDIANX sur une colonne d'entiers retournent des types de données mixtes, de type integer ou double, l'expression de colonne calculée suivante retourne une erreur comme résultat : `MedianNumberCarsOwned = MEDIAN(DimCustomer[NumberCarsOwned])`.
- Pour éviter les types de données mixtes, modifiez l'expression pour qu'elle retourne toujours le type de données double, par exemple :

`MedianNumberCarsOwned = MEDIANX(DimCustomer, CONVERT([NumberCarsOwned], DOUBLE))`.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Requête DAX

```DAX
EVALUATE { CONVERT(DATE(1900, 1, 1), INTEGER) }
```

retourne :

Agrandir le tableau

[Value]
2

Commentaires

Cette page a-t-elle été utile ? Yes No

COS

Article • 20/10/2023

Retourne le cosinus de l'angle donné.

Syntaxe

```
DAX
COS(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Obligatoire. Angle en radians dont vous souhaitez le cosinus.

Valeur de retour

Retourne le cosinus de l'angle donné.

Remarques

Si l'angle est exprimé en degrés, multipliez l'angle par PI()/180 ou utilisez la fonction RADIANS pour convertir l'angle en radians.

Exemple

Agrandir le tableau

Formule	Description	Résultat
= COS(1,047)	Cosinus de 1 047 radians	0,5001711
= COS(60*PI()/180)	Cosinus de 60 degrés	0,5
= COS(RADIANS(60))	Cosinus de 60 degrés	0,5

Commentaires

Cette page a-t-elle été utile ? Yes No

COSH

Article • 20/10/2023

Retourne le cosinus hyperbolique d'un nombre.

Syntaxe

```
DAX

COSH(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Obligatoire. Tout nombre réel pour lequel vous souhaitez trouver le cosinus hyperbolique.

Valeur de retour

Cosinus hyperbolique d'un nombre.

Remarques

- La formule du cosinus hyperbolique est la suivante :

$$\mathrm{COSH}(z) = \frac{e^{z} + e^{-z}}{2}$$

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Formule	Description	Résultat
= COSH(4)	Cosinus hyperbolique de 4	27,308233
= COSH(EXP(1))	Cosinus hyperbolique de la base du logarithme naturel.	7,6101251

Commentaires

Cette page a-t-elle été utile ? Yes No

COT

Article • 20/10/2023

Retourne la cotangente d'un angle calculé en radians.

Syntaxe

```
DAX

COT (<number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Angle en radians dont vous souhaitez la cotangente.

Valeur de retour

Cotangente de l'angle donné.

Remarques

- La valeur absolue du nombre doit être inférieure à 2^27 et ne peut pas être égale à 0.
- Si le nombre est en dehors de ses contraintes, une erreur est retournée.
- Si le nombre n'est pas une valeur numérique, une erreur est retournée.

Exemple

La requête DAX suivante :

```
DAX

EVALUATE { COT(30) }
```

retourne :

Agrandir le tableau

[Value]
-0.156119952161659

Commentaires

Cette page a-t-elle été utile ? Yes No

COTH

Article • 20/10/2023

Retourne la cotangente hyperbolique d'un angle hyperbolique.

Syntaxe

DAX

```
COTH (<number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Angle hyperbolique en radians pour lequel vous voulez obtenir la cotangente hyperbolique.

Valeur de retour

Cotangente hyperbolique de l'angle donné.

Remarques

- La cotangente hyperbolique est une analogue de la cotangente ordinaire (circulaire).
- La valeur absolue du nombre doit être inférieure à 2^{27} et ne peut pas être égale à 0.
- Si le nombre est en dehors de ses contraintes, une erreur est retournée
- Si le nombre n'est pas une valeur numérique, une erreur est retournée.
- L'équation suivante est utilisée :

$$\mathrm{COTH}(N) = \frac{1}{\mathrm{TANH}(N)} = \frac{\mathrm{COSH(N)}}{\mathrm{SINH(N)}} = \frac{e^N + e^{-N}}{e^N - e^{-N}}$$

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La requête DAX suivante :

```
DAX

EVALUATE { COTH(2) }
```

retourne :

Agrandir le tableau

[Value]
1.03731472072755

Commentaires

Cette page a-t-elle été utile ? Yes No

DEVISE

Article • 20/10/2023

Évalue l'argument et retourne le résultat en utilisant un type de données monétaire.

Syntaxe

```
DAX

CURRENCY(<value>)
```

Paramètres

Agrandir le tableau

Terme	Définition
valeur	Toute expression DAX qui retourne une valeur scalaire unique où l'expression doit être évaluée une seule fois avant toutes les autres opérations.

Valeur renvoyée

Valeur de l'expression évaluée et retournée comme valeur de type monétaire.

Notes

- La fonction CURRENCY arrondit à la cinquième décimale significative, en valeur, pour retourner le quatrième chiffre décimal. L'arrondi est effectué si la 5e décimale significative est égale ou supérieure à 5. Par exemple, si la valeur est 3,6666666666666, la conversion en devise retourne $3.6667. Par contre, si la valeur est 3,0123456789, la conversion en devise retourne $3.0123.
- Si les données de l'expression sont de type TrueFalse, la fonction CURRENCY(<TrueFalse>) retourne $1.0000 pour les valeurs True et $0.0000 pour les valeurs False.
- Si les données de l'expression sont de type Text, la fonction CURRENCY(<Text>) tente de convertir le texte en nombre. Si la conversion réussit, le nombre est converti en valeur monétaire. Sinon, une erreur est retournée.

- Si les données de l'expression sont de type DateHeure, la fonction CURRENCY(<DateHeure>) convertit la valeur DateHeure en nombre, puis ce nombre en valeur monétaire. Les valeurs DateHeure ont une partie entière qui représente le nombre de jours entre la date spécifiée et le 1er mars 1900 et une fraction qui représente la fraction d'un jour (où 12 heures ou midi correspond à 0,5 jour). Si la valeur de l'expression n'est pas une valeur DateHeure appropriée, une erreur est retournée.

Exemple

Convertit le nombre 1234.56 en type de données monétaire.

DAX

```
= CURRENCY(1234.56)
```

Renvoie la valeur 1234,56000.

Commentaires

Cette page a-t-elle été utile ? Yes No

DEGREES

Article • 20/10/2023

Convertit les radians en degrés.

Syntaxe

```
DAX
DEGREES(angle)
```

Paramètres

Agrandir le tableau

Terme	Définition
angle	Obligatoire. Angle en radians que vous souhaitez convertir.

Exemple

Agrandir le tableau

Formule	Description	Résultat
= DEGREES(PI())	Degrés de pi radians	180

Commentaires

Cette page a-t-elle été utile ? Yes No

DIVIDE

Article • 20/10/2023

Effectue une division et retourne un résultat alternatif ou BLANK() en cas de division par 0.

Syntaxe

DAX

```
DIVIDE(<numerator>, <denominator> [,<alternateresult>])
```

Paramètres

Agrandir le tableau

Terme	Définition
numerator	Dividende ou nombre à diviser.
denominator	Diviseur ou nombre par lequel le dividende doit être divisé.
alternateresult	(Facultatif) Valeur retournée quand une division par zéro génère une erreur. Quand elle est omise, la valeur par défaut est BLANK().

Valeur de retour

- Nombre décimal.

Remarques

- Le résultat alternatif en cas de division par 0 doit être une constante.
- Pour connaître les meilleures pratiques lors de l'utilisation de dividers, consultez Fonction Divide et opérateur de division (/) dans DAX.

Exemple

L'exemple suivant retourne 2.5.

```
DAX
= DIVIDE(5,2)
```

Exemple 1

L'exemple suivant retourne BLANK.

```
DAX
= DIVIDE(5,0)
```

Exemple 2

L'exemple suivant retourne 1.

```
DAX
= DIVIDE(5,0,1)
```

Contenu connexe

QUOTIENT, fonction
Fonctions mathématiques et trigonométriques

Commentaires

Cette page a-t-elle été utile ? Yes No

EVEN

Article • 20/10/2023

Retourne le nombre arrondi à l'entier pair immédiatement supérieur. Vous pouvez utiliser cette fonction pour traiter des éléments se présentant par paires. Par exemple, une caisse d'emballage peut contenir des rangées d'un ou de deux articles. La caisse est pleine quand le nombre d'articles, arrondi au multiple de deux immédiatement supérieur, correspond à la capacité de la caisse.

Syntaxe

```DAX
EVEN(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Valeur à arrondir.

Valeur de retour

Retourne le nombre arrondi à l'entier pair immédiatement supérieur.

Remarques

- Si nombre n'est pas numérique, EVEN retourne la valeur d'erreur #VALUE! #NUM!.
- Quel que soit le signe du nombre, une valeur est arrondie vers le haut en s'éloignant de zéro. Si le nombre est un entier pair, il n'est pas arrondi.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Formule	Description	Résultat
= EVEN(1.5)	Arrondit la valeur 1.5 à l'entier pair le plus proche.	2
= EVEN(3)	Arrondit la valeur 3 à l'entier pair le plus proche.	4
= EVEN(2)	Arrondit la valeur 2 à l'entier pair le plus proche.	2
= EVEN(-1)	Arrondit la valeur -1 à l'entier pair le plus proche.	-2

Commentaires

Cette page a-t-elle été utile ? Yes No

EXP

Article • 20/10/2023

Retourne e élevé à la puissance d'un nombre donné. La constante e est égale à 2,71828182845904 et constitue la base du logarithme népérien.

Syntaxe

DAX

```
EXP(<number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Exposant appliqué à la base e. La constante e est égale à 2,71828182845904 et constitue la base du logarithme népérien.

Valeur de retour

Nombre décimal.

Exceptions

Remarques

- EXP est l'inverse de LN, qui est le logarithme népérien du nombre donné.
- Pour calculer les puissances de bases autres que e, utilisez l'opérateur d'élévation à une puissance (^). Pour plus d'informations, consultez Référence des opérateurs DAX.

Exemple

La formule suivante calcule e élevé à la puissance du nombre contenu dans la colonne `[Power]`.

```
DAX
```

```
= EXP([Power])
```

Contenu connexe

Fonctions mathématiques et trigonométriques
LN, fonction
EXP (fonction)
LOG, fonction
LOG, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

FACT

Article • 20/10/2023

Retourne la factorielle d'un nombre, égale à la série 1*2*3*...* et se terminant par le nombre donné.

Syntaxe

DAX

```
FACT(<number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre non négatif pour lequel vous souhaitez calculer la factorielle.

Valeur de retour

Nombre décimal.

Remarques

- Si le nombre n'est pas un entier, il est tronqué et une erreur est retournée. Si le résultat est trop grand, une erreur est retournée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La formule suivante retourne la factorielle pour la série d'entiers de la colonne, `[Values]`.

```
```

DAX

```
= FACT([Values])
```

Le tableau suivant présente les résultats attendus :

Agrandir le tableau

Valeurs	Résultats
0	1
1	1
2	2
3	6
4	24
5	120
170	7,257415615308E+306

Contenu connexe

Fonctions mathématiques et trigonométriques
TRUNC, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

FLOOR

Article • 14/12/2023

Arrondit vers le bas, en direction de zéro, un nombre à l'entier ou au multiple le plus proche de l'unité de précision.

Syntaxe

DAX

```
FLOOR(<number>, <significance>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Valeur numérique que vous voulez arrondir.
significance	Multiple auquel vous voulez arrondir. Les arguments **number** et **significance** doivent être tous les deux positifs ou tous les deux négatifs.

Valeur de retour

Nombre décimal.

Remarques

- Si l'un des arguments n'est pas numérique, FLOOR retourne la valeur d'erreur *#VALUE!**.
- Si le nombre et la précision ont des signes différents, FLOOR retourne la valeur d'erreur **#NUM!**.
- Quel que soit le signe du nombre, une valeur est arrondie en s'éloignant de zéro. Si le nombre est un multiple exact de précision, aucun arrondi n'est effectué.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au

niveau des lignes (RLS).

Exemple

La formule suivante prend les valeurs de la colonne [Coût total du produit] de la table, InternetSales, et arrondit au multiple de 0,1 le plus proche.

DAX

```
= FLOOR(InternetSales[Total Product Cost],.1)
```

Le tableau suivant montre les résultats attendus pour certains exemples de valeurs :

Agrandir le tableau

Valeurs	Résultat attendu
10.8423	10.8
8.0373	8
2.9733	2.9

Contenu connexe

Fonctions mathématiques et trigonométriques

Commentaires

Cette page a-t-elle été utile ? Yes No

GCD

Article • 20/10/2023

Retourne le plus grand commun diviseur (PGCD) de deux entiers ou plus. Le plus grand commun diviseur est le plus grand entier qui divise à la fois number1 et number2 sans reste.

Syntaxe

DAX

```
GCD(number1, [number2], ...)
```

Paramètres

Agrandir le tableau

Terme	Définition
number1, number2, ...	Number1 est obligatoire, les nombres suivants sont facultatifs. Valeurs de 1 à 255. Si une valeur n'est pas un entier, elle est tronquée.

Valeur de retour

Plus grand commun diviseur de deux entiers ou plus.

Remarques

- Si un argument n'est pas numérique, le PGCD retourne la valeur d'erreur #VALUE! #NUM!.
- Si un argument est inférieur à zéro, le PGCD retourne la valeur d'erreur #NUM! #NUM!.
- Un divise toute valeur de façon égale.
- Un nombre premier n'a que lui-même et un comme diviseurs.
- Si un paramètre du PGCD est >=2^53, le PGCD retourne la valeur d'erreur #NUM! #NUM!.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Formule	Description	Résultat
= GCD(5, 2)	Plus grand commun diviseur de 5 et 2.	1
= GCD(24, 36)	Plus grand commun diviseur de 24 et 36.	12
= GCD(7, 1)	Plus grand commun diviseur de 7 et 1.	1

Commentaires

Cette page a-t-elle été utile ? Yes No

INT

Article • 20/10/2023

Arrondit un nombre à l'entier immédiatement inférieur.

Syntaxe

DAX

```
INT(<number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre à arrondir à un entier inférieur

Valeur de retour

Nombre entier.

Remarques

Les fonctions TRUNC et INT sont similaires en ce sens que les deux retournent des entiers. TRUNC supprime la partie fractionnaire du nombre. INT arrondit les nombres à l'entier immédiatement inférieur, en fonction de la valeur de la partie fractionnaire du nombre. INT et TRUNC ne sont différentes que lorsque vous utilisez des nombres négatifs : `TRUNC(-4.3)` retourne -4, mais `INT(-4.3)` retourne -5, car -5 est le nombre inférieur.

Exemple

L'expression suivante arrondit la valeur à 1. Si vous utilisez la fonction ROUND, le résultat est 2.

DAX

```
= INT(1.5)
```

Contenu connexe

Fonctions mathématiques et trigonométriques
ROUND, fonction
ROUNDUP, fonction
ROUNDDOWN, fonction
MROUND, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

ISO.CEILING

Article • 20/10/2023

Arrondit un nombre à l'entier ou au multiple de précision le plus proche.

Syntaxe

DAX

```
ISO.CEILING(<number>[, <significance>])
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre que vous souhaitez arrondir ou référence à une colonne contenant des nombres.
significance	(Facultatif) Multiple de précision auquel vous voulez arrondir le nombre. Par exemple, pour arrondir à l'entier le plus proche, entrez 1. Si l'unité de précision n'est pas spécifiée, le nombre est arrondi à l'entier le plus proche.

Valeur de retour

Nombre, du même type que l'argument *number*, arrondi comme spécifié.

Remarques

Il existe deux fonctions CEILING dans DAX. Elles diffèrent par les aspects suivants :

- La fonction CEILING émule le comportement de la fonction CEILING dans Excel.
- La fonction ISO.CEILING suit le comportement défini par la norme ISO pour déterminer la valeur de plafond.

Les deux fonctions retournent la même valeur pour les nombres positifs, mais des valeurs différentes pour les nombres négatifs. Quand vous utilisez un multiple de précision positif, les fonctions CEILING et ISO.CEILING arrondissent les nombres négatifs

à la valeur supérieure (vers l'infini positif). Quand vous utilisez un multiple de précision négatif, la fonction CEILING arrondit les nombres négatifs à la valeur inférieure (vers l'infini négatif) et la fonction ISO.CEILING arrondit les nombres négatifs à la valeur supérieure (vers l'infini positif).

Le résultat est généralement du même type que la valeur de précision (argument « significance »), à quelques différences près :

- Si le premier argument est de type monétaire, le résultat est de type monétaire.
- Si l'argument facultatif n'est pas inclus, le résultat est de type entier.
- Si l'argument de précision est de type booléen, le résultat est de type entier.
- Si l'argument de précision n'est pas de type numérique, le résultat est de type réel.

Exemple : Nombres positifs

La formule suivante retourne 4.45. Elle peut être utile si vous souhaitez éviter d'utiliser des unités inférieures dans vos tarifs. Si un produit existant est au prix de 4,42 $, vous pouvez utiliser ISO.CEILING pour arrondir les prix à l'unité supérieure la plus proche de cinq cents.

DAX

```
= ISO.CEILING(4.42,0.05)
```

Exemple : nombres négatifs

La formule suivante retourne la valeur de plafond ISO -4.40.

DAX

```
= ISO.CEILING(-4.42,0.05)
```

Contenu connexe

Fonctions mathématiques et trigonométriques
FLOOR, fonction
CEILING, fonction
ROUNDUP, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

LCM

Article • 20/10/2023

Retourne le plus petit commun multiple des entiers. Le plus petit commun multiple est l'entier positif le plus petit qui est un multiple de tous les entiers en argument number1, number2, etc. Utilisez LCM pour ajouter des fractions avec des dénominateurs différents.

Syntaxe

DAX

```
LCM(number1, [number2], ...)
```

Paramètres

Agrandir le tableau

Terme	Définition
number1, number2, ...	number1 est obligatoire, les nombres suivants sont facultatifs. Valeurs 1 à 255 dont vous voulez le plus petit commun multiple. Si la valeur n'est pas un entier, elle est tronquée.

Valeur de retour

Retourne le plus petit commun multiple des entiers.

Remarques

- Si un argument n'est pas numérique, LCM retourne la valeur d'erreur #NUM!.
- Si un argument est inférieur à zéro, LCM retourne la valeur d'erreur #NUM!.
- Si LCM(a, b) >= 2 ^ 53, LCM retourne la valeur d'erreur #NUM!.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Formule	Description	Résultat
= LCM(5, 2)	Plus petit commun multiple de 5 et 2.	10
= LCM(24, 36)	Plus petit commun multiple de 24 et 36.	72

Commentaires

Cette page a-t-elle été utile ? Yes No

LN

Article • 14/12/2023

Retourne le logarithme naturel d'un nombre. Les logarithmes naturels sont basés sur la constante e (2,71828182845904).

Syntaxe

```
DAX

LN(<number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre positif dont vous voulez obtenir le logarithme naturel.

Valeur de retour

Nombre décimal.

Remarques

La fonction LN est l'inverse de la fonction EXP.

Exemple

L'exemple suivant retourne le logarithme naturel du nombre dans la colonne, `[Values]`.

```
DAX

= LN([Values])
```

Contenu connexe

Fonctions mathématiques et trigonométriques
EXP (fonction)

Commentaires

Cette page a-t-elle été utile ? Yes No

LOG

Article • 14/12/2023

Retourne le logarithme d'un nombre dans la base que vous spécifiez.

Syntaxe

```DAX
LOG(<number>,<base>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre positif dont vous voulez obtenir le logarithme.
base	Base du logarithme. En cas d'omission, la base 10 est utilisée.

Valeur de retour

Nombre décimal.

Remarques

Vous pouvez recevoir une erreur si la valeur est trop grande pour être affichée.

La fonction LOG10 est similaire, mais retourne toujours le logarithme commun, c'est-à-dire le logarithme de base 10.

Exemple

Les formules suivantes retournent le même résultat, 2.

```DAX
= LOG(100,10)
= LOG(100)
```

```
= LOG10(100)
```

Contenu connexe

Fonctions mathématiques et trigonométriques
EXP (fonction)
LOG, fonction
LOG, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

LOG10

Article • 14/12/2023

Retourne le logarithme de base 10 d'un nombre.

Syntaxe

```
DAX

LOG10(<number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre positif dont vous voulez obtenir le logarithme de base 10.

Valeur de retour

Nombre décimal.

Remarques

La fonction LOG vous permet de changer la base du logarithme, au lieu d'utiliser la base 10.

Exemple

Les formules suivantes retournent le même résultat, 2 :

```
DAX

= LOG(100,10)
= LOG(100)
= LOG10(100)
```

Contenu connexe

Fonctions mathématiques et trigonométriques
EXP (fonction)
LOG, fonction
LOG, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

MOD

Article • 20/10/2023

Retourne le reste d'une division d'un nombre par un diviseur. Le résultat a toujours le même signe que le diviseur.

Syntaxe

```
DAX

MOD(<number>, <divisor>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre dont vous voulez obtenir le reste après la division.
divisor	Nombre par lequel vous voulez diviser.

Valeur de retour

Nombre entier.

Remarques

- Si le diviseur a la valeur 0 (zéro), MOD retourne une erreur. Vous ne pouvez pas diviser par 0.
- La fonction MOD peut être exprimée dans les termes de la fonction INT : MOD(n, d) = n - d*INT(n/d)

Exemple 1

La formule suivante retourne 1, le reste de 3 divisé par 2.

```
DAX
```

```
= MOD(3,2)
```

Exemple 2

La formule suivante retourne -1, le reste de 3 divisé par 2. Notez que le signe est toujours identique au signe du diviseur.

DAX

```
= MOD(-3,-2)
```

Contenu connexe

Fonctions mathématiques et trigonométriques
ROUND, fonction
ROUNDUP, fonction
ROUNDDOWN, fonction
MROUND, fonction
INT, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

MROUND

Article • 20/10/2023

Retourne un nombre arrondi au multiple souhaité.

Syntaxe

DAX

```
MROUND(<number>, <multiple>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre à arrondir.
multiple	Multiple de l'argument de précision auquel vous voulez arrondir le nombre.

Valeur de retour

Nombre décimal.

Remarques

MROUND arrondit au nombre supérieur, à l'opposé de zéro, si le reste de la division de **number** par la valeur de **multiple** spécifiée est supérieur ou égal à la moitié de la valeur de **multiple**.

Exemple : Nombre de décimales

L'expression suivante arrondit 1,3 au multiple le plus proche de 0,2. Le résultat attendu est 1,4.

DAX

```
= MROUND(1.3,0.2)
```

Exemple : nombres négatifs

L'expression suivante arrondit -10 au multiple le plus proche de 0,3. Le résultat attendu est -9.

DAX

```
= MROUND(-10,-3)
```

Exemple : erreur

L'expression suivante retourne une erreur, car les nombres ont des signes différents.

DAX

```
= MROUND(5,-2)
```

Contenu connexe

Fonctions mathématiques et trigonométriques
ROUND, fonction
ROUNDUP, fonction
ROUNDDOWN, fonction
MROUND, fonction
INT, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

ODD

Article • 20/10/2023

Retourne le nombre arrondi à l'entier impair supérieur le plus proche.

Syntaxe

```
DAX

ODD(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Obligatoire. Valeur à arrondir.

Valeur de retour

Retourne le nombre arrondi à l'entier impair immédiatement supérieur.

Remarques

- Si nombre n'est pas numérique, ODD retourne la valeur d'erreur #NUM!.
- Quel que soit le signe du nombre, une valeur est arrondie vers le haut en s'éloignant de zéro. Si le nombre est un entier impair, aucun arrondi n'est effectué.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Formule	Description	Résultat
= ODD(1,5)	Arrondit 1,5 à l'entier impair supérieur le plus proche.	3
= ODD(3)	Arrondit 3 à l'entier impair supérieur le plus proche.	3
= ODD(2)	Arrondit 2 à l'entier impair le plus proche.	3
=ODD(-1)	Arrondit -1 à l'entier impair supérieur le plus proche.	-1
= ODD(-2)	Arrondit -2 (en s'éloignant de 0) à l'entier impair supérieur le plus proche.	-3

Commentaires

Cette page a-t-elle été utile ? Yes No

PI

Article • 14/12/2023

Retourne la valeur de Pi, 3,14159265358979, avec une précision de 15 chiffres.

Syntaxe

```DAX
PI()
```

Valeur de retour

Nombre décimal avec la valeur de Pi, 3,14159265358979, avec une précision de 15 chiffres.

Remarques

Pi est une constante mathématique. Dans DAX, Pi est représenté sous la forme d'un nombre réel d'une précision de 15 chiffres, comme dans Excel.

Exemple

La formule suivante calcule l'aire d'un cercle à partir du rayon indiqué dans la colonne, `[Radius]`.

```DAX
= PI()*([Radius]*2)
```

Contenu connexe

Fonctions mathématiques et trigonométriques

Commentaires

Cette page a-t-elle été utile ? Yes No

POWER

Article • 14/12/2023

Retourne le résultat d'un nombre élevé à une puissance.

Syntaxe

```
DAX

POWER(<number>, <power>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre de base, qui peut être n'importe quel nombre réel.
puissance	Exposant auquel le nombre de base est élevé.

Valeur de retour

Nombre décimal.

Exemple

L'exemple suivant retourne 25.

```
DAX

= POWER(5,2)
```

Contenu connexe

Fonctions mathématiques et trigonométriques

Commentaires

Cette page a-t-elle été utile ? Yes No

QUOTIENT

Article • 20/10/2023

Effectue une division et retourne uniquement la partie entière du résultat. Utilisez cette fonction quand vous souhaitez ignorer le reste de la division.

Syntaxe

DAX

```
QUOTIENT(<numerator>, <denominator>)
```

Paramètres

Agrandir le tableau

Terme	Définition
numerator	Dividende ou nombre à diviser.
denominator	Diviseur ou nombre par lequel le dividende doit être divisé.

Valeur de retour

Nombre entier.

Remarques

- Si l'un des arguments est non numérique, QUOTIENT retourne la valeur d'erreur **#VALUE!**.
- Vous pouvez utiliser une référence de colonne au lieu d'une valeur littérale pour l'un ou l'autre des arguments. Toutefois, si la colonne que vous référencez contient un 0 (zéro), une erreur est retournée pour l'ensemble de la colonne de valeurs.

Exemple

Les formules suivantes retournent le même résultat, 2.

DAX

```
= QUOTIENT(5,2)
```

DAX

```
= QUOTIENT(10/2,2)
```

Contenu connexe

Fonctions mathématiques et trigonométriques

Commentaires

Cette page a-t-elle été utile ? Yes No

RADIANS

Article • 20/10/2023

Convertit les degrés en radians.

Syntaxe

DAX

```
RADIANS(angle)
```

Paramètres

Agrandir le tableau

Terme	Définition
angle	Obligatoire. Angle en degrés que vous souhaitez convertir.

Exemple

Agrandir le tableau

Formule	Description	Résultat
= RADIANS(270)	270 degrés en radians (4,712389 ou 3π/2 radians)	4,712389

Commentaires

Cette page a-t-elle été utile ? Yes No

RAND

Article • 14/12/2023

Retourne un nombre aléatoire supérieur ou égal à 0 et inférieur à 1, distribué équitablement. Le nombre retourné change chaque fois que la cellule qui contient cette fonction est recalculée.

Syntaxe

```DAX
RAND()
```

Valeur renvoyée

Nombre décimal.

Remarques

- Le recalcul dépend de différents facteurs, notamment du mode de recalcul du modèle (**Manuel** ou **Automatique**) et de l'actualisation (ou non) des données.
- Comme d'autres fonctions volatiles qui n'ont pas de valeurs fixes, la fonction RAND n'est pas toujours recalculée. Par exemple, en général, l'exécution d'une requête ou un filtrage n'entraîne pas la réévaluation de ces fonctions. Toutefois, les résultats de ces fonctions sont recalculés quand la colonne entière est recalculée. Ceci se produit notamment lors d'une actualisation à partir d'une source de données externe ou de la modification manuelle de données entraînant la réévaluation des formules qui contiennent ces fonctions.
- La fonction RAND est toujours recalculée si elle est utilisée dans la définition d'une mesure.
- La fonction RAND ne peut pas retourner le résultat zéro, pour éviter des erreurs telles qu'une division par zéro.

Exemples

Pour générer un nombre réel aléatoire compris entre deux autres nombres, utilisez :

DAX

```
= RAND()*(b-a)+a
```

Pour générer un nombre aléatoire supérieur à 0 et inférieur à 1 :

DAX

```
= RAND()
```

Pour générer un nombre aléatoire supérieur à 0 et inférieur à 100

DAX

```
= RAND()*100
```

Pour générer un nombre entier aléatoire supérieur à 0 et inférieur à 100

DAX

```
INT(RAND()*100)
```

Contenu connexe

Fonctions mathématiques et trigonométriques
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

RANDBETWEEN

Article • 14/12/2023

Retourne un nombre aléatoire compris entre les nombres que vous spécifiez.

Syntaxe

```
DAX

RANDBETWEEN(<bottom>,<top>)
```

Paramètres

Agrandir le tableau

Terme	Définition
Bas	Entier le plus petit retourné par la fonction.
Haut	Entier le plus grand retourné par la fonction.

Valeur de retour

Nombre entier.

Remarques

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La formule suivante retourne un nombre aléatoire compris entre 1 et 10.

```
DAX

= RANDBETWEEN(1,10)
```

Contenu connexe

Fonctions mathématiques et trigonométriques
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

ROUND

Article • 14/12/2023

Arrondit un nombre au nombre de chiffres spécifié.

Syntaxe

DAX

```
ROUND(<number>, <num_digits>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre à arrondir.
num_digits	Nombre de chiffres auquel arrondir le nombre. Une valeur négative arrondit les chiffres à gauche de la décimale. La valeur zéro arrondit le nombre à l'entier le plus proche.

Valeur de retour

Nombre décimal.

Remarques

- Si **num_digits** est supérieur à 0 (zéro), le nombre est arrondi selon le nombre spécifié de décimales.
- Si **num_digits** est égal à 0, le nombre est arrondi à l'entier le plus proche.
- Si **num_digits** est inférieur à 0, le nombre est arrondi à gauche de la décimale.
- Fonctions connexes
  - Pour arrondir systématiquement vers le haut (en s'éloignant de zéro), utilisez la fonction ROUNDUP.

- Pour arrondir systématiquement vers le bas (vers zéro), utilisez la fonction ROUNDDOWN.
- Pour arrondir un nombre à un multiple spécifique (par exemple, au multiple de 0,5 le plus proche), utilisez la fonction MROUND.
- Utilisez les fonctions TRUNC et INT pour obtenir la partie entière du nombre.

Exemple 1

La formule suivante arrondit la valeur 2.15 à une décimale vers le haut. Le résultat attendu est 2.2.

DAX

```
= ROUND(2.15,1)
```

Exemple 2

La formule suivante arrondit la valeur 21.5 à gauche de la décimale. Le résultat attendu est 20.

DAX

```
= ROUND(21.5,-1)
```

Contenu connexe

Fonctions mathématiques et trigonométriques
ROUND
ROUNDDOWN
MROUND
INT
TRUNC

Commentaires

Cette page a-t-elle été utile ? Yes No

ROUNDDOWN

Article • 20/10/2023

Arrondit un nombre vers le bas, vers zéro.

Syntaxe

DAX

```
ROUNDDOWN(<number>, <num_digits>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre réel à arrondir à l'entier inférieur.
num_digits	Nombre de chiffres auquel arrondir le nombre. Une valeur négative arrondit à gauche de la virgule décimale ; la valeur zéro arrondit à l'entier le plus proche.

Valeur de retour

Nombre décimal.

Remarques

- Si **num_digits** est supérieur à 0 (zéro), la valeur de **number** est arrondie à l'entier inférieur selon le nombre spécifié de décimales.
- Si **num_digits** est égal à 0, la valeur de **number** est arrondie à l'entier immédiatement inférieur.
- Si **num_digits** est inférieur à 0, la valeur de **number** est arrondie à l'entier inférieur à gauche de la virgule décimale.
- La fonction ROUNDDOWN affiche le même comportement que ROUND, sauf qu'elle arrondit toujours un nombre à l'entier inférieur. La fonction INT arrondit

également à l'entier inférieur, mais le résultat est toujours un entier, tandis qu'avec ROUNDDOWN, vous pouvez contrôler la précision du résultat.

Exemple 1

L'exemple suivant arrondit 3,14159 à trois décimales. Le résultat attendu est 3,141.

```
DAX

= ROUNDDOWN(3.14159,3)
```

Exemple 2

L'exemple suivant arrondit la valeur de 31415,92654 à 2 décimales à gauche de la virgule décimale. Le résultat attendu est 31400.

```
DAX

= ROUNDDOWN(31415.92654, -2)
```

Contenu connexe

Fonctions mathématiques et trigonométriques
ROUND
ROUNDUP
ROUNDDOWN
MROUND
INT

Commentaires

Cette page a-t-elle été utile ? Yes No

ROUNDUP

Article • 14/12/2023

Arrondit un nombre vers le haut en s'éloignant de 0 (zéro).

Syntaxe

DAX

```
ROUNDUP(<number>, <num_digits>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre réel que vous voulez arrondir vers le haut.
num_digits	Nombre de chiffres souhaité pour le nombre arrondi. Si la valeur de **num_digits** est négative, l'arrondi s'effectue à gauche de la décimale. Si **num_digits** est égal à zéro ou est omis, le nombre (**number**) est arrondi à l'entier le plus proche.

Valeur de retour

Nombre décimal.

Remarques

- Si **num_digits** est supérieur à 0 (zéro), le nombre est arrondi vers le haut selon le nombre spécifié de décimales.
- Si **num_digits** est égal à 0, le nombre est arrondi à l'entier supérieur le plus proche.
- Si **num_digits** est inférieure à 0, le nombre est arrondi vers le haut à gauche de la décimale.
- La fonction ROUNDUP affiche le même comportement que ROUND, sauf qu'elle arrondit toujours le nombre vers le haut.

Exemple

La formule suivante arrondit Pi à quatre décimales. Le résultat attendu est 3,1416.

```
DAX

= ROUNDUP(PI(),4)
```

Exemple : décimaux en deuxième argument

La formule suivante arrondit 1,3 au multiple le plus proche de 0,2. Le résultat attendu est 2.

```
DAX

= ROUNDUP(1.3,0.2)
```

Exemple : nombre négatif en deuxième argument

La formule suivante arrondit la valeur de la colonne **FreightCost** ; les résultats attendus sont présentés dans le tableau suivant :

```
DAX

= ROUNDUP([Values],-1)
```

Quand **num_digits** a une valeur inférieure à zéro, le nombre de chiffres à gauche de la décimal augmente de la valeur que vous spécifiez.

Agrandir le tableau

FreightCost	Résultat attendu
13.25	20
2.45	10
25.56	30
1.34	10

FreightCost	Résultat attendu
345.01	350

Contenu connexe

Fonctions mathématiques et trigonométriques
ROUND
ROUNDDOWN
MROUND
INT

Commentaires

Cette page a-t-elle été utile ? Yes No

SIGN

Article • 14/12/2023

Détermine le signe d'un nombre, le résultat d'un calcul ou une valeur dans une colonne. La fonction retourne 1 si le nombre est positif, 0 (zéro) si le nombre est zéro ou -1 si le nombre est négatif.

Syntaxe

DAX

```
SIGN(<number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	N'importe quel nombre réel, une colonne qui contient des nombres ou une expression qui prend la valeur d'un nombre.

Valeur de retour

Nombre entier. Les valeurs de retour possibles sont 1, 0 et -1.

Agrandir le tableau

Valeur de retour	Description
1	Le nombre est positif
0	Le nombre est zéro
-1	Le nombre est négatif

Exemple

La formule suivante retourne le signe du résultat de l'expression qui calcule le prix de vente moins le coût.

DAX

```
= SIGN( ([Sale Price] - [Cost]) )
```

Contenu connexe

Fonctions mathématiques et trigonométriques

Commentaires

Cette page a-t-elle été utile ? Yes No

SIN

Article • 20/10/2023

Retourne le sinus de l'angle donné.

Syntaxe

```
DAX
SIN(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Obligatoire. Angle en radians dont vous souhaitez le sinus.

Valeur de retour

Retourne le sinus de l'angle donné.

Remarques

Si un argument est exprimé en degrés, multipliez-le par PI()/180 ou utilisez la fonction RADIANS pour le convertir en radians.

Exemple

Agrandir le tableau

Formule	Description	Résultat
= SIN(PI())	Sinus de pi en radians (0 approximativement).	0.0
= SIN(PI()/2)	Sinus de pi/2 en radians.	1.0
= SIN(30*PI()/180)	Sinus de 30 degrés.	0,5

Formule	Description	Résultat
= SIN(RADIANS(30))	Sinus de 30 degrés.	0,5

Commentaires

Cette page a-t-elle été utile ? Yes No

SINH

Article • 20/10/2023

Retourne le sinus hyperbolique d'un nombre.

Syntaxe

```
DAX

SINH(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Obligatoire. Tout nombre réel.

Valeur de retour

Retourne le sinus hyperbolique d'un nombre.

Remarques

- La formule du sinus hyperbolique est la suivante :

$$\text{SINH}(z) = \frac{e^z - e^{-z}}{2}$$

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Probabilité d'obtention d'un résultat inférieur à 1.03 secondes.

```
DAX
```

```
= 2.868*SINH(0.0342\*1.03)
```

Retourne 0,1010491

Commentaires

Cette page a-t-elle été utile ? Yes No

SQRT

Article • 14/12/2023

Retourne la racine carrée d'un nombre.

Syntaxe

```DAX
SQRT(<number>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre dont vous voulez obtenir la racine carrée, une colonne qui contient des nombres ou une expression qui prend la valeur d'un nombre.

Valeur de retour

Nombre décimal.

Remarques

Si le nombre est négatif, la fonction SQRT retourne une erreur.

Exemple

La formule suivante,

```DAX
= SQRT(25)
```

Contenu connexe

Commentaires

Cette page a-t-elle été utile ? Yes No

SQRTPI

Article • 20/10/2023

Retourne la racine carrée de (nombre * pi).

Syntaxe

```DAX
SQRTPI(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Obligatoire. Nombre par lequel est multiplié pi.

Valeur de retour

Retourne la racine carrée de (number * pi).

Exemple

Agrandir le tableau

Formule	Description	Résultats
= SQRTPI(1)	Racine carrée de pi.	1,772454
= SQRTPI(2)	Racine carrée de 2 * pi.	2,506628

Commentaires

Cette page a-t-elle été utile ? Yes No

TAN

Article • 20/10/2023

Retourne la tangente de l'angle donné.

Syntaxe

```
DAX

TAN(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Obligatoire. Angle en radians dont vous voulez la tangente.

Valeur de retour

Retourne la tangente de l'angle donné.

Remarques

Si l'argument est exprimé en degrés, multipliez-le par PI()/180 ou utilisez la fonction RADIANS pour le convertir en radians.

Exemple

Agrandir le tableau

Formule	Description	Résultat
= TAN(0,785)	Tangente de 0,785 radians (0,99920)	0,99920
= TAN(45*PI()/180)	Tangente de 45 degrés (1)	1
= TAN(RADIANS(45))	Tangente de 45 degrés (1)	1

Commentaires

Cette page a-t-elle été utile ? Yes No

TANH

Article • 20/10/2023

Retourne la tangente hyperbolique d'un nombre.

Syntaxe

```
DAX

TANH(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Obligatoire. Tout nombre réel.

Valeur de retour

Retourne la tangente hyperbolique d'un nombre.

Remarques

- La formule de la tangente hyperbolique est la suivante :

$$\text{TANH}(z) = \frac{\text{SINH}(z)}{\text{COSH}(z)}$$

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Formule	Description	Résultat
= TANH(-2)	Tangente hyperbolique de -2 (-0.96403)	-0.964028
= TANH(0)	Tangente hyperbolique de 0 (0)	0
= TANH(0.5)	Tangente hyperbolique de 0.5 (0.462117)	0.462117

Commentaires

Cette page a-t-elle été utile ? Yes No

TRUNC

Article • 14/12/2023

Tronque un nombre en entier en supprimant la partie décimale, ou fractionnaire, du nombre.

Syntaxe

DAX

```
TRUNC(<number>,<num_digits>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre à tronquer.
num_digits	Nombre spécifiant la précision de la troncation. Si l'argument est omis, 0 (zéro).

Valeur de retour

Nombre entier.

Remarques

Les fonctions TRUNC et INT sont similaires en ce sens que les deux retournent des entiers. TRUNC supprime la partie fractionnaire du nombre. INT arrondit les nombres à l'entier immédiatement inférieur en fonction de la valeur de la partie fractionnaire du nombre. Les fonctions INT et TRUNC ne sont différentes que si vous utilisez des nombres négatifs : `TRUNC(-4.3)` retourne -4, mais `INT(-4.3)` retourne -5, car -5 est le nombre inférieur.

Exemple 1

La formule suivante retourne la valeur 3, soit la partie entière de pi.

```
DAX
= TRUNC(PI())
```

Exemple 2

La formule suivante retourne la valeur -8, soit la partie entière de -8.9.

```
DAX
= TRUNC(-8.9)
```

Contenu connexe

Fonctions mathématiques et trigonométriques
ROUND
ROUNDUP
ROUNDDOWN
MROUND
INT

Commentaires

Cette page a-t-elle été utile ? Yes No

Autres fonctions

Article • 20/10/2023

Ces fonctions effectuent des actions uniques qui n'entrent dans aucune des catégories.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
BLANK	Retourne un espace.
ERROR	Génère une erreur avec un message d'erreur.
EVALUATEANDLOG	Retourne la valeur du premier argument et la journalise dans un événement de profileur de journal d'évaluation DAX.
TOCSV	Retourne une table sous forme de chaîne au format CSV.
TOJSON	Retourne une table sous forme de chaîne au format JSON.

Commentaires

Cette page a-t-elle été utile ? Yes No

BLANK

Article • 20/10/2023

Retourne un espace.

Syntaxe

```DAX
BLANK()
```

Valeur de retour

Espace.

Remarques

- Les espaces ne sont pas équivalents aux valeurs Null. DAX utilise des espaces pour les valeurs Null de base de données et pour les cellules vides dans Excel.
- Certaines fonctions DAX traitent des cellules vides de manière légèrement différente de Microsoft Excel. Les espaces et les chaînes vides ("") ne sont pas toujours équivalents, mais certaines opérations peuvent les traiter ainsi.

Exemple

L'exemple suivant montre comment vous pouvez utiliser des espaces dans des formules. La formule calcule le ratio des ventes entre les canaux Revendeurs et Internet. Toutefois, avant d'essayer de calculer le ratio, il convient de rechercher les valeurs zéro pour le dénominateur. Si le dénominateur a la valeur zéro, une valeur vide doit être retournée ; dans le cas contraire, le ratio est calculé.

```DAX
= IF( SUM(InternetSales_USD[SalesAmount_USD])= 0   , BLANK()   ,
SUM(ResellerSales_USD[SalesAmount_USD])/SUM(InternetSales_USD[SalesAmount_US
D])   )
```

La table indique les résultats attendus lorsque cette formule est utilisée pour créer une visualisation de table.

Agrandir le tableau

Étiquettes de ligne	Accessories	Bikes	Clothing	Total général
2005		2,65		2,89
2006		3.33		4,03
2007	1,04	2,92	6,63	3,51
2008	0,41	1,53	2,00	1,71
Total général	0,83	2,51	5,45	2,94

Dans la source de données d'origine, la colonne évaluée par la fonction BLANK peut comporter du texte, des chaînes vides ou des valeurs Null. Si la source de données d'origine était une base de données SQL Server, les valeurs Null et les chaînes vides sont différents types de données. Toutefois, pour cette opération, un cast de type implicite est effectué et DAX les traite comme étant identiques.

Contenu connexe

Fonctions de texte
ISBLANK, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

ERROR

Article • 20/10/2023

Génère une erreur avec un message d'erreur.

Syntaxe

DAX

```
ERROR(<text>)
```

Paramètres

Agrandir le tableau

Terme	Définition
texte	Chaîne de texte contenant un message d'erreur.

Valeur de retour

Aucune

Remarques

- La fonction ERROR peut être placée dans une expression DAX partout où une valeur scalaire est attendue.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple 1

La requête DAX suivante :

DAX

```
DEFINE
MEASURE DimProduct[Measure] =
        IF(
            SELECTEDVALUE(DimProduct[Color]) = "Red",
            ERROR("red color encountered"),
            SELECTEDVALUE(DimProduct[Color])
        )
EVALUATE SUMMARIZECOLUMNS(DimProduct[Color], "Measure", [Measure])
ORDER BY [Color]
```

échoue et génère un message d'erreur contenant « red color encountered ».

Exemple 2

La requête DAX suivante :

DAX

```
DEFINE
MEASURE DimProduct[Measure] =
        IF(
            SELECTEDVALUE(DimProduct[Color]) = "Magenta",
            ERROR("magenta color encountered"),
            SELECTEDVALUE(DimProduct[Color])
        )
EVALUATE SUMMARIZECOLUMNS(DimProduct[Color], "Measure", [Measure])
ORDER BY [Color]
```

retourne la table suivante :

Agrandir le tableau

DimProduct[Color]	[Measure]
Noir	Noir
Bleu	Blue
Grey	Grey
Multiple	Multi
N/D	N/A
Rouge	Red
Argent	Silver

DimProduct[Color]	[Measure]
Silver\Black	Silver\Black
White	White
Jaune	Yellow

Comme Magenta ne figure pas parmi les couleurs de produit, la fonction ERROR n'est pas exécutée.

Commentaires

Cette page a-t-elle été utile ? Yes No

EVALUATEANDLOG

Article • 20/10/2023

Retourne la valeur du premier argument et la journalise dans un événement de profileur de journal d'évaluation DAX. Cette fonction est entièrement fonctionnelle dans Power BI Desktop uniquement. Elle agit comme une fonction directe simple dans d'autres environnements.

Syntaxe

```DAX
EVALUATEANDLOG(<Value>, [Label], [MaxRows])
```

Paramètres

Agrandir le tableau

Terme	Définition
Valeur	Toute expression scalaire ou expression de table à évaluer et journaliser.
Étiquette	(Facultatif) Chaîne constante incluse dans le texte json et la colonne Label de l'événement de journal d'évaluation DAX qui peut être utilisée pour identifier facilement une instance de l'appel de fonction.
MaxRows	(Facultatif) Nombre maximal de lignes dans le texte json de l'événement de journal d'évaluation DAX quand le premier argument est une expression de table. La valeur par défaut est 10.

Valeur renvoyée

Valeur du premier argument.

La structure JSON journalisée dans un événement de profileur de journal d'évaluation DAX comprend les éléments suivants :

- « expression » est la version textuelle du premier argument.
- « label » est le paramètre Label quand il est spécifié dans l'expression.
- « inputs » est une liste de colonnes dans le contexte d'évaluation qui affecte les valeurs du premier argument.

- « outputs » est une liste d'une seule colonne [Value] quand le premier argument est une expression scalaire et une liste de colonnes de sortie quand le premier argument est une expression de table.
- « data » est une liste de valeurs d'entrée et de valeurs de sortie quand le premier argument est une expression scalaire, et une liste de valeurs d'entrée et de lignes de sortie correspondantes quand le premier argument est une expression de table.
- « rowCount » est le nombre de lignes quand le premier argument est une expression de table. Même si le nombre de lignes dans la sortie json est tronqué par le paramètre MaxRows, rowCount est le nombre réel de lignes sans troncation.

Remarques

- Les événements de trace peuvent être capturés avec SQL Server Profiler et de l'outil open source DAX Debug Output.

- Cette fonction peut être utilisée avec presque n'importe quelle sous-expression dans une expression DAX, sans que cela affecte la validité de l'ensemble de l'expression.

- Quand le premier argument est évalué plusieurs fois dans une requête unique, la fonction génère un seul événement de journal d'évaluation DAX qui contient à la fois les valeurs d'entrée et les valeurs de sortie correspondantes.

- Quand le paramètre label est spécifié, sa valeur est retournée dans la sortie json et dans la colonne Label de l'événement de journal d'évaluation DAX.

- Si le premier argument est une expression de table, seules les lignes MaxRows supérieures sont affichées dans l'événement de journal d'évaluation DAX.

- Dans certains cas, cette fonction n'est pas exécutée en raison d'optimisations.

- Si l'événement de journal d'évaluation DAX comprend plus d'un million de caractères, il est tronqué pour conserver la structure json correcte.

Exemple 1

La requête DAX suivante :

```
DAX

evaluate
SUMMARIZE(
    EVALUATEANDLOG(FILTER(Sales, [ProductKey] = 528)),
    Sales[SalesTerritoryKey],
```

```
    "sum",
    sum(Sales[Sales Amount])
)
```

Retourne l'événement de journal d'évaluation DAX suivant :

JSON


```
{
    "expression": "FILTER(Sales, [ProductKey] = 528)",
    "inputs": [],
    "outputs": ["'Sales'[SalesOrderLineKey]", "'Sales'[ResellerKey]",
"'Sales'[CustomerKey]", "'Sales'[ProductKey]", "'Sales'[OrderDateKey]",
"'Sales'[DueDateKey]", "'Sales'[ShipDateKey]", "'Sales'[SalesTerritoryKey]",
"'Sales'[Order Quantity]", "'Sales'[Unit Price]", "'Sales'[Extended
Amount]", "'Sales'[Product Standard Cost]", "'Sales'[Total Product Cost]",
"'Sales'[Sales Amount]", "'Sales'[Unit Price Discount Pct]"],
    "data": [
        {
            "input": [],
            "rowCount": 3095,
            "output": [
                [52174001, -1, 23785, 528, 20190707, 20190717, 20190714, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [52173001, -1, 26278, 528, 20190707, 20190717, 20190714, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [52082001, -1, 23831, 528, 20190705, 20190715, 20190712, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [52054002, -1, 11207, 528, 20190704, 20190714, 20190711, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [52036001, -1, 25337, 528, 20190704, 20190714, 20190711, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [51939002, -1, 23670, 528, 20190702, 20190712, 20190709, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [51911002, -1, 11746, 528, 20190701, 20190711, 20190708, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [51379003, -1, 13745, 528, 20190612, 20190622, 20190619, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [51264002, -1, 11282, 528, 20190605, 20190615, 20190612, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0],
                [51184003, -1, 11263, 528, 20190531, 20190610, 20190607, 1,
1, 4.99, 4.99, 1.8663, 1.8663, 4.99, 0.0]
            ]
        }
    ]
}
```


Exemple 2

La requête DAX suivante avec un argument scalaire et différents attributs :

DAX

```
evaluate
SELECTCOLUMNS(
    TOPN(5, Customer),
    [Customer],
    "Customer",
    EVALUATEANDLOG([Customer] & ", " & [Country-Region], "customerLog")
)
```

Retourne l'événement de journal d'évaluation DAX suivant :

JSON

```
{
    "expression": "[Customer] & \", \" & [Country-Region]",
    "label": "customerLog",
    "inputs": ["'Customer'[Customer]", "'Customer'[Country-Region]"],
    "data": [
        {
            "input": ["Russell Xie", "United States"],
            "output": "Russell Xie, United States"
        },
        {
            "input": ["Savannah Baker", "United States"],
            "output": "Savannah Baker, United States"
        },
        {
            "input": ["Maurice Tang", "United States"],
            "output": "Maurice Tang, United States"
        },
        {
            "input": ["Emily Wood", "United States"],
            "output": "Emily Wood, United States"
        },
        {
            "input": ["Meghan Hernandez", "United States"],
            "output": "Meghan Hernandez, United States"
        }
    ]
}
```

Contenu connexe

TOCSV
TOJSON

Commentaires

Cette page a-t-elle été utile ? Yes No

TOCSV

Article • 14/12/2023

Retourne une table sous forme de chaîne au format CSV.

Syntaxe

DAX

```
TOCSV(<Table>, [MaxRows], [Delimiter], [IncludeHeaders])
```

Paramètres

Agrandir le tableau

Terme	Définition
Table de charge de travail	Table à convertir en CSV.
MaxRows	(Facultatif) Nombre maximal de lignes à convertir. La valeur par défaut est de 10 lignes.
Délimiteur	(Facultatif) Délimiteur de colonne. La valeur par défaut est la virgule (« , »).
IncludeHeaders	(Facultatif) Spécifie un en-tête avec le nom de colonne comme première ligne. La valeur par défaut est True.

Valeur renvoyée

Chaîne avec une représentation CSV de la table.

Exemple

La requête DAX suivante :

DAX

```
EVALUATE
{TOCSV(DimSalesTerritory)}
```

Retourne :

```
'DimSalesTerritory'[SalesTerritoryKey],'DimSalesTerritory'[SalesTerritoryAlt
ernateKey],'DimSalesTerritory'[SalesTerritoryRegion],'DimSalesTerritory'[Sal
esTerritoryCountry],'DimSalesTerritory'[SalesTerritoryGroup]
1,1,Northwest,United States,North America
2,2,Northeast,United States,North America
3,3,Central,United States,North America
4,4,Southwest,United States,North America
5,5,Southeast,United States,North America
6,6,Canada,Canada,North America
7,7,France,France,Europe
8,8,Germany,Germany,Europe
9,9,Australia,Australia,Pacific
10,10,United Kingdom,United Kingdom,Europe
```

Contenu connexe

TOJSON
EVALUATEANDLOG

Commentaires

Cette page a-t-elle été utile ? Yes No

TOJSON

Article • 14/12/2023

Retourne une table sous forme de chaîne au format JSON.

Syntaxe

DAX

```
TOJSON(<Table>, [MaxRows])
```

Paramètres

Agrandir le tableau

Terme	Définition
Table de charge de travail	Table à convertir en JSON.
MaxRows	(Facultatif) Nombre maximal de lignes à convertir. La valeur par défaut est de 10 lignes.

Valeur renvoyée

Chaîne avec une représentation JSON de la table. La représentation contient les noms de colonnes en tant que « header », le nombre de lignes en tant que « rowCount » et les valeurs en tant que « data ».

Exemple

La requête DAX suivante :

DAX

```
EVALUATE
{TOJSON(DimSalesTerritory)}
```

Retourne :

JSON


```
{
"header": ["'DimSalesTerritory'[SalesTerritoryKey]",
"'DimSalesTerritory'[SalesTerritoryAlternateKey]",
"'DimSalesTerritory'[SalesTerritoryRegion]",
"'DimSalesTerritory'[SalesTerritoryCountry]",
"'DimSalesTerritory'[SalesTerritoryGroup]"],<br>
"rowCount": 11,
"data": [
[1, 1, "Northwest", "United States", "North America"],
[2, 2, "Northeast", "United States", "North America"],
[3, 3, "Central", "United States", "North America"],
[4, 4, "Southwest", "United States", "North America"],
[5, 5, "Southeast", "United States", "North America"],
[6, 6, "Canada", "Canada", "North America"],
[7, 7, "France", "France", "Europe"],
[8, 8, "Germany", "Germany", "Europe"],
[9, 9, "Australia", "Australia", "Pacific"],
[10, 10, "United Kingdom", "United Kingdom", "Europe"]
]
}
```


Contenu connexe

TOCSV
EVALUATEANDLOG

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonctions parents et enfants

Article • 20/10/2023

Ces fonctions gèrent les données présentées sous forme de hiérarchies parent/enfant. Pour en savoir plus, consultez Présentation des fonctions pour les hiérarchies parent-enfant dans DAX.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
PATH	Retourne une chaîne de texte délimitée avec les identificateurs de tous les parents de l'identificateur actuel.
PATHCONTAINS	Retourne la valeur TRUE si l'*élément* spécifié existe dans le *chemin* spécifié.
PATHITEM	Retourne l'élément à la *position* spécifiée à partir d'une chaîne résultant de l'évaluation d'une fonction PATH.
PATHITEMREVERSE	Retourne l'élément à la *position* spécifiée à partir d'une chaîne résultant de l'évaluation d'une fonction PATH.
PATHLENGTH	Retourne le nombre de parents à l'élément spécifié dans un résultat PATH donné, y compris self.

Commentaires

Cette page a-t-elle été utile ? Yes No

Présentation des fonctions pour les hiérarchies parent-enfant dans DAX

Article • 20/10/2023

DAX fournit cinq fonctions pour aider les utilisateurs à gérer les données qui se présentent sous la forme d'une hiérarchie parent-enfant dans leurs modèles. Grâce à ces fonctions, un utilisateur peut obtenir l'intégralité du lignage des parents d'une ligne, le nombre de niveaux du lignage jusqu'au parent supérieur, l'identité du parent situé n niveaux au-dessus de la ligne actuelle et l'identité du nième descendant à partir du sommet de la hiérarchie de la ligne actuelle. Il est aussi possible de savoir si un parent donné est un parent dans la hiérarchie de la ligne actuelle.

Fonctions parent-enfant dans DAX

La table suivante contient une hiérarchie parent-enfant sur les colonnes **EmployeeKey** et **ParentEmployeeKey**. Elle est utilisée dans tous les exemples de fonctions.

Agrandir le tableau

EmployeeKey	ParentEmployeeKey
112	
14	112
3	14
11	3
13	3
162	3
117	162
221	162
81	162

Dans la table ci-dessus, vous pouvez voir qu'aucun parent n'est défini pour l'employé 112, que l'employé 14 a pour responsable l'employé 112 (ParentEmployeeKey), que l'employé 3 a pour responsable l'employé 14 et que les employés 11, 13 et 162 ont pour responsable l'employé 3. L'exemple ci-dessus nous

permet de comprendre que l'employé 112 n'a aucun supérieur et qu'il est en charge de tous les employés indiqués ici. Par ailleurs, l'employé 3 relève de l'employé 14 et les employés 11, 13 et 162 de l'employé 3.

La table suivante présente les fonctions disponibles, une brève description de la fonction et un exemple de la fonction sur les mêmes données que celles indiquées ci-dessus.

Fonction PATH – Retourne un texte délimité contenant les identificateurs de tous les parents de la ligne actuelle, en commençant par le niveau le plus ancien ou le plus élevé jusqu'au niveau actuel.

Agrandir le tableau

EmployeeKey	ParentEmployeeKey	Chemin
112		112
14	112	112|14
3	14	112|14|3
11	3	112|14|3|11
13	3	112|14|3|13
162	3	112|14|3|162
117	162	112|14|3|162|117
221	162	112|14|3|162|221
81	162	112|14|3|162|81

Fonction PATHLENGTH – Retourne le nombre de niveaux dans une fonction PATH() donnée, en commençant au niveau actuel jusqu'au niveau parent le plus ancien ou le plus élevé. Dans l'exemple suivant, la colonne PathLength est définie en tant que « `= PATHLENGTH([Path])` ». L'exemple inclut toutes les données de l'exemple Path() pour vous aider à mieux comprendre le mode opératoire de cette fonction.

Agrandir le tableau

EmployeeKey	ParentEmployeeKey	Chemin	PathLength
112		112	1
14	112	112|14	2
3	14	112|14|3	3

EmployeeKey	ParentEmployeeKey	Chemin	PathLength
11	3	112|14|3|11	4
13	3	112|14|3|13	4
162	3	112|14|3|162	4
117	162	112|14|3|162|117	5
221	162	112|14|3|162|221	5
81	162	112|14|3|162|81	5

Fonction PATHITEM – Retourne l'élément situé à la position spécifiée dans un résultat de type PATH(), en partant de la gauche. Dans l'exemple suivant, la colonne PathItem – 4e élément à partir de la gauche est définie en tant que « `= PATHITEM([Path], 4)` ». Cet exemple, qui utilise les données de l'exemple Path(), retourne l'élément EmployeeKey situé en quatrième position dans la chaîne Path en partant de la gauche.

Agrandir le tableau

EmployeeKey	ParentEmployeeKey	Chemin	PathItem - 4e élément à partir de la gauche
112		112	
14	112	112|14	
3	14	112|14|3	
11	3	112|14|3|11	11
13	3	112|14|3|13	13
162	3	112|14|3|162	162
117	162	112|14|3|162|117	162
221	162	112|14|3|162|221	162
81	162	112|14|3|162|81	162

Fonction PATHITEMREVERSE – Retourne l'élément situé à la *position* spécifiée dans un résultat de type PATH(), en partant de la droite.
Dans l'exemple suivant, la colonne PathItemReverse – 3e élément à partir de la droite est définie en tant que « `= PATHITEMREVERSE([Path], 3)` ». Cet exemple, qui utilise les

données de l'exemple Path(), retourne l'élément EmployeeKey situé en troisième position dans la chaîne Path en partant de la droite.

Agrandir le tableau

EmployeeKey	ParentEmployeeKey	Chemin	PathItemReverse - 3e élément à partir de la droite
112		112	
14	112	112|14	
3	14	112|14|3	112
11	3	112|14|3|11	14
13	3	112|14|3|13	14
162	3	112|14|3|162	14
117	162	112|14|3|162|117	3
221	162	112|14|3|162|221	3
81	162	112|14|3|162|81	3

Fonction PATHCONTAINS – Retourne **TRUE** si l'*élément* spécifié existe dans le *chemin* spécifié. Dans l'exemple suivant, la colonne PathContains – Employé 162 est définie en tant que « `= PATHCONTAINS([Path], "162")` ». Cet exemple retourne **TRUE** si le chemin donné contient l'employé 162. Cet exemple utilise les résultats de l'exemple Path() ci-dessus.

Agrandir le tableau

EmployeeKey	ParentEmployeeKey	Chemin	PathContains - Employé 162
112		112	FAUX
14	112	112|14	false
3	14	112|14|3	FAUX
11	3	112|14|3|11	FAUX
13	3	112|14|3|13	FAUX
162	3	112|14|3|162	VRAI
117	162	112|14|3|162|117	VRAI

Commentaires

Cette page a-t-elle été utile ? Yes No

PATH

Article • 20/10/2023

Retourne une chaîne de texte délimitée contenant les identificateurs de tous les parents de l'identificateur actuel, en commençant par le niveau le plus ancien et en continuant jusqu'au niveau actuel.

Syntaxe

DAX

```
PATH(<ID_columnName>, <parent_columnName>)
```

Paramètres

Agrandir le tableau

Terme	Définition
ID_columnName	Nom d'une colonne existante contenant l'identificateur unique des lignes de la table. Il ne peut pas s'agir d'une expression. La valeur *ID_columnName* doit être de type texte ou entier. Son type de données doit également correspondre à celui de la colonne référencée dans *parent_columnName*.
parent_columnName	Nom d'une colonne existante contenant l'identificateur unique du parent de la ligne actuelle. Il ne peut pas s'agir d'une expression. La valeur *parent_columnName* doit être de type texte ou entier. Son type de données doit également correspondre à celui de la valeur *ID_columnName*.

Valeur renvoyée

Chaîne de texte délimitée avec les identificateurs de tous les parents de l'identificateur actuel.

Notes

- Cette fonction est utilisée dans les tables possédant un type de hiérarchie interne pour retourner les éléments associés à la valeur de ligne actuelle. Par exemple,

dans une table d'employés contenant des employés, les responsables des employés et les responsables des responsables, vous pouvez retourner le chemin qui connecte un employé à son ou sa responsable.

- Le chemin n'est pas restreint à un seul niveau de relations parent-enfant. Il peut retourner des lignes associées situées plusieurs niveaux au-dessus de la ligne de départ spécifiée.
  - Le délimiteur utilisé pour séparer les ascendants est la barre verticale « | ».
  - Le même type de données (texte ou entier) doit être utilisé pour les valeurs *ID_columnName* et *parent_columnName*.
  - La valeur *parent_columnName* doit être présente dans *ID_columnName*. Autrement dit, vous ne pouvez pas rechercher un parent si aucune valeur n'est disponible au niveau enfant.
  - Si *parent_columnName* est vide, PATH() retourne la valeur *ID_columnName*. En d'autres termes, si vous recherchez le responsable d'un employé mais que la colonne *parent_columnName* est vide, la fonction PATH retourne simplement l'ID d'employé.
  - Si *ID_columnName* contient des doublons et que la valeur *parent_columnName* est la même pour ces doublons, PATH() retourne la valeur *parent_columnName* commune. Toutefois, si la valeur *parent_columnName* est différente pour ces doublons, PATH() retourne une erreur. En d'autres termes, si vous avez deux listes pour le même ID d'employé et que l'ID de responsable est le même pour les deux, la fonction PATH retourne l'ID de ce responsable. Toutefois, si deux ID d'employé identiques sont associés à des ID de responsable différents, la fonction PATH retourne une erreur.
  - Si *ID_columnName* est vide, PATH() retourne une valeur vide.
  - Si *ID_columnName* contient une barre verticale « | », PATH() retourne une erreur.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant crée une colonne calculée qui liste tous les responsables de chaque employé.

DAX

```
= PATH(Employee[EmployeeKey], Employee[ParentEmployeeKey])
```

Commentaires

Cette page a-t-elle été utile ? Yes No

PATHCONTAINS

Article • 20/10/2023

Retourne **TRUE** si l'élément (*item*) spécifié existe dans le chemin (*path*) spécifié.

Syntaxe

```
DAX
PATHCONTAINS(<path>, <item>)
```

Paramètres

Agrandir le tableau

Terme	Définition
path	Chaîne créée à la suite de l'évaluation d'une fonction PATH.
item	Expression Texte à rechercher dans le résultat de path.

Valeur renvoyée

Valeur **TRUE** si *item* existe dans *path* ; sinon, **FALSE**.

Notes

- Si *item* est un nombre entier, il est converti en texte, puis la fonction est évaluée. En cas d'échec de la conversion, la fonction retourne une erreur.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant crée une colonne calculée qui prend un ID de responsable et vérifie un ensemble d'employés. Si l'ID de responsable fait partie de la liste de responsables

retournée par la fonction PATH, la fonction PATHCONTAINS retourne la valeur true ; sinon, elle retourne la valeur false.

DAX

```
= PATHCONTAINS(PATH(Employee[EmployeeKey], Employee[ParentEmployeeKey]),
"23")
```

Commentaires

Cette page a-t-elle été utile ? Yes No

PATHITEM

Article • 20/10/2023

Retourne l'élément à la *position* spécifiée à partir d'une chaîne résultant de l'évaluation d'une fonction PATH. Les positions sont comptées de gauche à droite.

Syntaxe

DAX

```
PATHITEM(<path>, <position>[, <type>])
```

Paramètres

Agrandir le tableau

Terme	Définition
path	Chaîne de texte sous la forme des résultats d'une fonction PATH.
position	Expression entière spécifiant la position de l'élément à retourner.
type	(Facultatif) Énumération qui définit le type de données du résultat :

énumération de type

Agrandir le tableau

Énumération	Autre énumération	Description
TEXT	0	Les résultats sont retournés avec le type de données text (Valeur par défaut.)
INTEGER	1	Les résultats sont retournés sous forme d'entiers.

Valeur renvoyée

Identificateur retourné par la fonction PATH à la position spécifiée dans la liste d'identificateurs. Les éléments retournés par la fonction PATH sont classés du plus éloigné à l'actuel.

Notes

- Cette fonction peut être utilisée pour retourner un niveau spécifique à partir d'une hiérarchie retournée par une fonction PATH. Par exemple, vous pouvez retourner uniquement les responsables de niveau supérieur pour tous les employés.
- Si vous spécifiez un nombre pour *position* qui est inférieur à un (1) ou supérieur au nombre d'éléments dans *path*, la fonction PATHITEM retourne une valeur vide (BLANK).
- Si *type* n'est pas un élément d'énumération valide, une erreur est retournée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant retourne le responsable de troisième niveau de l'employé actuel ; il prend les ID de l'employé et du responsable comme entrée d'une fonction PATH qui retourne une chaîne avec la hiérarchie des parents à l'employé actuel. À partir de cette chaîne, PATHITEM retourne la troisième entrée sous la forme d'un entier.

DAX

```
= PATHITEM(PATH(Employee[EmployeeKey], Employee[ParentEmployeeKey]), 3, 1)
```

Commentaires

Cette page a-t-elle été utile ? Yes No

PATHITEMREVERSE

Article • 20/10/2023

Retourne l'élément à la *position* spécifiée à partir d'une chaîne résultant de l'évaluation d'une fonction PATH. Les positions sont comptées à rebours de droite à gauche.

Syntaxe

DAX

```
PATHITEMREVERSE(<path>, <position>[, <type>])
```

Paramètres

Agrandir le tableau

Terme	Définition
path	Chaîne de texte résultant de l'évaluation d'une fonction PATH.
position	Expression entière spécifiant la position de l'élément à retourner. La position est comptée à rebours de droite à gauche.
type	(Facultatif) Énumération qui définit le type de données du résultat :

énumération de type

Agrandir le tableau

Énumération	Autre énumération	Description
TEXT	0	Les résultats sont retournés avec le type de données text (Valeur par défaut.)
INTEGER	1	Les résultats sont retournés sous forme d'entiers.

Valeur renvoyée

Ascendant à la position n dans le chemin donné, en comptant de la position actuelle vers la plus ancienne.

Notes

- Cette fonction peut être utilisée pour obtenir un élément individuel d'une hiérarchie résultant d'une fonction PATH.
- Cette fonction inverse l'ordre standard de la hiérarchie afin que les éléments les plus proches soient listés en premier. Par exemple, si la fonction PATH retourne une liste de responsables au-dessus d'un employé dans une hiérarchie, la fonction PATHITEMREVERSE retourne le responsable immédiat de l'employé à la position 2, car la position 1 contient l'ID de l'employé.
- Si le nombre spécifié pour *position* est inférieur à un (1) ou supérieur au nombre d'éléments dans *path*, la fonction PATHITEM retourne une valeur vide (BLANK).
- Si *type* n'est pas un élément d'énumération valide, une erreur est retournée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant utilise une colonne d'ID d'employé comme entrée d'une fonction PATH et inverse la liste des éléments grands-parents qui sont retournés. La position spécifiée est 3 et le type de retour est 1. La fonction PATHITEMREVERSE retourne donc un entier représentant le responsable situé deux niveaux hiérarchiques au-dessus de l'employé.

DAX

```
= PATHITEMREVERSE(PATH(Employee[EmployeeKey], Employee[ParentEmployeeKey]),
3, 1)
```

Commentaires

Cette page a-t-elle été utile ? Yes No

PATHLENGTH

Article • 20/10/2023

Retourne le nombre de parents à l'élément spécifié dans un résultat PATH donné, y compris self.

Syntaxe

```
DAX

PATHLENGTH(<path>)
```

Paramètres

Agrandir le tableau

Terme	Définition
path	Expression textuelle résultant de l'évaluation d'une fonction PATH.

Valeur renvoyée

Nombre d'éléments parents de l'élément spécifié dans un résultat PATH donné, y compris l'élément spécifié.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant prend un ID d'employé comme entrée pour une fonction PATH et retourne la liste des responsables au-dessus de cet employé dans la hiérarchie. La fonction PATHLENGTH prend ce résultat et compte les différents niveaux d'employés et de responsables, y compris celui de l'employé avec lequel vous avez commencé.

```
```

DAX

```
= PATHLENGTH(PATH(Employee[EmployeeKey], Employee[ParentEmployeeKey]))
```

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonctions de relation

Article • 20/10/2023

Les fonctions de cette catégorie permettent de gérer et d'utiliser les relations entre les tables.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
CROSSFILTER	Spécifie la direction du filtrage croisé à utiliser dans un calcul pour une relation existant entre deux colonnes.
RELATED	Retourne une valeur associée provenant d'une autre table.
RELATEDTABLE	Évalue une expression de table dans un contexte modifié par les filtres spécifiés.
USERELATIONSHIP	Spécifie la relation à utiliser dans un calcul spécifique, comme celle qui existe entre columnName1 et columnName2.

Commentaires

Cette page a-t-elle été utile ? Yes No

CROSSFILTER

Article • 20/10/2023

Spécifie la direction du filtrage croisé à utiliser dans un calcul pour une relation existant entre deux colonnes.

Syntaxe

DAX

```
CROSSFILTER(<columnName1>, <columnName2>, <direction>)
```

Paramètres

Agrandir le tableau

Terme	Définition
columnName1	Nom complet d'une colonne existante spécifié avec la syntaxe DAX standard. Il représente généralement le côté « plusieurs » de la relation à utiliser. Si les arguments sont fournis dans l'ordre inverse, la fonction les permute avant de les utiliser. Cet argument ne peut pas être une expression.
columnName2	Nom complet d'une colonne existante spécifié avec la syntaxe DAX standard. Il représente généralement le côté « un » ou le côté de la recherche de la relation à utiliser. Si les arguments sont fournis dans l'ordre inverse, la fonction les permute avant de les utiliser. Cet argument ne peut pas être une expression.
Direction	Direction du filtre croisé à utiliser. Doit prendre l'une des valeurs suivantes : **None** : Aucun filtrage croisé n'est effectué sur cette relation. **Both** : Les filtres sur les deux côtés filtrent l'autre côté. **OneWay** : Les filtres sur un côté ou le côté de la recherche d'une relation filtrent l'autre côté. Cette option ne peut pas être utilisée avec une relation un-à-un. N'utilisez pas cette option sur une relation plusieurs-à-plusieurs, car il est impossible de déterminer le côté de la recherche. Utilisez plutôt OneWay_LeftFiltersRight ou OneWay_RightFiltersLeft. **OneWay_LeftFiltersRight** : les filtres sur le côté de <columnName1> filtrent le côté de <columnName2>. Cette option ne peut pas être utilisée avec une relation un-à-un ou plusieurs-à-un.

Terme	Définition
	OneWay_RightFiltersLeft : les filtres sur le côté de <columnName2> filtrent le côté de <columnName1>. Cette option ne peut pas être utilisée avec une relation un-à-un ou plusieurs-à-un.

Valeur renvoyée

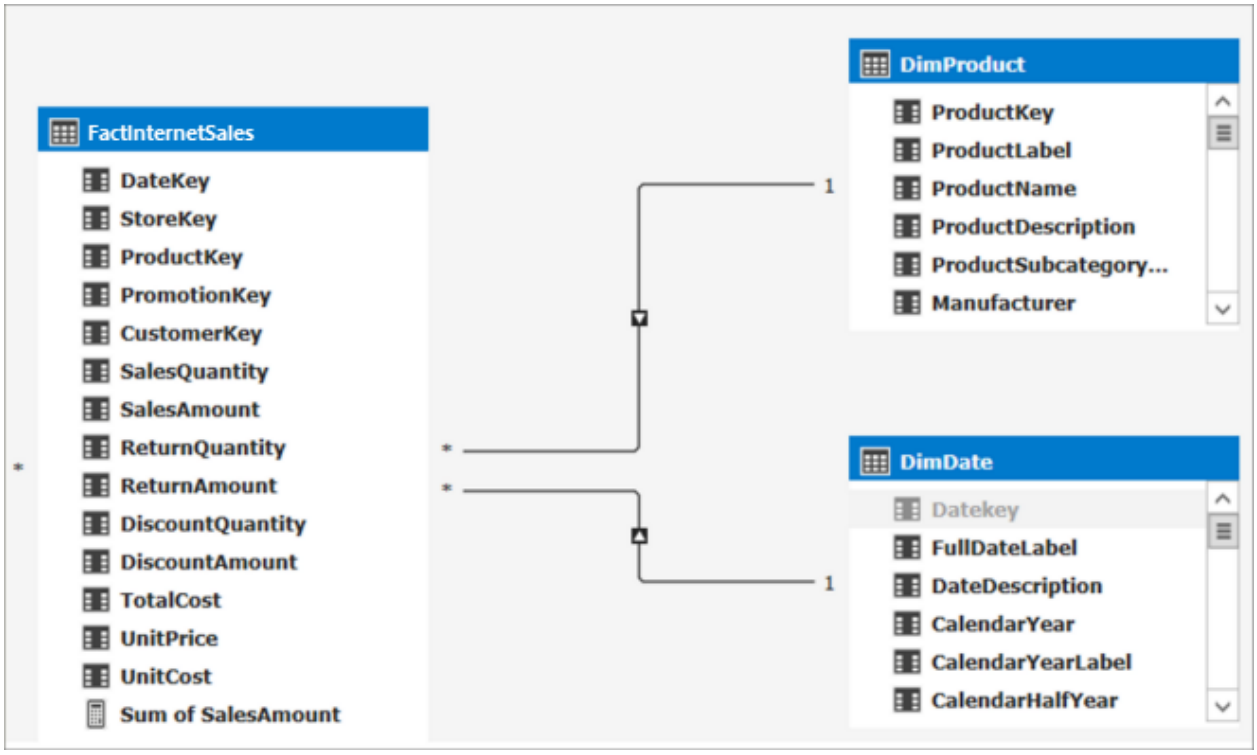
La fonction ne retourne aucune valeur. Elle définit uniquement la direction du filtrage croisé pour la relation indiquée et pour la durée de la requête.

Notes

- Dans le cas d'une relation 1:1, il n'y a pas de différence entre les directions one et both.
- CROSSFILTER peut être utilisé uniquement dans les fonctions qui utilisent un filtre comme argument, par exemple : CALCULATE, CALCULATETABLE, CLOSINGBALANCEMONTH, CLOSINGBALANCEQUARTER, CLOSINGBALANCEYEAR, OPENINGBALANCEMONTH, OPENINGBALANCEQUARTER, OPENINGBALANCEYEAR, TOTALMTD, TOTALQTD et TOTALYTD.
- CROSSFILTER utilise les relations existantes du modèle en identifiant les relations par leurs colonnes de fin.
- Avec la fonction CROSSFILTER, la définition du filtrage croisé d'une relation n'a pas d'importance. Le fait de définir la relation pour filtrer sur une ou deux directions dans le modèle n'affecte pas l'utilisation de la fonction. La fonction CROSSFILTER remplace toute définition de filtrage croisé existante.
- Une erreur est retournée si l'une des colonnes nommées comme argument ne fait pas partie d'une relation ou si les arguments appartiennent à des relations différentes.
- Si des expressions CALCULATE sont imbriquées et que plusieurs expressions CALCULATE contiennent une fonction CROSSFILTER, c'est la fonction CROSSFILTER la plus intérieure qui prévaut en cas de conflit ou d'ambiguïté.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Dans le modèle de diagramme suivant, DimProduct et DimDate présentent une relation de direction unique avec FactInternetSales.



On ne peut obtenir, par défaut, le nombre de produits vendus par année :

Row Labels	Sum of SalesAmount	Distinct Count of ProductKey
2005	$3,266,373.66	606
2006	$6,530,343.53	606
2007	$9,791,060.30	606
2008	$9,770,899.74	606
2009		606
2010		606
Grand Total	**$29,358,677.22**	**606**

Il existe deux façons d'obtenir le nombre de produits par année :

- Activez le filtrage croisé bidirectionnel sur la relation. Cela modifiera la façon dont fonctionnent les filtres pour toutes les données entre ces deux tables.
- Utilisez la fonction CROSSFILTER pour modifier la façon dont fonctionnent les relations pour cette mesure uniquement.

Avec DAX, vous pouvez utiliser la fonction CROSSFILTER pour modifier le comportement de la direction du filtre croisé entre deux colonnes définies par une relation. Dans ce cas, l'expression DAX se présente comme suit :

DAX

```
BiDi:= CALCULATE([Distinct Count of ProductKey],
CROSSFILTER(FactInternetSales[ProductKey], DimProduct[ProductKey] , Both))
```

En utilisant la fonction CROSSFILTER dans notre expression de mesure, nous obtenons les résultats attendus :

Row Labels	Sum of SalesAmount	Distinct Count of ProductKey	BiDi
2005	$3,266,373.66	606	25
2006	$6,530,343.53	606	56
2007	$9,791,060.30	606	133
2008	$9,770,899.74	606	102
2009		606	
2010		606	
Grand Total	**$29,358,677.22**	**606**	**606**

Commentaires

Cette page a-t-elle été utile ? Yes No

RELATED

Article • 20/10/2023

Retourne une valeur associée provenant d'une autre table.

Syntaxe

```DAX
RELATED(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne contenant les valeurs à récupérer.

Valeur de retour

Valeur unique associée à la ligne actuelle.

Remarques

- La fonction RELATED nécessite une relation entre la table actuelle et la table avec les informations associées. Vous spécifiez la colonne contenant les données désirées, et la fonction suit une relation plusieurs-à-un pour extraire la valeur de la colonne spécifiée dans la table associée. En l'absence de relation, vous devez en créer une.
- Quand la fonction RELATED effectue une recherche, elle examine toutes les valeurs dans la table spécifiée, quels que soient les filtres qui ont pu être appliqués.
- La fonction RELATED ayant besoin d'un contexte de ligne, elle peut uniquement être utilisée dans une expression de colonne calculée, où le contexte de ligne actuel n'est pas ambigu, ou comme fonction imbriquée dans une expression qui utilise une fonction d'analyse de table. Une fonction d'analyse de table telle que

SUMX obtient la valeur de la ligne actuelle, puis analyse une autre table à la recherche d'instances de cette valeur.

- La fonction RELATED ne peut pas être utilisée pour extraire une colonne dans une relation limitée.

Exemple

Dans l'exemple suivant, la mesure Ventes sur Internet hors États-Unis est créée pour produire un rapport de ventes excluant les ventes aux États-Unis. Pour créer la mesure, la table InternetSales_USD doit être filtrée pour exclure toutes les ventes appartenant aux États-Unis dans la table SalesTerritory. Les États-Unis, en tant que pays, apparaissent 5 fois dans la table SalesTerritory ; une fois pour chacune des régions suivantes : Nord-Ouest, Nord-Est, Centre, Sud-Ouest et Sud-Est.

Pour filtrer les ventes sur Internet et créer la mesure, une première approche peut être d'ajouter une expression de filtre, comme ceci :

DAX

```
FILTER('InternetSales_USD'
, 'InternetSales_USD'[SalesTerritoryKey]<>1 &&
'InternetSales_USD'[SalesTerritoryKey]<>2 &&
'InternetSales_USD'[SalesTerritoryKey]<>3 &&
'InternetSales_USD'[SalesTerritoryKey]<>4 &&
'InternetSales_USD'[SalesTerritoryKey]<>5)
```

Cependant, cette approche n'est pas intuitive, est sujette aux erreurs de frappe et peut ne pas fonctionner si l'une des régions existantes est amenée à être divisée à l'avenir.

Une meilleure approche consiste à utiliser la relation existante entre InternetSales_USD et SalesTerritory et à déclarer explicitement que le pays doit être différent des États-Unis. Pour ce faire, créez une expression de filtre comme suit :

DAX

```
FILTER( 'InternetSales_USD',
RELATED('SalesTerritory'[SalesTerritoryCountry])<>"United States")
```

Cette expression utilise la fonction RELATED pour rechercher la valeur du pays dans la table SalesTerritory, en commençant par la valeur de la colonne clé, SalesTerritoryKey, dans la table InternetSales_USD. Le résultat de la recherche est utilisé par la fonction Filter pour déterminer si la ligne InternetSales_USD est filtrée ou non.

> **Notes**
>
> Si l'exemple ne fonctionne pas, vous devrez peut-être créer une relation entre les tables.

DAX

```
= SUMX(FILTER( 'InternetSales_USD'
            ,  RELATED('SalesTerritory'[SalesTerritoryCountry])
               <>"United States"
             )
     ,'InternetSales_USD'[SalesAmount_USD])
```

La table suivante affiche uniquement les totaux de chaque région pour prouver que l'expression de filtre dans la mesure, Ventes sur Internet hors États-Unis, fonctionne comme prévu.

Agrandir le tableau

Row Labels	Internet Sales	Non USA Internet Sales
Australie	4 999 021,84 $	4 999 021,84 $
Canada	1,343,109.10 $	1,343,109.10 $
France	2 490 944,57 $	2 490 944,57 $
Allemagne	2 775 195,60 $	2 775 195,60 $
Royaume-Uni	5 057 076,55 $	5 057 076,55 $
États-Unis	9 389 479,79 $	
Total général	26 054 827,45 $	16 665 347,67 $

La table suivante montre un exemple de résultat si vous utilisiez cette mesure dans un visuel de table de rapport :

Agrandir le tableau

Row Labels	Accessories	Bikes	Clothing	Grand Total
2005		1 526 481,95 $		1 526 481,95 $
2006		3 554 744,04 $		3 554 744,04 $

Row Labels	Accessories	Bikes	Clothing	Grand Total
2007	156 480,18 $	5 640 106,05 $	70 142,77 $	5 866 729,00 $
2008	228 159,45 $	5 386 558,19 $	102 675,04 $	5 717 392,68 $
Total général	384 639,63 $	16 107 890,23 $	172 817,81 $	16 665 347,67 $

Contenu connexe

RELATEDTABLE
Fonctions de filtrage

Commentaires

Cette page a-t-elle été utile ? Yes No

RELATEDTABLE

Article • 20/10/2023

Évalue une expression de table dans un contexte modifié par les filtres spécifiés.

Syntaxe

```DAX
RELATEDTABLE(<tableName>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableName	Nom d'une table existante, spécifié avec la syntaxe DAX standard. Il ne peut pas s'agir d'une expression.

Valeur de retour

Table de valeurs.

Notes

- La fonction RELATEDTABLE change le contexte dans lequel les données sont filtrées et évalue l'expression dans le nouveau contexte spécifié.
- Cette fonction est un raccourci pour la fonction CALCULATETABLE sans expression logique.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant utilise la fonction RELATEDTABLE pour créer une colonne calculée avec les ventes sur Internet dans la table Product Category :

```
DAX
= SUMX( RELATEDTABLE('InternetSales_USD')
     , [SalesAmount_USD])
```

Le tableau suivant présente les résultats :

Agrandir le tableau

Product Category Key	Product Category AlternateKey	Product Category Name	Internet Sales
1	1	Bikes	$28,318,144.65
2	2	Components	
3	3	Clothing	$339,772.61
4	4	Accessories	$700,759.96

Contenu connexe

CALCULATETABLE
Fonctions de filtrage

Commentaires

Cette page a-t-elle été utile ? Yes No

USERELATIONSHIP

Article • 20/10/2023

Spécifie la relation à utiliser dans un calcul spécifique, comme celle qui existe entre columnName1 et columnName2.

Syntaxe

```
DAX

USERELATIONSHIP(<columnName1>,<columnName2>)
```

Paramètres

Agrandir le tableau

Terme	Définition
columnName1	Nom complet d'une colonne existante spécifié avec la syntaxe DAX standard. Il représente généralement le côté « plusieurs » de la relation à utiliser. Si les arguments sont fournis dans l'ordre inverse, la fonction les permute avant de les utiliser. Cet argument ne peut pas être une expression.
columnName2	Nom complet d'une colonne existante spécifié avec la syntaxe DAX standard. Il représente généralement le côté « un » ou le côté de la recherche de la relation à utiliser. Si les arguments sont fournis dans l'ordre inverse, la fonction les permute avant de les utiliser. Cet argument ne peut pas être une expression.

Valeur de retour

La fonction ne retourne aucune valeur. Elle autorise uniquement la relation indiquée pendant la durée du calcul.

Remarques

- USERELATIONSHIP peut être utilisé uniquement dans les fonctions qui utilisent un filtre comme argument, par exemple : CALCULATE, CALCULATETABLE, CLOSINGBALANCEMONTH, CLOSINGBALANCEQUARTER, CLOSINGBALANCEYEAR, OPENINGBALANCEMONTH, OPENINGBALANCEQUARTER, OPENINGBALANCEYEAR, TOTALMTD, TOTALQTD et TOTALYTD.

- USERELATIONSHIP ne peut pas être utilisé quand la sécurité au niveau des lignes est définie pour la table dans laquelle la mesure est incluse. Par exemple, `CALCULATE(SUM([SalesAmount]), USERELATIONSHIP(FactInternetSales[CustomerKey], DimCustomer[CustomerKey]))` retourne une erreur si la sécurité au niveau des lignes est définie pour DimCustomer.
- USERELATIONSHIP utilise les relations existantes du modèle, en identifiant les relations par leurs colonnes de fin.
- Dans USERELATIONSHIP, l'état d'une relation n'a pas d'importance. Autrement dit, que la relation soit active ou non, cela n'a pas d'effet sur l'utilisation de la fonction. Même si la relation est inactive, elle est utilisée et remplace les autres relations actives qui peuvent être présentes dans le modèle, mais qui ne sont pas mentionnées dans les arguments de la fonction.
- Une erreur est retournée si l'une des colonnes nommées comme argument ne fait pas partie d'une relation ou si les arguments appartiennent à des relations différentes.
- Si plusieurs relations sont nécessaires pour joindre la table A à la table B dans un calcul, chaque relation doit être indiquée dans une fonction USERELATIONSHIP différente.
- Si des expressions CALCULATE sont imbriquées et que plusieurs expressions CALCULATE contiennent une fonction USERELATIONSHIP, c'est la fonction USERELATIONSHIP la plus intérieure qui prévaut en cas de conflit ou d'ambiguïté.
- Jusqu'à 10 fonctions USERELATIONSHIP peuvent être imbriquées ; cependant, votre expression peut avoir un niveau d'imbrication supérieur. Par exemple, l'exemple d'expression ci-dessous est imbriquée sur 3 niveaux, alors que USERELATIONSHIP est imbriquée sur seulement 2 niveaux : `=CALCULATE(CALCULATE( CALCULATE( <anyExpression>, USERELATIONSHIP( t1[colA], t2[colB])), t99[colZ]=999), USERELATIONSHIP( t1[colA], t2[colA]))`.
- Pour les relations 1 à 1, la fonction USERELATIONSHIP active la relation dans une direction uniquement. En particulier, les filtres peuvent uniquement passer de la table *columnName2* à la table *columnName1*. Si vous souhaitez effectuer un filtrage croisé bidirectionnel, deux fonctions USERELATIONSHIP avec une direction opposée peuvent être utilisées dans le même calcul. Par exemple : `CALCULATE(..., USERELATIONSHIP(T1[K], T2[K]), USERELATIONSHIP(T2[K], T1[K]))`.

Exemple

L'exemple suivant montre comment remplacer la relation par défaut active entre les tables InternetSales et DateTime. La relation par défaut existe entre la colonne OrderDate de la table InternetSales et la colonne Date de la table DateTime.

Pour calculer la somme des ventes Internet et autoriser un découpage par date d'expédition (ShippingDate) et non par la traditionnelle date de commande (OrderDate), créez une mesure, [InternetSales par ShippingDate], à l'aide de l'expression suivante :

DAX

```
= CALCULATE(SUM(InternetSales[SalesAmount]),
USERELATIONSHIP(InternetSales[ShippingDate], DateTime[Date]))
```

Les relations entre InternetSales[ShipmentDate] et DateTime[Date] doivent exister et ne doivent pas constituer la relation active. De même, la relation entre InternetSales[OrderDate] et DateTime[Date] doit exister et doit être la relation active.

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonctions statistiques

Article • 20/10/2023

Les fonctions statistiques calculent des valeurs liées aux distributions statistiques et à la probabilité, telles que l'écart type et le nombre de permutations.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
BETA.DIST	Retourne la distribution Bêta.
BETA.INV	Retourne l'inverse de la fonction de densité de probabilité cumulative Bêta (BETA.DIST).
CHISQ.DIST	Retourne la distribution du Khi-deux.
CHISQ.DIST.RT	Retourne la probabilité unilatérale à droite de la distribution du Khi-deux.
CHISQ.INV	Retourne l'inverse de la probabilité unilatérale à gauche de la distribution du Khi-deux.
CHISQ.INV.RT	Retourne l'inverse de la probabilité unilatérale à droite de la distribution du Khi-deux.
COMBIN	Retourne le nombre de combinaisons pour un nombre d'éléments donné.
COMBINA	Retourne le nombre de combinaisons (avec les répétitions) pour un nombre d'éléments donné.
CONFIDENCE.NORM	L'intervalle de confiance est une plage de valeurs.
CONFIDENCE.T	Retourne l'intervalle de confiance pour la moyenne d'une population en utilisant une distribution T de Student.
EXPON.DIST	Retourne la distribution exponentielle.
GEOMEAN	Retourne la moyenne géométrique des nombres dans une colonne.
GEOMEANX	Retourne la moyenne géométrique d'une expression évaluée pour chaque ligne d'une table.
LINEST	Utilise la méthode des moindres carrés pour calculer une ligne droite qui correspond le mieux aux données spécifiées.

Fonction	Description
LINESTX	Utilise la méthode des moindres carrés pour calculer une ligne droite qui correspond le mieux aux données spécifiées. Les données sont le résultat des expressions évaluées pour chaque ligne d'une table.
MEDIAN	Retourne la valeur médiane des nombres d'une colonne.
MEDIANX	Retourne le nombre médian d'une expression évaluée pour chaque ligne d'une table.
NORM.DIST	Retourne la distribution normale pour la moyenne et l'écart type spécifiés.
NORM.INV	Inverse de la distribution cumulative normale pour la moyenne et l'écart type spécifiés.
NORM.S.DIST	Retourne la distribution normale standard (avec une moyenne égale à zéro et un écart type égal à un).
NORM.S.INV	Retourne l'inverse de la distribution cumulative normale standard.
PERCENTILE.EXC	Retourne le énième centile k des valeurs d'une plage, où k se trouve dans la plage de 0 à 1, non compris.
PERCENTILE.INC	Retourne le énième centile k des valeurs d'une plage, où k se trouve dans la plage de 0 à 1, inclus.
PERCENTILEX.EXC	Retourne le centile d'une expression évaluée pour chaque ligne d'une table.
PERCENTILEX.INC	Retourne le centile d'une expression évaluée pour chaque ligne d'une table.
PERMUT	Retourne le nombre de permutations d'un nombre donné d'objets qui peuvent être choisis parmi des objets de nombre.
POISSON.DIST	Retourne la distribution de Poisson.
RANK.EQ	Retourne le classement d'un nombre dans une liste de nombres.
RANKX	Retourne le classement d'un nombre dans une liste de nombres pour chaque ligne de l'argument *table*.
SAMPLE	Retourne un échantillon de N lignes de la table spécifiée.
STDEV.P	Retourne l'écart type de l'ensemble de la population.
STDEV.S	Retourne l'écart type d'un échantillon de population.
STDEVX.P	Retourne l'écart type de l'ensemble de la population.
STDEVX.S	Retourne l'écart type d'un échantillon de la population.

Fonction	Description
T.DIST	Retourne la distribution T unilatérale à gauche de Student.
T.DIST.2T	Retourne la distribution T bilatérale de Student.
T.DIST.RT	Retourne la distribution T de Student unilatérale à droite.
T.INV	Retourne l'inverse unilatéral à gauche de la distribution T de Student.
T.INV.2t	Retourne l'inverse bilatéral de la distribution T de Student.
VAR.P	Retourne la variance de l'ensemble de la population.
VAR.S	Retourne la variance d'un échantillon de la population.
VARX.P	Retourne la variance de l'ensemble de la population.
VARX.S	Retourne la variance d'un échantillon de population.

Commentaires

Cette page a-t-elle été utile ? Yes No

BETA.DIST

Article • 20/10/2023

Retourne la distribution Bêta. La distribution Bêta est couramment utilisée pour étudier la variation du pourcentage d'une valeur dans des échantillons. Il peut s'agir, par exemple, de la fraction d'une journée que des gens passent à regarder la télévision.

Syntaxe

DAX

```
BETA.DIST(x,alpha,beta,cumulative,[A],[B])
```

Paramètres

Agrandir le tableau

Terme	Définition
x	Valeur entre A et B à laquelle évaluer la fonction
Alpha	Paramètre de la distribution.
Bêta	Paramètre de la distribution.
A	facultatif. Limite inférieure de l'intervalle x.
B	facultatif. Limite supérieure de l'intervalle x.

Valeur renvoyée

Retourne la distribution Bêta.

Notes

- Si un argument n'est pas numérique, BETA.DIST retourne la valeur d'erreur #VALUE! #NUM!.
- Si un argument n'est pas un entier, il est arrondi.
- Si alpha ≤ 0 ou beta ≤ 0, BETA.DIST retourne la valeur d'erreur #NUM! #NUM!.

- Si x < A, x > B ou A = B, BETA.DIST retourne la valeur d'erreur #NUM! #NUM!.
- Si vous omettez les valeurs A et B, BETA.DIST utilise la distribution Bêta cumulée standard de sorte que A = 0 et B = 1.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Commentaires

Cette page a-t-elle été utile ? Yes No

BETA.INV

Article • 20/10/2023

Retourne l'inverse de la fonction de densité de probabilité cumulative Bêta (BETA.DIST).

Si la probabilité = BETA.DIST(x,...TRUE), alors BETA.INV(probabilité,...) = x. La distribution Bêta peut être utilisée dans la planification de projet pour modéliser les durées d'exécution probables pour une durée d'exécution attendue et une variabilité données.

Syntaxe

DAX

```
BETA.INV(probability,alpha,beta,[A],[B])
```

Paramètres

Agrandir le tableau

Terme	Définition
Probabilité	Probabilité associée à la distribution Bêta.
Alpha	Paramètre de la distribution.
Bêta	Paramètre de la distribution.
A	facultatif. Limite inférieure de l'intervalle x.
B	facultatif. Limite supérieure de l'intervalle x.

Valeur renvoyée

Retourne l'inverse de la fonction de densité de probabilité cumulative Bêta (BETA.DIST).

Notes

- Si un argument n'est pas numérique, BETA.INV retourne la valeur d'erreur #NUM!.
- Si un argument n'est pas un entier, il est arrondi.

- Si alpha ≤ 0 ou beta ≤ 0, BETA. INV retourne la valeur d'erreur #NUM!.
- Si la probabilité ≤ 0 ou si la probabilité > 1, BETA.INV retourne la valeur d'erreur #NUM!.
- Si vous omettez les valeurs de A et B, BETA.INV utilise la distribution Bêta cumulée standard, de sorte que A = 0 et B = 1.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Commentaires

Cette page a-t-elle été utile ? Yes No

CHISQ.DIST

Article • 20/10/2023

Retourne la distribution du Khi-deux.

La distribution du Khi-deux est couramment utilisée pour étudier la variation du pourcentage d'une valeur dans des exemples, telle qu'une fraction de journée passée à regarder la télévision.

Syntaxe

DAX

```
CHISQ.DIST(<x>, <deg_freedom>, <cumulative>)
```

Paramètres

Agrandir le tableau

Terme	Définition
x	Valeur à laquelle vous voulez évaluer la distribution.
Deg_freedom	Nombre de degrés de liberté.
cumulative	Valeur logique qui détermine la forme de la fonction. Si la valeur cumulative est TRUE, la fonction CHISQ.DIST retourne la fonction de distribution cumulative. Si la valeur est FALSE, elle retourne la fonction de probabilité de densité.

Valeur de retour

Distribution du Khi-deux.

Remarques

- Si x ou deg_freedom n'est pas numérique, une erreur est retournée.
- Si deg_freedom n'est pas un entier, il est arrondi.
- Si x < 0, une erreur est retournée.

- Si deg_freedom < 1 ou deg_freedom > 10^10, une erreur est retournée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La requête DAX suivante :

```
DAX
EVALUATE { CHISQ.DIST(2, 2, TRUE) }
```

retourne :

Agrandir le tableau

[Value]
0.632120558828558

Commentaires

Cette page a-t-elle été utile ? Yes No

CHISQ.DIST.RT

Article • 20/10/2023

Retourne la probabilité unilatérale à droite de la distribution du Khi-deux.

La distribution du Khi-deux est associée à un test Khi-deux. Utilisez le test Khi-deux pour comparer les valeurs observées et les valeurs attendues. Par exemple, une expérimentation génétique peut prévoir que la prochaine génération de plantes présentera un certain jeu de couleurs. En comparant les résultats observés à ceux attendus, vous pouvez déterminer si votre hypothèse initiale est valide.

Syntaxe

DAX

```
CHISQ.DIST.RT(<x>, <deg_freedom>)
```

Paramètres

Agrandir le tableau

Terme	Définition
x	Valeur à laquelle vous voulez évaluer la distribution.
Deg_freedom	Nombre de degrés de liberté.

Valeur de retour

Probabilité unilatérale à droite de la distribution du Khi-deux.

Remarques

- Si x ou deg_freedom n'est pas numérique, une erreur est retournée.
- Si deg_freedom n'est pas un entier, il est arrondi.
- Si x < 0, une erreur est retournée.
- Si deg_freedom < 1 ou deg_freedom > 10^10, une erreur est retournée.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La requête DAX suivante :

```
DAX

EVALUATE { CHISQ.DIST.RT(2, 5) }
```

retourne :

Agrandir le tableau

[Value]
0.84914503608461

Commentaires

Cette page a-t-elle été utile ? Yes No

CHISQ.INV

Article • 20/10/2023

Retourne l'inverse de la probabilité unilatérale à gauche de la distribution du Khi-deux.

La distribution du Khi-deux est couramment utilisée pour étudier la variation du pourcentage d'une valeur dans des exemples, telle qu'une fraction de journée passée à regarder la télévision.

Syntaxe

```DAX
CHISQ.INV(probability,deg_freedom)
```

Paramètres

Agrandir le tableau

Terme	Définition
Probabilité	Probabilité associée à la distribution du Khi-deux.
Deg_freedom	Nombre de degrés de liberté.

Valeur de retour

Retourne l'inverse de la probabilité unilatérale à gauche de la distribution du Khi-deux.

Remarques

- Si l'argument n'est pas numérique, CHISQ.INV retourne la valeur d'erreur #VALUE!.
- Si la probabilité est < à 0 ou > à 1, CHISQ.INV retourne #NUM! #NUM!.
- Si deg_freedom n'est pas un entier, il est arrondi.
- Si deg_freedom < 0 ou si deg_freedom > 10^10, CHISQ.INV retourne #NUM! #NUM!.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Formule	Description	Résultat
= CHISQ.INV(0.93,1)	Inverse de la probabilité unilatérale à gauche de la distribution du Khi-deux pour 0.93, avec 1 degré de liberté.	5.318520074
= CHISQ.INV(0.6,2)	Inverse de la probabilité unilatérale à gauche de la distribution du Khi-deux pour 0.6, avec 2 degrés de liberté.	1.832581464

Commentaires

Cette page a-t-elle été utile ? Yes No

CHISQ.INV.RT

Article • 20/10/2023

Retourne l'inverse de la probabilité unilatérale à droite de la distribution du Khi-deux.

Si la probabilité = CHISQ.DIST.RT(x,...), alors CHISQ.INV.RT(probabilité,...) = x. Utilisez cette fonction pour comparer les résultats observés à ceux attendus et déterminer si votre hypothèse initiale est valide.

Syntaxe

DAX

```
CHISQ.INV.RT(probability,deg_freedom)
```

Paramètres

Agrandir le tableau

Terme	Définition
Probabilité	Probabilité associée à la distribution du Khi-deux.
Deg_freedom	Nombre de degrés de liberté.

Valeur de retour

Retourne l'inverse de la probabilité unilatérale à droite de la distribution du Khi-deux.

Notes

- Si un argument n'est pas numérique, CHISQ.INV.RT retourne #VALUE! #NUM!.
- Si la probabilité est < à 0 ou > à 1, CHISQ.INV.RT retourne #NUM! #NUM!.
- Si deg_freedom n'est pas un entier, il est arrondi.
- Si deg_freedom est < à 1, CHISQ.INV.RT retourne #NUM! #NUM!.
- Pour une valeur de probabilité donnée, CHISQ.INV.RT recherche cette valeur x de sorte que CHISQ.DIST.RT(x, deg_freedom) = la probabilité. La précision de

CHISQ.INV.RT dépend donc de la précision de CHISQ.DIST.RT. CHISQ.INV.RT utilise une technique de recherche itérative. En l'absence de convergence de la recherche après 64 itérations, la fonction retourne #N/A comme valeur d'erreur.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Commentaires

Cette page a-t-elle été utile ? Yes No

COMBIN

Article • 20/10/2023

Retourne le nombre de combinaisons pour un nombre d'éléments donné. Utilisez COMBIN pour déterminer le nombre total possible de groupes pour un nombre donné d'éléments.

Syntaxe

DAX

```
COMBIN(number, number_chosen)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre d'éléments.
number_chosen	Nombre d'éléments dans chaque combinaison.

Valeur de retour

Retourne le nombre de combinaisons pour un nombre d'éléments donné.

Remarques

- Les arguments numériques sont tronqués en entiers.
- Si un argument n'est pas numérique, COMBIN retourne la #NUM!.
- Si number < 0, number_chosen < 0 ou number < number_chosen, COMBIN retourne la #NUM!.
- Une combinaison est un ensemble ou sous-ensemble d'éléments, quel que soit leur ordre interne. Il faut distinguer les combinaisons des permutations dans lesquelles l'ordre interne est important.

- Le nombre de combinaisons est le suivant, où number = n et number_chosen = k :

$$\binom{n}{k} = \frac{P_{k,n}}{k!} = \frac{n!}{k!(n-k)!}$$

  Où

$$P_{k,n} = \frac{n!}{(n-k)!}$$

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Formule	Description	Résultats
= COMBIN(8,2)	Équipes de deux personnes pouvant être formées à partir de 8 candidats.	28

Commentaires

Cette page a-t-elle été utile ? Yes No

COMBINA

Article • 20/10/2023

Retourne le nombre de combinaisons (avec les répétitions) pour un nombre d'éléments donné.

Syntaxe

DAX

```
COMBINA(number, number_chosen)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Doit être supérieur ou égal à 0 et supérieur ou égal à number_chosen. Les valeurs non entières sont tronquées.
number_chosen	Doit être supérieur ou égal à 0. Les valeurs non entières sont tronquées.

Valeur de retour

Retourne le nombre de combinaisons (avec les répétitions) pour un nombre d'éléments donné.

Remarques

- Si la valeur d'un argument ne respecte par les contraintes correspondantes, la fonction COMBINA retourne la valeur d'erreur #NUM! #NUM!.
- Si l'un des arguments est une valeur non numérique, la fonction COMBINA retourne la valeur d'erreur #VALUE! #NUM!.
- L'équation suivante est utilisée. N correspond au paramètre number et M au paramètre number_chosen :

$$\binom{N+M-1}{N-1}$$

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Formule	Description	Résultat
= COMBINA(4,3)	Retourne le nombre de combinaisons (avec les répétitions) pour 4 et 3.	20
= COMBINA(10,3)	Retourne le nombre de combinaisons (avec les répétitions) pour 10 et 3.	220

Commentaires

Cette page a-t-elle été utile ? Yes No

CONFIDENCE.NORM

Article • 20/10/2023

L'intervalle de confiance est une plage de valeurs. La moyenne de votre échantillon, x, se trouve au centre de cette plage et la plage est x ± CONFIDENCE.NORM. Par exemple, si x est la moyenne de l'échantillon des délais de livraison pour les produits commandés par courrier électronique, x ± CONFIDENCE.NORM est une plage de moyennes de population. Pour toute moyenne de population, µ0, la probabilité dans cette plage d'obtenir une moyenne de l'échantillon plus éloignée de µ0 que x est supérieure à la valeur alpha ; pour toute moyenne de population, µ0, la probabilité hors de cette plage d'obtenir une moyenne de l'échantillon plus éloignée de µ0 que x est inférieure à alpha. En d'autres termes, supposons que nous utilisons x, standard_dev et size pour construire un test bilatéral à un niveau de précision alpha de l'hypothèse que la moyenne de population est µ0. Par la suite, nous ne rejetterons pas cette hypothèse si µ0 figure dans l'intervalle de confiance et rejetterons cette hypothèse si µ0 ne figure pas dans l'intervalle de confiance. L'intervalle de confiance ne nous permet pas de déduire qu'il y a la probabilité 1 - alpha que le délai de livraison de notre prochain paquet se trouve dans l'intervalle de confiance.

Syntaxe

DAX

```
CONFIDENCE.NORM(alpha,standard_dev,size)
```

Paramètres

Agrandir le tableau

Terme	Définition
alpha	Niveau de précision utilisé pour calculer le niveau de confiance. Le niveau de confiance est égal à 100*(1 - alpha) %. En d'autres termes, une valeur alpha de 0,05 indique un niveau de confiance de 95 pour cent.
standard_dev	Écart type de la population pour la plage de données, supposé connu.
standard_dev,size	Taille de l'échantillon.

Valeur de retour

Plage de valeurs

Remarques

- Si un argument n'est pas numérique, CONFIDENCE.NORM retourne la #NUM!.
- Si alpha ≤ 0 ou alpha ≥ 1, CONFIDENCE.NORM retourne la #NUM!.
- Si standard_dev ≤ 0, CONFIDENCE.NORM retourne la #NUM!.
- Si size n'est pas un entier, il est arrondi.
- Si size < 1, CONFIDENCE.NORM retourne la #NUM!.
- Si nous supposons que la valeur alpha est égale à 0,05, nous devons calculer la zone sous la courbe normale standard qui est égale à (1 - alpha) ou 95 pour cent. Cette valeur est ± 1,96. L'intervalle de confiance est donc :

$$\overline{x} \pm 1.96\left(\frac{\sigma}{\sqrt{n}}\right)$$

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Commentaires

Cette page a-t-elle été utile ? Yes No

CONFIDENCE.T

Article • 20/10/2023

Retourne l'intervalle de confiance pour la moyenne d'une population en utilisant une distribution T de Student.

Syntaxe

DAX

```
CONFIDENCE.T(alpha,standard_dev,size)
```

Paramètres

Agrandir le tableau

Terme	Définition
alpha	Niveau de précision utilisé pour calculer le niveau de confiance. Le niveau de confiance est égal à 100*(1 - alpha) %. En d'autres termes, une valeur alpha de 0,05 indique un niveau de confiance de 95 pour cent.
standard_dev	Écart type de la population pour la plage de données, supposé connu.
size	Taille de l'échantillon.

Valeur de retour

Retourne l'intervalle de confiance pour la moyenne d'une population en utilisant une distribution T de Student.

Remarques

- Si un argument n'est pas numérique, CONFIDENCE.T retourne la valeur d'erreur #VALUE! #NUM!.
- Si alpha ≤ 0 ou alpha ≥ 1, CONFIDENCE.T retourne la valeur d'erreur #NUM! #NUM!.
- Si standard_dev ≤ 0, CONFIDENCE.T retourne la valeur d'erreur #NUM! #NUM!.

- Si le paramètre size n'est pas un entier, il est arrondi.
- Si le paramètre size est égal à 1, CONFIDENCE.T retourne la valeur d'erreur #DIV/0! #NUM!.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Agrandir le tableau

Formule	Description	Résultat
= CONFIDENCE.T(0.05,1,50)	Intervalle de confiance pour la moyenne d'une population basé sur une taille d'échantillon de 50, avec un niveau de précision de 5 % et un écart type de 1. Il est basé sur la distribution T de Student.	0.284196855

Commentaires

Cette page a-t-elle été utile ? Yes No

EXPON.DIST

Article • 20/10/2023

Retourne la distribution exponentielle. Utilisez EXPO.DIST pour modéliser le temps séparant des événements, tel le temps qu'il faut pour recevoir de l'argent d'un guichet automatique. Par exemple, vous pouvez utiliser EXPON.DIST pour déterminer la probabilité que le processus ne dépasse pas 1 minute.

Syntaxe

```
DAX

EXPON.DIST(x,lambda,cumulative)
```

Paramètres

Agrandir le tableau

Terme	Définition
x	Obligatoire. Valeur de la fonction.
lambda	Obligatoire. Valeur du paramètre.
cumulative	Obligatoire. Valeur logique qui indique la forme de la fonction exponentielle à fournir. Si la valeur cumulative est TRUE, EXPON.DIST retourne la fonction de distribution cumulative ; si la valeur est FALSE, la fonction de probabilité de densité est retournée.

Valeur de retour

Retourne la distribution exponentielle.

Remarques

- Si x ou lambda n'est pas numérique, EXPON.DIST retourne la #NUM!.
- Si x ou lambda n'est pas un entier, il est arrondi.
- Si x < 0, EXPO.DIST retourne la #NUM!.

- Si lambda ≤ 0, EXPON.DIST retourne la #NUM!.
- L'équation de la fonction de densité de probabilité est la suivante :

$$f(x;\lambda) = \lambda e^{-\lambda x}$$

- L'équation de la fonction de distribution cumulée est la suivante :

$$F(x;\lambda) = 1 - e^{-\lambda x}$$

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Commentaires

Cette page a-t-elle été utile ? Yes No

GEOMEAN

Article • 14/12/2023

Retourne la moyenne géométrique des nombres dans une colonne.

Pour retourner la moyenne géométrique d'une expression évaluée pour chaque ligne d'une table, utilisez la fonction GEOMEANX.

Syntaxe

```
DAX

GEOMEAN(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne contenant les nombres pour lesquels la moyenne géométrique doit être calculée.

Valeur de retour

Nombre décimal.

Remarques

- Seuls les nombres de la colonne sont comptés. Les espaces, les valeurs logiques et le texte sont ignorés.
- GEOMEAN(Table[Colonne]) équivaut à GEOMEANX(Table, Table[Colonne])
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L’exemple suivant calcule la moyenne géométrique de la colonne Return dans la table Investment :

```
DAX
= GEOMEAN( Investment[Return] )
```

Contenu connexe

GEOMEANX, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

GEOMEANX

Article • 14/12/2023

Retourne la moyenne géométrique d'une expression évaluée pour chaque ligne d'une table.

Pour retourner la moyenne géométrique des nombres d'une colonne, utilisez la fonction GEOMEAN.

Syntaxe

```
DAX

GEOMEANX(<table>, <expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table contenant les lignes pour lesquelles l'expression est évaluée.
expression	Expression à évaluer pour chaque ligne de la table.

Valeur renvoyée

Nombre décimal.

Remarques

- La fonction GEOMEANX prend une table comme premier argument ou une expression qui retourne une table. Le deuxième argument est une colonne qui contient les nombres dont vous voulez calculer la moyenne géométrique ou une expression qui correspond à une colonne.
- Seuls les nombres de la colonne sont comptés. Les valeurs vides, les valeurs logiques et le texte sont ignorés.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L’exemple suivant calcule la moyenne géométrique de la colonne ReturnPct dans la table Investment :

DAX

```
= GEOMEANX( Investments, Investments[ReturnPct] + 1 )
```

Contenu connexe

GEOMEAN, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

LINEST

Article • 20/10/2023

Utilise la méthode des moindres carrés pour calculer une ligne droite qui correspond le mieux aux données spécifiées, puis retourne une table décrivant la ligne. L'équation de la ligne est sous la forme : y = **Slope1***x1 + **Slope2***x2 + ... + **Intercept**.

Syntaxe

```
DAX

LINEST ( <columnY>, <columnX>[, …][, <const>] )
```

Paramètres

Agrandir le tableau

Terme	Définition
colonneY	Colonne des valeurs y connues. Doit avoir un type scalaire.
colonneX	Colonnes des valeurs x connues. Doit avoir un type scalaire. Au moins une doit être fournie.
const	(Facultatif) Valeur TRUE/FALSE constante spécifiant s'il faut forcer la constante **Intercept** pour qu'elle soit égale à 0. Si la valeur EST TRUE ou omise, la valeur **Intercept** est calculée normalement ; si la valeur est FALSE, la valeur **Intercept** est définie sur zéro.

Valeur retournée

Table monoligne décrivant la ligne, plus des statistiques supplémentaires. Voici les colonnes disponibles :

- **Slope1**, **Slope2**, ..., **SlopeN** : coefficients correspondant à chaque valeur x ;
- **Intercept** : valeur intercept ;
- **StandardErrorSlope1**, **StandardErrorSlope2**, ..., **StandardErrorSlopeN** : valeurs d'erreur standard pour les coefficients **Slope1**, **Slope2**, ..., **SlopeN** ;
- **StandardErrorIntercept** : valeur d'erreur standard pour la constante **Intercept** ;
- **CoefficientOfDetermination** : coefficient de détermination (r^2). Compare les valeurs y estimées et réelles, et pointe vers les valeurs de 0 à 1 : plus la valeur est élevée, plus la corrélation dans l'échantillon est élevée ;
- **StandardError** : erreur standard pour l'estimation y ;
- **FStatistic** : statistique F, ou valeur observée par F. Utilisez la statistique F pour déterminer si la relation observée entre les variables dépendantes et indépendantes se produit par hasard ;
- **DegreesOfFreedom** : degrés de liberté. Utilisez cette valeur pour vous aider à trouver des valeurs F critiques dans une table statistique et à déterminer un niveau de confiance pour le modèle ;
- **RegressionSumOfSquares** : somme de régression des carrés ;
- **ResidualSumOfSquares** : somme résiduelle des carrés.

Notes

La <colonneY> et les <colonnesX> doivent toutes appartenir à la même table.

Exemple 1

La requête DAX suivante :

```
DAX

EVALUATE LINEST(
    'FactInternetSales'[SalesAmount],
    'FactInternetSales'[TotalProductCost]
)
```

Retourne une table monoligne avec dix colonnes :

Agrandir le tableau

Slope1	Intercepter	StandardErrorSlope1	StandardErrorIntercept	CoefficientOfDetermination
1.67703250456677	6.34550460373026	0.000448675725548806	0.279131821917317	0.995695557281456

Agrandir le tableau

StandardError	FStatistic	DegreesOfFreedom	RegressionSumOfSquares	ResidualSumOfSquares
60.9171030357485	13970688.6139993	60396	51843736761.658	224123120.339218

- **Slope1** et **Intercept** : coefficients du modèle linéaire calculé ;
- **StandardErrorSlope1** et **StandardErrorIntercept** : valeurs d'erreur standard pour les coefficients ci-dessus ;
- **CoefficientOfDetermination**, **StandardError**, **FStatistic**, **DegreesOfFreedom**, **RegressionSumOfSquares** et **ResidualSumOfSquares** : statistiques de régression sur le modèle.

Pour une vente Internet donnée, ce modèle prédit le montant de la vente selon la formule suivante :

```
SalesAmount = Slope1 * TotalProductCost + Intercept
```

Exemple 2

La requête DAX suivante :

```
DAX

EVALUATE LINEST(
    'DimCustomer'[TotalSalesAmount],
    'DimCustomer'[YearlyIncome],
    'DimCustomer'[TotalChildren],
```

```
    'DimCustomer'[BirthDate]
)
```

Retourne une table monoligne avec quatorze colonnes :

- Slope1
- Slope2
- Slope3
- Intercepter
- StandardErrorSlope1
- StandardErrorSlope2
- StandardErrorSlope3
- StandardErrorIntercept
- CoefficientOfDetermination
- StandardError
- FStatistic
- DegreesOfFreedom
- RegressionSumOfSquares
- ResidualSumOfSquares

Pour un client donné, ce modèle prédit le total des ventes selon la formule suivante (la date de naissance est automatiquement convertie en nombre) :

```
TotalSalesAmount = Slope1 * YearlyIncome + Slope2 * TotalChildren + Slope3 * BirthDate +
Intercept
```

Contenu connexe

LINESTX
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

LINESTX

Article • 20/10/2023

Utilise la méthode des moindres carrés pour calculer une ligne droite qui correspond le mieux aux données spécifiées, puis retourne une table décrivant la ligne. Les données sont le résultat des expressions évaluées pour chaque ligne d'une table. L'équation de la ligne est sous la forme : y = **Slope1***x1 + **Slope2***x2 + ... + **Intercept**.

Syntaxe

DAX

```
LINESTX ( <table>, <expressionY>, <expressionX>[, …][, <const>] )
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table contenant les lignes pour lesquelles les expressions seront évaluées.
expressionY	Expression à évaluer pour chaque ligne de la table, afin d'obtenir les valeurs y connues. Doit avoir un type scalaire.
expressionY	Expressions à évaluer pour chaque ligne de la table, afin d'obtenir les valeurs x connues. Doit avoir un type scalaire. Au moins une doit être fournie.
const	(Facultatif) Valeur TRUE/FALSE constante spécifiant s'il faut forcer la constante **Intercept** pour qu'elle soit égale à 0. Si la valeur EST TRUE ou omise, la valeur **Intercept** est calculée normalement ; si la valeur est FALSE, la valeur **Intercept** est définie sur zéro.

Valeur retournée

Table monoligne décrivant la ligne, plus des statistiques supplémentaires. Voici les colonnes disponibles :

- **Slope1**, **Slope2**, ..., **SlopeN** : coefficients correspondant à chaque valeur x ;
- **Intercept** : valeur intercept ;
- **StandardErrorSlope1**, **StandardErrorSlope2**, ..., **StandardErrorSlopeN** : valeurs d'erreur standard pour les coefficients **Slope1**, **Slope2**, ..., **SlopeN** ;
- **StandardErrorIntercept** : valeur d'erreur standard pour la constante **Intercept** ;
- **CoefficientOfDetermination** : coefficient de détermination (r^2). Compare les valeurs y estimées et réelles, et pointe vers les valeurs de 0 à 1 : plus la valeur est élevée, plus la

corrélation dans l'échantillon est élevée ;

- **StandardError** : erreur standard pour l'estimation y ;
- **FStatistic** : statistique F, ou valeur observée par F. Utilisez la statistique F pour déterminer si la relation observée entre les variables dépendantes et indépendantes se produit par hasard ;
- **DegreesOfFreedom** : degrés de liberté. Utilisez cette valeur pour vous aider à trouver des valeurs F critiques dans une table statistique et à déterminer un niveau de confiance pour le modèle ;
- **RegressionSumOfSquares** : somme de régression des carrés ;
- **ResidualSumOfSquares** : somme résiduelle des carrés.

Exemple 1

La requête DAX suivante :

```
DAX

DEFINE VAR TotalSalesByRegion = SUMMARIZECOLUMNS(
    'Sales Territory'[Sales Territory Key],
    'Sales Territory'[Population],
    "Total Sales", SUM(Sales[Sales Amount])
)
EVALUATE LINESTX(
    'TotalSalesByRegion',
    [Total Sales],
    [Population]
)
```

Retourne une table monoligne avec dix colonnes :

Agrandir le tableau

Slope1	Intercepter	StandardErrorSlope1	StandardErrorIntercept	CoefficientOfDetermination
6.42271517588	-410592.76216	0.24959467764561	307826.343996223	0.973535860750193

Agrandir le tableau

StandardError	FStatistic	DegreesOfFreedom	RegressionSumOfSquares	ResidualSumOfSquares
630758.1747292	662.165707642	18	263446517001130	7161405749781.07

- **Slope1** et **Intercept** : coefficients du modèle linéaire calculé ;
- **StandardErrorSlope1** et **StandardErrorIntercept** : valeurs d'erreur standard pour les coefficients ci-dessus ;
- **CoefficientOfDetermination**, **StandardError**, **FStatistic**, **DegreesOfFreedom**, **RegressionSumOfSquares** et **ResidualSumOfSquares** : statistiques de régression sur le modèle.

Pour un territoire de vente donné, ce modèle prédit le total des ventes selon la formule suivante :

```
Total Sales = Slope1 * Population + Intercept
```

Exemple 2

La requête DAX suivante :

DAX

```
DEFINE VAR TotalSalesByCustomer = SUMMARIZECOLUMNS(
    'Customer'[Customer ID],
    'Customer'[Age],
    'Customer'[NumOfChildren],
    "Total Sales", SUM(Sales[Sales Amount])
)
EVALUATE LINESTX(
    'TotalSalesByCustomer',
    [Total Sales],
    [Age],
    [NumOfChildren]
)
```

Retourne une table monoligne avec douze colonnes :

Agrandir le tableau

Slope1	Slope2	Intercepter	StandardErrorSlope1
69.0435458093763	33.005949841721	-871.118539339539	0.872588875481658

Agrandir le tableau

StandardErrorSlope2	StandardErrorIntercept	CoefficientOfDetermination	StandardError
6.21158863903435	26.726292527427	0.984892920482022	68.5715034014342

Agrandir le tableau

FStatistic	DegreesOfFreedom	RegressionSumOfSquares	ResidualSumOfSquares
3161.91535144391	97	29734974.9782379	456098.954637092

Pour un client donné, ce modèle prédit le total des ventes selon la formule suivante :

```
```

```
Total Sales = Slope1 * Age + Slope2 * NumOfChildren + Intercept
```

Contenu connexe

LINEST
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

MEDIAN

Article • 20/10/2023

Retourne la valeur médiane des nombres d'une colonne.

Pour retourner la valeur médiane d'une expression évaluée pour chaque ligne d'une table, utilisez la fonction MEDIANX.

Syntaxe

DAX

```
MEDIAN(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne contenant les nombres pour lesquels la valeur médiane doit être calculée.

Valeur de retour

Nombre décimal.

Remarques

- Seuls les nombres de la colonne sont comptés. Les espaces sont ignorés. Les valeurs logiques, les dates et le texte ne sont pas pris en charge.
- MEDIAN(Table[Column]) équivaut à MEDIANX(Table, Table[Column]).
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L’exemple suivant calcule la valeur médiane d’une colonne nommée Age dans une table nommée Customers :

```
DAX
= MEDIAN( Customers[Age] )
```

Contenu connexe

MEDIANX, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

MEDIANX

Article • 14/12/2023

Retourne le nombre médian d'une expression évaluée pour chaque ligne d'une table.

Pour retourner la valeur médiane des nombres d'une colonne, utilisez la fonction MEDIAN.

Syntaxe

```
DAX

MEDIANX(<table>, <expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table contenant les lignes pour lesquelles l'expression est évaluée.
expression	Expression à évaluer pour chaque ligne de la table.

Valeur renvoyée

Nombre décimal.

Remarques

- La fonction MEDIANX prend une table comme premier argument ou une expression qui retourne une table. Le deuxième argument est une colonne qui contient les nombres dont vous voulez calculer la valeur médiane ou une expression qui est évaluée à une colonne.
- Seuls les nombres de la colonne sont comptés.
- Les valeurs logiques et le texte sont ignorés.
- MEDIANX n'ignore pas les valeurs vides, contrairement à MEDIAN.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant calcule l'âge médian des clients qui vivent aux États-Unis.

DAX

```
= MEDIANX( FILTER(Customers, RELATED( Geography[Country]="USA" ) ),
Customers[Age] )
```

Contenu connexe

MEDIAN, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

NORM.DIST

Article • 14/12/2023

Retourne la distribution normale pour la moyenne et l'écart type spécifiés.

Syntaxe

DAX

```
NORM.DIST(X, Mean, Standard_dev, Cumulative)
```

Paramètres

Agrandir le tableau

Terme	Définition
X	Valeur pour laquelle vous voulez la distribution.
Moyenne	Moyenne arithmétique de la distribution.
Standard_dev	Écart type de la distribution.
Cumulative*	Valeur logique qui détermine la forme de la fonction. Si la valeur cumulative est TRUE, la fonction NORM.DIST retourne la fonction de distribution cumulative. Si la valeur est FALSE, elle retourne la fonction de probabilité de densité.

Valeur renvoyée

Distribution normale pour la moyenne et l'écart type spécifiés.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

DAX

```
EVALUATE { NORM.DIST(42, 40, 1.5, TRUE) }
```

retourne :

Agrandir le tableau

[Value]
0.908788780274132

Contenu connexe

NORM.S.DIST, fonction
NORM.INV, fonction
NORM.S.INV

Commentaires

Cette page a-t-elle été utile ? Yes No

NORM.INV

Article • 14/12/2023

Inverse de la distribution cumulative normale pour la moyenne et l'écart type spécifiés.

Syntaxe

```DAX
NORM.INV(Probability, Mean, Standard_dev)
```

Paramètres

Agrandir le tableau

Terme	Définition
Probabilité	Probabilité correspondant à la distribution normale.
Moyenne	Moyenne arithmétique de la distribution.
Standard_dev	Écart type de la distribution.

Valeur de retour

Retourne l'inverse de la distribution cumulative normale pour la moyenne et l'écart type spécifiés.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

```DAX
EVALUATE { NORM.INV(0.908789, 40, 1.5) }
```

retourne :

Agrandir le tableau

[Value]
42.00000200956628780274132

Contenu connexe

NORM.S.INV
NORM.S.DIST, fonction
NORM.DIST, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

NORM.S.DIST

Article • 14/12/2023

Retourne la distribution normale standard (avec une moyenne égale à zéro et un écart type égal à un).

Syntaxe

DAX

```
NORM.S.DIST(Z, Cumulative)
```

Paramètres

Agrandir le tableau

Terme	Définition
Z	Valeur pour laquelle vous voulez la distribution.
Cumulative	La valeur cumulative est une valeur logique qui détermine la forme de la fonction. Si la valeur cumulative est TRUE, NORM.S.DIST retourne la fonction de distribution cumulative ; si la valeur est FALSE, il retourne la fonction de probabilité de densité.

Valeur renvoyée

Loi normale standard (a une moyenne égale à zéro et un écart type égal à un).

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

DAX

```
EVALUATE { NORM.S.DIST(1.333333, TRUE) }
```

retourne :

Agrandir le tableau

[Value]
0,908788725604095

Contenu connexe

NORM.INV, fonction
NORM.DIST, fonction
NORM.S.INV

Commentaires

Cette page a-t-elle été utile ? Yes No

NORM.S.INV

Article • 14/12/2023

Retourne l'inverse de la distribution cumulative normale standard. La distribution a une moyenne égale à zéro et un écart type égal à un.

Syntaxe

```
DAX

NORM.S.INV(Probability)
```

Paramètres

Agrandir le tableau

Terme	Définition
Probabilité	Probabilité correspondant à la distribution normale.

Valeur renvoyée

Inverse de la distribution cumulative normale standard. La distribution a une moyenne égale à zéro et un écart type égal à un.

Notes

Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

```
DAX

EVALUATE { NORM.S.INV(0.908789) }
```

retourne :

Agrandir le tableau

[Value]
1.33333467304411

Contenu connexe

NORM.INV
NORM.S.DIST, fonction
NORM.DIST, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

PERCENTILE.EXC

Article • 14/12/2023

Retourne le k-ième centile des valeurs d'une plage, k figurant dans la plage 0-1, limites exclues.

Pour retourner le centile d'une expression évaluée pour chaque ligne d'une table, utilisez la fonction PERCENTILEX.EXC.

Syntaxe

```
DAX

PERCENTILE.EXC(<column>, <k>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne contenant les valeurs qui définissent la position relative.
k	Valeur du centile dans la plage 0-1, limites exclues.

Valeur de retour

K-ième centile des valeurs d'une plage, k figurant dans la plage 0-1, limites exclues.

Remarques

- Si le paramètre column est vide, la valeur BLANK() est retournée.
- Si k est égal à zéro ou est une valeur vide, le classement centile 1/(n+1) retourne la plus petite valeur. Si la valeur est zéro, elle est hors limites et une erreur est retournée.
- Si k n'est pas une valeur numérique ou n'est pas compris dans la plage 0-1, une erreur est retournée.

- Si k n’est pas un multiple de 1/(n + 1), la fonction PERCENTILE.EXC effectue une interpolation pour déterminer la valeur au k-ième centile.
- PERCENTILE.EXC effectue une interpolation quand la valeur du centile spécifié est comprise entre deux valeurs dans le tableau. Si elle ne peut pas effectuer d’interpolation pour le centile k spécifié, une erreur est retournée.
- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Contenu connexe

PERCENTILEX.EXC

Commentaires

Cette page a-t-elle été utile ? Yes No

PERCENTILE.INC

Article • 20/10/2023

Retourne le énième centile k des valeurs d'une plage, où k se trouve dans la plage de 0 à 1, inclus.

Pour retourner le centile d'une expression évaluée pour chaque ligne d'une table, utilisez la fonction PERCENTILEX.INC.

Syntaxe

```
DAX

PERCENTILE.INC(<column>, <k>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne contenant les valeurs qui définissent la position relative.
k	Valeur du centile dans la plage de 0 à 1, inclus.

Valeur de retour

Énième centile k des valeurs d'une plage, où k se trouve dans la plage de 0 à 1, inclus.

Remarques

- Si le paramètre column est vide, la valeur BLANK() est retournée.
- Si k est égal à zéro ou est une valeur vide, le classement centile 1/(n+1) retourne la plus petite valeur. Si la valeur est zéro, elle est hors limites et une erreur est retournée.
- Si k n'est pas une valeur numérique ou n'est pas compris dans la plage 0-1, une erreur est retournée.

- Si k n'est pas un multiple de 1/(n + 1), la fonction PERCENTILE.INC effectue une interpolation pour déterminer la valeur au énième centile k.
- PERCENTILE.INC effectue une interpolation quand la valeur du centile spécifié est comprise entre deux valeurs dans le tableau. Si elle ne peut pas effectuer d'interpolation pour le centile k spécifié, une erreur est retournée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Contenu connexe

PERCENTILEX.INC

Commentaires

Cette page a-t-elle été utile ? Yes No

PERCENTILEX.EXC

Article • 20/10/2023

Retourne le centile d'une expression évaluée pour chaque ligne d'une table.

Pour retourner le centile des nombres d'une colonne, utilisez la fonction PERCENTILE.EXC.

Syntaxe

DAX

```
PERCENTILEX.EXC(<table>, <expression>, k)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table contenant les lignes pour lesquelles l'expression est évaluée.
expression	Expression à évaluer pour chaque ligne de la table.
k	Valeur de centile souhaitée dans la plage comprise entre 0 et 1 (exclus).

Valeur de retour

Centile d'une expression évaluée pour chaque ligne d'une table.

Remarques

- Si k est égal à zéro ou est une valeur vide, le classement centile 1/(n+1) retourne la plus petite valeur. Si la valeur est zéro, elle est hors limites et une erreur est retournée.
- Si k n'est pas une valeur numérique ou n'est pas compris dans la plage 0-1, une erreur est retournée.

- Si k n'est pas un multiple de 1/(n + 1), PERCENTILEX.EXC effectue une interpolation pour déterminer la valeur au k-ième centile.
- PERCENTILEX.EXC effectue une interpolation quand la valeur du centile spécifié est comprise entre deux valeurs dans le tableau. Si elle ne peut pas effectuer d'interpolation pour le centile k spécifié, une erreur est retournée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Contenu connexe

PERCENTILE.EXC

Commentaires

Cette page a-t-elle été utile ? Yes No

PERCENTILEX.INC

Article • 20/10/2023

Retourne le centile d'une expression évaluée pour chaque ligne d'une table.

Pour retourner le centile des nombres d'une colonne, utilisez PERCENTILE.INC.

Syntaxe

DAX

```
PERCENTILEX.INC(<table>, <expression>;, k)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table contenant les lignes pour lesquelles l'expression est évaluée.
expression	Expression à évaluer pour chaque ligne de la table.
k	Valeur de centile souhaitée dans la plage comprise entre 0 et 1 (inclus).

Valeur de retour

Centile d'une expression évaluée pour chaque ligne d'une table.

Remarques

- Si k est égal à zéro ou est une valeur vide, le classement centile 1/(n-1) retourne la plus petite valeur. Si la valeur est zéro, elle est hors limites et une erreur est retournée.
- Si k n'est pas numérique ou n'est pas compris dans la plage 0-1, une erreur est retournée.
- Si k n'est pas un multiple de 1/(n - 1), PERCENTILEX.EXC effectue une interpolation pour déterminer la valeur au énième centile k.

- PERCENTILEX.INC effectue une interpolation quand la valeur du centile spécifié est comprise entre deux valeurs dans le tableau. S'il ne peut pas effectuer d'interpolation pour le centile k spécifié, une erreur est retournée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Contenu connexe

PERCENTILE.INC

Commentaires

Cette page a-t-elle été utile ? Yes No

PERMUT

Article • 20/10/2023

Retourne le nombre de permutations d'un nombre donné d'objets qui peuvent être choisis parmi des objets de nombre. Une permutation est un ensemble ou un sous-ensemble d'objets ou d'événements dont l'ordre interne est important. Il faut distinguer les permutations des combinaisons, pour lesquelles l'ordre interne n'est pas important. Utilisez cette fonction pour les calculs de probabilité de type loterie.

Syntaxe

DAX

```
PERMUT(number, number_chosen)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Obligatoire. Entier qui indique le nombre d'objets.
number_chosen	Obligatoire. Entier qui indique le nombre d'objets dans chaque permutation.

Valeur de retour

Retourne le nombre de permutations d'un nombre donné d'objets qui peuvent être choisis parmi des objets de nombre.

Remarques

- Les deux arguments sont tronqués en entiers.
- Si le paramètre number ou number_chosen n'est pas numérique, la fonction PERMUT retourne la valeur d'erreur #VALUE! #NUM!.
- Si number ≤ 0 ou si number_chosen < 0, la fonction PERMUT retourne la valeur d'erreur #NUM! #NUM!.

- Si number < number_chosen, la fonction PERMUT retourne la valeur d'erreur #NUM! #NUM!.
- L'équation pour le nombre de permutations est la suivante :

$$P_{k,n} = \frac{n!}{(n-k)!}$$

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La formule suivante montre les permutations possibles pour un groupe de 3 objets où 2 sont choisis :

```
DAX
= PERMUT(3,2)
```

Result,

6

Commentaires

Cette page a-t-elle été utile ? Yes No

POISSON.DIST

Article • 20/10/2023

Retourne la distribution de Poisson. Une application courante de la distribution de Poisson est la prévision du nombre d'événements sur une période de temps spécifique, comme le nombre de voitures arrivant à une gare de péage sur 1 minute.

Syntaxe

DAX

```
POISSON.DIST(x,mean,cumulative)
```

Paramètres

Agrandir le tableau

Terme	Définition
x	Obligatoire. Nombre d'événements.
mean	Obligatoire. Valeur numérique attendue.
cumulative	Obligatoire. Valeur logique qui détermine la forme de la distribution de probabilité retournée. Si cumulative a la valeur TRUE, POISSON.DIST retourne la probabilité de Poisson cumulative que le nombre d'événements aléatoires qui se produisent sera compris entre zéro et x, inclus. Si sa valeur est FALSE, elle retourne la fonction de masse de la probabilité de Poisson que le nombre d'événements qui se produisent sera exactement x.

Valeur de retour

Retourne la distribution de Poisson.

Remarques

- Si x n'est pas un entier, il est arrondi.
- Si x ou mean n'est pas numérique, POISSON.DIST retourne la valeur d'erreur #VALUE!.

- Si x < 0, POISSON.DIST retourne la valeur d'erreur #NUM!.
- Si mean < 0, POISSON.DIST retourne la #NUM!.
- La formule POISSON.DIST est calculée comme suit.
  - Pour cumulative = FALSE :

$$\text{POISSON} = \frac{e^{-\lambda}\lambda^{x}}{x!}$$

  - Pour cumulative = TRUE :

$$\text{CUMPOISSON} = \sum_{k=0}^{x} \frac{e^{-\lambda}\lambda^{x}}{k!}$$

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Commentaires

Cette page a-t-elle été utile ? Yes No

RANK.EQ

Article • 20/10/2023

Retourne le classement d'un nombre dans une liste de nombres.

Syntaxe

DAX

```
RANK.EQ(<value>, <columnName>[, <order>])
```

Paramètres

Agrandir le tableau

Terme	Définition
valeur	Toute expression DAX qui retourne une valeur scalaire unique dont le classement doit être recherché. L'expression doit être évaluée une seule fois avant l'évaluation de la fonction, et sa valeur doit être passée à la liste d'arguments.
columnName	Nom d'une colonne existante par rapport à laquelle les rangs seront déterminés. Il ne peut pas s'agir d'une expression ou d'une colonne créée à l'aide des fonctions suivantes : ADDCOLUMNS, ROW ou SUMMARIZE.
order	(Facultatif) Valeur qui spécifie comment classer le paramètre *value* (ordre croissant ou décroissant) :

valeurs d'ordre

Agrandir le tableau

valeur	Valeur alternative	Description
0 (zéro)	FALSE	Le classement est effectué dans l'ordre décroissant de *columnName*. Si le paramètre *value* est égal au nombre le plus élevé dans *columnName*, alors **RANK.EQ** retourne 1.
1	TRUE	Le classement est effectué dans l'ordre croissant de *columnName*. Si le paramètre *value* est égal au nombre le plus faible dans *columnName*, alors **RANK.EQ** retourne 1.

Valeur renvoyée

Nombre indiquant le rang du paramètre *value* parmi les nombres de *columnName*.

Notes

- *columnName* ne peut pas faire référence à une colonne créée à l'aide des fonctions ADDCOLUMNS, ROW ou SUMMARIZE.I
- Si le paramètre *value* ne se trouve pas dans *columnName* ou s'il est vide, *RANK.EQ* retourne une valeur vide.
- Si le paramètre *value* contient des valeurs en double, le même rang leur est affecté. Le rang affecté suivant correspond au rang plus le nombre de valeurs en double. Par exemple, si cinq (5) valeurs sont classées à égalité au rang 11, la valeur suivante sera classée au rang 16 (11 + 5).
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple 1

L'exemple suivant crée une colonne calculée qui classe les valeurs de la colonne SalesAmount_USD (de la table *InternetSales_USD*) par rapport à l'ensemble des nombres de la même colonne.

DAX

```
= RANK.EQ(InternetSales_USD[SalesAmount_USD],
InternetSales_USD[SalesAmount_USD])
```

Exemple 2

L'exemple suivant classe un sous-ensemble de valeurs par rapport à un échantillon donné. Supposons que vous disposiez d'une table contenant des étudiants locaux avec leurs performances à un test national spécifique et de l'ensemble des notes obtenues à ce test. La colonne calculée suivante vous donne le classement national de chacun des étudiants locaux.

DAX

```
= RANK.EQ(Students[Test_Score], NationalScores[Test_Score])
```

Commentaires

Cette page a-t-elle été utile ? Yes No

RANKX

Article • 20/10/2023

Retourne le classement d'un nombre dans une liste de nombres pour chaque ligne de l'argument *table*.

Syntaxe

```DAX
RANKX(<table>, <expression>[, <value>[, <order>[, <ties>]]])
```

Paramètres

table
Toute expression DAX qui retourne une table de données sur laquelle l'expression est évaluée.

expression
Toute expression DAX qui retourne une valeur scalaire unique. L'expression est évaluée pour chaque ligne de l'argument *table* de sorte que toutes les valeurs pouvant être classées soient générées. Consultez la section Remarques pour comprendre le comportement de la fonction quand l'*expression* prend une valeur vide.

value
(Facultatif) Toute expression DAX qui retourne une valeur scalaire unique dont le classement doit être recherché. Consultez la section Remarques pour comprendre le comportement de la fonction quand le paramètre *value* n'est pas trouvé dans l'expression.

Quand le paramètre *value* est omis, la valeur de l'expression à la ligne actuelle est utilisée à la place.

order
(Facultatif) Valeur qui spécifie comment classer le paramètre *value* (ordre croissant ou décroissant) :

Agrandir le tableau

valeur	Valeur alternative	Description
0 (zéro)	FAUX	Le classement s'effectue dans l'ordre décroissant des valeurs de l'expression. Si la valeur est égale au nombre le plus élevé dans l'expression, RANKX retourne 1. Il s'agit de la valeur par défaut quand le paramètre order est omis.
1	VRAI	Le classement s'effectue dans l'ordre croissant des valeurs de l'expression. Si la valeur est égale au nombre le plus faible dans l'expression, RANKX retourne 1.

ties
(Facultatif) Énumération qui définit comment déterminer le classement en présence d'égalités.

Agrandir le tableau

énumération	Description
Ignorer	Le rang qui suit une égalité correspond au rang de l'égalité plus le nombre de valeurs incluses dans l'égalité. Par exemple, si cinq (5) valeurs sont classées à égalité au rang 11, la valeur suivante sera classée au rang 16 (11 + 5). Il s'agit de la valeur par défaut quand le paramètre *ties* est omis.
Dense	Le rang qui suit une égalité est le rang suivant. Par exemple, si cinq (5) valeurs sont classées à égalité au rang 11, la valeur suivante sera classée au rang 12.

Valeur de retour

Rang du paramètre *value* parmi toutes les valeurs possibles de l'*expression* évaluées pour toutes les lignes de nombres de la *table*.

Notes

- Si l'*expression* ou le paramètre *value* prend une valeur vide, il est traité comme un 0 (zéro) pour toutes les expressions qui génèrent un nombre ou comme du texte vide pour toutes les expressions de texte.
- Si le paramètre *value* ne fait pas partie des valeurs possibles de l'*expression*, RANKX l'ajoute temporairementaux valeurs de l'*expression* et effectue une réévaluation pour déterminer son rangcorrect.

- Vous pouvez ignorer les arguments facultatifs en plaçant une virgule (,) dans la liste d'arguments, par exemple RANKX(Inventory, [InventoryCost],,,"Dense")
- Les comparaisons liées à l'égalité (=, <>, >= et <=) entre des valeurs avec le type de données *Nombre décimal* peuvent retourner des résultats inattendus lors de l'utilisation de la fonction RANKX. Des résultats incorrects peuvent se produire, car les valeurs avec le type de données *Nombre décimal* sont stockées en tant que nombres à virgule flottante conformément à la norme IEEE 754 et présentent des limitations inhérentes de précision. Pour éviter d'obtenir des résultats inattendus, modifiez le type de données en *Nombre décimal fixe* ou effectuez un arrondi forcé à l'aide de ROUND.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La colonne calculée suivante dans la table Products calcule le classement des ventes Internet de chaque produit.

DAX

```
= RANKX(ALL(Products), SUMX(RELATEDTABLE(InternetSales), [SalesAmount]))
```

Commentaires

Cette page a-t-elle été utile ? Yes No

SAMPLE

Article • 20/10/2023

Retourne un échantillon de n lignes de la table spécifiée.

Syntaxe

DAX

```
SAMPLE(<n_value>, <table>, <orderBy_expression>, [<order>[,
<orderBy_expression>, [<order>]]…])
```

Paramètres

Agrandir le tableau

Terme	Définition
n_value	Nombre de lignes à retourner. Il s'agit de toute expression DAX qui retourne une valeur scalaire unique quand l'expression doit être évaluée plusieurs fois (pour chaque ligne/contexte). Si une valeur (ou expression) non entière est entrée, le résultat est converti en entier.
table	Toute expression DAX qui retourne une table de données à partir de laquelle extraire les « n » lignes d'échantillon.
orderBy_expression	Toute expression DAX scalaire où la valeur obtenue est évaluée pour chaque ligne de la *table*.
order	(Facultatif) Valeur qui spécifie comment trier les valeurs *orderBy_expression*, par ordre croissant ou décroissant : 0 (zéro), trie par ordre décroissant les valeurs *order_by*. 1, classe dans l'ordre croissant de *order_by*.

Valeur renvoyée

Une table contenant les n lignes d'échantillon de *table*, ou une table vide si la valeur *n_value* est inférieure ou égale à 0 (zéro). Si des arguments OrderBy sont fournis, l'exemple est stable et déterministe ; il retourne la première ligne, la dernière ligne et les lignes uniformément réparties entre elles. Si aucun classement n'est spécifié, l'échantillon est aléatoire, non stable et non déterministe.

Notes

- Si n_value est inférieur ou égal à 0 (zéro), SAMPLE retourne une table vide.
- Afin d'empêcher la duplication de valeurs dans l'échantillon, la table fournie en deuxième argument doit être groupée en fonction de la colonne utilisée pour le tri.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Commentaires

Cette page a-t-elle été utile ? Yes No

STDEV.S

Article • 20/10/2023

Retourne l'écart type d'un échantillon de population.

Syntaxe

DAX

```
STDEV.S(<ColumnName>)
```

Paramètres

Agrandir le tableau

Terme	Définition
columnName	Nom, généralement complet, d'une colonne existante utilisant la syntaxe DAX standard. Il ne peut pas s'agir d'une expression.

Valeur de retour

Nombre qui représente l'écart type d'un échantillon de population.

Exceptions

Remarques

- STDEV.S part du principe que la colonne fait référence à un échantillon de la population. Si vos données représentent l'ensemble de la population, calculez l'écart type à l'aide de STDEV.P.
- STDEV.S utilise la formule suivante :

  $\sqrt{[\sum(x - \tilde{x})^2/(n-1)]}$

  où $\tilde{x}$ représente la valeur moyenne de x pour l'échantillon de la population et n représente la taille de la population.

- Les lignes vides sont exclues par filtrage de *columnName* et ne sont pas prises en compte dans les calculs.
- Une erreur est retournée si *columnName* contient moins de 2 lignes non vides.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant illustre la formule pour une mesure qui calcule l'écart type de la colonne SalesAmount_USD, alors que la table InternetSales_USD correspond à l'échantillon de population.

DAX

```
= STDEV.S(InternetSales_USD[SalesAmount_USD])
```

Commentaires

Cette page a-t-elle été utile ? Yes No

STDEV.P,

Article • 20/10/2023

Retourne l'écart type de l'ensemble de la population.

Syntaxe

```
DAX

STDEV.P(<ColumnName>)
```

Paramètres

Agrandir le tableau

Terme	Définition
columnName	Nom, généralement complet, d'une colonne existante utilisant la syntaxe DAX standard. Il ne peut pas s'agir d'une expression.

Valeur de retour

Nombre représentant l'écart type de l'ensemble de la population.

Notes

- STDEV.P considère que la colonne fait référence à l'ensemble du remplissage. Si vos données représentent un échantillon de la population, calculez l'écart type à l'aide de STDEV.S.
- STDEV.P utilise la formule suivante :

  $\sqrt{[\sum(x - \tilde{x})^2/n]}$

  où $\tilde{x}$ représente la valeur moyenne de x pour l'ensemble de la population et n la taille de la population.
- Les lignes vides sont exclues par filtrage de *columnName* et ne sont pas prises en compte dans les calculs.

- Une erreur est retournée si *columnName* contient moins de 2 lignes non vides.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant illustre la formule d'une mesure qui calcule l'écart type de la colonne SalesAmount_USD alors que la table InternetSales_USD correspond à l'ensemble de la population.

DAX

```
= STDEV.P(InternetSales_USD[SalesAmount_USD])
```

Commentaires

Cette page a-t-elle été utile ? Yes No

STDEVX.S

Article • 20/10/2023

Retourne l'écart type d'un échantillon de population.

Syntaxe

DAX

```
STDEVX.S(<table>, <expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Toute expression DAX qui retourne une valeur scalaire unique où l'expression doit être évaluée plusieurs fois (pour chaque ligne/contexte).
expression	Toute expression DAX qui retourne une valeur scalaire unique où l'expression doit être évaluée plusieurs fois (pour chaque ligne/contexte).

Valeur de retour

Nombre avec l'écart type d'un échantillon de remplissage.

Exceptions

Remarques

- STDEVX.S évalue l'*expression* pour chaque ligne de la *table* et retourne l'écart type de l'*expression*, à supposer que la *table* fasse référence à un échantillon du remplissage. Si la *table* représente l'ensemble du remplissage, calculez l'écart type à l'aide de STDEVX.P.
- STDEVX.S utilise la formule suivante :

  $\sqrt{[\sum(x - \tilde{x})^2/(n-1)]}$

où $\tilde{x}$ représente la valeur moyenne de x pour l'ensemble de la population et n la taille de la population.

- Les lignes vides sont exclues par filtrage de *columnName* et ne sont pas prises en compte dans les calculs.
- Une erreur est retournée si *columnName* contient moins de 2 lignes non vides.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant illustre la formule d'une colonne calculée qui évalue l'écart type du prix unitaire par produit pour un échantillon de remplissage, quand la formule est utilisée dans la table Product.

DAX

```
= STDEVX.S(RELATEDTABLE(InternetSales_USD), InternetSales_USD[UnitPrice_USD]
- (InternetSales_USD[DiscountAmount_USD]/InternetSales_USD[OrderQuantity]))
```

Commentaires

Cette page a-t-elle été utile ? Yes No

STDEVX.P,

Article • 20/10/2023

Retourne l'écart type de l'ensemble de la population.

Syntaxe

DAX

```
STDEVX.P(<table>, <expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Toute expression DAX qui retourne une valeur scalaire unique où l'expression doit être évaluée plusieurs fois (pour chaque ligne/contexte).
expression	Toute expression DAX qui retourne une valeur scalaire unique où l'expression doit être évaluée plusieurs fois (pour chaque ligne/contexte).

Valeur de retour

Nombre qui représente l'écart type de l'ensemble de la population.

Remarques

- STDEVX.P évalue *expression* pour chaque ligne de *table* et retourne l'écart type de l'expression, à supposer que la table fasse référence à l'ensemble de la population. Si les données dans *table* représentent un échantillon de la population, vous devez plutôt calculer l'écart type à l'aide de STDEVX.S.
- STDEVX.P utilise la formule suivante :

  $\sqrt{[\sum(x - \tilde{x})^2/n]}$

  où $\tilde{x}$ représente la valeur moyenne de x pour l'ensemble de la population et n la taille de la population.

- Les lignes vides sont exclues par filtrage de *columnName* et ne sont pas prises en compte dans les calculs.
- Une erreur est retournée si *columnName* contient moins de 2 lignes non vides.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant illustre la formule d'une colonne calculée qui calcule l'écart type du prix unitaire par produit, quand la formule est utilisée dans la table *Product*.

DAX

```
= STDEVX.P(RELATEDTABLE(InternetSales_USD), InternetSales_USD[UnitPrice_USD]
- (InternetSales_USD[DiscountAmount_USD]/InternetSales_USD[OrderQuantity]))
```

Commentaires

Cette page a-t-elle été utile ? Yes No

T.DIST

Article • 14/12/2023

Retourne la distribution T unilatérale à gauche de Student.

Syntaxe

DAX

```
T.DIST(X,Deg_freedom,Cumulative)
```

Paramètres

Agrandir le tableau

Terme	Définition
X	Valeur numérique à laquelle évaluer la distribution.
Deg_freedom	Entier qui indique le nombre de degrés de liberté.
Cumulative	Valeur logique qui détermine la forme de la fonction. Si la valeur cumulative est TRUE, T.DIST retourne la fonction de distribution cumulative, si la valeur est FALSE, il retourne la fonction de probabilité de densité.

Valeur renvoyée

Loi unilatérale à gauche de Student.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

DAX

```
EVALUATE { T.DIST(60, 1, TRUE) }
```

Retourne :

Agrandir le tableau

[Value]
0.994695326367377

Contenu connexe

T.DIST.2T
T.DIST.RT
T.INV
T.INV.2t

Commentaires

Cette page a-t-elle été utile ? Yes No

T.DIST.2T

Article • 14/12/2023

Retourne la distribution T bilatérale de Student.

Syntaxe

```
DAX

T.DIST.2T(X,Deg_freedom)
```

Paramètres

Agrandir le tableau

Terme	Définition
X	Valeur numérique à laquelle évaluer la distribution.
Deg_freedom	Entier qui indique le nombre de degrés de liberté.

Valeur renvoyée

Distribution T bilatérale de Student.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

```
DAX

EVALUATE { T.DIST.2T(1.959999998, 60) }
```

retourne :

Agrandir le tableau

[Value]
0.054644929975921

Contenu connexe

T.DIST
T.DIST.RT
T.INV
T.INV.2t

Commentaires

Cette page a-t-elle été utile ? Yes No

T.DIST.RT

Article • 14/12/2023

Retourne la distribution T de Student unilatérale à droite.

Syntaxe

```
DAX

T.DIST.RT(X,Deg_freedom)
```

Paramètres

Agrandir le tableau

Terme	Définition
X	Valeur numérique à laquelle évaluer la distribution.
Deg_freedom	Entier qui indique le nombre de degrés de liberté.

Valeur renvoyée

Loi unilatérale à droite de Student.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

```
DAX

EVALUATE { T.DIST.RT(1.959999998, 60) }
```

retourne :

Agrandir le tableau

[Value]
0,0273224649879605

Contenu connexe

T.DIST
T.DIST.2T
T.INV
T.INV.2t

Commentaires

Cette page a-t-elle été utile ? Yes No

T.INV

Article • 14/12/2023

Retourne l'inverse unilatéral à gauche de la distribution T de Student.

Syntaxe

```
DAX

T.INV(Probability,Deg_freedom)
```

Paramètres

Agrandir le tableau

Terme	Définition
Probabilité	Probabilité associée à la distribution T de Student.
Deg_freedom	Nombre de degrés de liberté avec lequel caractériser la distribution.

Valeur de retour

Inverse unilatéral à gauche de la distribution T de Student.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

```
DAX

EVALUATE { T.INV(0.75, 2) }
```

retourne :

Agrandir le tableau

[Value]
0.816496580927726

Contenu connexe

T.INV.2T
T.DIST
T.DIST.2T
T.DIST.RT

Commentaires

Cette page a-t-elle été utile ? Yes No

T.INV.2T

Article • 14/12/2023

Retourne l'inverse bilatéral de la distribution T de Student.

Syntaxe

```
DAX

T.INV.2T(Probability,Deg_freedom)
```

Paramètres

Agrandir le tableau

Terme	Définition
Probabilité	Probabilité associée à la distribution T de Student.
Deg_freedom	Nombre de degrés de liberté avec lequel caractériser la distribution.

Valeur de retour

Inverse bilatéral de la distribution T de Student.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

```
DAX

EVALUATE { T.INV.2T(0.546449, 60) }
```

retourne :

Agrandir le tableau

[Value]
0.606533075825759

Contenu connexe

T.INV
T.DIST
T.DIST.2T
T.DIST.RT

Commentaires

Cette page a-t-elle été utile ? Yes No

VAR.S

Article • 20/10/2023

Retourne la variance d'un échantillon de population.

Syntaxe

```DAX
VAR.S(<columnName>)
```

Paramètres

Agrandir le tableau

Terme	Définition
columnName	Nom, généralement complet, d'une colonne existante utilisant la syntaxe DAX standard. Il ne peut pas s'agir d'une expression.

Valeur de retour

Nombre avec la variance d'un échantillon de population.

Notes

- VAR.S part du principe que la colonne fait référence à un échantillon de la population. Si vos données représentent l'ensemble de la population, calculez la variance à l'aide de VAR.P.
- VAR.S utilise la formule suivante :

  $\sum(x - \tilde{x})^2/(n-1)$

  $\tilde{x}$ représentant la valeur moyenne de x pour l'échantillon de population

  et n représente la taille de la population
- Les lignes vides sont exclues par filtrage de *columnName* et ne sont pas prises en compte dans les calculs.

- Une erreur est retournée si *columnName* contient moins de 2 lignes non vides.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant illustre la formule d'une mesure qui calcule la variance de la colonne SalesAmount_USD à partir de la table InternetSales_USD pour un échantillon de population.

DAX

```
= VAR.S(InternetSales_USD[SalesAmount_USD])
```

Commentaires

Cette page a-t-elle été utile ? Yes No

VAR.P,

Article • 20/10/2023

Retourne la variance de l'ensemble de la population.

Syntaxe

```DAX
VAR.P(<columnName>)
```

Paramètres

Agrandir le tableau

Terme	Définition
columnName	Nom, généralement complet, d'une colonne existante utilisant la syntaxe DAX standard. Il ne peut pas s'agir d'une expression.

Valeur de retour

Nombre avec la variance de l'ensemble de la population.

Remarques

- VAR.P considère que la colonne fait référence à l'ensemble du remplissage. Si vos données représentent un échantillon de la population, calculez la variance à l'aide de VAR.S.
- VAR.P utilise la formule suivante :

  $\sum(x - \tilde{x})^2/n$

  où $\tilde{x}$ représente la valeur moyenne de x pour l'ensemble de la population

  et n représente la taille de la population
- Les lignes vides sont exclues par filtrage de *columnName* et ne sont pas prises en compte dans les calculs.

- Une erreur est retournée si *columnName* contient moins de 2 lignes non vides.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant illustre la formule d'une mesure qui estime la variance de la colonne SalesAmount_USD à partir de la table InternetSales_USD pour l'ensemble de la population.

DAX

```
= VAR.P(InternetSales_USD[SalesAmount_USD])
```

Commentaires

Cette page a-t-elle été utile ? Yes No

VARX.S

Article • 20/10/2023

Retourne la variance d'un échantillon de population.

Syntaxe

DAX

```
VARX.S(<table>, <expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Toute expression DAX qui retourne une table de données.
expression	Toute expression DAX qui retourne une valeur scalaire unique où l'expression doit être évaluée plusieurs fois (pour chaque ligne/contexte).

Valeur de retour

Nombre qui représente la variance d'un échantillon de population.

Notes

- VARX.S évalue l'*expression* pour chaque ligne de la *table* et retourne la variance de l'*expression*, à supposer que la *table* fasse référence à un échantillon de population. Si la *table* représente la population entière, vous devez calculer la variance à l'aide de VARX.P.
- VAR.S utilise la formule suivante :

  $\sum(x - \tilde{x})^2/(n-1)$

  $\tilde{x}$ représentant la valeur moyenne de x pour l'échantillon de population

  et n représente la taille de la population

- Les lignes vides sont exclues par filtrage de *columnName* et ne sont pas prises en compte dans les calculs.
- Une erreur est retournée si *columnName* contient moins de 2 lignes non vides.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant illustre la formule d'une colonne calculée qui évalue la variance du prix unitaire par produit pour un échantillon de population quand la formule est utilisée dans la table Product.

DAX

```
= VARX.S(InternetSales_USD, InternetSales_USD[UnitPrice_USD] -
(InternetSales_USD[DiscountAmount_USD]/InternetSales_USD[OrderQuantity]))
```

Commentaires

Cette page a-t-elle été utile ? Yes No

VARX.P

Article • 20/10/2023

Retourne la variance de l'ensemble de la population.

Syntaxe

DAX

```
VARX.P(<table>, <expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Toute expression DAX qui retourne une table de données.
expression	Toute expression DAX qui retourne une valeur scalaire unique où l'expression doit être évaluée plusieurs fois (pour chaque ligne/contexte).

Valeur de retour

Nombre avec la variance de l'ensemble de la population.

Remarques

- VARX.P évalue <expression> pour chaque ligne de <table> et retourne l'écart de <expression> en partant de l'hypothèse que <table> fait référence à l'ensemble de la population. Si la <table> représentent un échantillon de la population, calculez la variance à l'aide de VARX.S.
- VARX.P utilise la formule suivante :

  $\sum(x - \tilde{x})^2/n$

  où $\tilde{x}$ représente la valeur moyenne de x pour l'ensemble de la population

  et n représente la taille de la population

- Les lignes vides sont exclues par filtrage de *columnName* et ne sont pas prises en compte dans les calculs.
- Une erreur est retournée si *columnName* contient moins de 2 lignes non vides.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant illustre la formule d'une colonne calculée qui calcule la variance du prix unitaire par produit quand la formule est utilisée dans la table Product.

DAX

```
= VARX.P(InternetSales_USD, InternetSales_USD[UnitPrice_USD] –
(InternetSales_USD[DiscountAmount_USD]/InternetSales_USD[OrderQuantity]))
```

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonctions de manipulation de tables

Article • 20/10/2023

Ces fonctions retournent une table ou manipulent des tables existantes.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
ADDCOLUMNS	Ajoute des colonnes calculées à la table ou à l'expression de table donnée.
ADDMISSINGITEMS	Ajoute à une table les combinaisons d'éléments de plusieurs colonnes si ces éléments n'existent pas déjà.
CROSSJOIN	Retourne une table contenant le produit cartésien de toutes les lignes de toutes les tables figurant dans les arguments.
CURRENTGROUP	Retourne un ensemble de lignes à partir de l'argument table d'une expression GROUPBY.
DATATABLE	Fournit un mécanisme permettant de déclarer un ensemble inline de valeurs de données.
DETAILROWS	Évalue une expression de lignes de détails définie pour une mesure et retourne les données.
DISTINCT (colonne)	Retourne une table à une colonne qui contient les valeurs distinctes de la colonne spécifiée.
DISTINCT (table)	Retourne une table en supprimant les lignes en double d'une autre table ou expression.
EXCEPT	Retourne les lignes d'une table qui n'apparaissent pas dans une autre table.
FILTERS	Retourne une table de valeurs directement appliquées comme filtres à *columnName*.
GENERATE	Retourne une table avec le produit cartésien entre chaque ligne de *table1* et la table résultant de l'évaluation de *table2* dans le contexte de la ligne actuelle de *table1*.
GENERATEALL	Retourne une table avec le produit cartésien entre chaque ligne de *table1* et la table résultant de l'évaluation de *table2* dans le contexte de la ligne actuelle de *table1*.

Fonction	Description
GENERATESERIES	Retourne une table à une seule colonne qui contient les valeurs d'une série arithmétique.
GROUPBY	Tout comme la fonction SUMMARIZE, GROUPBY n'utilise pas implicitement la fonction CALCULATE pour les colonnes d'extension qu'elle ajoute.
IGNORE	Modifie SUMMARIZECOLUMNS en omettant des expressions spécifiques de l'évaluation BLANK/NULL.
INTERSECT	Retourne l'intersection de lignes de deux tables, en conservant les doublons.
NATURALINNERJOIN	Effectue une jointure interne d'une table avec une autre table.
NATURALLEFTOUTERJOIN	Effectue une jointure de LeftTable avec RightTable.
ROLLUP	Modifie le comportement de SUMMARIZE en ajoutant des lignes de cumul au résultat des colonnes définies par le paramètre groupBy_columnName.
ROLLUPADDISSUBTOTAL	Modifie le comportement de SUMMARIZECOLUMNS en ajoutant des lignes de cumul/sous-total au résultat basé sur les colonnes groupBy_columnName.
ROLLUPISSUBTOTAL	Associe des groupes de cumul à la colonne ajoutée par ROLLUPADDISSUBTOTAL dans une expression ADDMISSINGITEMS.
ROLLUPGROUP	Modifie le comportement de SUMMARIZE et SUMMARIZECOLUMNS en ajoutant des lignes de cumul au résultat des colonnes définies par le paramètre groupBy_columnName.
ROW	Retourne une table à une seule ligne qui contient des valeurs qui sont le résultat des expressions fournies à chaque colonne.
SELECTCOLUMNS	Ajoute des colonnes calculées à la table ou à l'expression de table donnée.
SUBSTITUTEWITHINDEX	Retourne une table qui représente une semi-jointure gauche des deux tables fournies comme arguments.
SUMMARIZE	Retourne une table récapitulative des totaux demandés sur un ensemble de groupes.
SUMMARIZECOLUMNS	Retourne une table récapitulative sur un ensemble de groupes.
Constructeur de table	Retourne une table d'une ou plusieurs colonnes.
TOPN	Retourne les N premières lignes de la table spécifiée.

Fonction	Description
TREATAS	Applique le résultat d'une expression de table comme filtres aux colonnes d'une table non associée.
UNION	Crée une table d'union (jointure) à partir d'une paire de tables.
VALUES	Retourne une table à une colonne qui contient les valeurs distinctes de la table ou colonne spécifiée.

Commentaires

Cette page a-t-elle été utile ? Yes No

ADDCOLUMNS

Article • 20/10/2023

Ajoute des colonnes calculées à la table ou à l'expression de table donnée.

Syntaxe

```DAX
ADDCOLUMNS(<table>, <name>, <expression>[, <name>, <expression>]…)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Toute expression DAX qui retourne une table de données.
name	Nom donné à la colonne, placé entre guillemets doubles.
expression	Toute expression DAX qui retourne une expression scalaire, évaluée pour chaque ligne de *table*.

Valeur renvoyée

Table avec toutes ses colonnes d'origine et celles qui ont été ajoutées.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant retourne une version étendue de la table Product Category qui comprend les valeurs des ventes totales du canal Revendeurs et les ventes sur Internet.

```DAX
ADDCOLUMNS(ProductCategory,
                , "Internet Sales", SUMX(RELATEDTABLE(InternetSales_USD), InternetSales_USD[SalesAmount_USD])
                , "Reseller Sales", SUMX(RELATEDTABLE(ResellerSales_USD), ResellerSales_USD[SalesAmount_USD]))
```

La table suivante présente un aperçu des données telles qu'elles seraient reçues par toute fonction attendant de recevoir une table :

Agrandir le tableau

ProductCategory[ProductCategoryName]	ProductCategory[ProductCategoryAlternateKey]	ProductCategory[ProductCategoryKey]	[Internet Sales]	[Res Sales
Bikes	1	1	25107749,77	6308
Components	2	2		1120
Clothing	3	3	306157,5829	1669
Accessories	4	4	640920,1338	5343

Commentaires

Cette page a-t-elle été utile ?

Yes No

ADDMISSINGITEMS

Article • 20/10/2023

Ajoute des lignes avec des valeurs vides à une table retournée par SUMMARIZECOLUMNS.

Syntaxe

DAX

```
ADDMISSINGITEMS ( [<showAll_columnName> [, <showAll_columnName> [, … ] ] ],
<table> [, <groupBy_columnName> [, [<filterTable>] [, <groupBy_columnName>
[, [<filterTable>] [, … ] ] ] ] ] ] )
```

Paramètres

Agrandir le tableau

Terme	Définition
showAll_columnName	(Facultatif) Colonne pour laquelle des éléments sans données doivent être retournés pour les mesures utilisées. Si cet argument n'est pas spécifié, toutes les colonnes sont retournées.
table	Table SUMMARIZECOLUMNS.
groupBy_columnName	(Facultatif) Colonne sur laquelle effectuer le regroupement dans l'argument de table fourni.
filterTable	(Facultatif) Expression de table qui définit les lignes retournées.

Valeur renvoyée

Table avec une ou plusieurs colonnes.

Notes

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Avec SUMMARIZECOLUMNS

Une table retournée par SUMMARIZECOLUMNS inclut uniquement les lignes contenant des valeurs. En incorporant une expression SUMMARIZECOLUMNS dans une expression ADDMISSINGITEMS, les lignes ne contenant aucune valeur sont également retournées.

Exemple

Sans ADDMISSINGITEMS, la requête suivante :

DAX

```
SUMMARIZECOLUMNS(
    'Sales'[CustomerId],
    "Total Qty", SUM ( Sales[TotalQty] )
)
```

Retourne :

Agrandir le tableau

CustomerId	TotalQty
A	5
B	3
C	3
E	2

Avec ADDMISSINGITEMS, la requête suivante :

DAX

```
EVALUATE
ADDMISSINGITEMS (
    'Sales'[CustomerId],
    SUMMARIZECOLUMNS(
        'Sales'[CustomerId],
        "Total Qty", SUM ( Sales[TotalQty] )
    ),
    'Sales'[CustomerId]
)
```

Retourne :

Agrandir le tableau

CustomerId	TotalQty
A	5
B	3
C	3
D	
E	2
F	

Commentaires

Cette page a-t-elle été utile ? Yes No

CROSSJOIN

Article • 20/10/2023

Retourne une table contenant le produit cartésien de toutes les lignes de toutes les tables figurant dans les arguments. Les colonnes de la nouvelle table correspondent à toutes les colonnes de toutes les tables des arguments.

Syntaxe

DAX

```
CROSSJOIN(<table>, <table>[, <table>]…)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Toute expression DAX qui retourne une table de données

Valeur renvoyée

Table contenant le produit cartésien de toutes les lignes de toutes les tables figurant dans les arguments.

Notes

- Les noms de colonnes des arguments *table* doivent tous être différents dans toutes les tables, sans quoi une erreur est retournée.
- Le nombre total de lignes retournées par CROSSJOIN() est égal au produit du nombre de lignes de toutes les tables figurant dans les arguments. De plus, le nombre total de colonnes dans la table de résultats est la somme du nombre de colonnes dans toutes les tables. Par exemple, si **TableA** a **rA** lignes et **cA** colonnes, si **TableB** a **rB** lignes et **cB** colonnes, et si **TableC** a **rC** lignes et **cC** colonnes, la table obtenue a **rA** × **rB** × **rC** lignes et **cA** + **cB** + **cC** colonnes.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant montre les résultats de l'application de CROSSJOIN() à deux tables : **Colors** et **Stationery**.

La table **Colors** contient des couleurs (Color) et des modèles (Pattern) :

Agrandir le tableau

Couleur	Modèle
Rouge	Horizontal Stripe
Vert	Vertical Stripe
Bleu	Crosshatch

La table **Stationery** contient les polices et la présentation :

Agrandir le tableau

Police	Présentation
serif	embossed
sans-serif	engraved

L'expression permettant de générer la jointure croisée est présentée ci-dessous :

```
DAX

CROSSJOIN( Colors, Stationery)
```

Lorsque l'expression ci-dessus est utilisée chaque fois qu'une expression de table est attendue, les résultats de l'expression sont les suivants :

Agrandir le tableau

Couleur	Modèle	Police	Présentation
Rouge	Horizontal Stripe	serif	embossed

Couleur	Modèle	Police	Présentation
Vert	Vertical Stripe	serif	embossed
Bleu	Crosshatch	serif	embossed
Rouge	Horizontal Stripe	sans-serif	engraved
Vert	Vertical Stripe	sans-serif	engraved
Bleu	Crosshatch	sans-serif	engraved

Commentaires

Cette page a-t-elle été utile ? Yes No

CURRENTGROUP

Article • 20/10/2023

Retourne un ensemble de lignes à partir de l'argument de table d'une expression GROUPBY, qui appartiennent à la ligne actuelle du résultat de GROUPBY.

Syntaxe

```
DAX

CURRENTGROUP ( )
```

Paramètres

Aucune

Valeur renvoyée

Lignes dans l'argument de table de la fonction GROUPBY correspondant à un groupe de valeurs des arguments groupBy_columnName.

Notes

- Cette fonction s'utilise uniquement dans une expression GROUPBY.
- La fonction ne prend aucun argument et est seulement prise en charge en tant que premier argument de l'une des fonctions d'agrégation suivantes : AVERAGEX, COUNTAX, COUNTX, GEOMEANX, MAXX, MINX, PRODUCTX, STDEVX.S, STDEVX.P, SUMX, VARX.S, VARX.P.

Exemple

Consultez GROUPBY.

Commentaires

Cette page a-t-elle été utile ? Yes No

DATATABLE

Article • 20/10/2023

Fournit un mécanisme permettant de déclarer un ensemble inline de valeurs de données.

Syntaxe

DAX

```
DATATABLE (ColumnName1, DataType1, ColumnName2, DataType2..., {{Value1,
Value2...}, {ValueN, ValueN+1...}...})
```

Paramètres

Agrandir le tableau

Terme	Définition
ColumnName	Toute expression DAX qui retourne une table.
DataType	Énumération contenant : INTEGER, DOUBLE, STRING, BOOLEAN, CURRENCY, DATETIME
Value	Argument unique utilisant la syntaxe Excel pour une constante de tableau unidimensionnel, imbriquée pour fournir un tableau de tableaux. Cet argument représente l'ensemble des valeurs de données qui figureront dans la table. Par exemple, { {values in row1}, {values in row2}, {values in row3}, etc. } Où {values in row1} est un ensemble d'expressions de constantes séparées par des virgules, à savoir une combinaison de constantes, associées à des fonctions de base comme DATE, TIME et BLANK, à un opérateur plus entre DATE et TIME et à un opérateur moins unaire pour exprimer des valeurs négatives. Les valeurs valides sont les suivantes : 3, -5, BLANK(), "2009-04-15 02:45:21". Les valeurs ne peuvent pas référencer des éléments en dehors de l'expression immédiate, ni référencer des colonnes, tables, relations ou autre élément. Les valeurs manquantes sont traitées de la même façon que BLANK(). Par exemple, {1,2,BLANK(),4} et {1,2,,4} sont des expressions identiques.

Valeur renvoyée

Table déclarant un ensemble inline de valeurs.

Notes

- Contrairement à DATATABLE, le constructeur de table autorise les expressions scalaires comme valeurs d'entrée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

DAX

```
= DataTable("Name", STRING,
               "Region", STRING
               ,{
                        {" User1","East"},
                        {" User2","East"},
                        {" User3","West"},
                        {" User4","West"},
                        {" User4","East"}
                }
           )
```

Commentaires

Cette page a-t-elle été utile ? Yes No

DETAILROWS

Article • 20/10/2023

Évalue une expression de lignes de détails définie pour une mesure et retourne les données.

Syntaxe

```DAX
DETAILROWS([Measure])
```

Paramètres

Agrandir le tableau

Terme	Définition
Measure	Nom d'une mesure.

Valeur renvoyée

Table avec les données retournées par l'expression de lignes de détails. Si aucune expression de lignes de détails n'est définie, les données pour la table contenant la mesure sont retournées.

Commentaires

Cette page a-t-elle été utile ? Yes No

DISTINCT (colonne)

Article • 20/10/2023

Retourne une table à une colonne qui contient les valeurs distinctes de la colonne spécifiée. En d'autres termes, les valeurs dupliquées sont supprimées et seules les valeurs uniques sont retournées.

> **Notes**
>
> Cette fonction ne peut pas être utilisée pour retourner des valeurs dans une cellule ou une colonne dans une feuille de calcul. Au lieu de cela, vous imbriquez la fonction DISTINCT dans une formule pour obtenir une liste de valeurs distinctes qui peuvent être passées à une autre fonction, puis comptées, additionnées ou utilisées pour d'autres opérations.

Syntaxe

DAX

```
DISTINCT(<column>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne dont les valeurs uniques doivent être retournées. Ou il peut s'agir d'une expression retournant une colonne.

Valeur de retour

Colonne de valeurs uniques.

Remarques

- Les résultats de DISTINCT sont affectés par le contexte de filtre actuel. Par exemple, si vous utilisez la formule de l'exemple suivant pour créer une mesure, les résultats

changent chaque fois que la table a été filtrée pour afficher uniquement une région ou une période particulière.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Fonctions connexes

Il existe une autre version de la fonction DISTINCT, DISTINCT (table), qui retourne une table en supprimant les lignes dupliquées d'une autre table ou expression.

La fonction VALUES est semblable à DISTINCT ; elle peut également être utilisée pour retourner une liste de valeurs uniques et retourne généralement les mêmes résultats que DISTINCT. Toutefois, dans certains contextes, VALUES retourne une valeur spéciale supplémentaire. Pour plus d'informations, consultez VALUES, fonction.

Exemple

La formule suivante compte le nombre de clients uniques qui ont généré des commandes sur Internet. La table qui suit illustre les résultats possibles quand la formule est ajoutée à un rapport.

DAX

```
= COUNTROWS(DISTINCT(InternetSales_USD[CustomerKey]))
```

Vous ne pouvez pas coller la liste des valeurs retournées par DISTINCT directement dans une colonne. Passez plutôt les résultats de la fonction DISTINCT à une autre fonction qui compte, filtre ou agrège les valeurs à l'aide de la liste. Pour rendre l'exemple aussi simple que possible, la table de valeurs distinctes a été passée à la fonction COUNTROWS.

Agrandir le tableau

Étiquettes de ligne	Accessories	Bikes	Clothing	Total général
2005		1013		1013
2006		2677		2677
2007	6792	4875	2867	9309

Étiquettes de ligne	Accessories	Bikes	Clothing	Total général
2008	9435	5451	4196	11377
Total général	15114	9132	6852	18484

Notez également que les résultats ne sont pas additifs. Autrement dit, le nombre total de clients uniques en *2007* n'est pas la somme des clients uniques dans les catégories *Accessoires*, *Vélos* et *Habillement* pour cette année. En effet, un client peut être compté dans plusieurs groupes.

Contenu connexe

Fonctions de filtrage
FILTER, fonction
RELATED, fonction
VALUES, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

DISTINCT (table)

Article • 14/12/2023

Retourne une table en supprimant les lignes en double d'une autre table ou expression.

Syntaxe

```DAX
DISTINCT(<table>)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table dont les lignes uniques doivent être retournées. La table peut également être une expression produisant une table.

Valeur renvoyée

Table contenant uniquement des lignes distinctes.

Fonctions connexes

Une autre version de la fonction DISTINCT, DISTINCT (colonne), accepte un nom de colonne comme paramètre d'entrée.

Exemple

Cette requête :

```DAX
EVALUATE DISTINCT( { (1, "A"), (2, "B"), (1, "A") } )
```

Retourne la table :

Agrandir le tableau

[Value1]	[Value2]
1	Un
2	G

Contenu connexe

Fonctions de filtrage
DISTINCT (colonne)
FILTER, fonction
RELATED, fonction
VALUES, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

EXCEPT

Article • 20/10/2023

Retourne les lignes de la première table de l'expression qui n'apparaissent pas dans la deuxième table.

Syntaxe

DAX

```
EXCEPT(<table_expression1>, <table_expression2>)
```

Paramètres

Agrandir le tableau

Terme	Définition
Table_expression	Toute expression DAX qui retourne une table.

Valeur de retour

Table qui contient les lignes d'une table moins toutes les lignes d'une autre table.

Notes

- Si une ligne apparaît dans les deux tables, elle et ses doublons ne sont pas présents dans le jeu de résultats. Si une ligne apparaît uniquement dans table_expression1, elle et ses doublons apparaissent dans le jeu de résultats.
- Les noms de colonne correspondent aux noms de colonnes de table_expression1.
- La table retournée possède un lignage basé sur les colonnes de table_expression1, quel que soit le lignage des colonnes de la seconde table. Par exemple, si la première colonne de la première table_expression a un lignage sur la colonne C1 de base du modèle, Except réduira les lignes en fonction de la disponibilité des valeurs de la première colonne de la seconde table_expression et conservera intact le lignage sur la colonne C1 de base.

- Les deux tables doivent avoir le même nombre de colonnes.
- Les colonnes sont comparées en fonction de leur positionnement et avec une comparaison des données sans forçage de type.
- L'ensemble de lignes retournées dépend de l'ordre des deux expressions.
- La table retournée n'inclut pas les colonnes des tables associées à table_expression1.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

States1

Agrandir le tableau

State
A
B
B
B
C
D
D

States2

Agrandir le tableau

State
B
C
D

State
D
D
E
E
E

Except(States1, States2)

Agrandir le tableau

State
A

Except(States2, States1)

Agrandir le tableau

State
E
E
E

Commentaires

Cette page a-t-elle été utile ? Yes No

FILTERS

Article • 20/10/2023

Retourne les valeurs directement appliquées comme filtres à *columnName*.

Syntaxe

```
DAX

FILTERS(<columnName>)
```

Paramètres

Agrandir le tableau

Terme	Description
columnName	Nom d'une colonne existante, spécifié avec la syntaxe DAX standard. Il ne peut pas s'agir d'une expression.

Valeur de retour

Retourne les valeurs directement appliquées comme filtres à *columnName*.

Remarques

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant montre comment déterminer le nombre de filtres directs que comporte une colonne.

```
DAX

= COUNTROWS(FILTERS(ResellerSales_USD[ProductKey]))
```

Cet exemple vous permet de savoir combien de filtres directs ont été appliqués à ResellerSales_USD[ProductKey] dans le contexte d'évaluation de l'expression.

Commentaires

Cette page a-t-elle été utile ? Yes No

GENERATE

Article • 20/10/2023

Retourne une table avec le produit cartésien entre chaque ligne de *table1* et la table résultant de l'évaluation de *table2* dans le contexte de la ligne actuelle de *table1*.

Syntaxe

```DAX
GENERATE(<table1>, <table2>)
```

Paramètres

Agrandir le tableau

Terme	Définition
table1	Toute expression DAX qui retourne une table.
table2	Toute expression DAX qui retourne une table.

Valeur de retour

Table avec le produit cartésien entre chaque ligne de *table1* et table résultant de l'évaluation de *table2* dans le contexte de la ligne actuelle de *table1*

Remarques

- Si l'évaluation de *table2* pour la ligne actuelle de *table1* retourne une table vide, la table de résultats ne contient pas la ligne actuelle de *table1*. Cette fonction se distingue de GENERATEALL() qui inclut la ligne actuelle de *table1* dans les résultats, et les colonnes correspondant à *table2* présentent des valeurs null pour cette ligne.
- Tous les noms de colonne de *table1* et *table2* doivent être différents. Sinon, une erreur est retournée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Dans l'exemple suivant, l'utilisateur souhaite une table récapitulative des ventes par région et par catégorie de produit pour le réseau de revendeurs, à l'image de la table suivante :

Agrandir le tableau

SalesTerritory[SalesTerritoryGroup]	ProductCategory[ProductCategoryName]	[Reseller Sales]
Europe	Accessories	$ 142,227.27
Europe	Bikes	$ 9,970,200.44
Europe	Clothing	$ 365,847.63
Europe	Components	$ 2,214,440.19
Amérique du Nord	Accessories	$ 379,305.15
Amérique du Nord	Bikes	$ 52,403,796.85
Amérique du Nord	Clothing	$ 1,281,193.26
Amérique du Nord	Components	$ 8,882,848.05
Pacifique	Accessories	$ 12,769.57
Pacifique	Bikes	$ 710,677.75
Pacifique	Clothing	$ 22,902.38
Pacifique	Composants	$ 108,549.71

La formule suivante génère la table ci-dessus :

DAX

```
GENERATE(
SUMMARIZE(SalesTerritory, SalesTerritory[SalesTerritoryGroup])
,SUMMARIZE(ProductCategory
, [ProductCategoryName]
, "Reseller Sales", SUMX(RELATEDTABLE(ResellerSales_USD),
ResellerSales_USD[SalesAmount_USD])
```

```
)
)
```

1. La première instruction SUMMARIZE, `SUMMARIZE(SalesTerritory, SalesTerritory[SalesTerritoryGroup])`, génère une table des groupes de secteurs, où chaque ligne est un groupe de secteurs, comme ci-dessous :

   Agrandir le tableau

SalesTerritory[SalesTerritoryGroup]
Amérique du Nord
Europe
Pacifique
N/A

2. La deuxième instruction SUMMARIZE, `SUMMARIZE(ProductCategory, [ProductCategoryName], "Reseller Sales", SUMX(RELATEDTABLE(ResellerSales_USD), ResellerSales_USD[SalesAmount_USD]))`, génère une table de groupes de catégories de produit avec les ventes des revendeurs pour chaque groupe, comme ci-dessous :

   Agrandir le tableau

ProductCategory[ProductCategoryName]	[Reseller Sales]
Bikes	$ 63,084,675.04
Composants	$ 11,205,837.96
Clothing	$ 1,669,943.27
Accessories	$ 534,301.99

3. Cependant, si vous évaluez la table ci-dessus dans le contexte de chaque ligne de la table des groupes de secteurs, vous obtenez des résultats différents pour chaque secteur.

Commentaires

Cette page a-t-elle été utile ? Yes No

GENERATEALL

Article • 20/10/2023

Retourne une table avec le produit cartésien entre chaque ligne de *table1* et la table résultant de l'évaluation de *table2* dans le contexte de la ligne actuelle de *table1*.

Syntaxe

DAX

```
GENERATEALL(<table1>, <table2>)
```

Paramètres

Agrandir le tableau

Terme	Définition
table1	Toute expression DAX qui retourne une table.
table2	Toute expression DAX qui retourne une table.

Valeur de retour

Table avec le produit cartésien entre chaque ligne de *table1* et table résultant de l'évaluation de *table2* dans le contexte de la ligne actuelle de *table1*

Remarques

- Si l'évaluation de *table2* pour la ligne actuelle de *table1* retourne une table vide, alors la ligne actuelle de *table1* sera incluse dans les résultats et les colonnes correspondant à *table2* auront des valeurs Null pour cette ligne. Ceci diffère de la fonction GENERATE() qui **n'inclut pas** la ligne actuelle de *table1* dans les résultats.
- Tous les noms de colonne de *table1* et *table2* doivent être différents. Sinon, une erreur est retournée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Dans l'exemple suivant, l'utilisateur souhaite obtenir une table de synthèse des ventes par région et par catégorie de produit pour le réseau de revendeurs comme la table suivante :

Agrandir le tableau

SalesTerritory[SalesTerritoryGroup]	ProductCategory[ProductCategoryName]	[Reseller Sales]
Europe	Accessories	$ 142,227.27
Europe	Bikes	$ 9,970,200.44
Europe	Clothing	$ 365,847.63
Europe	Components	$ 2,214,440.19
N/A	Accessories	
N/A	Bikes	
N/A	Clothing	
N/A	Components	
Amérique du Nord	Accessories	$ 379,305.15
Amérique du Nord	Bikes	$ 52,403,796.85
Amérique du Nord	Clothing	$ 1,281,193.26
Amérique du Nord	Components	$ 8,882,848.05
Pacifique	Accessories	$ 12,769.57
Pacifique	Bikes	$ 710,677.75
Pacifique	Clothing	$ 22,902.38
Pacifique	Composants	$ 108,549.71

La formule suivante génère la table ci-dessus :

DAX

```
GENERATEALL(
SUMMARIZE(SalesTerritory, SalesTerritory[SalesTerritoryGroup])
,SUMMARIZE(ProductCategory
, [ProductCategoryName]
, "Reseller Sales", SUMX(RELATEDTABLE(ResellerSales_USD),
ResellerSales_USD[SalesAmount_USD])
)
)
```

1. La première instruction SUMMARIZE génère une table des groupes de secteurs, où chaque ligne est un groupe de secteurs comme ceux indiqués ci-dessous :

Agrandir le tableau

SalesTerritory[SalesTerritoryGroup]
Amérique du Nord
Europe
Pacifique
N/A

2. La deuxième instruction SUMMARIZE génère une table de groupes de catégories de produit avec les ventes des revendeurs pour chaque groupe comme indiqué ci-dessous :

Agrandir le tableau

ProductCategory[ProductCategoryName]	**[Reseller Sales]**
Bikes	$ 63,084,675.04
Composants	$ 11,205,837.96
Clothing	$ 1,669,943.27
Accessories	$ 534,301.99

3. Toutefois, si vous évaluez la table ci-dessus dans le contexte de chaque ligne de la table des groupes de secteurs, vous obtenez des résultats différents pour chaque secteur.

Commentaires

Cette page a-t-elle été utile ? Yes No

GENERATESERIES

Article • 20/10/2023

Retourne une table à une seule colonne contenant les valeurs d'une série arithmétique, autrement dit une série dans laquelle chaque valeur diffère de la précédente par une quantité constante. Le nom de la colonne retournée est Value.

Syntaxe

DAX

```
GENERATESERIES(<startValue>, <endValue>[, <incrementValue>])
```

Paramètres

Agrandir le tableau

Terme	Définition
startValue	Valeur initiale utilisée pour générer la séquence.
endValue	Valeur de fin utilisée pour générer la séquence.
incrementValue	(Facultatif) Valeur d'incrément de la séquence. Quand elle est omise, la valeur par défaut est 1.

Valeur renvoyée

Table à une seule colonne contenant les valeurs d'une série arithmétique. Le nom de la colonne est Value.

Notes

- Quand endValue est inférieur à startValue, une table vide est retournée.
- incrementValue doit être une valeur positive.
- La séquence s'arrête à la dernière valeur inférieure ou égale à endValue.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au

niveau des lignes (RLS).

Exemple 1

La requête DAX suivante :

```
DAX
EVALUATE GENERATESERIES(1, 5)
```

Retourne la table à une seule colonne suivante :

Agrandir le tableau

[Value]
1
2
3
4
5

Exemple 2

La requête DAX suivante :

```
DAX
EVALUATE GENERATESERIES(1.2, 2.4, 0.4)
```

Retourne la table à une seule colonne suivante :

Agrandir le tableau

[Value]
1.2
1.6
2

[Value]
2.4

Exemple 3

La requête DAX suivante :

```
DAX

EVALUATE GENERATESERIES(CURRENCY(10), CURRENCY(12.4), CURRENCY(0.5))
```

Retourne la table à une seule colonne suivante :

Agrandir le tableau

[Value]
10
10.5
11
11.5
12

Commentaires

Cette page a-t-elle été utile ? Yes No

GROUPBY

Article • 20/10/2023

La fonction GROUPBY est similaire à la fonction SUMMARIZE. Toutefois, GROUPBY n'utilise pas implicitement CALCULATE sur les colonnes d'extension qu'elle ajoute. GROUPBY permet d'utiliser une nouvelle fonction, CURRENTGROUP, au sein des fonctions d'agrégation dans les colonnes d'extension qu'elle ajoute. GROUPBY s'utilise pour effectuer plusieurs agrégations dans une analyse de table unique.

Syntaxe

DAX

```
GROUPBY (<table> [, <groupBy_columnName> [, <groupBy_columnName> [, …]]] [,
<name>, <expression> [, <name>, <expression> [, …]]])
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Toute expression DAX qui retourne une table de données.
groupBy_columnName	Nom d'une colonne existante dans la table (ou dans une table associée) selon lequel les données doivent être regroupées. Ce paramètre ne peut pas être une expression.
name	Nom donné à une nouvelle colonne qui est ajoutée à la liste des colonnes GroupBy, placé entre guillemets doubles.
expression	Une des fonctions d'agrégation X avec CURRENTGROUP() comme premier argument. Consultez la section « Avec CURRENTGROUP » plus loin pour voir la liste complète des fonctions d'agrégation X prises en charge.

Valeur renvoyée

Table avec les colonnes sélectionnées pour les arguments groupBy_columnName et les colonnes d'extension désignées par les arguments name.

Notes

- La fonction GROUPBY effectue les opérations suivantes :

    1. Elle commence avec la table spécifiée (et toutes les tables associées dans la direction « à un »).

    2. Elle crée un regroupement avec toutes les colonnes GroupBy (qui doivent exister dans la table depuis l'étape 1).

    3. Chaque groupe correspond à une ligne du résultat, mais représente un ensemble de lignes dans la table d'origine.

    4. Pour chaque groupe, elle évalue les colonnes d'extension qui sont ajoutées. Contrairement à la fonction SUMMARIZE, aucune fonction CALCULATE implicite n'est exécutée et le groupe n'est pas inclus dans le contexte de filtre.

- Chaque colonne pour laquelle vous définissez un nom doit avoir une expression correspondante ; dans le cas contraire, une erreur est retournée. Le premier argument, name, définit le nom de la colonne dans les résultats. Le deuxième argument, expression, définit le calcul effectué pour obtenir la valeur de chaque ligne de cette colonne.

- groupBy_columnName doit être dans la table ou dans une table associée.

- Chaque nom doit être placé entre guillemets doubles.

- La fonction regroupe un ensemble sélectionné de lignes en un ensemble de lignes de synthèse selon les valeurs d'une ou plusieurs colonnes groupBy_columnName. Une ligne est retournée pour chaque groupe.

- GROUPBY est principalement utilisé pour effectuer des agrégations sur les résultats intermédiaires d'expressions de table DAX. Pour réaliser efficacement des agrégations sur des tables physiques dans le modèle, utilisez la fonction SUMMARIZECOLUMNS ou SUMMARIZE.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Avec CURRENTGROUP

CURRENTGROUP peut uniquement être utilisé dans une expression qui définit une colonne d'extension dans la fonction GROUPBY. En effet, CURRENTGROUP retourne un

ensemble de lignes à partir de l'argument table de GROUPBY, qui appartiennent à la ligne actuelle du résultat de GROUPBY. La fonction CURRENTGROUP ne prend pas d'arguments et elle est seulement prise en charge en tant que premier argument de l'une des fonctions d'agrégation suivantes : AVERAGEX, COUNTAX, COUNTX, GEOMEANX, MAXX, MINX, PRODUCTX, STDEVX.S, STDEVX.P, SUMX, VARX.S, VARX.P.

Exemple

L'exemple suivant calcule d'abord les ventes totales regroupées par pays et catégorie de produit sur les tables physiques à l'aide de la fonction SUMMARIZECOLUMNS. Il utilise ensuite la fonction GROUPBY pour analyser le résultat intermédiaire de la première étape et connaître ainsi le nombre maximal de ventes dans chaque pays pour les différentes catégories de produits.

DAX

```
DEFINE
VAR SalesByCountryAndCategory =
SUMMARIZECOLUMNS(
Geography[Country],
Product[Category],
"Total Sales", SUMX(Sales, Sales[Price] * Sales[Qty])
)

EVALUATE
GROUPBY(
SalesByCountryAndCategory,
Geography[Country],
"Max Sales", MAXX(CURRENTGROUP(), [Total Sales])
)
```

Contenu connexe

SUMMARIZE, fonction
SUMMARIZECOLUMNS

Commentaires

Cette page a-t-elle été utile ? Yes No

IGNORE

Article • 20/10/2023

Modifie le comportement de la fonction SUMMARIZECOLUMNS en omettant les expressions spécifiques de l'évaluation BLANK/NULL. Les lignes pour lesquelles toutes les expressions qui n'utilisent pas IGNORE retournent BLANK/NULL sont exclues, indépendamment du fait que les expressions qui utilisent IGNORE prennent ou non la valeur BLANK/NULL. Cette fonction peut uniquement être utilisée dans une expression SUMMARIZECOLUMNS.

Syntaxe

DAX

```
IGNORE(<expression>)
```

Avec SUMMARIZECOLUMNS,

DAX

```
SUMMARIZECOLUMNS(<groupBy_columnName>[, < groupBy_columnName >]…,
[<filterTable>]…[, <name>, IGNORE(…)]…)
```

Paramètres

Agrandir le tableau

Terme	Définition
expression	Toute expression DAX qui retourne une seule valeur (et non une table).

Valeur renvoyée

La fonction ne retourne pas de valeur.

Notes

La syntaxe IGNORE peut uniquement être utilisée en tant qu'argument d'expression pour SUMMARIZECOLUMNS.

Exemple

Consultez SUMMARIZECOLUMNS.

Commentaires

Cette page a-t-elle été utile ? Yes No

INTERSECT

Article • 20/10/2023

Retourne l'intersection de lignes de deux tables, en conservant les doublons.

Syntaxe

DAX

```
INTERSECT(<table_expression1>, <table_expression2>)
```

Paramètres

Agrandir le tableau

Terme	Définition
Table_expression	Toute expression DAX qui retourne une table.

Valeur de retour

Table contenant toutes les lignes de table_expression1 qui se trouvent également dans table_expression2

Exceptions

Remarques

- Intersect n'est pas une fonction commutative. En général, Intersect(T1, T2) ne retourne pas le même jeu de résultats qu'Intersect(T2, T1).
- Les lignes en double sont conservées. Si table_expression1 et table_expression2 contiennent une même ligne, cette ligne et tous les doublons dans table_expression_1 sont inclus dans le jeu de résultats.
- Les noms de colonne correspondent aux noms de colonnes de table_expression1.
- La table retournée possède un lignage basé sur les colonnes de table_expression1, quel que soit le lignage des colonnes de la seconde table. Par exemple, si la

première colonne de la première table retournée par table_expression a un lignage sur la colonne C1 de base du modèle, la fonction Intersect réduit les lignes en fonction de l'intersection sur la première colonne de la seconde table retournée et conserve intact le lignage sur la colonne C1 de base.

- Les colonnes sont comparées en fonction de leur positionnement et avec une comparaison des données sans forçage de type.
- La table retournée n'inclut pas les colonnes des tables associées à table_expression1.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

States1

Agrandir le tableau

State
A
A
B
B
B
C
D
D

States2

Agrandir le tableau

State
B

State
C
D
D
D
E

Intersect(States1, States2)

Agrandir le tableau

State
B
B
B
C
D
D

Intersect(States2, States1)

Agrandir le tableau

State
B
C
D
D
D

Commentaires

Cette page a-t-elle été utile ? Yes No

NATURALINNERJOIN

Article • 14/12/2023

Effectue une jointure interne d'une table avec une autre table.

Syntaxe

DAX

```
NATURALINNERJOIN(<LeftTable>, <RightTable>)
```

Paramètres

Agrandir le tableau

Terme	Définition
LeftTable	Expression de table définissant la table sur le côté gauche de la jointure.
RightTable	Expression de table définissant la table sur le côté droit de la jointure.

Valeur renvoyée

Table comprenant uniquement les lignes pour lesquelles les valeurs des colonnes communes spécifiées sont présentes dans les deux tables. La table retournée a les colonnes communes de la table de gauche et les autres colonnes des deux tables.

Notes

- Les tables sont jointes sur des colonnes communes (par nom) dans les deux tables. Si les deux tables n'ont pas de noms de colonnes communs, une erreur est retournée.
- Il n'existe pas de garantie d'ordre de tri des résultats.
- Les colonnes auxquelles s'effectue la jointure doivent avoir le même type de données dans les deux tables.
- La jointure s'effectue seulement aux colonnes de la même table source (avec le même lignage). Par exemple, Products[ProductID], WebSales[ProductdID],

StoreSales[ProductdID] avec des relations plusieurs-à-un entre WebSales et StoreSales et la table Products basée sur la colonne ProductID, et les tables WebSales et StoreSales sont jointes à [ProductID].

- Une sémantique de comparaison stricte est utilisée lors de la jointure. Il n'y a pas de forçage de type. Par exemple, 1 n'est pas égal à 1.0.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Contenu connexe

NATURALLEFTOUTERJOIN

Commentaires

Cette page a-t-elle été utile ? Yes No

NATURALLEFTOUTERJOIN

Article • 20/10/2023

Effectue une jointure de LeftTable avec RightTable en utilisant la sémantique Left Outer Join.

Syntaxe

DAX

```
NATURALLEFTOUTERJOIN(<LeftTable>, <RightTable>)
```

Paramètres

Agrandir le tableau

Terme	Définition
LeftTable	Expression de table définissant la table sur le côté gauche de la jointure.
RightTable	Expression de table définissant la table sur le côté droit de la jointure.

Valeur renvoyée

Une table qui inclut uniquement les lignes de RightTable pour lesquelles les valeurs dans les colonnes communes spécifiées sont également présentes dans LeftTable. La table retournée contient les colonnes communes de la table de gauche et les autres colonnes des deux tables.

Notes

- Les tables sont jointes sur des colonnes communes (par nom) dans les deux tables. Si les deux tables n'ont pas de noms de colonnes communs, une erreur est retournée.
- Il n'existe pas de garantie d'ordre de tri des résultats.
- Les colonnes auxquelles s'effectue la jointure doivent avoir le même type de données dans les deux tables.

- La jointure s'effectue seulement aux colonnes de la même table source (avec le même lignage). Par exemple, Products[ProductID], WebSales[ProductdID], StoreSales[ProductdID] avec des relations plusieurs-à-un entre WebSales et StoreSales et la table Products basée sur la colonne ProductID, et les tables WebSales et StoreSales sont jointes à [ProductID].
- Une sémantique de comparaison stricte est utilisée lors de la jointure. Il n'y a pas de forçage de type. Par exemple, 1 n'est pas égal à 1.0.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Contenu connexe

NATURALINNERJOIN

Commentaires

Cette page a-t-elle été utile ? Yes No

ROLLUP

Article • 20/10/2023

Modifie le comportement de la fonction SUMMARIZE en ajoutant des lignes de cumul au résultat des colonnes définies par le paramètre groupBy_columnName. Cette fonction s'utilise uniquement dans une expression SUMMARIZE.

Syntaxe

DAX

```
ROLLUP ( <groupBy_columnName> [, <groupBy_columnName> [, … ] ] )
```

Avec SUMMARIZE,

DAX

```
SUMMARIZE(<table>, <groupBy_columnName>[, <groupBy_columnName>]…[,
ROLLUP(<groupBy_columnName>[,< groupBy_columnName>…])][, <name>,
<expression>]…)
```

Paramètres

Agrandir le tableau

Terme	Définition
groupBy_columnName	Nom qualifié d'une colonne existante ou fonction ROLLUPGROUP à utiliser pour créer des groupes de synthèse en fonction des valeurs qui s'y trouvent. Ce paramètre ne peut pas être une expression.

Valeur renvoyée

Cette fonction ne retourne pas de valeur. Elle spécifie uniquement l'ensemble de colonnes dont les sous-totaux doivent être calculés.

Notes

Cette fonction s'utilise uniquement dans une expression SUMMARIZE.

Exemple

Consultez SUMMARIZE.

Commentaires

Cette page a-t-elle été utile ? Yes No

ROLLUPADDISSUBTOTAL

Article • 20/10/2023

Modifie le comportement de la fonction SUMMARIZECOLUMNS en ajoutant des lignes de cumul/sous-total au résultat basé sur les colonnes groupBy_columnName. Cette fonction peut uniquement être utilisée dans une expression SUMMARIZECOLUMNS.

Syntaxe

DAX

```
ROLLUPADDISSUBTOTAL ( [<grandtotalFilter>], <groupBy_columnName>, <name> [,
[<groupLevelFilter>] [, <groupBy_columnName>, <name> [, [<groupLevelFilter>]
[, … ] ] ] ] )
```

Paramètres

Agrandir le tableau

Terme	Définition
grandtotalFilter	(Facultatif) Filtre à appliquer au niveau du total général (grandtotal).
groupBy_columnName	Nom d'une colonne existante utilisé pour créer des groupes de synthèse en fonction des valeurs qui s'y trouvent. Il ne peut pas s'agir d'une expression.
name	Nom d'une colonne ISSUBTOTAL. Les valeurs de la colonne sont calculées à l'aide de la fonction ISSUBTOTAL.
groupLevelFilter	(Facultatif) Filtre à appliquer au niveau actuel.

Valeur renvoyée

La fonction ne retourne pas de valeur.

Notes

None

Exemple

Consultez SUMMARIZECOLUMNS.

Commentaires

Cette page a-t-elle été utile ? Yes No

ROLLUPGROUP

Article • 20/10/2023

Modifie le comportement des fonctions SUMMARIZE et SUMMARIZECOLUMNS en ajoutant des lignes de cumul au résultat des colonnes définies par le paramètre groupBy_columnName. Cette fonction s'utilise uniquement dans une expression SUMMARIZE ou SUMMARIZECOLUMNS.

Syntaxe

DAX

```
ROLLUPGROUP ( <groupBy_columnName> [, <groupBy_columnName> [, … ] ] )
```

Paramètres

Agrandir le tableau

Terme	Définition
groupBy_columnName	Nom qualifié d'une colonne existante ou fonction ROLLUPGROUP à utiliser pour créer des groupes de synthèse en fonction des valeurs qui s'y trouvent. Ce paramètre ne peut pas être une expression.

Valeur renvoyée

Cette fonction ne retourne pas de valeur. Elle marque un ensemble de colonnes à traiter comme groupe unique pendant le calcul des sous-totaux par ROLLUP ou ROLLUPADDISSUBTOTAL.

Notes

La fonction ROLLUPGROUP s'utilise uniquement en tant qu'argument groupBy_columnName pour ROLLUP, ROLLUPADDISSUBTOTAL ou ROLLUPISSUBTOTAL.

Exemple

Consultez SUMMARIZE et SUMMARIZECOLUMNS.

Commentaires

Cette page a-t-elle été utile ? Yes No

ROLLUPISSUBTOTAL

Article • 20/10/2023

Associe des groupes de cumul à la colonne ajoutée par ROLLUPADDISSUBTOTAL. Cette fonction s'utilise uniquement dans une expression ADDMISSINGITEMS.

Syntaxe

DAX

```
ROLLUPISSUBTOTAL ( [<grandTotalFilter>], <groupBy_columnName>,
<isSubtotal_columnName> [, [<groupLevelFilter>] [, <groupBy_columnName>,
<isSubtotal_columnName> [, [<groupLevelFilter>] [, … ] ] ] ] )
```

Paramètres

Agrandir le tableau

Terme	Définition
grandTotalFilter	(Facultatif) Filtre à appliquer au niveau du total général (grandtotal).
groupBy_columnName	Nom d'une colonne existante utilisé pour créer des groupes de synthèse en fonction des valeurs qui s'y trouvent. Il ne peut pas s'agir d'une expression.
isSubtotal_columnName	Nom d'une colonne ISSUBTOTAL. Les valeurs de la colonne sont calculées à l'aide de la fonction ISSUBTOTAL.
groupLevelFilter	(Facultatif) Filtre à appliquer au niveau actuel.

Valeur renvoyée

Aucune

Notes

Cette fonction s'utilise uniquement dans une expression ADDMISSINGITEMS.

Commentaires

Cette page a-t-elle été utile ? Yes No

ROW, fonction

Article • 20/10/2023

Retourne une table à une seule ligne qui contient des valeurs qui sont le résultat des expressions fournies à chaque colonne.

Syntaxe

DAX

```
ROW(<name>, <expression>[[,<name>, <expression>]…])
```

Paramètres

Agrandir le tableau

Terme	Définition
name	Nom donné à la colonne, placé entre guillemets doubles.
expression	Toute expression DAX qui retourne une valeur scalaire unique à remplir. *name*.

Valeur renvoyée

Table à une seule ligne.

Notes

- Les arguments doivent toujours se présenter par paires *name* et *expression*.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant retourne une table à une ligne avec le total des ventes pour Internet et les réseaux de revendeurs.

DAX

```
ROW("Internet Total Sales (USD)", SUM(InternetSales_USD[SalesAmount_USD]),
         "Resellers Total Sales (USD)",
SUM(ResellerSales_USD[SalesAmount_USD]))
```

Commentaires

Cette page a-t-elle été utile ? Yes No

SELECTCOLUMNS

Article • 20/10/2023

Retourne une table avec les colonnes sélectionnées de la table et les nouvelles colonnes spécifiées par les expressions DAX.

Syntaxe

DAX

```
SELECTCOLUMNS(<Table>, [<Name>], <Expression>, <Name>], …)
```

Paramètres

Agrandir le tableau

Terme	Définition
Table de charge de travail	Toute expression DAX qui retourne une table.
Nom	Nom donné à la colonne, placé entre guillemets doubles.
Expression	Toute expression qui retourne une valeur scalaire comme une référence de colonne, un entier ou une valeur de chaîne.

Valeur de retour

Table avec le même nombre de lignes que la table spécifiée comme premier argument. La table renvoyée comporte une colonne pour chaque paire d'arguments <Name>, <Expression> et chaque expression est évaluée dans le contexte d'une ligne à partir de l'argument <Table> spécifié.

Remarques

SELECTCOLUMNS a la même signature que ADDCOLUMNS et a le même comportement sauf qu'au lieu de commencer par la <Table> spécifiée, SELECTCOLUMNS commence par une table vide avant d'ajouter des colonnes.

Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Pour la table suivante nommée **Client** :

Agrandir le tableau

Pays ou région	State	Count	Total
IND	JK	20	800
IND	MH	25	1 000
IND	WB	10	900
États-Unis	CA	5	500
États-Unis	WA	10	900

DAX

```
SELECTCOLUMNS(Customer, "Country, State", [Country]&", "&[State])
```

Retourne :

Agrandir le tableau

Pays, État
IND, JK
IND, MH
IND, WB
USA, CA
USA, WA

Commentaires

Cette page a-t-elle été utile ? Yes No

SUBSTITUTEWITHINDEX

Article • 20/10/2023

Retourne une table qui représente une semi-jointure gauche des deux tables fournies comme arguments. La semi-jointure est effectuée à l'aide de colonnes communes, déterminées par les noms des colonnes communes et le type de données commun. Les colonnes qui sont jointes sont remplacées par une seule colonne dans la table retournée qui est de type entier et contient un index. L'index est une référence dans la table de jointure droite basé sur un ordre de tri spécifié.

Les colonnes de la table de droite/seconde fournie qui n'existent pas dans la table de gauche/première fournie ne sont pas incluses dans la table retournée et ne sont pas utilisées pour la jointure.

L'index commence à 0 (base 0) et est incrémenté de un pour chaque ligne supplémentaire dans la table de jointure de droite/seconde fournie. L'index est basé sur l'ordre de tri spécifié pour la table de jointure de droite/seconde.

Syntaxe

DAX

```
SUBSTITUTEWITHINDEX(<table>, <indexColumnName>, <indexColumnsTable>,
[<orderBy_expression>, [<order>][, <orderBy_expression>, [<order>]]…])
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table à filtrer en effectuant une semi-jointure gauche avec la table spécifiée comme troisième argument (indexColumnsTable). Il s'agit de la table sur le côté gauche de la semi-jointure gauche. Lla table retournée contient les mêmes colonnes que cette table, sauf que toutes les colonnes communes aux deux tables seront remplacées par une colonne d'index unique dans la table retournée.
indexColumnName	Chaîne qui spécifie le nom de la colonne d'index qui remplace toutes les colonnes communes dans les deux tables fournies comme arguments à cette fonction.

Terme	Définition
indexColumnsTable	Seconde table de la semi-jointure gauche. Il s'agit du tableau à droite de la semi-jointure gauche. Seules les valeurs présentes dans cette table seront retournées par la fonction. Aussi, les colonnes de cette table (basées sur les noms de colonne) seront remplacées par une colonne d'index unique dans la table retournée par cette fonction.
orderBy_expression	Toute expression DAX où la valeur de résultat est utilisée pour spécifier l'ordre de tri souhaité de la table indexColumnsTable afin de générer des valeurs d'index correctes. L'ordre de tri spécifié pour la table indexColumnsTable définit l'index de chaque ligne de la table et cet index est utilisé dans la table retournée pour représenter des combinaisons de valeurs dans indexColumnsTable, telles qu'elles apparaissent dans la table fournie comme premier argument de cette fonction.
order	(Facultatif) Valeur qui spécifie comment trier les valeurs orderBy_expression, par ordre croissant ou décroissant : Valeur : **Desc**. Valeur alternative : **0**(zéro)/**FALSE**. Trie par ordre décroissant les valeurs orderBy_expression. Il s'agit de la valeur par défaut quand le paramètre order est omis. Valeur : **ASC**. Valeur alternative : **1**/**TRUE**. Trie par ordre croissant les valeurs orderBy_expression.

Valeur renvoyée

Une table qui contient uniquement ces valeurs présentes dans la table indexColumnsTable et qui a une colonne d'index au lieu de toutes les colonnes présentes (par nom) dans la table indexColumnsTable.

Notes

- Cette fonction ne garantit pas l'ordre de tri des résultats.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Commentaires

Cette page a-t-elle été utile ?

Yes No

SUMMARIZE

Article • 20/10/2023

Retourne une table récapitulative des totaux demandés sur un ensemble de groupes.

Syntaxe

```
DAX

SUMMARIZE (<table>, <groupBy_columnName>[, <groupBy_columnName>]…[, <name>, <expression>]…)
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Toute expression DAX qui retourne une table de données.
groupBy_ColumnName	(Facultatif) Nom qualifié d'une colonne existante utilisé pour créer des groupes de synthèse en fonction des valeurs qui s'y trouvent. Ce paramètre ne peut pas être une expression.
name	Nom donné à une colonne de totaux ou de synthèse, placé entre guillemets doubles.
expression	Toute expression DAX qui retourne une valeur scalaire unique où l'expression doit être évaluée plusieurs fois (pour chaque ligne/contexte).

Valeur de retour

Table avec les colonnes sélectionnées pour les arguments *groupBy_columnName* et les colonnes synthétisées désignées par les arguments name.

Remarques

- Chaque colonne pour laquelle vous définissez un nom doit avoir une expression correspondante ; dans le cas contraire, une erreur est retournée. Le premier argument, name, définit le nom de la colonne dans les résultats. Le deuxième argument, expression, définit le calcul effectué pour obtenir la valeur de chaque ligne de cette colonne.
- groupBy_columnName doit être dans la *table* ou dans une table associée à la *table*.
- Chaque nom doit être placé entre guillemets doubles.
- La fonction regroupe un ensemble sélectionné de lignes en un ensemble de lignes de synthèse selon les valeurs d'une ou plusieurs colonnes groupBy_columnName. Une ligne est retournée pour chaque groupe.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant retourne un résumé des ventes du revendeur, regroupées autour de l'année civile et du nom de la catégorie de produits. Cette table de résultats vous permet d'effectuer une analyse sur les ventes du revendeur par année et par catégorie de produits.

```
DAX

SUMMARIZE(ResellerSales_USD
      , DateTime[CalendarYear]
      , ProductCategory[ProductCategoryName]
      , "Sales Amount (USD)", SUM(ResellerSales_USD[SalesAmount_USD])
```

```
      , "Discount Amount (USD)", SUM(ResellerSales_USD[DiscountAmount])
      )
```

La table suivante présente un aperçu des données telles qu'elles seraient reçues par toute fonction attendant de recevoir une table :

Agrandir le tableau

DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
2008	Bikes	12968255.42	36167.6592
2005	Bikes	6958251.043	4231.1621
2006	Bikes	18901351.08	178175.8399
2007	Bikes	24256817.5	276065.992
2008	Composants	2008052.706	39.9266
2005	Composants	574256.9865	0
2006	Composants	3428213.05	948.7674
2007	Composants	5195315.216	4226.0444
2008	Clothing	366507.844	4151.1235
2005	Clothing	31851.1628	90.9593
2006	Clothing	455730.9729	4233.039
2007	Clothing	815853.2868	12489.3835
2008	Accessories	153299.924	865.5945
2005	Accessories	18594.4782	4.293
2006	Accessories	86612.7463	1061.4872
2007	Accessories	275794.8403	4756.6546

Avec ROLLUP

L'ajout de la syntaxe ROLLUP modifie le comportement de la fonction SUMMARIZE en ajoutant des lignes de cumul au résultat sur les colonnes groupBy_columnName. ROLLUP s'utilise uniquement dans une expression SUMMARIZE.

Exemple

L'exemple suivant ajoute des lignes de cumul aux colonnes Group-By de l'appel de fonction SUMMARIZE :

DAX

```
SUMMARIZE(ResellerSales_USD
      , ROLLUP( DateTime[CalendarYear], ProductCategory[ProductCategoryName])
      , "Sales Amount (USD)", SUM(ResellerSales_USD[SalesAmount_USD])
      , "Discount Amount (USD)", SUM(ResellerSales_USD[DiscountAmount])
)
```

Retourne la table suivante

Agrandir le tableau

DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
2008	Bikes	12968255.42	36167.6592
2005	Bikes	6958251.043	4231.1621

DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
2006	Bikes	18901351.08	178175.8399
2007	Bikes	24256817.5	276065.992
2008	Composants	2008052.706	39.9266
2005	Composants	574256.9865	0
2006	Composants	3428213.05	948.7674
2007	Composants	5195315.216	4226.0444
2008	Clothing	366507.844	4151.1235
2005	Clothing	31851.1628	90.9593
2006	Clothing	455730.9729	4233.039
2007	Clothing	815853.2868	12489.3835
2008	Accessories	153299.924	865.5945
2005	Accessories	18594.4782	4.293
2006	Accessories	86612.7463	1061.4872
2007	Accessories	275794.8403	4756.6546
2008		15496115.89	41224.3038
2005		7582953.67	4326.4144
2006		22871907.85	184419.1335
2007		30543780.84	297538.0745
		76494758.25	527507.9262

Avec ROLLUPGROUP

Toutefois, l'ajout de ROLLUPGROUP dans une syntaxe ROLLUP peut être utilisé pour empêcher les sous-totaux partiels dans les lignes de cumul. ROLLUPGROUP s'utilise uniquement dans une expression ROLLUP, ROLLUPADDISSUBTOTAL ou ROLLUPISSUBTOTAL.

Exemple

L'exemple suivant affiche uniquement le total général de toutes les années et de toutes les catégories sans le sous-total de chaque année avec toutes les catégories :

DAX

```
SUMMARIZE(ResellerSales_USD
      , ROLLUP(ROLLUPGROUP( DateTime[CalendarYear], ProductCategory[ProductCategoryName]))
      , "Sales Amount (USD)", SUM(ResellerSales_USD[SalesAmount_USD])
      , "Discount Amount (USD)", SUM(ResellerSales_USD[DiscountAmount])
)
```

Retourne la table suivante

Agrandir le tableau

DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
2008	Bikes	12968255.42	36167.6592
2005	Bikes	6958251.043	4231.1621

DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
2006	Bikes	18901351.08	178175.8399
2007	Bikes	24256817.5	276065.992
2008	Composants	2008052.706	39.9266
2005	Composants	574256.9865	0
2006	Composants	3428213.05	948.7674
2007	Composants	5195315.216	4226.0444
2008	Clothing	366507.844	4151.1235
2005	Clothing	31851.1628	90.9593
2006	Clothing	455730.9729	4233.039
2007	Clothing	815853.2868	12489.3835
2008	Accessories	153299.924	865.5945
2005	Accessories	18594.4782	4.293
2006	Accessories	86612.7463	1061.4872
2007	Accessories	275794.8403	4756.6546
		76494758.25	527507.9262

Avec ISSUBTOTAL

ISSUBTOTAL vous permet de créer une autre colonne dans l'expression SUMMARIZE qui retourne True si la ligne contient des valeurs de sous-total pour la colonne fournie comme argument de ISSUBTOTAL ; sinon, retourne false. ISSUBTOTAL s'utilise uniquement dans une expression SUMMARIZE.

Exemple

L'exemple suivant génère une colonne ISSUBTOTAL pour chacune des colonnes ROLLUP dans l'appel de fonction SUMMARIZE donné :

DAX

```
SUMMARIZE(ResellerSales_USD
      , ROLLUP( DateTime[CalendarYear], ProductCategory[ProductCategoryName])
      , "Sales Amount (USD)", SUM(ResellerSales_USD[SalesAmount_USD])
      , "Discount Amount (USD)", SUM(ResellerSales_USD[DiscountAmount])
      , "Is Sub Total for DateTimeCalendarYear", ISSUBTOTAL(DateTime[CalendarYear])
      , "Is Sub Total for ProductCategoryName", ISSUBTOTAL(ProductCategory[ProductCategoryName])
)
```

Retourne la table suivante

Agrandir le tableau

[Is Sub Total for DateTimeCalendarYear]	[Is Sub Total for ProductCategoryName]	DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
FALSE	FAUX				
FAUX	FALSE	2008	Bikes	12968255.42	36167.6592
FALSE	FALSE	2005	Bikes	6958251.043	4231.1621
FAUX	FALSE	2006	Bikes	18901351.08	178175.8399

[Is Sub Total for DateTimeCalendarYear]	[Is Sub Total for ProductCategoryName]	DateTime[CalendarYear]	ProductCategory[ProductCategoryName]	[Sales Amount (USD)]	[Discount Amount (USD)]
FALSE	FALSE	2007	Bikes	24256817.5	276065.992
FALSE	FALSE	2008	Composants	2008052.706	39.9266
FALSE	FALSE	2005	Composants	574256.9865	0
FALSE	FALSE	2006	Composants	3428213.05	948.7674
FALSE	FALSE	2007	Composants	5195315.216	4226.0444
FALSE	FALSE	2008	Clothing	366507.844	4151.1235
FALSE	FALSE	2005	Clothing	31851.1628	90.9593
FALSE	FALSE	2006	Clothing	455730.9729	4233.039
FALSE	FALSE	2007	Clothing	815853.2868	12489.3835
FALSE	FALSE	2008	Accessories	153299.924	865.5945
FALSE	FALSE	2005	Accessories	18594.4782	4.293
FALSE	FALSE	2006	Accessories	86612.7463	1061.4872
FALSE	FALSE	2007	Accessories	275794.8403	4756.6546
false	VRAI				
FAUX	true	2008		15496115.89	41224.3038
false	true	2005		7582953.67	4326.4144
false	true	2006		22871907.85	184419.1335
false	true	2007		30543780.84	297538.0745
VRAI	TRUE			76494758.25	527507.9262

Contenu connexe

SUMMARIZECOLUMNS

Commentaires

Cette page a-t-elle été utile ? Yes No

SUMMARIZECOLUMNS

Article • 20/10/2023

Retourne une table récapitulative sur un ensemble de groupes.

Syntaxe

DAX

```
SUMMARIZECOLUMNS( <groupBy_columnName> [, < groupBy_columnName >]…,
[<filterTable>]…[, <name>, <expression>]…)
```

Paramètres

Agrandir le tableau

Terme	Définition
groupBy_columnName	Référence de colonne complète (Table[Column]) à une table de base pour laquelle les valeurs distinctes sont incluses dans la table retournée. Chaque colonne groupBy_columnName est soumise à une jointure croisée (tables différentes) ou existe automatiquement (même table) avec les colonnes spécifiées suivantes.
filterTable	Expression de table qui est ajoutée au contexte de filtre de toutes les colonnes spécifiées en tant qu'arguments groupBy_columnName. Les valeurs présentes dans la table de filtres sont utilisées pour filtrer avant que la jointure croisée/l'existence automatique ne soit effectuée.
name	Chaîne représentant le nom de colonne à utiliser pour l'expression suivante spécifiée.
expression	Toute expression DAX qui retourne une seule valeur (et non une table).

Valeur de retour

Table qui contient des combinaisons de valeurs des colonnes fournies en fonction du regroupement spécifié. Seules les lignes pour lesquelles au moins une des expressions fournies retourne une valeur non vide sont incluses dans la table retournée. Si toutes les expressions ont la valeur BLANK/NULL pour une ligne, cette ligne n'est pas incluse dans la table retournée.

Remarques

- Cette fonction ne garantit aucun ordre de tri des résultats.
- Une colonne ne peut pas être spécifiée plusieurs fois dans le paramètre groupBy_columnName. Par exemple, la formule suivante n'est pas valide.

  `SUMMARIZECOLUMNS( Sales[StoreId], Sales[StoreId] )`

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Contexte de filtre

Considérez la requête suivante :

DAX

```
SUMMARIZECOLUMNS (
    'Sales Territory'[Category],
    FILTER('Customer', 'Customer' [First Name] = "Alicia")
)
```

Dans cette requête, sans mesure, les colonnes groupBy ne contiennent aucune colonne de l'expression FILTER (par exemple, la table Customer). Le filtre n'est pas appliqué aux colonnes groupBy. Les tables Sales Territory et Customer peuvent être indirectement liées par le biais de la table de faits Reseller Sales. Étant donné qu'elles ne sont pas directement liées, l'expression de filtre ne présente aucune opération et les colonnes groupBy ne sont pas affectées.

Toutefois, avec cette requête :

DAX

```
SUMMARIZECOLUMNS (
    'Sales Territory'[Category], 'Customer' [Education],
    FILTER('Customer', 'Customer'[First Name] = "Alicia")
)
```

Les colonnes groupBy contiennent une colonne qui est affectée par le filtre et ce filtre est appliqué aux résultats des colonnes groupBy.

Avec IGNORE

La syntaxe IGNORE peut être utilisée pour modifier le comportement de la fonction SUMMARIZECOLUMNS en omettant des expressions spécifiques de l'évaluation BLANK/NULL. Les lignes pour lesquelles toutes les expressions qui n'utilisent pas IGNORE retournent BLANK/NULL sont exclues, indépendamment du fait que les expressions qui utilisent IGNORE sont ou non évaluées à la valeur BLANK/NULL. IGNORE peut uniquement être utilisé dans une expression SUMMARIZECOLUMNS.

Exemple

DAX

```
SUMMARIZECOLUMNS(
    Sales[CustomerId], "Total Qty",
    IGNORE( SUM( Sales[Qty] ) ),
    "BlankIfTotalQtyIsNot3", IF( SUM( Sales[Qty] )=3, 3 )
)
```

Cela cumule la colonne Sales[CustomerId], ce qui crée un sous-total pour tous les clients du regroupement donné. Sans IGNORE, le résultat est le suivant :

Agrandir le tableau

CustomerId	Total Qty	BlankIfTotalQtyIsNot3
A	5	
B	3	3
C	3	3

Avec IGNORE,

Agrandir le tableau

CustomerId	Total Qty	BlankIfTotalQtyIsNot3
B	3	3
C	3	3

Toutes les expressions ignorées,

DAX

```
SUMMARIZECOLUMNS(
    Sales[CustomerId], "Blank",
    IGNORE( BLANK() ), "BlankIfTotalQtyIsNot5",
    IGNORE( IF( SUM( Sales[Qty] )=5, 5 ) )
)
```

Même si les deux expressions retournent des valeurs vides pour certaines lignes, elles sont incluses, car il n'y a pas d'expressions non ignorées qui retournent des valeurs vides.

Agrandir le tableau

CustomerId	Vide	BlankIfTotalQtyIsNot5
A		5
B		
C		

Avec NONVISUAL

La fonction NONVISUAL marque un filtre de valeur dans la fonction SUMMARIZECOLUMNS comme n'affectant pas les valeurs de mesure, mais uniquement comme s'appliquant à des colonnes groupBy. NONVISUAL peut uniquement être utilisé dans une expression SUMMARIZECOLUMNS.

Exemple

DAX

```
DEFINE
MEASURE FactInternetSales[Sales] = SUM(FactInternetSales[Sales Amount])
EVALUATE
SUMMARIZECOLUMNS
(
    DimDate[CalendarYear],
    NONVISUAL(TREATAS({2007, 2008}, DimDate[CalendarYear])),
    "Sales", [Sales],
    "Visual Total Sales", CALCULATE([Sales],
ALLSELECTED(DimDate[CalendarYear]))
)
ORDER BY [CalendarYear]
```

Retourne le résultat où [Visual Total Sales] correspond au total sur toutes les années :

Agrandir le tableau

DimDate[CalendarYear]	[Sales]	[Visual Total Sales]
2007	9 791 060,30	29 358 677,22
2008	9 770 899,74	29 358 677,22

En revanche, la même requête *sans* la fonction NONVISUAL :

DAX

```
DEFINE
MEASURE FactInternetSales[Sales] = SUM(FactInternetSales[Sales Amount])
EVALUATE
SUMMARIZECOLUMNS
(
    DimDate[CalendarYear],
    TREATAS({2007, 2008}, DimDate[CalendarYear]),
    "Sales", [Sales],
    "Visual Total Sales", CALCULATE([Sales],
ALLSELECTED(DimDate[CalendarYear]))
)
ORDER BY [CalendarYear]
```

Retourne le résultat où [Visual Total Sales] correspond au total sur les deux années sélectionnées :

Agrandir le tableau

DimDate[CalendarYear]	[Sales]	[Visual Total Sales]
2007	9 791 060,30	19 561 960,04
2008	9 770 899,74	19 561 960,04

Avec ROLLUPADDISSUBTOTAL

L'ajout de la syntaxe ROLLUPADDISSUBTOTAL modifie le comportement de la fonction SUMMARIZECOLUMNS en ajoutant des lignes de cumul/sous-total au résultat basé sur les colonnes groupBy_columnName. ROLLUPADDISSUBTOTAL peut uniquement être utilisé dans une expression SUMMARIZECOLUMNS.

Exemple avec un sous-total unique

DAX

```
DEFINE
VAR vCategoryFilter =
  TREATAS({"Accessories", "Clothing"}, Product[Category])
VAR vSubcategoryFilter =
  TREATAS({"Bike Racks", "Mountain Bikes"}, Product[Subcategory])
EVALUATE
  SUMMARIZECOLUMNS
  (
    ROLLUPADDISSUBTOTAL
    (
      Product[Category], "IsCategorySubtotal", vCategoryFilter,
      Product[Subcategory], "IsSubcategorySubtotal", vSubcategoryFilter
    ),
    "Total Qty", SUM(Sales[Qty])
  )
  ORDER BY
  [IsCategorySubtotal] DESC, [Category],
  [IsSubcategorySubtotal] DESC, [Subcategory]
```

Retourne la table suivante :

Agrandir le tableau

Category	Sous-catégorie	IsCategorySubtotal	IsSubcategorySubtotal	Total Qty
		True	True	60398
Accessories		False	True	36092
Accessories	Porte-vélos	False	False	328
Bikes	VTT	False	False	4970
Clothing		False	True	9101

Exemple avec plusieurs sous-totaux

DAX

```
SUMMARIZECOLUMNS (
    Regions[State], ROLLUPADDISSUBTOTAL ( Sales[CustomerId],
"IsCustomerSubtotal" ),
    ROLLUPADDISSUBTOTAL ( Sales[Date], "IsDateSubtotal"), "Total Qty", SUM(
Sales[Qty] )
)
```

Les ventes sont regroupées par région, par client, par date, avec des sous-totaux pour 1. Ventes par région, par date 2. Ventes par région, par client 3. Regroupement sur le client et la date qui aboutit aux ventes par région.

Retourne la table suivante :

Agrandir le tableau

CustomerID	IsCustomerSubtotal	State	Total Qty	Date	IsDateSubtotal
A	FAUX	WA	5	7/10/2014	
B	FAUX	WA	1	7/10/2014	
B	FAUX	WA	2	7/11/2014	
C	FAUX	OR	2	7/10/2014	
C	FAUX	OR	1	7/11/2014	
	TRUE	WA	6	7/10/2014	
	TRUE	WA	2	7/11/2014	
	TRUE	OR	2	7/10/2014	
	TRUE	OR	1	7/11/2014	
A	FAUX	WA	5		TRUE
B	FAUX	WA	3		TRUE
C	FAUX	OR	3		VRAI
	VRAI	WA	8		VRAI
	VRAI	OR	3		TRUE

Avec ROLLUPGROUP

Comme avec la fonction SUMMARIZE, la fonction ROLLUPGROUP peut être utilisée avec ROLLUPADDISSUBTOTAL pour spécifier les groupes/précisions de synthèse (sous-totaux) à inclure, et réduire ainsi le nombre de lignes de sous-total retournées. ROLLUPGROUP peut uniquement être utilisé dans une expression SUMMARIZECOLUMNS ou SUMMARIZE.

Exemple avec plusieurs sous-totaux

DAX

```
SUMMARIZECOLUMNS(
    ROLLUPADDISSUBTOTAL( Sales[CustomerId], "IsCustomerSubtotal" ),
    ROLLUPADDISSUBTOTAL(ROLLUPGROUP(Regions[City], Regions[State]),
"IsCityStateSubtotal"),"Total Qty", SUM( Sales[Qty] )
)
```

Toujours avec un regroupement par ville et région, mais calculés ensemble pour le sous-total ; retourne la table suivante

Agrandir le tableau

State	CustomerId	IsCustomerSubtotal	Total Qty	City	IsCityStateSubtotal
WA	A	FAUX	2	Bellevue	FALSE
WA	B	FAUX	2	Bellevue	FALSE
WA	A	FAUX	3	Redmond	FALSE
WA	B	false	1	Redmond	FALSE
OR	C	FAUX	3	Portland	FALSE
WA		TRUE	4	Bellevue	FALSE
WA		TRUE	4	Redmond	FALSE
OR		TRUE	3	Portland	FALSE
	A	false	5		FAUX
	B	FAUX	3		TRUE
	C	FAUX	3		VRAI
		TRUE	11		VRAI

Contenu connexe

SUMMARIZE

Commentaires

Cette page a-t-elle été utile ?

Yes No

Constructeur de table

Article • 20/10/2023

Retourne une table d'une ou plusieurs colonnes.

Syntaxe

DAX

```
{ <scalarExpr1>, <scalarExpr2>, … }
{ ( <scalarExpr1>, <scalarExpr2>, … ), ( <scalarExpr1>, <scalarExpr2>, … ),
… }
```

Paramètres

Agrandir le tableau

Terme	Définition
scalarExprN	Toute expression DAX qui retourne une valeur scalaire.

Valeur de retour

Table d'une ou plusieurs colonnes. Lorsqu'il n'y a qu'une seule colonne, le nom de la colonne est Value. Quand il y a N colonnes où N > 1, les noms des colonnes de gauche à droite sont Value1, Value2, ..., ValueN.

Remarques

- La première syntaxe retourne une table d'une seule colonne. La seconde syntaxe retourne une table d'une ou plusieurs colonnes.
- Le nombre d'expressions scalaires doit être le même pour toutes les lignes.
- Lorsque les types de données des valeurs d'une colonne sont différents dans des lignes différentes, toutes les valeurs sont converties en un type de données commun.

Exemple 1

Les requêtes DAX suivantes :

```
DAX

EVALUATE { 1, 2, 3 }
```

et

```
DAX

EVALUATE { (1), (2), (3) }
```

Retournent la table suivante d'une seule colonne :

Agrandir le tableau

[Value]
1
2
3

Exemple 2

La requête DAX suivante :

```
DAX

EVALUATE
    {
        (1.5, DATE(2017, 1, 1), CURRENCY(199.99), "A"),
        (2.5, DATE(2017, 1, 2), CURRENCY(249.99), "B"),
        (3.5, DATE(2017, 1, 3), CURRENCY(299.99), "C")
    }
```

Retourne :

Agrandir le tableau

[Value1]	[Value2]	[Value3]	[Value4]
1.5	1/1/2017	199.99	A

[Value1]	[Value2]	[Value3]	[Value4]
2.5	2/1/2017	249.99	B
3,5	1/3/2017	299.99	C

Exemple 3

La requête DAX suivante :

```
DAX
EVALUATE { 1, DATE(2017, 1, 1), TRUE, "A" }
```

Retourne la table suivante d'une seule colonne de type de données String :

Agrandir le tableau

[Value]
1
1/1/2017
VRAI
A

Commentaires

Cette page a-t-elle été utile ? Yes No

TOPN

Article • 20/10/2023

Retourne les N premières lignes de la table spécifiée.

Syntaxe

DAX

```
TOPN(<N_Value>, <Table>, <OrderBy_Expression>, [<Order>[,
<OrderBy_Expression>, [<Order>]]…])
```

Paramètres

Agrandir le tableau

Paramètre	Définition
N_Value	Nombre de lignes à retourner. Toute expression DAX qui retourne une valeur scalaire où l'expression doit être évaluée plusieurs fois (pour chaque ligne/contexte). Consultez Remarques pour mieux comprendre quand le nombre de lignes retournées pourrait être supérieur à *n_value*.
Table de charge de travail	Toute expression DAX qui retourne une table de données à partir de laquelle extraire les « n » premières lignes. Consultez Remarques pour mieux comprendre quand une table vide est retournée.
OrderBy_Expression	(Facultatif) Toute expression DAX où la valeur obtenue est utilisée pour trier la table et évaluée pour chaque ligne de la *table*.
JSON	(Facultatif) Une valeur qui spécifie comment trier les valeurs *OrderBy_Expression* : - **0** (zéro) ou **FALSE**. Trie dans l'ordre décroissant des valeurs de *Commande*. Valeur par défaut lorsque le paramètre *Commande* est omis. - **1** ou **TRUE**. Rangs dans l'ordre croissant de *OrderBy*.

Valeur retournée

Une table avec les N premières lignes de *Table* ou une table vide si *N_Value* est 0 (zéro) ou moins. Les lignes ne sont pas triées dans un ordre particulier.

Notes

- S'il y a une égalité, dans les valeurs *Order_By*, à la N-ième ligne de la table, toutes les lignes liées sont renvoyées. Ensuite, lorsqu'il y a des liens à la N-ième ligne, la fonction peut retourner plus de n lignes.
- Si N_Value est 0 (zéro) ou moins, TOPN renvoie une table vide.
- TOPN ne garantit aucun ordre de tri des résultats.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La formule de mesure suivante renvoie les 10 produits les plus vendus par montant des ventes.

DAX

```
= SUMX(
        TOPN(
            10,
            SUMMARIZE(
                    InternetSales,
                    InternetSales[ProductKey],
                    "TotalSales", SUM(InternetSales[SalesAmount])
            ),
            [TotalSales], DESC
        ),
        [TotalSales]
)
```

Commentaires

Cette page a-t-elle été utile ? Yes No

TREATAS

Article • 14/12/2023

Applique le résultat d'une expression de table comme filtres aux colonnes d'une table non associée.

Syntaxe

```
DAX
TREATAS(table_expression, <column>[, <column>[, <column>[,…]]]} )
```

Paramètres

Agrandir le tableau

Terme	Définition
table_expression	Expression produisant une table.
colonne	Une ou plusieurs colonnes existantes. Il ne peut pas s'agir d'une expression.

Valeur de retour

Table contenant toutes les lignes de la ou des colonnes qui se trouvent également dans table_expression.

Remarques

- Le nombre de colonnes spécifié doit correspondre au nombre de colonnes dans l'expression de table. Les colonnes doivent aussi être dans le même ordre.
- Si une valeur retournée dans l'expression de table n'existe pas dans la colonne, elle est ignorée. Par exemple, TREATAS({"Red", "Green", "Yellow"}, DimProduct[Color]) définit un filtre sur la colonne DimProduct[Color] avec trois valeurs : "Red", "Green" et "Yellow". Si "Yellow" n'existe pas dans DimProduct[Color], les valeurs de filtre effectives sont "Red" et "Green".
- Cette fonction est idéale quand il n'existe pas de relation entre les tables. Si vous avez plusieurs relations entre les tables impliquées, utilisez USERELATIONSHIP à la

place.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Dans l'exemple suivant, le modèle contient deux tables de produits non liées. Si un utilisateur applique un filtre à DimProduct1[ProductCategory] en sélectionnant Bikes, Seats et Tires, le même filtre (Bikes, Seats et Tires) est appliqué à DimProduct2[ProductCategory].

DAX

```
CALCULATE(
SUM(Sales[Amount]),
TREATAS(VALUES(DimProduct1[ProductCategory]), DimProduct2[ProductCategory])
)
```

Contenu connexe

INTERSECT
FILTER
USERELATIONSHIP

Commentaires

Cette page a-t-elle été utile ? Yes No

UNION

Article • 20/10/2023

Crée une table d'union (jointure) à partir d'une paire de tables.

Syntaxe

DAX

```
UNION(<table_expression1>, <table_expression2> [,<table_expression>]…)
```

Paramètres

Agrandir le tableau

Terme	Définition
table_expression	Toute expression DAX qui retourne une table.

Valeur de retour

Table qui contient toutes les lignes de chacune des deux expressions de table.

Remarques

- Les deux tables doivent avoir le même nombre de colonnes.
- Les colonnes sont combinées par position dans leur table respective.
- Les noms de colonnes de la table retournée correspondent aux noms de colonnes de table_expression1.
- Les lignes en double sont conservées.
- La table retournée présente une traçabilité, le cas échéant. Par exemple, si la première colonne de chaque table_expression présente une traçabilité vers la même colonne de base C1 du modèle, la première colonne du résultat de la fonction UNION présentera une traçabilité vers C1. Cependant, si les colonnes combinées présentent une traçabilité vers des colonnes de base différentes, ou s'il

existe une colonne d'extension, la colonne obtenue dans UNION ne présentera pas de traçabilité.

- Quand les types de données diffèrent, le type de données obtenu est déterminé par les règles de forçage de type de données.
- La table retournée ne contient pas de colonnes des tables associées.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'expression suivante crée une union en combinant la table USAInventory et la table INDInventory dans une table unique :

```
DAX

UNION(UsaInventory, IndInventory)
```

USAInventory

Agrandir le tableau

Pays/région	State	Count	Total
USA	CA	5	500
États-Unis	WA	10	900

INDInventory

Agrandir le tableau

Pays/région	State	Count	Total
IND	JK	20	800
IND	MH	25	1 000
IND	WB	10	900

Table retournée

Agrandir le tableau

Pays/région	State	Count	Total
USA	CA	5	500
États-Unis	WA	10	900
IND	JK	20	800
IND	MH	25	1 000
IND	WB	10	900

Commentaires

Cette page a-t-elle été utile ? Yes No

VALUES

Article • 14/12/2023

Quand le paramètre d'entrée est un nom de colonne, retourne une table à une colonne qui contient les valeurs distinctes de la colonne spécifiée. Les valeurs dupliquées sont supprimées et seules les valeurs uniques sont retournées. Une valeur vide peut être ajoutée. Quand le paramètre d'entrée est un nom de table, retourne les lignes de la table spécifiée. Les lignes en double sont conservées. Une ligne vide peut être ajoutée.

> **Notes**
>
> Cette fonction ne peut pas être utilisée pour retourner des valeurs dans une cellule ou une colonne d'une feuille de calcul. Au lieu de cela, vous l'utilisez comme fonction intermédiaire, imbriquée dans une formule, pour obtenir une liste de valeurs distinctes qui peuvent être comptées ou utilisées pour filtrer ou additionner d'autres valeurs.

Syntaxe

DAX

```
VALUES(<TableNameOrColumnName>)
```

Paramètres

Agrandir le tableau

Terme	Définition
TableName ou ColumnName	Colonne à partir de laquelle des valeurs uniques doivent être retournées, ou table à partir de laquelle des lignes doivent être retournées.

Valeur de retour

Quand le paramètre d'entrée est un nom de colonne, une table à une seule colonne est retournée. Quand le paramètre d'entrée est un nom de table, une table des mêmes colonnes est retournée.

Remarques

- Quand vous utilisez la fonction VALUES dans un contexte qui a été filtré, les valeurs uniques retournées par VALUES sont affectées par le filtre. Par exemple, si vous filtrez par région et que vous retournez une liste de valeurs pour la ville, la liste inclut uniquement les villes dans les régions autorisées par le filtre. Pour retourner toutes les villes, quels que soient les filtres existants, vous devez utiliser la fonction ALL pour supprimer les filtres de la table. Le deuxième exemple illustre l'utilisation de la fonction ALL avec VALUES.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

- Pour connaître les meilleures pratiques d'utilisation de la fonction VALUES, voir Utiliser SELECTEDVALUE au lieu de VALUES.

Fonctions connexes

Dans la plupart des scénarios, lorsque l'argument est un nom de colonne, les résultats de la fonction VALUES sont identiques à ceux de la fonction **DISTINCT**. Les deux fonctions suppriment les doublons et retournent une liste des valeurs possibles dans la colonne spécifiée. Toutefois, la fonction VALUES peut également retourner une valeur vide. Cette valeur vide est utile dans les cas où vous recherchez des valeurs distinctes dans une table associée, mais qu'une valeur utilisée dans la relation est manquante dans une table. Dans la terminologie de base de données, il s'agit d'une violation de l'intégrité référentielle. De telles incompatibilités dans les données peuvent se produire quand une table est mise à jour et que la table associée ne l'est pas.

Lorsque l'argument est un nom de table, le résultat de la fonction VALUES retourne toutes les lignes de la table spécifiée plus une ligne vide, en cas de violation de l'intégrité référentielle. La fonction DISTINCT supprime les lignes en double et retourne des lignes uniques dans la table spécifiée.

> ⓘ **Notes**
>
> La fonction DISTINCT accepte comme argument un nom de colonne ou une expression de table valide, mais la fonction VALUES accepte uniquement un nom de colonne ou un nom de table comme argument.

Le tableau suivant résume l'incompatibilité entre les données qui peut se produire dans deux tables associées lorsque l'intégrité référentielle n'est pas conservée.

Agrandir le tableau

Table MyOrders	Table MySales
1er juin	Ventes du 1er juin
2 juin	Ventes du 2 juin
(aucune date de commande n'a été entrée)	Ventes du 3 juin

Si vous utilisez la fonction DISTINCT pour retourner une liste de dates, seules deux dates sont retournées. Toutefois, si vous utilisez la fonction VALUES, la fonction retourne les deux dates et un membre vide supplémentaire. En outre, toute ligne de la table MySales qui n'a pas de date correspondante dans la table MyOrders sera « mise en correspondance » avec ce membre inconnu.

Exemple

La formule suivante compte le nombre de factures uniques (commandes client) et produit les résultats suivants quand elles sont utilisées dans un rapport qui contient les noms des catégories de produits :

```
DAX
= COUNTROWS(VALUES('InternetSales_USD'[SalesOrderNumber]))
```

retourne :

Agrandir le tableau

Étiquettes de ligne	Nombre de factures
Accessories	18 208
Bikes	15 205
Clothing	7 461
Total général	27 659

Contenu connexe

FILTER, fonction
COUNTROWS, fonction
Fonctions de filtrage

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonctions de texte

Article • 20/10/2023

DAX (Data Analysis Expressions) comprend un ensemble de fonctions de texte basées sur la bibliothèque de fonctions de chaîne dans Excel, mais qui ont été modifiées pour fonctionner avec des tables et des colonnes dans les modèles tabulaires. Cette section décrit les fonctions de texte disponibles dans le langage DAX.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
COMBINEVALUES	joint deux chaînes de texte ou plus en une seule.
CONCATENATE	Concatène deux chaînes de texte.
CONCATENATEX	Concatène le résultat d'une expression évaluée pour chaque ligne d'une table.
EXACT	Compare deux chaînes de texte et retourne TRUE si elles sont exactement identiques ; sinon, FALSE.
FIND	Retourne la position de départ d'une chaîne de texte à l'intérieur d'une autre chaîne de texte.
FIXED	Arrondit un nombre au nombre spécifié de décimales et retourne le résultat sous forme de texte.
FORMAT	Convertit une valeur en texte au format spécifié.
LEFT	Retourne le nombre spécifié de caractères à partir du début d'une chaîne de texte.
LEN	Retourne le nombre de caractères dans une chaîne de texte.
LOWER	Convertit toutes les lettres d'une chaîne de texte en minuscules.
MID	Retourne une chaîne de caractères à partir du milieu d'une chaîne de texte, en fonction d'une position de départ et d'une longueur.
REPLACE	REPLACE remplace une partie d'une chaîne de texte, en fonction du nombre de caractères que vous spécifiez, par une autre chaîne de texte.
REPT	Répète du texte un certain nombre de fois.
RIGHT	RIGHT retourne le ou les derniers caractères d'une chaîne de texte, en

Fonction	Description
	fonction du nombre de caractères que vous spécifiez.
SEARCH	Retourne le numéro du caractère où apparaît pour la première fois un caractère ou une chaîne de texte spécifique selon une lecture de gauche à droite.
SUBSTITUTE	Remplace un texte existant par un nouveau texte dans une chaîne de texte.
TRIM	Supprime tous les espaces d'un texte à l'exception des espaces uniques entre les mots.
UNICHAR	Retourne le caractère Unicode référencé par la valeur numérique.
UNICODE	Retourne le code numérique correspondant au premier caractère de la chaîne de texte.
UPPER	Convertit toutes les lettres d'une chaîne de texte en majuscules.
VALUE	Convertit en nombre une chaîne de texte qui représente un nombre.

Commentaires

Cette page a-t-elle été utile ? Yes No

COMBINEVALUES

Article • 20/10/2023

joint deux chaînes de texte ou plus en une seule. L'objectif premier de cette fonction est de prendre en charge les relations multicolonnes dans les modèles DirectQuery. Pour plus d'informations, consultez **Remarques**.

Syntaxe

DAX

```
COMBINEVALUES(<delimiter>, <expression>, <expression>[, <expression>]…)
```

Paramètres

Agrandir le tableau

Terme	Définition
delimiter	Séparateur à utiliser pendant la concaténation. Doit être une valeur constante.
expression	Expression DAX dont la valeur sera jointe en une chaîne de texte unique.

Valeur de retour

Une chaîne concaténée.

Remarques

- La fonction COMBINEVALUES suppose, mais sans confirmer, que lorsque les valeurs d'entrée sont différentes, les chaînes de sortie sont également différentes. Selon cette hypothèse, quand la fonction COMBINEVALUES est utilisée pour créer des colonnes calculées afin de produire une relation qui joint plusieurs colonnes de deux tables DirectQuery, une condition de jointure optimisée est générée au moment de la requête. Par exemple, si les utilisateurs veulent créer une relation entre Table1(Column1, Column2) et Table2(Column1, Column2), ils peuvent créer deux colonnes calculées, une sur chaque table, comme suit :

  DAX

```
Table1[CalcColumn] = COMBINEVALUES(",", Table1[Column1],
Table1[Column2])
```

et

DAX

```
Table2[CalcColumn] = COMBINEVALUES(",", Table2[Column1],
Table2[Column2])
```

Ils peuvent ensuite créer une relation entre `Table1[CalcColumn]` et `Table2[CalcColumn]`. Contrairement aux autres fonctions et opérateurs DAX, qui sont traduits littéralement en opérateurs et fonctions SQL correspondants, la relation ci-dessus génère un prédicat de jointure SQL comme suit :

DAX

```
(Table1.Column1 = Table2.Column1 OR Table1.Column1 IS NULL AND
Table2.Column1 IS NULL)
```

et

DAX

```
(Table1.Column2 = Table2.Column2 OR Table1.Column2 IS NULL AND
Table2.Column2 IS NULL)
```

- Le prédicat de jointure peut éventuellement fournir des performances de requêtes bien supérieures à celles qui impliquent des fonctions et des opérateurs SQL complexes.
- La fonction COMBINEVALUES s'appuie sur les utilisateurs pour choisir le délimiteur approprié afin de garantir que les combinaisons uniques de valeurs d'entrée produisent des chaînes de sortie distinctes, mais elle ne confirme pas que l'hypothèse est exacte. Par exemple, si les utilisateurs choisissent `"| "` comme délimiteur, mais qu'une ligne dans Table1 contient `Table1[Column1] = "| "` et `Table2 [Column2] = " "`, alors qu'une ligne de Table2 contient `Table2[Column1] = " "` et `Table2[Column2] = "| "`, les deux sorties concaténées sont les mêmes `"|| "`, ce qui semble indiquer que les deux lignes constituent une correspondance dans l'opération de jointure. Les deux lignes ne sont pas jointes si les deux tables proviennent de la même source DirectQuery, bien qu'elles soient regroupées si les deux tables sont importées.

Exemple

La requête DAX suivante :

DAX

```
EVALUATE
DISTINCT (
    SELECTCOLUMNS ( Date, "Month", COMBINEVALUES ( ", ", [MonthName],
[CalendarYear] ) )
)
```

Retourne la table à une seule colonne suivante :

Agrandir le tableau

[Mois]
Janvier, 2020
Février, 2020
Mars, 2020
Avril, 2020
Mai 2020
Juin 2020
Juillet 2020
Août 2020
Septembre 2020
Octobre 2020
Novembre 2020
Décembre, 2020
Janvier, 2021
Janvier, 2021
Février, 2021
Mars 2021
Avril 2021

[Mois]
Mai, 2021
Juin 2021
Juillet 2021
Août 2021
Septembre 2021
Octobre 2021
Novembre 2021
Décembre 2021

Commentaires

Cette page a-t-elle été utile ? Yes No

CONCATENATE

Article • 22/01/2024

Concatène deux chaînes de texte.

Syntaxe

DAX

```
CONCATENATE(<text1>, <text2>)
```

Paramètres

Agrandir le tableau

Terme	Définition
text1	Première chaîne de texte à joindre en une chaîne de texte unique. La chaîne peut inclure du texte ou des nombres. Vous pouvez également utiliser des références de colonnes.
text2	Deuxième chaîne de texte à joindre en une seule chaîne de texte. La chaîne peut inclure du texte ou des nombres. Vous pouvez également utiliser des références de colonne.

Valeur de retour

Une chaîne concaténée.

Remarques

- La fonction CONCATENATE concatène deux chaînes de texte. Les éléments concaténés peuvent être du texte, des nombres, des valeurs booléennes représentées sous forme de texte ou une combinaison de ces éléments. Vous pouvez également utiliser une référence de colonne si la colonne contient les valeurs appropriées.

- Dans DAX, la fonction CONCATENATE accepte deux arguments uniquement, alors qu'elle en accepte jusqu'à 255 dans Excel. Si vous devez concaténer plusieurs colonnes, vous pouvez créer une série de calculs ou utiliser l'opérateur de concaténation (**&**) pour toutes les joindre dans une expression plus simple.

- Si vous souhaitez utiliser des chaînes de texte directement (plutôt qu'une référence de colonne), vous devez placer chaque chaîne entre guillemets doubles.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple : concaténation de littéraux

L'exemple de formule crée une valeur de chaîne en combinant deux valeurs de chaîne que vous spécifiez comme arguments.

DAX

```
= CONCATENATE("Hello ", "World")
```

Exemple : concaténation de chaînes dans des colonnes

L'exemple de formule retourne le nom complet du client tel qu'il figure dans un répertoire. Notez qu'une fonction imbriquée est utilisée comme deuxième argument. Il s'agit d'un moyen de concaténer plusieurs chaînes, quand plus de deux valeurs doivent être utilisées comme arguments.

DAX

```
= CONCATENATE(Customer[LastName], CONCATENATE(", ", Customer[FirstName]))
```

Exemple : concaténation conditionnelle de chaînes dans des colonnes

L'exemple de formule crée une colonne calculée dans la table Customer avec le nom complet du client, combinant le prénom, l'initiale du deuxième prénom et le nom. En l'absence de deuxième prénom, le nom suit immédiatement le prénom. En présence d'un deuxième prénom, seule son initiale est utilisée. Elle est suivie d'un point.

DAX

```
= CONCATENATE( [FirstName]&" ", CONCATENATE( IF( LEN([MiddleName])>1,
```

```
LEFT([MiddleName],1)&" ", ""), [LastName]))
```

Cette formule utilise des fonctions CONCATENATE et IF imbriquées, ainsi que l'opérateur d'esperluette (&) pour concaténer de manière conditionnelle trois valeurs de chaîne et ajouter des espaces en tant que séparateurs.

Exemple : concaténation de colonnes avec des types de données différents

L'exemple suivant montre comment concaténer des valeurs dans des colonnes contenant différents types de données. Si la valeur que vous concaténez est numérique, elle est convertie implicitement en texte. Si les deux valeurs sont numériques, elles sont toutes les deux converties en texte et concaténées comme s'il s'agissait de chaînes.

Agrandir le tableau

Description de produit	Abréviation du produit (colonne 1 de la clé composite)	Numéro du produit (colonne 2 de la clé composite)	Nouvelle colonne clé générée
Vélo tout terrain	MTN	40	VTT40
Vélo tout terrain	MTN	42	VTT42

DAX

```
= CONCATENATE('Products'[Product abbreviation],'Products'[Product number])
```

Dans DAX, la fonction CONCATENATE accepte deux arguments uniquement, alors qu'elle en accepte jusqu'à 255 dans Excel. Si vous devez ajouter d'autres arguments, vous pouvez utiliser l'opérateur d'esperluette (&). Par exemple, la formule suivante génère les résultats VTT-40 et VTT-42.

DAX

```
= [Product abbreviation] & "-" & [Product number]
```

Contenu connexe

CONCATENATEX

Commentaires

Cette page a-t-elle été utile ? Yes No

CONCATENATEX

Article • 20/10/2023

Concatène le résultat d'une expression évaluée pour chaque ligne d'une table.

Syntaxe

```
DAX

CONCATENATEX(<table>, <expression>[, <delimiter> [, <orderBy_expression> [, <order>]]...])
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Table contenant les lignes pour lesquelles l'expression est évaluée.
expression	Expression à évaluer pour chaque ligne de la *table*.
delimiter	(Facultatif) Séparateur à utiliser pendant la concaténation.
orderBy_expression	(Facultatif) Toute expression DAX dont la valeur du résultat est utilisée pour trier les valeurs concaténées dans la chaîne de sortie. Elle est évaluée pour chaque ligne de la *table*.
order	(Facultatif) Valeur qui spécifie comment trier les valeurs *orderBy_expression*, par ordre croissant ou décroissant.

Le paramètre facultatif **order** accepte les valeurs suivantes :

Agrandir le tableau

Valeur	Autres valeurs	Description
0 (zéro)	FALSE, DESC	Trie par ordre décroissant les valeurs *orderBy_expression*. Il s'agit de la valeur par défaut quand le paramètre *order* est omis.
1	TRUE, ASC	Trie par ordre croissant les valeurs *orderBy_expression*.

Valeur renvoyée

Une chaîne concaténée.

Remarques

- Cette fonction prend comme premier argument une table ou une expression qui retourne une table. Le deuxième argument est une colonne qui contient les valeurs que vous voulez concaténer ou une expression qui retourne une valeur.
- Les valeurs concaténées ne sont pas nécessairement triées dans un ordre particulier, sauf si *orderBy_expression* est spécifié.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

Table Employees

Agrandir le tableau

FirstName	LastName
Alan	Brewer
Michael	Blythe

La formule suivante :

```
DAX
= CONCATENATEX(Employees, [FirstName] & " " & [LastName], ",")
```

Retourne les informations suivantes :
« Alan Brewer, Michael Blythe »

Commentaires

Cette page a-t-elle été utile ?

Yes No

EXACT

Article • 20/10/2023

Compare deux chaînes de texte et retourne TRUE si elles sont exactement identiques ; sinon FALSE est retourné. EXACT respecte la casse, mais ignore les différences de mise en forme. La fonction EXACT respecte la casse

Syntaxe

DAX

```
EXACT(<text1>,<text2>)
```

Paramètres

Agrandir le tableau

Terme	Définition
text1	Première colonne ou chaîne de texte qui contient du texte.
text2	Seconde colonne ou chaîne de texte qui contient du texte.

Valeur renvoyée

Vrai ou faux. (Valeur booléenne)

Exemple

La formule suivante utilisée dans une colonne calculée dans la table Product vérifie la valeur de Product pour la ligne actuelle par rapport à la valeur de Model pour la ligne actuelle, puis retourne True si elles sont identiques et False si elles sont différentes.

Les exemples de cet article peuvent être utilisés avec l'exemple de modèle Adventure Works DW 2020 Power BI Desktop. Pour obtenir le modèle, consultez Exemple de modèle DAX.

DAX

```
= EXACT([Product], [Model])
```

Contenu connexe

Fonctions de texte

Commentaires

Cette page a-t-elle été utile ? Yes No

FIND

Article • 20/10/2023

Retourne la position de départ d'une chaîne de texte à l'intérieur d'une autre chaîne de texte. La fonction FIND respecte la casse.

Syntaxe

DAX

```
FIND(<find_text>, <within_text>[, [<start_num>][, <NotFoundValue>]])
```

Paramètres

Agrandir le tableau

Terme	Définition
find_text	Texte à rechercher. Utilisez des guillemets doubles (texte vide) pour rechercher le premier caractère dans **within_text**.
within_text	Texte contenant le texte à rechercher.
start_num	(Facultatif) Caractère à partir duquel commencer la recherche ; s'il est omis, **start_num** = 1. Le premier caractère dans **within_text** est le caractère numéro 1.
NotFoundValue	(Facultatif, mais fortement recommandé) Valeur qui doit être retournée quand l'opération ne trouve pas de sous-chaîne correspondante, généralement 0, -1 ou BLANK(). Si non spécifié, une erreur est renvoyée.

Valeur renvoyée

Nombre qui indique le point de départ de la chaîne de texte à rechercher.

Remarques

- À la différence de Microsoft Excel qui fournit plusieurs versions de la fonction FIND pour prendre en charge les langages de jeu de caractères codés sur un octet (SBCS) et de jeu de caractères codés sur deux octets (DBCS), DAX utilise Unicode et compte chaque caractère de façon identique, ce qui vous évite d'avoir à utiliser une version propre à chaque type de caractère.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).
- La fonction FIND ne prend pas en charge les caractères génériques. Pour utiliser des caractères génériques, choisissez la fonction SEARCH.

Exemple

La requête DAX suivante recherche la position de la première lettre de « Bike » dans la chaîne qui contient le nom du revendeur. Si elle est introuvable, Blank est retourné.

N'oubliez pas que FIND respecte la casse. Dans cet exemple, si « bike » est utilisé dans l'argument <find_text>, aucun résultat n'est retourné. Utilisez SEARCH pour ne pas respecter la casse.

Les exemples de cet article peuvent être utilisés avec l'exemple de modèle Adventure Works DW 2020 Power BI Desktop. Pour obtenir le modèle, consultez Exemple de modèle DAX.

DAX

```
EVALUATE
CALCULATETABLE (
    ADDCOLUMNS (
        TOPN ( 10, SUMMARIZE('Reseller', [Reseller], [Business Type])),
        "Position of Bike", FIND ( "Bike", 'Reseller'[Reseller], 1, BLANK ()
)
    ),
    'Reseller'[Business Type] IN { "Specialty Bike Shop", "Value Added
Reseller", "Warehouse"}
)
```

Retourne :

Agrandir le tableau

Reseller	Business Type	Position de Bike
Volume Bike Sellers	Warehouse	8
Mass Market Bikes	Value Added Reseller	13
Twin Cycles	Value Added Reseller	
Rich Department Store	Warehouse	

Reseller	Business Type	Position de Bike
Rental Gallery	Specialty Bike Shop	
Budget Toy Store	Warehouse	
Global Sports Outlet	Warehouse	
Online Bike Catalog	Warehouse	8
Helmets and Cycles	Value Added Reseller	
Jumbo Bikes	Specialty Bike Shop	7

Contenu connexe

SEARCH
Fonctions de texte

Commentaires

Cette page a-t-elle été utile ? Yes No

FIXED

Article • 20/10/2023

Arrondit un nombre au nombre spécifié de décimales et retourne le résultat sous forme de texte. Vous pouvez spécifier que le résultat soit retourné avec ou sans virgules.

Syntaxe

```
DAX
FIXED(<number>, <decimals>, <no_commas>)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre que vous souhaitez arrondir et convertir en texte, ou colonne contenant un nombre.
decimals	(Facultatif) Nombre de chiffres après la virgule décimale ; si l'argument est omis, 2.
no_commas	(Facultatif) Valeur logique : si la valeur est 1, les virgules ne sont pas affichées dans le texte retourné ; si la valeur est 0 ou si l'argument est omis, les virgules sont affichées dans le texte retourné.

Valeur renvoyée

Nombre représenté sous forme de texte.

Remarques

- Si la valeur utilisée pour le paramètre **decimals** est négative, **number** est arrondi à gauche de la virgule décimale.
- Si vous omettez le paramètre **decimals**, sa valeur est supposée être 2.
- Si **no_commas** a la valeur 0 ou est omis, le texte retourné inclut des virgules comme d'habitude.

- La principale différence entre la mise en forme d'une cellule contenant un nombre à l'aide d'une commande et la mise en forme d'un nombre directement avec la fonction FIXED est que FIXED convertit son résultat en texte. Un nombre mis en forme à l'aide d'une commande du menu de mise en forme est toujours un nombre.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La formule suivante utilisée dans une colonne calculée obtient la valeur numérique de la ligne actuelle dans Product[Tarif Price] et la retourne sous forme de texte avec 2 décimales et aucune virgule.

Les exemples de cet article peuvent être utilisés avec l'exemple de modèle Adventure Works DW 2020 Power BI Desktop. Pour obtenir le modèle, consultez Exemple de modèle DAX.

```
DAX

= FIXED([List Price],2,1)
```

Contenu connexe

CEILING
FLOOR
ISO.CEILING
MROUND
ROUND
ROUNDDOWN
ROUNDUP

Commentaires

Cette page a-t-elle été utile ? Yes No

FORMAT

Article • 20/10/2023

Convertit une valeur en texte au format spécifié.

Syntaxe

DAX

```
FORMAT(<value>, <format_string>[, <locale_name>])
```

Paramètres

Agrandir le tableau

Terme	Définition
valeur	Valeur ou expression qui prend une valeur unique.
format_string	Chaîne avec le modèle de mise en forme.
locale_name	(Facultatif) Nom des paramètres régionaux à utiliser par la fonction. Les valeurs possibles sont les chaînes acceptées par la fonction API Windows LocaleNameToLCID().

Valeur de retour

Chaîne contenant le paramètre **value** mis en forme comme défini par **format_string**.

> ⓘ Notes
>
> Si le paramètre **value** a la valeur BLANK, la fonction retourne une chaîne vide.
>
> Si **format_string** a la valeur BLANK, la valeur est mise en forme avec un format de date ou nombre général (en fonction du type de données **value**).

Remarques

- Les chaînes de format prédéfinies utilisent la propriété de culture du modèle lors de la mise en forme du résultat. Par défaut, la propriété de culture du modèle est définie en fonction des paramètres régionaux utilisateur de l'ordinateur. Pour les nouveaux modèles de Power BI Desktop, la propriété de culture du modèle peut être changée dans Options > Paramètres régionaux > Langue du modèle. Pour Analysis Services, la culture de modèle est définie en fonction de la propriété Langue définie au départ par l'instance.

- Les chaînes de format prises en charge comme argument de la fonction DAX FORMAT sont basées sur les chaînes de format utilisées par Visual Basic (OLE Automation) et non celles utilisées par le .NET Framework. Vous pouvez donc obtenir des résultats inattendus ou une erreur si l'argument ne correspond à aucune chaîne de format définie. Par exemple, « p » comme abréviation de « pourcentage » n'est pas pris en charge. Les chaînes que vous spécifiez comme argument de la fonction FORMAT qui ne sont pas incluses dans la liste des chaînes de format prédéfinies sont gérées comme chaînes de format personnalisées ou littéraux de chaîne.

- Pour en savoir plus sur la spécification de paramètres régionaux avec FORMAT, regardez cette vidéo.

- L'utilisation de FORMAT transforme un résultat de mesure en type de données texte. Si le résultat de la mesure est de type de données numérique à l'origine, en cas d'utilisation de FORMAT, la mesure ne peut pas être utilisée sur les visuels où la section des valeurs nécessite un type de données numérique, comme des graphiques. Dans Power BI, vous pouvez également utiliser des **Chaînes de format dynamique pour des mesures** qui spécifient une chaîne de format conditionnel qui conserve le type de données numérique de la mesure.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemples

Chaînes de format

DAX

```
= FORMAT( 12345.67, "General Number")
= FORMAT( 12345.67, "Currency")
= FORMAT( 12345.67, "Fixed")
```

```
= FORMAT( 12345.67, "Standard")
= FORMAT( 12345.67, "Percent")
= FORMAT( 12345.67, "Scientific")
```

Retourne les informations suivantes :

12345.67 "General Number" affiche le nombre sans aucune mise en forme.

$12 345,67 "Currency" affiche le nombre avec le format de devise correspondant à vos paramètres régionaux. L'exemple montre ici le format de devise par défaut pour les États-Unis.

12345.67 "Fixed" affiche au moins un chiffre à gauche du séparateur décimal et deux chiffres à droite.

12,345.67 "Standard" affiche au moins un chiffre à gauche du séparateur décimal et deux chiffres à droite, et comprend des séparateurs de milliers. L'exemple montre ici le format par défaut des nombres pour les États-Unis.

1,234,567.00 % "Percent" affiche le nombre sous forme de pourcentage (multiplié par 100) avec une mise en forme et le signe de pourcentage à droite du nombre, séparé par un seul espace.

1.23E+04 "Scientific" affiche le nombre dans une notation scientifique avec deux chiffres décimaux.

DateTime avec locale_name facultatif

DAX

```
= FORMAT( dt"2020-12-15T12:30:59", BLANK(), "en-US" )
= FORMAT( dt"2020-12-15T12:30:59", BLANK(), "en-GB" )
= FORMAT( dt"2020-12-15T12:30:59", "mm/dd/yyyy", "en-GB" )
```

Retourne les informations suivantes :

12/15/2020 12:30:59 PM Où le mois précède le jour et l'heure est au format 12 heures.

15/12/2020 12:30:59 Où le jour précède le mois et l'heure est au format 24 heures.

12/15/2020 12:30:59 Où le mois précède le jour et l'heure est au format 24 heures. Comme une chaîne de format non dépendante des paramètres régionaux est spécifiée, les paramètres régionaux ne sont pas appliqués et le format non conforme aux paramètres régionaux est retourné.

Formats numériques prédéfinis

Les formats numériques prédéfinis suivants peuvent être spécifiés dans l'argument **format_string** :

Agrandir le tableau

Format	Description
`"General Number"`	Affiche le nombre sans séparateurs de milliers.
`"Currency"`	Affiche le nombre avec des séparateurs de milliers, le cas échéant ; affiche deux chiffres à droite du séparateur décimal. La sortie est basée sur les paramètres régionaux du système.
`"Fixed"`	Affiche au moins un chiffre à gauche et deux chiffres à droite du séparateur décimal.
`"Standard"`	Affiche le nombre avec des séparateurs de milliers, au moins un chiffre à gauche et deux chiffres à droite du séparateur décimal.
`"Percent"`	Affiche le nombre multiplié par 100 avec un signe de pourcentage (%) ajouté de suite à droite ; affiche toujours deux chiffres à droite du séparateur décimal.
`"Scientific"`	Utilise la notation scientifique standard, en fournissant deux chiffres significatifs.
`"Yes/No"`	Affiche No si le nombre est égal à 0 ; dans le cas contraire, affiche Yes.
`"True/False"`	Affiche False si le nombre est égal à 0 ; dans le cas contraire, affiche True.
`"On/Off"`	Affiche Off si le nombre est égal à 0 ; dans le cas contraire, affiche On.

Formats numériques personnalisés

Une expression de format personnalisée pour les nombres peut comporter de une à trois sections séparées par des points-virgules. Si l'argument de chaîne de format contient l'un des formats numériques nommés, une seule section est autorisée.

Agrandir le tableau

Si vous utilisez :	Le résultat est
Une seule section	L'expression de format s'applique à toutes les valeurs.

Si vous utilisez :	Le résultat est
Deux sections	La première section s'applique aux valeurs positives et aux zéros, et la seconde aux valeurs négatives.
Trois sections	La première section s'applique aux valeurs positives, la deuxième aux valeurs négatives et la troisième aux zéros.

DAX

```
"$#,##0;($#,##0)"
```

Si vous incluez des points-virgules sans valeur entre eux, la section manquante est définie avec le format de la valeur positive. Par exemple, le format suivant affiche les valeurs positives et négatives d'après le format défini dans la première section et affiche « Zéro » si la valeur est zéro.

DAX

```
"$#,##0"
```

Si vous incluez des points-virgules sans valeur entre eux, la section manquante est représentée par le format de la valeur positive.

Caractères de format numérique personnalisés

Les caractères de format numérique personnalisés suivants peuvent être spécifiés dans l'argument **format_string** :

Agrandir le tableau

Caractère	Description
Aucune	Affichez le nombre sans mise en forme.
(**0**)	Espace réservé de chiffre. Affiche un chiffre ou un zéro. Si l'expression contient un chiffre à l'emplacement où la valeur 0 apparaît dans la chaîne de format, elle est affichée ; sinon, affiche un zéro à cette position. Si le nombre contient moins de chiffres que de zéros (de part et d'autre du séparateur décimal) dans l'expression de format, affiche des zéros au début ou à la fin. Si le nombre contient plus de chiffres à droite du séparateur décimal qu'il y a de zéros à droite du séparateur décimal dans l'expression de format, arrondit le nombre à autant de décimales qu'il y a de zéros. Si le nombre contient plus de chiffres à gauche du séparateur décimal qu'il y a de zéros

Caractère	Description
	à gauche du séparateur décimal dans l'expression de format, affiche les chiffres supplémentaires sans modification.
(#)	Espace réservé de chiffre. Affiche un chiffre ou rien. Si l'expression contient un chiffre à l'emplacement où la valeur # apparaît dans la chaîne de format, elle est affichée. Sinon, n'affiche rien à cette position. Ce symbole fonctionne comme l'espace réservé de chiffre 0, sauf que les zéros de début et de fin ne sont pas affichés si le nombre a le même nombre ou un nombre de chiffres inférieur à celui des caractères # de part et d'autre du séparateur décimal dans l'expression de format.
(.)	Espace réservé de décimale. Dans certains paramètres régionaux, une virgule est utilisée comme séparateur décimal. L'espace réservé de décimale détermine le nombre de chiffres affichés à gauche et à droite du séparateur décimal. Si l'expression de format contient uniquement des signes numériques à gauche de ce symbole, les nombres inférieurs à 1 commencent par un séparateur décimal. Pour afficher un zéro non significatif avec les nombres fractionnaires, utilisez 0 comme premier espace réservé de chiffre à gauche du séparateur décimal. Le caractère réel utilisé comme espace réservé décimal dans le résultat mis en forme dépend du format numérique reconnu par votre système.
(%)	Espace réservé de pourcentage. L'expression est multipliée par 100. Le caractère de pourcentage (%) est inséré à l'emplacement où il apparaît dans la chaîne de format.
(,)	Séparateur des milliers. Dans certains paramètres régionaux, un point est utilisé comme séparateur des milliers. Le séparateur des milliers sépare les milliers des centaines au sein d'un nombre qui a quatre décimales ou plus à gauche du séparateur décimal. L'utilisation standard du séparateur de milliers est spécifiée si le format contient un séparateur de milliers entouré par des espaces réservés de chiffre (**0** ou **#**). La présence de deux séparateurs des milliers contigus, ou d'un séparateur des milliers juste à gauche du séparateur décimal (qu'une décimale soit ou non spécifiée), signifie « mettre à l'échelle le nombre en le divisant par 1 000, en l'arrondissant le cas échéant ». Par exemple, vous pouvez utiliser la chaîne de format « ##0,, » pour représenter 100 millions par 100. Les nombres inférieurs à 1 million sont affichés comme 0. Deux séparateurs de milliers adjacents dans toute position autre qu'immédiatement à gauche du séparateur décimal sont traités simplement comme spécifiant l'utilisation d'un séparateur de milliers. Le caractère réel utilisé pour le séparateur de milliers dans le résultat mis en forme dépend du format numérique reconnu par votre système.
(:)	Séparateur d'heure. Pour certains paramètres régionaux, d'autres caractères peuvent être utilisés pour représenter le séparateur d'heure. Le séparateur d'heure sépare les heures, les minutes et les secondes lors de la mise en forme des valeurs d'heure. Le caractère réel utilisé comme séparateur d'heure dans le résultat mis en forme est déterminé par vos paramètres système.
(/)	Séparateur de date. Pour certains paramètres régionaux, d'autres caractères peuvent être utilisés pour représenter le séparateur de date. Le séparateur de date sépare le jour, le mois et l'année lorsque les valeurs de date sont mises en forme. Le caractère

Caractère	Description
	réel utilisé comme séparateur de date dans le résultat mis en forme est déterminé par vos paramètres système.
(**E- E+ e- e+**)	Format scientifique. Si l'expression de format contient au moins un espace réservé de chiffre (**0** ou **#**) à droite de E-, E+, e- ou e+, le nombre est affiché au format scientifique et E ou e est inséré entre le nombre et son exposant. Le nombre d'espaces réservés de chiffre à droite détermine le nombre de chiffres dans l'exposant. Utilisez E- ou e- pour placer un signe moins à côté des exposants négatifs. Utilisez E+ ou e+ pour insérer un signe moins à côté des exposants négatifs et un signe plus à côté des exposants positifs.
- + $ ( )	Affiche un caractère littéral. Pour afficher un caractère autre que les caractères listés, faites-le précéder d'une barre oblique inverse (`\`) ou placez-le entre guillemets doubles (" ").
(\)	Affiche le caractère suivant dans la chaîne de format. Pour afficher un caractère ayant une signification spéciale sous forme de caractère littéral, faites-le précéder d'une barre oblique inverse (`\`). La barre oblique inverse elle-même n'est pas affichée. L'utilisation d'une barre oblique inverse équivaut à mettre le caractère suivant entre des guillemets doubles. Pour afficher une barre oblique inverse, utilisez deux barres obliques inverses (`\\`). Exemples de caractères qui ne peuvent pas être affichés comme caractères littéraux : caractères de mise en forme de date et d'heure (a, c, d, h, m, n, p, q, s, t, w, y, / et :), caractères de mise en forme numérique (#, 0, %, E, e, virgule et point) et caractères de mise en forme de chaîne (@, &, <, > et !).
("ABC")	Affiche la chaîne à l'intérieur des guillemets doubles (" ").

Formats de date et d'heure prédéfinis

Les formats date/heure prédéfinis suivants peuvent être spécifiés dans l'argument **format_string**. Quand vous utilisez des formats autres que ceux-ci, ils sont interprétés comme un format date/heure personnalisé :

Agrandir le tableau

Format	Description
`"General Date"`	Affiche une date et/ou une heure. Par exemple, 3/12/2008 11:07:31. L'affichage de la date est déterminé par les paramètres régionaux actuels de l'application.
`"Long Date"` ou `"Medium Date"`	Affiche une date en fonction du format de date longue des paramètres régionaux actuels. Par exemple, Mercredi 12 mars 2008.
`"Short Date"`	Affiche une date en fonction du format de date courte des paramètres régionaux actuels. Par exemple, 3/12/2008.

Format	Description
`"Long Time"` ou	Affiche une heure en fonction du format d'heure longue des paramètres régionaux actuels, qui inclut généralement les heures, les minutes et les secondes. Par exemple, 11:07:31.
`"Medium Time"`	Affiche une heure au format 12 heures. Par exemple, 11:07 AM.
`"Short Time"`	Affiche une heure au format 24 heures. Par exemple, 11:07.

Formats date/heure personnalisés

Les caractères de format suivants peuvent être spécifiés dans l'argument **format_string** pour créer des formats date/heure personnalisés :

Agrandir le tableau

Caractère	Description
(:)	Séparateur d'heure. Pour certains paramètres régionaux, d'autres caractères peuvent être utilisés pour représenter le séparateur d'heure. Le séparateur d'heure sépare les heures, les minutes et les secondes lors de la mise en forme des valeurs d'heure. Le caractère réel utilisé comme séparateur d'heure dans le résultat mis en forme est déterminé par vos paramètres système.
(/)	Séparateur de date. Pour certains paramètres régionaux, d'autres caractères peuvent être utilisés pour représenter le séparateur de date. Le séparateur de date sépare le jour, le mois et l'année lorsque les valeurs de date sont mises en forme. Le caractère réel utilisé comme séparateur de date dans le résultat mis en forme est déterminé par vos paramètres système.
(\)	Barre oblique inverse. Affiche le caractère suivant sous forme de caractère littéral. De ce fait, il n'est pas interprété en tant que caractère de mise en forme.
(")	Guillemet double. Le texte placé entre guillemets doubles est affiché. De ce fait, il n'est pas interprété en tant que caractères de mise en forme.
c	Affiche la date sous la forme `ddddd` et l'heure sous la forme `ttttt`, dans cet ordre. Affiche uniquement les informations de date si le numéro de série de la date ne comporte pas de partie fractionnaire. Affiche uniquement les informations d'heure s'il n'y a pas de partie entière.
d	Affiche le jour sous forme de nombre sans zéro non significatif (1-31).
jj	Affiche le jour sous forme de nombre avec un zéro non significatif (01-31).
ddd	Affiche le jour sous forme abrégée (Dim-Sam). Localisé.

Caractère	Description
dddd	Affiche le jour sous forme de nom entier (Dimanche-Samedi). Localisé.
ddddd	Affiche la date complète (jour, mois et année), conformément au format de date courte défini sur votre système. Le format de date courte par défaut est `mm/dd/yyyy`.
dddddd	Affiche un numéro de série de date complète (jour, mois et année), conformément au paramètre de date longue reconnu par votre système. Le format de date longue par défaut est `dddd, mmmm d, yyyy`.
w	Affiche le jour de la semaine sous la forme d'un nombre (de 1 pour dimanche à 7 pour samedi).
ww	Affiche la semaine de l'année sous forme de nombre (1-54).
m	Affiche le mois sous forme de nombre sans zéro non significatif (1-12). Si `m` suit immédiatement `h` ou `hh`, les minutes sont affichées à la place du mois.
MM	Affiche le mois sous forme de nombre avec un zéro non significatif (01-12). Si `mm` suit immédiatement `h` ou `hh`, les minutes sont affichées à la place du mois.
mmm	Affiche le mois sous forme abrégée (Jan-Déc). Localisé.
mmmm	Affiche le mois sous forme de nom de mois complet (Janvier-Décembre). Localisé.
q	Affiche le trimestre de l'année sous forme de nombre (1-4).
o	Affiche le jour de l'année sous forme de nombre (1-366).
yy	Affiche l'année sous forme de nombre à 2 chiffres (00-99).
aaaa	Affiche l'année sous forme de nombre à 4 chiffres (100-9999).
h	Affiche l'heure sous forme de nombre sans zéro non significatif (0-23).
hh	Affiche l'heure sous forme de nombre avec un zéro non significatif (00-23).
n	Affiche les minutes sous forme de nombre sans zéro non significatif (0-59).
nn	Affiche les minutes sous forme de nombre avec un zéro non significatif (00-59).
s	Affiche les secondes sous forme de nombre sans zéro non significatif (0-59).
ss	Affiche les secondes sous forme de nombre avec un zéro non significatif (00-59).
ttttt	Affiche l'heure complète (heures, minutes et secondes), dans une mise en forme qui reprend le séparateur horaire défini dans les paramètres horaires de votre système. Un zéro non significatif apparaît si l'option correspondante est sélectionnée et si l'heure est antérieure à 10:00 A.M. ou P.M. Le format d'heure par défaut est `h:mm:ss`.
AM/PM	Utilisez l'horloge de 12 heures et affichez un AM en majuscules avec toute heure

Caractère	Description
	avant midi ; un PM en majuscules avec toute heure comprise entre midi et 23h59.
AM/PM	Utilise l'horloge de 12 heures et affiche am en minuscules avec toute heure avant midi et pm en minuscules avec toute heure comprise entre midi et 23h59.
A/P	Utilise l'horloge de 12 heures et affiche un « A » majuscule avec toute heure avant midi et un « P » majuscule avec toute heure comprise entre midi et 23h59.
A/P	Utilise l'horloge de 12 heures et affiche un « a » minuscule avec toute heure avant midi et un « p » minuscule avec toute heure comprise entre midi et 23h59.
AMPM	Utilise l'horloge de 12 heures et affiche le littéral de chaîne AM comme défini par votre système avec toute heure avant midi et le littéral de chaîne PM comme défini par votre système avec toute heure comprise entre midi et 23h59. AMPM peut être spécifié en majuscules ou en minuscules. La casse de la chaîne affichée correspond à celle de la chaîne définie par vos paramètres système. Le format par défaut est AM/PM. Si votre système est défini sur une horloge de 24 heures, la chaîne est généralement définie comme étant une chaîne vide.

Le format de date/heure utilise les paramètres régionaux de l'utilisateur actuel pour mettre en forme la chaîne. Prenons pour exemple de date le 25 juin 2020. Quand cette date est mise en forme avec la chaîne de format « m/d/yyyy », elle se présente comme ceci :

- Les paramètres régionaux utilisateur sont États-Unis d'Amérique (en-US) : « 6/25/2020 »
- Les paramètres régionaux sont Allemagne (de-DE) : « 6.25.2020 »

Exemples de formats date/heure personnalisés

Les exemples suivants utilisent la date/heure jeudi 25 juin 2020, à 13:23:45. L'Allemagne (de-DE) utilise un système de 24 heures. Elle n'a pas d'équivalent de la notation AM/PM.

Agrandir le tableau

Format	Résultat (en-US)	Résultat (de-DE)
`"c"`	06/25/2020 13:23:45	25.06.2020 13:23:45
`"d"`	25	25
`"dd"`	25	25
`"ddd"`	Thu	Do
`"dddd"`	Thursday	Donnerstag

Format	Résultat (en-US)	Résultat (de-DE)
`"ddddd"`	06/25/2020	25.06.2020
`"dddddd"`	Thursday, June 25, 2020	Donnerstag, 25. Juni 2020
`"w"`	5	5
`"ww"`	26	26
`"m"`	6	6
`"mm"`	06	06
`"mmm"`	Jun	Jun
`"mmmm"`	June	Juni
`"q"`	2	2
`"y"`	177	177
`"yy"`	20	20
`"yyyy"`	2020	2020
`"""Year"" yyyy"`	Year 2020	Year 2020
`"yyyy \Qq"`	2020 Q2	2020 Q2
`"dd/mm/yyyy"`	25/06/2020	25.06.2020
`"mm/dd/yyyy"`	06/25/2020	06.25.2020
`"h:nn:ss"`	13:23:45	13:23:45
`"h:nn:ss AMPM"`	1:23:45 PM	1:23:45
`"hh:nn:ss"`	13:23:45	13:23:45
`"hh:nn:ss AMPM"`	01:23:45 PM	01:23:45
`"ttttt"`	13:23:45	13:23:45
`"ttttt AMPM"`	13:23:45 PM	13:23:45
`"mm/dd/yyyy hh:nn:ss AMPM"`	06/25/2020 01:23:45 PM	6.25.2020 01:23:45

Commentaires

Cette page a-t-elle été utile ? Yes No

LEFT

Article • 14/12/2023

Retourne le nombre spécifié de caractères à partir du début d'une chaîne de texte.

Syntaxe

DAX

```
LEFT(<text>, <num_chars>)
```

Paramètres

Agrandir le tableau

Terme	Définition
texte	Chaîne de texte qui contient les caractères que vous voulez extraire ou une référence à une colonne qui contient du texte.
num_chars	(facultatif) Nombre de caractères que LEFT doit extraire ; si l'argument est omis, 1.

Valeur renvoyée

Chaîne de texte.

Remarques

- Alors que Microsoft Excel contient des fonctions différentes pour utiliser le texte dans les langues à caractères codés sur un ou deux octets, DAX utilise Unicode et stocke tous les caractères sur une même longueur, si bien qu'une seule fonction suffit.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple suivant retourne les cinq premiers caractères du nom de la société dans la colonne [ResellerName] et les cinq premières lettres du code géographique dans la colonne [GeographyKey], puis les concatène pour créer un identificateur.

```
DAX
= CONCATENATE(LEFT('Reseller'[ResellerName],LEFT(GeographyKey,3))
```

Si l'argument **num_chars** est un nombre supérieur au nombre de caractères disponibles, la fonction retourne le nombre maximal de caractères disponibles et ne génère pas d'erreur. Par exemple, la colonne [GeographyKey] contient des nombres tels que 1, 12 et 311 ; par conséquent, le résultat a également une longueur variable.

Contenu connexe

Fonctions de texte

Commentaires

Cette page a-t-elle été utile ? Yes No

LEN

Article • 20/10/2023

Retourne le nombre de caractères dans une chaîne de texte.

Syntaxe

DAX

```
LEN(<text>)
```

Paramètres

Agrandir le tableau

Terme	Définition
texte	Texte dont vous souhaitez rechercher la longueur ou colonne contenant du texte. Les espaces comptent pour des caractères.

Valeur de retour

Nombre entier indiquant le nombre de caractères dans la chaîne de texte.

Remarques

- Alors que Microsoft Excel propose des fonctions différentes pour les langues à caractères codés sur un ou deux octets, DAX utilise Unicode et stocke tous les caractères sur une même longueur.
- LEN compte toujours chaque caractère comme un seul (1) caractère, quel que soit le paramètre de la langue par défaut.
- Si vous utilisez LEN avec une colonne contenant des valeurs non textuelles, comme des dates ou des valeurs booléennes, la fonction caste implicitement la valeur en texte selon le format actuel de la colonne.

Exemple

La formule suivante additionne les longueurs des adresses dans les colonnes [AddressLine1] et [AddressLine2].

```DAX
= LEN([AddressLine1])+LEN([AddressLin2])
```

Commentaires

Cette page a-t-elle été utile ? Yes No

LOWER

Article • 14/12/2023

Convertit toutes les lettres d'une chaîne de texte en minuscules.

Syntaxe

```
DAX

LOWER(<text>)
```

Paramètres

Agrandir le tableau

Terme	Définition
texte	Texte que vous souhaitez convertir en minuscules, ou référence à une colonne qui contient du texte.

Valeur renvoyée

Texte en minuscules.

Remarques

Les caractères qui ne sont pas des lettres ne sont pas changés. Par exemple, la formule `=LOWER("123ABC")` retourne **123abc**.

Exemple

La formule suivante obtient chaque ligne de la colonne, [ProductCode], et convertit la valeur en minuscules. Les nombres de la colonne ne sont pas affectés.

```
DAX

= LOWER('New Products'[ProductCode])
```

Contenu connexe

Fonctions de texte

Commentaires

Cette page a-t-elle été utile ? Yes No

MID

Article • 20/10/2023

Retourne une chaîne de caractères à partir du milieu d'une chaîne de texte, en fonction d'une position de départ et d'une longueur.

Syntaxe

DAX

```
MID(<text>, <start_num>, <num_chars>)
```

Paramètres

Agrandir le tableau

Terme	Définition
texte	La chaîne de texte dont vous voulez extraire les caractères ou une colonne qui contient du texte.
start_num	Position du premier caractère à extraire. Les positions commencent à 1.
num_chars	Nombre de caractères à retourner.

Valeur renvoyée

Une chaîne de texte de la longueur spécifiée.

Remarques

Alors que Microsoft Excel propose des fonctions différentes pour les langues à caractères codés sur un ou deux octets, DAX utilise Unicode et stocke tous les caractères avec la même longueur.

Exemples

L'expression suivante :

```DAX
MID("abcde",2,3))
```

retourne « **bcd** » .

L'expression suivante :

```DAX
MID('Reseller'[ResellerName],1,5))
```

retourne le même résultat que `LEFT([ResellerName],5)`. Les deux expressions retournent les cinq premières lettres de la colonne `[ResellerName]`.

Contenu connexe

Fonctions de texte

Commentaires

Cette page a-t-elle été utile ? Yes No

REPLACE

Article • 20/10/2023

REPLACE remplace une partie d'une chaîne de texte, en fonction du nombre de caractères que vous spécifiez, par une autre chaîne de texte.

Syntaxe

DAX

```
REPLACE(<old_text>, <start_num>, <num_chars>, <new_text>)
```

Paramètres

Agrandir le tableau

Terme	Définition
old_text	Chaîne de texte qui contient les caractères à remplacer ou référence à une colonne qui contient du texte.
start_num	Position du caractère dans **old_text** à remplacer par **new_text**.
num_chars	Nombre de caractères à remplacer. **Avertissement :** Si l'argument, *num_chars*, est un espace ou fait référence à une colonne qui donne une valeur vide, la chaîne pour *new_text* est insérée à la position *start_num* sans qu'aucun caractère ne soit remplacé. Il s'agit du même comportement que dans Excel.
new_text	Texte de remplacement pour les caractères spécifiés dans **old_text**.

Valeur renvoyée

Chaîne de texte.

Remarques

- Alors que Microsoft Excel propose des fonctions différentes pour les langues à caractères codés sur un ou deux octets, DAX utilise Unicode et stocke donc tous les caractères sur une même longueur.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La formule suivante crée une colonne calculée qui remplace les deux premiers caractères du code de produit dans la colonne [ProductCode] par un nouveau code à deux lettres, OB.

DAX

```
= REPLACE('New Products'[Product Code],1,2,"OB")
```

Contenu connexe

Fonctions de texte
SUBSTITUTE, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

REPT

Article • 14/12/2023

Répète du texte un certain nombre de fois. Utilisez REPT pour remplir une cellule avec un certain nombre d'instances d'une chaîne de texte.

Syntaxe

DAX

```
REPT(<text>, <num_times>)
```

Paramètres

Agrandir le tableau

Terme	Définition
texte	Texte à répéter.
num_times	Nombre positif spécifiant le nombre de répétitions du texte.

Valeur renvoyée

Chaîne contenant les modifications.

Remarques

- Si **number_times** est égal à 0 (zéro), la fonction REPT retourne une valeur vide.
- Si **number_times** n'est pas un entier, sa valeur est tronquée.
- Le résultat de la fonction REPT ne peut pas dépasser 32 767 caractères, ou REPT retourne une erreur.

Exemple : Répétition de chaînes littérales

L'exemple suivant retourne la chaîne, 85, répétée trois fois.

```
```

```
DAX
= REPT("85",3)
```

Exemple : répétition de valeurs de colonne

L'exemple suivant retourne la chaîne dans la colonne, [MyText], répétée le nombre de fois figurant dans la colonne, [MyNumber]. Étant donné que la formule s'étend à la totalité de la colonne, la chaîne obtenue dépend du texte et de la valeur numérique figurant dans chaque ligne.

```
DAX
= REPT([MyText],[MyNumber])
```

Agrandir le tableau

MyText	MyNumber	CalculatedColumn1
Texte	2	TextText
Numéro	0	
85 %	3	858585

Contenu connexe

Fonctions de texte

Commentaires

Cette page a-t-elle été utile ? Yes No

RIGHT

Article • 20/10/2023

RIGHT retourne le ou les derniers caractères d'une chaîne de texte, en fonction du nombre de caractères que vous spécifiez.

Syntaxe

DAX

```
RIGHT(<text>, <num_chars>)
```

Paramètres

Agrandir le tableau

Terme	Définition
texte	Chaîne de texte qui contient les caractères que vous voulez extraire ou une référence à une colonne qui contient du texte.
num_chars	(facultatif) Nombre de caractères que RIGHT doit extraire ; si l'argument est omis, 1. Vous pouvez également utiliser une référence à une colonne qui contient des nombres.

Si la référence de colonne ne contient pas de texte, elle est implicitement convertie en texte.

Valeur renvoyée

Chaîne de texte contenant les caractères les plus à droite spécifiés.

Remarques

- RIGHT compte toujours chaque caractère, qu'il soit sur un ou deux octets, comme 1, indépendamment du paramètre de langue par défaut.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple : retour d'un nombre constant de caractères

La formule suivante retourne les deux derniers chiffres du code produit dans la table New Products.

```
DAX
= RIGHT('New Products'[ProductCode],2)
```

Exemple : utilisation d'une référence de colonne pour spécifier le nombre de caractères

La formule suivante retourne un nombre variable de chiffres à partir du code produit dans la table New Products, en fonction du nombre dans la colonne, MyCount. S'il n'y a aucune valeur dans la colonne, MyCount, ou si la valeur est un espace, RIGHT retourne également un blanc.

```
DAX
= RIGHT('New Products'[ProductCode],[MyCount])
```

Contenu connexe

Fonctions de texte
LEFT
MID

Commentaires

Cette page a-t-elle été utile ? Yes No

SEARCH

Article • 20/10/2023

Retourne le numéro du caractère où apparaît pour la première fois un caractère ou une chaîne de texte spécifique selon une lecture de gauche à droite. La recherche ne respecte pas la casse mais respecte les accents.

Syntaxe

DAX

```
SEARCH(<find_text>, <within_text>[, [<start_num>][, <NotFoundValue>]])
```

Paramètres

Agrandir le tableau

Terme	Définition
find_text	Texte à rechercher. Vous pouvez utiliser des caractères génériques (le point d'interrogation (?) et l'astérisque (*)) dans **find_text**. Un point d'interrogation correspond à n'importe quel caractère unique ; un astérisque correspond à n'importe quelle séquence de caractères. Si vous souhaitez rechercher un point d'interrogation ou un astérisque réel, tapez un tilde (~) avant le caractère.
within_text	Texte dans lequel vous voulez rechercher **find_text** ou colonne contenant le texte.
start_num	(Facultatif) Position du caractère dans **within_text** à partir de laquelle vous voulez démarrer la recherche. Si l'argument est omis, la valeur est 1.
NotFoundValue	(Facultatif, mais fortement recommandé) Valeur qui doit être retournée quand l'opération ne trouve pas de sous-chaîne correspondante, généralement 0, -1 ou BLANK(). Si non spécifié, une erreur est renvoyée.

Valeur de retour

Numéro de la position de départ de la première chaîne de texte à partir du premier caractère de la deuxième chaîne de texte.

Remarques

- La fonction de recherche ne respecte pas la casse. Si la recherche porte sur « N », la première occurrence de « N » ou « n » sera trouvée.
- La fonction de recherche respecte les accents. Si la recherche porte sur le caractère « á », la première occurrence de « á » sera trouvée, mais pas les occurrences de « a », « à », « A » ou « Á ».
- Vous pouvez utiliser la fonction SEARCH pour déterminer l'emplacement d'un caractère ou d'une chaîne de texte dans une autre chaîne de texte, puis utiliser la fonction MID pour retourner le texte ou la fonction REPLACE pour modifier le texte.
- Si **find_text** est introuvable dans **within_text**, la formule retourne une erreur. Ce comportement est similaire à celui d'Excel, qui retourne #VALUE si la sous-chaîne est introuvable. Les valeurs Null dans **within_text** sont interprétées comme une chaîne vide dans ce contexte.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La requête DAX suivante recherche la position de la première lettre de « cycle » dans la chaîne qui contient le nom du revendeur. Si elle est introuvable, Blank est retourné.

SEARCH ne respecte pas la casse. Dans cet exemple, si « cycle » ou « Cycle » est utilisé dans l'argument <find_text>, les résultats sont retournés dans les deux cas. Utilisez FIND pour respecter la casse.

Les exemples de cet article peuvent être utilisés avec l'exemple de modèle Adventure Works DW 2020 Power BI Desktop. Pour obtenir le modèle, consultez Exemple de modèle DAX.

DAX

```
EVALUATE
CALCULATETABLE (
    ADDCOLUMNS (
        TOPN ( 10, SUMMARIZE('Reseller', [Reseller], [Business Type])),
        "Position of cycle", SEARCH ( "cycle", 'Reseller'[Reseller], 1,
BLANK () )
    ),
```

```
    'Reseller'[Business Type] IN { "Specialty Bike Shop", "Value Added
Reseller", "Warehouse"}
)
```

Retourne :

Agrandir le tableau

Reseller	Business Type	Position de cycle
Volume Bike Sellers	Warehouse	
Mass Market Bikes	Value Added Reseller	
Twin Cycles	Value Added Reseller	6
Rich Department Store	Warehouse	
Rental Gallery	Specialty Bike Shop	
Budget Toy Store	Warehouse	
Global Sports Outlet	Warehouse	
Online Bike Catalog	Warehouse	
Helmets and Cycles	Value Added Reseller	13
Jumbo Bikes	Specialty Bike Shop	

Contenu connexe

FIND
REPLACE
Fonctions de texte

Commentaires

Cette page a-t-elle été utile ? Yes No

SUBSTITUTE

Article • 14/12/2023

Remplace un texte existant par un nouveau texte dans une chaîne de texte.

Syntaxe

DAX

```
SUBSTITUTE(<text>, <old_text>, <new_text>, <instance_num>)
```

Paramètres

Agrandir le tableau

Terme	Définition
texte	Texte dans lequel vous souhaitez remplacer des caractères ou référence à une colonne contenant du texte.
old_text	Texte existant à remplacer.
new_text	Texte de remplacement de **old_text**.
instance_num	(Facultatif) Occurrence de **old_text** à remplacer. En cas d'omission, chaque instance de **old_text** est remplacée.

Valeur renvoyée

Chaîne de texte.

Remarques

- Utilisez la fonction SUBSTITUTE pour remplacer un texte spécifique dans une chaîne de texte. Utilisez la fonction REPLACE pour remplacer tout texte de longueur variable figurant à un emplacement spécifique d'une chaîne de texte.
- La fonction SUBSTITUTE respecte la casse. Si la casse des paramètres **text** et **old_text** est différente, la fonction SUBSTITUTE ne remplace pas le texte.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple : substitution dans une chaîne

La formule suivante crée une copie de la colonne [Product Code] dans laquelle toutes les occurrences de l’ancien code de produit **PA** sont remplacées par le nouveau code de produit **NW**.

DAX

```
= SUBSTITUTE([Product Code], "NW", "PA")
```

Contenu connexe

Fonctions de texte
REPLACE

Commentaires

Cette page a-t-elle été utile ? Yes No

TRIM

Article • 14/12/2023

Supprime tous les espaces d'un texte à l'exception des espaces uniques entre les mots.

Syntaxe

```
DAX

TRIM(<text>)
```

Paramètres

Agrandir le tableau

Terme	Définition
text	Texte dont vous souhaitez supprimer les espaces ou colonne contenant du texte.

Valeur renvoyée

Chaîne avec les espaces supprimés.

Remarques

- Utilisez la fonction TRIM sur le texte que vous avez reçu d'une autre application et dont l'espacement peut être irrégulier.
- À l'origine, la fonction TRIM a été conçue pour tronquer le caractère d'espace ASCII 7 bits (valeur 32) dans du texte. Dans le jeu de caractères Unicode, il existe un caractère d'espace supplémentaire appelé espace insécable dont la valeur décimale est 160. Ce caractère est couramment utilisé dans les pages web sous la forme de l'entité HTML . La fonction TRIM elle-même ne supprime pas ce caractère d'espace insécable. Pour obtenir un exemple montrant comment supprimer les deux caractères d'espace dans du texte, consultez Suppression d'espaces et de caractères non imprimables dans du texte.

Exemple

La formule suivante crée une chaîne ne contenant pas d'espace blanc de fin.

```
DAX
= TRIM("A column with trailing spaces.   ")
```

Quand vous créez la formule, elle est propagée sur la ligne telle que vous l'avez tapée. Vous voyez donc la chaîne d'origine dans chaque formule, et les résultats ne sont pas visibles. Toutefois, quand la formule est évaluée, la chaîne est tronquée.

Vous pouvez vous assurer que la formule génère le résultat correct en vérifiant la longueur de la colonne calculée créée par la formule précédente comme suit :

```
DAX
= LEN([Calculated Column 1])
```

Contenu connexe

Fonctions de texte

Commentaires

Cette page a-t-elle été utile ? Yes No

UNICHAR

Article • 20/10/2023

Retourne le caractère Unicode référencé par la valeur numérique.

Syntaxe

DAX

```
UNICHAR(number)
```

Paramètres

Agrandir le tableau

Terme	Définition
nombre	Nombre Unicode qui représente le caractère.

Valeur de retour

Caractère représenté par le nombre Unicode.

Remarques

- Si les caractères XML ne sont pas valides, UNICHAR retourne une erreur.
- Si les nombres Unicode sont des substituts partiels et que les types de données ne sont pas valides, UNICHAR retourne une erreur.
- Si les nombres sont des valeurs numériques hors de la plage autorisée, UNICHAR retourne une erreur.
- Si le nombre est égal à zéro (0), UNICHAR retourne une erreur.
- Le caractère Unicode retourné peut être une chaîne de caractères, par exemple en code UTF-8 ou UTF-16.

Exemple

L'exemple suivant retourne le caractère représenté par le nombre Unicode 66 (A majuscule).

```
DAX

= UNICHAR(65)
```

L'exemple suivant retourne le caractère représenté par le nombre Unicode 32 (espace).

```
DAX

= UNICHAR(32)
```

L'exemple suivant retourne le caractère représenté par le nombre Unicode 9733 (caractère ★).

```
DAX

= UNICHAR(9733)
```

Commentaires

Cette page a-t-elle été utile ? Yes No

UNICODE

Article • 20/10/2023

Retourne le nombre (point de code) correspondant au premier caractère du texte.

Syntaxe

```
DAX

UNICODE( <Text> )
```

Paramètres

Agrandir le tableau

Terme	Définition
Texte	Le texte est le caractère dont vous voulez la valeur Unicode.

Valeur renvoyée

Code numérique pour le premier caractère d'une chaîne de texte.

Commentaires

Cette page a-t-elle été utile ? Yes No

UPPER

Article • 14/12/2023

Convertit toutes les lettres d'une chaîne de texte en majuscules.

Syntaxe

DAX

```
UPPER (<text>)
```

Paramètres

Agrandir le tableau

Terme	Définition
texte	Texte ou référence à une colonne qui contient du texte à convertir en majuscules.

Valeur renvoyée

Même texte en majuscules.

Exemple

La formule suivante convertit toutes les lettres de la chaîne de la colonne [ProductCode] en majuscules. Les caractères non alphabétiques ne sont pas affectés.

DAX

```
= UPPER(['New Products'[Product Code])
```

Contenu connexe

Fonctions de texte
LOWER, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

VALEUR

Article • 20/10/2023

Convertit en nombre une chaîne de texte qui représente un nombre.

Syntaxe

```
DAX

VALUE(<text>)
```

Paramètres

Agrandir le tableau

Terme	Définition
texte	Texte à convertir.

Valeur de retour

Nombre converti en type de données décimal.

Remarques

- La valeur transmise en tant que paramètre **text** peut être dans l'un quelconque des formats de constante, de nombre, de date ou d'heure reconnus par l'application ou les services que vous utilisez. Si **text** n'est pas dans l'un de ces formats, une erreur est retournée.
- Vous n'avez généralement pas besoin d'utiliser la fonction VALUE dans une formule, car le moteur convertit implicitement le texte en nombres si nécessaire.
- Vous pouvez également utiliser des références de colonnes. Par exemple, si vous avez une colonne qui contient des types numériques mixtes, la fonction VALUE peut être utilisée pour convertir toutes les valeurs en un même type de données numérique. Toutefois, si vous utilisez la fonction VALUE avec une colonne qui contient du texte et des nombres mixtes, la colonne entière est marquée d'une

erreur, car toutes les valeurs de toutes les lignes ne peuvent pas être converties en nombres.

Exemple

La formule suivante convertit la chaîne typée « 3 » en valeur numérique 3.

```
DAX
= VALUE("3")
```

Contenu connexe

Fonctions de texte

Commentaires

Cette page a-t-elle été utile ? Yes No

Fonctions d’intelligence temporelle

Article • 20/10/2023

DAX (Data Analysis Expressions) comprend également des fonctions d’intelligence temporelle qui vous permettent de manipuler des données en utilisant des périodes de temps (jours, mois, trimestres et années, entre autres), puis de générer et comparer des calculs sur ces périodes. Avant d’utiliser des fonctions Time Intelligence, assurez-vous de marquer l’une des tables contenant la colonne de date comme Table de dates.

Dans cette catégorie

Agrandir le tableau

Fonction	Description
CLOSINGBALANCEMONTH	Évalue l’expression à la dernière date du mois dans le contexte actuel.
CLOSINGBALANCEQUARTER	Évalue l’expression à la dernière date du trimestre dans le contexte actuel.
CLOSINGBALANCEYEAR	Évalue l’expression à la dernière date de l’année dans le contexte actuel.
DATEADD	Retourne une table qui contient une colonne de dates déplacées en avançant ou en remontant dans le temps en fonction du nombre d’intervalles spécifié à partir des dates dans le contexte actuel.
DATESBETWEEN	Retourne une table contenant une colonne de dates qui commence à une date de début spécifiée et se poursuit jusqu’à une date de fin spécifiée.
DATESINPERIOD	Retourne une table contenant une colonne de dates qui commence à une date de début spécifiée et se poursuit pendant le nombre et le type d’intervalles de dates spécifiés.
DATESMTD	Retourne une table qui contient une colonne des dates du mois jusqu’à ce jour, dans le contexte actuel.
DATESQTD	Retourne une table qui contient une colonne des dates du trimestre jusqu’à ce jour, dans le contexte actuel.
DATESYTD	Retourne une table qui contient une colonne des dates de l’année jusqu’à ce jour, dans le contexte actuel.

Fonction	Description
ENDOFMONTH	Retourne la dernière date du mois dans le contexte actuel pour la colonne de dates spécifiée.
ENDOFQUARTER	Retourne la dernière date du trimestre dans le contexte actuel pour la colonne de dates spécifiée.
ENDOFYEAR	Retourne la dernière date de l'année dans le contexte actuel pour la colonne de dates spécifiée.
FIRSTDATE	Retourne la première date dans le contexte actuel pour la colonne de dates spécifiée.
FIRSTNONBLANK	Retourne la première valeur de la colonne, column, filtrée par le contexte actuel, où l'expression n'est pas vide
LASTDATE	Retourne la dernière date dans le contexte actuel pour la colonne de dates spécifiée.
LASTNONBLANK	Retourne la dernière valeur de la colonne, column, filtrée par le contexte actuel, où l'expression n'est pas vide.
NEXTDAY	Retourne une table qui contient une colonne de toutes les dates du jour suivant, selon la première date spécifiée dans la colonne dates du contexte actuel.
NEXTMONTH	Retourne une table qui contient une colonne de toutes les dates du mois suivant, selon la première date dans la colonne dates du contexte actuel.
NEXTQUARTER	Retourne une table qui contient une colonne de toutes les dates du trimestre suivant, selon la première date spécifiée dans la colonne dates du contexte actuel.
NEXTYEAR	Retourne une table qui contient une colonne de toutes les dates de l'année suivante, selon la première date dans la colonne dates du contexte actuel.
OPENINGBALANCEMONTH	Évalue l'expression à la première date du mois dans le contexte actuel.
OPENINGBALANCEQUARTER	Évalue l'expression à la première date du trimestre dans le contexte actuel.
OPENINGBALANCEYEAR	Évalue l'expression à la première date de l'année dans le contexte actuel.
PARALLELPERIOD	Retourne une table qui contient une colonne de dates qui représente une période parallèle aux dates de la colonne dates spécifiée, dans le contexte actuel, avec les dates décalées d'un

Fonction	Description
	certain nombre d'intervalles en avançant ou en remontant dans le temps.
PREVIOUSDAY	Retourne une table qui contient une colonne de toutes les dates représentant le jour qui précède la première date de la colonne dates, dans le contexte actuel.
PREVIOUSMONTH	Retourne une table qui contient une colonne de toutes les dates du mois précédent, selon la première date de la colonne dates, dans le contexte actuel.
PREVIOUSQUARTER	Retourne une table qui contient une colonne de toutes les dates du trimestre précédent, selon la première date de la colonne dates, dans le contexte actuel.
PREVIOUSYEAR	Retourne une table qui contient une colonne de toutes les dates de l'année précédente, selon la dernière date de la colonne dates, dans le contexte actuel.
SAMEPERIODLASTYEAR	Retourne une table qui contient une colonne de dates déplacées d'une année en remontant dans le temps à partir des dates de la colonne dates spécifiée dans le contexte actuel.
STARTOFMONTH	Retourne la première date du mois dans le contexte actuel pour la colonne de dates spécifiée.
STARTOFQUARTER	Retourne la première date du trimestre dans le contexte actuel pour la colonne de dates spécifiée.
STARTOFYEAR	Retourne la première date de l'année dans le contexte actuel pour la colonne de dates spécifiée.
TOTALMTD	Évalue la valeur de l'expression pour le mois jusqu'à ce jour, dans le contexte actuel.
TOTALQTD	Évalue la valeur de l'expression pour les dates du trimestre jusqu'à ce jour, dans le contexte actuel.
TOTALYTD	Évalue la valeur de cumul annuel jusqu'à ce jour de l'expression dans le contexte actuel.

Commentaires

Cette page a-t-elle été utile ? Yes No

CLOSINGBALANCEMONTH

Article • 20/10/2023

Évalue l'**expression** à la dernière date du mois dans le contexte actuel.

Syntaxe

```
DAX
CLOSINGBALANCEMONTH(<expression>,<dates>[,<filter>])
```

Paramètres

Agrandir le tableau

Paramètre	Définition
expression	Expression retournant une valeur scalaire.
dates	Colonne contenant des dates.
filter	(Facultatif) Expression qui spécifie un filtre à appliquer au contexte actuel.

Valeur de retour

Valeur scalaire qui représente l'**expression** évaluée à la dernière date du mois dans le contexte actuel.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

> ⓘ Notes
>
> Les contraintes sur les expressions booléennes sont décrites dans **CALCULATE, fonction**.

> ⓘ Notes
>
> Des restrictions s'appliquent à l'expression **filter** ; elles sont décrites dans **CALCULATE, fonction**.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule la valeur d'inventaire de fin de mois des produits.

DAX

```
=
CLOSINGBALANCEMONTH(SUMX(ProductInventory,ProductInventory[UnitCost]*Product
Inventory[UnitsBalance]),DateTime[DateKey])
```

Contenu connexe

Fonctions d'intelligence temporelle
CLOSINGBALANCEYEAR, fonction
CLOSINGBALANCEQUARTER, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

CLOSINGBALANCEQUARTER

Article • 15/12/2023

Évalue l'**expression** à la dernière date du trimestre dans le contexte actuel.

Syntaxe

```
DAX
CLOSINGBALANCEQUARTER(<expression>,<dates>[,<filter>])
```

Paramètres

Agrandir le tableau

Terme	Définition
expression	Expression retournant une valeur scalaire.
dates	Colonne contenant des dates.
filter	(Facultatif) Expression qui spécifie un filtre à appliquer au contexte actuel.

Valeur de retour

Valeur scalaire qui représente la valeur **expression** évaluée à la dernière date du trimestre dans le contexte actuel.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

> ⓘ Notes
>
> Les contraintes sur les expressions booléennes sont décrites dans **CALCULATE, fonction**.

> ⓘ Notes
>
> Des restrictions s'appliquent à l'expression **filter** ; elles sont décrites dans **CALCULATE, fonction**.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule la valeur d'inventaire de fin de trimestre de l'inventaire des produits.

DAX

```
=
CLOSINGBALANCEQUARTER(SUMX(ProductInventory,ProductInventory[UnitCost]*Produ
ctInventory[UnitsBalance]),DateTime[DateKey])
```

Contenu connexe

Fonctions d'intelligence temporelle
CLOSINGBALANCEYEAR, fonction
CLOSINGBALANCEMONTH, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

CLOSINGBALANCEYEAR

Article • 15/12/2023

Évalue l'**expression** à la dernière date de l'année dans le contexte actuel.

Syntaxe

DAX

```
CLOSINGBALANCEYEAR(<expression>,<dates>[,<filter>][,<year_end_date>])
```

Paramètres

Agrandir le tableau

Terme	Définition
expression	Expression retournant une valeur scalaire.
dates	Colonne contenant des dates.
filter	(Facultatif) Expression qui spécifie un filtre à appliquer au contexte actuel.
year_end_date	(Facultatif) Chaîne littérale avec une date qui définit la date de fin d'année. La valeur par défaut est le 31 décembre.

Valeur de retour

Valeur scalaire qui représente l'**expression** évaluée à la dernière date de l'année dans le contexte actuel.

Remarques

- Le paramètre **year_end_date** est un littéral de chaîne d'une date. Ses paramètres régionaux sont identiques à ceux du client où le classeur a été créé. La partie année de la date est ignorée.
- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.

- Une expression de table qui retourne une seule colonne de valeurs de date/heure
- Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

> ⓘ **Notes**
>
> Les contraintes sur les expressions booléennes sont décrites dans **CALCULATE, fonction**.

> ⓘ **Notes**
>
> Des restrictions s'appliquent à l'expression **filter** ; elles sont décrites dans **CALCULATE, fonction**.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule la valeur d'inventaire de fin d'année des produits.

DAX

```
=
CLOSINGBALANCEYEAR(SUMX(ProductInventory,ProductInventory[UnitCost]*ProductI
nventory[UnitsBalance]),DateTime[DateKey])
```

Contenu connexe

Fonctions d'intelligence temporelle
CLOSINGBALANCEYEAR, fonction
CLOSINGBALANCEQUARTER, fonction
CLOSINGBALANCEMONTH, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

DATEADD

Article • 14/12/2023

Retourne une table qui contient une colonne de dates avancées ou reculées dans le temps du nombre d'intervalles spécifié par rapport aux dates dans le contexte actuel.

Syntaxe

DAX

```
DATEADD(<dates>,<number_of_intervals>,<interval>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne qui contient des dates.
number_of_intervals	Entier qui spécifie le nombre d'intervalles à ajouter aux dates ou à soustraire des dates.
interval	Intervalle selon lequel les dates doivent être avancées ou reculées. L'intervalle peut avoir l'une des valeurs suivantes : `year`, `quarter`, `month`, `day`

Valeur de retour

Table contenant une seule colonne de valeurs de date.

Remarques

L'argument **dates** peut être l'un des éléments suivants :

- Une référence à une colonne de date/heure
- Une expression de table qui retourne une seule colonne de valeurs de date/heure
- Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

> ⓘ Notes
>
> Les contraintes sur les expressions booléennes sont décrites dans la rubrique **CALCULATE, fonction**.

- Si le nombre spécifié pour **number_of_intervals** est positif, les dates dans la colonne **dates** sont avancées dans le temps ; s'il est négatif, les dates dans la colonne **dates** sont reculées dans le temps.
- Le paramètre **interval** est une énumération, pas un ensemble de chaînes ; les valeurs ne doivent donc pas être placées entre guillemets. De plus, si vous utilisez les valeurs `year`, `quarter`, `month` ou `day`, vous devez les spécifier en toutes lettres.
- La table de résultats inclut uniquement les dates qui existent dans la colonne **dates**.
- Si les dates dans le contexte actuel ne constituent pas un intervalle contigu, la fonction retourne une erreur.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple de décalage d'un ensemble de dates

La formule suivante calcule des dates qui sont antérieures d'un an aux dates dans le contexte actuel.

DAX

```
= DATEADD(DateTime[DateKey],-1,year)
```

Contenu connexe

Fonctions d'intelligence temporelle
Fonctions de date et heure

Commentaires

Cette page a-t-elle été utile ? Yes No

DATESBETWEEN

Article • 20/10/2023

Retourne une table contenant une colonne de dates qui commence à une date de début spécifiée et se poursuit jusqu'à une date de fin spécifiée.

Cette fonction peut être passée en tant que filtre à la fonction CALCULATE. Utilisez-la pour filtrer une expression sur une plage de dates personnalisée.

> ⓘ **Notes**
>
> Si vous spécifiez des intervalles de dates standard tels que des jours, des mois, des trimestres ou des années, il est recommandé d'utiliser la fonction **DATESINPERIOD**, qui est plus adaptée.

Syntaxe

```
DAX

DATESBETWEEN(<Dates>, <StartDate>, <EndDate>)
```

Paramètres

Agrandir le tableau

Terme	Définition
Dates	Colonne de date.
StartDate	Expression de date.
EndDate	Expression de date.

Valeur renvoyée

Table contenant une seule colonne de valeurs de date.

Remarques

- Dans le cas d'utilisation le plus courant, **Dates** est une référence à la colonne de date d'une table de dates marquée.
- Si **StartDate** est BLANK, alors **StartDate** sera la valeur la plus ancienne dans la colonne **Dates**.
- Si **EndDate** est BLANK, alors **EndDate** sera la dernière valeur dans la colonne **Dates**.
- Les dates utilisées comme **StartDate** et **EndDate** sont inclusives. Ainsi, par exemple, si la valeur **StartDate** est le 1er juillet 2019, cette date sera incluse dans la table renvoyée (à condition que la date existe dans la colonne **Dates**).
- La table renvoyée ne peut contenir que des dates stockées dans la colonne **Dates**. Ainsi, par exemple, si la colonne **Dates** commence le 1er juillet 2017 et que la valeur **StartDate** est le 1er juillet 2016, la table renvoyée commencera le 1er juillet 2017.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La définition de mesure de la table **Sales** suivante utilise la fonction DATESBETWEEN pour produire un calcul LTD (*life-to-date*). Life-to-date représente le cumul d'une mesure dans le temps depuis le tout début.

Notez que la formule utilise la fonction MAX. Cette fonction retourne la date la plus récente incluse dans le contexte de filtre. Par conséquent, la fonction DATESBETWEEN retourne une table de dates entre la date la plus ancienne et la date la plus récente.

Les exemples de cet article peuvent être utilisés avec l'exemple de modèle Adventure Works DW 2020 Power BI Desktop. Pour obtenir le modèle, consultez Exemple de modèle DAX.

DAX

```
Customers LTD =
CALCULATE(
    DISTINCTCOUNT(Sales[CustomerKey]),
    DATESBETWEEN(
        'Date'[Date],
        BLANK(),
        MAX('Date'[Date])
```

```
    )
)
```

Supposons que la date la plus ancienne présente dans la table **Date** soit le 1er juillet 2017. Quand un rapport filtre la mesure sur le mois de juin 2020, la fonction DATESBETWEEN retourne une plage de dates comprises entre le 1er juillet 2017 et le 30 juin 2020.

Contenu connexe

- Fonctions d'intelligence temporelle (DAX)
- Fonctions de date et heure (DAX)
- DATESINPERIOD, fonction (DAX)

Commentaires

Cette page a-t-elle été utile ? Yes No

DATESINPERIOD

Article • 20/10/2023

Retourne une table contenant une colonne de dates qui commence à une date de début spécifiée et se poursuit pendant le nombre et le type d'intervalles de dates spécifiés.

Cette fonction peut être passée en tant que filtre à la fonction CALCULATE. Utilisez-la pour filtrer une expression par intervalles de date standard (jours, mois, trimestres ou années).

Syntaxe

```
DAX

DATESINPERIOD(<dates>, <start_date>, <number_of_intervals>, <interval>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne de date.
start_date	Expression de date.
number_of_intervals	Entier qui spécifie le nombre d'intervalles à ajouter aux dates ou à soustraire des dates.
interval	Intervalle selon lequel les dates doivent être avancées ou reculées. Cet intervalle peut avoir l'une des valeurs suivantes : `DAY`, `MONTH`, `QUARTER` et `YEAR`

Valeur renvoyée

Table contenant une seule colonne de valeurs de date.

Remarques

- Dans le cas d'usage le plus courant, **dates** référence la colonne de date d'une table de dates marquée.

- Si le nombre spécifié pour **number_of_intervals** est positif, les dates sont avancées dans le temps ; s'il est négatif, les dates sont reculées dans le temps.
- Le paramètre **interval** est une énumération. Les valeurs valides sont `DAY`, `MONTH`, `QUARTER` et `YEAR`. Étant donné qu'il s'agit d'une énumération, les valeurs ne sont pas passées sous forme de chaînes. Vous ne devez donc pas les placer entre guillemets.
- La table retournée peut contenir uniquement des dates stockées dans la colonne **dates**. Par exemple, si la colonne **dates** commence au 1er juillet 2017 et que la valeur de **start_date** est le 1er juillet 2016, la table retournée commence au 1er juillet 2017.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La définition de mesure de la table **Sales** suivante utilise la fonction DATESINPERIOD pour calculer le chiffre d'affaires de l'année précédente (PY).

Notez que la formule utilise la fonction MAX. Cette fonction retourne la date la plus récente incluse dans le contexte de filtre. Par conséquent, la fonction DATESINPERIOD retourne une table de dates à partir de la date la plus récente de l'année dernière.

Les exemples de cet article peuvent être utilisés avec l'exemple de modèle Adventure Works DW 2020 Power BI Desktop. Pour obtenir le modèle, consultez Exemple de modèle DAX.

DAX

```
Revenue PY =
CALCULATE(
    SUM(Sales[Sales Amount]),
    DATESINPERIOD(
        'Date'[Date],
        MAX('Date'[Date]),
        -1,
        YEAR
    )
)
```

Supposons que le rapport est filtré sur le mois de juin 2020. La fonction MAX retourne la date du 30 juin 2020. La fonction DATESINPERIOD retourne ensuite une plage de dates

comprises entre le 1er juillet 2019 et le 30 juin 2020. Il s'agit d'une année de valeurs de date à partir du 30 juin 2020 de l'année dernière.

Contenu connexe

Fonctions d'intelligence temporelle (DAX)
Fonctions de date et heure (DAX)
DATESBETWEEN, fonction (DAX)

Commentaires

Cette page a-t-elle été utile ? Yes No

DATESMTD

Article • 14/12/2023

Retourne une table qui contient une colonne des dates du mois jusqu'à ce jour, dans le contexte actuel.

Syntaxe

DAX

```
DATESMTD(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne qui contient des dates.

Valeur renvoyée

Table contenant une seule colonne de valeurs de date.

Remarques

L'argument **dates** peut être l'un des éléments suivants :

- Une référence à une colonne de date/d'heure.
- Une expression de table qui retourne une seule colonne de valeurs de date/heure
- Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

> **Notes**
>
> Les contraintes sur les expressions booléennes sont décrites dans la rubrique **CALCULATE, fonction**.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L’exemple de formule suivant crée une mesure qui calcule le « cumul mensuel à ce jour » des ventes sur Internet.

DAX

```
= CALCULATE(SUM(InternetSales_USD[SalesAmount_USD]),
DATESMTD(DateTime[DateKey]))
```

Contenu connexe

Fonctions d’intelligence temporelle
Fonctions de date et heure
DATESYTD, fonction
DATESQTD, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

DATESQTD

Article • 14/12/2023

Retourne une table qui contient une colonne des dates du trimestre jusqu'à ce jour, dans le contexte actuel.

Syntaxe

DAX

```
DATESQTD(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne qui contient des dates.

Valeur renvoyée

Table contenant une seule colonne de valeurs de date.

Remarques

L'argument **dates** peut être l'un des éléments suivants :

- Une référence à une colonne de date/d'heure.
- Une expression de table qui retourne une seule colonne de valeurs de date/heure
- Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

> **Notes**
>
> Les contraintes sur les expressions booléennes sont décrites dans la rubrique **CALCULATE, fonction**.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L’exemple de formule suivant crée une mesure qui calcule le total cumulé trimestriel des ventes sur Internet.

DAX

```
= CALCULATE(SUM(InternetSales_USD[SalesAmount_USD]),
DATESQTD(DateTime[DateKey]))
```

Contenu connexe

Fonctions d’intelligence temporelle
Fonctions de date et heure
DATESYTD, fonction
DATESMTD, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

DATESYTD

Article • 20/10/2023

Retourne une table qui contient une colonne des dates de l'année jusqu'à ce jour, dans le contexte actuel.

Syntaxe

DAX

```
DATESYTD(<dates> [,<year_end_date>])
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne qui contient des dates.
year_end_date	(Facultatif) Chaîne littérale avec une date qui définit la date de fin d'année. La valeur par défaut est le 31 décembre.

Valeur renvoyée

Table contenant une seule colonne de valeurs de date.

Remarques

L'argument **dates** peut être l'un des éléments suivants :

- Une référence à une colonne de date/heure
- Une expression de table qui retourne une seule colonne de valeurs de date/heure
- Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

> ⓘ **Notes**

> Les contraintes sur les expressions booléennes sont décrites dans la rubrique **CALCULATE, fonction**.

- Le paramètre **year_end_date** est un littéral de chaîne d'une date. Ses paramètres régionaux sont identiques à ceux du client où le classeur a été créé. La partie année de la date est ignorée. Selon les paramètres régionaux, le format peut être « m-jj » ou « jj-m ».
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule le « total cumulé » des ventes sur Internet.

DAX

```
= CALCULATE(SUM(InternetSales_USD[SalesAmount_USD]),
DATESYTD(DateTime[DateKey]))
```

L'exemple de formule suivant crée une mesure qui calcule le « Total cumulé de l'année fiscale » pour les ventes sur Internet à l'aide des paramètres régionaux US pour le format Date.

DAX

```
= CALCULATE(
    SUM(InternetSales_USD[SalesAmount_USD]),
    DATESYTD(DateTime[DateKey],
        "6-30"
        )
)
```

Contenu connexe

Fonctions d'intelligence temporelle
Fonctions de date et heure
DATESMTD, fonction
DATESQTD, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

ENDOFMONTH

Article • 20/10/2023

Retourne la dernière date du mois dans le contexte actuel pour la colonne de dates spécifiée.

Syntaxe

```DAX
ENDOFMONTH(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne qui contient des dates.

Valeur de retour

Table contenant une seule colonne et une seule ligne avec une valeur de date.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui retourne la fin du mois, pour le contexte actuel.

```
DAX
= ENDOFMONTH(DateTime[DateKey])
```

Contenu connexe

Fonctions de date et heure
Fonctions d'intelligence temporelle
ENDOFYEAR, fonction
ENDOFQUARTER, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

ENDOFQUARTER

Article • 20/10/2023

Retourne la dernière date du trimestre dans le contexte actuel pour la colonne de dates spécifiée.

Syntaxe

DAX

```
ENDOFQUARTER(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne qui contient des dates.

Valeur de retour

Table contenant une seule colonne et une seule ligne avec une valeur de date.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/heure
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui retourne la fin du trimestre pour le contexte actuel.

```
DAX
= ENDOFQUARTER(DateTime[DateKey])
```

Contenu connexe

Fonctions de date et heure
Fonctions d'intelligence temporelle
ENDOFYEAR, fonction
ENDOFMONTH, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

ENDOFYEAR

Article • 20/10/2023

Retourne la dernière date de l'année dans le contexte actuel pour la colonne de dates spécifiée.

Syntaxe

DAX

```
ENDOFYEAR(<dates> [,<year_end_date>])
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne qui contient des dates.
year_end_date	(Facultatif) Chaîne littérale avec une date qui définit la date de fin d'année. La valeur par défaut est le 31 décembre.

Valeur de retour

Table contenant une seule colonne et une seule ligne avec une valeur de date.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/heure
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.

- Le paramètre **year_end_date** est un littéral de chaîne d'une date. Ses paramètres régionaux sont identiques à ceux du client où le classeur a été créé. La partie année de la date est ignorée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui retourne la fin de l'exercice se terminant le 30 juin pour le contexte actuel.

DAX

```
= ENDOFYEAR(DateTime[DateKey],"06/30/2004")
```

Contenu connexe

Fonctions de date et heure
Fonctions d'intelligence temporelle
ENDOFMONTH, fonction
ENDOFQUARTER, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

FIRSTDATE

Article • 14/12/2023

Retourne la première date dans le contexte actuel pour la colonne de dates spécifiée.

Syntaxe

```DAX
FIRSTDATE(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne qui contient des dates.

Valeur de retour

Table contenant une seule colonne et une seule ligne avec une valeur de date.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.
- Quand le contexte actuel est une date unique, les fonctions FIRSTDATE et LASTDATE retournent la même date.

- La valeur de retour est une table qui contient une seule colonne et une seule valeur. Par conséquent, cette fonction peut être utilisée comme argument d'une fonction qui exige une table dans ses arguments. De même, la valeur retournée peut être utilisée chaque fois qu'une valeur de date est nécessaire.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui obtient la première date à laquelle une vente a été réalisée dans le canal de ventes Internet pour le contexte actuel.

DAX

```
= FIRSTDATE('InternetSales_USD'[SaleDateKey])
```

Contenu connexe

Fonctions de date et heure
Fonctions d'intelligence temporelle
LASTDATE, fonction
FIRSTNONBLANK, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

FIRSTNONBLANK

Article • 14/12/2023

Retourne la première valeur de la colonne (**column**) filtrée par le contexte actuel, où l'expression n'est pas vide.

Syntaxe

```
DAX
FIRSTNONBLANK(<column>,<expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Expression de colonne.
expression	Expression dont les valeurs vides sont évaluées pour chaque valeur de **column**.

Valeur renvoyée

Table contenant une seule colonne et une seule ligne avec la première valeur calculée.

Remarques

- L'argument **column** peut être l'un des éléments suivants :
  - Référence à une colonne
  - Table contenant une seule colonne
- Expression booléenne qui définit une table à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.
- Cette fonction est généralement utilisée pour retourner la première valeur d'une colonne pour laquelle l'expression n'est pas vide. Par exemple, vous pouvez obtenir la dernière valeur correspondant aux ventes d'un produit.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Contenu connexe

LASTNONBLANK, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

FIRSTNONBLANKVALUE

Article • 20/10/2023

Évalue une expression filtrée par les valeurs triées d'une colonne et retourne la première valeur de l'expression qui n'est pas vide.

Syntaxe

DAX

```
FIRSTNONBLANKVALUE(<column>, <expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne ou expression qui retourne une table comprenant une seule colonne.
expression	Expression dont chaque valeur de <column> est évaluée.

Valeur retournée

Première valeur non vide de <expression> correspondant aux valeurs triées de <column>.

Remarques

- L'argument column peut être l'un des éléments suivants :
  - Référence à une colonne
  - Table contenant une seule colonne
- Cette fonction est différente de FIRSTNONBLANK, car la <column> est ajoutée au contexte de filtre pour l'évaluation de <expression>.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La requête DAX suivante :

```DAX
EVALUATE
SUMMARIZECOLUMNS(
  DimProduct[Class],
  "FNBV",
  FIRSTNONBLANKVALUE(
    DimDate[Date],
    SUM(FactInternetSales[SalesAmount])
   )
)
```

Retourne :

Agrandir le tableau

DimProduct[Class]	[FNBV]
L	699,0982
H	13778,24
M	1000,4375
	533,83

Commentaires

Cette page a-t-elle été utile ? Yes No

LASTDATE

Article • 20/10/2023

Retourne la dernière date dans le contexte actuel pour la colonne de dates spécifiée.

Syntaxe

DAX

```
LASTDATE(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne qui contient des dates.

Valeur de retour

Table contenant une seule colonne et une seule ligne avec une valeur de date.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/heure
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.
- Quand le contexte actuel est une date unique, les fonctions FIRSTDATE et LASTDATE retournent la même date.

- Techniquement, la valeur de retour est une table qui contient une seule colonne et une seule valeur. Par conséquent, cette fonction peut être utilisée comme argument d'une fonction qui exige une table dans ses arguments. De même, la valeur retournée peut être utilisée chaque fois qu'une valeur de date est nécessaire.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui obtient la dernière date pour le contexte actuel, à laquelle une vente a été réalisée dans le canal de ventes Internet.

DAX

```
= LASTDATE('InternetSales_USD'[SaleDateKey])
```

Contenu connexe

Fonctions de date et heure
Fonctions d'intelligence temporelle
FIRSTDATE, fonction
LASTNONBLANK, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

LASTNONBLANK

Article • 20/10/2023

Retourne la dernière valeur de la colonne (**column**) filtrée par le contexte actuel, où l'expression n'est pas vide.

Syntaxe

```
DAX

LASTNONBLANK(<column>,<expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Expression de colonne.
expression	Expression dont les valeurs vides sont évaluées pour chaque valeur de **column**.

Valeur renvoyée

Table contenant une seule colonne et une seule ligne avec la dernière valeur calculée.

Remarques

- L'argument **column** peut être l'un des éléments suivants :
  - Référence à une colonne
  - Table contenant une seule colonne
  - Expression booléenne qui définit une table à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.
- Cette fonction est généralement utilisée pour retourner la dernière valeur d'une colonne pour laquelle l'expression n'est pas vide. Par exemple, vous pouvez obtenir la dernière valeur correspondant aux ventes d'un produit.

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Contenu connexe

FIRSTNONBLANK, fonction
Fonctions statistiques

Commentaires

Cette page a-t-elle été utile ? Yes No

LASTNONBLANKVALUE

Article • 20/10/2023

Évalue une expression filtrée par les valeurs triées d'une colonne et retourne la dernière valeur de l'expression qui n'est pas vide.

Syntaxe

```
DAX

LASTNONBLANKVALUE(<column>, <expression>)
```

Paramètres

Agrandir le tableau

Terme	Définition
colonne	Colonne ou expression qui retourne une table comprenant une seule colonne.
expression	Expression dont chaque valeur de <column> est évaluée.

Valeur retournée

Dernière valeur non vide de <expression> correspondant aux valeurs triées de <column>.

Remarques

- L'argument column peut être l'un des éléments suivants :
  - Référence à une colonne
  - Table contenant une seule colonne
- Cette fonction est différente de LASTNONBLANK, car la <column> est ajoutée au contexte de filtre pour l'évaluation de <expression>.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

La requête DAX suivante :

```DAX
EVALUATE
SUMMARIZECOLUMNS(
  DimProduct[Class],
  "LNBV",
  LASTNONBLANKVALUE(
    DimDate[Date],
    SUM(FactInternetSales[SalesAmount])
   )
)
```

Retourne :

Agrandir le tableau

DimProduct[Class]	[LNBV]
L	132,44
H	137,6
M	84,97
	2288,6

Commentaires

Cette page a-t-elle été utile ? Yes No

NEXTDAY

Article • 20/10/2023

Retourne une table qui contient une colonne de toutes les dates du jour suivant, selon la première date spécifiée dans la colonne **dates** du contexte actuel.

Syntaxe

```
DAX

NEXTDAY(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne contenant des dates.

Valeur renvoyée

Table contenant une seule colonne de valeurs de date.

Remarques

- Cette fonction retourne toutes les dates du jour suivant jusqu'à la première date du paramètre d'entrée. Par exemple, si la première date de l'argument **dates** fait référence au 10 juin 2009, cette fonction retourne toutes les dates égales au 11 juin 2009.
- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule les ventes sur Internet du « jour suivant ».

DAX

```
= CALCULATE(SUM(InternetSales_USD[SalesAmount_USD]),
NEXTDAY('DateTime'[DateKey]))
```

Contenu connexe

Fonctions d'intelligence temporelle
Fonctions de date et heure
NEXTQUARTER, fonction
NEXTMONTH, fonction
NEXTYEAR, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

NEXTMONTH

Article • 20/10/2023

Retourne une table qui contient une colonne de toutes les dates du mois suivant, selon la première date dans la colonne **dates** du contexte actuel.

Syntaxe

DAX

```
NEXTMONTH(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne contenant des dates.

Valeur renvoyée

Table contenant une seule colonne de valeurs de date.

Remarques

- Cette fonction retourne toutes les dates, du jour suivant jusqu'à la première date du paramètre d'entrée. Par exemple, si la première date de l'argument **dates** référence le 10 juin 2009, cette fonction retourne toutes les dates du mois de juillet 2009.
- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule les ventes Internet du mois suivant.

DAX

```
= CALCULATE(SUM(InternetSales_USD[SalesAmount_USD]),
NEXTMONTH('DateTime'[DateKey]))
```

Contenu connexe

Fonctions d'intelligence temporelle
Fonctions de date et heure
NEXTDAY, fonction
NEXTQUARTER, fonction
NEXTYEAR, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

NEXTQUARTER

Article • 14/12/2023

Retourne une table qui contient une colonne de toutes les dates du trimestre suivant, selon la première date spécifiée dans la colonne **dates** du contexte actuel.

Syntaxe

DAX

```
NEXTQUARTER(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne contenant des dates.

Valeur renvoyée

Table contenant une seule colonne de valeurs de date.

Remarques

- Cette fonction retourne toutes les dates du trimestre suivant en fonction de la première date du paramètre d'entrée. Par exemple, si la première date de la colonne **dates** est le 10 juin 2009, cette fonction retourne toutes les dates du trimestre de juillet à septembre 2009.
- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule les ventes Internet du trimestre suivant.

DAX

```
= CALCULATE(SUM(InternetSales_USD[SalesAmount_USD]),
NEXTQUARTER('DateTime'[DateKey]))
```

Contenu connexe

Fonctions d'intelligence temporelle
Fonctions de date et heure
NEXTDAY, fonction
NEXTMONTH, fonction
NEXTYEAR, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

NEXTYEAR

Article • 14/12/2023

Retourne une table qui contient une colonne de toutes les dates de l'année suivante, selon la première date dans la colonne **dates** du contexte actuel.

Syntaxe

DAX

```
NEXTYEAR(<dates>[,<year_end_date>])
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne contenant des dates.
year_end_date	(Facultatif) Chaîne littérale avec une date qui définit la date de fin d'année. La valeur par défaut est le 31 décembre.

Valeur renvoyée

Table contenant une seule colonne de valeurs de date.

Remarques

- Cette fonction retourne toutes les dates de l'année suivante en fonction de la première date de la colonne d'entrée. Par exemple, si la première date de la colonne **dates** fait référence à l'année 2007, la fonction retourne toutes les dates de l'année 2008.
- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure

- Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.
- Le paramètre **year_end_date** est un littéral de chaîne d'une date. Ses paramètres régionaux sont identiques à ceux du client où le classeur a été créé. La partie année de la date est ignorée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule les ventes Internet de « l'année suivante ».

DAX

```
= CALCULATE(SUM(InternetSales_USD[SalesAmount_USD]),
NEXTYEAR('DateTime'[DateKey]))
```

Contenu connexe

Fonctions d'intelligence temporelle
Fonctions de date et heure
NEXTDAY, fonction
NEXTQUARTER, fonction
NEXTMONTH, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

OPENINGBALANCEMONTH

Article • 14/12/2023

Évalue l'**expression** à la première date du mois dans le contexte actuel.

Syntaxe

```
DAX
OPENINGBALANCEMONTH(<expression>,<dates>[,<filter>])
```

Paramètres

Agrandir le tableau

Terme	Définition
expression	Expression retournant une valeur scalaire.
dates	Colonne contenant des dates.
filter	(Facultatif) Expression qui spécifie un filtre à appliquer au contexte actuel.

Valeur de retour

Valeur scalaire qui représente l'**expression** évaluée à la première date du mois dans le contexte actuel.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.

- L'expression **filter** présente les restrictions décrites dans la rubrique CALCULATE, fonction.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule la valeur d'inventaire de début de mois des produits.

DAX

```
=
OPENINGBALANCEMONTH(SUMX(ProductInventory,ProductInventory[UnitCost]*Product
Inventory[UnitsBalance]),DateTime[DateKey])
```

Contenu connexe

OPENINGBALANCEYEAR, fonction
OPENINGBALANCEQUARTER, fonction
Fonctions d'intelligence temporelle
CLOSINGBALANCEMONTH, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

OPENINGBALANCEQUARTER

Article • 14/12/2023

Évalue la valeur **expression** à la première date du trimestre dans le contexte actuel.

Syntaxe

DAX

```
OPENINGBALANCEQUARTER(<expression>,<dates>[,<filter>])
```

Paramètres

Agrandir le tableau

Terme	Définition
expression	Expression retournant une valeur scalaire.
dates	Colonne qui contient des dates.
filte*	(Facultatif) Expression qui spécifie un filtre à appliquer au contexte actuel.

Valeur de retour

Valeur scalaire qui représente la valeur **expression** évaluée à la première date du trimestre dans le contexte actuel.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.

- L'expression **filter** présente les restrictions décrites dans la rubrique CALCULATE, fonction.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule la valeur d'inventaire de début de trimestre de l'inventaire des produits.

DAX

```
=
OPENINGBALANCEQUARTER(SUMX(ProductInventory,ProductInventory[UnitCost]*Produ
ctInventory[UnitsBalance]),DateTime[DateKey])
```

Contenu connexe

OPENINGBALANCEYEAR, fonction
OPENINGBALANCEMONTH, fonction
Fonctions d'intelligence temporelle
CLOSINGBALANCEQUARTER, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

OPENINGBALANCEYEAR

Article • 20/10/2023

Évalue l'**expression** à la première date de l'année dans le contexte actuel.

Syntaxe

DAX

```
OPENINGBALANCEYEAR(<expression>,<dates>[,<filter>][,<year_end_date>])
```

Paramètres

Agrandir le tableau

Terme	Définition
expression	Expression retournant une valeur scalaire.
dates	Colonne contenant des dates.
filter	(Facultatif) Expression qui spécifie un filtre à appliquer au contexte actuel.
year_end_date	(Facultatif) Chaîne littérale avec une date qui définit la date de fin d'année. La valeur par défaut est le 31 décembre.

Valeur de retour

Valeur scalaire qui représente l'**expression** évaluée à la première date de l'année dans le contexte actuel.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.
- L'expression **filter** présente les restrictions décrites dans la rubrique CALCULATE, fonction.
- Le paramètre **year_end_date** est un littéral de chaîne d'une date. Ses paramètres régionaux sont identiques à ceux du client où le classeur a été créé. La partie année de la date est ignorée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule la valeur d'inventaire de début d'année des produits.

DAX

```
=
OPENINGBALANCEYEAR(SUMX(ProductInventory,ProductInventory[UnitCost]*ProductI
nventory[UnitsBalance]),DateTime[DateKey])
```

Contenu connexe

OPENINGBALANCEQUARTER, fonction
OPENINGBALANCEMONTH, fonction
Fonctions d'intelligence temporelle
CLOSINGBALANCEYEAR, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

PARALLELPERIOD

Article • 20/10/2023

Retourne une table qui contient une colonne de dates représentant une période parallèle aux dates de la colonne **dates** spécifiée, dans le contexte actuel, avec les dates décalées d'un certain nombre d'intervalles en avançant ou en remontant dans le temps.

Syntaxe

DAX

```
PARALLELPERIOD(<dates>,<number_of_intervals>,<interval>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne qui contient des dates.
number_of_intervals	Entier qui spécifie le nombre d'intervalles à ajouter aux dates ou à soustraire des dates.
interval	Intervalle selon lequel les dates doivent être avancées ou reculées. Cet intervalle peut avoir l'une des valeurs suivantes : `year`, `quarter`, `month`.

Valeur de retour

Table contenant une seule colonne de valeurs de date.

Remarques

- Cette fonction prend l'ensemble de dates actuel dans la colonne spécifiée par **dates**, décale la première date et la dernière date du nombre d'intervalles spécifié, puis retourne toutes les dates contiguës figurant entre les deux dates décalées. Si l'intervalle est une plage partielle du mois, du trimestre ou de l'année, tous les mois partiels dans le résultat sont également remplis pour compléter l'intervalle entier.

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/heure
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.

- Si le nombre spécifié pour **number_of_intervals** est positif, les dates dans la colonne **dates** sont avancées dans le temps ; s'il est négatif, les dates dans la colonne **dates** sont reculées dans le temps.

- Le paramètre **interval** est une énumération, pas un ensemble de chaînes ; les valeurs ne doivent donc pas être placées entre guillemets. En outre, les valeurs `year`, `quarter` et `month` doivent être épelées intégralement lorsque vous les utilisez.

- La table de résultats comprend uniquement les dates qui figurent dans les valeurs de la colonne de table sous-jacente.

- La fonction PARALLELPERIOD est similaire à la fonction DATEADD, si ce n'est que PARALLELPERIOD retourne toujours des périodes entières au niveau de granularité donné, alors que des périodes partielles sont retournées par DATEADD. Par exemple, si vous avez sélectionné des dates qui commencent au 10 juin et se terminent le 21 juin de la même année, et que vous souhaitez décaler cette sélection d'un mois vers l'avant, la fonction PARALLELPERIOD retourne toutes les dates du mois suivant (du 1er juillet au 31 juillet). Toutefois, si la fonction DATEADD est utilisée à la place, le résultat inclut uniquement les dates du 10 juillet au 21 juillet.

- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule les ventes Internet de l'année précédente.

```
DAX
```

```
= CALCULATE(SUM(InternetSales_USD[SalesAmount_USD]),
PARALLELPERIOD(DateTime[DateKey],-1,year))
```

Contenu connexe

Fonctions d'intelligence temporelle
Fonctions de date et heure
DATEADD, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

PREVIOUSDAY

Article • 20/10/2023

Retourne une table qui contient une colonne de toutes les dates représentant le jour qui précède la première date de la colonne **dates**, dans le contexte actuel.

Syntaxe

```
DAX
PREVIOUSDAY(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne contenant des dates.

Valeur renvoyée

Table contenant une seule colonne de valeurs de date.

Remarques

- Cette fonction détermine la première date dans le paramètre d'entrée, puis retourne toutes les dates correspondant au jour précédant cette première date. Par exemple, si la première date dans l'argument **dates** fait référence au 10 juin 2009, cette fonction retourne toutes les dates égales au 9 juin 2009.
- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE, fonction.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule les ventes Internet du jour précédent.

DAX

```
= CALCULATE(SUM(InternetSales_USD[SalesAmount_USD]),
PREVIOUSDAY('DateTime'[DateKey]))
```

Contenu connexe

Fonctions d'intelligence temporelle
Fonctions de date et heure
PREVIOUSMONTH, fonction
PREVIOUSQUARTER, fonction
PREVIOUSYEAR, fonction

Commentaires

Cette page a-t-elle été utile ? Yes No

PREVIOUSMONTH

Article • 20/10/2023

Renvoie une table contenant une colonne de toutes les dates du mois précédent, en fonction de la première date de la colonne **Dates**, dans le contexte actuel.

Syntaxe

DAX

```
PREVIOUSMONTH(<Dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
Dates	Colonne contenant des dates.

Valeur renvoyée

Table contenant une seule colonne de valeurs de date.

Remarques

- Cette fonction retourne toutes les dates du mois précédent en utilisant la première date de la colonne d'entrée. Par exemple, si la première date de l'argument **Dates** fait référence au 10 juin 2009, cette fonction renvoie toutes les dates du mois de mai 2009.
- L'argument **Dates** peut être l'un des suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule les « ventes du mois précédent » pour les ventes sur Internet.

DAX

```
= CALCULATE(SUM(InternetSales_USD[SalesAmount_USD]),
PREVIOUSMONTH('DateTime'[DateKey]))
```

Contenu connexe

Fonctions d'intelligence temporelle
Fonctions de date et heure
PREVIOUSDAY
PREVIOUSQUARTER
PREVIOUSYEAR

Commentaires

Cette page a-t-elle été utile ? Yes No

PREVIOUSQUARTER

Article • 14/12/2023

Retourne une table qui contient une colonne de toutes les dates du trimestre précédent, basée sur la première date de la colonne **dates**, dans le contexte actuel.

Syntaxe

DAX

```
PREVIOUSQUARTER(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne contenant des dates.

Valeur renvoyée

Table contenant une seule colonne de valeurs de date.

Remarques

- Cette fonction retourne toutes les dates du trimestre précédent en utilisant la première date de la colonne d'entrée. Par exemple, si la première date de l'argument **dates** est le 10 juin 2009, cette fonction retourne toutes les dates du trimestre de janvier à mars 2009.
- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule les « ventes du trimestre précédent » pour les ventes sur Internet.

DAX

```
= CALCULATE(SUM(InternetSales_USD[SalesAmount_USD]),
PREVIOUSQUARTER('DateTime'[DateKey]))
```

Contenu connexe

Fonctions d'intelligence temporelle
Fonctions de date et heure
PREVIOUSMONTH
PREVIOUSDAY
PREVIOUSYEAR

Commentaires

Cette page a-t-elle été utile ? Yes No

PREVIOUSYEAR

Article • 20/10/2023

Retourne une table qui contient une colonne de toutes les dates de l'année précédente, selon la dernière date de la colonne **dates**, dans le contexte actuel.

Syntaxe

DAX

```
PREVIOUSYEAR(<dates>[,<year_end_date>])
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne contenant des dates.
year_end_date	(Facultatif) Chaîne littérale avec une date qui définit la date de fin d'année. La valeur par défaut est le 31 décembre.

Valeur renvoyée

Table contenant une seule colonne de valeurs de date.

Remarques

- Cette fonction retourne toutes les dates de l'année précédente selon la dernière date du paramètre d'entrée. Par exemple, si la date la plus récente de l'argument **dates** fait référence à l'année 2009, cette fonction retourne toutes les dates pour l'année 2008 jusqu'à la date **year_end_date** spécifiée.

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure

    - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE.
- Le paramètre **year_end_date** est un littéral de chaîne d'une date. Ses paramètres régionaux sont identiques à ceux du client où le classeur a été créé. La partie année de la date est ignorée.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule les ventes Internet de l'année précédente.

DAX

```
= CALCULATE(SUM(InternetSales_USD[SalesAmount_USD]),
PREVIOUSYEAR('DateTime'[DateKey]))
```

Contenu connexe

Fonctions d'intelligence temporelle
Fonctions de date et heure
PREVIOUSMONTH
PREVIOUSDAY
PREVIOUSQUARTER

Commentaires

Cette page a-t-elle été utile ? Yes No

SAMEPERIODLASTYEAR

Article • 20/10/2023

Retourne une table qui contient une colonne de dates déplacées d'une année en remontant dans le temps à partir des dates de la colonne **dates** spécifiée dans le contexte actuel.

Syntaxe

DAX

```
SAMEPERIODLASTYEAR(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne contenant des dates.

Valeur renvoyée

Table à une seule colonne de valeurs de date.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/heure
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE.
- Les dates retournées sont les mêmes que celles retournées par cette formule équivalente : `DATEADD(dates, -1, year)`

- Cette fonction n’est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L’exemple de formule suivant crée une mesure qui calcule les ventes de l’année précédente du revendeur.

DAX

```
= CALCULATE(SUM(ResellerSales_USD[SalesAmount_USD]),
SAMEPERIODLASTYEAR(DateTime[DateKey]))
```

Contenu connexe

Fonctions d’intelligence temporelle
Fonctions de date et heure
PREVIOUSYEAR
PARALLELPERIOD

Commentaires

Cette page a-t-elle été utile ? Yes No

STARTOFMONTH

Article • 20/10/2023

Retourne la première date du mois dans le contexte actuel pour la colonne de dates spécifiée.

Syntaxe

```
DAX

STARTOFMONTH(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne qui contient des dates.

Valeur de retour

Table contenant une seule colonne et une seule ligne avec une valeur de date.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui retourne le début du mois pour le contexte actuel.

```
DAX
= STARTOFMONTH(DateTime[DateKey])
```

Contenu connexe

Fonctions de date et heure
Fonctions d'intelligence temporelle
STARTOFYEAR
STARTOFQUARTER

Commentaires

Cette page a-t-elle été utile ? Yes No

STARTOFQUARTER

Article • 14/12/2023

Retourne la première date du trimestre dans le contexte actuel pour la colonne de dates spécifiée.

Syntaxe

```
DAX

STARTOFQUARTER(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne qui contient des dates.

Valeur de retour

Table contenant une seule colonne et une seule ligne avec une valeur de date.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui retourne le début du trimestre, pour le contexte actuel.

```
DAX
= STARTOFQUARTER(DateTime[DateKey])
```

Contenu connexe

Fonctions de date et heure
Fonctions d'intelligence temporelle
STARTOFYEAR
STARTOFMONTH

Commentaires

Cette page a-t-elle été utile ? Yes No

STARTOFYEAR

Article • 14/12/2023

Retourne la première date de l'année dans le contexte actuel pour la colonne de dates spécifiée.

Syntaxe

```
DAX
STARTOFYEAR(<dates>)
```

Paramètres

Agrandir le tableau

Terme	Définition
dates	Colonne qui contient des dates.
YearEndDate	(Facultatif) Valeur de date de fin d'année.

Valeur de retour

Table contenant une seule colonne et une seule ligne avec une valeur de date.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au

niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui retourne le début de l'année pour le contexte actuel.

```
DAX
= STARTOFYEAR(DateTime[DateKey])
```

Contenu connexe

Fonctions de date et heure
Fonctions d'intelligence temporelle
STARTOFQUARTER
STARTOFMONTH

Commentaires

Cette page a-t-elle été utile ? Yes No

TOTALMTD

Article • 20/10/2023

Évalue la valeur de l'**expression** pour le mois en cours jusqu'à ce jour dans le contexte actuel.

Syntaxe

DAX

```
TOTALMTD(<expression>,<dates>[,<filter>])
```

Paramètres

Agrandir le tableau

Paramètre	Définition
expression	Expression retournant une valeur scalaire.
dates	Colonne contenant des dates.
filter	(Facultatif) Expression qui spécifie un filtre à appliquer au contexte actuel.

Valeur de retour

Valeur scalaire qui représente l'**expression** évaluée pour le mois en cours jusqu'à ce jour, par rapport à la colonne **dates**.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE.

- Des restrictions s'appliquent à l'expression **filter** ; elles sont décrites dans la rubrique CALCULATE.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule la somme ou le total des ventes Internet du mois en cours jusqu'à ce jour.

DAX

```
= TOTALMTD(SUM(InternetSales_USD[SalesAmount_USD]),DateTime[DateKey])
```

Contenu connexe

ALL
CALCULATE
TOTALYTD
TOTALQTD

Commentaires

Cette page a-t-elle été utile ? Yes No

TOTALQTD

Article • 14/12/2023

Évalue la valeur de l'**expression** pour les dates du trimestre jusqu'à ce jour, dans le contexte actuel.

Syntaxe

DAX

```
TOTALQTD(<expression>,<dates>[,<filter>])
```

Paramètres

Agrandir le tableau

Paramètre	Définition
expression	Expression retournant une valeur scalaire.
dates	Colonne contenant des dates.
filter	(Facultatif) Expression qui spécifie un filtre à appliquer au contexte actuel.

Valeur de retour

Valeur scalaire qui représente l'**expression** évaluée pour toutes les dates du trimestre en cours jusqu'à ce jour, selon les dates figurant dans **dates**.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne
- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE.

- Des restrictions s'appliquent à l'expression **filter** ; elles sont décrites dans la rubrique CALCULATE.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule la somme ou le total des ventes Internet cumulé sur le trimestre.

DAX

```
= TOTALQTD(SUM(InternetSales_USD[SalesAmount_USD]),DateTime[DateKey])
```

Contenu connexe

ALL
CALCULATE
TOTALYTD
TOTALMTD

Commentaires

Cette page a-t-elle été utile ? Yes No

TOTALYTD

Article • 20/10/2023

Évalue la valeur de l'**expression** pour l'année en cours jusqu'à ce jour dans le contexte actuel.

Syntaxe

```
DAX

TOTALYTD(<expression>,<dates>[,<filter>][,<year_end_date>])
```

Paramètres

Agrandir le tableau

Paramètre	Définition
expression	Expression retournant une valeur scalaire.
dates	Colonne contenant des dates.
filter	(Facultatif) Expression qui spécifie un filtre à appliquer au contexte actuel.
year_end_date	(Facultatif) Chaîne littérale avec une date qui définit la date de fin d'année. La valeur par défaut est le 31 décembre.

Valeur de retour

Valeur scalaire qui représente l'**expression** évaluée pour l'année en cours jusqu'à ce jour, par rapport à la colonne **dates**.

Remarques

- L'argument **dates** peut être l'un des éléments suivants :
  - Une référence à une colonne de date/d'heure.
  - Une expression de table qui retourne une seule colonne de valeurs de date/heure
  - Une expression booléenne qui définit une table de valeurs de date/heure à une seule colonne

- Les contraintes sur les expressions booléennes sont décrites dans la rubrique CALCULATE.
- Des restrictions s'appliquent à l'expression **filter** ; elles sont décrites dans la rubrique CALCULATE.
- Le paramètre **year_end_date** est un littéral de chaîne d'une date. Ses paramètres régionaux sont identiques à ceux du client où le classeur a été créé. La partie année de la date n'est pas nécessaire et est ignorée. Par exemple, la formule suivante spécifie le paramètre year_end_date 6/30 (exercice) pour un classeur dont les paramètres régionaux sont définis sur en-US.

  DAX

  ```
  = TOTALYTD(SUM(InternetSales_USD[SalesAmount_USD]),DateTime[DateKey],
  ALL('DateTime'), "6/30")
  ```

  Dans cet exemple, year_end_date peut être spécifié sous les formes « 6/30 », « juin 30 » ou « 30 juin » ou par toute chaîne résolue en mois/jour. Toutefois, il est recommandé de spécifier le paramètre year_end_date au format « mois/jour » (comme indiqué) pour garantir la résolution correcte de la chaîne en date.
- Cette fonction n'est pas prise en charge pour une utilisation en mode DirectQuery quand elle est utilisée dans des colonnes calculées ou des règles de sécurité au niveau des lignes (RLS).

Exemple

L'exemple de formule suivant crée une mesure qui calcule la somme ou le total des ventes Internet de l'année en cours jusqu'à ce jour.

DAX

```
= TOTALYTD(SUM(InternetSales_USD[SalesAmount_USD]),DateTime[DateKey])
```

Contenu connexe

ALL
CALCULATE
DATESYTD
TOTALMTD
TOTALQTD

Commentaires

Cette page a-t-elle été utile ? Yes No

Instructions

Article • 20/10/2023

Dans cette catégorie

Agrandir le tableau

.	Description
DEFINE	(Mot clé) Introduit une instruction avec une ou plusieurs définitions d'entité qui peuvent être appliquées à une ou plusieurs instructions EVALUATE.
EVALUATE	(Mot clé) Introduit une instruction contenant une expression de table nécessaire pour exécuter une requête DAX.
MEASURE	(Mot clé) Introduit une définition de mesure qui peut être utilisée dans une ou plusieurs instructions EVALUATE d'une requête.
ORDER BY	(Mot clé) Introduit une instruction qui définit l'ordre de tri des résultats de la requête retournés par une instruction EVALUATE.
START AT	(Mot clé) Introduit une instruction qui définit la valeur de départ à laquelle sont retournés les résultats de la requête d'une instruction ORDER BY.
VAR	(Mot clé) Stocke le résultat d'une expression sous forme de variable nommée, qui peut ensuite être passée comme argument à d'autres expressions de mesure.

Commentaires

Cette page a-t-elle été utile ? Yes No

DÉFINIR

Article • 20/10/2023

Introduit une instruction avec une ou plusieurs définitions d'entité qui peuvent être appliquées à une ou plusieurs instructions EVALUATE d'une requête DAX.

Syntaxe

DAX

```
[DEFINE
    (
     (MEASURE <table name>[<measure name>] = <scalar expression>) |
     (VAR <var name> = <table or scalar expression>) |
     (TABLE <table name> = <table expression>) |
     (COLUMN <table name>[<column name>] = <scalar expression>) |
    ) +
]

(EVALUATE <table expression>) +
```

Paramètres

Agrandir le tableau

Terme	Définition
Entité	MEASURE, VAR, TABLE[1] ou COLUMN[1].
name	Nom d'une définition de mesure, variable, table ou colonne. Il ne peut pas s'agir d'une expression. Le nom n'a pas besoin d'être unique. Le nom existe seulement pour la durée de la requête.
expression	Toute expression DAX qui retourne une table ou une valeur scalaire. L'expression peut utiliser n'importe quelle entité définie. Si vous devez convertir une expression scalaire en expression de table, wrappez l'expression dans un constructeur de table avec des accolades `{}` ou utilisez la fonction `ROW()` pour retourner une seule table de lignes.

[1]**Attention :** Les définitions de TABLE et COLUMN limitées aux requêtes sont destinées à une utilisation interne uniquement. Même si vous pouvez définir des expressions TABLE et COLUMN pour une requête sans erreur de syntaxe, elles peuvent produire des erreurs d'exécution et ne sont pas recommandées.

Remarques

- Une requête DAX peut avoir plusieurs instructions EVALUATE, mais seulement une instruction DEFINE. Les définitions de l'instruction DEFINE peuvent s'appliquer à toutes les instructions EVALUATE de la requête.
- Au moins une définition est obligatoire dans une instruction DEFINE.
- Les définitions de mesure pour une requête remplacent les mesures d'un modèle du même nom.
- Les noms de VAR ont des restrictions uniques. Pour en savoir plus, consultez VAR - Paramètres.
- Pour en savoir plus sur l'utilisation d'une instruction DEFINE, consultez Requêtes DAX.

Contenu connexe

EVALUATE
VAR
MEASURE
Requêtes DAX

Commentaires

Cette page a-t-elle été utile ? Yes No

EVALUATE

Article • 14/12/2023

Introduit une instruction contenant une expression de table nécessaire dans une requête DAX.

Syntaxe

```DAX
EVALUATE <table>
```

Paramètres

Agrandir le tableau

Terme	Définition
tableau	Expression de table

Valeur de retour

Résultat d'une expression de table.

Remarques

- Une requête DAX peut contenir plusieurs instructions EVALUATE.
- Pour en savoir plus sur l'utilisation des instructions EVALUATE, consultez Requêtes DAX.

Exemple

```DAX
EVALUATE
    'Internet Sales'
```

Retourne toutes les lignes et colonnes de la table Internet Sales sous la forme d'une table.

Contenu connexe

ORDER BY
START AT
DEFINE
VAR
Requêtes DAX

Commentaires

Cette page a-t-elle été utile ? Yes No

MEASURE

Article • 20/10/2023

Introduit une définition de mesure dans une instruction DEFINE d'une requête DAX.

Syntaxe

DAX

```
[DEFINE
    (
      MEASURE <table name>[<measure name>] = <scalar expression>
    ) +
]

(EVALUATE <table expression>) +
```

Paramètres

Agrandir le tableau

Terme	Définition
Nom de la table	Nom d'une table contenant la mesure.
measure name	Nom de la mesure. Il ne peut pas s'agir d'une expression. Le nom n'a pas besoin d'être unique. Le nom existe seulement pour la durée de la requête.
scalar_expression	Expression DAX qui retourne une valeur scalaire.

Valeur retournée

Résultat calculé de l'expression de mesure.

Remarques

- Les définitions de mesure pour une requête remplacent les mesures du modèle du même nom pendant la durée de la requête. Elles n'affectent pas la mesure du modèle.

- L'expression de mesure peut être utilisée avec n'importe quelle autre expression dans la même requête.
- Pour en savoir plus sur l'utilisation des instructions MEASURE, consultez Requêtes DAX.

Contenu connexe

DEFINE
EVALUATE
VAR
Requêtes DAX

Commentaires

Cette page a-t-elle été utile ? Yes No

ORDER BY

Article • 14/12/2023

Introduit une instruction qui définit l'ordre de tri des résultats de requête retournés par une instruction EVALUATE dans une requête DAX.

Syntaxe

DAX

```
[ORDER BY {<expression> [{ASC | DESC}]}[, …]]
```

Paramètres

Agrandir le tableau

Terme	Définition
expression	Toute expression DAX qui retourne une valeur scalaire unique.
ASC	Ordre de tri croissant (par défaut).
DESC	Ordre de tri décroissant.

Valeur renvoyée

Résultat d'une instruction EVALUATE dans l'ordre croissant (ASC) ou décroissant (DESC).

Remarques

Pour en savoir plus sur l'utilisation des instructions ORDER BY, consultez Requêtes DAX.

Contenu connexe

START AT
EVALUATE
VAR
DEFINE
Requêtes DAX

Commentaires

Cette page a-t-elle été utile ? Yes No

START AT

Article • 14/12/2023

Introduit une instruction qui définit la valeur de départ à laquelle sont retournés les résultats de requête d'une clause ORDER BY dans une instruction EVALUATE d'une requête DAX.

Syntaxe

DAX

```
[START AT {<value>|<parameter>} [, …]]
```

Paramètres

Agrandir le tableau

Terme	Définition
valeur	Valeur constante. Il ne peut pas s'agir d'une expression.
parameter	Nom d'un paramètre dans une instruction XMLA précédé du caractère `@`.

Remarques

- Les arguments de START AT ont une correspondance un-à-un avec les colonnes de l'instruction ORDER BY. Il peut y avoir autant d'arguments dans l'instruction START AT que dans l'instruction ORDER BY, mais pas plus. Le premier argument de l'instruction START AT définit la valeur de départ de la colonne 1 des colonnes ORDER BY. Le deuxième argument de l'instruction START AT définit la valeur de départ de la colonne 2 des colonnes ORDER BY dans les lignes correspondant à la première valeur de la colonne 1.
- Pour en savoir plus sur l'utilisation des instructions START AT, consultez Requêtes DAX.

Contenu connexe

ORDER BY
EVALUATE
VAR
DEFINE
Requêtes DAX

Commentaires

Cette page a-t-elle été utile ? Yes No

VAR

Article • 20/10/2023

Stocke le résultat d'une expression sous forme de variable nommée, qui peut ensuite être passée comme argument à d'autres expressions de mesure. Une fois que les valeurs obtenues ont été calculées pour une expression de variable, ces valeurs ne changent pas, même si la variable est référencée dans une autre expression.

Syntaxe

DAX

```
VAR <name> = <expression>
```

Paramètres

Agrandir le tableau

Terme	Définition
name	Nom de la variable (identificateur). Les délimiteurs ne sont pas pris en charge. Par exemple, 'varName' ou [varName] génèrent une erreur. Jeu de caractères pris en charge : a-z, A-Z, 0-9. Les caractères 0-9 ne peuvent pas être utilisés comme premier caractère. __ (double trait de soulignement) est autorisé comme préfixe pour le nom de l'identificateur. Aucun autre caractère spécial n'est pris en charge. Les mots clés réservés ne sont pas autorisés. Les noms des tables existantes ne sont pas autorisés. Les espaces vides ne sont pas autorisées.
expression	Expression DAX qui retourne une valeur scalaire ou de table.

Valeur de retour

Variable nommée contenant le résultat de l'argument d'expression.

Notes

- Une expression passée comme argument à VAR peut contenir une autre déclaration VAR.
- Lors du référencement d'une variable :
  - Les mesures ne peuvent pas faire référence à des variables définies en dehors de l'expression de mesure, mais peuvent faire référence à des variables d'étendue fonctionnelle définies dans l'expression.
  - Les variables peuvent faire référence à des mesures.
  - Les variables peuvent faire référence à des variables définies précédemment.
  - Les colonnes contenues dans des variables de table ne peuvent pas être référencées au moyen d'une syntaxe TableName[ColumnName].
- Pour connaître les meilleures pratiques lors de l'utilisation de VAR, consultez Utiliser des variables pour améliorer vos formules DAX.
- Pour en savoir plus sur l'utilisation de VAR dans une requête DAX, consultez Requêtes DAX.

Exemple

Pour calculer un pourcentage de croissance d'une année sur l'autre sans utiliser de variable, vous pouvez créer trois mesures distinctes. Cette première mesure calcule la somme des montants des ventes :

DAX

```
Sum of SalesAmount = SUM(SalesTable[SalesAmount])
```

Une deuxième mesure calcule le montant des ventes pour l'année précédente :

DAX

```
SalesAmount PreviousYear =
    CALCULATE([Sum of SalesAmount],
    SAMEPERIODLASTYEAR(Calendar[Date])
    )
```

Vous pouvez ensuite créer une troisième mesure qui combine les deux autres mesures pour calculer le pourcentage de croissance. Notez que la mesure Sum of SalesAmount est utilisée à deux reprises : une première fois pour déterminer s'il existe une vente, et une deuxième fois pour calculer un pourcentage.

DAX

```
Sum of SalesAmount YoY%: =
    IF([Sum of SalesAmount] ,
        DIVIDE(([Sum of SalesAmount] - [SalesAmount PreviousYear]), [Sum of
SalesAmount])
    )
```

En utilisant une variable, vous pouvez créer une seule mesure qui calcule le même résultat :

DAX

```
YoY% = VAR Sales = SUM(SalesTable[SalesAmount])

VAR SalesLastYear =
    CALCULATE ( SUM ( SalesTable[SalesAmount] ), SAMEPERIODLASTYEAR (
'Calendar'[Date] ) )

    return if(Sales, DIVIDE(Sales - SalesLastYear, Sales))
```

En utilisant une variable, vous pouvez obtenir le même résultat, mais de façon plus lisible. Par ailleurs, comme le résultat de l'expression est stocké dans la variable, les performances de la mesure peuvent être considérablement améliorées, car elle n'a pas besoin d'être recalculée à chaque utilisation.

Contenu connexe

Utiliser des variables pour améliorer vos formules DAX
Requêtes DAX

Commentaires

Cette page a-t-elle été utile ? Yes No

Glossaire DAX

Article • 20/10/2023

Requête analytique

Les visuels Power BI interrogent un modèle de données à l'aide d'une *requête analytique*. Une requête analytique s'efforce de réduire les volumes de données potentiellement importants et les complexités des modèles en trois phases distinctes : filtrage, regroupement et résumé. Une requête analytique est créée automatiquement dès lors que des champs sont affectés aux puits de visuels de rapport. Les auteurs de rapports peuvent contrôler le comportement des affectations de champs en renommant les champs, en modifiant la technique de résumé ou en désactivant le résumé pour obtenir un regroupement. Pendant la phase de conception d'un rapport, il est possible d'ajouter des filtres au rapport, à une page de rapport ou à un visuel. En mode lecture, les filtres peuvent être modifiés dans le volet **Filtres** ou par des interactions avec des segments et d'autres visuels (filtrage croisé).

BLANK

DAX définit l'absence d'une valeur avec BLANK. C'est l'équivalent de la valeur NULL en SQL, mais leur comportement n'est pas tout à fait identique. Il s'inscrit davantage dans la logique d'Excel dans sa façon de définir une cellule vide. BLANK est évalué à zéro ou à une chaîne vide quand il est combiné avec d'autres opérations. Par exemple, BLANK + 20 = 20. Utilisez toujours des majuscules ; le pluriel est BLANKs, avec un « s » minuscule.

Colonne calculée

Calcul de modèle qui vise à ajouter une colonne à un modèle tabulaire en écrivant une formule DAX. La formule doit retourner une valeur scalaire, et elle est évaluée pour chaque ligne de la table. Une colonne calculée peut être ajoutée à une table en mode de stockage Importer ou DirectQuery.

Mesure calculée

Dans la modélisation tabulaire, il n'existe pas ce concept de *mesure calculée*. Il y est plutôt question de *mesure*. Le mot *calculé* est utilisé pour décrire les tables calculées et les colonnes calculées. Il s'agit d'un moyen de les distinguer des tables et des colonnes issues de Power Query. Le concept de mesure est étranger à Power Query.

Table calculée

Calcul de modèle qui vise à ajouter une table à un modèle tabulaire en écrivant une formule DAX. La formule doit retourner un objet table. La table qui en résulte utilise le mode de stockage Importer.

Calcul

Processus délibéré qui transforme une ou plusieurs entrées en un ou plusieurs résultats. Dans un modèle de données tabulaires, un calcul peut être un objet modèle : une table calculée, une colonne calculée ou une mesure.

Contexte

Décrit l'environnement dans lequel une formule DAX est évaluée. Il existe deux types de contexte : le *contexte de ligne* et le *contexte de filtre*. Le contexte de ligne représente la « ligne actuelle ». Il sert à évaluer les formules de colonne calculée et les expressions utilisées par les itérateurs de table. Le contexte de filtre sert à évaluer les mesures. Il représente les filtres appliqués directement aux colonnes de modèle et aux filtres propagés par des relations de modèle.

DAX

Le langage DAX (Data Analysis Expressions) est un langage de formules pour Power Pivot dans Excel, Power BI, Azure Analysis Services et la modélisation tabulaire dans SQL Server Analysis Services. Vous pouvez aussi utiliser DAX pour ajouter des calculs de modèle de données et définir des règles de sécurité au niveau des lignes (RLS).

Sécurité dynamique

Quand des règles de sécurité au niveau des lignes (RLS) sont mises en place en utilisant l'identité de l'utilisateur du rapport. Les règles filtrent les tables de modèle en utilisant le nom de compte de l'utilisateur, ce qui peut être effectué avec les fonctions USERNAME ou USERPRINCIPALNAME. Consultez Sécurité au niveau des lignes.

Expression

Unité de logique DAX qui fait l'objet d'une évaluation avant qu'un résultat soit retourné. Les expressions peuvent déclarer des variables. Dans ce cas, une sous-expression leur

est affectée et elles doivent inclure une instruction RETURN qui génère une expression finale. Les expressions sont construites à l'aide d'objets modèle (tables, colonnes ou mesures), de fonctions, d'opérateurs ou de constantes.

Champ

Ressource de modèle de données présentée dans le volet **Champs**. Les champs servent à configurer des filtres de rapport et des visuels. Les champs sont constitués de colonnes de modèle, de niveaux hiérarchiques et de mesures.

Formule

Une ou plusieurs expressions DAX servant à définir un calcul de modèle. Les expressions internes sont appelées sous-expressions. Le pluriel est *formules*.

Fonction

Les fonctions DAX contiennent des arguments qui permettent de transmettre des paramètres. Les formules peuvent utiliser de nombreux appels de fonction, avec la possibilité d'imbriquer des fonctions dans d'autres fonctions. Dans une formule, les noms des fonctions doivent être suivis de parenthèses. Les paramètres entre parenthèses sont transmis.

Mesure implicite

Calcul généré automatiquement en configurant un visuel Power BI pour résumer les valeurs de colonne. Les colonnes **numériques** prennent en charge la plus large palette de résumés, notamment : Somme, Moyenne, Minimum, Maximum, Nombre (distinct), Nombre, Écart type, Variance ou Médiane. Les colonnes d'autres types de données peuvent aussi être résumées. Les colonnes **Texte** peuvent être résumées avec : Première (par ordre alphabétique), Dernière (par ordre alphabétique), Nombre (distinct) ou Nombre. Les colonnes **Date** peuvent être résumées avec : Plus ancienne, Plus récente, Nombre (distinct) ou Nombre. Les colonnes **booléennes** peuvent être résumées avec : Nombre (distinct) ou Nombre.

Fonction itératrice

Fonction DAX qui énumère toutes les lignes d'une table donnée et évalue une expression donnée pour chaque ligne. Elle offre de la flexibilité et permet de contrôler la

façon dont les calculs de modèle résument les données.

MDX

Le langage MDX (Multidimensional Expressions) est un langage de formules pour les modèles multidimensionnels SQL Server Analysis Services (aussi appelés *cubes*). Si MDX permet d'interroger des modèles tabulaires, il ne permet pas en revanche de définir des mesures implicites. Il permet seulement d'interroger les mesures déjà définies dans le modèle.

Mesure

Calcul qui effectue le résumé. Les mesures sont soit *implicites*, soit *explicites*. Une mesure explicite est un calcul ajouté à un modèle de données tabulaire en écrivant une formule DAX. Une formule de mesure doit retourner une valeur scalaire. Dans le volet **Champs**, les mesures explicites sont parées d'une icône de calculatrice. Les mesures explicites s'avèrent nécessaires quand le modèle est interrogé à l'aide d'expressions multidimensionnelles (MDX), comme c'est le cas quand la fonction d'analyse est utilisée dans Excel. En général, une mesure explicite s'appelle tout simplement une mesure.

Groupe de mesures

Table de modèle qui contient au moins une mesure et aucune hiérarchie ni colonne visible. Dans le volet **Champs**, chaque groupe de mesures est paré d'une icône à plusieurs calculatrices. Les groupes de mesures sont listés en haut du volet **Champs** et triés par ordre alphabétique des noms.

Calcul de modèle

Formule nommée servant à ajouter une table calculée, une colonne calculée ou une mesure à un modèle de données tabulaire. Sa structure est <NOM> = <FORMULE>. La plupart des calculs sont ajoutés par les modeleurs de données dans Power BI Desktop, mais il est également possible d'ajouter des mesures à un rapport à connexion active. Consultez Mesures de rapport.

Mesures rapides

Fonctionnalité dans Power BI Desktop qui dispense de l'écriture de formules DAX pour les mesures couramment définies. Les mesures rapides incluent la moyenne par

catégorie, le rang et la différence par rapport à la ligne de base.

Mesures de rapport

Également appelées *mesures au niveau du rapport*. Elles sont ajoutées à un rapport à connexion active dans Power BI Desktop en écrivant une formule DAX, mais uniquement pour les connexions aux modèles Power BI ou aux modèles tabulaires Analysis Services.

Sécurité au niveau des lignes

Également appelée *RLS*. Technique de conception visant à limiter l'accès à des sous-ensembles de données pour des utilisateurs spécifiques. Dans un modèle tabulaire, cela est possible en créant des rôles de modèle. Les rôles sont assortis de règles, qui sont des expressions DAX destinées à filtrer les lignes de table.

Scalaire

Dans DAX, une valeur scalaire est une valeur unique. Une valeur scalaire peut être de n'importe quel type de données : Decimal, Integer, DateTime, String, Currency, Boolean (Décimal, Entier, DateHeure, Chaîne, Devise, Booléen). Une valeur scalaire peut être le résultat d'une expression calculée à partir de plusieurs valeurs. Par exemple, une fonction d'agrégation telle que MAX() retourne une valeur maximale unique à partir d'un ensemble de valeurs à partir duquel effectuer l'évaluation.

Résumé

Opération appliquée aux valeurs d'une colonne. Consultez mesure.

Intelligence temporelle

L'intelligence temporelle porte sur les calculs dans le temps, comme le cumul annuel jusqu'à ce jour (YTD).

Fonctions d'intelligence temporelle

DAX comprend de nombreuses fonctions d'intelligence temporelle. Chaque fonction d'intelligence temporelle obtient son résultat en modifiant le contexte de filtre pour les filtres de date. Exemples de fonctions : TOTALYTD et SAMEPERIODLASTYEAR.

Valeur, valeurs

Données à visualiser.

Paramètre de scénario

Fonctionnalité Power BI Desktop qui permet d'accepter une entrée utilisateur via des segments. Chaque paramètre crée une table calculée à une seule colonne et une mesure qui retourne une seule valeur sélectionnée. La mesure peut être utilisée dans des calculs de modèle pour répondre à l'entrée de l'utilisateur.

Commentaires

Cette page a-t-elle été utile ? Yes No

Opérateurs DAX

Article • 20/10/2023

Le langage DAX (Data Analysis Expression) a recours à des opérateurs pour créer des expressions qui comparent des valeurs, effectuent des calculs arithmétiques ou utilisent des chaînes.

Types d'opérateurs

Il existe quatre types d'opérateurs de calcul : les opérateurs arithmétiques, les opérateurs de comparaison, les opérateurs de concaténation de texte et les opérateurs logiques.

Opérateurs arithmétiques

Pour effectuer des opérations mathématiques de base comme des additions, des soustractions ou des multiplications, combiner des nombres et produire des résultats numériques, utilisez les opérateurs arithmétiques suivants.

Agrandir le tableau

Opérateur arithmétique	Signification	Exemple
+ (signe plus)	Addition	3+3
– (signe moins)	Soustraction ou signe	3–1–1
* (astérisque)	Multiplication	3*3
/ (barre oblique)	Division	3/3
^ (caret)	Élévation à une puissance	16^4

Notes

Le signe plus peut fonctionner à la fois comme *opérateur binaire* et comme *opérateur unaire*. Un opérateur binaire nécessite des nombres de chaque côté de l'opérateur et effectue une addition. Si vous utilisez des valeurs non numériques dans une formule DAX des deux côtés de l'opérateur binaire, DAX tente de les caster en types de données numériques. En revanche, l'opérateur unaire peut être appliqué à n'importe quel type d'argument. Le symbole plus n'affecte ni le type ni

la valeur et est simplement ignoré ; par contre, l'opérateur moins crée une valeur négative s'il est appliqué à une valeur numérique.

Opérateurs de comparaison

Utilisez les opérateurs suivants pour comparer deux valeurs. Quand deux valeurs sont comparées à l'aide de ces opérateurs, le résultat est une valeur logique, TRUE ou FALSE.

Agrandir le tableau

Opérateur de comparaison	Signification	Exemple
=	Égal à	[Region] = "USA"
==	Strictement égal à	[Region] == "USA"
>	Supérieur à	[Sales Date] > "Jan 2009"
<	Inférieur à	[Sales Date] < "Jan 1 2009"
>=	Supérieur ou égal à	[Amount] >= 20000
<=	Inférieur ou égal à	[Amount] <= 100
<>	Différent de	[Region] <> "USA"

Tous les opérateurs de comparaison sauf == considèrent les valeurs vides comme étant égales au nombre 0, à une chaîne vide (""), à DATE(1899, 12, 30) ou à FALSE. [Column] = 0 est donc TRUE quand la valeur de [Column] est 0 ou vide. En revanche, [Column] == 0 est TRUE uniquement quand la valeur de [Column] est 0.

Opérateur de concaténation de texte

Utilisez une esperluette (**&**) pour joindre ou concaténer deux chaînes de texte (ou plus) afin de produire un seul bloc de texte.

Agrandir le tableau

Opérateur de texte	Signification	Exemple
& (esperluette)	Connecte ou concatène deux valeurs pour produire une valeur de texte continue.	[Region] & ", " & [City]

Opérateurs logiques

Utilisez les opérateurs logiques (&&) et (||) pour combiner des expressions afin de produire un résultat unique.

Agrandir le tableau

Opérateur de texte	Signification	Exemples
&& (double esperluette)	Crée une condition AND entre deux expressions ayant chacune un résultat booléen. Si les deux expressions retournent TRUE, la combinaison des expressions retourne également TRUE ; sinon, elle retourne FALSE.	([Region] = "France") && ([BikeBuyer] = "yes"))
|| (symbole de barre verticale double)	Crée une condition OR entre deux expressions logiques. Si l'une des expressions retourne TRUE, le résultat est TRUE. Le résultat est FALSE uniquement quand les deux expressions sont FALSE.	(([Region] = "France") || ([BikeBuyer] = "yes"))
IN	Crée une condition OR logique entre chaque ligne comparée à une table. Remarque : La syntaxe du constructeur de table utilise des accolades.	'Product'[Color] IN { "Red", "Blue", "Black" }

Opérateurs et ordre de précédence

Dans certains cas, l'ordre dans lequel le calcul est effectué peut affecter la valeur de retour. Il est donc important de comprendre comment l'ordre est déterminé et comment vous pouvez le changer pour obtenir les résultats souhaités.

Ordre de calcul

Une expression évalue les opérateurs et les valeurs dans un ordre spécifique. Toutes les expressions commencent toujours par un signe égal (=). Le signe égal indique que les caractères qui s'ensuivent constituent une expression.

Après le signe égal se trouvent les éléments à calculer (opérandes), lesquels sont séparés par des opérateurs de calcul. Les expressions sont toujours lues de gauche à droite, mais vous pouvez contrôler dans une certaine mesure l'ordre dans lequel les éléments sont regroupés à l'aide de parenthèses.

Précédence des opérateurs

Si vous combinez plusieurs opérateurs dans une même formule, les opérations sont classées selon le tableau suivant. Si des opérateurs ont une valeur de précédence égale, ils sont classés de gauche à droite. Par exemple, si une expression contient un opérateur de multiplication et un opérateur de division, ils sont évalués dans l'ordre dans lequel ils apparaissent dans l'expression, de gauche à droite.

Agrandir le tableau

Opérateur	Description
^	Élévation à une puissance
–	Signe (comme dans –1)
* et /	Multiplication et division
+ et –	Addition et soustraction
&	Connecte deux chaînes de texte (concaténation)
=,==,<,>,<=,>=,<>,IN	Comparaison
NOT	NOT (opérateur unaire)

Utilisation de parenthèses pour contrôler l'ordre de calcul

Pour changer l'ordre d'évaluation, vous devez placer entre parenthèses la partie de la formule à calculer en premier. Par exemple, la formule suivante produit 11 parce que la multiplication est calculée avant l'addition. La formule multiplie 2 par 3, puis ajoute 5 au résultat.

DAX

```
=5+2*3
```

En revanche, si vous utilisez des parenthèses pour changer la syntaxe, l'ordre est modifié. Désormais, 5 et 2 sont additionnés, puis le résultat est multiplié par 3 pour produire 21.

DAX

```
=(5+2)*3
```

Dans l'exemple suivant, les parenthèses placées autour de la première partie de la formule forcent le calcul à évaluer l'expression `(3 + 0.25)` en premier et à diviser le

résultat par le résultat de l'expression (`3 - 0.25)`.

```
DAX

=(3 + 0.25)/(3 - 0.25)
```

Dans l'exemple suivant, l'opérateur d'élévation à une puissance est appliqué en premier, conformément aux règles de précédence des opérateurs, puis l'opérateur de signe est appliqué. Le résultat de cette expression est -4.

```
DAX

=-2^2
```

Pour faire en sorte que l'opérateur de signe soit appliqué en premier à la valeur numérique, vous pouvez utiliser des parenthèses pour contrôler les opérateurs, comme dans l'exemple suivant. Le résultat de cette expression est 4.

```
DAX

= (-2)^2
```

Compatibilité

À l'instar de Microsoft Excel, DAX permet de gérer et de comparer facilement différents types de données. Toutefois, le moteur de calcul sous-jacent étant basé sur SQL Server Analysis Services, il propose en plus les fonctionnalités avancées d'un magasin de données relationnelles, notamment une prise en charge plus riche des types de date et d'heure. Dans certains cas, les résultats des calculs ou le comportement des fonctions peuvent différer de ceux d'Excel. En outre, DAX prend en charge plus de types de données qu'Excel. Cette section décrit les principales différences.

Forçage des types de données des opérandes

En général, les deux opérandes de chaque côté d'un opérateur doivent être du même type de données. Toutefois, dans certains cas, si les types de données sont différents, DAX les convertit en un type de données commun pour appliquer l'opérateur :

1. Les deux opérandes sont convertis pour obtenir le plus grand type de données commun possible.
2. L'opérateur est appliqué, si possible.

Par exemple, supposons que vous souhaitiez combiner deux nombres. Un nombre provient d'une formule, par exemple =`[Price] * .20`, et le résultat peut contenir plusieurs décimales. L'autre nombre est un entier qui est fourni sous la forme d'une valeur de chaîne.

Dans ce cas, DAX convertit les deux nombres en nombres réels dans un format numérique, en utilisant le plus grand format numérique pouvant stocker les deux types de nombres. DAX applique ensuite la multiplication.

Selon la combinaison de types de données, le forçage de type peut ne pas être appliqué aux opérations de comparaison. Pour obtenir la liste complète des types de données pris en charge par DAX, consultez Types de données pris en charge dans les modèles tabulaires et Types de données dans Power BI Desktop.

Les entiers, nombres réels, devises, valeurs de date/heure et valeurs vides sont considérés comme des valeurs numériques à des fins de comparaison. La valeur vide est égale à zéro lors d'une comparaison. Les combinaisons de types de données suivantes sont prises en charge pour les opérations de comparaison.

Agrandir le tableau

Type de données côté gauche	Type de données côté droit
Numérique	Numérique
Booléen	Boolean
String	Chaîne

D'autres comparaisons de types de données mixtes retournent une erreur. Par exemple, une formule comme ="1" > 0 retourne une erreur indiquant que *les opérations DAX de comparaison ne prennent pas en charge la comparaison des valeurs de type Texte avec des valeurs de type Entier*.

Agrandir le tableau

Types de données utilisés dans DAX	Types de données utilisés dans Excel
Nombres (I8, R8)	Nombres (R8)
Chaîne	Chaîne
Booléen	Boolean
Date/Heure	Variante

Types de données utilisés dans DAX	Types de données utilisés dans Excel
Devise	Devise

Différences dans l'ordre de précédence

L'ordre de précédence des opérations dans les formules DAX est fondamentalement le même que dans Microsoft Excel, mais certains opérateurs Excel tels que le pourcentage ne sont pas pris en charge. Par ailleurs, les plages ne sont pas prises en charge.

Chaque fois que vous copiez et collez des formules à partir d'Excel, veillez à examiner attentivement la formule car certains opérateurs ou éléments qu'elle contient peuvent ne pas être valides. En cas de doute sur l'ordre dans lequel les opérations sont effectuées, il est recommandé d'utiliser des parenthèses pour contrôler l'ordre des opérations et lever toute ambiguïté quant au résultat.

Contenu connexe

Syntaxe DAX
Nommage des paramètres DAX

Commentaires

Cette page a-t-elle été utile ? Yes No

Requêtes DAX

Article • 20/10/2023

Les clients de création de rapports comme Power BI et Excel exécutent des requêtes DAX chaque fois qu'un champ est placé sur un rapport ou qu'un filtre est appliqué. En utilisant SQL Server Management Studio (SSMS), Power BI Report Builder et des outils open source comme DAX Studio, vous pouvez créer et exécuter vos propres requêtes DAX. Les requêtes DAX retournent des résultats sous forme de table directement dans l'outil, ce qui vous permet de rapidement créer des formules DAX et tester leurs performances.

Avant de découvrir les requêtes, il est important de bien comprendre les bases de DAX. Si vous ne l'avez pas encore fait, n'hésitez pas à consulter Vue d'ensemble de DAX.

Mots clés

Les requêtes DAX ont une syntaxe simple composée d'un seul mot clé obligatoire, EVALUATE, et de plusieurs mots clés facultatifs : ORDER BY, START AT, DEFINE, MEASURE, VAR, TABLE et COLUMN. Chaque mot clé définit une instruction utilisée pour la durée de la requête.

EVALUATE (obligatoire)

Au niveau le plus basique, une requête DAX est une instruction **EVALUATE** contenant une expression de table. Au moins une instruction EVALUATE est obligatoire, mais une requête peut contenir n'importe quel nombre d'instructions EVALUATE.

Syntaxe d'EVALUATE

```DAX
EVALUATE <table>
```

Paramètres d'EVALUATE

Agrandir le tableau

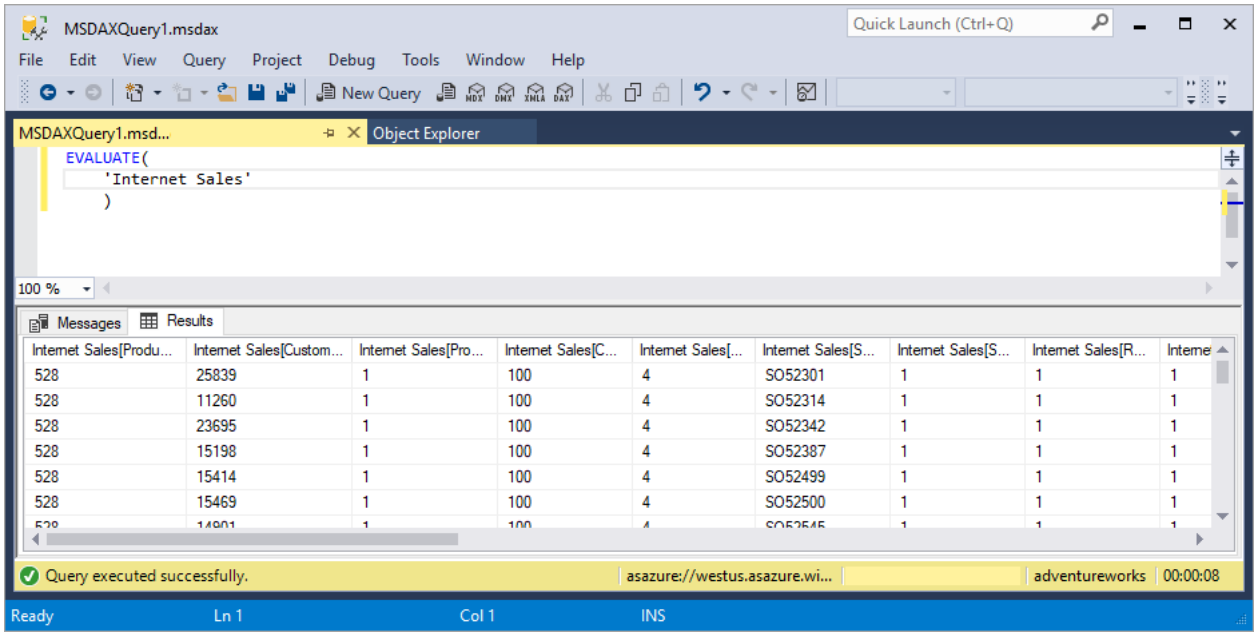
Terme	Définition
tableau	Expression de table.

Exemple d'EVALUATE

DAX

```
EVALUATE
    'Internet Sales'
```

Retourne toutes les lignes et colonnes de la table Internet Sales sous la forme d'une table.



ORDER BY (facultatif)

Le mot clé facultatif **ORDER BY** définit une ou plusieurs expressions utilisées pour trier les résultats d'une requête. Toute expression qui peut être évaluée pour chaque ligne du résultat est valide.

Syntaxe d'ORDER BY

DAX

```
EVALUATE <table>
[ORDER BY {<expression> [{ASC | DESC}]}[, …]]
```

Paramètres d'ORDER BY

Agrandir le tableau

Terme	Définition
expression	Toute expression DAX qui retourne une valeur scalaire unique.
ASC	Ordre de tri croissant (par défaut).
DESC	Ordre de tri décroissant.

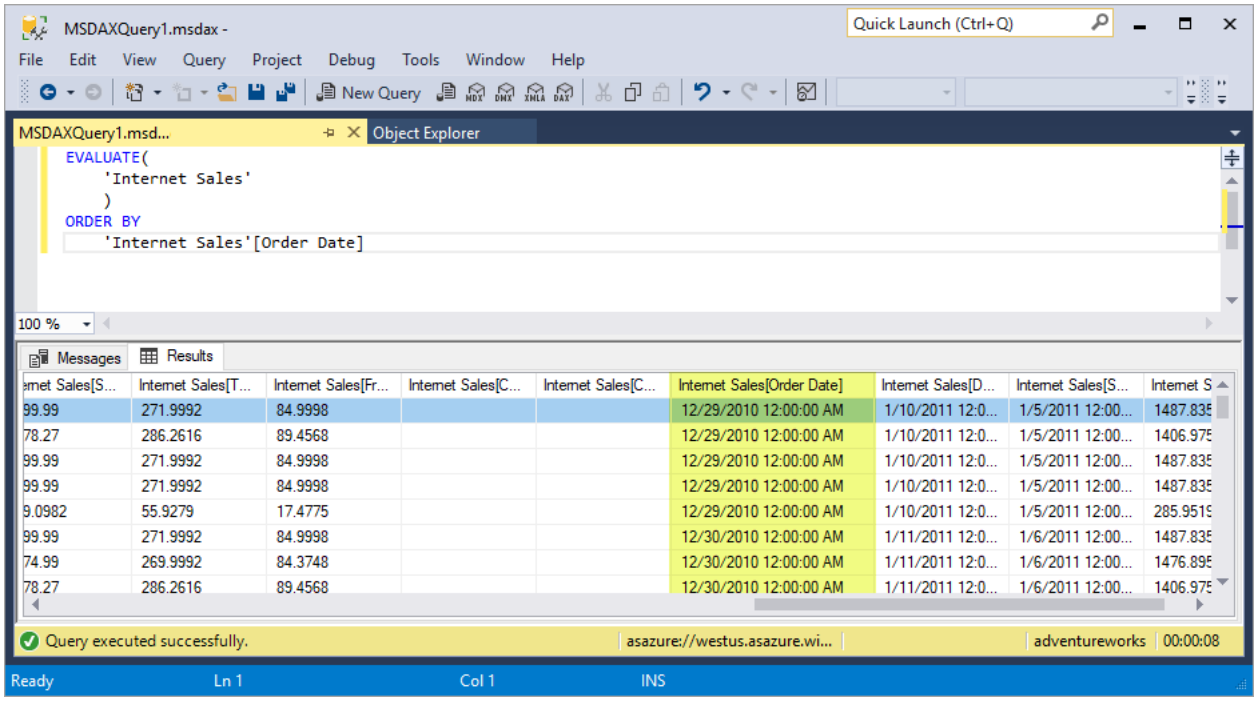
Exemple d'ORDER BY

DAX

```
EVALUATE
    'Internet Sales'

ORDER BY
    'Internet Sales'[Order Date]
```

Retourne toutes les lignes et colonnes de la table Internet Sales, dans l'ordre croissant du champ Order Date, sous forme de table.



START AT (facultatif)

Le mot clé facultatif **START AT** est utilisé dans une clause **ORDER BY**. Il définit la valeur à laquelle commencent les résultats de la requête.

Syntaxe de START AT

DAX

```
EVALUATE <table>
[ORDER BY {<expression> [{ASC | DESC}]}[, …]
[START AT {<value>|<parameter>} [, …]]]
```

Paramètres de START AT

Agrandir le tableau

Terme	Définition
valeur	Valeur constante. Il ne peut pas s'agir d'une expression.
parameter	Nom d'un paramètre dans une instruction XMLA précédé du caractère `@`.

Remarques concernant START AT

Les arguments de START AT ont des correspondances uniques avec les colonnes de la clause ORDER BY. La clause START AT peut inclure autant d'arguments qu'il en existe dans la clause ORDER BY, mais pas plus. Le premier argument de la clause START AT définit la valeur de départ de la colonne 1 des colonnes ORDER BY. Le deuxième argument de la clause START AT définit la valeur de départ de la colonne 2 des colonnes ORDER BY dans les lignes correspondant à la première valeur de la colonne 1.

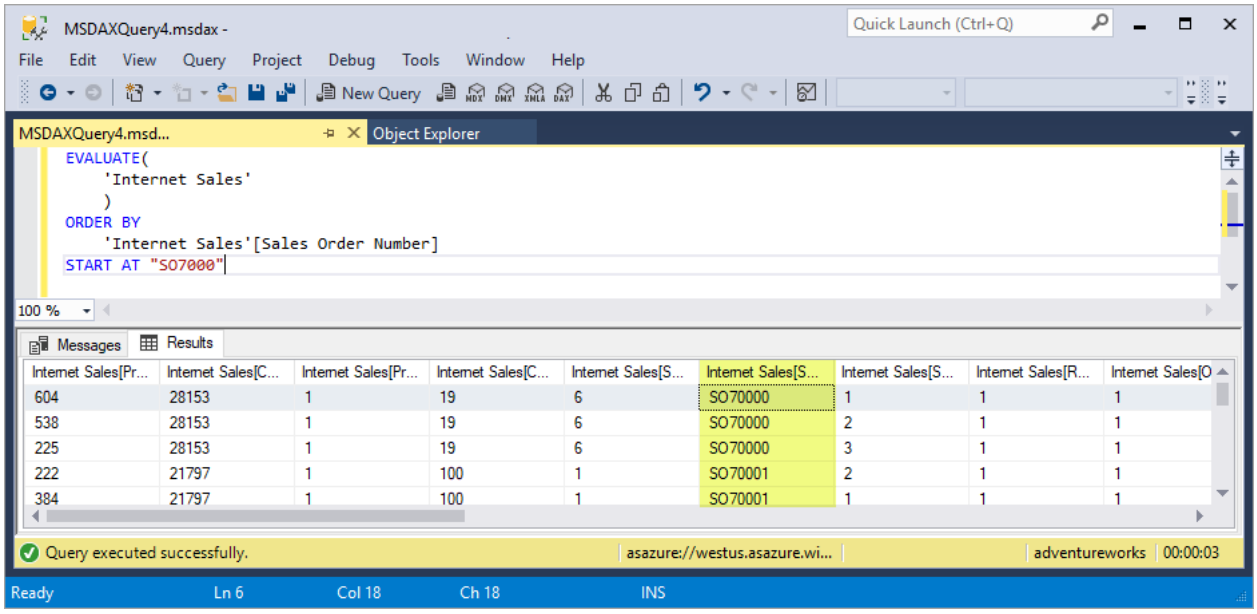
Exemple de START AT

DAX

```
EVALUATE
    'Internet Sales'

ORDER BY
    'Internet Sales'[Sales Order Number]
START AT "SO7000"
```

Retourne toutes les lignes et colonnes de la table Internet Sales, par ordre croissant du champ Sales Order Number, sous forme de table, en commençant par SO7000.



DEFINE (facultatif)

Le mot clé **DEFINE** facultatif introduit une ou plusieurs définitions d'entité calculées qui existent uniquement pendant la durée de la requête. Les définitions précèdent l'instruction EVALUATE et sont valides pour toutes les instructions EVALUATE de la requête. Les définitions peuvent être des variables, des mesures, des tables[1] et des colonnes[1]. Les définitions peuvent référencer d'autres définitions qui apparaissent avant ou après la définition actuelle. Au moins une définition est obligatoire si le mot clé DEFINE est compris dans une requête.

Syntaxe de DEFINE

DAX

```
[DEFINE
    (
     (MEASURE <table name>[<measure name>] = <scalar expression>) |
     (VAR <var name> = <table or scalar expression>) |
     (TABLE <table name> = <table expression>) |
     (COLUMN <table name>[<column name>] = <scalar expression>) |
    ) +
]

(EVALUATE <table expression>) +
```

Paramètres de DEFINE

Agrandir le tableau

Terme	Définition
Entité	MEASURE, VAR, TABLE[1] ou COLUMN[1].
name	Nom d'une définition de mesure, variable, table ou colonne. Il ne peut pas s'agir d'une expression. Le nom n'a pas besoin d'être unique. Le nom existe seulement pour la durée de la requête.
expression	Toute expression DAX qui retourne une table ou une valeur scalaire. L'expression peut utiliser n'importe quelle entité définie. Si vous devez convertir une expression scalaire en expression de table, wrappez l'expression dans un constructeur de table avec des accolades `{}` ou utilisez la fonction `ROW()` pour retourner une seule table de lignes.

[1]**Attention :** Les définitions de TABLE et COLUMN limitées aux requêtes sont destinées à une utilisation interne uniquement. Même si vous pouvez définir des expressions TABLE et COLUMN pour une requête sans erreur de syntaxe, elles peuvent produire des erreurs d'exécution et ne sont pas recommandées.

Remarques concernant DEFINE

- Une requête DAX peut avoir plusieurs instructions EVALUATE, mais seulement une instruction DEFINE. Les définitions de l'instruction DEFINE peuvent s'appliquer à toutes les instructions EVALUATE de la requête.
- Au moins une définition est obligatoire dans une instruction DEFINE.
- Les définitions de mesure d'une requête remplacent les mesures du modèle du même nom, mais sont uniquement utilisées dans la requête. Elles n'affectent pas la mesure du modèle.
- Les noms de VAR ont des restrictions uniques. Pour en savoir plus, consultez VAR - Paramètres.

Exemple de DEFINE

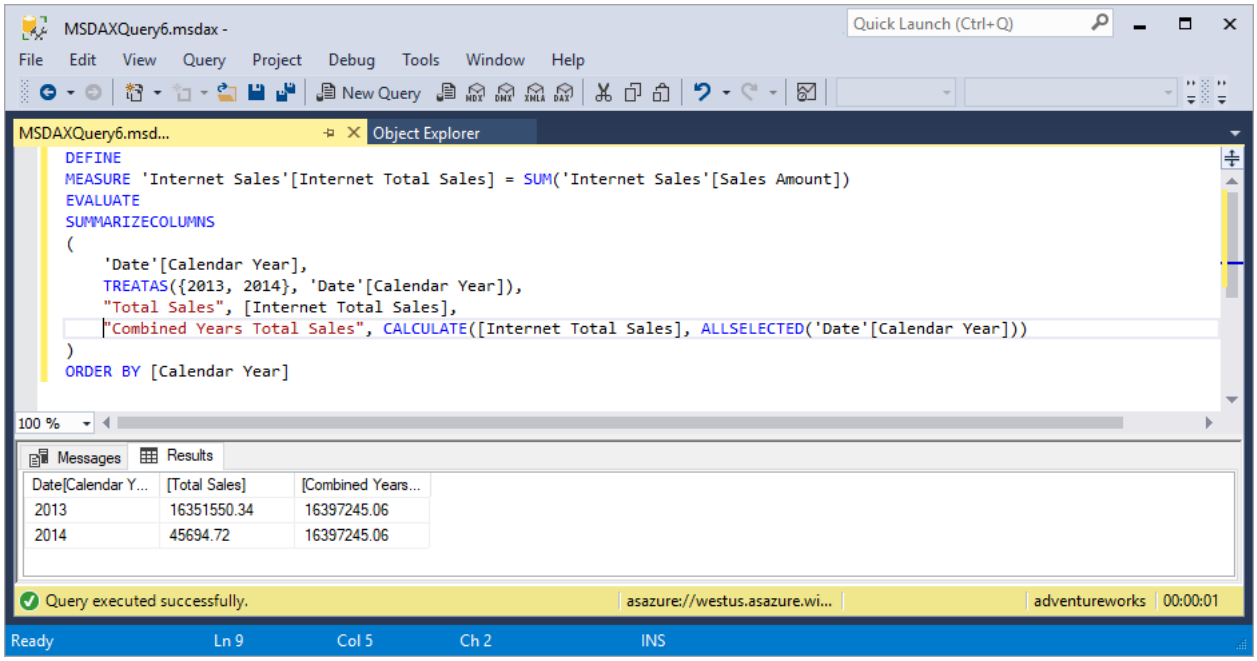
DAX

```
DEFINE
    MEASURE 'Internet Sales'[Internet Total Sales] =
        SUM ( 'Internet Sales'[Sales Amount] )

EVALUATE
SUMMARIZECOLUMNS (
```

```
    'Date'[Calendar Year],
    TREATAS (
        {
            2013,
            2014
        },
        'Date'[Calendar Year]
    ),
    "Total Sales", [Internet Total Sales],
    "Combined Years Total Sales",
        CALCULATE (
            [Internet Total Sales],
            ALLSELECTED ( 'Date'[Calendar Year] )
        )
)
ORDER BY [Calendar Year]
```

Retourne les ventes totales calculées pour les années 2013 et 2014 ainsi que les ventes totales calculées combinées pour les années 2013 et 2014 sous la forme d'une table. La mesure Internet Total Sales dans l'instruction DEFINE est utilisée dans les expressions Total Sales et Combined Years Total Sales.



Paramètres dans les requêtes DAX

Vous pouvez paramétrer une instruction de requête DAX correctement définie et la réutiliser à volonté en apportant simplement quelques modifications aux valeurs de paramètre.

La méthode Execute (XMLA) inclut un élément de collection Parameters (XMLA) qui permet de définir des paramètres et de leur affecter une valeur. Dans la collection, chaque élément Parameter (XMLA) définit le nom du paramètre et sa valeur.

Référencez les paramètres XMLA en faisant précéder leur nom du caractère `@`.
N'importe où dans la syntaxe où une valeur est autorisée, la valeur peut être remplacée par un appel de paramètre. Tous les paramètres XMLA sont tapés sous forme de texte.

> **Important**
>
> Les paramètres définis dans la section de paramètres et qui ne sont pas utilisés dans l'élément **<STATEMENT>** génèrent une réponse d'erreur en XMLA. Les paramètres utilisés et non définis dans l'élément **<Parameters>** génèrent une réponse d'erreur en XMLA.

Contenu connexe

Instructions DAX
SUMMARIZECOLUMNS
TREATAS
FILTER

Commentaires

Cette page a-t-elle été utile ? Yes No

Conventions de nommage des paramètres DAX

Article • 20/10/2023

Les noms de paramètre sont normalisés dans la référence DAX pour faciliter l'utilisation et la compréhension des fonctions.

Noms de paramètre

Agrandir le tableau

Terme	Définition
expression	Toute expression DAX qui retourne une valeur scalaire unique où l'expression doit être évaluée plusieurs fois (pour chaque ligne/contexte).
value	Toute expression DAX qui retourne une valeur scalaire unique où l'expression doit être évaluée une seule fois avant toutes les autres opérations.
tableau	Toute expression DAX qui retourne une table de données.
tableName	Nom d'une table existante, spécifié avec la syntaxe DAX standard. Il ne peut pas s'agir d'une expression.
columnName	Nom, généralement complet, d'une colonne existante utilisant la syntaxe DAX standard. Il ne peut pas s'agir d'une expression.
name	Constante de chaîne qui sera utilisée pour fournir le nom d'un nouvel objet.
order	Énumération utilisée pour déterminer l'ordre de tri.
ties	Énumération utilisée pour déterminer la gestion des valeurs d'égalité.
type	Énumération utilisée pour déterminer le type de données pour PathItem et PathItemReverse.

Attribution d'un préfixe aux noms de paramètre ou utilisation du préfixe seul

Agrandir le tableau

Terme	Définition
prefixing	Les noms de paramètre peuvent être qualifiés plus précisément avec un préfixe descriptif de la façon dont l'argument est utilisé et pour éviter toute lecture ambiguë des paramètres. Par exemple : Result_ColumnName : fait référence à une colonne existante utilisée pour obtenir les valeurs de résultat dans la fonction LOOKUPVALUE(). Search_ColumnName : fait référence à une colonne existante utilisée pour rechercher une valeur dans la fonction LOOKUPVALUE().
omitting	Les noms de paramètre sont omis si le préfixe est suffisamment clair pour décrire le paramètre. Par exemple, au lieu d'avoir la syntaxe suivante DATE (Year_Value, Month_Value, Day_Value), il est plus clair pour l'utilisateur de lire DATE (Year, Month, Day) ; la répétition par trois fois de la valeur du suffixe n'ajoute rien à une meilleure compréhension de la fonction et gêne la lecture inutilement. Toutefois, si le paramètre préfixé est Year_columnName, le nom du paramètre et le préfixe sont conservés pour s'assurer que l'utilisateur comprend que le paramètre requiert une référence à une colonne Years existante.

Commentaires

Cette page a-t-elle été utile ? Yes No

Syntaxe DAX

Article • 20/10/2023

Cet article décrit la syntaxe et les conditions requises du langage d'expression de formule DAX.

Conditions requises de la syntaxe

Une formule DAX commence toujours par un signe égal (=). Après le signe égal, vous pouvez fournir une expression qui prend une valeur scalaire, ou une expression qui peut être convertie en valeur scalaire. Leurs thèmes sont les suivants :

- Une constante scalaire, ou une expression qui utilise un opérateur scalaire (+,-,*,/,>=,...,&&, ...)
- Des références à des colonnes ou des tables. Le langage DAX utilise toujours des tables et des colonnes comme entrées de fonctions, jamais un tableau ou un ensemble arbitraire de valeurs.
- Des opérateurs, constantes et valeurs fournis comme partie d'une expression.
- Le résultat d'une fonction et de ses arguments obligatoires. Certaines fonctions DAX retournent une table au lieu d'une valeur scalaire et doivent être enveloppées (wrapped) dans une fonction qui évalue la table et retourne une valeur scalaire , sauf si la table a une seule colonne et une seule ligne, elle est traitée comme une valeur scalaire.

  La plupart des fonctions DAX demandent un ou plusieurs arguments, qui peuvent inclure des tables, des colonnes, des expressions et des valeurs. Toutefois, certaines fonctions, telles que PI, ne nécessitent aucun argument, mais nécessitent toujours des parenthèses pour indiquer l'argument null. Par exemple, vous devez toujours taper PI(), et non PI. Vous pouvez également imbriquer des fonctions dans d'autres fonctions.
- Des expressions. Une expression peut contenir un ou tous les éléments suivants : opérateurs, constantes ou références à des colonnes.

Par exemple, toutes les formules ci-dessous sont valides.

Agrandir le tableau

Formule	Résultat
= 3	3
= "**Sales**"	**Sales**
= 'Sales'[Amount]	Si vous utilisez cette formule dans la table Sales, vous obtiendrez la valeur de la colonne Amount dans la table Sales pour la ligne actuelle.
= (0,03 * [Amount]) =0.03 * [Amount]	Trois pour cent de la valeur dans la colonne Amount de la table actuelle. Même si cette formule peut être utilisée pour calculer un pourcentage, le résultat n'est pas affiché sous forme de pourcentage, sauf si vous appliquez la mise en forme dans la table.
= PI()	Valeur de la constante pi.

Les formules peuvent se comporter différemment selon leur utilisation. Vous devez toujours connaître le contexte et la façon dont les données que vous utilisez dans la formule sont liées à d'autres données qui risquent d'être utilisées dans le calcul.

Conditions de nommage

Un modèle de données contient souvent plusieurs tables. Ensemble, les tables et leurs colonnes contiennent une base de données stockée dans le moteur d'analytique en mémoire (VertiPaq). Dans cette base de données, toutes les tables doivent avoir des noms uniques. Les noms des colonnes doivent également être uniques dans chaque table. Tous les noms d'objet sont *insensibles à la casse* ; par exemple, les noms **SALES** et **Sales** représentent la même table.

Chaque colonne et chaque mesure que vous ajoutez à un modèle de données existant doivent appartenir à une table spécifique. Vous spécifiez la table qui contient la colonne implicitement quand vous créez une colonne calculée dans une table, ou explicitement quand vous créez une mesure et spécifiez le nom de la table où la définition de la mesure doit être stockée.

Quand vous utilisez une table ou une colonne comme entrée d'une fonction, vous devez généralement *qualifier* le nom de la colonne. Le nom *qualifié complet* d'une colonne est le nom de la table, suivi du nom de la colonne entre crochets : par exemple, 'U.S. Sales'[Products]. Un nom qualifié complet est toujours obligatoire quand vous référencez une colonne dans les contextes suivants :

- En tant qu'argument de la fonction, VALUES
- En tant qu'argument des fonctions, ALL ou ALLEXCEPT

- Dans un argument de filtre pour les fonctions, CALCULATE ou CALCULATETABLE
- En tant qu'argument de la fonction, RELATEDTABLE
- En tant qu'argument d'une fonction Time Intelligence

Un nom de colonne *non qualifié* se limite au nom de la colonne entre crochets : par exemple, [Sales Amount]. Par exemple, quand vous référencez une valeur scalaire à partir de la même ligne de la table actuelle, vous pouvez utiliser le nom de colonne non qualifié.

Si le nom d'une table contient des espaces, des mots clés réservés ou des caractères non autorisés, vous devez placer le nom de la table entre guillemets simples. Vous devez également mettre les noms de table entre guillemets si le nom contient des caractères qui ne sont pas dans la plage de caractères alphanumériques ANSI, que vos paramètres régionaux prennent en charge ou non le jeu de caractères. Par exemple, si vous ouvrez un classeur qui contient des noms de table écrits en caractères cyrilliques, tels que « Таблица », le nom de la table doit être placé entre guillemets, même s'il ne contient pas d'espaces.

> ⓘ **Notes**
>
> Pour faciliter la saisie des noms complets des colonnes, utilisez la fonctionnalité d'autocomplétion dans l'éditeur de formule.

Tables

- Les noms de table sont obligatoires chaque fois que la colonne provient d'une table différente de la table actuelle. Les noms de table doivent être uniques dans la base de données.
- Les noms de table doivent être placés entre guillemets simples s'ils contiennent des espaces, d'autres caractères spéciaux ou des caractères alphanumériques qui ne font pas partie de l'alphabet.

Mesures

- Les noms de mesure doivent toujours être entre crochets.
- Les noms de mesure peuvent contenir des espaces.

- Chaque nom de mesure doit être unique dans un modèle. Par conséquent, le nom de la table est facultatif devant un nom de mesure quand une mesure existante est référencée. Toutefois, lorsque vous créez une mesure, vous devez toujours spécifier une table dans laquelle la définition de la mesure est stockée.

Colonnes

Les noms de colonne doivent être uniques dans le contexte d'une table ; cependant, plusieurs tables peuvent avoir des colonnes avec le même nom (la désambiguïsation vient avec le nom de la table).

En général, les colonnes peuvent être référencées sans référencer la table de base à laquelle elles appartiennent, sauf s'il risque d'y avoir un conflit de noms à résoudre ou avec certaines fonctions qui demandent que les noms de colonne soient qualifiés complets.

Mots clés réservés

Si le nom que vous utilisez pour une table est le même qu'un mot clé réservé Analysis Services, une erreur est générée et vous devez renommer la table. Toutefois, vous pouvez utiliser des mots clés dans les noms d'objet si le nom de l'objet est entre crochets (pour les colonnes) ou entre guillemets (pour les tables).

> ⓘ **Notes**
>
> Les guillemets peuvent être représentés par plusieurs caractères différents, en fonction de l'application. Si vous collez des formules à partir d'un document externe ou d'une page web, veillez à vérifier le code ASCII du caractère utilisé pour les guillemets ouvrants et fermants afin de vous assurer qu'ils sont identiques. Sinon, DAX peut ne pas reconnaître les symboles comme des guillemets, ce qui rend la référence non valide.

Caractères spéciaux

Les caractères et les types de caractères suivants ne sont pas valides dans les noms de table, de colonne ou de mesure :

- Espaces de début ou de fin ; à moins que les espaces ne soient encadrés par des délimiteurs de nom, des crochets ou des apostrophes simples.
- Caractères de contrôle

- Les caractères suivants qui ne sont pas valides dans les noms d'objet :

  .,;':/*|?&%$!+=()[]{}<>

Exemples de noms d'objet

Le tableau suivant présente des exemples de noms d'objet :

Agrandir le tableau

Types d'objet	Exemples	Commentaire
Nom du tableau	**Sales**	Si le nom de table ne contient pas d'espaces ni d'autres caractères spéciaux, il n'est pas nécessaire de le mettre entre guillemets.
Nom du tableau	**'Canada Sales'**	Si le nom contient des espaces, des tabulations ou d'autres caractères spéciaux, mettez le nom entre guillemets simples.
Nom qualifié complet de colonne	**Sales[Amount]**	Le nom de la table précède le nom de la colonne et le nom de la colonne est entre crochets.
Nom qualifié complet de mesure	**Sales[Profit]**	Le nom de la table précède le nom de la mesure et le nom de la mesure est entre crochets. Dans certains contextes, un nom qualifié complet est toujours obligatoire.
Nom de colonne non qualifié	**[Amount]**	Le nom non qualifié est juste le nom de la colonne entre crochets. Les contextes dans lesquels vous pouvez utiliser le nom non qualifié incluent des formules dans une colonne calculée au sein de la même table ou dans une fonction d'agrégation qui analyse la même table.
Colonne qualifiée complète dans une table avec des espaces	**'Canada Sales'[Qty]**	Le nom de la table contient des espaces, il doit donc être entouré de guillemets simples.

Autres restrictions

La syntaxe nécessaire pour chaque fonction et le type d'opération qu'elle peut effectuer varient grandement selon la fonction. En général toutefois, les règles suivantes s'appliquent à toutes les formules et expressions :

- Les formules et expressions DAX ne peuvent pas modifier ni insérer des valeurs individuelles dans les tables.

- Vous ne pouvez pas créer de lignes calculées en utilisant DAX. Vous pouvez créer uniquement des colonnes calculées et des mesures.
- Lorsque vous définissez des colonnes calculées, vous pouvez imbriquer des fonctions à n'importe quel niveau.
- DAX contient plusieurs fonctions qui retournent une table. En général, vous utilisez les valeurs retournées par ces fonctions comme entrée pour d'autres fonctions, qui nécessitent une table comme entrée.

Opérateurs et constantes DAX

Le tableau suivant liste les opérateurs pris en charge par DAX. Pour plus d'informations sur la syntaxe des opérateurs individuels, consultez Opérateurs DAX.

Agrandir le tableau

Type d'opérateur	Symbole et utilisation
Opérateur de parenthèse	() ordre de priorité et regroupement des arguments
Opérateurs arithmétiques	+ (addition) - (signe de soustraction) * (multiplication) / (division) ^ (élévation à une puissance)
Opérateurs de comparaison	= (égal à) > (supérieur à) < (inférieur à) >= (supérieur ou égal à) <= (inférieur ou égal à) <> (différent de)
Opérateur de concaténation de texte	& (concaténation)

Type d'opérateur	Symbole et utilisation
Opérateurs logiques	&& (and) || (ou)

Type de données

Vous n'avez pas besoin de caster, convertir ou spécifier le type de données d'une colonne ou d'une valeur que vous utilisez dans une formule DAX. Lorsque vous utilisez des données dans une formule DAX, DAX identifie automatiquement les types de données dans les colonnes référencées et les valeurs que vous tapez, puis effectue des conversions implicites si nécessaire pour effectuer l'opération spécifiée.

Par exemple, si vous essayez d'ajouter un nombre à une valeur de date, le moteur interprète l'opération dans le contexte de la fonction et convertit les nombres en un type de données courant, puis présente le résultat dans le format prévu, une date.

Toutefois, il existe certaines limitations sur les valeurs qui peuvent être converties avec succès. Si une valeur ou une colonne a un type de données qui est incompatible avec l'opération en cours, DAX retourne une erreur. De plus, DAX ne fournit pas de fonctions qui vous laissent explicitement changer, convertir ou caster le type de données des données existantes que vous avez importées dans un modèle de données.

> ⓘ **Important**
>
> DAX ne prend pas en charge l'utilisation du type de données Variant. Par conséquent, quand vous chargez ou importez des données dans un modèle de données, les données de chaque colonne sont généralement d'un type de données cohérent.

Certaines fonctions retournent des valeurs scalaires, notamment des chaînes, tandis que d'autres marchent avec des nombres, entiers et réels, ou des dates et des heures. Le type de données nécessaire pour chaque fonction est décrit dans la section Fonctions DAX.

Vous pouvez utiliser des tables contenant plusieurs colonnes et plusieurs lignes de données comme argument d'une fonction. Certaines fonctions retournent aussi des tables, qui sont stockées en mémoire et peuvent être utilisées comme arguments pour d'autres fonctions.

Date et heure

DAX stocke les valeurs de date et d'heure à l'aide du type de données DateHeure utilisé par Microsoft SQL Server. Le format DateHeure utilise un nombre à virgule flottante où les valeurs Date correspondent à la partie entière représentant le nombre de jours depuis le 30 décembre 1899. Les valeurs d'heure correspondent à la partie décimale d'une valeur d'une date où les heures, les minutes et les secondes sont représentées par des fractions décimales d'un jour. Les fonctions de date et d'heure DAX convertissent implicitement les arguments en type de données DateHeure.

> ⓘ **Notes**
>
> La valeur DateTime maximale exacte prise en charge par DAX est le 31 décembre 9999 00:00:00.

Littéral de date et d'heure

À partir de la version d'août 2021 de Power BI Desktop, les valeurs Date et DateHeure de DAX peuvent être spécifiées en tant que littéral au format `dt"YYYY-MM-DD"`, `dt"YYYY-MM-DDThh:mm:ss"` ou `dt"YYYY-MM-DD hh:mm:ss"`. Quand elles sont spécifiées en tant que littéral, il n'est pas nécessaire d'utiliser les fonctions DATE, TIME, DATEVALUE et TIMEVALUE dans l'expression.

Par exemple, l'expression suivante utilise des fonctions DATE et TIME pour filtrer sur OrderDate :

DAX

```
EVALUATE
FILTER (
        FactInternetSales,
        [OrderDate] > (DATE(2015,1,9) + TIME(2,30,0)) &&[OrderDate] <
(DATE(2015,12,31) + TIME(11,59,59))
)
```

La même expression de filtre peut être spécifiée en tant que littéral :

DAX

```
EVALUATE
FILTER (
        FactInternetSales,
        [OrderDate] > dt"2015-1-9T02:30:00" && [OrderDate] < dt"2015-12-
```

```
31T11:59:59"
)
```

> **Notes**
>
> Le format DAX littéral de type Date et DateHeure n'est pas pris en charge dans toutes les versions de Power BI Desktop, d'Analysis Services et de Power Pivot dans Excel. Les fonctionnalités DAX qui sont nouvelles ou mises à jour sont généralement introduites dans Power BI Desktop, puis incluses ultérieurement dans Analysis Services et Power Pivot dans Excel.

Commentaires

Cette page a-t-elle été utile ? Yes No